saroj@interview-prep:~/master-handbook$

$ cat README.md

DevOps / SRE — Master Interview Handbook

Consolidated from every prior interview handbook · organized by technology, not by company

Tabs: 36 **Format:** search + mastery tracking per topic, saved locally **Built from:** 26 source handbooks

1494	0	0%
TOTAL ITEMS	MASTERED	OVERALL

AWS

- [ ] Explain Region, Availability Zone, and Edge Location. How do they shape your architecture?

IF ASKED, SAY THIS
Region — an independent geographic area (e.g. us-east-1); an isolation and data-residency boundary with its own service endpoints. Regions fail independently.

- **Region** — an independent geographic area (e.g. `us-east-1`); an isolation and data-residency boundary with its own service endpoints. Regions fail independently.
- **Availability Zone (AZ)** — one or more discrete datacenters within a Region with independent power/cooling/networking, linked by low-latency fiber. Each Region has a minimum of three.
- **Edge Location** — hundreds of CloudFront/Route 53 points of presence that cache content close to users.

PRACTICAL NOTE
DESIGN RULE:
spread across
AZS FOR HIGH AVAILABILITY
within a Region, and across
REGIONS FOR DR
and data residency. AZ names are randomized per account, so use the stable
AZ ID
when you need a fixed physical reference. Always run production in at least 3 AZs.

- [ ] What is the AWS Shared Responsibility Model, and where does the line move?

IF ASKED, SAY THIS
AWS is responsible for security of the cloud (physical datacenters, hardware, hypervisor, managed-service infrastructure, network backbone). The customer is responsible for security in the cloud (data, IAM, OS patching on EC2, security-group config, encryption settings). The line shifts with abstraction: with EC2 you patch the OS; with RDS AWS does; with S3/Lambda you mainly own access config and your data.

AWS is responsible for security **of** the cloud (physical datacenters, hardware, hypervisor, managed-service infrastructure, network backbone). The customer is responsible for security **in** the cloud (data, IAM, OS patching on EC2, security-group config, encryption settings). The line shifts with abstraction: with EC2 you patch the OS; with RDS AWS does; with S3/Lambda you mainly own access config and your data.

PRACTICAL NOTE
A public S3 bucket leak is always a
CUSTOMER
misconfiguration — AWS provides the controls; you must set them. Most "AWS breaches" are customer-side. Knowing exactly which half you own per service is the heart of cloud security reasoning.

- [ ] Why use AWS Organizations and multiple accounts? What are SCPs?

IF ASKED, SAY THIS
The AWS account is the strongest isolation boundary . Separate accounts (dev/staging/prod, plus log-archive and security accounts) give blast-radius isolation, clean per-environment billing, and distinct guardrails. Organizations manages them centrally; Service Control Policies (SCPs) set the maximum permissions for accounts in an OU — they never grant, only bound. Control Tower automates a governed multi-account landing zone.

The AWS account is the **strongest isolation boundary**. Separate accounts (dev/staging/prod, plus log-archive and security accounts) give blast-radius isolation, clean per-environment billing, and distinct guardrails. **Organizations** manages them centrally; **Service Control Policies (SCPs)** set the *maximum* permissions for accounts in an OU — they never grant, only bound. **Control Tower** automates a governed multi-account landing zone.

PRACTICAL NOTE

Effective permission = intersection of SCP and IAM; an SCP `Deny` overrides any IAM `Allow`. Typical SCPs: deny disabling CloudTrail, deny unapproved Regions, deny leaving the org. The management account should run no workloads.

- [ ] Name the AWS Well-Architected pillars and give one practice each.

IF ASKED, SAY THIS

Six pillars: Operational Excellence (IaC, runbooks, observability), Security (least privilege, encryption), Reliability (multi-AZ, auto-recovery, tested DR), Performance Efficiency (right instance types, caching), Cost Optimization (right-sizing, Savings Plans), and Sustainability (efficient resource/Region choices, Graviton).

Six pillars: **Operational Excellence** (IaC, runbooks, observability), **Security** (least privilege, encryption), **Reliability** (multi-AZ, auto-recovery, tested DR), **Performance Efficiency** (right instance types, caching), **Cost Optimization** (right-sizing, Savings Plans), and **Sustainability** (efficient resource/Region choices, Graviton).

PRACTICAL NOTE

Memorize the six — it's a Solutions Architect staple and a genuine design checklist. The master reliability principle:
DESIGN FOR FAILURE
— assume any instance, AZ, or Region can vanish and architect to survive it.

- [ ] Walk through what happens when you run an AWS CLI/API call. What's the control plane vs data plane?

IF ASKED, SAY THIS

The CLI/SDK resolves credentials, builds the request, and SigV4-signs it with your keys, sending it over HTTPS to a regional service endpoint (e.g. ec2.us-east-1.amazonaws.com). AWS verifies the signature (authentication) and IAM evaluates the layered policies (authorization, default-deny). If allowed, the service performs the action.

The CLI/SDK resolves credentials, builds the request, and **SigV4-signs** it with your keys, sending it over HTTPS to a **regional service endpoint** (e.g. `ec2.us-east-1.amazonaws.com`). AWS verifies the signature (authentication) and IAM evaluates the layered policies (authorization, default-deny). If allowed, the service performs the action.

Control plane = managing resources (`RunInstances`, `CreateBucket`); **data plane** = using them (serving traffic, `GetObject`). They scale and fail independently.

PRACTICAL NOTE

A few services are global (IAM, Route 53, CloudFront, billing), anchored in `us-east-1`. The most common "my resource is missing" cause is the
WRONG REGION
. Designing request paths to not depend on the control plane (which can have its own outages) improves resilience.

- [ ] IAM users vs groups vs roles vs policies — what's the difference?

IF ASKED, SAY THIS

User — a long-lived identity for a person or legacy app (has credentials).

- **User** — a long-lived identity for a person or legacy app (has credentials).
- **Group** — a collection of users to attach policies to; not an identity you assume.
- **Role** — an identity with **no long-term credentials** that is *assumed* to get temporary STS credentials; used by services, cross-account access, and federation.
- **Policy** — a JSON document granting/denying permissions, attached to users, groups, or roles (identity-based) or to resources (resource-based).

PRACTICAL NOTE

Best practice: humans federate via IAM Identity Center (SSO) to
ASSUME ROLES
(short-lived creds), apps on EC2/ECS/EKS use
ROLES
, and you avoid IAM users with static keys entirely. Attach policies to groups/roles, not individual users.

- [ ] Explain IAM policy evaluation logic. What wins?

IF ASKED, SAY THIS

Evaluation is default- deny , and an explicit Deny always wins over any Allow. AWS aggregates: identity-based policies, resource-based policies, permission boundaries (a ceiling on the identity), SCPs (org-level ceiling), and session policies. Roughly: the request is allowed only if it's permitted by identity/resource policy AND within the permission boundary AND within the SCP, with no explicit deny anywhere.

Evaluation is default-**deny**, and an **explicit Deny always wins** over any Allow. AWS aggregates: identity-based policies, resource-based policies, **permission boundaries** (a ceiling on the identity), **SCPs** (org-level ceiling), and session policies. Roughly: the request is allowed only if it's permitted by identity/resource policy AND within the permission boundary AND within the SCP, with no explicit deny anywhere.

PRACTICAL NOTE

Permission boundary and SCP

INTERSECT
(limit) — they never grant. For
CROSS-ACCOUNT
access you need both: an identity policy in the calling account allowing `sts:AssumeRole`, and a trust/resource policy in the target account allowing that principal. Use the IAM Policy Simulator to debug "AccessDenied."

- [ ] How does an EC2 instance (or ECS task / Lambda) get AWS credentials securely?

IF ASKED, SAY THIS
Attach an IAM role (via an instance profile for EC2, a task role for ECS, the execution role for Lambda). The SDK/CLI retrieves short-lived credentials automatically — from the Instance Metadata Service (IMDS) on EC2, or the task-role endpoint on ECS. You never store access keys.

Attach an **IAM role** (via an instance profile for EC2, a task role for ECS, the execution role for Lambda). The SDK/CLI retrieves short-lived credentials automatically — from the **Instance Metadata Service (IMDS)** on EC2, or the task-role endpoint on ECS. You never store access keys.

PRACTICAL NOTE
Hard-coding keys in an AMI, image, or repo is the most common catastrophic AWS mistake. Use
IMDSV2
(session-token required) to defend against SSRF stealing instance credentials. On EKS, use
IRSA
(IAM Roles for Service Accounts) so pods get scoped roles via OIDC.

- [ ] How do you let GitHub Actions deploy to AWS without long-lived keys?

IF ASKED, SAY THIS
Use OIDC federation : configure GitHub's OIDC provider as a trusted identity provider in IAM, then create a role whose trust policy allows that provider, scoped by repo/branch claims. The workflow assumes the role and gets short-lived STS credentials — no stored access keys.

Use **OIDC federation**: configure GitHub's OIDC provider as a trusted identity provider in IAM, then create a role whose **trust policy** allows that provider, scoped by repo/branch claims. The workflow assumes the role and gets short-lived STS credentials — no stored access keys.

PRACTICAL NOTE
This eliminates the #1 CI secret-leak risk. Scope the trust policy tightly (specific repo, specific branch/environment) so a fork or another repo can't assume it. The same pattern works for GitLab and other OIDC-capable CI systems.

- [ ] Permission boundaries vs SCPs vs identity policies — when do you use each?

IF ASKED, SAY THIS
Identity policy — grants permissions to a user/role.

- **Identity policy** — grants permissions to a user/role.
- **Permission boundary** — a *ceiling* on what an identity policy can grant; used so you can delegate role-creation to teams without letting them escalate privilege.
- **SCP** — an org/OU-wide *ceiling* applied across whole accounts.

PRACTICAL NOTE
Boundaries and SCPs both
LIMIT, NEVER GRANT
. Classic use of a boundary: let developers create IAM roles for their apps, but bound those roles so they can't grant themselves admin. SCPs enforce org guardrails (allowed Regions, mandatory logging) regardless of account-level IAM.

- [ ] What are your IAM security best practices?

IF ASKED, SAY THIS
Harden and stop using the root user (MFA on, locked away). Enforce least privilege at the narrowest scope. Prefer roles + short-lived credentials (Identity Center/STS) over IAM users and static keys; never commit keys. Require MFA . Use IAM Access Analyzer to find unintended external access and over-broad grants, and run regular access reviews.

Harden and stop using the **root user** (MFA on, locked away). Enforce **least privilege** at the narrowest scope. Prefer **roles + short-lived credentials** (Identity Center/STS) over IAM users and static keys; never commit keys. Require **MFA**. Use IAM Access Analyzer to find unintended external access and over-broad grants, and run regular access reviews.

PRACTICAL NOTE
Start broad-deny and add specific allows rather than granting `*` and trimming later. Rotate any unavoidable long-lived keys. Tag-based and condition-based policies (e.g. `aws:SourceIp`, `aws:RequestedRegion`) add guardrails.

- [ ] What are the core components of a VPC?

IF ASKED, SAY THIS
A VPC is your isolated software-defined network. Core pieces: subnets (a CIDR slice tied to one AZ), route tables (where traffic goes), an Internet Gateway (IGW) (public internet access), a NAT Gateway (outbound internet for private subnets), security groups and NACLs (firewalls), and optionally peering/Transit Gateway/VPN/Direct Connect/endpoints for connectivity.

A **VPC** is your isolated software-defined network. Core pieces: **subnets** (a CIDR slice tied to one AZ), **route tables** (where traffic goes), an **Internet Gateway (IGW)** (public internet access), a **NAT Gateway** (outbound internet for private subnets), **security groups** and **NACLs** (firewalls), and optionally peering/Transit Gateway/VPN/Direct Connect/endpoints for connectivity.

PRACTICAL NOTE
A subnet is
PUBLIC
if its route table sends `0.0.0.0/0` to an IGW and instances have public IPs; otherwise it's private. Plan non-overlapping CIDRs so VPCs can peer. Reserve address space for growth (especially EKS pods with the VPC CNI).

- [ ] How does an instance in a private subnet reach the internet, and how is that different from inbound access?

IF ASKED, SAY THIS
Outbound: route the private subnet's 0.0.0.0/0 to a NAT Gateway in a public subnet, which SNATs traffic out via the IGW. The instance can reach the internet, but the internet cannot initiate connections to it. Inbound access to a private instance comes via a load balancer in public subnets or a bastion/SSM Session Manager — never a public IP on the instance.

Outbound: route the private subnet's `0.0.0.0/0` to a **NAT Gateway** in a public subnet, which SNATs traffic out via the IGW. The instance can reach the internet, but the internet **cannot initiate** connections to it. Inbound access to a private instance comes via a load balancer in public subnets or a bastion/SSM Session Manager — never a public IP on the instance.

PRACTICAL NOTE
COST GOTCHA:
NAT Gateways charge per hour *and* per GB processed — a frequent surprise on the bill. For AWS-service traffic, use
VPC ENDPOINTS
to bypass NAT entirely. Prefer
SSM SESSION MANAGER
over bastions/SSH (no open port, fully audited).

- [ ] Security Group vs Network ACL — what's the difference?

IF ASKED, SAY THIS
Security Group — stateful , instance/ENI level, allow-only rules (return traffic auto-allowed). Evaluated as a whole (no order). Default-deny inbound, allow-all outbound.

- **Security Group** — **stateful**, instance/ENI level, **allow-only** rules (return traffic auto-allowed). Evaluated as a whole (no order). Default-deny inbound, allow-all outbound.
- **Network ACL (NACL)** — **stateless**, subnet level, supports **allow and deny**, rules processed in numbered order. You must explicitly allow return traffic (ephemeral ports).

PRACTICAL NOTE
Use SGs as the primary control (simpler, stateful); use NACLs for coarse subnet-level deny (e.g. block a malicious IP range). Forgetting to allow ephemeral return ports in a NACL is a classic "why is traffic dropped" bug. SGs can reference other SGs as sources — cleaner than IP ranges.

- [ ] VPC peering vs Transit Gateway — when do you outgrow peering?

IF ASKED, SAY THIS
VPC peering is a one-to-one, non-transitive connection between two VPCs over the AWS backbone. With many VPCs it becomes an unmanageable full mesh (N×(N−1)/2 connections), and you can't route A→C through B. Transit Gateway (TGW) is a hub that all VPCs (and on-prem via VPN/DX) attach to, providing transitive routing and centralized control.

VPC peering is a one-to-one, **non-transitive** connection between two VPCs over the AWS backbone. With many VPCs it becomes an unmanageable full mesh (N×(N−1)/2 connections), and you can't route A→C through B. **Transit Gateway (TGW)** is a hub that all VPCs (and on-prem via VPN/DX) attach to, providing **transitive** routing and centralized control.

PRACTICAL NOTE
Move to TGW once you have more than a handful of VPCs or need on-prem + multi-VPC routing. Overlapping CIDRs can't be peered or attached — plan address space org-wide. TGW supports route tables per attachment for segmentation.

- [ ] VPN vs Direct Connect for hybrid connectivity?

IF ASKED, SAY THIS
Site-to-Site VPN — IPsec tunnels over the public internet; quick to set up, encrypted, but subject to internet latency/jitter and bandwidth limits. Direct Connect (DX) — a dedicated private physical link to AWS; consistent low latency, higher and predictable bandwidth, lower data-transfer cost at scale, but slower to provision and not encrypted by default.

Site-to-Site VPN — IPsec tunnels over the public internet; quick to set up, encrypted, but subject to internet latency/jitter and bandwidth limits. **Direct Connect (DX)** — a dedicated private physical link to AWS; consistent low latency, higher and predictable bandwidth, lower data-transfer cost at scale, but slower to provision and not encrypted by default.

PRACTICAL NOTE
Common pattern:
DX AS PRIMARY WITH A VPN BACKUP
(DX isn't inherently redundant — a single DX is a single point of failure). Run encryption over DX (MACsec or a VPN over DX) for sensitive traffic. Attach both to a Transit Gateway for centralized hybrid routing.

☐ What are VPC endpoints (PrivateLink), and Gateway vs Interface endpoints? ▲

IF ASKED, SAY THIS
VPC endpoints let resources reach AWS services privately, without traversing the public internet or a NAT Gateway. Gateway endpoints (S3 and DynamoDB only) add a route-table entry — free. Interface endpoints (PrivateLink) put an ENI with a private IP in your subnet for most other services — billed hourly + per GB.

VPC endpoints let resources reach AWS services privately, without traversing the public internet or a NAT Gateway. **Gateway endpoints** (S3 and DynamoDB only) add a route-table entry — free. **Interface endpoints** (PrivateLink) put an ENI with a private IP in your subnet for most other services — billed hourly + per GB.

PRACTICAL NOTE
Endpoints improve security (traffic stays on the AWS network) and can cut NAT costs (especially S3 gateway endpoints for heavy S3 traffic). Interface endpoints rely on private DNS so the service FQDN resolves to the private IP — misconfigured DNS is the usual failure.

☐ Explain Route 53 routing policies. ▲

IF ASKED, SAY THIS
Simple — one record, no logic.

- **Simple** — one record, no logic.
- **Weighted** — split traffic by percentage (canary, A/B).
- **Latency-based** — route to the Region with lowest latency for the user.
- **Failover** — primary/secondary with health checks (active-passive DR).
- **Geolocation / Geoproximity** — route by user location.
- **Multivalue answer** — return multiple healthy IPs (basic client-side balancing).

PRACTICAL NOTE
Route 53 is a global service and a key DR tool — failover routing + health checks gives cross-region failover at the DNS layer. Remember DNS
TTL CACHING
delays failover; keep TTLs low for records you may need to fail over quickly.

☐ Compare EBS volume types and how you choose. ▲

IF ASKED, SAY THIS
gp3 (general-purpose SSD) — the default; baseline 3,000 IOPS and you provision IOPS/throughput independently of size. Best price/perf for most workloads.

- **gp3** (general-purpose SSD) — the default; baseline 3,000 IOPS and you provision IOPS/throughput independently of size. Best price/perf for most workloads.
- **io2 / io2 Block Express** (provisioned IOPS SSD) — highest IOPS and durability for critical databases.
- **st1** (throughput HDD) — big sequential workloads (logs, big data).
- **sc1** (cold HDD) — infrequently accessed, cheapest.

PRACTICAL NOTE
gp3 decouples IOPS from capacity (gp2 tied IOPS to size — a common reason to migrate). EBS is AZ-scoped; snapshots (to S3) are how you back up and move volumes across AZs/Regions. For ephemeral high-IOPS scratch, instance store (local NVMe) is faster but lost on stop.

☐ On-Demand vs Reserved vs Savings Plans vs Spot — how do you optimize compute cost? ▲

IF ASKED, SAY THIS

On-Demand — pay per second, no commitment; for spiky/unpredictable load.

- **On-Demand** — pay per second, no commitment; for spiky/unpredictable load.
- **Reserved Instances / Savings Plans** — 1- or 3-year commitment for ~up to 70% off; Savings Plans are more flexible (commit to $/hour of compute, not a specific instance). For steady baseline load.
- **Spot** — up to ~90% off spare capacity, but can be reclaimed with a 2-minute warning; for fault-tolerant, interruptible work (batch, CI, stateless workers).

PRACTICAL NOTE
Typical mix: Savings Plans for the steady baseline, On-Demand for variable headroom, Spot for interruptible scale-out. Spot needs graceful handling of the interruption notice and diversification across instance types/AZs to avoid mass reclamation.

How does an Auto Scaling Group work, and what makes scaling reliable?

IF ASKED, SAY THIS
An ASG maintains a desired count of instances from a launch template , spread across AZs, and replaces unhealthy ones (using EC2 or ELB health checks). Scaling policies: target tracking (keep a metric like CPU at a target — simplest), step , and scheduled . Lifecycle hooks let you run setup/drain logic on launch/terminate.

An **ASG** maintains a desired count of instances from a **launch template**, spread across AZs, and replaces unhealthy ones (using EC2 or ELB health checks). Scaling policies: **target tracking** (keep a metric like CPU at a target — simplest), **step**, and **scheduled**. **Lifecycle hooks** let you run setup/drain logic on launch/terminate.

PRACTICAL NOTE
Use
ELB HEALTH CHECKS
(not just EC2 status) so the ASG replaces instances that are running but not serving. Spread across ≥3 AZs.
QUOTA EXHAUSTION
(vCPU limits) during a spike is a classic "it didn't scale when we needed it" incident — pre-request headroom and watch quotas.

What are placement groups, and when do you use each type?

IF ASKED, SAY THIS
Cluster — packs instances close together in one AZ for lowest latency / highest throughput (HPC, tightly-coupled compute). Less resilient (single AZ).

- **Cluster** — packs instances close together in one AZ for lowest latency / highest throughput (HPC, tightly-coupled compute). Less resilient (single AZ).
- **Spread** — places instances on distinct hardware (max 7 per AZ) to minimize correlated failure; for small numbers of critical instances.
- **Partition** — groups instances into partitions on separate racks; for large distributed systems (HDFS, Cassandra) that are partition-aware.

PRACTICAL NOTE
Cluster optimizes performance at the cost of fault isolation; spread/partition optimize fault isolation. Match the group to whether your workload values raw latency (cluster) or failure independence (spread/partition).

IMDSv1 vs IMDSv2 — why does it matter for security?

IF ASKED, SAY THIS
The Instance Metadata Service serves instance data and the role's temporary credentials at 169.254.169.254 . IMDSv1 is a simple request/response — an SSRF vulnerability in an app can be tricked into fetching credentials. IMDSv2 requires a session token obtained via a PUT (with a hop limit), which SSRF typically can't perform, blocking the attack.

The Instance Metadata Service serves instance data and the role's temporary credentials at `169.254.169.254`. **IMDSv1** is a simple request/response — an **SSRF** vulnerability in an app can be tricked into fetching credentials. **IMDSv2** requires a session token obtained via a PUT (with a hop limit), which SSRF typically can't perform, blocking the attack.

PRACTICAL NOTE
Enforce IMDSv2 (require tokens, set hop limit to 1) on all instances — it's a cheap, high-impact hardening step. SSRF stealing instance credentials has caused major real-world breaches; IMDSv2 is the direct mitigation.

ALB vs NLB vs GWLB — which do you choose?

IF ASKED, SAY THIS
ALB (Application LB) — Layer 7 (HTTP/S). Host/path-based routing, WebSocket, redirects, OIDC auth, WAF integration. For web apps/microservices/containers.

- **ALB** (Application LB) — Layer 7 (HTTP/S). Host/path-based routing, WebSocket, redirects, OIDC auth, WAF integration. For web apps/microservices/containers.

- **NLB** (Network LB) — Layer 4 (TCP/UDP/TLS). Ultra-high throughput, very low latency, static IPs / PrivateLink. For non-HTTP, extreme performance, or when you need fixed IPs.
- **GWLB** (Gateway LB) — Layer 3; inserts third-party virtual appliances (firewalls, IDS/IPS) transparently into the traffic path.

PRACTICAL NOTE
(CLB is the legacy Classic LB — avoid for new builds.) ALB for L7 features, NLB for L4 speed/static IPs, GWLB for inline security appliances. ALB routes to
TARGET GROUPS
(instances, IPs, or Lambda) with health checks.

☐ How do target groups, health checks, and sticky sessions work? ▲

IF ASKED, SAY THIS
A target group is the pool the LB routes to (EC2 instances, IPs, containers, or Lambda). The LB runs health checks against each target and only sends traffic to healthy ones; failing targets are drained. Sticky sessions (cookie-based on ALB) pin a client to one target — needed for stateful apps, but they undermine even load distribution and resilience.

A **target group** is the pool the LB routes to (EC2 instances, IPs, containers, or Lambda). The LB runs **health checks** against each target and only sends traffic to healthy ones; failing targets are drained. **Sticky sessions** (cookie-based on ALB) pin a client to one target — needed for stateful apps, but they undermine even load distribution and resilience.

PRACTICAL NOTE
Prefer
STATELESS
apps (store session in Redis/DynamoDB) so you don't need stickiness. Tune health-check path/thresholds carefully — too aggressive and you flap healthy targets out; too lax and you keep sending traffic to broken ones.
CONNECTION DRAINING
(deregistration delay) lets in-flight requests finish during deploys/scale-in.

☐ How do an ALB and an Auto Scaling Group work together for a resilient web tier? ▲

IF ASKED, SAY THIS
The ASG launches instances across AZs and registers them with the ALB's target group; the ALB distributes requests to healthy targets and the ASG replaces unhealthy ones. Target-tracking scaling adds/removes instances based on load (e.g. requests-per-target or CPU). The result is a self-healing, auto-scaling, multi-AZ tier behind one endpoint.

The ASG launches instances across AZs and registers them with the ALB's target group; the ALB distributes requests to healthy targets and the ASG replaces unhealthy ones. Target-tracking scaling adds/removes instances based on load (e.g. requests-per-target or CPU). The result is a self-healing, auto-scaling, multi-AZ tier behind one endpoint.

PRACTICAL NOTE
Use the ALB health check as the ASG's health source so non-serving instances are recycled. Connection draining + lifecycle hooks give graceful instance retirement. This ALB + ASG pattern is the canonical AWS HA web tier.

☐ Explain S3 storage classes and lifecycle policies. ▲

IF ASKED, SAY THIS
S3 Standard — frequent access, default.

- **S3 Standard** — frequent access, default.
- **Intelligent-Tiering** — auto-moves objects between tiers by access pattern (no retrieval fees) — great when access is unpredictable.
- **Standard-IA / One Zone-IA** — infrequent access (One Zone is cheaper but single-AZ).
- **Glacier Instant / Flexible / Deep Archive** — archival, cheapest storage, retrieval latency from ms to hours.

Lifecycle policies automate transitions and expiration (e.g. Standard → IA at 30d → Glacier at 90d → delete at 365d).

PRACTICAL NOTE
Lifecycle automation is a major cost lever for logs/backups. Watch minimum-storage-duration charges and Glacier retrieval latency/cost — don't archive data you might need fast. Intelligent-Tiering is the safe default when you don't know the access pattern.

☐ What durability and availability does S3 provide, and what does versioning add? ▲

IF ASKED, SAY THIS
S3 Standard is designed for 11 nines (99.999999999%) of durability — objects are redundantly stored across multiple devices and AZs — and high availability. Versioning keeps every version of an object so you can recover from accidental overwrites/deletes (a delete just adds a delete marker).

S3 Standard is designed for **11 nines (99.999999999%) of durability** — objects are redundantly stored across multiple devices and AZs — and high availability. **Versioning** keeps every version of an object so you can recover from accidental overwrites/deletes (a delete just adds a delete marker).

PRACTICAL NOTE
Durability ≠ availability ≠ backup. Versioning + MFA-delete + lifecycle protects against accidental and malicious deletion (and ransomware). For cross-Region protection use
CROSS-REGION REPLICATION
.
OBJECT LOCK
(WORM) gives immutability for compliance.

- [ ] How do you encrypt S3 data? Compare SSE-S3, SSE-KMS, and SSE-C.

IF ASKED, SAY THIS
SSE-S3 — AWS manages the keys entirely (AES-256). Simplest.

- **SSE-S3** — AWS manages the keys entirely (AES-256). Simplest.
- **SSE-KMS** — keys in KMS; you get key policies, rotation, and CloudTrail audit of key use. Use **S3 Bucket Keys** to cut KMS request costs at scale.
- **SSE-C** — you supply the key on each request; AWS doesn't store it. Maximum control, maximum operational burden.

PRACTICAL NOTE
Encryption at rest is on by default. Choose SSE-KMS when you need auditability/controlled key access/separation of duties (most regulated workloads). Without Bucket Keys, SSE-KMS on a high-throughput bucket can generate large KMS API costs.

- [ ] Bucket policy vs IAM policy vs ACL — and how do buckets end up public?

IF ASKED, SAY THIS
IAM policies attach to identities; bucket policies are resource-based policies on the bucket (good for cross-account and conditions like aws:SourceVpce); ACLs are the legacy object/bucket grant mechanism (AWS now recommends disabling ACLs). Buckets become public via a permissive bucket policy/ACL granting * .

IAM policies attach to identities; **bucket policies** are resource-based policies on the bucket (good for cross-account and conditions like `aws:SourceVpce`); **ACLs** are the legacy object/bucket grant mechanism (AWS now recommends disabling ACLs). Buckets become public via a permissive bucket policy/ACL granting `*`.

PRACTICAL NOTE
Enable
S3 BLOCK PUBLIC ACCESS
at the account and bucket level — it overrides any policy/ACL that would make objects public, and is the single best guard against the classic open-bucket leak. Use IAM Access Analyzer to detect external access. Disable ACLs and use policies + Block Public Access.

- [ ] What is S3's consistency model, and how do you get high request performance?

IF ASKED, SAY THIS
S3 now provides strong read-after-write consistency for all operations (GET/PUT/LIST) — a PUT is immediately readable (it was eventually consistent for overwrites/deletes before Dec 2020). For performance, S3 scales per prefix (thousands of requests/sec per prefix), so spread keys across prefixes for very high throughput; use multipart upload for large objects and parallelism.

S3 now provides **strong read-after-write consistency** for all operations (GET/PUT/LIST) — a PUT is immediately readable (it was eventually consistent for overwrites/deletes before Dec 2020). For performance, S3 scales per **prefix** (thousands of requests/sec per prefix), so spread keys across prefixes for very high throughput; use **multipart upload** for large objects and parallelism.

PRACTICAL NOTE
The old advice to randomize key prefixes for performance is largely obsolete (S3 auto-scales), but spreading load across prefixes still helps at extreme scale. Strong consistency simplifies pipelines that used to need workarounds. Use
TRANSFER ACCELERATION
or CloudFront for global upload/download latency.

- [ ] RDS Multi-AZ vs read replicas — what problem does each solve?

IF ASKED, SAY THIS
Multi-AZ is for high availability : a synchronous standby in another AZ; on failure AWS fails over automatically (DNS CNAME flips). In classic Multi-AZ the standby is not readable — it's pure resilience, not scaling. Read replicas are for read scaling : asynchronous copies you can read from (and can be cross-Region), but they're not an automatic-failover HA mechanism by themselves.

Multi-AZ is for **high availability**: a synchronous standby in another AZ; on failure AWS fails over automatically (DNS CNAME flips). In classic Multi-AZ the standby is **not readable** — it's pure resilience, not scaling. **Read replicas** are for **read scaling**: asynchronous copies you can read from (and can be cross-Region), but they're not an automatic-failover HA mechanism by themselves.

PRACTICAL NOTE
HA → Multi-AZ; read scaling → read replicas; you often use both. The newer
MULTI-AZ DB CLUSTER

deployment adds two *readable* standbys with faster failover. A read replica can be promoted to a standalone primary (useful for DR or migration).

- [ ] Describe Aurora's architecture and why it differs from standard RDS.

IF ASKED, SAY THIS
Aurora separates compute from a distributed, log-structured storage layer that replicates 6 copies across 3 AZs and self-heals. Up to 15 read replicas share that same storage (so replica lag is minimal and adding replicas doesn't copy data). It exposes a writer endpoint and a load-balanced reader endpoint , and fails over in seconds by promoting a replica.

Aurora separates compute from a **distributed, log-structured storage layer** that replicates 6 copies across 3 AZs and self-heals. Up to 15 read replicas share that same storage (so replica lag is minimal and adding replicas doesn't copy data). It exposes a **writer endpoint** and a load-balanced **reader endpoint**, and fails over in seconds by promoting a replica.

PRACTICAL NOTE
Because storage is shared and distributed, Aurora gives near-synchronous replicas, fast failover, and continuous backup to S3 with point-in-time restore.
AURORA SERVERLESS V2
autoscales capacity. It's MySQL/PostgreSQL-compatible, so a common upgrade path from RDS.

- [ ] DynamoDB fundamentals: partition keys, capacity modes, GSI vs LSI, DAX, Streams.

IF ASKED, SAY THIS
DynamoDB is a managed key-value/document store with single-digit-ms latency. Data is partitioned by the partition key 's hash — a good (high-cardinality, evenly-accessed) key avoids hot partitions . Capacity: on-demand (pay per request, auto-scales) or provisioned (set RCU/WCU, cheaper for steady load). GSI = alternate partition/sort key, own capacity, eventually consistent; LSI = same partition key, alternate sort key, defined at creation. DAX = in-memory cache; Streams = change feed for event-driven processing.

DynamoDB is a managed key-value/document store with single-digit-ms latency. Data is partitioned by the **partition key**'s hash — a good (high-cardinality, evenly-accessed) key avoids **hot partitions**. Capacity: **on-demand** (pay per request, auto-scales) or **provisioned** (set RCU/WCU, cheaper for steady load). **GSI** = alternate partition/sort key, own capacity, eventually consistent; **LSI** = same partition key, alternate sort key, defined at creation. **DAX** = in-memory cache; **Streams** = change feed for event-driven processing.

PRACTICAL NOTE
Hot partitions (e.g. a key like a fixed date or a celebrity user) cause throttling even with capacity to spare — design the key for even distribution. Reads are eventually consistent by default; request strongly-consistent reads when needed (costs more, not available on GSIs). Model access patterns first (single-table design).

- [ ] When do you reach for ElastiCache, and Redis vs Memcached?

IF ASKED, SAY THIS
ElastiCache offloads the database with an in-memory cache for hot reads, session stores, leaderboards, rate limiting, and pub/sub. Redis — rich data structures, persistence, replication, cluster mode, pub/sub, geospatial; the default choice. Memcached — simple, multi-threaded key-value cache that scales horizontally; use when you just need a plain, large, sharded cache with no persistence/replication.

ElastiCache offloads the database with an in-memory cache for hot reads, session stores, leaderboards, rate limiting, and pub/sub. **Redis** — rich data structures, persistence, replication, cluster mode, pub/sub, geospatial; the default choice. **Memcached** — simple, multi-threaded key-value cache that scales horizontally; use when you just need a plain, large, sharded cache with no persistence/replication.

PRACTICAL NOTE
Caching strategies:
CACHE-ASIDE
(lazy load on miss) is most common;
WRITE-THROUGH
keeps cache fresh at write cost. Mind cache
INVALIDATION
and stampede protection (a popular key expiring under load). Redis cluster mode + replicas gives HA.

- [ ] Explain the Lambda execution model, cold starts, and concurrency controls.

IF ASKED, SAY THIS
Lambda runs your code in ephemeral execution environments it creates on demand. A cold start is the first-invocation latency to spin up a new environment (worse for large packages, VPC-attached, or heavy init). Reserved concurrency caps/guarantees a function's concurrency; provisioned concurrency keeps environments pre-warmed to eliminate cold starts for latency-sensitive functions.

Lambda runs your code in ephemeral execution environments it creates on demand. A **cold start** is the first-invocation latency to spin up a new environment (worse for large packages, VPC-attached, or heavy init). **Reserved concurrency** caps/guarantees a function's concurrency; **provisioned concurrency** keeps environments pre-warmed to eliminate cold starts for latency-sensitive functions.

PRACTICAL NOTE

Reduce cold starts: smaller packages, lighter runtimes, move work out of the handler init, and provisioned concurrency for user-facing paths. The account has a default concurrency limit (~1,000, soft) shared across functions — reserved concurrency on one function reduces what's available to others. Idempotency matters (retries can double-invoke).

SQS vs SNS vs EventBridge — when do you use each?

IF ASKED, SAY THIS

SQS — a queue : one consumer group pulls and processes messages; decouples and buffers load. Standard (at-least-once, best-effort order) or FIFO (ordered, exactly-once).

- **SQS** — a **queue**: one consumer group pulls and processes messages; decouples and buffers load. Standard (at-least-once, best-effort order) or FIFO (ordered, exactly-once).
- **SNS** — **pub/sub fan-out**: one message pushed to many subscribers (queues, Lambda, HTTP).
- **EventBridge** — an event bus with **content-based routing rules**, schema registry, and SaaS/AWS-service event sources; richer filtering than SNS.

PRACTICAL NOTE

Classic pattern:
`SNS → MULTIPLE SQS QUEUES`
(fan-out with durable buffering per consumer). Always configure a
`DEAD-LETTER QUEUE (DLQ)`
for poison messages. EventBridge when you need event routing/filtering and integration with many event producers; SQS when you need a durable work queue.

What is API Gateway, and REST API vs HTTP API?

IF ASKED, SAY THIS

API Gateway fronts your APIs with auth, throttling, caching, request/response transformation, and routing to Lambda/HTTP/AWS services. HTTP API is the newer, cheaper, lower-latency option with core features (JWT auth, CORS). REST API is the older, fuller-featured option (request validation, API keys/usage plans, WAF, private endpoints, fine-grained transformations).

API Gateway fronts your APIs with auth, throttling, caching, request/response transformation, and routing to Lambda/HTTP/AWS services. **HTTP API** is the newer, cheaper, lower-latency option with core features (JWT auth, CORS). **REST API** is the older, fuller-featured option (request validation, API keys/usage plans, WAF, private endpoints, fine-grained transformations).

PRACTICAL NOTE

Use HTTP API by default for simple Lambda/proxy backends (lower cost/latency); use REST API when you need its advanced features. Set
`THROTTLING/USAGE PLANS`
to protect backends, and a Lambda authorizer or Cognito/JWT for auth.

What are Step Functions and when would you use them?

IF ASKED, SAY THIS

Step Functions orchestrate multi-step workflows as a state machine — sequencing Lambda functions and AWS service calls with built-in retries, error handling, parallel branches, and human approval steps, without you writing orchestration glue. Standard workflows (long-running, exactly-once, auditable) vs Express (high-volume, short, cheaper).

Step Functions orchestrate multi-step workflows as a state machine — sequencing Lambda functions and AWS service calls with built-in retries, error handling, parallel branches, and human approval steps, without you writing orchestration glue. **Standard** workflows (long-running, exactly-once, auditable) vs **Express** (high-volume, short, cheaper).

PRACTICAL NOTE

Reach for Step Functions when a process has branching, retries, long waits, or coordination across many services — it replaces brittle chains of Lambdas calling Lambdas. The visual state machine + execution history makes complex workflows debuggable and auditable.

ECS vs EKS, and Fargate vs EC2 launch type?

IF ASKED, SAY THIS

ECS is AWS's own container orchestrator — simpler, deeply integrated, no Kubernetes to manage. EKS is managed Kubernetes — portable, huge ecosystem, but more operational surface. Launch type: Fargate is serverless (no nodes to manage/patch, pay per task) ; EC2 means you manage the node fleet (more control, can be cheaper at high steady utilization, supports things Fargate doesn't like daemonsets/GPUs in some cases).

ECS is AWS's own container orchestrator — simpler, deeply integrated, no Kubernetes to manage. **EKS** is managed Kubernetes — portable, huge ecosystem, but more operational surface. **Launch type: Fargate** is serverless (no nodes to manage/patch, pay per task) ; **EC2** means you manage the node fleet (more control, can be cheaper at high steady utilization, supports things Fargate doesn't like daemonsets/GPUs in some cases).

PRACTICAL NOTE

Choose ECS for simplicity in an AWS-only shop; EKS for Kubernetes portability/standardization. Fargate to eliminate node ops; EC2 when you need node-level control or better economics at scale. Fargate Spot cuts cost for interruptible tasks.

- [ ] What is ECR and how do clusters pull from it securely?

IF ASKED, SAY THIS
ECR is AWS's managed container registry. Clusters authenticate via IAM (the node/task/execution role gets ECR pull permissions) rather than static registry credentials. It supports image scanning (basic and enhanced via Inspector), lifecycle policies to expire old images, immutable tags, and cross-Region/cross-account replication.

ECR is AWS's managed container registry. Clusters authenticate via IAM (the node/task/execution role gets ECR pull permissions) rather than static registry credentials. It supports image **scanning** (basic and enhanced via Inspector), lifecycle policies to expire old images, immutable tags, and cross-Region/cross-account replication.

PRACTICAL NOTE
Use
IMMUTABLE TAGS
+ scanning in CI, and lifecycle policies to control storage cost. Private ECR access via a VPC interface endpoint keeps pulls off the public internet. Grant least-privilege pull/push via IAM, not shared credentials.

- [ ] What is IRSA on EKS and why is it the right way to give pods AWS access?

IF ASKED, SAY THIS
IRSA (IAM Roles for Service Accounts) federates a Kubernetes service account to an IAM role via the cluster's OIDC provider. Pods using that service account get scoped, short-lived AWS credentials automatically — no node-wide role, no static keys. Each workload gets exactly the permissions it needs.

IRSA (IAM Roles for Service Accounts) federates a Kubernetes **service account** to an IAM role via the cluster's OIDC provider. Pods using that service account get **scoped, short-lived** AWS credentials automatically — no node-wide role, no static keys. Each workload gets exactly the permissions it needs.

PRACTICAL NOTE
Without IRSA, pods would inherit the broad node instance-profile role (over-privileged, shared by all pods on the node). IRSA enforces least privilege per workload. (EKS Pod Identity is a newer alternative that simplifies the setup.) This is the EKS analogue of least-privilege task roles in ECS.

- [ ] How do CloudWatch metrics, logs, and alarms fit together?

IF ASKED, SAY THIS
CloudWatch Metrics are numeric time-series (built-in per service, plus custom metrics you publish). Logs are streamed from apps/agents into log groups, queryable with Logs Insights . Alarms evaluate a metric against a threshold and trigger actions (SNS notification, Auto Scaling, Lambda remediation). Dashboards visualize it all.

CloudWatch Metrics are numeric time-series (built-in per service, plus custom metrics you publish). **Logs** are streamed from apps/agents into log groups, queryable with **Logs Insights**. **Alarms** evaluate a metric against a threshold and trigger actions (SNS notification, Auto Scaling, Lambda remediation). **Dashboards** visualize it all.

PRACTICAL NOTE
Alert on
SYMPTOMS
(error rate, latency, SLO burn), not just CPU. Composite alarms reduce noise. Metric filters turn log patterns into metrics. EMF (Embedded Metric Format) emits custom metrics from logs cheaply. For high-cardinality/observability at scale, many teams pair CloudWatch with Prometheus/Grafana.

- [ ] CloudTrail vs Config vs CloudWatch — what does each answer?

IF ASKED, SAY THIS
CloudTrail — who did what, when : an audit log of every API call. For security forensics and compliance.

- **CloudTrail** — **who did what, when**: an audit log of every API call. For security forensics and compliance.
- **AWS Config** — **resource configuration state and compliance over time**: tracks how resources are configured, records changes, and evaluates them against rules.
- **CloudWatch** — **operational telemetry**: metrics, logs, alarms about how things are performing.

PRACTICAL NOTE
Audit/forensics → CloudTrail; configuration drift/compliance → Config; performance/health → CloudWatch. Enable CloudTrail org-wide to an immutable log-archive account and protect it with an SCP that forbids disabling it. Config rules can auto-remediate non-compliant resources.

- [ ] What does AWS X-Ray provide?

IF ASKED, SAY THIS
X-Ray is distributed tracing for AWS apps: it follows a request across services (API Gateway → Lambda → DynamoDB, etc.), building a service map and showing where latency and errors occur. It answers "which hop in this distributed request is slow or failing."

X-Ray is distributed tracing for AWS apps: it follows a request across services (API Gateway → Lambda → DynamoDB, etc.), building a service map and showing where latency and errors occur. It answers "which hop in this distributed request is slow or failing."

PRACTICAL NOTE
Tracing is the third observability pillar (with metrics and logs) and essential for microservices/serverless root-cause. Many teams now use **OPENTELEMETRY**
(via the AWS Distro for OpenTelemetry) to stay vendor-neutral while still exporting to X-Ray or other backends.

- [ ] Explain KMS and envelope encryption.

IF ASKED, SAY THIS
KMS manages encryption keys. With envelope encryption , KMS doesn't encrypt your large data directly: it issues a data key , which encrypts the data locally; the data key itself is encrypted by the KMS key (CMK) and stored alongside the ciphertext. To decrypt, the service asks KMS to decrypt the data key, then uses it locally. This scales encryption without sending bulk data to KMS.

KMS manages encryption keys. With **envelope encryption**, KMS doesn't encrypt your large data directly: it issues a **data key**, which encrypts the data locally; the data key itself is encrypted by the KMS **key (CMK)** and stored alongside the ciphertext. To decrypt, the service asks KMS to decrypt the data key, then uses it locally. This scales encryption without sending bulk data to KMS.

PRACTICAL NOTE
Access is governed by
KEY POLICIES
(and grants) — separate from IAM, so you get separation of duties (an admin with S3 access may still be denied decrypt). Customer-managed keys support rotation and CloudTrail audit of every use. Deleting/disabling a key renders its data unreadable — powerful but operationally dangerous.

- [ ] Secrets Manager vs SSM Parameter Store — which and when?

IF ASKED, SAY THIS
Both store config/secrets and integrate with IAM/KMS. Secrets Manager adds automatic rotation (built-in for RDS and custom via Lambda), cross-Region replication, and is purpose-built for secrets — at a per-secret cost. SSM Parameter Store is cheaper/free for standard parameters, great for general config and secrets you rotate yourself (SecureString via KMS).

Both store config/secrets and integrate with IAM/KMS. **Secrets Manager** adds **automatic rotation** (built-in for RDS and custom via Lambda), cross-Region replication, and is purpose-built for secrets — at a per-secret cost. **SSM Parameter Store** is cheaper/free for standard parameters, great for general config and secrets you rotate yourself (SecureString via KMS).

PRACTICAL NOTE
Use Secrets Manager when you need managed rotation (database credentials); use Parameter Store for general config and lower-cost secret storage. Both let apps fetch values at runtime via IAM-scoped roles — never bake secrets into images.

- [ ] What do GuardDuty, Security Hub, WAF, and Shield each do?

IF ASKED, SAY THIS
GuardDuty — threat detection from logs (VPC Flow, CloudTrail, DNS); flags anomalies like crypto-mining or credential exfiltration.

- **GuardDuty** — threat detection from logs (VPC Flow, CloudTrail, DNS); flags anomalies like crypto-mining or credential exfiltration.
- **Security Hub** — aggregates findings (GuardDuty, Inspector, Config) and benchmarks posture against standards (CIS, AWS FSBP).
- **WAF** — Layer 7 web firewall (rules for SQLi/XSS, rate limiting) on ALB/CloudFront/API Gateway.
- **Shield** — DDoS protection (Standard free; Advanced adds higher-layer protection and cost protection).

PRACTICAL NOTE
GuardDuty = detection, Security Hub = aggregation/posture, WAF = app-layer filtering, Shield = DDoS. Together with Config (compliance) and Inspector (vuln scanning) they form a layered defense. Enable GuardDuty + Security Hub org-wide for centralized visibility.

- [ ] What are the four DR strategies, cheapest to most resilient, and how do RTO/RPO drive the choice?

IF ASKED, SAY THIS
Backup & Restore — restore from backups after disaster. Cheapest; high RTO (hours), RPO = backup frequency.

- **Backup & Restore** — restore from backups after disaster. Cheapest; high RTO (hours), RPO = backup frequency.

- **Pilot Light** — core data replicated and minimal services on standby; scale up on failover. Lower RTO.
- **Warm Standby** — a scaled-down but running full stack in the DR Region; scale up on failover. Low RTO.
- **Active-Active (Multi-Region)** — both Regions serve traffic; near-zero RTO/RPO. Most expensive and complex.

PRACTICAL NOTE
RTO
(max time to restore) and
RPO
(max data loss) set by the business pick the strategy and the cost. Tight RTO forbids backup-restore; tight RPO forbids async-only replication. The plan is only real if you
TEST FAILOVER
(game days) and keep capacity available in the secondary Region.

☐ How do you build a multi-Region active-passive architecture on AWS? ▲

IF ASKED, SAY THIS
Replicate data to the secondary Region (S3 Cross-Region Replication, RDS/Aurora cross-Region replicas or global database, DynamoDB global tables), keep infrastructure defined as code so the standby is reproducible, and use Route 53 failover routing with health checks to flip traffic to the secondary when the primary is unhealthy. Warm standby keeps minimal capacity running to absorb the cutover.

Replicate data to the secondary Region (S3 Cross-Region Replication, RDS/Aurora cross-Region replicas or global database, DynamoDB global tables), keep infrastructure defined as code so the standby is reproducible, and use **Route 53 failover routing** with health checks to flip traffic to the secondary when the primary is unhealthy. Warm standby keeps minimal capacity running to absorb the cutover.

PRACTICAL NOTE
Watch DNS TTL on failover. DynamoDB global tables and Aurora Global Database give active-active-ish data layers with low RPO. The hard part is
DATA
consistency and ensuring the standby has enough warm capacity — a cold standby can't absorb full load instantly.

☐ What does Multi-AZ give you, and why is it the baseline for production? ▲

IF ASKED, SAY THIS
Multi-AZ spreads resources across physically separate datacenters within a Region, so the loss of one AZ (power, network, flood) doesn't take down your service. It's the foundational, low-effort high-availability move — load balancers, ASGs, and managed databases all support it. SLA improves to 99.99% for compute spread across zones.

Multi-AZ spreads resources across physically separate datacenters within a Region, so the loss of one AZ (power, network, flood) doesn't take down your service. It's the foundational, low-effort high-availability move — load balancers, ASGs, and managed databases all support it. SLA improves to 99.99% for compute spread across zones.

PRACTICAL NOTE
Single-AZ "production" works in the demo and fails in the real incident. Always run prod across ≥3 AZs. Multi-AZ handles in-Region failures; pair it with multi-Region only when you need protection against a full-Region event or data residency.

☐ How do you approach AWS cost optimization? ▲

IF ASKED, SAY THIS
Make spend visible and bounded: tag everything for cost allocation, use Cost Explorer and Budgets with alarms . Then optimize: right-size over-provisioned resources, buy Savings Plans/RIs for steady load and use Spot for interruptible work, delete idle resources, move data to cheaper S3 tiers via lifecycle, and watch hidden costs (NAT Gateway processing, cross-AZ/egress data transfer, idle EBS/EIPs).

Make spend visible and bounded: **tag** everything for cost allocation, use **Cost Explorer** and **Budgets with alarms**. Then optimize: **right-size** over-provisioned resources, buy **Savings Plans/RIs** for steady load and use **Spot** for interruptible work, delete idle resources, move data to cheaper S3 tiers via lifecycle, and watch hidden costs (NAT Gateway processing, cross-AZ/egress data transfer, idle EBS/EIPs).

PRACTICAL NOTE
NAT Gateway and data-transfer/egress are the classic surprise line items — use VPC endpoints and keep chatty traffic in-AZ. The error budget / unused capacity is also a cost signal: consistently idle headroom means you over-provisioned. Make cost a continuous practice (FinOps), not a quarterly panic.

☐ Reserved Instances vs Savings Plans vs Spot — quick cost guidance? ▲

IF ASKED, SAY THIS
Savings Plans — commit to a steady $/hour of compute for 1–3 years for up to ~70% off; more flexible than RIs (apply across instance families/Regions/Lambda/Fargate). Reserved Instances — similar discount but tied to specific attributes (less flexible; Convertible RIs add some flexibility). Spot — up to ~90% off spare capacity, reclaimable with a 2-minute notice; for fault-tolerant workloads.

Savings Plans — commit to a steady $/hour of compute for 1–3 years for up to ~70% off; more flexible than RIs (apply across instance families/Regions/Lambda/Fargate). **Reserved Instances** — similar discount but tied to specific attributes (less flexible; Convertible RIs add some

flexibility). **Spot** — up to ~90% off spare capacity, reclaimable with a 2-minute notice; for fault-tolerant workloads.

> **PRACTICAL NOTE**
> Cover your stable baseline with Savings Plans, run interruptible/stateless scale-out on Spot, and leave On-Demand for unpredictable headroom. This blend typically cuts compute cost dramatically versus all-On-Demand.

- [ ] Task definition, task, and service — how do these three relate?

> **IF ASKED, SAY THIS**
> A task definition is the blueprint — JSON describing one or more containers, their images, CPU/memory, port mappings, IAM roles, and volumes; it's versioned (each update creates a new revision, never mutates in place). A task is one running instance of that definition. A service is the controller that keeps a desired c

A **task definition** is the blueprint — JSON describing one or more containers, their images, CPU/memory, port mappings, IAM roles, and volumes; it's versioned (each update creates a new revision, never mutates in place). A **task** is one running instance of that definition. A **service** is the controller that keeps a desired count of tasks running, replaces failed ones, and wires them into load balancing and auto-scaling — analogous to a Kubernetes Deployment sitting on top of a Pod template.

- [ ] Fargate vs. EC2 launch type — how do you actually decide?

> **IF ASKED, SAY THIS**
> Fargate removes the underlying instance entirely — AWS manages the host, you pay per task's vCPU/memory-seconds. Default choice for most services: no AMI patching, no capacity planning, scales to zero between traffic, and you can't get host-level SSH access even if you wanted to (which is itself a security win for regu

Fargate removes the underlying instance entirely — AWS manages the host, you pay per task's vCPU/memory-seconds. Default choice for most services: no AMI patching, no capacity planning, scales to zero between traffic, and you can't get host-level SSH access even if you wanted to (which is itself a security win for regulated workloads).

EC2 launch type still wins when you need: GPU instances (Fargate GPU support is limited/pricier at scale), very high sustained CPU/memory density where Fargate's per-task pricing gets expensive vs. bin-packing many tasks onto reserved/spot instances, host-level daemons (custom log/metric agents that need host access), or Windows containers with specific host requirements.

- [ ] Task role vs. task execution role — this trips people up constantly. What's the actual difference?

> **IF ASKED, SAY THIS**
> Execution role : used by the ECS agent itself, before your application code ever runs — pulling the container image from ECR, writing container logs to CloudWatch, decrypting SSM/Secrets Manager values referenced in the task definition's secrets block.

Execution role: used by the ECS agent itself, before your application code ever runs — pulling the container image from ECR, writing container logs to CloudWatch, decrypting SSM/Secrets Manager values referenced in the task definition's `secrets` block.

Task role: assumed by your application code at runtime — what lets your app call `s3:GetObject`, publish to SNS, write to DynamoDB. This is the one that follows least-privilege per-service, not per-cluster.

⚠ The classic mistake: granting broad permissions to the execution role "to make it work," when the app's own S3/DynamoDB calls actually need the task role. Interviewers ask this specifically to see if you understand the boundary between "what the platform needs to run the container" and "what the container's code needs to do its job."

- [ ] What does awsvpc network mode actually change vs. the older bridge mode?

> **IF ASKED, SAY THIS**
> In awsvpc mode, every task gets its own elastic network interface (ENI) with its own private IP, directly in the VPC subnet — meaning its own security group, independent of whatever host it happens to land on. That's what makes per-task security groups possible (task A can only talk to the RDS instance, task B can only

In `awsvpc` mode, every task gets its own elastic network interface (ENI) with its own private IP, directly in the VPC subnet — meaning its own security group, independent of whatever host it happens to land on. That's what makes per-task security groups possible (task A can only talk to the RDS instance, task B can only talk to the queue) instead of one shared host-level security group covering everything co-located on an EC2 instance under bridge mode. It's mandatory for Fargate, and generally recommended for EC2 launch type too — bridge/host modes still exist mainly for legacy or very port-dense workloads where ENI limits per instance become the bottleneck.

- [ ] How do you auto-scale an ECS service on something other than CPU — say, SQS queue depth?

> **IF ASKED, SAY THIS**
> Application Auto Scaling supports target tracking on any CloudWatch metric, not just the built-in CPU/memory ones. For a queue-worker service you'd publish (or use the existing) ApproximateNumberOfMessagesVisible metric and set a target tracking policy aiming for, say, "5 messages per running task" — Auto Scaling then

Application Auto Scaling supports target tracking on any CloudWatch metric, not just the built-in CPU/memory ones. For a queue-worker service you'd publish (or use the existing) `ApproximateNumberOfMessagesVisible` metric and set a target tracking policy aiming for, say, "5 messages per running task" — Auto Scaling then adds/removes tasks to hold that ratio. This is the standard pattern for decoupled, queue-driven workers where CPU alone doesn't reflect real backlog.

```
resource "aws_appautoscaling_policy" "queue_depth" {
  policy_type = "TargetTrackingScaling"
  target_tracking_scaling_policy_configuration {
    target_value = 5
    customized_metric_specification {
      metric_name = "ApproximateNumberOfMessagesVisible"
      namespace   = "AWS/SQS"
      statistic   = "Average"
      dimensions { name = "QueueName", value = aws_sqs_queue.jobs.name }
    }
  }
}
```

- [ ] Rolling update vs. blue/green on ECS — what's actually different operationally?

IF ASKED, SAY THIS
Rolling (the ECS-native default) replaces tasks in place, governed by minimumHealthyPercent / maximumPercent — e.g. 100/200 means it can double capacity temporarily while draining old tasks, never dropping below full capacity. Simple, but the old and new versions serve traffic side by side for the whole rollout, which

Rolling (the ECS-native default) replaces tasks in place, governed by `minimumHealthyPercent` / `maximumPercent` — e.g. 100/200 means it can double capacity temporarily while draining old tasks, never dropping below full capacity. Simple, but the old and new versions serve traffic side by side for the whole rollout, which matters if you can't tolerate mixed-version behavior.

Blue/green via CodeDeploy stands up an entirely new task set behind a second (or the same, with listener rules) ALB target group, shifts traffic — all at once or linearly/canary over time — and keeps the old task set warm for a configurable bake time so a bad deploy is an instant rollback (flip traffic back) rather than a redeploy. Costs more (double capacity during the switch) but gives you a real rollback button instead of "roll forward with a fix."

- [ ] What's a capacity provider strategy, and how does it help with cost efficiency?

IF ASKED, SAY THIS
A capacity provider strategy tells a service how to split placement across multiple capacity sources with weighted ratios — most commonly FARGATE and FARGATE_SPOT . A base of on-demand Fargate keeps a guaranteed floor of stable capacity, and a weight split above that base routes the bulk of scale-out tasks to Spot at u

A capacity provider strategy tells a service how to split placement across multiple capacity sources with weighted ratios — most commonly `FARGATE` and `FARGATE_SPOT` . A `base` of on-demand Fargate keeps a guaranteed floor of stable capacity, and a `weight` split above that base routes the bulk of scale-out tasks to Spot at up to ~70% lower cost, with ECS automatically replacing Spot tasks that get reclaimed. For EC2 launch type, a capacity provider tied to an Auto Scaling Group with *managed scaling* lets ECS itself drive the ASG size based on actual task placement demand, rather than you hand-tuning scaling policies on the ASG separately.

- [ ] How do ECS services find each other without hardcoded IPs?

IF ASKED, SAY THIS
Two patterns, for different traffic shapes. North-south (external/ALB-fronted) traffic goes through target groups — the ALB tracks healthy task IPs itself, you never see them. East-west (service-to-service) traffic typically uses AWS Cloud Map : each service registers its tasks' IPs under a DNS namespace (e.g. orders.i

Two patterns, for different traffic shapes. North-south (external/ALB-fronted) traffic goes through target groups — the ALB tracks healthy task IPs itself, you never see them. East-west (service-to-service) traffic typically uses **AWS Cloud Map**: each service registers its tasks' IPs under a DNS namespace (e.g. `orders.internal`), and ECS keeps that registration in sync as tasks come and go, so other services just resolve `orders.internal` via standard DNS. For richer east-west needs — retries, circuit breaking, mTLS between services — that's where you'd bring in App Mesh or a service mesh sidecar rather than relying on Cloud Map alone.

- [ ] The JD asks for both ECS and EKS. When would you actually reach for ECS over EKS on a new service?

IF ASKED, SAY THIS
Good framing for an interview, not a religious war: ECS wins on operational simplicity — no control plane to manage or upgrade, a much smaller AWS-specific IAM/networking surface to reason about, faster for a small platform team to run well. It's the right default for straightforward service-and-worker workloads that d

Good framing for an interview, not a religious war: ECS wins on operational simplicity — no control plane to manage or upgrade, a much smaller AWS-specific IAM/networking surface to reason about, faster for a small platform team to run well. It's the right default for straightforward service-and-worker workloads that don't need Kubernetes-specific ecosystem tooling.

EKS earns its extra operational weight when you need portability across clouds/on-prem, the CNCF ecosystem (Helm charts vendors ship as their primary distribution format, Argo CD/Flux GitOps, custom operators/CRDs), or finer-grained scheduling control (affinity/anti-affinity, custom schedulers, DaemonSets for node-level agents). A pragmatic answer for a platform team: standardize on EKS as the default for anything ecosystem-dependent, keep ECS available for teams that just need "run this container reliably" without the Kubernetes learning curve.

- [ ] A service's tasks keep cycling: START → RUNNING → STOPPED every 30–60 seconds. Where do you look first?

> **IF ASKED, SAY THIS**
> Order of investigation: aws ecs describe-tasks for the stoppedReason field first — it's usually blunt and specific ("Essential container in task exited," "CannotPullContainerError," a failed health check). If it's an exit, check CloudWatch Logs for that task's log stream for an application crash. If it's CannotPullCont

Order of investigation: `aws ecs describe-tasks` for the `stoppedReason` field first — it's usually blunt and specific ("Essential container in task exited," "CannotPullContainerError," a failed health check). If it's an exit, check CloudWatch Logs for that task's log stream for an application crash. If it's `CannotPullContainerError`, verify the execution role has ECR pull permissions and that the subnet has a route to ECR (NAT gateway or VPC endpoint — a common gap in fully private subnets).

If tasks start healthy then get killed specifically by the ALB, it's almost always the target group health check — wrong path, wrong port, or a health-check grace period too short for a slow-starting app (the service scheduler kills tasks the ALB reports unhealthy, even if the app would've passed 10 seconds later).

- [ ] List all EC2 instances with their ID, state, type, and public IP — both AWS CLI and boto3.

> **IF ASKED, SAY THIS**
> CLI, shaping the output server-side-parsed with --query so it's immediately readable:

CLI, shaping the output server-side-parsed with `--query` so it's immediately readable:

```
aws ec2 describe-instances \
  --query 'Reservations[].Instances[].{ID:InstanceId,State:State.Name,Type:InstanceType,IP:PublicIpAddress}' \
  --output table
```

boto3 — note the nested `Reservations → Instances` structure, a common gotcha even for a single instance:

```
import boto3
ec2 = boto3.client("ec2")
for reservation in ec2.describe_instances()["Reservations"]:
    for i in reservation["Instances"]:
        print(i["InstanceId"], i["State"]["Name"], i["InstanceType"], i.get("PublicIpAddress"))
```

The `.get()` on PublicIpAddress matters — an instance with no public IP won't have that key at all, and a plain `i["PublicIpAddress"]` would raise a KeyError instead of returning None.

- [ ] Get one specific field out of a large describe-instances response, usable directly as a shell variable.

> **IF ASKED, SAY THIS**
> Scope to the one instance, drill to the exact field with --query , and use --output text so you get a bare value instead of quoted JSON:

Scope to the one instance, drill to the exact field with `--query`, and use `--output text` so you get a bare value instead of quoted JSON:

```
PRIVATE_IP=$(aws ec2 describe-instances \
  --instance-ids i-0123456789abcdef0 \
  --query 'Reservations[0].Instances[0].PrivateIpAddress' \
  --output text)
echo "$PRIVATE_IP"
```

The `[0]` indexing on both Reservations and Instances is required even though you only asked for one instance — the API always returns arrays. Forgetting `--output text` is the other common miss: with the default JSON output you'd get a quoted string ("10.0.1.23") that then needs stripping before it's usable as a clean shell variable.

- [ ] List all S3 buckets along with their region. Why isn't this a single simple command?

> **IF ASKED, SAY THIS**
> list-buckets deliberately doesn't return region — you have to call get-bucket-location per bucket separately. A bash loop combining both:

`list-buckets` deliberately doesn't return region — you have to call `get-bucket-location` per bucket separately. A bash loop combining both:

```
for b in $(aws s3api list-buckets --query 'Buckets[].Name' --output text); do
  region=$(aws s3api get-bucket-location --bucket "$b" --query 'LocationConstraint' --output text)
  echo "$b: ${region:-us-east-1}"
done
```

The `${region:-us-east-1}` fallback isn't cosmetic — `get-bucket-location` returns `None`/null specifically for buckets in us-east-1, a legacy quirk from when that was the only region and had no location constraint to report. Forgetting that fallback is a real, common source of a script printing

"bucket: None" instead of the actual region.

- [ ] List every object in an S3 bucket under a prefix, correctly handling pagination for a bucket that could have millions of objects.

IF ASKED, SAY THIS
boto3's paginator abstraction, never manual NextContinuationToken / IsTruncated handling — it's easy to get that logic subtly wrong and silently miss objects past the first page (1000 by default):

boto3's paginator abstraction, never manual `NextContinuationToken` / `IsTruncated` handling — it's easy to get that logic subtly wrong and silently miss objects past the first page (1000 by default):

```
import boto3
s3 = boto3.client("s3")
paginator = s3.get_paginator("list_objects_v2")
for page in paginator.paginate(Bucket="my-bucket", Prefix="logs/2026/"):
    for obj in page.get("Contents", []):
        print(obj["Key"], obj["Size"])
```

The equivalent CLI command is `aws s3api list-objects-v2 --bucket my-bucket --prefix logs/2026/` — the CLI paginates internally for you by default. If you only need a simple listing rather than full object metadata, `aws s3 ls s3://my-bucket/logs/2026/ --recursive` is the simpler, friendlier command for interactive use.

- [ ] Given a VPC, identify which of its subnets are public vs. private from a script alone, with no console access.

IF ASKED, SAY THIS
“Public” isn't a stored attribute on a subnet — it's inferred from whether its route table has a route to an Internet Gateway. The logic: for each subnet, find its associated route table, then check that table's routes for a destination of 0.0.0.0/0 whose target is an IGW (an ID starting with igw-) rather than a NAT G

“Public” isn't a stored attribute on a subnet — it's inferred from whether its route table has a route to an Internet Gateway. The logic: for each subnet, find its associated route table, then check that table's routes for a destination of `0.0.0.0/0` whose target is an IGW (an ID starting with `igw-`) rather than a NAT Gateway or nothing.

```
import boto3
ec2 = boto3.client("ec2")
route_tables = ec2.describe_route_tables()["RouteTables"]

def is_public(subnet_id):
    for rt in route_tables:
        assoc_ids = [a.get("SubnetId") for a in rt["Associations"]]
        if subnet_id not in assoc_ids:
            continue
        for route in rt["Routes"]:
            if route.get("DestinationCidrBlock") == "0.0.0.0/0" and str(route.get("GatewayId", "")).startswith("igw-"):
                return True
    return False
```

The edge case worth naming out loud in an interview: a subnet with no explicit route-table association falls back to the VPC's *main* route table — a script that only checks explicit associations will silently mis-classify those subnets.

- [ ] Write a script to find every security group with SSH open to the whole internet — a real compliance check.

IF ASKED, SAY THIS
The from_port is None check matters — a rule allowing all ports (like an overly-permissive "allow all traffic" rule) has FromPort / ToPort as None rather than -1 or 0 / 65535 , and a naive port-range comparison would throw a TypeError on it instead of correctly flagging it as an even worse finding than a scoped port-22

```
import boto3
ec2 = boto3.client("ec2")
offenders = []

for sg in ec2.describe_security_groups()["SecurityGroups"]:
    for perm in sg.get("IpPermissions", []):
        from_port = perm.get("FromPort")
        to_port = perm.get("ToPort")
        if from_port is None or to_port is None:
            continue
        if from_port <= 22 <= to_port:
            for r in perm.get("IpRanges", []):
```

```
                if r.get("CidrIp") == "0.0.0.0/0":
                    offenders.append(sg["GroupId"])

print(f"Open-SSH security groups: {offenders}")
```

The `from_port is None` check matters — a rule allowing **all** ports (like an overly-permissive "allow all traffic" rule) has `FromPort` / `ToPort` as `None` rather than `-1` or `0` / `65535`, and a naive port-range comparison would throw a TypeError on it instead of correctly flagging it as an even worse finding than a scoped port-22 rule.

- [ ] List IAM roles and show each one's actual permissions. Why does this take more than one API call per role?

IF ASKED, SAY THIS
A role's effective permissions come from several places that each need a separate call: list_attached_role_policies for managed policies (returns ARNs, not the policy content itself — you'd need get_policy + get_policy_version per one to see the actual JSON), list_role_policies for inline policies (returns just names —

A role's effective permissions come from several places that each need a separate call: `list_attached_role_policies` for managed policies (returns ARNs, not the policy content itself — you'd need `get_policy` + `get_policy_version` per one to see the actual JSON), `list_role_policies` for inline policies (returns just names — need `get_role_policy` per name for the document), and the role's trust policy, which comes back directly on `get_role` as `AssumeRolePolicyDocument`.

People underestimate this in a scripting exercise — “show me what this role can do” sounds like one call but is genuinely a fan-out across three or four API calls per role, and the trust policy (who can *assume* the role) is a completely separate document from the permission policies (what the role can *do* once assumed) — conflating the two is a common mistake.

- [ ] What's the first command you should run at the top of any AWS automation script, and why?

IF ASKED, SAY THIS
Returns the Account ID, ARN, and UserId of whatever credentials the script is actually running as — boto3 equivalent is boto3.client("sts").get_caller_identity() . It costs nothing and takes milliseconds, and it's the cheapest possible guard against a genuinely common, genuinely damaging class of incident: a script tha

```
aws sts get-caller-identity
```

Returns the Account ID, ARN, and UserId of whatever credentials the script is actually running as — boto3 equivalent is `boto3.client("sts").get_caller_identity()`. It costs nothing and takes milliseconds, and it's the cheapest possible guard against a genuinely common, genuinely damaging class of incident: a script that assumes it's pointed at dev but is actually running with prod credentials because of a stale environment variable or a misconfigured profile.

In any script with a destructive action (delete, terminate, modify), I'd assert the returned Account ID matches an expected value before proceeding, and fail loudly if it doesn't — a one-line check that has saved real production incidents.

- [ ] Write a Python script to find all unencrypted EBS volumes across every region in the account.

IF ASKED, SAY THIS
The part worth calling out explicitly: EC2/EBS resources are region-scoped, so there's no single “describe all volumes” call — you have to enumerate regions first (IAM, S3 bucket listing, and Route 53 are the main services where a single global call is enough; most others, including EC2, need this per-region loop). Fil

```
import boto3

def find_unencrypted_volumes():
    ec2_global = boto3.client("ec2", region_name="us-east-1")
    regions = [r["RegionName"] for r in ec2_global.describe_regions()["Regions"]]
    offenders = {}
    for region in regions:
        ec2 = boto3.client("ec2", region_name=region)
        vols = ec2.describe_volumes(Filters=[{"Name": "encryption", "Values": ["false"]}])["Volumes"]
        if vols:
            offenders[region] = [v["VolumeId"] for v in vols]
    return offenders
```

The part worth calling out explicitly: EC2/EBS resources are region-scoped, so there's no single “describe all volumes” call — you have to enumerate regions first (IAM, S3 bucket listing, and Route 53 are the main services where a single global call is enough; most others, including EC2, need this per-region loop). Filtering server-side with `Filters=[{"Name":"encryption","Values":["false"]}]` is meaningfully better than fetching everything and filtering in Python — far less data over the wire on an account with a large fleet.

- [ ] List all Lambda functions and flag any running a deprecated or soon-to-be-deprecated runtime.

IF ASKED, SAY THIS

Worth mentioning even without memorizing the exact current deprecation list: AWS deprecates Lambda runtimes on a rolling schedule, and a function on a deprecated runtime can still be invoked but can no longer be updated — which turns into a nasty surprise the day someone actually needs to ship a fix. A scheduled versio

```
import boto3
lam = boto3.client("lambda")
DEPRECATED = {"python3.7", "python3.8", "nodejs14.x", "nodejs16.x"}

paginator = lam.get_paginator("list_functions")
for page in paginator.paginate():
    for fn in page["Functions"]:
        if fn.get("Runtime") in DEPRECATED:
            print(f"DEPRECATED: {fn['FunctionName']} — {fn['Runtime']}")
```

Worth mentioning even without memorizing the exact current deprecation list: AWS deprecates Lambda runtimes on a rolling schedule, and a function on a deprecated runtime can still be invoked but can no longer be *updated* — which turns into a nasty surprise the day someone actually needs to ship a fix. A scheduled version of this exact script feeding into a monthly hygiene report is a very realistic, low-effort piece of automation for this role.

- [ ] Describe every CloudWatch alarm currently in ALARM state, across the account.

IF ASKED, SAY THIS
boto3 equivalent, same server-side filter via StateValue :

```
aws cloudwatch describe-alarms \
  --state-value ALARM \
  --query 'MetricAlarms[].{Name:AlarmName,Metric:MetricName,Reason:StateReason}' \
  --output table
```

boto3 equivalent, same server-side filter via `StateValue`:

```
import boto3
cw = boto3.client("cloudwatch")
alarms = cw.describe_alarms(StateValue="ALARM")["MetricAlarms"]
for a in alarms:
    print(a["AlarmName"], a["MetricName"], a["StateReason"])
```

Filtering with `StateValue="ALARM"` server-side, rather than fetching every alarm and filtering client-side, is the same principle as the security-group and EBS scripts above — let the API narrow the result set whenever it can, instead of paying for the full fetch every time.

- [ ] Find every AWS resource tagged Owner=team-x, across many different resource types, without writing a separate describe call per service.

IF ASKED, SAY THIS
The Resource Groups Tagging API is built exactly for this — one call across most taggable resource types, instead of hand-rolling separate EC2/S3/RDS/Lambda describe calls and stitching results together yourself:

The Resource Groups Tagging API is built exactly for this — one call across most taggable resource types, instead of hand-rolling separate EC2/S3/RDS/Lambda describe calls and stitching results together yourself:

```
import boto3
tagging = boto3.client("resourcegroupstaggingapi")

paginator = tagging.get_paginator("get_resources")
for page in paginator.paginate(TagFilters=[{"Key": "Owner", "Values": ["team-x"]}]):
    for res in page["ResourceTagMappingList"]:
        print(res["ResourceARN"])
```

This is the single highest-leverage API to know for a compliance/inventory/cost-attribution script — it's the difference between a five-line script and separately querying a dozen services and merging the results by hand.

- [ ] Update your local kubeconfig to talk to an EKS cluster from the CLI. What's actually happening under the hood?

IF ASKED, SAY THIS
This writes an entry to ~/.kube/config , but critically it does not embed a static credential. Instead it configures kubectl to invoke aws eks get-token at request time — generating a short-lived, IAM-signed bearer token fresh for every API call, using whatever AWS credentials are active in your shell at that moment.

```
aws eks update-kubeconfig --name my-cluster --region ap-south-1
```

This writes an entry to `~/.kube/config`, but critically it does **not** embed a static credential. Instead it configures kubectl to invoke `aws eks get-token` at request time — generating a short-lived, IAM-signed bearer token fresh for every API call, using whatever AWS credentials are active in your shell at that moment.

That token authenticates you as an AWS principal; whether that principal can actually *do* anything inside the cluster is a completely separate check against the `aws-auth` ConfigMap (or EKS access entries on newer clusters), which maps IAM principals to Kubernetes RBAC identities. This is exactly why a user with perfectly valid AWS credentials but no mapping in `aws-auth` gets a Kubernetes-level "Unauthorized" — they authenticated fine at the AWS layer and failed entirely at the Kubernetes authorization layer, two genuinely separate systems that this one command bridges.

- [ ] What's the actual difference between --query and --filters/--filter in the AWS CLI, and why does it matter at scale?

IF ASKED, SAY THIS
--filters (naming varies slightly by service) is server-side — AWS only returns resources matching the filter, so less data crosses the wire and fewer pagination round-trips are needed. --query is client-side JMESPath, applied locally to the full response after it's already been fetched — it only reshapes or extracts f

--filters (naming varies slightly by service) is **server-side** — AWS only returns resources matching the filter, so less data crosses the wire and fewer pagination round-trips are needed. **--query** is **client-side** JMESPath, applied locally to the full response *after* it's already been fetched — it only reshapes or extracts fields from data you already paid to receive, it doesn't reduce what AWS sends.

On a small account the distinction barely matters. On an account with thousands of EC2 instances or S3 objects, using `--filters` wherever the service supports it (not every API does) is a real, measurable performance difference — fewer API calls, faster scripts, and for pay-per-request-adjacent services, sometimes a real cost difference too. The practical rule: filter server-side with `--filters` whenever possible, and reach for `--query` to shape the output of whatever's left.

- [ ] What's JMESPath, and what's a genuinely tricky JMESPath query you'd actually need in real work?

IF ASKED, SAY THIS
JMESPath is the query language --query uses to filter and reshape JSON responses. Basic field selection and simple projections are easy; the genuinely tricky part is filtering an array based on a nested condition — for example, getting instance IDs only for instances tagged Environment=prod :

JMESPath is the query language `--query` uses to filter and reshape JSON responses. Basic field selection and simple projections are easy; the genuinely tricky part is **filtering an array based on a nested condition** — for example, getting instance IDs only for instances tagged `Environment=prod`:

```
aws ec2 describe-instances \
  --query "Reservations[].Instances[?Tags[?Key=='Environment' \&\& Value=='prod']].InstanceId[]" \
  --output text
```

Two things trip people up here: the nested `[?...]` filter-projection syntax (filtering the Tags array *inside* a filter on Instances), and the trailing empty `[]` — without it, the result is a nested array-of-arrays (one sub-array per Reservation) instead of a single flat list of instance IDs, which silently breaks a naive `for id in $(...)` loop expecting one ID per line.

- [ ] You need one of these one-off describe scripts to run on a schedule and alert on findings, not just print to a terminal. How do you evolve it into real automation?

IF ASKED, SAY THIS
Wrap the already-tested core logic in a Lambda function on an EventBridge Scheduler trigger, rather than a cron job on some box someone has to remember exists. Parameterize account/region/thresholds through environment variables instead of hardcoding them, so the same function can be reused across environments.

Wrap the already-tested core logic in a Lambda function on an EventBridge Scheduler trigger, rather than a cron job on some box someone has to remember exists. Parameterize account/region/thresholds through environment variables instead of hardcoding them, so the same function can be reused across environments.

Critically, don't just let it log and hope someone reads CloudWatch Logs — explicitly publish findings to SNS (or emit a custom CloudWatch metric via `put_metric_data` that an alarm watches) so a real finding is actually alertable, not just discoverable by someone who happens to go looking. And give the function a clean, explicit "0 offenders found" vs. "N offenders found, see details" return convention — the same success/failure discipline from the Python & Scripting section earlier — so it's obvious from the outside whether a run was clean or needs attention, without anyone having to read the full log.

- [ ] What's cross-zone load balancing, and why would you turn it on or off?

IF ASKED, SAY THIS
Without it, an ALB/NLB distributes traffic evenly across AZs but NOT evenly across individual targets if AZs have unequal target counts -- 2 targets in one AZ and 8 in another means the 2 each get 4x the traffic of the 8. Cross-zone load balancing spreads traffic evenly across ALL targets regardless of which AZ they're in. ALBs have it on by default; NLBs have it off by default (and enabling it can add cross-AZ data transfer cost).

The NLB-off-by-default-with-a-cost-implication detail is the part worth knowing precisely -- it's a real, non-obvious operational surprise (uneven load on an NLB with imbalanced AZ target counts) that a lot of people don't discover until they've already hit it.

- [ ] What's connection draining (deregistration delay), and what happens if it's not configured correctly during a deploy?

IF ASKED, SAY THIS
When a target is deregistered (e.g. during a rolling deploy), the load balancer stops sending NEW requests to it immediately, but keeps existing in-flight connections alive for a configurable window (default 300s) so they can finish gracefully instead of being cut off mid-request. Set it too short and in-flight requests get dropped during every deploy; set it too long and deploys take unnecessarily long to complete since the pipeline often waits for full drain.

This is exactly the mechanism that determines whether a rolling deployment is actually zero-downtime for END USERS, not just for the deployment process itself -- a deploy that "succeeds" from the pipeline's perspective can still be dropping real in-flight user requests if this value is misconfigured.

- [ ] How do you tune health check settings, and what's the actual tradeoff between aggressive and lenient thresholds?

IF ASKED, SAY THIS
Key knobs: interval (how often to check), timeout (how long to wait for a response), healthy/unhealthy threshold (consecutive successes/failures needed to flip state). Aggressive (short interval, low unhealthy-threshold) detects real failures fast but is prone to false positives from transient blips, pulling healthy targets out unnecessarily. Lenient is more tolerant of noise but slower to actually remove a genuinely failing target from rotation.

The right answer names the tradeoff explicitly rather than just "set it aggressively for fast detection" -- a target flapping in and out of rotation due to overly-aggressive health checks is itself a real production problem, not just a theoretical risk.

- [ ] Users report intermittent errors, but your application logs show nothing wrong. What load-balancer-level artifact do you check?

IF ASKED, SAY THIS
ALB/ELB access logs (delivered to S3) -- they capture every request at the load balancer level, including ones that never reached your application at all (a 502/504 from the LB itself, a connection that timed out before the app ever saw it). If application logs are clean but users see errors, the load balancer's own logs are often where the actual failure is recorded, since a failure between LB and target never generates an application-level log entry.

This is a genuinely important escalation step to name -- "my app logs are clean" is not the same as "nothing is wrong," and checking LB access logs specifically is how you catch failures happening in the gap between the load balancer and the application.

- [ ] How would you test that a DR plan actually works, rather than trusting it looks correct on paper?

IF ASKED, SAY THIS
Regular DR drills actually failing over (or genuinely exercising the real mechanisms) rather than a tabletop exercise alone — the gap between "the runbook says this works" and "we've actually done it" is where DR plans fail during a real event. Validate not just that compute comes back up in the DR region, but that it

Regular DR drills actually failing over (or genuinely exercising the real mechanisms) rather than a tabletop exercise alone — the gap between "the runbook says this works" and "we've actually done it" is where DR plans fail during a real event.

Validate not just that compute comes back up in the DR region, but that it can reach its dependencies — including the same third-party/ERP connectivity discussed earlier, since a DR environment that can't reach the ERP because the network path was never replicated looks complete but isn't.

- [ ] How would you build resilience against a third-party or ERP system being unavailable, for a system with critical external dependencies?

IF ASKED, SAY THIS
Circuit breaker and graceful degradation for the third-party dependency, a queue-based buffer so upstream processes (RPA, internal services) can keep submitting work while the downstream ERP is temporarily unreachable, and a clear, tested reprocessing path for the backlog once the dependency recovers — "wait and hope n

Circuit breaker and graceful degradation for the third-party dependency, a queue-based buffer so upstream processes (RPA, internal services) can keep submitting work while the downstream ERP is temporarily unreachable, and a clear, **tested** reprocessing path for the backlog once the dependency recovers — "wait and hope nothing was lost" isn't a real resilience strategy for a financial/ERP integration.

- [ ] Design a secure, highly-available 3-tier web application on AWS — walk me through the full architecture.

IF ASKED, SAY THIS
This is close to exactly what was asked yesterday, and it's a standard architect-interview pattern — very likely to recur in some form. Lead with a clean high-level flow, then be ready to drill into any layer when asked — yesterday's interviewer "gradually extracted" the design rather than asking for everything upfront

This is close to exactly what was asked yesterday, and it's a standard architect-interview pattern — very likely to recur in some form. Lead with a clean high-level flow, then be ready to drill into any layer when asked — yesterday's interviewer "gradually extracted" the design rather than asking for everything upfront, and that's the more natural way to actually deliver this.

High-level flow: Client → Route 53 (DNS) → CloudFront (CDN/edge) → WAF → Application Load Balancer → Auto Scaling group of EC2 or EKS pods → RDS (Multi-AZ) for relational data, S3 for static/object storage.

Layer in when asked: Networking — VPC with public subnets (ALB, NAT Gateway) and private subnets (compute, database), Internet Gateway for

public egress, NAT Gateway for private outbound-only access. Security — IAM roles (never long-lived keys) for compute to reach other services, Secrets Manager for DB credentials, security groups scoped per tier. Observability — CloudWatch metrics/alarms/dashboards, CloudTrail for audit. Resilience — Multi-AZ everywhere it matters, a backup strategy for RDS/S3, DR posture if asked.

The delivery habit worth rehearsing: state the client-to-database flow in one breath first (8-10 words per hop), then offer "happy to go deeper on security, scaling, or resilience — where would you like me to start" rather than front-loading every detail unprompted. That's closer to how it was actually asked yesterday, and it reads as confidence rather than a memorized diagram.

- [ ] How would you design a zero-downtime schema change for a production RDS database?

IF ASKED, SAY THIS
Additive-first changes (add a new column as nullable, backfill it, only make it required in a later deploy) rather than a single blocking migration that locks the table. For genuinely large tables, an online schema-change approach (building a shadow table and cutting over) rather than an in-place ALTER that locks for t

Additive-first changes (add a new column as nullable, backfill it, only make it required in a later deploy) rather than a single blocking migration that locks the table. For genuinely large tables, an online schema-change approach (building a shadow table and cutting over) rather than an in-place ALTER that locks for the duration.

Application code deployed to tolerate both the old and new schema during the transition window (read/write compatibility on both sides), so the schema change and the application deploy don't have to be perfectly simultaneous.

- [ ] Route 53 — weighted routing, failover routing, and traffic shifting — walk through when you'd use each and how you'd set it up.

IF ASKED, SAY THIS
Weighted routing for gradual traffic shifting between versions/environments — assign each record a relative weight, adjust over time for a canary-style rollout at the DNS layer. Failover routing for active-passive DR — a primary record with associated health checks, and a secondary record Route 53 automatically starts

Weighted routing for gradual traffic shifting between versions/environments — assign each record a relative weight, adjust over time for a canary-style rollout at the DNS layer. **Failover routing** for active-passive DR — a primary record with associated health checks, and a secondary record Route 53 automatically starts serving if the primary's health check fails. **Weighted + latency-based combined** for multi-region active-active wanting both geographic optimization and controlled traffic distribution.

Worth mentioning explicitly: health checks are what make failover routing actually work — a failover policy with no real health check behind it doesn't actually fail over, it just sits there.

- [ ] How would you decide between Lambda, ECS/EKS, and EC2 for a given workload when architecting a new service?

IF ASKED, SAY THIS
Lambda for event-driven, short-duration (≤15 min), stateless work with unpredictable/spiky traffic — pay per invocation, zero infra management, but cold starts and the time limit rule it out for long-running or highly latency-sensitive synchronous work. ECS/EKS for containerized services needing more runtime control or longer-running proc

Lambda for event-driven, short-duration (≤15 min), stateless work with unpredictable/spiky traffic — pay per invocation, zero infra management, but cold starts and the time limit rule it out for long-running or highly latency-sensitive synchronous work.

ECS/EKS for containerized services needing more runtime control or longer-running processes — ECS for AWS-native simplicity without managing a control plane, EKS for Kubernetes portability/ecosystem. **EC2** when you need full OS-level control or licensing tied to instance identity. Most modern architectures end up a mix — Lambda for glue, ECS/EKS for core services, EC2 for special cases.

- [ ] Walk through designing a REST API on API Gateway backed by Lambda — what are the key architectural decisions?

IF ASKED, SAY THIS
Integration type (Lambda proxy integration is the common default — passes the full request through, your function handles routing/parsing). Authorization (IAM auth, Cognito authorizer, or a custom Lambda authorizer for OAuth/JWT). Stage strategy (dev/staging/prod stages, canary deployments for safer rollouts). Throttling/usage plans (prot

Integration type (Lambda proxy integration is the common default — passes the full request through, your function handles routing/parsing). **Authorization** (IAM auth, Cognito authorizer, or a custom Lambda authorizer for OAuth/JWT). **Stage strategy** (dev/staging/prod stages, canary deployments for safer rollouts). **Throttling/usage plans** (protecting backend Lambda concurrency, API keys for third-party rate-limiting). **Request validation** at the Gateway level — catching malformed requests before they even invoke Lambda.

- [ ] How would you architect EC2 for a workload that needs to be highly available across a region?

IF ASKED, SAY THIS
Auto Scaling Group spanning multiple AZs (never single-AZ for anything production-critical), behind an ALB/NLB distributing traffic and health-checking instances, with launch templates (not launch configurations) for a consistent, reproducible config. Scaling policies tied to actual demand signals (target tracking on CPU/request count, or

Auto Scaling Group spanning multiple AZs (never single-AZ for anything production-critical), behind an ALB/NLB distributing traffic and health-checking instances, with **launch templates** (not launch configurations) for a consistent, reproducible config.

Scaling policies tied to actual demand signals (target tracking on CPU/request count, or a custom CloudWatch metric closer to real load), and instance refresh capability for rolling out AMI updates without a big-bang cutover.

- [ ] What's your approach to S3 architecture for an enterprise platform — bucket structure, security, cost management?

IF ASKED, SAY THIS
Bucket-per-purpose or a well-namespaced prefix structure depending on how many teams/environments need genuinely separate access boundaries. Security: block public access at the account level by default, bucket policies plus IAM, SSE-KMS for anything sensitive, versioning plus MFA delete for anything that can't tolerate accidental deletio

Bucket-per-purpose or a well-namespaced prefix structure depending on how many teams/environments need genuinely separate access boundaries. **Security:** block public access at the account level by default, bucket policies plus IAM, SSE-KMS for anything sensitive, versioning plus MFA delete for anything that can't tolerate accidental deletion.

Cost: lifecycle policies transitioning cold data to IA/Glacier automatically, and Storage Lens/Cost Explorer to see where spend actually concentrates.

- [ ] ECS vs. EKS specifically — what would push you toward one over the other for a new containerized platform?

IF ASKED, SAY THIS
ECS when the org is AWS-native without existing Kubernetes investment — simpler control plane, Fargate removes even node management, generally faster to production without deep K8s expertise. EKS when you need Kubernetes-ecosystem portability (multi-cloud, existing Helm charts/operators) or the team is already K8s-centric. For a role arch

ECS when the org is AWS-native without existing Kubernetes investment — simpler control plane, Fargate removes even node management, generally faster to production without deep K8s expertise. **EKS** when you need Kubernetes-ecosystem portability (multi-cloud, existing Helm charts/operators) or the team is already K8s-centric.

For a role architecting across enterprise clients with varying existing investment, being able to make and justify this call per-engagement matters more than a fixed default.

- [ ] What's Fargate, and when would you choose it over EC2 launch type for ECS/EKS?

IF ASKED, SAY THIS
Serverless compute for containers — no EC2 instances to provision, patch, or scale yourself; billed per task/pod resource allocation. Choose it when operational simplicity matters more than fine-grained cost optimization or specialized instance requirements (GPU workloads, high-density bin-packing still often favor self-managed EC2 node g

Serverless compute for containers — no EC2 instances to provision, patch, or scale yourself; billed per task/pod resource allocation. Choose it when operational simplicity matters more than fine-grained cost optimization or specialized instance requirements (GPU workloads, high-density bin-packing still often favor self-managed EC2 node groups).

Common pattern: Fargate for most general services, EC2 node groups reserved for workloads with specific compute needs Fargate doesn't support well.

- [ ] How would you design ECR into a broader CI/CD and deployment architecture?

IF ASKED, SAY THIS
Private repositories per service/team with push permissions restricted to CI/CD service roles (not broad developer push access), image scanning enabled gating promotion of vulnerable images, lifecycle policies expiring untagged/old images automatically, and immutable tags for anything deployed to production — so a tag can't be silently ov

Private repositories per service/team with push permissions restricted to CI/CD service roles (not broad developer push access), **image scanning enabled** gating promotion of vulnerable images, lifecycle policies expiring untagged/old images automatically, and **immutable tags** for anything deployed to production — so a tag can't be silently overwritten after the fact, a real audit/reproducibility concern.

- [ ] Walk through your approach to VPC architecture for a new enterprise platform — subnet design, routing.

IF ASKED, SAY THIS
Multi-AZ subnet design with clear tiering — public subnets for internet-facing resources (load balancers, NAT gateways), private subnets for application compute, often a further-isolated data subnet tier for databases with no internet route at all. Route tables scoped per tier (public → internet gateway, private → outbound via NAT, data →

Multi-AZ subnet design with clear tiering — public subnets for internet-facing resources (load balancers, NAT gateways), private subnets for application compute, often a further-isolated data subnet tier for databases with no internet route at all.

Route tables scoped per tier (public → internet gateway, private → outbound via NAT, data → no outbound internet route). **CIDR planning up front** matters more than expected — undersizing a VPC's address space is genuinely painful to fix later, especially once peering/Transit Gateway attachments exist.

- [ ] When would you use Transit Gateway versus VPC peering for connecting multiple VPCs?

IF ASKED, SAY THIS
VPC peering for a small, stable number of point-to-point connections — simple, no extra cost beyond data transfer, but no transitive routing (each pair needs its own connection — an unmanageable mesh past a handful of VPCs). Transit Gateway beyond a few VPCs, or for hub-and-spoke connectivity across many VPCs and on-prem networks — centra

VPC peering for a small, stable number of point-to-point connections — simple, no extra cost beyond data transfer, but no transitive routing (each pair needs its own connection — an unmanageable mesh past a handful of VPCs).

Transit Gateway beyond a few VPCs, or for hub-and-spoke connectivity across many VPCs and on-prem networks — centralized routing, transitive routing support, and the natural hub for connecting Direct Connect/VPN cleanly into a multi-VPC environment.

- [ ] Direct Connect vs. Site-to-Site VPN — how would you decide which to recommend to a client for hybrid connectivity?

IF ASKED, SAY THIS
Direct Connect for consistent, high-bandwidth, low-latency needs — a dedicated physical connection bypassing the public internet, right for latency-sensitive workloads or substantial predictable bandwidth (large migrations, real-time hybrid traffic). VPN for faster setup, lower cost, when volume/latency sensitivity doesn't justify Direct

Direct Connect for consistent, high-bandwidth, low-latency needs — a dedicated physical connection bypassing the public internet, right for latency-sensitive workloads or substantial predictable bandwidth (large migrations, real-time hybrid traffic).

VPN for faster setup, lower cost, when volume/latency sensitivity doesn't justify Direct Connect's cost and lead time (provisioning a circuit can take weeks). Common pattern: VPN as interim connectivity while DX provisions, and as a DX failover path even after.

- [ ] How would you use PrivateLink in an architecture connecting to third-party or ERP systems?

IF ASKED, SAY THIS
PrivateLink exposes a service via a private endpoint inside your VPC, so traffic never traverses the public internet — directly relevant given this JD's ERP integration (SAP/Oracle/BaaN) and third-party connectivity scope, where keeping that traffic off the public internet is often a real security/compliance requirement. Where the ERP/thi

PrivateLink exposes a service via a private endpoint inside your VPC, so traffic never traverses the public internet — directly relevant given this JD's ERP integration (SAP/Oracle/BaaN) and third-party connectivity scope, where keeping that traffic off the public internet is often a real security/compliance requirement.

Where the ERP/third party doesn't natively support PrivateLink, the alternative is routing through Direct Connect/VPN to reach it privately, or a VPC endpoint service if you're exposing an internal service to a partner.

- [ ] How would you architect Route 53 and DNS for a multi-region or hybrid environment?

IF ASKED, SAY THIS
Public hosted zones with routing policy matched to the need — latency-based for global multi-region services, failover for active-passive DR, weighted for canary/blue-green traffic shifting. Private hosted zones for internal-only resolution, associated with the specific VPCs that need them. For hybrid environments, Route 53 Resolver (inbo

Public hosted zones with routing policy matched to the need — latency-based for global multi-region services, failover for active-passive DR, weighted for canary/blue-green traffic shifting. Private hosted zones for internal-only resolution, associated with the specific VPCs that need them.

For hybrid environments, **Route 53 Resolver** (inbound/outbound endpoints) lets on-prem DNS and VPC DNS resolve each other's names — frequently needed but easy to overlook when integrating with on-prem ERP systems.

- [ ] What's your approach to load balancer selection — ALB vs NLB vs Gateway Load Balancer — for different parts of this kind of platform?

IF ASKED, SAY THIS
ALB for HTTP/HTTPS needing Layer 7 features — path/host-based routing, WebSocket support, native integration with API-style microservices and ECS/EKS target groups. NLB for ultra-low-latency, high-throughput TCP/UDP, or when you need a static/Elastic IP for the load balancer (common when a partner system needs to allowlist a fixed IP — re

ALB for HTTP/HTTPS needing Layer 7 features — path/host-based routing, WebSocket support, native integration with API-style microservices and ECS/EKS target groups. **NLB** for ultra-low-latency, high-throughput TCP/UDP, or when you need a static/Elastic IP for the load balancer (common when a partner system needs to allowlist a fixed IP — relevant again for ERP/third-party connectivity). **Gateway Load Balancer** specifically for inserting third-party virtual appliances transparently into the traffic path.

- [ ] How would you design network security (security groups, NACLs) for a platform integrating with sensitive third-party/ERP systems?

IF ASKED, SAY THIS
Security groups as the primary, stateful control — scoped tightly per tier (LB SG allows inbound from internet on 443 only, app-tier SG allows inbound only from the LB SG, data-tier SG allows inbound only from the app-tier SG) rather than broad CIDR rules. NACLs as a coarser, stateless secondary layer at the subnet boundary. For third-par

Security groups as the primary, stateful control — scoped tightly per tier (LB SG allows inbound from internet on 443 only, app-tier SG allows inbound only from the LB SG, data-tier SG allows inbound only from the app-tier SG) rather than broad CIDR rules. NACLs as a coarser, stateless secondary

layer at the subnet boundary.

For third-party/ERP connectivity specifically, **egress rules deserve the same scrutiny as ingress** — an app tier that can reach anywhere outbound is a real data-exfiltration risk if compromised.

- [ ] What's the relationship between AWS Organizations, Control Tower, and Landing Zone — how do they fit together?

> **IF ASKED, SAY THIS**
> Organizations is the underlying multi-account management primitive (consolidated billing, OU structure, SCPs). Control Tower is a managed service on top that automates setting up a well-architected multi-account environment — it deploys a Landing Zone (foundational accounts like log archive and audit, baseline SCPs, account provisioning v

Organizations is the underlying multi-account management primitive (consolidated billing, OU structure, SCPs). **Control Tower** is a managed service on top that automates setting up a well-architected multi-account environment — it deploys a **Landing Zone** (foundational accounts like log archive and audit, baseline SCPs, account provisioning via Account Factory) so you're not hand-building that governance scaffolding yourself.

In short: Organizations is the primitive, Control Tower is the automation layer, Landing Zone is the resulting baseline it sets up.

- [ ] How would you design an OU (Organizational Unit) structure for a new enterprise AWS Organization?

> **IF ASKED, SAY THIS**
> Driven by how governance/policy actually needs to differ, not just org-chart mirroring — a common pattern separates Security/Log Archive (highly restricted), Infrastructure/Shared Services (networking, CI/CD tooling), and workload OUs split by environment or business unit, each with SCPs matching that OU's risk profile. The goal: policies

Driven by how governance/policy actually needs to differ, not just org-chart mirroring — a common pattern separates Security/Log Archive (highly restricted), Infrastructure/Shared Services (networking, CI/CD tooling), and workload OUs split by environment or business unit, each with SCPs matching that OU's risk profile.

The goal: policies applied at the right altitude — org-wide for universal guardrails, OU-level for environment/BU-specific rules — without constant per-account exceptions.

- [ ] What FinOps practices would you build into a platform like this from the start, rather than bolting on later?

> **IF ASKED, SAY THIS**
> Tagging enforced at creation time (SCPs denying creation without required tags, or Config rules flagging non-compliance), since untagged resources are nearly impossible to attribute to a cost center retroactively. Budgets and anomaly detection alerting proactively rather than discovering overspend at month-end. Regular right-sizing review

Tagging enforced at creation time (SCPs denying creation without required tags, or Config rules flagging non-compliance), since untagged resources are nearly impossible to attribute to a cost center retroactively. Budgets and anomaly detection alerting proactively rather than discovering overspend at month-end. Regular right-sizing review (Compute Optimizer recommendations, idle/underutilized resource checks) as an ongoing habit, not a one-time exercise — cost drift happens continuously as workloads evolve.

- [ ] How would you approach cost allocation and chargeback for a multi-account environment serving multiple business units or client engagements?

> **IF ASKED, SAY THIS**
> Consolidated billing at the Organizations level with cost allocation tags and/or account-level separation (one account per business unit/client makes chargeback straightforward — cost is just that account's bill), combined with Cost Explorer/Cost and Usage Reports sliced by tag for anything shared across accounts (shared services, central

Consolidated billing at the Organizations level with cost allocation tags and/or account-level separation (one account per business unit/client makes chargeback straightforward — cost is just that account's bill), combined with Cost Explorer/Cost and Usage Reports sliced by tag for anything shared across accounts (shared services, centralized logging) needing proportional allocation.

- [ ] What governance guardrails would you put in place at the Organization level to prevent cost or security surprises, without being so restrictive that teams can't move?

> **IF ASKED, SAY THIS**
> SCPs for hard boundaries that should never be crossed regardless of who's asking (preventing CloudTrail/Config disabling, restricting to approved regions, blocking root-account use outside true emergencies), combined with softer advisory guardrails (Config rules that flag rather than block, budget alerts rather than hard caps) for things

SCPs for hard boundaries that should never be crossed regardless of who's asking (preventing CloudTrail/Config disabling, restricting to approved regions, blocking root-account use outside true emergencies), combined with softer advisory guardrails (Config rules that flag rather than block, budget alerts rather than hard caps) for things needing human judgment.

The balance matters — over-restricting with SCPs on things that should be a conversation is how platform teams end up bypassed via exception requests that erode the whole governance model's credibility.

Azure

- [ ] Explain the Azure resource hierarchy: Management Group, Subscription, Resource Group, Resource.

IF ASKED, SAY THIS
Management Groups organize many subscriptions for governance at scale; subscriptions are billing and access boundaries; resource groups are logical containers grouping resources with a shared lifecycle; and resources are the individual services. RBAC and Azure Policy applied at a higher level inheri

- Management Groups sit at the top to organize multiple subscriptions and apply governance (policies, RBAC) at scale.
- Subscriptions are billing and access boundaries containing resources. Resource Groups are logical containers grouping related resources with a shared lifecycle.
- Resources are the actual services (VMs, storage, AKS). RBAC and Azure Policy can be applied at any level and inherit downward.

- [ ] What is Microsoft Entra ID (Azure AD) and how does it relate to RBAC?

IF ASKED, SAY THIS
Microsoft Entra ID (formerly Azure AD) is Azure's identity provider, holding users, groups, service principals, and managed identities with SSO and conditional access. Azure RBAC then grants access by assigning roles like Contributor or Reader to those identities at a chosen scope, inheriting downwa

- Microsoft Entra ID (formerly Azure AD) is the cloud identity provider — users, groups, service principals, and managed identities, plus SSO and conditional access.
- Azure RBAC grants access by assigning roles (Owner, Contributor, Reader, or custom) to identities at a scope (management group/subscription/RG/resource).
- Managed identities let Azure resources authenticate to other services without stored credentials — analogous to AWS IAM roles.

- [ ] What is AKS and what does Azure manage for you?

IF ASKED, SAY THIS
AKS is Azure's managed Kubernetes: Azure operates the control plane for you (free) while you manage and pay for worker node pools. It integrates with Entra ID for authentication, Azure CNI or kubenet for networking, Azure Monitor for observability, and the cluster autoscaler for node scaling. Node p

- Azure Kubernetes Service (AKS) is managed Kubernetes. Azure runs and manages the control plane (API server, etcd, scheduler) for free; you manage and pay for the worker node pools.
- It integrates with Entra ID for auth, Azure CNI/kubenet for networking, Azure Monitor for observability, and the cluster autoscaler for node scaling.
- Node pools let you separate workloads (system vs user, GPU pools, spot pools).

- [ ] Compare Azure VMs, App Service, Container Instances, and Functions.

IF ASKED, SAY THIS
Azure VMs give full IaaS control with OS management. App Service is managed PaaS for web apps and APIs with built-in scaling and deployment slots. Container Instances run serverless single containers without orchestration, and Functions are serverless event-driven compute billed on consumption. As w

- Azure VMs: full IaaS control, you manage the OS. App Service: managed PaaS for web apps/APIs with built-in scaling and deployment slots.
- Azure Container Instances (ACI): serverless single containers, fast start, no orchestration. Azure Functions: serverless event-driven functions, consumption billing.
- Spectrum from control (VMs) to minimal ops (Functions); AKS sits in the middle for full orchestration.

- [ ] Explain Azure networking: VNet, NSG, and peering.

IF ASKED, SAY THIS
A Virtual Network (VNet) is the isolated network split into subnets, and Network Security Groups filter traffic at the subnet or NIC level with priority-ordered allow/deny rules — roughly the equivalent of AWS Security Groups plus NACLs. VNet peering links VNets for private cross-network or cross-re

- A Virtual Network (VNet) is the isolated network, divided into subnets. Network Security Groups (NSGs) filter traffic at subnet or NIC level with priority-ordered allow/deny rules.
- VNet peering connects VNets (same or cross-region) for private, low-latency communication. A NAT Gateway provides outbound internet for private subnets.
- Azure Firewall / Application Gateway add L7 and centralized egress control.

- [ ] What is the difference between AKS System Node Pool and User Node Pool and why does it matter?

> **IF ASKED, SAY THIS**
> This is an AKS-specific question that tests real production experience: System Node Pool: - Runs kube-system pods: CoreDNS, kube-proxy, metrics-server, Container Insights agent - Has a taint: CriticalAddonsOnly=true:NoSchedule - Must always have at least 1 node running (cannot scale to zero) - Use S

This is an AKS-specific question that tests real production experience:

System Node Pool:
- Runs kube-system pods: CoreDNS, kube-proxy, metrics-server, Container Insights agent
- Has a taint: CriticalAddonsOnly=true:NoSchedule
- Must always have at least 1 node running (cannot scale to zero)
- Use Standard_D2s_v3 or similar - does not need to be large

User Node Pool:
- Runs your application workloads
- Can scale to zero (cost saving for dev/staging during off-hours)
- Can be Spot instances (60-70% cost saving for stateless workloads)
- Can have multiple pools with different VM sizes (GPU pool for ML, high-memory for analytics)

Why the separation matters in production:
Without separation, your payment service and CoreDNS share the same nodes. If a misbehaving payment pod causes OOM on the node, CoreDNS gets evicted too. Now ALL pods in the cluster lose DNS resolution. Everything breaks.

With system pool tainted: your app pods cannot be scheduled on the system pool even by accident. CoreDNS is protected. System components stay stable regardless of what application workloads do.

Setting it up:
```
az aks nodepool add --cluster-name mycluster --resource-group rg --name apppool --node-count 3 --node-vm-size Standard_D4s_v3 --mode User --spot-max-price -1
```

- [ ] How does Azure Workload Identity work and how is it different from Pod Managed Identity (the old way)?

> **IF ASKED, SAY THIS**
> Workload Identity is the modern, secure way for AKS pods to authenticate to Azure services without credentials: Old way - Pod Managed Identity (AAD Pod Identity): - A DaemonSet intercepts IMDS requests from pods and swaps identities - Required privileged DaemonSet with access to Azure AD - Had a kno

Workload Identity is the modern, secure way for AKS pods to authenticate to Azure services without credentials:

Old way - Pod Managed Identity (AAD Pod Identity):
- A DaemonSet intercepts IMDS requests from pods and swaps identities
- Required privileged DaemonSet with access to Azure AD
- Had a known security issue: any pod on the node could potentially steal another pod's identity
- Microsoft deprecated this approach

New way - Workload Identity:
1. AKS cluster has OIDC issuer URL configured
2. You create a managed identity in Azure
3. You create a federated credential linking the managed identity to a specific K8s ServiceAccount
4. Pod runs with that ServiceAccount
5. Azure AD Workload Identity webhook injects AZURE_CLIENT_ID, AZURE_TENANT_ID, AZURE_FEDERATED_TOKEN_FILE into the pod
6. Azure SDK uses these to call Azure AD and get short-lived tokens

Setup commands:
```
az identity create --name payments-identity --resource-group rg az aks update --enable-oidc-issuer --enable-workload-identity az identity federated-credential create --name payments-federated --identity-name payments-identity --issuer $(az aks show --query oidcIssuerProfile.issuerUrl -o tsv) --subject system:serviceaccount:production:payments-sa
```

Why better: No DaemonSet, no privileged access. Each pod gets its own identity. Token exchange happens at the Azure AD level, not on the node. Completely audit-able. Same principle as AWS IRSA.

- [ ] Explain Azure Data Factory and Databricks together. How do you ensure pipeline reliability as an SRE?

> **IF ASKED, SAY THIS**
> The modern financial data platform on Azure: Architecture: Medallion Pattern On-prem SQL Server → ADF Copy Activity → ADLS Gen2 Bronze layer → Databricks notebook → Silver layer (cleaned) → Gold layer (aggregated) → Azure SQL → Power BI ADF role - orchestrator only: - Moves data, does NOT transform

The modern financial data platform on Azure:

Architecture: Medallion Pattern
On-prem SQL Server → ADF Copy Activity → ADLS Gen2 Bronze layer → Databricks notebook → Silver layer (cleaned) → Gold layer (aggregated) → Azure SQL → Power BI

ADF role - orchestrator only:
- Moves data, does NOT transform it. Transformation is Databricks' job.
- Pipeline triggers: schedule, event (blob landed), manual, tumbling window
- Activities: Copy Data, Execute Databricks Notebook, Stored Procedure, Web, Lookup, If Condition
- Integration Runtimes: Azure IR (cloud-to-cloud), Self-hosted IR (on-prem connectivity)

Databricks role - transformation engine:
- Apache Spark cluster runs Python/SQL/Scala notebooks
- Delta Lake: ACID transactions + time travel (query data as it was yesterday = audit trail)
- Unity Catalog: fine-grained access control + lineage tracking (mandatory for financial compliance)

SRE reliability for ADF + Databricks:
1. Pipeline SLO: "Regulatory report must complete by 06:00 AM daily." Alert at 05:30 if not finished.
2. ADF retry policy: max retries=3, interval=5min on every activity. Handles transient Azure service hiccups.
3. Databricks cluster config: auto-scaling min=4 max=20 workers, auto-terminate after 30min idle.
4. Data quality gates in Databricks: row count check - if today's rows are 20% below yesterday, fail the pipeline. Prevents silent data corruption reaching reporting layer.
5. Delta Lake time travel recovery: if bad data written: `RESTORE TABLE silver.transactions TO VERSION AS OF 10` - data restored in seconds.
6. Cost alert: DBU (Databricks Unit) budget alarm via Azure Cost Management - page if daily spend exceeds threshold.

- [ ] What's the one-line mental model for when Azure is the natural fit over AWS?

IF ASKED, SAY THIS
Azure = a Microsoft-integrated city. If the company already runs Office 365, Active Directory, SQL Server, or .NET, Azure is the natural home — everything integrates natively, and Azure AD/Entra ID is the identity backbone for essentially everything, not just Azure-native services.

This is a good framing to lead with in an interview because it shows you're choosing the cloud based on organizational fit, not brand preference — the same instinct interviewers want to see in an AWS-vs-Azure architecture-decision question.

- [ ] Give the AWS-to-Azure service mapping you'd want ready for a dual-cloud interview.

IF ASKED, SAY THIS
VPC→VNet, EC2→VM, EKS→AKS, Lambda→Azure Functions, S3→Blob Storage, RDS→Azure SQL, DynamoDB→Cosmos DB, ElastiCache→Azure Cache for Redis, IAM Roles/IRSA→Managed/Workload Identity, CloudTrail→Activity Log, GuardDuty→Defender for Cloud, AWS Config→Azure Policy, Secrets Manager→Key Vault, ECR→ACR, Kinesis→Event Hub, SNS/SQS→Service Bus, CloudWatch→Azure Monitor, X-Ray→App Insights.

Having this mapping ready cold is what separates 'knows AWS and has heard of Azure' from 'can genuinely operate in either' in an interview — pick 3-4 pairs you can explain the subtle differences on (not just the name mapping), e.g. Managed Identity vs IRSA both remove the need for stored credentials but plug into the identity model differently.

- [ ] AKS system node pool vs user node pool — what's the difference and why does it matter operationally?

IF ASKED, SAY THIS
The system pool runs cluster-critical pods (CoreDNS, kube-proxy, metrics-server) and carries a CriticalAddonsOnly taint so your workloads can't land there and starve cluster services — it must always have at least 1 node and can't scale to zero. The user pool runs your actual application workloads and is where autoscaling/spot instances belong.

A common production mistake is putting everything in one pool — under load, application pods can then starve CoreDNS or metrics-server of resources, which causes cluster-wide symptoms (DNS flakiness, HPA misbehaving) that look nothing like their actual root cause.

- [ ] Walk through Azure's core networking building blocks — VNet, subnets, NSGs, route tables — and how they map to AWS.

IF ASKED, SAY THIS
A VNet is the direct equivalent of a VPC — your isolated address space. Subnets work the same conceptual way, though in Azure they're not AZ-pinned by default the way AWS subnets are. NSGs (Network Security Groups) map to AWS Security Groups for allow/deny rules, but NSGs can attach at either the su

A **VNet** is the direct equivalent of a VPC — your isolated address space. **Subnets** work the same conceptual way, though in Azure they're not AZ-pinned by default the way AWS subnets are. **NSGs** (Network Security Groups) map to AWS Security Groups for allow/deny rules, but NSGs can attach at either the subnet level or the NIC level — AWS Security Groups only attach at the ENI (instance) level, so Azure gives you an extra layer of network-level policy that AWS achieves instead through NACLs at the subnet.

Route tables (User Defined Routes in Azure) work the same way conceptually — explicit next-hop rules per subnet. The one genuine conceptual difference worth knowing cold: Azure NSGs process rules by priority number (lower number wins) across both inbound and outbound in one rule set, where AWS splits Security Group rules (stateful, allow-only) from NACLs (stateless, explicit allow+deny) as two separate layers.

- [ ] Explain Azure AD / Entra ID and RBAC — how does it differ conceptually from AWS IAM?

IF ASKED, SAY THIS
Entra ID (formerly Azure AD) is fundamentally an identity provider first — it's the same identity plane behind Microsoft 365, so users/groups are organization-wide, not per-cloud-account the way IAM users are scoped to one AWS account. Azure RBAC is the authorization layer on top — role assignments

Entra ID (formerly Azure AD) is fundamentally an identity provider first — it's the same identity plane behind Microsoft 365, so users/groups are organization-wide, not per-cloud-account the way IAM users are scoped to one AWS account. **Azure RBAC** is the authorization layer on top — role assignments (built-in roles like Contributor/Reader, or custom roles) scoped to a management group, subscription, resource group, or individual resource, in a hierarchy that inherits downward.

The conceptual difference from AWS IAM: AWS policies are typically attached directly to a role/user as JSON; Azure RBAC role *assignments* are a separate object binding a principal (user/group/service principal/Managed Identity) to a role definition at a specific scope — and Managed Identity is Azure's equivalent of an IAM role for a resource (a VM or Function gets an identity without any credential to manage at all), directly comparable to instance-profile roles in AWS.

- [ ] What's the difference between AKS and EKS operationally — upgrades, node pools, networking?

> **IF ASKED, SAY THIS**
> Upgrades — both require an explicit, deliberate control-plane version bump (no silent auto-upgrade by default in either), then a node pool/node group upgrade to match. AKS's node-image-upgrade and node-surge behavior during upgrade is broadly similar in spirit to EKS managed node group updates.

Upgrades — both require an explicit, deliberate control-plane version bump (no silent auto-upgrade by default in either), then a node pool/node group upgrade to match. AKS's node-image-upgrade and node-surge behavior during upgrade is broadly similar in spirit to EKS managed node group updates.

Node pools vs. node groups — conceptually equivalent; AKS supports mixing Windows and Linux node pools in the same cluster more natively, which comes up more often in enterprise/.NET-heavy shops.

Networking — AKS defaults to kubenet (simpler, NAT'd, no native VNet IP per pod) or Azure CNI (every pod gets a routable VNet IP, closer to how AWS VPC CNI behaves by default on EKS). If IP address planning at scale matters, Azure CNI Overlay or AWS's own IP-prefix-delegation mode on EKS are the two clouds' respective answers to “we're running out of IPs for pods” — worth knowing both exist since it's a real, recurring operational pain point on both platforms.

- [ ] How does Azure Monitor / Log Analytics compare to CloudWatch, in practical, hands-on terms?

> **IF ASKED, SAY THIS**
> CloudWatch splits metrics, logs, and traces into distinct sub-products (CloudWatch Metrics, CloudWatch Logs, X-Ray) that you correlate yourself. Azure Monitor is more unified — Log Analytics workspaces hold both logs and many metrics together, queryable with KQL (Kusto Query Language), which in prac

CloudWatch splits metrics, logs, and traces into distinct sub-products (CloudWatch Metrics, CloudWatch Logs, X-Ray) that you correlate yourself. Azure Monitor is more unified — Log Analytics workspaces hold both logs and many metrics together, queryable with KQL (Kusto Query Language), which in practice I've found more expressive for ad hoc cross-cutting queries than CloudWatch Logs Insights' query syntax, though that's partly just familiarity.

Practically, the workflow difference that matters most day to day: CloudWatch Alarms are metric-threshold-based and fairly simple to reason about; Azure Monitor alerts can fire off a full KQL query result, which is more powerful but means the alert logic itself needs the same review rigor as a real query — easier to write something subtly wrong that either over- or under-fires.

- [ ] Walk through setting up a hybrid connection between on-prem and Azure — ExpressRoute vs. VPN Gateway, when do you use which?

> **IF ASKED, SAY THIS**
> VPN Gateway (site-to-site IPsec) — encrypted tunnel over the public internet, quick to stand up, lower cost, but subject to internet latency/jitter and a hard practical throughput ceiling. Fine for moderate traffic, dev/test connectivity, or as a backup path.

VPN Gateway (site-to-site IPsec) — encrypted tunnel over the public internet, quick to stand up, lower cost, but subject to internet latency/jitter and a hard practical throughput ceiling. Fine for moderate traffic, dev/test connectivity, or as a backup path.

ExpressRoute — a private, dedicated circuit through a connectivity provider, bypassing the public internet entirely — far more predictable latency and much higher throughput, but a real procurement lead time (physical circuit provisioning) and meaningfully higher cost. I'd reach for ExpressRoute for latency-sensitive production traffic or genuinely high-volume transfer, and VPN Gateway for everything else, including as an automatic failover path even when ExpressRoute is primary — the AWS equivalent split is Direct Connect vs. Site-to-Site VPN, same tradeoff shape.

- [ ] You need to choose between AWS and Azure for a new workload with no existing preference — what factors actually drive that decision in practice?

> **IF ASKED, SAY THIS**
> In practice, “which cloud is technically better” loses to “where does this organization already have real operational muscle” far more often than a green-field comparison would suggest.

- **Where the org already has depth** — existing IAM/Entra ID setup, existing landing zone, existing team skillset. Rebuilding governance from scratch on a second cloud is a real, recurring cost that's easy to underestimate.
- **Ecosystem fit** — heavy Microsoft shop (AD, .NET, M365) leans Azure for the identity/integration story; AI/ML-heavy or startup-ecosystem-heavy workloads have historically leaned AWS for breadth of managed services, though both have converged a lot.
- **Compliance/data residency** — sometimes a specific regulatory requirement or client contract already names a cloud.
- **Specific managed service fit** — occasionally one cloud just has the exact service you need and the other's equivalent is meaningfully less mature.

In practice, “which cloud is technically better” loses to “where does this organization already have real operational muscle” far more often than a green-field comparison would suggest.

- [ ] How do you keep IAM/RBAC policy intent consistent when managing access across both AWS and Azure for the same team?

IF ASKED, SAY THIS
I don't try to force one literal policy syntax across both — IAM JSON and Azure RBAC role definitions are structurally different enough that a direct 1:1 mapping fights both systems. Instead I keep one source-of-truth document (or a shared Terraform module per cloud, called from the same access-requ

I don't try to force one literal policy syntax across both — IAM JSON and Azure RBAC role definitions are structurally different enough that a direct 1:1 mapping fights both systems. Instead I keep one source-of-truth document (or a shared Terraform module per cloud, called from the same access-request process) defining *roles by intent* — “read-only observability access,” “deploy-only pipeline access,” “break-glass admin” — and implement that same intent natively in each cloud's own idiom.

Both get managed through the same Terraform-driven process and the same access-review cadence, so “who can do what” is auditable consistently even though the underlying policy objects look different — consistency of process and intent, not a forced identical syntax that would compromise both.

- [ ] What's your approach to cost governance and tagging when an org runs workloads on both AWS and Azure?

IF ASKED, SAY THIS
A shared minimum tag/label schema enforced on both clouds — owner, environment, cost-center, project — with the same key names on both sides (AWS tags and Azure tags/labels are functionally equivalent, just named slightly differently in the UI) so a combined cost report can actually group by the sam

A shared minimum tag/label schema enforced on both clouds — owner, environment, cost-center, project — with the same key names on both sides (AWS tags and Azure tags/labels are functionally equivalent, just named slightly differently in the UI) so a combined cost report can actually group by the same dimensions across providers.

Enforcement, not just convention: policy-as-code (AWS Config rules / Azure Policy) that flags or blocks untagged resource creation, checked in the same Terraform pipeline gate as everything else. For the actual reporting, either each cloud's native cost tool per-account plus a manual roll-up, or a cross-cloud cost tool (CloudHealth, Windsor.ai-style aggregation) if the spend genuinely justifies the extra tooling — I wouldn't add that overhead until dual-cloud spend is large enough that manual roll-up is genuinely painful.

- [ ] How would you design monitoring so one on-call engineer gets a unified view across AWS and Azure resources, instead of switching consoles mid-incident?

IF ASKED, SAY THIS
Standardize on one place metrics/logs actually land, regardless of source cloud — typically Prometheus (via CloudWatch Exporter and Azure Monitor's Prometheus-remote-write integration) feeding one Grafana instance, or both clouds' logs shipped into one central store (Splunk/ELK/Loki) via their respe

Standardize on one place metrics/logs actually land, regardless of source cloud — typically Prometheus (via CloudWatch Exporter and Azure Monitor's Prometheus-remote-write integration) feeding one Grafana instance, or both clouds' logs shipped into one central store (Splunk/ELK/Loki) via their respective native forwarders. The on-call engineer's dashboard and alert routing point at that one place, not at CloudWatch console plus Azure Monitor console separately.

Alerting follows the same principle — one alerting pipeline (Alertmanager or PagerDuty as the single aggregation point) receiving from both clouds' native alert sources, so a page looks and routes the same way regardless of which cloud the underlying resource lives in. The goal is that during an incident, “which cloud is this in” is a fact you look up, not a context-switch you have to make to even start investigating.

- [ ] What are the practical gotchas when migrating a workload from AWS to Azure or vice versa — where do ‘equivalent’ services actually behave differently?

IF ASKED, SAY THIS
IAM model shape — porting an IAM policy's exact logic into RBAC (or back) is rarely 1:1; Azure's scope hierarchy and AWS's resource-ARN-based conditions don't map cleanly, so access design usually needs re-thinking, not just re-typing. Managed DB behavior — RDS and Azure Database for the same engine

- **IAM model shape** — porting an IAM policy's exact logic into RBAC (or back) is rarely 1:1; Azure's scope hierarchy and AWS's resource-ARN-based conditions don't map cleanly, so access design usually needs re-thinking, not just re-typing.
- **Managed DB behavior** — RDS and Azure Database for the same engine (say Postgres) can differ in exact parameter group defaults, extension availability, and failover timing — test failover behavior explicitly, don't assume parity.
- **Load balancer/ingress semantics** — ALB vs. Azure Application Gateway/Front Door have different path-routing and header-manipulation capabilities; ingress annotations rarely translate directly.
- **Networking defaults** — subnet-to-AZ pinning behaves differently, and default outbound-internet behavior (NAT Gateway vs. Azure's default outbound access, which has its own quirks and is being deprecated by Azure in some SKUs) is an easy silent gotcha.
- **Cost model shape** — data egress and load balancer pricing structures differ enough that a cost-neutral migration on paper can land differently in the actual bill.

- [ ] How do your Azure certifications translate into hands-on operational value for a role like this, alongside your AWS experience?

IF ASKED, SAY THIS
AZ-104 (Azure Administrator) is squarely the day-to-day operational layer this role needs — VNets, storage, VM/AKS management, monitoring, backup — the same operational muscle as hands-on AWS work, just in Azure's idiom. SC-300 (Identity and Access Administrator) and SC-100 (Cybersecurity Architect)

AZ-104 (Azure Administrator) is squarely the day-to-day operational layer this role needs — VNets, storage, VM/AKS management, monitoring, backup — the same operational muscle as hands-on AWS work, just in Azure's idiom. SC-300 (Identity and Access Administrator) and SC-100

(Cybersecurity Architect) go further than most SRE candidates go on the identity/security side — Entra ID, Conditional Access, RBAC design, security architecture patterns — which matters directly for a role that explicitly wants practical understanding across both clouds' identity and security models, not just compute and networking.

Concretely, that combination means I'm not translating AWS instincts into Azure on the fly during an incident — the certifications reflect real, tested depth on the Azure side that sits alongside genuine hands-on AWS production experience, which is exactly the dual-cloud shape this JD is asking for rather than a gap to explain around.

EKS

- [ ] What does AWS actually manage in "managed control plane," and what's still on you?

> **IF ASKED, SAY THIS**
> AWS runs and patches the API server, etcd, and the core controllers across multiple AZs, scales the control plane itself, and handles its availability SLA — you never SSH into a control plane node or see etcd directly. What's still yours: the data plane (worker nodes — patching, sizing, right-sizing instance types), cl

AWS runs and patches the API server, etcd, and the core controllers across multiple AZs, scales the control plane itself, and handles its availability SLA — you never SSH into a control plane node or see etcd directly. What's still yours: the data plane (worker nodes — patching, sizing, right-sizing instance types), cluster version upgrades (you choose when to move to a new K8s minor version, AWS doesn't force it until the version hits end-of-standard-support), add-on versions (VPC CNI, CoreDNS, kube-proxy), and everything above the infrastructure line — RBAC, network policy, workload scheduling, ingress.

- [ ] Explain IRSA — how does a pod get scoped AWS permissions instead of inheriting the node's role?

> **IF ASKED, SAY THIS**
> Before IRSA, every pod on a node inherited whatever IAM instance profile the node itself had — meaning any workload on a node could call anything the node's role allowed, a massive over-privilege problem on shared nodes. IAM Roles for Service Accounts fixes this by giving the cluster an OIDC identity provider, then let

Before IRSA, every pod on a node inherited whatever IAM instance profile the node itself had — meaning any workload on a node could call anything the node's role allowed, a massive over-privilege problem on shared nodes. IAM Roles for Service Accounts fixes this by giving the *cluster* an OIDC identity provider, then letting you annotate a Kubernetes ServiceAccount with an IAM role ARN that trusts that OIDC provider, scoped to that specific service account via a trust policy condition.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: orders-api
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/orders-api-role
```

The EKS Pod Identity webhook injects short-lived, auto-rotating credentials into any pod using that service account — no static keys, no node-wide blast radius. Note: AWS also now offers **EKS Pod Identity** as a newer, simpler alternative to IRSA (no OIDC provider setup, a direct trust to the EKS Pod Identity service) — worth mentioning both since a lot of production clusters are still on IRSA and migrating.

- [ ] Managed node groups vs. self-managed nodes vs. Fargate profiles on EKS — when does each make sense?

> **IF ASKED, SAY THIS**
> Managed node groups are the default: AWS handles the ASG, drains/upgrades nodes in order during a version bump, and integrates with the console — you still choose instance types, AMI, and scaling policy. Self-managed nodes give you full control over the AMI (custom bootstrap scripts, a hardened golden image, non-standa

Managed node groups are the default: AWS handles the ASG, drains/upgrades nodes in order during a version bump, and integrates with the console — you still choose instance types, AMI, and scaling policy. **Self-managed** nodes give you full control over the AMI (custom bootstrap scripts, a hardened golden image, non-standard kernel modules) at the cost of owning the lifecycle yourself — mostly for specialized workloads or strict compliance imaging requirements. **Fargate profiles** run individual pods with no node to manage at all, matched by namespace/label selectors — good for bursty, small, or untrusted workloads where per-pod isolation matters, but you lose DaemonSets, host networking, and some node-level customization on anything scheduled to Fargate.

- [ ] A large EKS cluster keeps hitting "not enough IP addresses" for new pods, even though nodes aren't CPU/memory constrained. What's happening?

> **IF ASKED, SAY THIS**
> Classic VPC CNI IP exhaustion. By default, the Amazon VPC CNI assigns each pod a real routable IP from the VPC subnet — so the number of pods a node can run is capped by (ENIs per instance type × IPs per ENI), not by CPU/memory. On smaller instance types packed with many small pods, you hit the IP ceiling long before c

Classic VPC CNI IP exhaustion. By default, the Amazon VPC CNI assigns each pod a real routable IP from the VPC subnet — so the number of pods a node can run is capped by (ENIs per instance type × IPs per ENI), not by CPU/memory. On smaller instance types packed with many small pods, you hit the IP ceiling long before compute limits.

⚠ Fixes interviewers want to hear, roughly in order of how much they cost to adopt: (1) enable **prefix delegation** so each ENI slot gets a /28 prefix instead of one IP — dramatically raises pod density per node with minimal change; (2) size subnets correctly up front (a /24 fills up fast at real cluster scale); (3) for extreme density, consider an alternate CNI like Cilium in overlay/eBPF mode that doesn't consume one VPC IP per pod at all.

- [ ] How does an Ingress object on EKS actually become a real ALB in AWS?

> **IF ASKED, SAY THIS**

The AWS Load Balancer Controller (successor to the older "ALB Ingress Controller" name) runs as a deployment in-cluster, watches for Ingress objects annotated with kubernetes.io/ingress.class: alb , and calls the AWS API to provision and configure an actual Application Load Balancer, target groups, and listener rules t

The **AWS Load Balancer Controller** (successor to the older "ALB Ingress Controller" name) runs as a deployment in-cluster, watches for Ingress objects annotated with `kubernetes.io/ingress.class: alb`, and calls the AWS API to provision and configure an actual Application Load Balancer, target groups, and listener rules to match — reconciling continuously, the same controller pattern as everything else in Kubernetes. The same controller also provisions Network Load Balancers for Service objects of type `LoadBalancer` with the right annotations. Target type matters: `instance` mode routes through node ports (extra hop), `ip` mode registers pod IPs directly in the target group — the latter is standard on EKS with the VPC CNI since pods already have routable VPC IPs.

- [ ] Walk through upgrading an EKS cluster a minor version — what order, and what can go wrong?

IF ASKED, SAY THIS

Control plane first (AWS upgrades it in place, brief API server disruption but no workload impact), then upgrade managed add-ons (VPC CNI, CoreDNS, kube-proxy) to versions compatible with the new control plane, then roll node groups — for managed node groups this can be a guided in-place update or, more safely, standin

Control plane first (AWS upgrades it in place, brief API server disruption but no workload impact), then upgrade managed add-ons (VPC CNI, CoreDNS, kube-proxy) to versions compatible with the new control plane, then roll node groups — for managed node groups this can be a guided in-place update or, more safely, standing up a new node group on the target version and cordoning/draining the old one so you can roll back by just re-pointing workloads if something's wrong.

What breaks in practice: Kubernetes only supports a skew of a few minor versions between control plane and kubelet, so nodes left too far behind become unsupported; deprecated/removed API versions (a classic one historically was `extensions/v1beta1` Ingress) silently fail if manifests or Helm charts haven't been updated; and any admission webhook or CRD-based operator not yet compatible with the new API server version can start rejecting or mishandling objects cluster-wide.

- [ ] EBS CSI driver vs. EFS CSI driver — how do you decide which a workload needs?

IF ASKED, SAY THIS

EBS is block storage, AZ-bound, ReadWriteOnce — one pod (or a set of pods on the same node) at a time, the default for a database or anything needing low-latency block I/O. If that pod gets rescheduled to a different AZ, the volume has to move with it or the pod stays Pending, which is why EBS-backed stateful workloads

EBS is block storage, AZ-bound, `ReadWriteOnce` — one pod (or a set of pods on the same node) at a time, the default for a database or anything needing low-latency block I/O. If that pod gets rescheduled to a different AZ, the volume has to move with it or the pod stays Pending, which is why EBS-backed stateful workloads typically pin via topology-aware provisioning or run in a single AZ deliberately. EFS is NFS, works across AZs, and supports `ReadWriteMany` — the right choice when multiple pods (potentially in different AZs) need to read/write the same files concurrently, like shared config, ML model artifacts, or a legacy app that assumes a shared filesystem. EFS throughput and per-operation latency are both worse than EBS, so it's not a default — use it because you specifically need shared access, not as a general-purpose volume.

- [ ] Cluster Autoscaler vs. Karpenter — what's the actual architectural difference?

IF ASKED, SAY THIS

Cluster Autoscaler works against fixed Auto Scaling Groups — it watches for unschedulable pods, picks which ASG to scale based on which node group's instance type would fit, and bumps that ASG's desired count. You're still choosing node group shapes ahead of time.

Cluster Autoscaler works against fixed Auto Scaling Groups — it watches for unschedulable pods, picks which ASG to scale based on which node group's instance type would fit, and bumps that ASG's desired count. You're still choosing node group shapes ahead of time.

Karpenter skips ASGs entirely — it watches unschedulable pods and provisions the right EC2 instance type and size directly via the EC2 API, matched to what the pending pods actually need (bin-packing more precisely, mixing Spot and on-demand per NodePool policy), and can deprovision/consolidate nodes far faster since it isn't bound to ASG scaling semantics. Generally faster scale-up, better bin-packing, and less node-group sprawl to maintain — most new EKS builds default to Karpenter today, but Cluster Autoscaler is still extremely common in existing estates, so know both.

- [ ] What's an "EKS managed add-on," and why does it matter for upgrade planning?

IF ASKED, SAY THIS

Managed add-ons (VPC CNI, CoreDNS, kube-proxy, EBS/EFS CSI drivers, Pod Identity Agent) are operated by EKS itself — AWS publishes tested version compatibility matrices per Kubernetes version, and you can update them through the EKS API/Terraform provider rather than manually applying manifests. The practical payoff: b

Managed add-ons (VPC CNI, CoreDNS, kube-proxy, EBS/EFS CSI drivers, Pod Identity Agent) are operated by EKS itself — AWS publishes tested version compatibility matrices per Kubernetes version, and you can update them through the EKS API/Terraform provider rather than manually applying manifests. The practical payoff: before a cluster version upgrade, you check the add-on compatibility matrix first, since an incompatible CNI or CoreDNS version is one of the most common causes of "upgrade succeeded, cluster is broken" — pods failing to get IPs or DNS resolution silently failing cluster-wide.

- [ ] Nodes intermittently flip to NotReady for 30–60 seconds under load, then recover on their own. Where do you look?

IF ASKED, SAY THIS

Start with kubectl describe node for the condition transitions and reason — usually kubelet failing its own liveness ping to the API server. Common root causes at this specific symptom (transient, self-recovering): CPU pressure on the node starving the kubelet process itself of scheduling time (check if a pod without C

Start with `kubectl describe node` for the condition transitions and reason — usually `kubelet` failing its own liveness ping to the API server. Common root causes at this specific symptom (transient, self-recovering): CPU pressure on the node starving the kubelet process itself of scheduling time (check if a pod without CPU limits is starving neighbors — the classic "noisy neighbor"), or the VPC CNI's `ipamd` struggling to keep up with high pod churn on that node. Also check `kube-proxy` and CNI pod logs on that specific node — if they're crash-looping or OOMing under load, the node reports unhealthy even though the underlying EC2 instance is fine. Instance-level: a burstable instance type (T-family) exhausting CPU credits under sustained load presents almost identically — check `CPUCreditBalance` in CloudWatch before assuming it's a Kubernetes-layer problem at all.

- [ ] Least privilege for EKS workload IAM specifically — IRSA vs Pod Identity.

IF ASKED, SAY THIS

Both scope IAM permissions to a specific ServiceAccount instead of the whole node, which is the core least-privilege win over node-level roles. IRSA does this via an OIDC trust relationship per cluster that has to be explicitly configured and kept in sync; EKS Pod Identity (the newer mechanism) simp

Both scope IAM permissions to a specific ServiceAccount instead of the whole node, which is the core least-privilege win over node-level roles. IRSA does this via an OIDC trust relationship per cluster that has to be explicitly configured and kept in sync; EKS Pod Identity (the newer mechanism) simplifies this with a cluster-level association managed through an EKS API instead of hand-wired OIDC trust policies, and doesn't require touching the IAM role's trust policy every time you add a new cluster. Either way, the actual least-privilege discipline is the same: one role per workload scoped to exactly the actions/resources it needs, never a shared "apps-can-do-everything" role reused across unrelated services.

- [ ] EKS Fargate profiles specifically — how do they decide which pods run on Fargate vs. EC2 nodes in a mixed cluster?

IF ASKED, SAY THIS

A Fargate profile matches pods by namespace (and optionally label selectors) — any pod scheduled into a matching namespace runs on Fargate automatically, no node group needed for it; everything else schedules onto your regular managed/self-managed node groups. Common pattern: put low-traffic or spiky batch/cron workloads on Fargate profiles to avoid paying for idle node capacity, keep steady-state high-throughput services on regular node groups where per-pod Fargate pricing is less cost-effective.

The real limitation to know cold: Fargate pods can't use DaemonSets, hostNetwork, or hostPort, and have more constrained storage options (no hostPath) — so anything relying on a node-level agent (many logging/monitoring DaemonSets) needs an alternative sidecar-based approach on Fargate.

- [ ] How would you actually design VPC/subnet CIDR sizing for an EKS cluster to avoid the IP-exhaustion problem before it happens?

IF ASKED, SAY THIS

The VPC CNI assigns a real VPC IP to every pod by default, so subnet sizing must account for (nodes x max-pods-per-node), not just node count — undersized subnets are the single most common EKS capacity surprise. Fix proactively with larger CIDR blocks, secondary CIDR ranges added to the VPC, or switching the CNI to prefix delegation mode (assigns /28 IP prefixes to ENIs instead of one IP each, dramatically raising pod density per subnet).

Prefix delegation is the detail that shows real EKS depth — it's the standard modern fix for IP exhaustion without needing to re-architect the VPC, and it's a one-line CNI configuration change (`ENABLE_PREFIX_DELEGATION=true`) rather than a network redesign.

- [ ] How do you get cluster and pod-level logs off an EKS cluster for real observability, beyond just kubectl logs?

IF ASKED, SAY THIS

Control-plane logging (API server, audit, scheduler, controller-manager) ships to CloudWatch Logs once enabled per log type on the cluster — it's off by default and costs extra, so enabling only the types you actually need (typically api and audit) matters for cost. Pod/application logs need a separate path: a DaemonSet log shipper (Fluent Bit is the standard choice, often via the CloudWatch Container Insights or an ADOT/OpenTelemetry setup) forwarding to CloudWatch, OpenSearch, or wherever your log stack lives.

A common interview trap: assuming EKS logs "just work" like a managed service dashboard — in reality both control-plane and data-plane (pod) logging are opt-in and require explicit setup, which is exactly the kind of gap that bites teams during their first real incident on a new cluster.

- [ ] What EKS-specific network security controls exist beyond standard Kubernetes NetworkPolicy?

IF ASKED, SAY THIS

Security groups for pods (assign an actual AWS security group directly to specific pods via the VPC CNI, enforced at the ENI level — stronger and AWS-native compared to NetworkPolicy, which depends on the CNI plugin implementing enforcement). Also: private API server endpoint access (disable the public endpoint or restrict it by CIDR) so the control plane itself isn't internet-reachable, which is a standard baseline hardening step in regulated environments.

Worth naming explicitly: standard Calico/Cilium NetworkPolicy still works and is usually the first line of defense, but security groups for pods add AWS-native, ENI-level enforcement as a second layer — the two aren't mutually exclusive and pair well in a defense-in-depth answer.

- [ ] Bottlerocket vs. Amazon Linux 2/2023 as the EKS node AMI — why would you choose the purpose-built option?

IF ASKED, SAY THIS
Bottlerocket is a minimal, container-optimized OS with no shell/package manager by default and an immutable root filesystem — smaller attack surface, faster boot, and updates are applied as atomic image swaps (with automatic rollback on failed boot) rather than in-place package updates that can leave a node in a half-patched state.

The tradeoff to mention: Bottlerocket's lack of a general-purpose shell means some debugging workflows (SSH in and poke around) don't work the same way — you debug via a separate "admin container" instead, which is a real operational adjustment for teams used to Amazon Linux.

- [ ] Lay out VPC networking fundamentals: what actually provides isolation and routing between subnets?

IF ASKED, SAY THIS
A VPC is a logically isolated network with a CIDR block; subnets carve that into smaller ranges, each pinned to one Availability Zone. A subnet is "public" purely by convention — meaning its route table has a route to an Internet Gateway (IGW) for 0.0.0.0/0 — versus "private," routing outbound traffic through a NAT Gateway instead (so ins

A VPC is a logically isolated network with a CIDR block; subnets carve that into smaller ranges, each pinned to one Availability Zone. A subnet is "public" purely by convention — meaning its route table has a route to an Internet Gateway (IGW) for `0.0.0.0/0` — versus "private," routing outbound traffic through a NAT Gateway instead (so instances can reach the internet outbound but aren't directly reachable inbound). Security is enforced at two layers with different semantics: **Security Groups** are stateful (return traffic is automatically allowed, you don't need an explicit outbound rule for a response to an allowed inbound request) and attached to individual ENIs/instances; **Network ACLs** are stateless (you must explicitly allow both directions) and apply at the subnet level as a coarser secondary layer.

- [ ] What's the actual difference between a NAT Gateway and a NAT Instance, and why does almost everyone use NAT Gateway now?

IF ASKED, SAY THIS
Both let private-subnet resources reach the internet outbound without being directly reachable inbound. A NAT Instance is a regular EC2 instance running NAT software you manage and patch yourself (single point of failure unless you build your own HA setup, and it's throughput-bounded by instance size). NAT Gateway is a fully-managed AWS s

Both let private-subnet resources reach the internet outbound without being directly reachable inbound. A NAT Instance is a regular EC2 instance running NAT software you manage and patch yourself (single point of failure unless you build your own HA setup, and it's throughput-bounded by instance size). NAT Gateway is a fully-managed AWS service — highly available within its AZ automatically, scales throughput without your intervention, no patching. The tradeoff is purely cost (NAT Gateway has an hourly charge plus per-GB data processing charge that can add up meaningfully at high outbound data volume) versus the operational burden of self-managing NAT instances — virtually all production setups now default to NAT Gateway, one per AZ for resilience, accepting the cost for the operational simplicity.

- [ ] Why does data transfer between Availability Zones cost money, and how does that actually shape architecture decisions?

IF ASKED, SAY THIS
Cross-AZ traffic (unlike same-AZ traffic) incurs a per-GB charge in both directions since AZs are physically separate data centers connected by AWS's own network backbone — this is a genuinely non-trivial cost driver at scale for chatty microservices architectures. It shapes real decisions: topology-aware routing (Kubernetes' topology.kub

Cross-AZ traffic (unlike same-AZ traffic) incurs a per-GB charge in both directions since AZs are physically separate data centers connected by AWS's own network backbone — this is a genuinely non-trivial cost driver at scale for chatty microservices architectures. It shapes real decisions: topology-aware routing (Kubernetes' `topology.kubernetes.io/zone` aware traffic routing/service mesh locality-aware load balancing to prefer same-AZ backends when possible), and it's a real argument for keeping tightly-coupled, high-volume-communicating services co-located rather than spread indiscriminately across AZs purely for HA — availability and cost pull in different directions here, and a senior answer acknowledges that tension explicitly rather than pretending "just spread across AZs" is free.

- [ ] What's VPC Peering vs Transit Gateway, and when does peering stop scaling?

IF ASKED, SAY THIS
VPC Peering is a direct, point-to-point, non-transitive connection between exactly two VPCs — non-transitive means if A peers with B and B peers with C, A cannot reach C through B; you'd need a direct A-C peering too. This creates an unmanageable full-mesh (N×(N-1)/2 connections) as VPC count grows past a handful. Transit Gateway is a cen

VPC Peering is a direct, point-to-point, non-transitive connection between exactly two VPCs — non-transitive means if A peers with B and B peers with C, A cannot reach C through B; you'd need a direct A-C peering too. This creates an unmanageable full-mesh (N×(N-1)/2 connections) as VPC count grows past a handful. Transit Gateway is a central hub — every VPC attaches to it once, and TGW handles routing between all attached VPCs (and on-premises connections) through one managed resource, turning what would be a full mesh into a hub-and-spoke, which is the standard answer for "how do you connect 20+ VPCs" and is also the natural on-ramp for hybrid connectivity (section 9.6), since Direct Connect/VPN attach to the Transit Gateway too.

- [ ] What does AWS actually manage for you in EKS versus what you're still responsible for?

IF ASKED, SAY THIS
AWS manages the control plane entirely — API server, etcd, scheduler, controller-manager, including their HA and patching across multiple AZs — you never directly access or operate these components. You remain responsible for: worker nodes (whether self-managed EC2, Managed Node Groups, or Fargate, you choose instance types/sizing/scaling

AWS manages the **control plane** entirely — API server, etcd, scheduler, controller-manager, including their HA and patching across multiple AZs — you never directly access or operate these components. You remain responsible for: **worker nodes** (whether self-managed EC2, Managed Node Groups, or Fargate, you choose instance types/sizing/scaling and — for anything but Fargate — OS-level patching), **cluster networking configuration** (VPC/subnet layout, security groups, the CNI plugin), **add-ons** (CoreDNS, kube-proxy, VPC CNI — AWS offers these as "EKS Add-ons" for easier lifecycle management, but you still choose versions/config), **IAM/RBAC mapping**, and obviously everything you deploy on top.

- [ ] An EKS node group fails to join the cluster after creation — walk the diagnosis.

IF ASKED, SAY THIS
Common causes in rough order of frequency: (1) the node's IAM role is missing required managed policies (AmazonEKSWorkerNodePolicy , AmazonEC2ContainerRegistryReadOnly , the CNI policy) or the aws-auth ConfigMap (or, in newer EKS, EKS Access Entries) doesn't map that node role to a valid cluster identity, so it authenticates to AWS fine

Common causes in rough order of frequency: (1) the node's IAM role is missing required managed policies (`AmazonEKSWorkerNodePolicy` , `AmazonEC2ContainerRegistryReadOnly` , the CNI policy) or the `aws-auth` ConfigMap (or, in newer EKS, EKS Access Entries) doesn't map that node role to a valid cluster identity, so it authenticates to AWS fine but the Kubernetes API rejects it; (2) security group rules don't allow the required control-plane-to-node and node-to-node communication ports; (3) subnets lack a route to the EKS API endpoint (private cluster endpoint configs are a common culprit — nodes in a subnet with no route to the private endpoint simply can't register); (4) the node's bootstrap script failed (check `/var/log/cloud-init-output.log` via SSM Session Manager on the instance) — often a version mismatch between the node AMI and the cluster's Kubernetes version.

- [ ] Walk through the actual mechanics of how IRSA delivers credentials to a pod.

IF ASKED, SAY THIS
The EKS cluster has an associated OIDC identity provider registered in IAM. The target IAM role's trust policy allows that specific OIDC provider, scoped to a specific namespace+ServiceAccount combination via a condition on the token's sub claim. A mutating webhook (the Pod Identity Webhook) automatically injects a projected service accou

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-app-sa
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/my-app-role
```

The EKS cluster has an associated OIDC identity provider registered in IAM. The target IAM role's trust policy allows that specific OIDC provider, scoped to a specific namespace+ServiceAccount combination via a condition on the token's `sub` claim. A mutating webhook (the Pod Identity Webhook) automatically injects a projected service account token volume and `AWS_ROLE_ARN` / `AWS_WEB_IDENTITY_TOKEN_FILE` env vars into any pod using an annotated ServiceAccount. The AWS SDK inside the app, on startup, automatically calls `AssumeRoleWithWebIdentity` using that token to obtain short-lived, auto-rotating credentials — all transparent to application code as long as it's using a reasonably current AWS SDK version that supports the web identity credential provider chain.

- [ ] A pod using IRSA is getting AccessDenied even though the IAM policy looks correct. What do you check, in order?

IF ASKED, SAY THIS
Is the ServiceAccount actually annotated correctly, and is the pod actually using that ServiceAccount (a very common miss: the Deployment spec doesn't set serviceAccountName , so it silently uses default instead of the intended one). Does the IAM role's trust policy (not just the permissions policy) correctly scope the sub condition to th

- Is the ServiceAccount actually annotated correctly, and is the pod actually *using* that ServiceAccount (a very common miss: the Deployment spec doesn't set `serviceAccountName` , so it silently uses `default` instead of the intended one).
- Does the IAM role's **trust policy** (not just the permissions policy) correctly scope the `sub` condition to the exact `system:serviceaccount:<namespace>:<name>` — a namespace or ServiceAccount name typo here is extremely common and produces exactly this symptom.
- Was the pod already running *before* the ServiceAccount annotation was added — the webhook injects credentials at pod creation time, so an existing pod won't pick up a newly-added annotation without being recreated.
- Is the SDK version in the application actually new enough to support the web identity credential chain, and is it not being overridden by some other credential source earlier in its credential provider chain (e.g. a stale `AWS_ACCESS_KEY_ID` env var accidentally set elsewhere taking priority).

- [ ] EBS vs EFS vs S3 — when is each the right storage choice, and why?

IF ASKED, SAY THIS
EBS — block storage, attached to exactly one instance/node at a time (AZ-local, as covered in the Kubernetes storage section), the right choice for a database's own data directory or anything needing low-latency block-level access. EFS — managed NFS, genuinely shared read-write across many instances/AZs simultaneously, the right fit for R

EBS — block storage, attached to exactly one instance/node at a time (AZ-local, as covered in the Kubernetes storage section), the right choice for a database's own data directory or anything needing low-latency block-level access. **EFS** — managed NFS, genuinely shared read-write across many instances/AZs simultaneously, the right fit for RWX Kubernetes workloads or shared config/asset storage across a fleet, at higher per-GB cost and higher latency than EBS. **S3** — object storage, not a filesystem at all (no POSIX semantics, no in-place partial-file edits) — the right choice for large unstructured data, backups, static assets, data lake storage, or anything accessed via API/HTTP rather than needing a real mounted filesystem.

- [ ] Explain RDS Multi-AZ vs Read Replicas — they sound similar but solve different problems.

> **IF ASKED, SAY THIS**
> Multi-AZ is purely for availability — a synchronously-replicated standby in a different AZ that RDS automatically fails over to if the primary fails; the standby is not readable/queryable during normal operation, it exists solely as a hot standby. Read Replicas are for read scaling — asynchronously-replicated, independently readable/query

Multi-AZ is purely for **availability** — a synchronously-replicated standby in a different AZ that RDS automatically fails over to if the primary fails; the standby is not readable/queryable during normal operation, it exists solely as a hot standby. **Read Replicas** are for **read scaling** — asynchronously-replicated, independently readable/queryable copies you can direct read-heavy traffic to, offloading the primary; a read replica can be promoted to a standalone writable instance manually but isn't an automatic failover target the way a Multi-AZ standby is. A common production pattern combines both: Multi-AZ on the primary for HA, plus one or more read replicas (possibly cross-region) for read scaling and geographic read latency — conflating the two in an interview answer is a common tell that the concepts are understood only shallowly.

- [ ] What's the EBS CSI driver's role in EKS storage, and what IAM permission does it specifically need?

> **IF ASKED, SAY THIS**
> The EBS CSI driver is the controller that translates Kubernetes PVC/StorageClass requests into actual EBS volume create/attach/detach/delete API calls — since Kubernetes core itself has deliberately moved away from in-tree cloud-provider-specific volume plugins toward this pluggable CSI (Container Storage Interface) model. It runs as a co

The EBS CSI driver is the controller that translates Kubernetes PVC/StorageClass requests into actual EBS volume create/attach/detach/delete API calls — since Kubernetes core itself has deliberately moved away from in-tree cloud-provider-specific volume plugins toward this pluggable CSI (Container Storage Interface) model. It runs as a controller Deployment (handles provisioning) plus a node DaemonSet (handles the actual mount on each node), and needs an IAM role (commonly via IRSA, tying back to section 9.3) with EC2 EBS permissions (`CreateVolume` , `AttachVolume` , `DeleteVolume` , etc.) — a very common real-world troubleshooting scenario is PVCs stuck Pending simply because this driver's IRSA role is missing or misconfigured, which ties directly back to the Kubernetes storage-troubleshooting chapter.

- [ ] CloudWatch vs CloudTrail — what's the actual distinction people blur together?

> **IF ASKED, SAY THIS**
> CloudWatch is metrics, logs, and alarms about the behavior/performance of your resources — CPU utilization, request counts, application log streams, custom metrics. CloudTrail is an audit log of API calls/actions made against your AWS account — who (which IAM identity) did what (which API action) to which resource, when, from where — the

CloudWatch is metrics, logs, and alarms about the *behavior/performance* of your resources — CPU utilization, request counts, application log streams, custom metrics. **CloudTrail** is an audit log of *API calls/actions* made against your AWS account — who (which IAM identity) did what (which API action) to which resource, when, from where — the record you'd pull to answer "who deleted that S3 bucket" or "who changed this security group," which CloudWatch fundamentally doesn't capture at all since it's not tracking control-plane API activity, only resource behavior/metrics.

- [ ] How does the Datadog/Prometheus agent actually get metrics out of a managed AWS service you can't install an agent on (e.g. RDS)?

> **IF ASKED, SAY THIS**
> For genuinely managed services with no accessible host to run an agent on, metrics come from the CloudWatch API instead — Datadog's AWS integration (or the Prometheus cloudwatch_exporter) polls CloudWatch on your behalf and re-exposes those metrics in the observability tool alongside your other data, rather than true agent-based collecti

For genuinely managed services with no accessible host to run an agent on, metrics come from the CloudWatch API instead — Datadog's AWS integration (or the Prometheus `cloudwatch_exporter`) polls CloudWatch on your behalf and re-exposes those metrics in the observability tool alongside your other data, rather than true agent-based collection. This is a meaningful practical distinction to know: CloudWatch-sourced metrics typically have coarser granularity (often 1 or 5-minute periods by default vs. sub-minute for agent-collected metrics) and added latency (CloudWatch itself has some delay before a metric is queryable), which matters when you're trying to alert on something time-sensitive against a managed service you don't directly control the collection interval for.

- [ ] Direct Connect vs Site-to-Site VPN — walk through the real tradeoffs, not just "one is faster."

> **IF ASKED, SAY THIS**
> Direct Connect is a dedicated physical network connection from your on-prem/colo facility into AWS (via a DX location), giving consistent, predictable low latency and high bandwidth, bypassing the public internet entirely — but it takes weeks (physical cross-connect provisioning) to set up and costs meaningfully more, and by itself isn't

Direct Connect is a dedicated physical network connection from your on-prem/colo facility into AWS (via a DX location), giving consistent, predictable low latency and high bandwidth, bypassing the public internet entirely — but it takes weeks (physical cross-connect provisioning) to set up and costs meaningfully more, and by itself isn't encrypted (you'd layer IPsec or MACsec on top for genuinely sensitive traffic). **Site-to-Site VPN** runs encrypted tunnels over the public internet — quick to provision (often same-day, purely configuration), lower cost, but subject to public internet latency/jitter/congestion, meaningfully less predictable for latency-sensitive workloads. A common resilient pattern uses Direct Connect as primary with VPN as an automatic failover path if the DX circuit goes down, since VPN's quick provisioning/lower dependency on physical infrastructure makes it a reasonable backup path.

- [ ] How does Transit Gateway tie into a genuinely hybrid (multi-VPC + on-prem) network design?

> **IF ASKED, SAY THIS**

Transit Gateway acts as the single hub both your VPCs (via TGW VPC attachments) and your on-prem network (via a Direct Connect Gateway attachment or VPN attachment to the TGW) connect into — meaning on-prem gets one connection into the TGW and, via TGW's route tables, can reach every attached VPC without a separate DX/VPN connection per V

Transit Gateway acts as the single hub both your VPCs (via TGW VPC attachments) and your on-prem network (via a Direct Connect Gateway attachment or VPN attachment to the TGW) connect into — meaning on-prem gets one connection into the TGW and, via TGW's route tables, can reach every attached VPC without a separate DX/VPN connection per VPC, and VPCs can reach on-prem the same way. TGW route tables let you segment this deliberately (not every VPC needs to reach every other VPC or on-prem network by default) — a genuinely hybrid design at any real scale is built around TGW as the routing hub rather than a tangle of individual VPC-to-on-prem VPN connections.

- [ ] How does DNS resolution work across a hybrid environment — how does an on-prem system resolve an AWS-hosted private resource, and vice versa?

IF ASKED, SAY THIS

Route 53 Resolver's inbound and outbound endpoints are the standard mechanism: an outbound endpoint lets AWS-side resources forward DNS queries for on-prem domains to your on-prem DNS servers (via the Direct Connect/VPN path), and an inbound endpoint lets on-prem systems query AWS Route 53 private hosted zones by pointing their own DNS re

Route 53 Resolver's inbound and outbound endpoints are the standard mechanism: an **outbound endpoint** lets AWS-side resources forward DNS queries for on-prem domains to your on-prem DNS servers (via the Direct Connect/VPN path), and an **inbound endpoint** lets on-prem systems query AWS Route 53 private hosted zones by pointing their own DNS resolution (via conditional forwarding rules on their on-prem DNS servers) at that inbound endpoint's IPs. Without this, on-prem systems have no way to resolve `*.internal.aws`-style private DNS names, and AWS-side resources have no way to resolve internal on-prem hostnames — this is a very concrete, commonly-tested piece of "what actually makes hybrid infrastructure work" beyond just having network connectivity.

- [ ] A newly-provisioned VPC can reach on-prem via Direct Connect, but a specific application still can't connect. What do you check, layer by layer?

IF ASKED, SAY THIS

Route tables — does the subnet's route table actually have a route toward on-prem's CIDR pointing at the TGW/DX gateway attachment? Security Groups/NACLs — is the specific port/protocol actually allowed both directions (remembering NACLs are stateless, unlike Security Groups)? On-prem side firewall — AWS-side config being perfect doesn't

- Route tables — does the subnet's route table actually have a route toward on-prem's CIDR pointing at the TGW/DX gateway attachment?
- Security Groups/NACLs — is the specific port/protocol actually allowed both directions (remembering NACLs are stateless, unlike Security Groups)?
- On-prem side firewall — AWS-side config being perfect doesn't help if the on-prem firewall doesn't have a matching allow rule for the new VPC's CIDR; this is genuinely a two-sided coordination problem, not purely an AWS-side troubleshooting exercise.
- BGP route propagation on the DX connection itself — confirm the new VPC's CIDR is actually being advertised over BGP and appearing in on-prem's routing table, not just configured in AWS's route tables (a config that's correct on the AWS side but never propagated is a classic DX troubleshooting trap).
- DNS (section above) — if the "can't connect" symptom is actually a resolution failure rather than a routing/firewall block, none of the above will show anything wrong.

- [ ] What's the difference between designing for Multi-AZ resilience versus genuine Multi-Region DR — why isn't Multi-AZ "enough"?

IF ASKED, SAY THIS

Multi-AZ protects against a single data center-level failure (power, hardware, a single facility issue) — AZs within a region are physically separate but still share the same regional control plane and, rarely but not never, a regional-scope event (a regional service degradation, a broad configuration error affecting an entire region) can

Multi-AZ protects against a single data center-level failure (power, hardware, a single facility issue) — AZs within a region are physically separate but still share the same regional control plane and, rarely but not never, a regional-scope event (a regional service degradation, a broad configuration error affecting an entire region) can affect every AZ in that region simultaneously. Multi-Region DR protects against that class of event by having infrastructure capable of running in a genuinely separate region — at real cost: cross-region data replication latency/consistency tradeoffs, doubled (or more) infrastructure to maintain and keep in sync, and meaningfully more complex failover orchestration (DNS-based regional failover, data reconciliation). Whether Multi-Region is actually warranted is a real business/cost conversation tied back to the RTO/RPO the business genuinely needs — not every workload justifies it.

- [ ] SCENARIO Scenario: you're asked to design connectivity for a new EKS cluster that needs to reach an on-prem Oracle database with strict, predictable low latency. Walk through the design.

IF ASKED, SAY THIS

Direct Connect (not VPN) as the primary path given the strict low-latency requirement — public internet-routed VPN's variable latency/jitter is a real risk for a strict-latency dependency like a synchronous DB connection. Route the DX connection through Transit Gateway so it's reusable for future VPCs rather than a single-VPC-scoped DX Ga

Direct Connect (not VPN) as the primary path given the strict low-latency requirement — public internet-routed VPN's variable latency/jitter is a real risk for a strict-latency dependency like a synchronous DB connection. Route the DX connection through Transit Gateway so it's reusable for future VPCs rather than a single-VPC-scoped DX Gateway association. Configure Route 53 Resolver inbound/outbound endpoints so the cluster can resolve the on-prem Oracle hostname by its actual internal DNS name rather than a hardcoded IP (hardcoded IPs break silently on any on-prem re-IP). Set up IRSA-scoped, least-privilege access for whatever service actually talks to Oracle (network reachability alone isn't the whole access story).

For resilience, add a Site-to-Site VPN as an automatic BGP-failover backup path for the rare DX outage, accepting degraded (not zero) service during that window rather than a hard requirement for zero-latency-variance during a genuine DX failure.

- [ ] SCENARIO Scenario: leadership asks for a realistic DR plan for a customer-facing service currently in one region, with a business-stated tolerance of "a few minutes of downtime, near-zero data loss." What do you propose, and what do you push back on?

> **IF ASKED, SAY THIS**
> "A few minutes RTO, near-zero RPO" maps directly to Warm Standby, not the cheaper Pilot Light pattern (too slow to meet a few-minutes RTO) and not necessarily full Active/Active (likely more cost/complexity than the stated requirement actually needs) — propose a scaled-down-but-running copy in a second region, with continuous data replica

"A few minutes RTO, near-zero RPO" maps directly to Warm Standby, not the cheaper Pilot Light pattern (too slow to meet a few-minutes RTO) and not necessarily full Active/Active (likely more cost/complexity than the stated requirement actually needs) — propose a scaled-down-but-running copy in a second region, with continuous data replication (e.g. cross-region RDS read replica ready for promotion, or a database technology with built-in multi-region replication) to keep RPO near-zero, and automated (not manual/runbook-only) DNS-based failover to hit the minutes-level RTO reliably, since a manual failover process realistically can't guarantee a few-minutes RTO once you account for human detection-and-decision time. The pushback worth raising explicitly: this needs to actually be *tested* periodically (a real failover drill, not just an architecture diagram) — an untested DR plan is not meaningfully different from no DR plan, since the first real test of most DR setups' gaps is the actual incident, which is exactly the wrong time to discover them.

Cloud Migration

- [ ] What are the '6 Rs' of cloud migration, and how do you actually pick one for a given workload?

IF ASKED, SAY THIS
Rehost (lift-and-shift, fastest, least benefit), Replatform (lift-and-shift-plus, e.g. swap self-managed MySQL for RDS without rearchitecting), Refactor/Re-architect (rebuild for cloud-native, slowest, most benefit), Repurchase (move to SaaS instead), Retire (decommission, don't migrate), Retain (leave it where it is, at least for now). Pick per-workload, not once for the whole portfolio -- a legacy app nobody wants to touch gets rehosted or retained; a core revenue system slated for real investment gets refactored.

The interview signal here is whether you apply this per-workload or treat migration as one-size-fits-all -- a real migration portfolio is almost always a mix, and being able to justify WHY a specific system got a specific R (not just naming all six) is what separates a rehearsed answer from a real one.

- [ ] Lift-and-shift vs. re-architecting during a migration -- what's the actual tradeoff, and is there a good middle path?

IF ASKED, SAY THIS
Lift-and-shift is fast and low-risk but usually forfeits most cloud-native benefit and can even cost MORE than on-prem if nothing is right-sized. Re-architecting captures real benefit (elasticity, managed services, cost optimization) but is slow and risky to do simultaneously with a migration. The common middle path: migrate first with minimal changes to get out of the datacenter on schedule, then modernize incrementally afterward with the deadline pressure removed.

Naming that middle path explicitly ("migrate then modernize" as two separate phases) is a strong, senior-sounding answer -- combining migration and re-architecture into one project is a classic way migrations blow their timeline and budget.

- [ ] How do you structure a migration into waves, and what determines what goes in wave 1?

IF ASKED, SAY THIS
Wave 1 should be low-risk AND genuinely representative -- not the easiest possible system (proves nothing) and not the hardest (too risky to learn on). A good wave 1 candidate: moderate complexity, low business criticality, few dependencies on systems not yet migrated. Its purpose is proving the migration pattern and tooling actually work before betting anything critical on it.

Dependency mapping BEFORE wave planning is the detail worth naming -- migrating a system whose database hasn't moved yet just recreates the same latency/coupling problem in a new location. A real migration plan sequences waves by dependency graph, not just team availability.

- [ ] How do you plan the actual cutover for a migrating workload -- the moment traffic moves from old to new?

IF ASKED, SAY THIS
Prefer a strategy that keeps a fast rollback path available: run old and new in parallel first (shadow traffic or dual-write) to validate the new environment under real load before cutting over live traffic, then cut over via DNS/load-balancer weight shifting (gradual, reversible) rather than a hard switch, with an explicit, rehearsed rollback plan and a defined go/no-go decision point.

The parallel-run/shadow-traffic step is what separates a mature cutover plan from a hopeful one -- it catches environment-specific issues (latency differences, config drift, missing IAM permissions) BEFORE they're customer-facing.

- [ ] How do you migrate a production database with minimal downtime, rather than just taking a backup and restoring it?

IF ASKED, SAY THIS
Use a continuous-replication tool (AWS DMS, or the target database's native replication) to do an initial full load, then keep source and target in sync via ongoing change-data-capture while the old database keeps serving live traffic -- cutover becomes: stop writes briefly, let replication lag drain, repoint the application, resume writes. Downtime shrinks from a full backup/restore window to a short, bounded write-freeze.

Being able to name the actual tool (AWS DMS specifically, or database-native logical replication) rather than describing the concept abstractly is what makes this answer land as real experience.

- [ ] How do you validate that a data migration was actually correct, not just that it completed without erroring?

IF ASKED, SAY THIS
Row counts matching is the weakest check alone -- also run checksums/hashes on data samples, compare aggregate business metrics between old and new (total order value, active user count) as a sanity check, and run the application's own test suite against migrated data. During a dual-write period, actively compare read results between old and new on a sample of real requests before fully cutting over.

The aggregate-business-metrics sanity check is worth volunteering -- a migration can pass every row-count and schema check while still being subtly wrong (a timezone shift, a truncated field), and a business number moving unexpectedly is often the first real signal something's off.

- [ ] During a migration, you inevitably have a period where some systems are on-prem and some are in the cloud. What does that hybrid period actually require operationally?

IF ASKED, SAY THIS
Reliable, low-latency connectivity between the two (VPN for lower traffic/cost-sensitivity, Direct Connect/ExpressRoute for consistent bandwidth and lower latency at scale), consistent identity (federating on-prem AD with cloud IAM rather than two separate identity systems), and unified observability -- a single pane of glass across both environments, since an incident during this period can span both.

Unified observability across the hybrid boundary is the piece teams most often underinvest in -- it's tempting to treat cloud and on-prem monitoring as separate concerns right when cross-environment issues are most likely.

- [ ] How do you approach cost modeling for a migration -- both making the business case and avoiding surprises after the move?

IF ASKED, SAY THIS
Before: model realistic cloud spend based on RIGHT-SIZED resources, not 1:1 hardware-spec matching (which usually overprovisions and erases expected savings); include often-forgotten costs like data-transfer/egress during the migration itself. After: tag everything from day one for cost attribution, and expect the first month's bill to have surprises -- egress costs and unused-but-still-running old infrastructure are the two most common ones.

The 1:1-hardware-matching point is worth stating explicitly -- a common mistake is sizing the new cloud instance to match the old physical server's specs exactly, which usually massively overshoots actual utilization and is a big reason migrations sometimes cost MORE than promised.

- [ ] A migrated service is now running in the cloud, but latency for some users got noticeably worse. What do you check?

IF ASKED, SAY THIS
Check geography first -- if the new region is physically farther from a user population than the old datacenter, that's pure speed-of-light latency, no tuning fixes it. Then check whether a dependency didn't fully migrate -- the app is now in the cloud but still calling back to an on-prem database over the hybrid link, adding a round-trip that didn't exist before. Then check right-sizing.

The still-calling-an-on-prem-dependency case is the classic migration-specific latency bug -- it's easy to migrate an application layer while a data layer lags behind, and the resulting cross-hybrid-boundary calls are often the actual cause.

- [ ] Walk me through a cloud migration you've been part of -- how do you structure this as an interview answer?

IF ASKED, SAY THIS
Structure: (1) scope -- what was migrating, from where to where, roughly how big/critical; (2) your specific role/ownership, not "the team"; (3) the approach -- which R, how waves were sequenced, how cutover was planned; (4) a real challenge that came up and how it was handled; (5) the outcome, ideally with a number.

Bring your own: your actual migration experience, sized honestly -- even a single service migrated, or a specific piece you owned within a larger effort, told with real specifics beats a vague description of a huge program you were only peripherally involved in.

- [ ] When migrating a monolith toward microservices as part of a cloud move, what's a safer pattern than a full rewrite?

IF ASKED, SAY THIS
The Strangler Fig pattern -- put a routing layer in front of the monolith, and incrementally peel off individual capabilities into new services one at a time, with the router directing traffic to whichever (old monolith or new service) currently owns that capability. The monolith shrinks gradually while the system stays fully functional and deployable throughout, rather than a risky big-bang rewrite.

This pattern is worth naming by name (Strangler Fig) -- it's a well-recognized answer to decomposing a monolith safely, and citing it signals you know an established approach rather than improvising on the spot.

- [ ] What's the actual difference between disaster recovery and a cloud migration, and why does conflating them cause problems?

IF ASKED, SAY THIS
DR is about resilience -- a standby environment ready to take over if the primary fails, ideally rarely or never actually used for real traffic. Migration is a one-way, permanent move of the primary environment itself. Conflating them causes problems when a migration is planned like a DR failover (assuming it's quick and reversible) when it's actually a much bigger, one-directional undertaking.

This distinction is worth drawing explicitly if a migration question drifts toward DR territory -- it shows you're not conflating two genuinely different disciplines that happen to both involve moving workloads between environments.

Architecture Scenarios

[AWS] Lambda (Account A) → private API on EC2 (Account B)

IF ASKED, SAY THIS
First I'd confirm it's just these two accounts talking one-to-one — if so, I don't reach for Transit Gateway or PrivateLink, those solve a different shape of problem. Lambda isn't in a VPC by default, so step one is attaching it to private subnets with a security group. Then a straightforward VPC peering connection, route table entries on both sides, and a security group rule on the EC2 side allowing only the Lambda's security group on the API port. That's private, minimal, and doesn't touch the internet.

Scenario: A Lambda function in Account A needs to call a REST API hosted on an EC2 instance in Account B. Traffic must stay private, no internet. Design it.

Watch out for: Reaching for PrivateLink, VPC Peering, and Transit Gateway all at once, without first noticing that Lambda isn't in any VPC yet — that's the step that actually unlocks the rest.

Clarifying questions to ask first:

- Just these two accounts, or will more join later?
- Same region?
- What's the traffic pattern — occasional calls or sustained volume?

Design:

1. Attach the Lambda function to private subnets in a VPC (it isn't in your VPC by default — this is the step that gets missed).
2. Since it's two accounts, one-to-one: start with VPC Peering, not Transit Gateway.
3. Add route table entries on both sides pointing the peer CIDR at the peering connection.
4. Security group on the EC2 instance allows inbound from the Lambda's security group on the API port only.
5. Attach the AWSLambdaVPCAccessExecutionRole so the function can create the ENIs it needs — no special cross-account IAM is required for the network path itself, peering is routing, not identity.

How the design changes if scale/scope grows: If a third VPC needs the same access, stop using peering — it isn't transitive, so a mesh of pairwise connections becomes unmanageable fast. That's when Transit Gateway earns its cost. If Account B is really a third party exposing one service to many outside accounts, that's PrivateLink instead of peering.

Likely follow-up: What if that EC2 API needs to be reachable by 50 different customer accounts? — PrivateLink — peering 50 VPCs is unmanageable and exposes more of the network than the one service actually needs.

[AWS] 3 business units, 3 accounts, one shared database service

IF ASKED, SAY THIS
Since more business units are coming, I wouldn't build pairwise VPC peering — it doesn't scale and doesn't route transitively, so spoke A could never reach the hub through spoke B anyway. I'd put a Transit Gateway in the middle, attach the shared-services VPC as the hub and each business unit as a spoke, and give each spoke a route table that only points to the hub, so spokes stay isolated from each other unless there's a reason not to.

Scenario: Three business units, each with their own AWS account and VPC, all need private connectivity to a shared-services VPC hosting a central database. More business units will onboard over the next year. Design the connectivity.

Watch out for: Standing up individual VPC Peering connections per business unit because it works today — it stops scaling the moment a fourth or fifth spoke shows up, and peering doesn't route transitively between spokes anyway.

Clarifying questions to ask first:

- Will spokes ever need to talk to each other, or only to the hub?
- Does traffic between spokes need central inspection or firewalling?

Design:

1. Use a hub-and-spoke topology built on Transit Gateway.
2. Attach the shared-services VPC to the TGW as the hub.
3. Attach each business unit's VPC to the TGW as a spoke.
4. Give each spoke its own TGW route table containing only a route to the hub — that isolates spokes from each other by default.
5. If traffic needs inspection, route it through a dedicated inspection VPC with AWS Network Firewall attached to the TGW.

How the design changes if scale/scope grows: If this is confirmed to stay at exactly two or three VPCs forever with no growth, plain peering is still cheaper and simpler — Transit Gateway has an hourly and per-GB cost that isn't justified for a permanently small, static topology.

Likely follow-up: How do you stop BU1 and BU2 talking to each other, only to the hub? — Separate TGW route tables per spoke — each spoke's route table has a route to the hub only, not to other spokes' CIDRs.

[AWS] SaaS API exposed to hundreds of customer VPCs

IF ASKED, SAY THIS
At that scale I wouldn't touch peering at all — PrivateLink is built for exactly this shape. I'd front the service with a Network Load Balancer, publish it as a VPC Endpoint Service, and each customer creates an Interface Endpoint in their own VPC. Traffic stays on the AWS backbone, nobody's exposing more of their network than the one API, and there's no CIDR planning headache across hundreds of customers.

Scenario: You're building a SaaS product on AWS. Customers want to reach your API privately from their own VPCs, no public internet, and you expect hundreds of customers. How do you expose the service?

Watch out for: Proposing VPC peering per customer — operationally impossible at that count, and it exposes your entire VPC's CIDR to every customer instead of just the one service.

Clarifying questions to ask first:

- Does access need to be bidirectional, or only customer-to-you?
- Any CIDR overlap risk across customer VPCs?

Design:

1. Put a Network Load Balancer in front of your service.
2. Create a VPC Endpoint Service backed by that NLB.
3. Share the service name with customers.
4. Each customer creates an Interface VPC Endpoint in their own VPC pointing at your service — no route table changes on either side, no CIDR overlap concern.

How the design changes if scale/scope grows: PrivateLink is one-directional — the customer initiates to you. If customers ever needed bidirectional, peer-level network access, that's not what PrivateLink is for.

Likely follow-up: How is this different from a public ALB with IP allowlisting? — PrivateLink never touches the public internet and avoids maintaining IP allowlists per customer — access is controlled by endpoint connections, not by IP ranges that change.

- [ ] [AWS] On-prem ERP needs AWS access in two weeks

IF ASKED, SAY THIS
Given the two-week window, Direct Connect is off the table regardless of how the traffic grows later — the physical provisioning alone usually takes longer than that. A Site-to-Site VPN stands up in hours: customer gateway on their side, virtual private gateway on ours, two tunnels for redundancy out of the box. That comfortably covers a few Mbps.

Scenario: Finance needs their on-prem ERP to reach a new AWS-hosted reporting service within two weeks. Traffic is moderate, a few Mbps. Design the connectivity.

Watch out for: Defaulting to Direct Connect because it 'sounds more enterprise' — it takes weeks for the physical cross-connect and doesn't fit a two-week deadline.

Clarifying questions to ask first:

- Is this traffic level expected to grow significantly?
- Is there already a VPN device on-prem?

Design:

1. Stand up a Site-to-Site VPN: a customer gateway on-prem, a virtual private gateway (or Transit Gateway VPN attachment) in AWS.
2. Provisioning is hours, not weeks — it fits the deadline.
3. IPsec encryption over the internet is sufficient for moderate throughput.
4. Each VPN connection gets two tunnels by default for redundancy.

How the design changes if scale/scope grows: If throughput needs grow past what a VPN reliably sustains, or latency becomes strict, that's the trigger to add Direct Connect later — keeping the VPN in place as the backup path.

Likely follow-up: What if Finance says throughput will 10x in six months? — Start the Direct Connect provisioning process now in parallel — it's slow — while the VPN carries traffic today; cut over once it's ready.

- [ ] [AWS] Permanent, high-throughput, low-latency link for DB replication

IF ASKED, SAY THIS
Since reliability matters more than cost and this is permanent, I'd go straight to Direct Connect rather than starting with VPN and migrating later — a dedicated connection avoids the public internet entirely and gives consistent low latency and throughput. I'd still keep a Site-to-Site VPN as an automatic failover path, and use a Direct Connect Gateway if this needs to reach more than one VPC or region.

Scenario: A customer needs a permanent, high-throughput, low-latency connection between their data center and AWS for continuous database replication. Cost is secondary to reliability.

Watch out for: Proposing VPN alone for a permanent, latency-sensitive, high-throughput workload — it's capped in practice and rides the public internet, which is exactly what this requirement is trying to avoid.

Clarifying questions to ask first:

- Is there a Direct Connect location or partner near their data center?
- Does this need multi-VPC or multi-region reach?

Design:

1. Provision AWS Direct Connect — a dedicated private connection via a Direct Connect location or partner.
2. Add a Site-to-Site VPN as a failover path for resilience (Direct Connect + VPN hybrid).
3. Use a Direct Connect Gateway if multiple VPCs or regions need to share the connection.

How the design changes if scale/scope grows: If timeline pressure ever trumps the reliability requirement, fall back to VPN first and run Direct Connect provisioning in parallel — that's the two-week scenario's answer, applied as an interim step here.

Likely follow-up: Why not just a bigger VPN? — VPN throughput is effectively capped per tunnel and it still traverses the public internet — it doesn't meet a hard low-latency, high-throughput requirement the way a dedicated connection does.

- [ ] [AWS] Lambda + EC2 both reaching an on-prem database over VPN

IF ASKED, SAY THIS
This is really the same first move as the cross-account Lambda question — Lambda has to join the VPC before it can reach anything private, on-prem included. Once it's in the same subnets as the EC2 instance, it's a standard hybrid routing problem: VPN to on-prem through a virtual private gateway, route table entries both directions, and security group egress allowing the DB port out.

Scenario: A Lambda function and an EC2 instance both need to reach a database that lives on-premises, over a VPN. Walk through the design, including routing.

Watch out for: Treating this as a brand-new problem instead of recognizing it's the same 'put Lambda in the VPC first' step from the cross-account scenario, now combined with standard hybrid routing.

Clarifying questions to ask first:

- Is this one VPC or several that need the on-prem route?

Design:

1. Attach the Lambda function to the same private subnets as the EC2 instance (it needs an ENI in the VPC to reach anything private, including on-prem).
2. Connect the VPC to on-prem via Site-to-Site VPN through a virtual private gateway (or Transit Gateway if multiple VPCs need the route).
3. Route tables in the private subnets point the on-prem CIDR at the VGW/TGW; the on-prem router points the VPC CIDR back through its VPN device.
4. Security group egress on Lambda and EC2 must allow the DB port outbound; the on-prem firewall must allow inbound from the AWS side CIDR.

How the design changes if scale/scope grows: If more than one VPC needs the same on-prem route, move the VPN attachment to a Transit Gateway instead of a per-VPC virtual private gateway — every attached VPC then shares one route to on-prem instead of duplicating VPN infrastructure.

Likely follow-up: Any gotcha with Lambda at higher concurrency here? — Each concurrent execution can consume an IP from the subnet — undersized subnets can hit IP exhaustion under high concurrency, so size subnets with that in mind.

- [ ] [AWS] Standard 3-tier web application

IF ASKED, SAY THIS
Public subnet for the ALB only. App tier sits in private subnets behind it, database tier in isolated private subnets with no internet route at all. Outbound-only access for the app tier comes from a NAT Gateway. And I'd chain the security groups — ALB accepts from the internet, app tier only accepts from the ALB's security group, database only accepts from the app tier's security group — so nothing skips a layer.

Scenario: Design a standard 3-tier web application on AWS: public-facing, with an application layer and a database layer.

Watch out for: Putting the application or database tier in a public subnet 'to keep it simple' — it works, but it's not the design you'd defend to a security-conscious customer.

Clarifying questions to ask first:

- Any existing VPC/network standards to follow?

Design:

1. Public subnets host an internet-facing Application Load Balancer.
2. Private subnets host the application tier (EC2/ECS/EKS) registered in the ALB's target group.
3. Isolated private subnets host RDS — no route to the internet at all.
4. A NAT Gateway in the public subnet gives the app tier outbound-only internet access for patches and package installs.
5. Security groups chain layer to layer: ALB SG allows 443 from the internet; app SG allows the app port only from the ALB SG; DB SG allows the DB port only from the app SG.

How the design changes if scale/scope grows: This is already the right baseline for most cases; the escalation is usually adding resilience (next scenario) rather than adding more network complexity here.

Likely follow-up: Why isolated subnets for RDS instead of just 'private'? — Isolated subnets have no route to a NAT Gateway or internet gateway at all — it removes an entire class of exfiltration risk even if the DB security group were ever misconfigured.

- [ ] [AWS] Make the 3-tier app resilient to an AZ failure and traffic spikes

IF ASKED, SAY THIS
Resilience here is mostly about spreading the existing tiers across AZs rather than adding new components. Subnets in at least two AZs for every tier, the app tier in an Auto Scaling Group across those AZs so it can both survive an AZ loss and scale out under load, and RDS Multi-AZ for a synchronous standby with automatic failover. Health checks at the ALB and ASG level handle the self-healing.

Scenario: Take that 3-tier architecture and make it resilient to an Availability Zone failure and able to handle traffic spikes.

Watch out for: Adding compute capacity everywhere without addressing the actual failure domain — an AZ outage isn't fixed by more instances in the same AZ.

Clarifying questions to ask first:

- What's the acceptable failover time if an AZ goes down?

Design:

1. Spread public, private, and DB subnets across at least two, ideally three, Availability Zones.
2. Put the app tier in an Auto Scaling Group spanning those AZs, with a target-tracking scaling policy on CPU or request count.
3. The ALB load balances across AZs automatically once targets exist in each.
4. Enable RDS Multi-AZ for a synchronous standby in a second AZ with automatic failover.
5. Rely on ALB and ASG health checks for self-healing — unhealthy targets are pulled and replaced automatically.

How the design changes if scale/scope grows: If the business needs protection against a full regional failure, not just an AZ, that's a different, heavier conversation — cross-region DR, covered separately.

☐ [AWS] Read-heavy app, slow static assets globally ▲

IF ASKED, SAY THIS
Two separate problems, two separate caches. For the read-heavy database load, ElastiCache in front of RDS absorbs the hot, repeated queries so they don't hit the database every time. For slow static assets globally, CloudFront caches them at edge locations close to the user instead of every request round-tripping to the origin region.

Scenario: The application is read-heavy and static assets load slowly for users outside your primary region. What do you add?

Watch out for: Scaling the database vertically to handle read load instead of addressing the access pattern with caching.

Clarifying questions to ask first:

- Is the read load on hot, repeated queries, or broad and varied?

Design:

1. Add ElastiCache (Redis or Memcached) in front of RDS for hot, frequently-read data — cuts direct database load.
2. Add CloudFront in front of the ALB and any S3-hosted assets for edge caching of static content.
3. Set sensible TTLs and an invalidation strategy for content that changes.

How the design changes if scale/scope grows: If the read pattern is broad rather than hot-key concentrated, caching helps less — that's when read replicas for horizontal read scaling become the better lever instead of, or alongside, caching.

Likely follow-up: How do you avoid serving stale cached content after a deploy? — Short TTLs on frequently-changing content plus explicit invalidation on deploy, versioned asset filenames for static assets so old cached copies simply stop being referenced.

☐ [AWS] End-to-end migration of an on-prem monolith ▲

IF ASKED, SAY THIS
I'd start with discovery, not architecture — mapping dependencies, data volume, and any compliance constraints, because that's what the 6 R's decision actually depends on. For a monolith on a tight deadline, rehost first and refactor later is often the right, defensible call. Target architecture follows the standard 3-tier pattern with hybrid connectivity during cutover. I'd pilot with the lowest-risk component, migrate in waves using MGN and DMS, shift traffic gradually with Route 53 weighted routing, and validate before decommissioning anything on-prem. And I'd frame the outcome in the customer's terms — cost, agility, reduced on-prem burden — not just 'it's running on EC2 now.'

Scenario: A customer wants to migrate a monolithic on-prem Java application to AWS. Walk through your approach end-to-end.

Watch out for: Answering entirely in operations and deployment language — 'we'd set up EC2, deploy the app, configure monitoring' — without covering discovery, the architecture decision, ownership, or the business outcome the customer actually cares about.

Clarifying questions to ask first:

- What's the downtime tolerance for cutover?
- Any compliance or data residency constraints?

Design:

1. Discovery and assessment first — inventory dependencies, data volume, and compliance constraints (AWS Application Discovery Service or a manual inventory).
2. Decide per component using the 6 R's — rehost, replatform, repurchase, refactor, retain, or retire — and justify the choice against the constraints (a tight deadline often means rehost first, refactor later, and that's a legitimate answer).
3. Design the target architecture — typically the 3-tier pattern above, plus hybrid connectivity for the cutover period.
4. Execute in waves: pilot a low-risk component first using AWS MGN for rehost and DMS for the database, then move the rest.
5. Cut over using Route 53 weighted routing for a gradual traffic shift, with a defined rollback plan.
6. Validate with performance and functional testing, and where feasible a parallel run before decommissioning on-prem.

How the design changes if scale/scope grows: For a large, tightly-coupled estate with many interdependencies, the discovery phase alone can justify bringing in AWS Application Discovery Service for automated dependency mapping rather than a manual inventory.

Likely follow-up: Who owns the account and network foundation versus the app migration itself? — The platform or landing-zone team owns account structure, networking, and central IAM guardrails; the app team owns the application-level migration decisions within that foundation — worth stating explicitly rather than implying you owned all of it.

[AWS] Database migration with minimal downtime

IF ASKED, SAY THIS
DMS handles this well — full load to get the bulk of the data across, then continuous change data capture so the target stays current while the source keeps serving production traffic. Once replication lag is effectively zero, the actual cutover is just repointing the connection string in a short window. I'd keep the source available read-only afterward as a rollback path rather than decommissioning it immediately.

Scenario: How do you migrate a production database to AWS with minimal downtime?

Watch out for: Proposing a straight dump-and-restore for a production database — it works for small, low-change-rate data, but implies a downtime window most production systems can't tolerate.

Clarifying questions to ask first:

- What's the actual acceptable downtime window?

Design:

1. Use AWS DMS for continuous replication — a full load followed by ongoing change data capture from source to target (RDS or Aurora).
2. Validate data consistency between source and target while replication is running.
3. Cut over in a short window once replication lag is near zero — repoint the application's connection string.
4. Keep the source running read-only for a period as a rollback path until confidence is established.

How the design changes if scale/scope grows: For a heterogeneous migration — different source and target engines — add the AWS Schema Conversion Tool ahead of DMS to handle schema and code translation before replication starts.

[AWS] Dividing ownership between platform team and application teams

IF ASKED, SAY THIS
I'd draw the line where most organizations actually draw it: the platform team owns the guardrails and shared infrastructure — account structure, networking, central security tooling — and my role is designing and operating within that framework. For example, I'd own the application architecture, the app-specific IAM roles, and the migration approach for that service, while working inside the account and network foundation the platform team provides. That's a more accurate answer than claiming ownership of the whole stack, and it's the honest one.

Scenario: In a migration project, how do you divide responsibility between the platform or landing-zone team and the application teams?

Watch out for: Implying you personally designed every layer, including the account structure and network foundation, when in most organizations that's a separate, dedicated function.

Design:

1. Platform team owns: AWS Organizations structure, account vending, the network foundation (VPC/Transit Gateway/hybrid connectivity), central IAM guardrails (SCPs), and central logging/security tooling.
2. Application team owns: the application's architecture within the provided landing zone, app-specific IAM roles scoped to least privilege within the guardrails, CI/CD for their service, and the specifics of their own data migration.

How the design changes if scale/scope grows: —

[AWS] Decouple one producer from several downstream consumers

IF ASKED, SAY THIS
SNS fan-out to SQS is the straightforward answer here — A publishes once to a topic, and every downstream service gets its own queue with independent processing and retry, without A ever knowing who's subscribed. If this grows into many event types with consumers that need to filter by content rather than just by topic, I'd reach for EventBridge instead, since it adds rule-based routing and a schema registry on top of the same fan-out idea.

Scenario: Service A needs to notify multiple downstream services when an order is placed, without A needing to know who's listening.

Watch out for: Having Service A call each downstream service directly — it couples A to every consumer's availability and makes adding a new consumer a code change in A.

Clarifying questions to ask first:

- How many event types, and do consumers need content-based filtering?

Design:

1. Publish to an SNS topic from Service A — A knows nothing about who's subscribed.
2. Each downstream service subscribes via its own SQS queue (the SNS fan-out to SQS pattern) for durability and independent retry.
3. If there are many event types and consumers need content-based routing rules, use EventBridge instead — it adds a schema registry and rule-based filtering.

How the design changes if scale/scope grows: EventBridge over SNS/SQS specifically when there are many distinct event sources or types and routing needs to be content-based rather than just topic-based.

[AWS] EC2 stops → cleanup, notify on-call, and audit log

IF ASKED, SAY THIS

One rule, multiple targets — EventBridge supports fan-out natively, so I wouldn't build one Lambda that does cleanup and notification and logging. The rule matches the EC2 stopped state-change event and fans out to three targets: a cleanup Lambda scoped to just that job, an SNS topic feeding a Slack webhook for on-call visibility, and CloudTrail handling the audit trail on its own. Each piece stays single-purpose and independently testable.

Scenario: When an EC2 instance stops, automatically trigger a Lambda to do cleanup. Now extend it: also notify the on-call team and log to a central audit trail.

Watch out for: Building a Lambda that does the cleanup and then also calls Slack and writes the audit log itself — one function doing three jobs instead of using EventBridge's native fan-out.

Design:

1. One EventBridge rule matching the EC2 state-change event (source aws.ec2, detail-type EC2 Instance State-change Notification, state stopped).
2. Target 1: a Lambda scoped only to cleanup — deregistering from a service registry, nothing else.
3. Target 2: an SNS topic that fans out to a Slack webhook for on-call notification.
4. Target 3: rely on CloudTrail plus the same rule (or a scheduled aggregation) for the audit trail, rather than the cleanup Lambda writing audit logs itself.
5. Scope the Lambda's execution role to exactly what cleanup requires — nothing broader.

How the design changes if scale/scope grows: —

- [ ] [AWS] Synchronous API call or async event flow?

IF ASKED, SAY THIS
The test I'd apply: if the caller needs an answer to proceed, it's synchronous — API Gateway and Lambda for order status makes sense because the UI is waiting on a value. If it's a backend service that just needs to know something happened and doesn't need to block on a response, that's async — EventBridge or SQS, which is also more resilient because it naturally retries if a downstream service is temporarily down.

Scenario: A front-end needs order status. Should that be a synchronous API call or an asynchronous event-driven flow?

Watch out for: Defaulting to 'everything should be event-driven' as a general best practice, without checking whether the caller actually needs an immediate answer.

Clarifying questions to ask first:

- Does the caller need the result to render something right now?

Design:

1. If the front-end needs an immediate value to render the UI: synchronous — API Gateway plus Lambda, or a direct service call.
2. If a backend service just needs to react to a state change without blocking the caller: asynchronous — EventBridge or SQS, which also survives downstream outages via retry and backoff.

How the design changes if scale/scope grows: —

- [ ] [AWS] Cross-account S3 access with no shared credentials

IF ASKED, SAY THIS
No access keys change hands here at all. I'd create a role in Account B with a trust policy naming Account A's role as a trusted principal, add a bucket policy on the S3 bucket that also allows that principal — both sides matter for cross-account S3 — and have the application in Account A call AssumeRole to get short-lived, automatically-expiring credentials.

Scenario: An application in Account A needs to read from an S3 bucket in Account B. No long-lived credentials should be shared. Design the access.

Watch out for: Creating an IAM user with an access key in Account B and handing the key to Account A — exactly the long-lived-credential pattern the requirement rules out.

Design:

1. Create an IAM role in Account B with a trust policy allowing Account A's role or service to assume it via sts:AssumeRole.
2. Add a resource-based bucket policy on the S3 bucket in Account B also allowing that specific principal from Account A — S3 cross-account access needs both the identity-based and resource-based side.
3. The application in Account A calls AssumeRole to get short-lived credentials — nothing long-lived changes hands.

How the design changes if scale/scope grows: —

- [ ] [AWS] Least-privilege Lambda: one bucket, one table

IF ASKED, SAY THIS
A dedicated role for this function, not a shared one — that alone prevents scope creep. The policy names the exact bucket ARN for GetObject and the exact table ARN for PutItem and UpdateItem, nothing wildcarded, nothing broader than what the function actually does. I'd also run IAM Access Analyzer against real usage to validate the policy isn't over- or under-scoped.

Scenario: A Lambda function needs to read from one specific S3 bucket and write to one specific DynamoDB table — nothing else. Design the IAM.

Watch out for: Reusing a broad, shared execution role 'because it already has the permissions' — it's fast but violates least privilege and widens blast radius if the function is ever compromised.

Design:

1. Give the function its own dedicated execution role, not a shared one.
2. Scope the policy to s3:GetObject on the specific bucket ARN, not a wildcard resource.

3. Scope dynamodb:PutItem and UpdateItem to the specific table ARN only.
4. No unrelated actions — no s3:DeleteBucket, no dynamodb:* wildcards.

How the design changes if scale/scope grows: —

☐ [AWS] Centralizing security visibility across an AWS Organization ▲

IF ASKED, SAY THIS
I'd centralize this in a dedicated Security account acting as delegated administrator for both Security Hub and GuardDuty, with AWS Organizations auto-enrolling every member account so findings roll up automatically instead of living scattered per account. Logging goes to a separate dedicated logging account. And I'd back it with SCPs at the OU level so member accounts can't disable GuardDuty or wander out of the guardrails.

Scenario: How do you centralize security visibility across a multi-account AWS Organization?

Watch out for: Enabling Security Hub and GuardDuty independently in every account with no aggregation — findings exist, but nobody has one place to see them all.

Design:

1. Designate a dedicated Security/Audit account (not the management account) as the delegated administrator for Security Hub and GuardDuty.
2. AWS Organizations integration auto-enrolls member accounts and aggregates their findings into that account.
3. Send org-wide CloudTrail and VPC Flow Logs to a dedicated logging account's S3 bucket with restricted access.
4. Use SCPs at the OU level to enforce guardrails — e.g., deny disabling GuardDuty, deny leaving the organization, restrict regions.

How the design changes if scale/scope grows: —

☐ [AWS] Multi-AZ vs Read Replica vs cross-region — when each applies ▲

IF ASKED, SAY THIS
Each one answers a different question. Multi-AZ is about surviving an AZ failure with automatic failover and near-zero data loss — that's an availability answer, same region. A read replica is about scaling read traffic, not failover — it's asynchronous and would need manual promotion if used for recovery. Cross-region is the actual DR answer for a full regional outage, typically a cross-region read replica or Aurora Global Database, with RPO tied to however far behind replication lag is at the moment of failure.

Scenario: When would you use Multi-AZ versus a Read Replica versus a cross-region setup?

Watch out for: Treating all three as interchangeable 'resilience' options instead of mapping each to a specific failure domain and RTO/RPO target.

Design:

1. Multi-AZ: HA for the primary within a region — synchronous standby, automatic failover, protects against an AZ failure. RTO roughly 60–120 seconds, RPO near zero.
2. Read Replica: scales reads, not failover protection by default — asynchronous, can be same-region or cross-region, can be manually promoted if needed.
3. Cross-region (cross-region read replica or Aurora Global Database): DR against a full regional failure — async replication, promoted manually or via a documented runbook, RPO depends on replication lag.

How the design changes if scale/scope grows: —

☐ [AWS] DR design: RTO 1 hour, RPO 15 minutes, secondary region ▲

IF ASKED, SAY THIS
A one-hour RTO and 15-minute RPO points to warm standby, not full active-active — active-active would meet the numbers but costs and complicates far more than the target requires, and a cold backup-only approach wouldn't meet either number. So: a scaled-down app tier already running in the DR region, Aurora Global Database or a cross-region replica for the data with replication lag monitored against the 15-minute target, Route 53 health checks doing the failover routing, and a runbook to scale up and promote during an actual event.

Scenario: Design DR for the 3-tier app with an RTO of 1 hour and an RPO of 15 minutes, secondary region.

Watch out for: Jumping straight to full active-active across regions — it technically satisfies the numbers but is far more cost and complexity than the target actually requires.

Design:

1. Warm standby pattern — not full active-active, not a cold backup-only approach.
2. App tier: a scaled-down copy running in the DR region, Auto Scaling Group held at minimum capacity.
3. Database tier: Aurora Global Database or a cross-region read replica, asynchronous replication, monitored to stay under the 15-minute RPO.
4. Route 53 health checks with a failover routing policy redirect traffic to the DR region on primary failure.
5. A documented runbook scales up the DR region's ASG and promotes the database replica during an actual failover.

How the design changes if scale/scope grows: For VM-heavy or lift-and-shift workloads specifically, AWS Elastic Disaster Recovery (DRS) is worth knowing about too — continuous block-level replication with orchestrated failover, a managed alternative to hand-building this pattern.

☐ [AWS] Design monitoring and alerting for a new service from scratch ▲

IF ASKED, SAY THIS

IF ASKED, SAY THIS

I'd anchor this in the four golden signals rather than just naming CloudWatch. Latency as p99 against the SLA, traffic for capacity and anomaly detection, errors as a concrete threshold like more than 10 5xx in 5 minutes, and saturation — CPU, memory, connection counts — because that's what predicts a failure before it happens rather than reporting one after the fact. Alarms route through SNS to PagerDuty for anything paging, Slack for warnings, and I'd use composite alarms so a single noisy metric doesn't page someone at 3 AM. And all of it defined as code next to the service, not clicked together manually.

Scenario: A new microservice just went to production. Design its monitoring and alerting strategy from scratch — which metrics, what thresholds, manual or automated?

Watch out for: Naming CloudWatch as the tool without a framework behind it — technically correct but exactly the 'generally correct but lacking detailed implementation' feedback pattern.

Clarifying questions to ask first:

- What's the SLA or acceptable latency/error target for this service?

Design:

1. Use the four golden signals as the mental model: latency, traffic, errors, saturation.
2. Latency: track p50/p90/p99 response time; alert if p99 breaches the SLA threshold for 5 consecutive minutes.
3. Traffic: request count/RPS — useful for capacity planning and spotting anomalies (a sudden drop usually means something upstream broke).
4. Errors: alert if the 5xx rate exceeds roughly 1% of requests, or a concrete count such as more than 10 5xx errors in 5 minutes.
5. Saturation: CPU or memory sustained above 80%, DB connections approaching max, queue depth growing — these predict failures before they happen.
6. Wire CloudWatch Alarms to SNS, routed to PagerDuty or Slack — separate a non-paging warning tier from a paging critical tier, and consider composite alarms (e.g., only page if errors AND latency are both elevated) to cut noise.
7. Define every alarm as code alongside the service (Terraform/CloudFormation) rather than clicking them in manually.

How the design changes if scale/scope grows: For a service with tight SLAs or many dependencies, add composite/anomaly-detection alarms and dependency-level tracing (X-Ray) on top of the golden-signals baseline.

Likely follow-up: How do you decide what pages someone versus what just posts to Slack? — Page for anything that's actively breaking the user experience right now or trending toward an outage (error rate, saturation); route to Slack for anything informational or trending but not yet urgent.

☐ [AWS] CloudWatch alerts: manual vs automated, and what triggers them ▲

IF ASKED, SAY THIS

Automated and defined as code every time — an alarm that only exists because someone clicked it in the console doesn't survive the next environment rebuild. And I'd always give concrete numbers rather than naming a metric in the abstract: CPU above 80% for 5 minutes, more than 10 5xx errors in a 5-minute window, RDS connections past about 80% of max_connections, pod restarts above zero in a short window. Composite alarms where it makes sense, so related signals don't each fire their own page.

Scenario: How do you create CloudWatch alerts — what metrics, manual or automated, what thresholds?

Watch out for: Listing metric names without concrete thresholds — the exact gap the interviewer called out: 'generally correct but lacking detailed implementation.'

Design:

1. Always automated, defined as code alongside the resource it monitors — not created by hand in the console.
2. Concrete thresholds, not vague ones: CPU above 80% sustained for 5 minutes, more than 10 5xx errors in 5 minutes, RDS connections above roughly 80% of max_connections, p99 latency above the SLA target, pod/container restarts above zero in a short window.
3. Use CloudWatch composite alarms to combine related signals and reduce false pages.

How the design changes if scale/scope grows: —

☐ [Azure] Function (Subscription A) → private API on a VM (Subscription B) ▲

IF ASKED, SAY THIS

Same shape of problem as the AWS version, so the same discipline applies: confirm it's really just two subscriptions talking one-to-one before reaching for anything heavier. The Function needs Regional VNet Integration first since it isn't in a VNet by default, then a straightforward VNet Peering connection between the two VNets — that works fine across subscriptions and even across tenants. One difference worth knowing: Azure peering creates the routing automatically, so there's no manual route table step the way there is in AWS.

Scenario: An Azure Function in Subscription A needs to call an API hosted on a VM in Subscription B, privately. Design it.

Watch out for: The same over-engineering instinct as the AWS version — reaching for ExpressRoute or a full hub-spoke build when it's a simple one-to-one case.

Clarifying questions to ask first:

- Just these two subscriptions, or will more join later?
- Same region, or does this cross regions too?

Design:

1. Give the Function App Regional VNet Integration so its outbound traffic routes through a VNet (Functions aren't in a VNet by default, same first move as the AWS Lambda case).
2. Peer the two VNets — VNet Peering works across subscriptions, and even across Entra ID tenants if configured; use global peering if the regions differ.
3. NSG on the VM's subnet or NIC allows inbound only from the Function's VNet/subnet on the API port.

4. No manual route table step needed for the basic case — VNet Peering auto-creates the system routes between the two VNets, unlike AWS where you add the route table entries yourself.

How the design changes if scale/scope grows: Three or more VNets needing mutual access — move to a hub VNet with peering to each spoke, or Virtual WAN if it needs to scale across regions. One service exposed to many outside tenants without full peering — that's Private Link Service instead.

> **Likely follow-up:** Does peering route transitively in Azure the way it doesn't in AWS? — No — VNet Peering isn't transitive either. Spoke-to-spoke through a hub still needs a hub VNet with a firewall/NVA, or Virtual WAN.

- [ ] [Azure] 3 business units, 3 subscriptions, one shared service

> **IF ASKED, SAY THIS**
> Same reasoning as the AWS hub-spoke case: with more business units coming, pairwise peering doesn't scale and isn't transitive anyway. I'd build a hub VNet for shared services with Azure Firewall for inspection, peer each business unit in as a spoke, and use UDRs to force spoke traffic through the firewall instead of routing direct. If this needs to span many regions, I'd reach for Virtual WAN instead of a hand-built hub — that's Azure's answer to hub-spoke at real scale.

Scenario: Three business units, each with their own subscription and VNet, all need private connectivity to a shared-services VNet. More business units will onboard over the next year.

Watch out for: Individual VNet peering per business unit — same scaling problem as AWS, plus no way to centrally inspect traffic between spokes without extra work.

Clarifying questions to ask first:

- Does spoke traffic need central inspection or firewalling?

Design:

1. Hub-and-spoke: a hub VNet holding shared services, with Azure Firewall or an NVA for centralized inspection.
2. Each business unit's VNet peers to the hub as a spoke.
3. User Defined Routes (UDRs) on the spokes force traffic through the firewall in the hub rather than going direct.
4. If this needs to scale across many regions and branches, move the hub to Azure Virtual WAN instead of a hand-built hub VNet.

How the design changes if scale/scope grows: Virtual WAN specifically once the hub-and-spoke needs to span multiple regions or many branch/VPN connections — a hand-built hub VNet stops being the efficient answer at that point.

- [ ] [Azure] SaaS API exposed to hundreds of customer VNets

> **IF ASKED, SAY THIS**
> This is Private Link Service, the direct equivalent of AWS PrivateLink — a Standard Load Balancer in front of the service, published as a Private Link Service, and each customer connects in via a Private Endpoint in their own VNet. One Azure-specific detail worth mentioning: connections need approval by default, which is a useful control point for onboarding, though it can be set to auto-approve for known subscriptions.

Scenario: You're building a SaaS product on Azure. Customers want to reach your API privately from their own VNets, and you expect hundreds of customers.

Watch out for: Proposing VNet peering per customer — the direct Azure analogue of the AWS PrivateLink trap, and it fails for the same reasons: doesn't scale, over-exposes the network.

Design:

1. Put a Standard Load Balancer in front of your service.
2. Create a Private Link Service referencing that load balancer.
3. Each customer creates a Private Endpoint in their own VNet connecting to your Private Link Service.
4. By default, connection requests need your approval — you can auto-approve specific subscriptions if that fits the onboarding flow.

How the design changes if scale/scope grows: —

- [ ] [Azure] On-prem ERP needs Azure access in two weeks

> **IF ASKED, SAY THIS**
> Same call as AWS and for the same reason — ExpressRoute doesn't fit a two-week deadline regardless of how the workload might grow later. A Site-to-Site VPN through a VPN Gateway in the GatewaySubnet stands up in hours and comfortably handles moderate traffic.

Scenario: Finance needs their on-prem ERP to reach a new Azure-hosted reporting service within two weeks. Traffic is moderate.

Watch out for: Reaching for ExpressRoute because it sounds more robust — same timeline mismatch as the AWS case, provisioning takes far longer than two weeks.

Design:

1. Site-to-Site VPN via a VPN Gateway, deployed into the reserved GatewaySubnet.
2. On-prem side needs a compatible VPN device.
3. Provisioning is hours, not the weeks ExpressRoute would take.

How the design changes if scale/scope grows: If throughput or latency requirements tighten later, add ExpressRoute alongside the VPN — Azure explicitly supports ExpressRoute and VPN coexistence, so the VPN doesn't need to be torn out.

- [ ] [Azure] Permanent, high-throughput, low-latency, compliance-sensitive link

> **IF ASKED, SAY THIS**
> Given reliability and compliance both point away from the public internet, this is ExpressRoute from the start rather than starting with VPN — a dedicated private connection with consistent throughput and latency. I'd still layer a Site-to-Site VPN as an automatic failover path, since Azure explicitly supports running both together.

Scenario: A customer needs a permanent, high-throughput, low-latency, compliance-sensitive connection between their data center and Azure. Cost is secondary to reliability.

Watch out for: Defaulting to VPN because it's simpler to set up, when the requirement explicitly rules that out (public-internet transit, capped throughput).

Design:

1. Provision ExpressRoute — a private, dedicated connection that never traverses the public internet.
2. Keep a Site-to-Site VPN as a failover path (ExpressRoute + VPN coexistence).
3. Use an ExpressRoute Gateway sized to the throughput requirement.

How the design changes if scale/scope grows: —

- [ ] [Azure] Function + VM both reaching an on-prem database over VPN

> **IF ASKED, SAY THIS**
> Same first move as the cross-subscription Function scenario — Regional VNet Integration puts the Function in the same VNet as the VM, and from there it's a standard hybrid routing problem: a VPN Gateway in the GatewaySubnet connecting to on-prem, UDRs if traffic needs to go through a firewall, and NSGs allowing the DB port outbound.

Scenario: An Azure Function and a VM both need to reach a database that lives on-premises, over a VPN. Walk through the design, including routing.

Watch out for: Treating the Function's networking as a separate problem from the VM's, instead of recognizing that Regional VNet Integration puts it on the same footing as the VM once it's done.

Design:

1. Give the Function Regional VNet Integration into the same VNet as the VM.
2. Connect that VNet to on-prem via a VPN Gateway in the GatewaySubnet.
3. Configure UDRs if traffic needs to pass through an NVA or firewall rather than routing direct.
4. NSGs on the relevant subnets allow the DB port outbound; the on-prem firewall allows inbound from the Azure-side address range.

How the design changes if scale/scope grows: —

- [ ] [Azure] Standard 3-tier web application

> **IF ASKED, SAY THIS**
> Structurally identical discipline to the AWS 3-tier answer, translated into Azure's building blocks: Application Gateway with WAF at the edge, App Service or VMSS behind it in a private subnet reached via Regional VNet Integration, and Azure SQL reached through a Private Endpoint rather than a public connection string. NSGs chain layer to layer the same way security groups do on AWS.

Scenario: Design a standard 3-tier web application on Azure: public-facing, with an application layer and a database layer.

Watch out for: Putting the app or database tier on a public IP directly instead of layering subnets the way you would on AWS — the same discipline applies across clouds.

Design:

1. Application Gateway (with WAF) in a public-facing subnet.
2. App Service with Regional VNet Integration, or a VMSS in a private subnet, behind the Application Gateway.
3. Azure SQL Database reached via a Private Endpoint in a data subnet — no public endpoint at all.
4. NSGs chain the same way as AWS security groups: Gateway subnet accepts from the internet, app subnet accepts only from the Gateway, data subnet accepts only from the app subnet.

How the design changes if scale/scope grows: —

> **Likely follow-up:** Application Gateway or Front Door here? — Application Gateway is regional and VNet-integrated, the right fit for this single-region 3-tier design; Front Door is global, edge-based, and comes in once there's a multi-region or global-latency requirement.

- [ ] [Azure] Make the 3-tier app resilient to a zone failure and traffic spikes

> **IF ASKED, SAY THIS**
> Zone-redundant configuration across the stack — Application Gateway v2, VMSS, and Azure SQL all support it — handles the zone-failure requirement, and VMSS autoscale rules on CPU or queue length handle the traffic-spike requirement. If the business also needs protection against a full regional outage, that's Azure SQL failover groups layered on top, which is really the DR conversation rather than the zone-resilience one.

Scenario: Take that 3-tier architecture and make it resilient to an Availability Zone failure and able to handle traffic spikes.

Watch out for: Scaling out without zone-awareness — more instances that all happen to sit in the same zone doesn't survive a zone failure.

Design:

1. Deploy Application Gateway v2, VMSS, and Azure SQL in a zone-redundant configuration across Availability Zones.
2. VMSS autoscale rules on CPU or queue length handle traffic spikes.
3. Azure SQL failover groups add automatic or manual regional failover if that's needed beyond zone redundancy.

How the design changes if scale/scope grows: —

- [ ] [Azure] Read-heavy app, slow static assets globally

> **IF ASKED, SAY THIS**
> Two caches for two problems, same as the AWS answer: Azure Cache for Redis absorbs the hot read traffic in front of Azure SQL, and Azure Front Door handles the global static-asset problem by caching at the edge close to the user instead of every request reaching back to the primary region. Front Door is the right call here specifically because it's global and edge-based, versus Application Gateway which is regional.

Scenario: The application is read-heavy and static assets load slowly for users outside your primary region. What do you add?

Watch out for: Reaching for a bigger Azure SQL tier to absorb read load instead of addressing the access pattern with caching, same trap as the AWS version.

Design:

1. Azure Cache for Redis in front of Azure SQL for hot, frequently-read data.
2. Azure Front Door in front of Application Gateway for global edge caching, routing, and WAF at the edge.
3. Sensible TTLs and cache invalidation for content that changes.

How the design changes if scale/scope grows: —

- [ ] [Azure] End-to-end migration of an on-prem monolith

> **IF ASKED, SAY THIS**
> Same structure as the AWS answer, Azure-native tools: Azure Migrate for discovery and dependency mapping first, then the same 6 R's decision per component — rehost first on a tight deadline is just as defensible here. Target architecture sits inside a proper Azure landing zone rather than an ad-hoc subscription. Execution uses Azure Migrate: Server Migration and DMS for the database, cutover via Traffic Manager or Front Door weighted routing, and validation before anything on-prem gets decommissioned. And I'd close with the business outcome, not just the technical steps.

Scenario: A customer wants to migrate a monolithic on-prem Java application to Azure. Walk through your approach end-to-end.

Watch out for: Same trap as the AWS version — staying in deployment language instead of covering discovery, the architecture decision, ownership, and the business outcome.

Clarifying questions to ask first:

- What's the downtime tolerance for cutover?
- Any compliance or data residency constraints?

Design:

1. Discovery and assessment first, using Azure Migrate for dependency mapping and right-sizing recommendations.
2. Decide per component using the same 6 R's framework — rehost, replatform, repurchase, refactor, retain, retire.
3. Design an Azure landing zone (management group hierarchy, policy, networking) as the target foundation.
4. Execute with Azure Migrate: Server Migration for rehost candidates, Azure Database Migration Service for the data layer.
5. Cut over using Traffic Manager or Front Door weighted routing for a gradual shift, with a rollback plan.
6. Validate with performance and functional testing before decommissioning on-prem.

How the design changes if scale/scope grows: —

> **Likely follow-up:** Who owns the landing zone versus the app migration? — The platform team owns the management group hierarchy, policy, and central networking; the app team owns the application-level migration decisions within that landing zone — the same ownership split as the AWS answer.

- [ ] [Azure] Database migration with minimal downtime

> **IF ASKED, SAY THIS**
> Azure DMS in online mode does the same job as AWS DMS here — continuous sync keeps the target current while the source keeps serving production traffic, and the actual cutover is a short window once lag is near zero.

Scenario: How do you migrate a production database to Azure with minimal downtime?

Watch out for: A straight export/import for a production database — implies a downtime window most production systems can't accept.

Design:

1. Azure Database Migration Service for online (continuous sync) migration to Azure SQL or a managed instance.
2. Validate data consistency while sync is running.
3. Cut over in a short window once replication lag is near zero.

How the design changes if scale/scope grows: —

- [ ] [Azure] Dividing ownership between platform team and application teams

> **IF ASKED, SAY THIS**
> Same honest framing as the AWS answer: the platform team owns the management group hierarchy, policy, central networking, and identity; my role is designing and operating within that — the application architecture, the app-specific RBAC, the CI/CD, and the migration approach for that specific service.

Scenario: In a migration project, how do you divide responsibility between the platform team and the application teams on Azure?

Watch out for: Implying personal ownership of the whole landing zone when that's typically a dedicated platform function.

Design:

1. Platform team owns: management group hierarchy, Azure Policy, central networking (hub VNet, ExpressRoute/VPN), and central identity (Entra ID, Conditional Access).
2. Application team owns: their resource group, app-specific RBAC scoped to least privilege, CI/CD for their service, and their own migration specifics.

How the design changes if scale/scope grows: —

- [ ] [Azure] Decouple one producer from several downstream consumers

> **IF ASKED, SAY THIS**
> Service Bus Topics with per-consumer Subscriptions — A publishes once, each downstream service has its own subscription with independent processing and, if needed, its own filter on which messages it cares about. Worth mentioning as a genuine cross-cloud difference: this is one service doing what AWS needs SNS and SQS together to do.

Scenario: Service A needs to notify multiple downstream services when an order is placed, without A needing to know who's listening.

Watch out for: Having A call each downstream service directly, coupling it to every consumer's availability — same trap as the AWS version.

Design:

1. Publish to a Service Bus Topic from Service A.
2. Each downstream service gets its own Subscription on that topic, optionally with its own filter — durable, independent processing per consumer.
3. This is a genuine Azure difference worth naming: Service Bus Topics and Subscriptions give you pub/sub with per-consumer filtering natively in one service, where AWS combines SNS for pub/sub with SQS for durability to get the same effect.

How the design changes if scale/scope grows: —

- [ ] [Azure] VM deallocated → cleanup, notify on-call, and audit log

> **IF ASKED, SAY THIS**
> The Azure-native pattern here is an Activity Log Alert watching for the VM deallocate operation, firing an Action Group that fans out to multiple actions — the cleanup Function is one action, a Teams or Slack webhook is another. The audit trail comes from routing the Activity Log itself to Log Analytics via diagnostic settings, so the cleanup Function stays scoped to just cleanup, same single-responsibility idea as the AWS EventBridge answer.

Scenario: When a VM is deallocated, automatically trigger a Function to do cleanup. Now extend it: also notify the on-call team and log to a central audit trail.

Watch out for: Building the cleanup Function to also handle notification and logging itself, instead of using Azure Monitor's native fan-out to an Action Group.

Design:

1. An Activity Log Alert watching for the VM deallocate operation (Microsoft.Compute/virtualMachines/deallocate/action).
2. The alert fires an Action Group with multiple actions: one calls the cleanup Function, another posts to Teams/Slack via a webhook or Logic App.
3. Route the Activity Log to a Log Analytics workspace via diagnostic settings for the audit trail, rather than having the Function write audit logs itself.

How the design changes if scale/scope grows: For custom application-level events rather than resource state changes, Event Grid with custom topics is the better fit than Activity Log Alerts — Activity Log is specifically for control-plane operations on Azure resources.

- [ ] [Azure] Synchronous API call or async event flow?

> **IF ASKED, SAY THIS**
> Same test as the AWS answer: does the caller need a value to proceed right now? If yes, synchronous — API Management in front of a Function for order status. If it's a backend service just reacting to something happening, async via Event Grid or Service Bus, which also tolerates a downstream outage better through retry.

Scenario: A front-end needs order status. Should that be a synchronous API call or an asynchronous event-driven flow?

Watch out for: Defaulting to 'event-driven is always better' without checking whether the caller needs an immediate answer — same trap as AWS.

Design:

1. Immediate value needed to render the UI: synchronous, API Management in front of a Function or App Service.
2. Backend reacting to a state change without blocking the caller: asynchronous, Event Grid or Service Bus.

How the design changes if scale/scope grows: —

- [ ] [Azure] Cross-subscription access with no shared credentials

IF ASKED, SAY THIS
This is genuinely simpler than the AWS equivalent — a Managed Identity on the Subscription A resource, given an RBAC role scoped to the specific storage account in Subscription B. There's no secret to create or rotate, and no separate trust-policy document to author the way AWS needs — it's one role assignment.

Scenario: An application in Subscription A needs to read from a Storage Account in Subscription B. No long-lived credentials should be shared. Design the access.

Watch out for: Generating a Storage Account access key or a service principal secret and handing it to Subscription A — exactly the long-lived-credential pattern the requirement excludes.

Design:

1. Enable a Managed Identity (system- or user-assigned) on the resource in Subscription A.
2. Assign that identity an RBAC role — Storage Blob Data Reader, scoped to the specific storage account — in Subscription B.
3. No secrets are created, stored, or rotated at any point; Azure manages the identity's credential lifecycle entirely.

How the design changes if scale/scope grows: —

Likely follow-up: How does this compare to AWS AssumeRole? — Same goal, less ceremony — Managed Identity removes credential management entirely rather than issuing short-lived credentials via a trust policy and an explicit assume-role call.

- [ ] [Azure] Least-privilege Function: one storage container, one Cosmos DB

IF ASKED, SAY THIS
A dedicated Managed Identity for this Function, with Storage Blob Data Reader scoped to the exact storage account and Cosmos DB Data Contributor scoped to the exact Cosmos account — never Owner or Contributor at a broader scope just because it's convenient.

Scenario: A Function needs to read from one Blob container and write to one Cosmos DB container — nothing else. Design the identity.

Watch out for: Assigning Contributor at the subscription or resource group scope because it's fast — the same over-broad-role trap as reusing a shared IAM role on AWS.

Design:

1. A user-assigned Managed Identity dedicated to this Function.
2. Storage Blob Data Reader scoped to the specific storage account (or container, via a custom role if that granularity is needed).
3. Cosmos DB Data Contributor scoped to the specific Cosmos account/container, not the subscription.

How the design changes if scale/scope grows: —

- [ ] [Azure] Centralizing security visibility across an Azure tenant

IF ASKED, SAY THIS
Tenant-wide Defender for Cloud enabled via Azure Policy so it's automatic for new subscriptions rather than something each team remembers to turn on, with Microsoft Sentinel aggregating everything centrally through a Log Analytics workspace as the SIEM layer. Guardrails come from Azure Policy at the management group level — deny public IPs, require encryption, that kind of enforcement rather than just visibility.

Scenario: How do you centralize security visibility across a multi-subscription Azure tenant?

Watch out for: Enabling Defender for Cloud independently per subscription with no tenant-wide policy or central aggregation.

Design:

1. Enable Microsoft Defender for Cloud tenant-wide via Azure Policy, so new subscriptions are auto-enrolled rather than manually configured.
2. Aggregate findings and logs centrally through Microsoft Sentinel as the SIEM, fed by a Log Analytics workspace.
3. Enforce guardrails with Azure Policy at the management group level — for example, denying public IPs or requiring encryption.

How the design changes if scale/scope grows: This is genuinely a strong area to lean into confidently given the SC-300 background — Entra ID, Conditional Access, and this exact security-governance pattern are certified ground, not a gap.

- [ ] [Azure] Availability Zones vs geo-replication vs Azure Site Recovery

IF ASKED, SAY THIS
Same mapping exercise as the AWS version. Availability Zones handle a datacenter-level failure within a region, directly comparable to Multi-AZ. Geo-replication is about read scaling and DR together, asynchronous, comparable to a cross-region read replica. Azure Site Recovery is the one without a tidy one-line AWS parallel — it's a managed DR-orchestration product specifically for VM workloads, closest to AWS Elastic Disaster Recovery, and it's worth knowing both exist rather than hand-building DR with snapshots and scripts.

Scenario: When would you use zone redundancy versus geo-replication versus Azure Site Recovery?

Watch out for: Treating all three as interchangeable 'resilience,' the same trap as the AWS Multi-AZ/Read Replica/cross-region question.

Design:

1. Availability Zones: HA within a region, protects against a datacenter-level failure — the Azure analogue of AWS Multi-AZ.
2. Geo-replication (Azure SQL active geo-replication, Cosmos DB multi-region): read scaling plus DR, asynchronous.

3. Azure Site Recovery: managed DR orchestration specifically for VM-based workloads — continuous replication to a secondary region plus orchestrated, tested failover and failback.

How the design changes if scale/scope grows: —

- [ ] [Azure] DR design: RTO 1 hour, RPO 15 minutes, secondary region

> **IF ASKED, SAY THIS**
> Same reasoning as the AWS answer — a 1-hour RTO and 15-minute RPO calls for warm standby, not full active-active. Azure Site Recovery keeps the VM tier continuously replicated but scaled down in the secondary region, an Azure SQL auto-failover group handles the database side within the RPO target, and Traffic Manager or Front Door does the failover routing. All of it backed by a tested runbook, not a plan that's only ever been read, never run.

Scenario: Design DR for the 3-tier app with an RTO of 1 hour and an RPO of 15 minutes, secondary region.

Watch out for: Jumping to full active-active across regions when warm standby satisfies the actual numbers at far lower cost and complexity.

Design:

1. Warm standby: VM-based tiers replicated continuously via Azure Site Recovery into the secondary region, scaled down until failover.
2. Azure SQL auto-failover group for the database tier — asynchronous geo-replication, monitored to stay under the 15-minute RPO.
3. Traffic Manager or Front Door for DNS-level failover routing to the secondary region.
4. A tested runbook to scale up the secondary region and trigger the failover group during an actual event.

How the design changes if scale/scope grows: —

- [ ] [Azure] Design monitoring and alerting for a new service from scratch

> **IF ASKED, SAY THIS**
> Same golden-signals framework as the AWS answer, Azure-native tools underneath: Application Insights gives latency percentiles and failure rate on the app tier out of the box, plus dependency tracking. Errors and saturation get concrete thresholds, not vague ones. Alerts — metric-based or KQL log queries against Log Analytics — route through Action Groups, and I'd mention Dynamic Thresholds specifically, since it's an ML-based alternative to static numbers that Azure offers natively.

Scenario: A new service just went to production. Design its monitoring and alerting strategy from scratch.

Watch out for: Naming Azure Monitor as the tool without a framework — same gap the interviewer flagged on the AWS side.

Clarifying questions to ask first:

- What's the SLA or acceptable latency/error target for this service?

Design:

1. Same four golden signals as the AWS answer: latency, traffic, errors, saturation.
2. Application Insights gives p50/p95/p99 latency and failure rate natively for the app tier, plus dependency tracking.
3. Errors: alert on a concrete threshold, e.g. failure rate above roughly 1% over 5 minutes.
4. Saturation: CPU/memory sustained above 80%, connection pool near max — the same predictive signals as AWS.
5. Azure Monitor Alerts — metric alerts plus log alerts written as KQL queries against Log Analytics — route to Action Groups (email, SMS, webhook, ITSM, Logic App).
6. Consider Dynamic Thresholds on metric alerts — an ML-based alternative to hand-picking static numbers, worth mentioning as an Azure-specific capability.

How the design changes if scale/scope grows: —

> **Likely follow-up:** Azure Monitor Workbooks — what are they for here? — Dashboarding on top of the same data — useful for a team-facing view of the golden signals, separate from the alerting pipeline itself.

- [ ] [Azure] Azure Monitor alerts: manual vs automated, and what triggers them

> **IF ASKED, SAY THIS**
> Automated and defined as code every time, same principle as AWS — an alert that only exists because someone clicked it into the portal doesn't survive a redeploy. Concrete numbers, not vague metric names: CPU above 80% for 5 minutes, failure rate above roughly 1%, p99 against the SLA. And I'd use Dynamic Thresholds where a static number is genuinely hard to pick with confidence, with action group severities separating what pages someone from what's just informational.

Scenario: How do you create Azure Monitor alerts — what metrics, manual or automated, what thresholds?

Watch out for: Same gap as AWS — naming metrics without concrete thresholds or without saying how they're deployed.

Design:

1. Automated, defined as code (Bicep/Terraform) alongside the resource, not clicked together in the portal.
2. Concrete thresholds: CPU above 80% for 5 minutes, failure rate above roughly 1% over 5 minutes, connection pool near max, p99 latency above the SLA target.
3. Use Dynamic Thresholds where a static number is hard to pick confidently, and action group severity tiers to separate paging from informational alerts.

How the design changes if scale/scope grows: —

Platform Engineering

- [ ] Define Platform Engineering in a way that distinguishes it from "DevOps" and from a traditional infrastructure/ops team — this exact question comes up constantly at the AVP level.

> **IF ASKED, SAY THIS**
> Platform Engineering is the discipline of designing and building self-service internal tooling and infrastructure — an Internal Developer Platform (IDP) — that lets product/application teams provision, deploy, and operate their own services safely and independently, without needing to individually understand or directly touch the underlyi

Platform Engineering is the discipline of designing and building **self-service internal tooling and infrastructure** — an Internal Developer Platform (IDP) — that lets product/application teams provision, deploy, and operate their own services safely and independently, without needing to individually understand or directly touch the underlying cloud/Kubernetes/networking complexity. Where DevOps is a cultural philosophy (dev and ops share responsibility for the full lifecycle) without prescribing who builds what, and a traditional ops/infra team is often a ticket-driven gatekeeper (app team requests, infra team executes), Platform Engineering explicitly treats the platform itself as a **product**, with application teams as its internal customers, and success measured by developer self-sufficiency and velocity rather than ticket-closure throughput.

- [ ] Why did the industry move toward platform engineering — what specific failure mode of pure "everyone owns their own infra" DevOps was it responding to?

> **IF ASKED, SAY THIS**
> The idealized "you build it, you run it" DevOps model, taken to its extreme without platform investment, means every application team independently re-solves the same infrastructure problems (their own CI/CD setup, their own Kubernetes manifests, their own monitoring conventions, their own way of handling secrets) — massive duplicated eff

The idealized "you build it, you run it" DevOps model, taken to its extreme without platform investment, means every application team independently re-solves the same infrastructure problems (their own CI/CD setup, their own Kubernetes manifests, their own monitoring conventions, their own way of handling secrets) — massive duplicated effort, wildly inconsistent security/reliability posture across teams, and a cognitive load on application engineers (needing deep infra expertise just to ship a feature) that actively slows product delivery rather than the intended speedup. Platform Engineering is the corrective: centralize the genuinely undifferentiated, repeatedly-reinvented infrastructure work into a platform team building reusable paved paths, while application teams keep true ownership and autonomy over their own service's business logic and operational decisions within those paved paths.

- [ ] What's the difference between an Internal Developer Platform (IDP) and just "a bunch of infrastructure tools"?

> **IF ASKED, SAY THIS**
> An IDP is specifically the integrated, self-service layer on top of that infrastructure — a developer requesting a new service gets a working CI/CD pipeline, a provisioned environment, standard observability, and a deployment path in one coherent, largely automated flow (often via a portal, CLI, or Git-based workflow), rather than needing

An IDP is specifically the **integrated, self-service layer** on top of that infrastructure — a developer requesting a new service gets a working CI/CD pipeline, a provisioned environment, standard observability, and a deployment path in one coherent, largely automated flow (often via a portal, CLI, or Git-based workflow), rather than needing to individually configure Terraform, wire up Datadog, request AWS IAM roles, and set up Argo CD by hand, one tool at a time, each with its own learning curve and its own ticket to some other team. "A bunch of infrastructure tools" is necessary but not sufficient — the IDP is specifically the cohesive, opinionated, self-service experience layered on top that makes those tools usable by an application developer without becoming an infrastructure expert themselves.

- [ ] Lay out the four fundamental team types from the Team Topologies framework, and give a concrete example of each in an SRE/platform context.

> **IF ASKED, SAY THIS**
> Stream-aligned team — organized around a single, continuous flow of work (a product area, a customer journey) — most application/feature teams are this. Platform team — provides self-service internal infrastructure/tooling capabilities (the IDP) to reduce cognitive load on stream-aligned teams — this is where SRE/Platform Engineering typi

- **Stream-aligned team** — organized around a single, continuous flow of work (a product area, a customer journey) — most application/feature teams are this.
- **Platform team** — provides self-service internal infrastructure/tooling capabilities (the IDP) to reduce cognitive load on stream-aligned teams — this is where SRE/Platform Engineering typically sits.
- **Enabling team** — temporary, high-expertise team that helps stream-aligned teams adopt a new capability or overcome a specific gap (e.g. a short-term embedded team helping app teams migrate to a new observability stack), then steps back once the capability is absorbed — deliberately not a permanent dependency.
- **Complicated-subsystem team** — owns a genuinely complex piece of the system requiring specialist deep expertise that would be inefficient to distribute (e.g. a dedicated team owning a shared, algorithmically complex billing/rating engine).

- [ ] What are the three core interaction modes between teams in this framework, and why does the mode matter, not just who talks to whom?

> **IF ASKED, SAY THIS**

Collaboration — teams working closely together, high communication bandwidth, typically for a bounded period while figuring something new out (appropriate but expensive to sustain indefinitely). X-as-a-Service — one team consumes another's capability through a well-defined, largely self-service interface with minimal ongoing direct intera

Collaboration — teams working closely together, high communication bandwidth, typically for a bounded period while figuring something new out (appropriate but expensive to sustain indefinitely). **X-as-a-Service** — one team consumes another's capability through a well-defined, largely self-service interface with minimal ongoing direct interaction needed (the target mode for a mature platform team's relationship with app teams — "here's the API/paved path, use it"). **Facilitating** — one team (often an enabling team) actively helps another improve, without doing the work for them. The mode matters because defaulting to permanent "Collaboration" between a platform team and every app team it supports doesn't scale — it recreates the ticket-driven bottleneck platform engineering was supposed to remove; the explicit goal of platform investment is to move that relationship toward X-as-a-Service over time.

- [ ] What is "cognitive load" in this framework, and how does it concretely justify platform investment to a skeptical business stakeholder?

IF ASKED, SAY THIS

Cognitive load is the total mental capacity a team needs to do its job — intrinsic (genuinely necessary for their actual domain), extraneous (unnecessary friction — needing to understand Kubernetes networking internals just to deploy a simple CRUD service), and germane (the focused learning specific to their task). A team with excessive e

Cognitive load is the total mental capacity a team needs to do its job — intrinsic (genuinely necessary for their actual domain), extraneous (unnecessary friction — needing to understand Kubernetes networking internals just to deploy a simple CRUD service), and germane (the focused learning specific to their task). A team with excessive extraneous load (constantly fighting infrastructure complexity that's not actually their core job) ships slower and with more errors, regardless of individual skill — this reframes "why invest in a platform team" from a vague efficiency argument into a specific, measurable claim: every unit of infrastructure complexity the platform absorbs into a self-service paved path is extraneous cognitive load removed from every single stream-aligned team that would otherwise have had to carry it individually, which is the concrete ROI argument for platform investment leadership actually responds to.

- [ ] Define "golden path" precisely — and the critical nuance about whether it's mandatory.

IF ASKED, SAY THIS

A golden path (or "paved road") is the officially supported, well-documented, tooled-and-automated way to accomplish a common task (spin up a new service, deploy to production, set up standard observability) — designed to be the easiest path specifically so teams choose it voluntarily because it's genuinely the path of least resistance, n

A golden path (or "paved road") is the officially supported, well-documented, tooled-and-automated way to accomplish a common task (spin up a new service, deploy to production, set up standard observability) — designed to be the easiest path specifically so teams choose it voluntarily because it's genuinely the path of least resistance, not because it's the only permitted path. The critical nuance senior interviewers probe for: a good golden path is **opinionated but not mandatory** — teams with a genuine, justified reason to deviate can, but they take on the operational burden of that deviation themselves rather than the platform team supporting every possible bespoke variant. A golden path that's actually a locked-down mandatory path tends to breed workarounds and resentment; one that's simply the easiest, best-supported option tends to get adopted voluntarily because it's genuinely good.

- [ ] What makes self-service infrastructure genuinely self-service, versus a request queue with extra automation steps?

IF ASKED, SAY THIS

Genuine self-service means an application team can go from "I need X" to "X exists and is working" with zero required intervention from another human team along the critical path — a developer fills out a scaffold (a template repo, a portal form, a CLI command) and the entire provisioning chain (repo creation, CI/CD wiring, cloud resource

Genuine self-service means an application team can go from "I need X" to "X exists and is working" with zero required intervention from another human team along the critical path — a developer fills out a scaffold (a template repo, a portal form, a CLI command) and the entire provisioning chain (repo creation, CI/CD wiring, cloud resource provisioning, initial monitoring setup) completes automatically. A request queue with automation still requires a human on the platform team to approve/trigger each step, even if the step itself is automated once triggered — that's faster than fully manual, but it's not self-service, and it doesn't scale the same way, since the platform team remains a bottleneck proportional to request volume rather than a fixed investment that scales with adoption.

- [ ] How do you build guardrails into a self-service platform without it becoming either unsafe (too permissive) or useless (too restrictive)?

IF ASKED, SAY THIS

Push safety into the paved path's defaults and automation rather than approval gates: sensible secure-by-default templates (a new service scaffold comes with least-privilege IAM, standard resource limits, and security scanning already wired in, not opt-in), policy-as-code enforcement (OPA/Kyverno admission control, the Terraform Sentinel/

Push safety into the paved path's defaults and automation rather than approval gates: sensible secure-by-default templates (a new service scaffold comes with least-privilege IAM, standard resource limits, and security scanning already wired in, not opt-in), policy-as-code enforcement (OPA/Kyverno admission control, the Terraform Sentinel/Conftest patterns from chapter 3) that blocks genuinely dangerous configurations automatically at request time rather than requiring a human reviewer to catch it, and clear, fast escape hatches for legitimate edge cases (a documented process for "I need something outside the golden path" that doesn't dead-end at "not supported") rather than either blocking everything non-standard or allowing everything unchecked. The goal is safety enforced by the system's design, not by a human gatekeeper reviewing every request — the latter is exactly the bottleneck self-service is meant to eliminate.

- [ ] What problem does a developer portal like Backstage actually solve, at its core?

IF ASKED, SAY THIS

At scale, "where's the documentation for this service," "who owns it," "what's its current on-call status," and "how do I even find what services exist" become genuinely hard organizational problems on their own, independent of any individual tool's quality — a Software Catalog (Backstage's core feature) is a centralized, structured inven

At scale, "where's the documentation for this service," "who owns it," "what's its current on-call status," and "how do I even find what services exist" become genuinely hard organizational problems on their own, independent of any individual tool's quality — a **Software Catalog** (Backstage's core feature) is a centralized, structured inventory of every service/component with standardized metadata (owner, on-call link, docs link, dependencies, tech stack) that makes an organization's systems actually discoverable and navigable, rather than tribal knowledge scattered across wikis, Slack history, and individual engineers' memory. Backstage specifically is an open-source (originally Spotify, now CNCF) framework for building this kind of portal, extensible via plugins to surface CI/CD status, cost data, security posture, and self-service scaffolding actions all in one place.

- [ ] What's a Backstage "Software Template," and how does it operationalize the golden path concept?

IF ASKED, SAY THIS
A Software Template is a scaffolding definition (parameters + a set of actions) that a developer fills out through the portal UI (service name, language, team ownership) to generate a brand-new, fully-wired repository — pre-configured with the standard CI/CD pipeline, the org's standard directory structure, initial observability hooks, an

A Software Template is a scaffolding definition (parameters + a set of actions) that a developer fills out through the portal UI (service name, language, team ownership) to generate a brand-new, fully-wired repository — pre-configured with the standard CI/CD pipeline, the org's standard directory structure, initial observability hooks, and correct Backstage catalog registration, all in one guided flow. This is the concrete implementation of "golden path" (section 10.3) as an actual clickable, self-service artifact rather than just a wiki page describing the recommended approach — turning documented best practice into the actual literal easiest path, which is what makes adoption happen in practice rather than staying aspirational.

- [ ] What does "treating the platform as a product" concretely change about how a platform team operates, day to day?

IF ASKED, SAY THIS
It means the platform team adopts real product-management discipline applied internally: they have identified internal "customers" (the application teams) whose actual needs and pain points are gathered directly (interviews, usage data, adoption metrics) rather than assumed; they prioritize their roadmap based on genuine internal customer

It means the platform team adopts real product-management discipline applied internally: they have identified internal "customers" (the application teams) whose actual needs and pain points are gathered directly (interviews, usage data, adoption metrics) rather than assumed; they prioritize their roadmap based on genuine internal customer impact rather than what's technically interesting to build; they measure success by adoption and developer satisfaction, not by "features shipped"; and — critically — they treat low adoption of a platform capability as a signal to investigate and fix (the product/feature has a problem), not as a discipline problem with the application teams for "not using it correctly." A platform team that mandates usage instead of earning it through genuine value is skipping the actual "as a product" discipline and just rebranding a mandate.

- [ ] How would you actually measure Developer Experience (DevEx) with real numbers, not just "developers seem happier"?

IF ASKED, SAY THIS
Combine system-level metrics with self-reported ones, since neither alone is sufficient: DORA metrics (deployment frequency, lead time for changes, change failure rate, time to restore service) as objective throughput/stability signals at the org level; time-to-first-deploy for a new service or new hire (a very concrete, measurable proxy

Combine system-level metrics with self-reported ones, since neither alone is sufficient: **DORA metrics** (deployment frequency, lead time for changes, change failure rate, time to restore service) as objective throughput/stability signals at the org level; **time-to-first-deploy for a new service or new hire** (a very concrete, measurable proxy for how good the golden path actually is in practice); and structured periodic developer satisfaction surveys (the SPACE framework — Satisfaction, Performance, Activity, Communication, Efficiency — is the commonly-cited model for this) capturing the qualitative, self-reported side that pure system metrics miss (a team can look fine on DORA metrics while still being quietly miserable due to unrelated friction). Citing DORA plus a structured qualitative framework by name, rather than a vague "developer happiness," is what distinguishes a genuinely informed answer at this level.

- [ ] How do you justify platform team headcount/investment to leadership in terms they actually weigh decisions in?

IF ASKED, SAY THIS
Translate platform investment into the same language leadership already uses for any other investment decision: engineering-hours saved in aggregate (if a golden path cuts new-service setup from 2 weeks to 2 hours, multiply that across however many new services get created per quarter — a concrete, defensible number), reduced incident fre

Translate platform investment into the same language leadership already uses for any other investment decision: engineering-hours saved in aggregate (if a golden path cuts new-service setup from 2 weeks to 2 hours, multiply that across however many new services get created per quarter — a concrete, defensible number), reduced incident frequency/severity tied to standardization (fewer bespoke, individually-misconfigured setups means fewer unique failure modes to debug), and faster time-to-market for new product initiatives (a measurable business-facing outcome, not just an internal engineering efficiency metric). The weakest version of this pitch is "good engineering practice" as the sole justification — the strongest, and the one an AVP-level conversation expects, ties platform investment directly to a business metric leadership already tracks and cares about.

- [ ] Lay out the spectrum of Kubernetes multi-tenancy models, from loosest to strictest isolation.

IF ASKED, SAY THIS
Namespace-based (soft multi-tenancy) — one shared cluster, tenants separated by namespace + RBAC + ResourceQuota + NetworkPolicy. Lowest operational overhead, but tenants still share the same control plane, node pool (unless further segmented), and blast radius for cluster-level issues. Namespace + stronger isolation (node-level segmentat

- **Namespace-based (soft multi-tenancy)** — one shared cluster, tenants separated by namespace + RBAC + ResourceQuota + NetworkPolicy. Lowest operational overhead, but tenants still share the same control plane, node pool (unless further segmented), and blast radius for cluster-level issues.
- **Namespace + stronger isolation (node-level segmentation)** — dedicated node pools per tenant (via taints/tolerations/affinity) on top of namespace isolation, reducing noisy-neighbor resource contention risk between tenants even though they still share one control plane.
- **Virtual clusters** (vCluster, Kamaji) — each tenant gets what looks like their own control plane/API server, actually running as a workload inside a shared host cluster — stronger API-level isolation than namespaces without the full cost of separate physical clusters.
- **Cluster-per-tenant (hard multi-tenancy)** — fully separate clusters, strongest isolation (a tenant can't even see the existence of another tenant's cluster), highest operational/cost overhead (N clusters to patch, upgrade, monitor instead of one).

☐ What specifically can a namespace NOT isolate, even with RBAC/ResourceQuota/NetworkPolicy all correctly configured — and why does that matter for a platform team's tenancy decision?

IF ASKED, SAY THIS
Namespaces don't isolate the kernel — pods across namespaces on the same node still share the same kernel, meaning a kernel-level exploit or a container escape vulnerability could theoretically cross tenant boundaries regardless of how well RBAC/NetworkPolicy are configured (those control API-level and network-level access, not kernel-lev

Namespaces don't isolate the **kernel** — pods across namespaces on the same node still share the same kernel, meaning a kernel-level exploit or a container escape vulnerability could theoretically cross tenant boundaries regardless of how well RBAC/NetworkPolicy are configured (those control API-level and network-level access, not kernel-level isolation). They also don't isolate **cluster-scoped resources** (CRDs, StorageClasses, the API server itself) or shared cluster-critical components (CoreDNS, cluster autoscaler) — a misbehaving or resource-exhausting tenant can degrade these shared components for everyone. For a platform team, this is the actual decision-driving question: for genuinely hostile-tenant or strict-compliance scenarios (unrelated external customers sharing infrastructure, strict regulatory data segregation requirements), namespace-based soft multi-tenancy's residual shared-kernel/shared-control-plane risk may be unacceptable, pushing the decision toward virtual clusters or cluster-per-tenant despite the added operational cost — versus internal teams within one trusted organization, where namespace-based isolation is very often a perfectly reasonable, much cheaper choice.

☐ SCENARIO Scenario: you're brought in as AVP to build a platform engineering function from scratch at a company that currently has 15 application teams each running their own bespoke infra. Where do you actually start?

IF ASKED, SAY THIS
Not with a big-bang platform build — start by talking to several of those 15 teams directly to find the genuinely shared, most-painful, most-duplicated problem (very often it's CI/CD setup or observability onboarding, but confirm rather than assume), and build the smallest viable golden path for exactly that one problem first, working clo

Not with a big-bang platform build — start by talking to several of those 15 teams directly to find the genuinely shared, most-painful, most-duplicated problem (very often it's CI/CD setup or observability onboarding, but confirm rather than assume), and build the smallest viable golden path for exactly that one problem first, working closely (Collaboration mode, per Team Topologies) with one or two willing early-adopter teams to prove real value before generalizing. Resist the urge to design a comprehensive platform roadmap upfront based on assumption — the fastest way to lose organizational trust in a new platform function is shipping something nobody asked for while the actual daily pain goes unaddressed. Use early adoption success (with real metrics — section 10.5) as the case to expand scope and get buy-in from the remaining teams, rather than mandating adoption from day one before the platform has proven itself.

☐ Scenario: an application team refuses to adopt the platform's golden path, insisting their service is "different" and needs a bespoke setup. As the platform lead, how do you handle it?

IF ASKED, SAY THIS
First, genuinely investigate whether they're right — sometimes "we're different" is accurate, and forcing a golden path that truly doesn't fit is exactly the mandatory-not-opinionated failure mode from section 10.3; the platform team should have a real process for evaluating and potentially incorporating a legitimate new requirement into

First, genuinely investigate whether they're right — sometimes "we're different" is accurate, and forcing a golden path that truly doesn't fit is exactly the mandatory-not-opinionated failure mode from section 10.3; the platform team should have a real process for evaluating and potentially incorporating a legitimate new requirement into the golden path itself, benefiting future teams with the same need. If, after genuine investigation, the objection is really just unfamiliarity/inertia rather than a real technical constraint, address it with data and support rather than a mandate: show the concrete cost of their bespoke path (their team now owns 100% of the operational burden and security posture the golden path would have absorbed — tie back to the cognitive-load argument in section 10.2) and offer hands-on support adopting it, rather than an authority-based "you have to use this." If they still opt out after that, they explicitly own the resulting operational burden and risk — that's a legitimate outcome of an opinionated-but-not-mandatory platform, not a failure of the platform team, as long as that tradeoff was made with genuinely informed consent, not just stonewalling.

☐ SCENARIO Scenario: six months into a platform initiative, adoption is low and a VP asks why the investment isn't paying off. How do you diagnose and respond, as the person accountable for the platform?

IF ASKED, SAY THIS
Treat low adoption as a genuine product signal to investigate, not a reason to get defensive or to blame application teams for not adopting — check concretely whether the golden path actually solves the real pain point teams have (re-validate against section 10.7's early-discovery step; priorities may have shifted, or the initial problem

Treat low adoption as a genuine product signal to investigate, not a reason to get defensive or to blame application teams for not adopting — check concretely whether the golden path actually solves the real pain point teams have (re-validate against section 10.7's early-discovery step; priorities may have shifted, or the initial problem selection may have been wrong), whether onboarding friction to the platform itself is the blocker (an overly complex "self-service" path that's actually still hard to use defeats its own purpose), and whether teams simply don't know it exists or don't trust it yet (a communication/trust problem, distinct from a product-quality problem, requiring a different fix — visible internal advocacy and a track record of

reliability, not more features). Bring the VP a specific, evidenced diagnosis and a concrete corrective plan with a defined next checkpoint, rather than either over-promising a quick fix or being purely defensive about the original approach — this is precisely the kind of accountable, data-driven leadership judgment an AVP-level panel is evaluating for in a question framed this way.

DevOps

- [ ] What is DevOps, at the level of a one-sentence answer you could open with?

> **IF ASKED, SAY THIS**
> DevOps is not a technology, it's a way of working, built on three core insights: small frequent changes are safer than large infrequent ones; the team that builds a service should run it in production; and feedback loops between production and development should be as short as possible.

Everything else — the CI/CD tooling, monitoring, on-call — exists in service of those three ideas, not as the definition of DevOps itself. Leading with this framing (rather than a tool list) signals seniority.

- [ ] What are the DORA metrics, and what does 'elite' actually look like on each?

> **IF ASKED, SAY THIS**
> Deployment Frequency, Lead Time for Changes, Mean Time to Restore, and Change Failure Rate. Elite: multiple deploys/day, <1hr commit-to-prod lead time, <1hr MTTR, and a low single-digit-percent change failure rate.

- **Deployment Frequency** — Elite: multiple times/day. Improve via CI/CD investment, automated testing, feature flags.
- **Lead Time for Changes** — Elite: under 1 hour from commit to production. Improve via smaller PRs, faster CI, trunk-based development.
- **Mean Time to Restore (MTTR)** — Elite: under 1 hour. Improve via better observability, runbooks, and automated rollback.
- **Change Failure Rate** — Elite: 0-15%. Improve via canary/progressive delivery and stronger pre-prod testing.

Quoting your own team's numbers against these bands (even approximately) is one of the highest-signal things you can do in a senior SRE/DevOps interview.

- [ ] CI/CD vs GitOps vs DevSecOps — how do these three actually differ, not just as buzzwords?

> **IF ASKED, SAY THIS**
> CI/CD (Jenkins/Harness/GitHub Actions) builds, tests, and pushes a deploy on every commit — focus is speed of delivery. GitOps (ArgoCD/Flux) makes Git the source of truth for cluster state and syncs it in — focus is consistency/auditability, and critically, CI never holds cluster credentials. DevSecOps weaves security scanning into every stage of the pipeline rather than as a separate gate at the end.

The GitOps security property is worth stating explicitly in an interview: because ArgoCD/Flux pulls from Git rather than CI pushing to the cluster, your CI system never needs cluster-admin credentials — a compromised CI pipeline can't directly touch production, it can only propose a Git change.

- [ ] What is DevOps and how is it different from what teams did before it existed?

> **IF ASKED, SAY THIS**
> DevOps is not a tool or a job title - it is a cultural and engineering movement: Before DevOps (the wall of confusion): Development team wrote code. When ready, they threw it "over the wall" to Operations. Operations team deployed it to production. When it failed in production (which it often did be

DevOps is not a tool or a job title - it is a cultural and engineering movement:

Before DevOps (the wall of confusion):
Development team wrote code. When ready, they threw it "over the wall" to Operations. Operations team deployed it to production. When it failed in production (which it often did because Ops environment was different from Dev), Dev blamed Ops for breaking it. Ops blamed Dev for writing bad code. Release cycles were quarterly or annual because each release was a major risky event.

What DevOps changed:
1. **You build it, you run it:** The team that writes the service is also responsible for running it in production. Development engineers carry the pager. This immediately incentivises writing reliable, observable code.
2. **Continuous Delivery:** Small, frequent deployments instead of large, infrequent ones. Smaller changes are easier to test, easier to roll back, and fail in smaller ways.
3. **Infrastructure as Code:** Operations work treated like software development: version controlled, tested, reviewed in pull requests.
4. **Feedback loops as tight as possible:** Developers see production metrics within minutes of deployment. If something breaks, they know immediately and fix it immediately.

The Three Ways (Gene Kim's model):
Flow (fast left-to-right delivery), Feedback (fast right-to-left signals), Continuous Learning (experiment and learn from failure)

How SRE implements DevOps: SRE is Google's specific implementation of DevOps principles. Error budgets make reliability quantitative. Postmortems make learning from failure systematic. Eliminating toil makes automation the default.

- [ ] What is the difference between CI, CD (Continuous Delivery), and CD (Continuous Deployment)?

> **IF ASKED, SAY THIS**
> These three terms are often confused. The distinction matters: CI - Continuous Integration: Every developer integrates their code into the shared main branch at least daily. Every integration is automatically built and tested. The goal is to detect integration problems as early as possible. Without

These three terms are often confused. The distinction matters:

CI - Continuous Integration:

Every developer integrates their code into the shared main branch at least daily. Every integration is automatically built and tested. The goal is to detect integration problems as early as possible. Without CI, developers work in isolation for weeks and then have a painful merge.
Tools: Jenkins, GitHub Actions, GitLab CI

Continuous Delivery:
Code is always in a deployable state. Every successful CI build produces an artifact that COULD be deployed to production. The actual deployment requires a human approval (button click, pull request merge, etc.).
Key insight: The decision to deploy is a business decision, not a technical one. The technical capability to deploy safely is always there.

Continuous Deployment:
Every successful CI build is automatically deployed to production - no human approval needed. This requires very high confidence in your automated tests and deployment pipeline.
Used by: Netflix, Amazon, companies with mature testing and feature flags.
Not appropriate for: Financial services (regulated change management requires approval), safety-critical systems

Which to use in financial services:
CI: always
Continuous Delivery: deploy frequently (multiple times per day to staging), but require CAB/ITSM approval for production (ServiceNow CHG)
Continuous Deployment: not used for regulated production environments - change management regulations require documented approval

The important nuance: You can have very fast Continuous Delivery. Harness + ServiceNow can auto-create, route for approval, and auto-close a CHG in 15 minutes. That is Continuous Delivery that also satisfies SOX requirements.

- [ ] What is DevSecOps and how do you embed security into a CI/CD pipeline without slowing teams down?

IF ASKED, SAY THIS
DevSecOps = security shifts left, from the end of the release cycle to the beginning: The problem with traditional security: Security review happened at the end: build the feature, test it, hand it to the security team, wait 2 weeks for pen test results, then discover you need to rewrite half the au

DevSecOps = security shifts left, from the end of the release cycle to the beginning:

The problem with traditional security:
Security review happened at the end: build the feature, test it, hand it to the security team, wait 2 weeks for pen test results, then discover you need to rewrite half the authentication layer. By that time, the developer has moved to a different project.

Security at each CI/CD stage (with approximate time cost):

Pre-commit (developer's laptop):
- Git hooks: detect secrets before they are committed (gitleaks, trufflehog) - 1 second
- IDE plugins: SonarLint highlights security issues as you type - 0 seconds overhead

Pull Request / CI build (fast gates, under 5 minutes):
- SAST (Static Application Security Testing): SonarQube, Semgrep - scan for SQL injection, XSS, hardcoded secrets - 2-3 minutes
- Dependency scan: OWASP Dependency Check, Snyk - find CVEs in your libraries - 1-2 minutes
- Secret scanning: detect any secrets that slipped through - 30 seconds

Docker build stage:
- Container vulnerability scan: Trivy, Grype - scan the image for OS and library CVEs - 1-2 minutes
- Image signing: Cosign - prove this image was built by your CI pipeline - 30 seconds

Deployment stage:
- OPA/Kyverno admission policies: reject deployments without resource limits, without non-root user, with :latest tags
- Infrastructure scan: Checkov scans Terraform before apply - 30 seconds

Production (continuous):
- Falco: runtime security - detects suspicious behaviour in running containers (shell spawned, unexpected network connection)
- AWS Security Hub / Defender for Cloud: continuous compliance assessment

How to avoid slowing teams down:
Make security gates fast (under 5 minutes total for all checks). Fail on CRITICAL and HIGH CVEs only (not medium - too many false positives). Allow teams to accept risk for specific CVEs with documented justification. Give developers actionable fix instructions, not just "you have a vulnerability".

- [ ] How is SRE actually different from traditional sysadmin/Ops, beyond the title?

IF ASKED, SAY THIS
Traditional Ops tends to be reactive and treats reliability as an unstated, unlimited requirement — 100% uptime as the implicit goal, with success measured by absence of incidents. SRE, as Google originally framed it, applies **engineering discipline to operations**: reliability targets are explicit

Traditional Ops tends to be reactive and treats reliability as an unstated, unlimited requirement — 100% uptime as the implicit goal, with success measured by absence of incidents. SRE, as Google originally framed it, applies **engineering discipline to operations**: reliability targets are explicit and quantified (SLOs), some unreliability is accepted on purpose (an error budget) rather than chased to zero, toil is treated as something to actively measure and eliminate through automation rather than accepted as the job, and postmortems are blameless and produce real engineering follow-through. The practical difference shows up in day-to-day priorities — an SRE team with error budget left might ship changes fast; the same team with a burned error budget deliberately slows down and prioritizes reliability work, which is a very different operating model from 'just try not to break anything, always.'

- [ ] What does 'designing for resilience' actually look like in practice?

IF ASKED, SAY THIS

Redundancy so no single instance/AZ/dependency failing takes the whole system down — multiple AZs, multiple instances behind a load balancer, no single database with no standby. **Graceful degradation** — a non-critical feature failing (say, a recommendations widget) shouldn't take down the core

Redundancy so no single instance/AZ/dependency failing takes the whole system down — multiple AZs, multiple instances behind a load balancer, no single database with no standby. **Graceful degradation** — a non-critical feature failing (say, a recommendations widget) shouldn't take down the core flow (checkout) around it; that usually means isolating failure domains and having sensible fallbacks rather than one unhandled exception cascading. **Circuit breakers** stop a struggling downstream dependency from being hammered with retries that make its recovery slower, or from exhausting the caller's own resources (connection pools, threads) waiting on a dependency that's clearly not coming back soon. **Retries with exponential backoff and jitter** handle the transient-failure case without a fleet of clients all retrying in lockstep and turning a brief blip into a thundering-herd outage. None of these are exotic — they're the concrete, implementable version of 'assume things will fail, and design so one failure doesn't cascade.'

- [ ] What does having an 'architecture mindset' mean day to day, beyond executing infrastructure tickets?

IF ASKED, SAY THIS
It means evaluating whether a proposed change is actually the *right* design before implementing it, not just the requested one — asking what happens at 10x the current scale, what the failure modes are, what it costs to operate and to undo later, rather than just making the immediate ask work. A co

It means evaluating whether a proposed change is actually the *right* design before implementing it, not just the requested one — asking what happens at 10x the current scale, what the failure modes are, what it costs to operate and to undo later, rather than just making the immediate ask work. A concrete, practical instinct that comes with seniority: reaching for the **simplest design that genuinely satisfies the requirement**, rather than the most sophisticated one available — a well-chosen NAT Gateway and a straightforward route table beats a needlessly elaborate Transit Gateway/PrivateLink setup for a problem that didn't actually need that complexity, and the more complex option usually costs more to build, operate, and hand off to the next engineer for very little real benefit if the requirement didn't call for it. Concretely, it shows up as asking 'why' one more time before building, and being willing to push back (with reasoning, not just an opinion) on a design that solves today's ticket but creates next quarter's incident.

- [ ] What does “automation-first” mean in an SRE context, and how do you decide what to automate first?

IF ASKED, SAY THIS
It means defaulting to a scripted, repeatable solution instead of a manual one, and treating manual repetition as a bug in the system rather than normal operations. Google's SRE model gives a useful test for what counts as toil : manual, repetitive, automatable, tactical, and work that scales linear

It means defaulting to a scripted, repeatable solution instead of a manual one, and treating manual repetition as a bug in the system rather than normal operations. Google's SRE model gives a useful test for what counts as **toil**: manual, repetitive, automatable, tactical, and work that scales linearly with the service instead of shrinking with better engineering.

To prioritize, I score candidate tasks on frequency × time cost × error-proneness × risk if skipped. High-frequency, error-prone, low-judgment tasks (log cleanup, cert renewal, routine scaling) go first — they have the fastest payback and the least chance of automating away something that actually needed a human's judgment.

- [ ] Walk through building an automated remediation workflow for a recurring low-risk incident, like a disk-space alert.

IF ASKED, SAY THIS
Alert fires → webhook to an automation runner (a Lambda/Azure Function, or a runner like Rundeck) → runner executes a pre-approved, idempotent script that clears rotated logs/temp files or grows the volume if it's cloud-native storage → posts the action and result back to the incident channel → auto

Alert fires → webhook to an automation runner (a Lambda/Azure Function, or a runner like Rundeck) → runner executes a pre-approved, idempotent script that clears rotated logs/temp files or grows the volume if it's cloud-native storage → posts the action and result back to the incident channel → auto-resolves the alert if the check now passes, or escalates to a human if it doesn't.

The guardrails matter more than the happy path: a dry-run mode you validate first, a hard cap on how much the automation is allowed to do per run (blast-radius limit), and an audit log of every action taken so a human can always reconstruct what happened.

- [ ] Why does idempotency matter for automation scripts, and how do you enforce it?

IF ASKED, SAY THIS
An orchestrator or a retry-on-failure alert handler will sometimes run the same automation twice. If the script isn't idempotent, a retry can create duplicate resources, double-apply a fix, or fail with a confusing secondary error instead of just no-op'ing safely.

An orchestrator or a retry-on-failure alert handler will sometimes run the same automation twice. If the script isn't idempotent, a retry can create duplicate resources, double-apply a fix, or fail with a confusing secondary error instead of just no-op'ing safely.

I enforce it with a check-then-act pattern — query current state before mutating (does the tag already exist, is the pod already restarted, is the record already created) — and prefer declarative tools that build this in for free, like Terraform's plan/apply model, over imperative scripts that just fire commands blindly.

- [ ] What workflow orchestration tools have you used or would use to chain multi-step automation, and when would you pick one over another?

IF ASKED, SAY THIS

IF ASKED, SAY THIS
Ansible is my default for agentless, push-based configuration across heterogeneous hosts — good fit when the steps are mostly “configure this fleet of machines.” For cloud-native, event-driven chains I'd reach for AWS Step Functions or Azure Logic Apps — visual state machines, native IAM/Managed-Ide

Ansible is my default for agentless, push-based configuration across heterogeneous hosts — good fit when the steps are mostly “configure this fleet of machines.” For cloud-native, event-driven chains I'd reach for AWS Step Functions or Azure Logic Apps — visual state machines, native IAM/Managed-Identity integration, built-in retry and error branches, no infrastructure of my own to run.

For self-hosted runbook automation across a mixed AWS+Azure+on-prem estate with strong RBAC and an audit trail, something like Rundeck or StackStorm fits better since it's cloud-agnostic and the trigger source can be anything. The real deciding factor is where the trigger and the target actually live — native cloud events stay in the native orchestrator; cross-platform runbooks go to a tool that doesn't care which cloud it's talking to.

- [ ] How would you automate certificate rotation for internal services before they expire?

IF ASKED, SAY THIS
Inside Kubernetes, cert-manager with an ACME issuer (Let’s Encrypt) or an internal CA handles this end to end — it renews before expiry and re-mounts the TLS secret automatically, so the workload just needs to reload on secret change. I'd still put an independent watchdog on top: a blackbox exporter

Inside Kubernetes, cert-manager with an ACME issuer (Let's Encrypt) or an internal CA handles this end to end — it renews before expiry and re-mounts the TLS secret automatically, so the workload just needs to reload on secret change. I'd still put an independent watchdog on top: a blackbox exporter probe (`probe_ssl_earliest_cert_expiry`) scraped by Prometheus, alerting at 30/14/7 days out, so a silent cert-manager failure doesn't go unnoticed until the outage.

For services outside Kubernetes, a scheduled job hits Vault/ACM/Key Vault's API to reissue and pushes the new cert through the same deployment pipeline as any other config change — never a manual copy onto a box.

- [ ] Explain event-driven automation vs. schedule-driven (cron) automation, with an SRE example of each.

IF ASKED, SAY THIS
Schedule-driven automation runs on a fixed clock regardless of system state — nightly backups, a weekly cost report, a CronJob that checks certificate expiry. It's predictable and easy to reason about, but by definition it reacts on a delay.

Schedule-driven automation runs on a fixed clock regardless of system state — nightly backups, a weekly cost report, a CronJob that checks certificate expiry. It's predictable and easy to reason about, but by definition it reacts on a delay.

Event-driven automation reacts to something happening — an alert firing triggers an autoscale action, an S3/Blob upload triggers a processing pipeline. It cuts time-to-remediation dramatically, but the handler has to be idempotent and safe to fire in bursts, because real triggers rarely arrive as one clean event — they arrive as retries, duplicates, and near-simultaneous fan-out.

- [ ] You're asked to reduce toil for a repetitive “restart the pod when a memory leak causes OOM” scenario. What's the automated fix vs. the automation band-aid, and how do you choose which to build first?

IF ASKED, SAY THIS
The pod-level liveness probe already gives you the reflex restart for free. If that's genuinely not enough — say the leak is fast enough to cause user-facing errors before the probe trips — I'd add a watcher that restarts earlier based on a memory-usage threshold and pages on every occurrence.

The pod-level liveness probe already gives you the reflex restart for free. If that's genuinely not enough — say the leak is fast enough to cause user-facing errors before the probe trips — I'd add a watcher that restarts earlier based on a memory-usage threshold and pages on every occurrence.

But I'd treat that as a band-aid, explicitly, and open a ticket to profile the actual leak (heap dump, memory profiler, or just bisecting recent deploys). Automating around a recurring root cause indefinitely just hides a bug behind a script — useful to buy time, not a substitute for fixing it, and I'd say exactly that in the postmortem so it doesn't quietly become the permanent state.

- [ ] What's a runbook, and how do you safely turn a manual runbook into full automation?

IF ASKED, SAY THIS
A runbook is the documented, step-by-step response to a known failure mode — written so any on-call engineer, not just the person who wrote it, can follow it under pressure.

A runbook is the documented, step-by-step response to a known failure mode — written so any on-call engineer, not just the person who wrote it, can follow it under pressure.

- Write the manual steps down precisely first — that's your automation spec.
- Script the read-only/diagnostic steps first; they carry no risk and build confidence in the tool.
- Script the remediation steps behind a manual trigger or approval gate.
- After several successful supervised runs, move the trigger to fully automatic, with alerting and a rollback path.
- Keep the original runbook as the documented human fallback — automation fails too, and someone needs to know what to do by hand.

- [ ] How do you handle credentials for automation that has to talk to multiple systems — say AWS, Azure, and Kubernetes in the same script?

IF ASKED, SAY THIS

Nothing hardcoded, nothing long-lived where I can avoid it. On the AWS side that means IAM roles assumed via OIDC federation (IRSA if it's running as a pod); on Azure, a Managed Identity; inside the cluster, a ServiceAccount bound to a narrowly-scoped Role, not cluster-admin.

Nothing hardcoded, nothing long-lived where I can avoid it. On the AWS side that means IAM roles assumed via OIDC federation (IRSA if it's running as a pod); on Azure, a Managed Identity; inside the cluster, a ServiceAccount bound to a narrowly-scoped Role, not cluster-admin.

Anything that has to be a real secret — an API token, a DB password — comes from a central store (Vault, Secrets Manager, Key Vault) injected at runtime, never baked into an image or committed to git. Each credential is scoped to exactly the verbs and resources that script needs, rotated on a schedule, and its use is logged so I can audit what the automation actually touched.

- [ ] Describe a workflow you'd build to automatically scale infrastructure ahead of a predictable traffic spike, like a known sale event.

IF ASKED, SAY THIS

I'd combine scheduled pre-scaling with reactive autoscaling as a backstop, not either alone. A CronJob or scheduled Lambda bumps HPA minReplicas (or ASG desired capacity) an hour or two ahead of the event based on last year's traffic curve, so pods are warm and past any slow cold-start before load a

I'd combine scheduled pre-scaling with reactive autoscaling as a backstop, not either alone. A CronJob or scheduled Lambda bumps HPA `minReplicas` (or ASG desired capacity) an hour or two ahead of the event based on last year's traffic curve, so pods are warm and past any slow cold-start before load actually arrives. HPA/Cluster Autoscaler/Karpenter stay active on top of that for anything the forecast underestimated.

I'd validate the target capacity with a load test beforehand rather than trust the forecast blindly, and schedule an automatic step-down back to baseline afterward — the same discipline that prevents the scale-up from silently becoming a permanent, unnecessary cost.

- [ ] What makes infrastructure-changing automation risky, and what safety nets do you build in?

IF ASKED, SAY THIS

The underlying principle: risk scales with blast radius × frequency ÷ human review, so the safety nets attack whichever of those terms is largest for that specific automation.

- **Dry-run by default** — plan/preview mode is the normal path; “actually apply” is the deliberate exception.
- **Canary the blast radius** — run against one resource or one small environment before the fleet.
- **Rate limits / circuit breakers** — cap how many remediation actions the automation can take per hour; past that cap it alerts a human instead of continuing to act, so a bad trigger loop can't cascade.
- **Full audit log** — every action taken is logged with enough detail to revert it by hand if needed.

The underlying principle: risk scales with blast radius × frequency ÷ human review, so the safety nets attack whichever of those terms is largest for that specific automation.

- [ ] Give a real example from your own background of a manual process you automated — what did it replace and what was the impact?

IF ASKED, SAY THIS

One concrete one: a cross-namespace pod restart-count reporter I built in Python, which used to be a manual kubectl get pods sweep across namespaces whenever someone suspected instability — now it runs on a schedule and flags anomalies automatically. Alongside it, a Docker image/container cleanup sc

One concrete one: a cross-namespace pod restart-count reporter I built in Python, which used to be a manual `kubectl get pods` sweep across namespaces whenever someone suspected instability — now it runs on a schedule and flags anomalies automatically. Alongside it, a Docker image/container cleanup script that used to be manual disk-space firefighting on build hosts.

Across deployment and release automation more broadly, that kind of work is a direct contributor to the ~60% reduction in manual deployment effort I've delivered in recent roles — I go deeper into the specific project and numbers in the Behavioral section.

- [ ] How does Ansible complement Terraform rather than compete with it?

IF ASKED, SAY THIS

Terraform provisions and tracks the desired state of infrastructure — create the VM, the network, the managed database — declaratively. Ansible handles configuration management and orchestration on top of what's already provisioned — install packages, configure files, orchestrate a rolling deploy —

Terraform provisions and tracks the desired state of infrastructure — create the VM, the network, the managed database — declaratively. Ansible handles configuration management and orchestration on top of what's already provisioned — install packages, configure files, orchestrate a rolling deploy — typically agentless, idempotent tasks rather than full state tracking.

A common pattern: Terraform provisions the servers or cluster, Ansible configures what runs on them. In a fully Kubernetes-native shop, Ansible's role naturally shrinks, since container images already bake in a lot of what Ansible used to configure by hand.

SRE

- [ ] What's the actual difference between an SLI, an SLO, and an SLA?

> **IF ASKED, SAY THIS**
> An SLI is the measured metric itself — e.g. the percentage of requests that succeed. An SLO is your internal target for that SLI over a window, e.g. 99.9% success over 30 days. An SLA is the external, often contractual, commitment to a customer with real consequences (service credits, penalties) if missed.

An **SLI** is the measured metric itself — e.g. the percentage of requests that succeed. An **SLO** is your internal target for that SLI over a window, e.g. 99.9% success over 30 days. An **SLA** is the external, often contractual, commitment to a customer with real consequences (service credits, penalties) if missed.

In practice SLAs are set looser than SLOs on purpose — you want your internal target to trip an alert and get fixed well before you're anywhere near breaching what you've promised a customer, especially in banking where SLA breaches can carry regulatory weight, not just reputational.

- [ ] How would you choose SLIs for a payments API specifically?

> **IF ASKED, SAY THIS**
> Anchor on user-facing signals, not infrastructure ones. For a payments API that typically means: successful-transaction rate (2xx / total, carefully excluding legitimate client-side declines from counting as your failures), latency percentiles (p95/p99 request duration), and a payments-specific correctness signal — the

Anchor on user-facing signals, not infrastructure ones. For a payments API that typically means: **successful-transaction rate** (2xx / total, carefully excluding legitimate client-side declines from counting as your failures), **latency percentiles** (p95/p99 request duration), and a payments-specific correctness signal — the percentage of transactions that reach a terminal state (approved or explicitly declined) within a defined time window, rather than being left 'stuck' in a pending state.

Avoid picking CPU or memory utilization as an SLI — those are useful as saturation signals and root-cause inputs, but they're causes, not the thing the customer actually experiences.

- [ ] What is an error budget, and how do you use it operationally, not just theoretically?

> **IF ASKED, SAY THIS**
> Error budget = 1 minus your SLO. A 99.9% SLO over 30 days gives you roughly 43 minutes of allowed unreliability. The operational value is that it turns 'reliability vs. velocity' from a political argument into a shared, data-driven decision: if the budget is healthy, teams can ship faster and take on more risk; if it's

Error budget = 1 minus your SLO. A 99.9% SLO over 30 days gives you roughly 43 minutes of allowed unreliability. The operational value is that it turns 'reliability vs. velocity' from a political argument into a shared, data-driven decision: if the budget is healthy, teams can ship faster and take on more risk; if it's nearly exhausted, the team shifts focus to reliability work and slows down risky releases.

The key is agreeing on this policy *before* you need it — deciding in the moment, under pressure, defeats the purpose.

- [ ] How do you calculate burn rate, and what does a burn-rate alert look like?

> **IF ASKED, SAY THIS**
> Burn rate is how fast you're consuming error budget relative to the rate that would exactly exhaust it by the end of the window. A burn rate of 1 means you're on pace to use the whole budget exactly by window end. A common fast-burn threshold (from Google's SRE workbook) is 14.4× sustained over a 1-hour window — that c

Burn rate is how fast you're consuming error budget relative to the rate that would exactly exhaust it by the end of the window. A burn rate of 1 means you're on pace to use the whole budget exactly by window end. A common fast-burn threshold (from Google's SRE workbook) is **14.4×** sustained over a 1-hour window — that consumes about 2% of a 30-day budget in a single hour.

Good practice is **multi-window, multi-burn-rate** alerting: require both a short window (e.g. 5 minutes) *and* a longer window (e.g. 1 hour) to both be breaching before paging, which filters out short blips while still catching real, sustained degradation fast.

- [ ] What SLIs would you define for a capital markets trading platform, given how latency-sensitive it is?

> **IF ASKED, SAY THIS**
> Tail latency dominates here far more than average: p99/p99.9 order-acknowledgment latency , order-to-fill latency, and market-data feed staleness/lag. You'd also track correctness SLIs like order-state consistency and reconciliation match rate.

Tail latency dominates here far more than average: **p99/p99.9 order-acknowledgment latency**, order-to-fill latency, and market-data feed staleness/lag. You'd also track correctness SLIs like order-state consistency and reconciliation match rate.

The important nuance for trading specifically is that **jitter and predictability** often matter as much as raw speed — consistent, predictable latency is a fairness concern between market participants, not just a UX one, so a system that's usually fast but occasionally has a latency outlier can be a bigger problem than one that's uniformly a bit slower.

- [ ] What actually happens, process-wise, when an error budget is exhausted?

IF ASKED, SAY THIS
This needs to be a pre-agreed governance response, not an ad hoc argument: freeze non-essential feature releases, redirect the team's focus to the specific reliability issues that consumed the budget (usually visible from recent incidents/postmortems), require extra sign-off for any deploy during the freeze, and commun

This needs to be a pre-agreed governance response, not an ad hoc argument: freeze non-essential feature releases, redirect the team's focus to the specific reliability issues that consumed the budget (usually visible from recent incidents/postmortems), require extra sign-off for any deploy during the freeze, and communicate clearly to stakeholders why velocity is intentionally slowing and for how long.

It shouldn't mean freezing *everything* indiscriminately — a well-tested, high-value reliability fix can still ship; a risky new feature shouldn't.

- [ ] How do you handle SLOs across a multi-region, multi-tier payment system?

IF ASKED, SAY THIS
Define SLOs per critical user journey or tier ('authorize payment', 'capture payment', 'settlement batch') rather than one blanket SLO — different tiers have very different customer impact and latency profiles. For multi-region, track both per-region and global-aggregate SLOs; a single struggling region can hide inside

Define SLOs per critical user journey or tier ('authorize payment', 'capture payment', 'settlement batch') rather than one blanket SLO — different tiers have very different customer impact and latency profiles. For multi-region, track both **per-region** and **global-aggregate** SLOs; a single struggling region can hide inside an otherwise-healthy global number.

Also remember **composite SLOs**: your service's SLO mathematically can't exceed the combined reliability of the critical dependencies it calls — if you depend on two services each at 99.9%, your own ceiling is lower than 99.9% unless you've built in redundancy.

- [ ] Explain the four golden signals and how you'd map them to SLIs for a real service.

IF ASKED, SAY THIS
Latency (response time — split successful vs. failed request latency, since failures are often fast-but-wrong or slow-and-timing-out, and blending them hides both), Traffic (demand — RPS/TPS), Errors (rate of failed requests), and Saturation (how 'full' the system is — queue depth, connection pool usage, resource utili

Latency (response time — split successful vs. failed request latency, since failures are often fast-but-wrong or slow-and-timing-out, and blending them hides both), **Traffic** (demand — RPS/TPS), **Errors** (rate of failed requests), and **Saturation** (how 'full' the system is — queue depth, connection pool usage, resource utilization).

Saturation is the one people under-use: it's a *leading* indicator that degrades before errors actually appear, which makes it valuable for catching problems early rather than after customers are already affected. You wouldn't necessarily track all four as customer-facing SLIs everywhere — pick the subset that's actually meaningful for the specific service.

- [ ] How would you define an SLO for a batch settlement job differently from a real-time API?

IF ASKED, SAY THIS
Batch settlement SLOs are about completion-by-deadline and data completeness/accuracy , not per-request latency — e.g. '99% of settlement runs complete within their T+1 window by 06:00'. They're also measured per run (often daily) rather than continuously.

Batch settlement SLOs are about **completion-by-deadline** and **data completeness/accuracy**, not per-request latency — e.g. '99% of settlement runs complete within their T+1 window by 06:00'. They're also measured **per run** (often daily) rather than continuously.

A real-time API's SLO is about availability and per-request latency percentiles measured continuously. Conflating the two — e.g. trying to apply a rolling 30-day success-rate SLO to a once-a-day batch job — gives you a number that technically exists but doesn't actually tell anyone anything useful.

- [ ] Walk through your incident response process from detection to resolution.

IF ASKED, SAY THIS
Detect (alert or report) → triage and assemble the right responders based on severity → mitigate first — rollback, failover, circuit-break — even before you fully understand root cause → communicate status to stakeholders on a regular cadence → confirm recovery against real metrics, not just 'looks fine' → close out an

Detect (alert or report) → **triage** and assemble the right responders based on severity → **mitigate first** — rollback, failover, circuit-break — even before you fully understand root cause → **communicate** status to stakeholders on a regular cadence → **confirm recovery** against real metrics, not just 'looks fine' → close out and schedule a blameless postmortem.

The core principle: mitigate fast, understand fully afterward. Don't let a full root-cause investigation delay stopping customer impact.

- [ ] What is a blameless postmortem, and why does it actually matter?

IF ASKED, SAY THIS
A structured review after an incident focused on systemic and process causes — 'why did our systems and processes allow this to happen' — rather than individual blame. It matters because a blame culture makes people hide mistakes, under-report near-misses, and avoid the honest detail you actually need to find real root

A structured review after an incident focused on systemic and process causes — 'why did our systems and processes allow this to happen' — rather than individual blame. It matters because a blame culture makes people hide mistakes, under-report near-misses, and avoid the honest detail you actually need to find real root causes. A blameless culture surfaces the true chain of contributing factors — usually several, rarely one person's fault — so the organization can fix the systemic gaps instead of just reprimanding whoever was on call.

- [ ] How would you structure an incident severity matrix for a banking platform?

IF ASKED, SAY THIS

Typically tiered by customer and business impact: SEV1 — customers unable to make payments platform-wide, or a data-integrity/security issue — immediate page, exec notification, possibly regulatory notification depending on jurisdiction and scope. SEV2 — significant degradation affecting a subset of customers or one re

Typically tiered by customer and business impact: **SEV1** — customers unable to make payments platform-wide, or a data-integrity/security issue — immediate page, exec notification, possibly regulatory notification depending on jurisdiction and scope. **SEV2** — significant degradation affecting a subset of customers or one region, still urgent. **SEV3** — minor degradation with a workaround. **SEV4** — cosmetic, no customer impact.

In banking specifically, the severity criteria should also fold in regulatory reporting thresholds — certain outage durations or scopes trigger mandatory notification — so classification needs to be aligned with legal/compliance, not just engineering judgment.

- [ ] How would you handle a P1 affecting payment processing during market hours?

IF ASKED, SAY THIS

Declare immediately and page the full response team. Mitigate first — if a recent deploy correlates with the onset, roll back right away rather than debugging forward under pressure. Bring in a dedicated incident commander to coordinate communication so engineers can stay focused on the fix. Open a stakeholder channel

Declare immediately and page the full response team. Mitigate first — if a recent deploy correlates with the onset, roll back right away rather than debugging forward under pressure. Bring in a dedicated incident commander to coordinate communication so engineers can stay focused on the fix. Open a stakeholder channel with regular, timed updates — even 'still investigating' on schedule beats silence, especially given market-hours sensitivity for compliance and business stakeholders. Only move to full root-cause analysis once impact is mitigated.

- [ ] How do you make sure postmortem action items actually get done, instead of quietly dying in a doc?

IF ASKED, SAY THIS

Every action item gets a single named owner and a due date — never 'the team'. Track them in the same system as regular work (JIRA) with visibility to leadership. Review outstanding items in a recurring reliability/ops review. And treat repeatedly-slipping critical action items as a signal worth escalating on its own —

Every action item gets a single named owner and a due date — never 'the team'. Track them in the same system as regular work (JIRA) with visibility to leadership. Review outstanding items in a recurring reliability/ops review. And treat repeatedly-slipping *critical* action items as a signal worth escalating on its own — it's a real data point about whether reliability work is actually being prioritized against feature work, and sometimes that's the more important conversation.

- [ ] What's your approach to on-call rotations and reducing on-call fatigue?

IF ASKED, SAY THIS

Keep the rotation large enough that no one is on-call too often. Have clear escalation paths so nobody's stuck alone outside their expertise. Actively drive down alert noise — it's the single biggest fatigue driver. Track 'pages per week' as an actual health metric and review it. Recognize and, where possible, compensa

Keep the rotation large enough that no one is on-call too often. Have clear escalation paths so nobody's stuck alone outside their expertise. Actively drive down alert noise — it's the single biggest fatigue driver. Track 'pages per week' as an actual health metric and review it. Recognize and, where possible, compensate on-call load appropriately. And close the loop: every incident's learnings should feed back into automation that reduces recurring pages for the same root cause, so the rotation gets quieter over time instead of staying static.

- [ ] How do you communicate incident status to non-technical stakeholders or regulators?

IF ASKED, SAY THIS

Translate into business impact and plain language: what's affected, how many customers or transactions, current status (investigating / mitigating / resolved), and when the next update will come — no jargon. For regulator-facing communication in banking specifically, this usually follows a formal, pre-defined template

Translate into business impact and plain language: what's affected, how many customers or transactions, current status (investigating / mitigating / resolved), and when the next update will come — no jargon. For regulator-facing communication in banking specifically, this usually follows a formal, pre-defined template and timeline (many jurisdictions have explicit operational-resilience incident-reporting rules), so knowing your organization's actual notification thresholds and escalating accurately and on time to compliance/legal matters as much as the technical fix itself.

- [ ] What's the difference between a root cause and a contributing factor?

IF ASKED, SAY THIS

'The root cause' framing — implying one single cause — is usually misleading. Most real incidents have several contributing factors that combined to create impact: a code bug, plus a monitoring gap that delayed detection, plus a runbook that was out of date. Good RCA lists every contributing factor and prioritizes fixe

'The root cause' framing — implying one single cause — is usually misleading. Most real incidents have several contributing factors that combined to create impact: a code bug, *plus* a monitoring gap that delayed detection, *plus* a runbook that was out of date. Good RCA lists every contributing factor and prioritizes fixes by risk-reduction leverage, rather than fixating on just the triggering bug — because if you fix only that and leave the detection gap, the next *different* trigger causes the same prolonged impact.

- [ ] How do you balance 'fix it fast' against 'understand it fully' during an active incident?

IF ASKED, SAY THIS

Default to fast mitigation using known-safe levers — rollback, failover, a feature flag off — even without full understanding, since customer impact accrues every minute. Save deep root-cause investigation for the postmortem, where you have full data and no time pressure. The exception is when the 'fast fix' itself car

Default to fast mitigation using known-safe levers — rollback, failover, a feature flag off — even without full understanding, since customer impact accrues every minute. Save deep root-cause investigation for the postmortem, where you have full data and no time pressure. The exception is when the 'fast fix' itself carries real risk — an untested failover, say — where it's worth a brief validation before acting. This is a judgment call built through experience and sharpened by practicing it in game days, not a fixed rule you can apply mechanically.

- [ ] The JD lists AWS, Azure, or Google Cloud — how do you handle it if this round probes a cloud platform you haven't worked hands-on with?

IF ASKED, SAY THIS

Don't bluff, and don't apologize for it either. Say plainly which platform is your depth — in your case, Azure (AZ-104, SC-300, SC-100 puts you well past surface-level on identity, governance, and infrastructure) and AWS (6+ years hands-on) — then bridge: most "AWS, Azure, or GCP" JD lines are really testing whether yo

Don't bluff, and don't apologize for it either. Say plainly which platform is your depth — in your case, Azure (AZ-104, SC-300, SC-100 puts you well past surface-level on identity, governance, and infrastructure) and AWS (6+ years hands-on) — then bridge: most "AWS, Azure, or GCP" JD lines are really testing whether your reliability thinking transfers across providers, not GCP-specific trivia.

A concrete bridge line works well: "I haven't run GCP in production, but the primitives map closely — Cloud Load Balancing health checks are functionally what Azure Front Door or an AWS ALB target group does for me, and I'd expect the ramp-up to be mostly naming, not concepts." That reads as systems thinking, not a gap.

- [ ] How would multi-region high availability for a payments platform actually differ if you built it on Azure versus AWS — not just renamed, but genuinely different?

IF ASKED, SAY THIS

The concepts line up — region plus zonal redundancy, global traffic steering, replicated data — but the primitives and their failure characteristics don't map one-to-one. On Azure you'd typically steer traffic with Front Door or Traffic Manager (DNS-based, so failover speed is bounded by client TTL/caching behavior) an

The concepts line up — region plus zonal redundancy, global traffic steering, replicated data — but the primitives and their failure characteristics don't map one-to-one. On Azure you'd typically steer traffic with **Front Door** or **Traffic Manager** (DNS-based, so failover speed is bounded by client TTL/caching behavior) and replicate state with **Cosmos DB** multi-region writes or geo-replicated SQL. On AWS you'd use **Route 53** health-check-based failover or **Global Accelerator** (anycast IP, faster failover than DNS since it doesn't wait on TTL expiry) with **DynamoDB Global Tables** or **Aurora Global Database**.

The genuinely different part is the consistency model conversation: Cosmos DB lets you pick a consistency level per-container along a five-point spectrum, while DynamoDB Global Tables is last-writer-wins by default. For a payments platform, that's not a footnote — it changes how you reason about double-processing risk during a region failover, and it's a good signal in an interview that you think about data correctness, not just uptime.

- [ ] Compare Azure Monitor and Log Analytics to Amazon CloudWatch for SRE observability. What's genuinely different, not just branding?

IF ASKED, SAY THIS

Azure Monitor is more unified out of the box — metrics, logs, and Application Insights (APM/tracing) share one data platform and one query language, KQL , which is expressive for correlation-heavy queries (joining a trace to the log line to the metric spike in one query). CloudWatch historically grew as more separate s

Azure Monitor is more unified out of the box — metrics, logs, and Application Insights (APM/tracing) share one data platform and one query language, **KQL**, which is expressive for correlation-heavy queries (joining a trace to the log line to the metric spike in one query). CloudWatch historically grew as more separate services — Metrics, Logs Insights, and X-Ray each with their own query syntax — though AWS has been closing that gap with CloudWatch Application Signals and cross-service dashboards.

Practically, the interview-relevant difference is alerting semantics: Azure Monitor alert rules plus **Action Groups** versus CloudWatch **Alarms** plus **SNS** both get you paged, but Azure's dynamic thresholds (ML-based baselining) versus CloudWatch anomaly detection bands are tuned differently and worth knowing you'd re-validate rather than assume behave identically after a migration either direction.

- [ ] What does least-privilege access for SRE tooling actually look like on Azure versus AWS?

IF ASKED, SAY THIS

On Azure it's RBAC scoped as tightly as the resource hierarchy allows (management group → subscription → resource group → resource), layered with Entra ID Conditional Access and, for genuinely least-privilege standing access, Privileged Identity Management (PIM) so an SRE requests just-in-time elevation to a role — say

On Azure it's **RBAC** scoped as tightly as the resource hierarchy allows (management group → subscription → resource group → resource), layered with **Entra ID Conditional Access** and, for genuinely least-privilege standing access, **Privileged Identity Management (PIM)** so an SRE requests just-in-time elevation to a role — say, production write access — for a time-boxed window with an approval and an audit trail, rather than holding it permanently.

On AWS the equivalent pattern is IAM roles assumed via **STS** for short-lived credentials, cross-account roles for prod isolation from lower environments, and permission boundaries to cap what a role can be escalated to even by someone who can edit IAM policies. The interview-worthy point either way: standing production access is itself a reliability and audit risk in banking, so the goal is credentials that expire, not credentials that are merely "restricted."

- [ ] How do health checks and automated failover actually differ between AWS (Route 53 / ALB) and Azure (Front Door / Traffic Manager)?

IF ASKED, SAY THIS
Traffic Manager is pure DNS-based failover — it changes which IP a DNS query resolves to, so failover speed is at the mercy of client and resolver TTL caching, sometimes minutes. Front Door operates at Layer 7 with anycast, so it can redirect existing and new connections much faster and also do path-based routing and W

Traffic Manager is pure DNS-based failover — it changes which IP a DNS query resolves to, so failover speed is at the mercy of client and resolver TTL caching, sometimes minutes. **Front Door** operates at Layer 7 with anycast, so it can redirect existing and new connections much faster and also do path-based routing and WAF in the same layer. **Route 53** health-check failover is DNS-based like Traffic Manager with the same TTL caveat, while an **ALB**'s target-group health checks handle failover within a region at the connection layer, and **Global Accelerator** is AWS's anycast answer to Front Door's speed advantage.

The question worth anticipating a follow-up on: what health check depth do you actually trust? A TCP-level check that the process is listening is not the same signal as an application-level check hitting a real dependency (DB connectivity, downstream payment processor reachability) — shallow health checks are a classic way a "healthy" instance keeps taking traffic while it's actually failing every request.

- [ ] What does the reliability pillar of a cloud Well-Architected Framework actually give you day to day, beyond being a compliance checklist?

IF ASKED, SAY THIS
Treated as a checklist it's box-ticking. Treated properly, both AWS's and Azure's reliability pillars are really asking three operational questions that map straight onto SRE practice: can you detect a failure fast (your monitoring and SLI choices), can you recover automatically without a human paging at 3am (your fail

Treated as a checklist it's box-ticking. Treated properly, both AWS's and Azure's reliability pillars are really asking three operational questions that map straight onto SRE practice: can you detect a failure fast (your monitoring and SLI choices), can you recover automatically without a human paging at 3am (your failover, autoscaling, and self-healing design), and have you actually tested that recovery works rather than assumed it (this is where chaos engineering and game days stop being a separate initiative and become the evidence behind a Well-Architected review).

The senior-level answer connects it back to error budgets: a Well-Architected review that isn't tied to your SLOs is just an audit exercise; one that is becomes the mechanism for deciding what redundancy is actually worth building next, instead of building redundancy everywhere uniformly regardless of cost.

- [ ] How do you get a product or engineering team to treat reliability as a shared responsibility instead of 'the SRE team's job'?

IF ASKED, SAY THIS
Ownership follows pain and visibility. If SRE carries the pager alone, reliability stays SRE's problem by default regardless of what anyone says in a kickoff meeting. Practical levers: put the service-owning team on the same on-call rotation as SRE (even a lighter secondary rotation), make SLOs something the owning tea

Ownership follows pain and visibility. If SRE carries the pager alone, reliability stays SRE's problem by default regardless of what anyone says in a kickoff meeting. Practical levers: put the service-owning team on the same on-call rotation as SRE (even a lighter secondary rotation), make SLOs something the owning team signs off on and reviews monthly rather than a dashboard SRE maintains for them, and make sure the owning team's engineers — not just SRE — are in the room for postmortems on their own service.

Error budgets help make this concrete rather than aspirational: when a team is burning its budget, that's a shared, numeric signal to slow down feature velocity together — not an SRE veto handed down from outside the team.

- [ ] What does 'acting as a reliability and performance advocate throughout the SDLC' concretely look like — where do you actually insert yourself?

IF ASKED, SAY THIS
Earlier than most people expect. At design review, before code exists: asking what happens when a dependency times out, what the retry and idempotency story is, what the SLI for this feature will be. At definition-of-done: a story isn't done if it hasn't had its reliability impact assessed, its dashboard wired, and its

Earlier than most people expect. At design review, before code exists: asking what happens when a dependency times out, what the retry and idempotency story is, what the SLI for this feature will be. At definition-of-done: a story isn't done if it hasn't had its reliability impact assessed, its dashboard wired, and its runbook updated — the same bar as "has tests." Before go-live: a load and, where it matters, a chaos pass in a pre-prod environment as a gate, not an afterthought squeezed in the week of release. After go-live: a short retro on whether the reliability assumptions from design actually held once real traffic hit it.

The distinction that signals seniority here: advocate doesn't mean gatekeeper. You're most effective embedded early enough that reliability is a design input, not a launch-blocking objection raised too late to act on cheaply.

- [ ] Describe how you'd push back on a delivery timeline for reliability reasons when you don't have the authority to simply say no.

IF ASKED, SAY THIS
Lead with quantified risk in language the business already uses, not engineering language: not "this isn't resilient enough" but "this ships without a tested failover path, and the last time a similar gap surfaced in production it was a specific number of minutes of payment processing downtime during market hours." The

Lead with quantified risk in language the business already uses, not engineering language: not "this isn't resilient enough" but "this ships without a tested failover path, and the last time a similar gap surfaced in production it was a specific number of minutes of payment processing downtime during market hours." Then don't just object — propose the cheaper alternative to a flat delay: ship behind a feature flag to a small percentage first, add a manual kill switch, or descope the riskiest dependency to a fast-follow.

What actually changes minds is data plus an alternative path, escalated through the same channel the timeline decision was made in — not raised as a late objection in standup. If it still ships as-is after that, your job was to make the risk visible and give the decision-maker a real choice, not to have the final word.

- [ ] How do you prioritize reliability investment against feature velocity when you don't own the product roadmap?

IF ASKED, SAY THIS
Translate reliability debt into the same units the roadmap is already prioritized in — usually cost and revenue risk, not engineering elegance. "This will take two sprints" loses to a shiny feature every time; "our last two incidents in this area cost this many hours of engineering time and had this customer impact, an

Translate reliability debt into the same units the roadmap is already prioritized in — usually cost and revenue risk, not engineering elegance. "This will take two sprints" loses to a shiny feature every time; "our last two incidents in this area cost this many hours of engineering time and had this customer impact, and here's the recurring risk if we don't fix the root cause" competes on the roadmap's own terms.

Error budgets again do real work here: a budget that's healthy is implicit license to prioritize features; a budget that's repeatedly blown is the trigger, agreed in advance rather than negotiated in the moment, for reliability work to jump the queue. Getting that trigger agreed upfront — before you need it — is most of the battle.

- [ ] How do you build a genuine partnership with QA so performance and resilience testing become a shared practice instead of something bolted on right before release?

IF ASKED, SAY THIS
Split ownership deliberately rather than SRE owning it end to end: QA typically knows the test cases and business-critical user journeys better than SRE does, while SRE typically owns the tooling, environments, and production-realistic data. Co-authoring test scenarios — not handing QA a tool and walking away — is what

Split ownership deliberately rather than SRE owning it end to end: QA typically knows the test cases and business-critical user journeys better than SRE does, while SRE typically owns the tooling, environments, and production-realistic data. Co-authoring test scenarios — not handing QA a tool and walking away — is what actually embeds the practice rather than making it one more handoff.

Moving these tests earlier in the pipeline matters as much as who owns them: a lightweight performance smoke test on every merge catches regressions QA would otherwise only find in a dedicated pre-release cycle, by which point a fix is expensive. Sharing production dashboards with QA — not just pass/fail test results — also closes the loop, so they can see whether what they tested for actually matches what happens under real traffic.

- [ ] Design a highly available, resilient architecture for a real-time payments-processing platform, from ingestion to settlement. Walk through it.

IF ASKED, SAY THIS
Start at the edge: an API gateway layer handling authentication, rate limiting, and — critically for payments — idempotency keys, so a client's retry after a timeout can't double-charge. Behind that, decouple ingestion from processing with a durable queue (Kafka or equivalent) so a downstream slowdown becomes backpress

Start at the edge: an API gateway layer handling authentication, rate limiting, and — critically for payments — idempotency keys, so a client's retry after a timeout can't double-charge. Behind that, decouple ingestion from processing with a durable queue (Kafka or equivalent) so a downstream slowdown becomes backpressure, not dropped transactions. Processing services are stateless and horizontally scaled behind Kubernetes HPA, so they scale independently of the queue and the database.

Data tier: a primary store with multi-AZ synchronous replication for zero data loss within a region, async cross-region replication for disaster recovery, and read replicas to keep reporting/reconciliation load off the transactional path. Settlement runs as a separate, checkpointed async job — not inline with the customer-facing request — so a slow settlement never becomes a slow checkout. Circuit breakers wrap every call to a downstream card network or processor you don't control. And SLOs and dashboards get defined alongside the architecture, not bolted on after launch — if you can't say what 'healthy' means for each layer before it ships, you'll find out the hard way in production.

- [ ] How would you architect a trading order-management system to survive a full regional outage with minimal data loss?

IF ASKED, SAY THIS

First pin down what 'minimal' actually means here in RPO terms, because it's the crux of the design trade-off: fully synchronous cross-region replication gets you close to zero data loss but adds real latency to every order — latency that matters when milliseconds affect trade execution. Fully async replication protect

First pin down what 'minimal' actually means here in RPO terms, because it's the crux of the design trade-off: fully synchronous cross-region replication gets you close to zero data loss but adds real latency to every order — latency that matters when milliseconds affect trade execution. Fully async replication protects latency but means a regional failure can lose in-flight orders. Most capital markets designs land on synchronous replication within a region (multi-AZ) for the zero-loss guarantee that matters most, plus async cross-region for disaster recovery, accepting a small, bounded, explicitly-agreed RPO for the true regional-loss scenario rather than pretending it can be zero for free.

Order sequencing across regions is the other hard part — you need a single source of truth for ordering (rather than two regions both accepting writes and reconciling later) to avoid trade-sequencing disputes, which usually means active-passive with fast, tested failover rather than active-active for the order book itself, even if other components run active-active. And this only counts as validated if you've actually run a game day simulating full region loss, not just documented the design — a regulator asking for evidence of DR testing will ask for the test, not the diagram.

- [ ] If asked to design the SLO framework and error-budget policy for a brand-new capital markets platform with no existing reliability practice, where do you start?

IF ASKED, SAY THIS

Start from what traders and counterparties actually feel, not from what's easy to instrument: order-acceptance latency, order-rejection rate, and whether settlement completes correctly — not CPU or pod-restart counts. Set the first SLOs from observed historical baselines rather than guessing a target and hoping reality

Start from what traders and counterparties actually feel, not from what's easy to instrument: order-acceptance latency, order-rejection rate, and whether settlement completes correctly — not CPU or pod-restart counts. Set the first SLOs from observed historical baselines rather than guessing a target and hoping reality matches it; an SLO set before you have data is usually wrong in one direction or the other and erodes trust in the whole framework the first time it's obviously miscalibrated.

Roll out consequences in phases rather than all at once: dashboards and visibility first so people trust the numbers, then error-budget-informed prioritization conversations once there's a track record, and only then any formal policy tying budget exhaustion to release gates. A framework that shows up on day one with hard enforcement and no track record tends to get worked around rather than adopted.

- [ ] You suspect the problem is the network layer itself, not the application or the database — how do you systematically confirm or rule that out in a cloud environment?

IF ASKED, SAY THIS

Work outward from the request path rather than guessing. Rule DNS in or out first — stale records, slow resolver responses, or a client caching an old IP after a failover can all look exactly like an application hang. Then check the connection itself: TCP handshake failures, exhausted connection pools, or a security gr

Work outward from the request path rather than guessing. Rule DNS in or out first — stale records, slow resolver responses, or a client caching an old IP after a failover can all look exactly like an application hang. Then check the connection itself: TCP handshake failures, exhausted connection pools, or a security group / NSG rule silently dropping traffic all fail before your application code ever runs, so they never show up in application logs at all — that absence is itself a signal.

A classic trap worth naming explicitly: a load balancer's health check passing while real traffic still fails, because the health check hits a lightweight endpoint on a different path than actual traffic exercises — the target looks healthy and keeps receiving requests it can't actually serve. Concrete tools that separate network from application definitively rather than by guesswork: VPC Flow Logs or NSG flow logs to see whether traffic is even reaching the host, traceroute/mtr for routing and cross-AZ or cross-region latency, and distributed tracing to see exactly where in the request path time is actually being spent. Ruling the network in or out early saves hours of debugging application code for what turns out to be a five-minute DNS TTL or firewall rule fix.

- [ ] What's the Five Whys technique, and what's its actual failure mode when used badly?

IF ASKED, SAY THIS

You ask "why did this happen" repeatedly, each answer prompting the next "why," until you reach a root cause rather than stopping at the first, most-visible symptom. The common failure mode: it's easy to stop too early (three whys instead of five, landing on a comfortable-sounding but still-surface-

You ask "why did this happen" repeatedly, each answer prompting the next "why," until you reach a root cause rather than stopping at the first, most-visible symptom. The common failure mode: it's easy to stop too early (three whys instead of five, landing on a comfortable-sounding but still-surface-level cause) or to follow a single linear chain when the real cause is actually several contributing factors intersecting at once — a Five Whys done as a strict single chain can produce a confident-sounding but incomplete or even wrong root cause. Treating it as a starting structure for the investigation rather than a rigid algorithm you must complete exactly five times is the practical fix.

- [ ] What's the role of an incident commander, and why separate that from "the person fixing it"?

IF ASKED, SAY THIS

The incident commander owns coordination — keeping the timeline, deciding on the response strategy, managing communication to stakeholders, and deciding when to pull in more people — deliberately not the same person doing the hands-on technical fix. The reason to separate the roles: someone deep in

The incident commander owns coordination — keeping the timeline, deciding on the response strategy, managing communication to stakeholders, and deciding when to pull in more people — deliberately not the same person doing the hands-on technical fix. The reason to separate the roles: someone deep in debugging a hard technical problem is the worst-positioned person to also be tracking who's been notified, whether the customer-facing status page needs updating, or whether this needs to escalate — task-switching between "deep technical focus" and "coordination and

communication" degrades both. At smaller scale one person sometimes does both, but the moment an incident runs long or involves multiple teams, splitting the roles is what keeps the response from falling apart.

- [ ] What are the four golden signals, and why these four specifically?

IF ASKED, SAY THIS
Latency (how long requests take — split successful vs. failed, since a fast error is a very different problem than a slow success), Traffic (demand on the system — requests/sec, or a workload-appropriate equivalent), Errors (rate of failed requests), and Saturation (how "full" the system is — CPU, m

Latency (how long requests take — split successful vs. failed, since a fast error is a very different problem than a slow success), **Traffic** (demand on the system — requests/sec, or a workload-appropriate equivalent), **Errors** (rate of failed requests), and **Saturation** (how "full" the system is — CPU, memory, queue depth, connection pool usage).

They're the minimum set that answers "is this service healthy" without needing deep familiarity with its internals — useful because an on-call engineer at 3am is often not the person who wrote the service, and these four together catch almost every real-world failure mode: a service can be up (no errors) yet unusable (latency), or fine now but about to fall over (saturation).

- [ ] RED method vs. USE method — what's each for, and when do you reach for one over the other?

IF ASKED, SAY THIS
RED (Rate, Errors, Duration) is request-centric — built for services: how many requests, how many failed, how long did they take. It's my default for anything with a request/response shape — an API, a microservice.

RED (Rate, Errors, Duration) is request-centric — built for services: how many requests, how many failed, how long did they take. It's my default for anything with a request/response shape — an API, a microservice.

USE (Utilization, Saturation, Errors) is resource-centric — built for infrastructure: CPU, disk, memory, network. I reach for USE when RED says "the service is slow/erroring" and I need to find *which resource* is the bottleneck causing it — the two methods compose naturally: RED tells you something's wrong at the service level, USE helps you find where at the infrastructure level.

- [ ] Explain RTO and RPO, and how they drive your DR architecture decisions.

IF ASKED, SAY THIS
RTO (Recovery Time Objective) — how long can the service be down before it's unacceptable. RPO (Recovery Point Objective) — how much data can you afford to lose, expressed as time ("up to the last 5 minutes").

RTO (Recovery Time Objective) — how long can the service be down before it's unacceptable. **RPO** (Recovery Point Objective) — how much data can you afford to lose, expressed as time ("up to the last 5 minutes").

They directly size the architecture: a tight RTO (minutes) needs warm or hot standby — infrastructure already running, ready to take traffic — not a cold rebuild-from-backup path. A tight RPO needs continuous/near-real-time replication (streaming DB replication, not nightly snapshots). Both cost money and complexity in direct proportion to how tight they are, so the real conversation with stakeholders is usually "what RTO/RPO can we actually afford," not "what's technically possible."

- [ ] Design an on-call rotation and escalation policy for a team of 5–6 engineers supporting a 24/7 service.

IF ASKED, SAY THIS
Weekly primary + secondary rotation — primary responds first, secondary is the immediate escalation if primary doesn't acknowledge within a set window (5–10 minutes) or explicitly needs help. That gives roughly one week on primary and one week on secondary per person per month at that team size, whi

Weekly primary + secondary rotation — primary responds first, secondary is the immediate escalation if primary doesn't acknowledge within a set window (5–10 minutes) or explicitly needs help. That gives roughly one week on primary and one week on secondary per person per month at that team size, which is sustainable without burning people out.

Escalation policy: unacknowledged page → secondary after N minutes → team lead/manager after another N minutes if still unresolved, with severity-based paging (SEV1 pages immediately, SEV3 can wait for business hours). I'd also cap consecutive on-call weeks, rotate the specific pairing so it's not always the same two people, and treat a rotation with a much higher-than-average page count as a signal to fix the underlying noise, not just something to endure.

- [ ] What is MTTA, and why do interviewers ask about it separately from MTTR?

IF ASKED, SAY THIS
MTTA is pure responsiveness — alert-to-acknowledgment, before any diagnosis starts. It's tracked separately from MTTR because a bad MTTA is an alerting/routing problem, while a bad MTTR is a diagnosis/tooling problem — different root causes, different fixes.

MTTA is the average time between an alert firing and someone actually acknowledging it and starting work — pure responsiveness, before any diagnosis or fix happens. It's tracked separately from MTTR because the two have different root causes and different fixes: a bad MTTA is an alerting or on-call routing problem, while a bad MTTR is a diagnosis or tooling problem. Naming that distinction up front signals you understand incident metrics as a system, not just as vocabulary.

- Chain: **MTTD** (detect) → **MTTA** (acknowledge) → **MTTR** (restore)
- MTTA problems = alerting & routing. MTTR problems = diagnosis & fix tooling.

- Keep the two causally separate in your answer — interviewers listen for this.

☐ How would you actually bring MTTA down on a team? ▲

IF ASKED, SAY THIS
Cut alert noise/dedupe first — the fastest lever. Then fix routing (service-ownership maps, not a shared queue), add tiered auto-escalation for unacknowledged P1s, and make sure paging actually reaches people (push/SMS, not email).

Start with noise: engineers stop reacting fast to alerts that cry wolf, so tune thresholds and dedupe before anything else. Then fix routing — service-ownership maps so the right on-call gets paged the first time, not a shared queue. Add tiered escalation so an unacknowledged P1 auto-escalates after a few minutes, and make sure paging actually reaches people through mobile push or SMS, not just email. Cite a real number if you have one — cutting alert noise by around 40% through fatigue remediation is exactly the kind of concrete result that lands better than the theory.

- Cut noise/dedupe first — usually the fastest lever
- Ownership-based routing beats one shared alert queue
- Tiered auto-escalation for unacknowledged P1s
- **Your ammunition:** ~40% alert-noise reduction through fatigue remediation

☐ How do you ensure SLA compliance? ▲

IF ASKED, SAY THIS
Keep the three terms straight: SLA is the external contract, SLO is your tighter internal target, SLI is what you actually measure. Compliance is proactive — dashboards tracking SLI against SLO, an error-budget policy, escalating on trend toward breach rather than after it happens.

Keep the three terms straight: SLA is the external commitment, such as 99.9% uptime, SLO is your internal target, often tighter than the SLA as a buffer, and SLI is the actual measured indicator. Compliance is mostly proactive — dashboards tracking SLI against SLO in real time, an error-budget policy so you know how much risk you can spend on releases before endangering the SLA, and escalating when you're trending toward breach rather than waiting until it happens. If you've owned SLOs for a production service, say so directly with the number — for example, maintaining a 99.9% uptime target.

- **SLA** = external contract, **SLO** = internal target, **SLI** = the measurement
- Error budget = 'how much risk can we spend on this release'
- Proactive trend-watching beats reactive breach response
- **Your ammunition:** owned a 99.9% uptime SLO target

☐ What does 'accuracy' mean in incident handling, beyond just speed? ▲

IF ASKED, SAY THIS
Accuracy means getting it right the first time — correct severity (not everything is a P1), routing to the right team without a bounce, and diagnosing the real root cause instead of a fast guess that lets it recur. Fast-but-wrong usually costs more than slightly-slower-but-right.

Accuracy means getting it right the first time: correct severity classification, since not everything is a P1 and inflating severity burns trust and wastes responders, routing to the right team without a bounce, and diagnosing the actual root cause instead of a fast guess that lets the same incident recur next week. Fast-but-wrong usually costs more than slightly-slower-but-right, because it creates rework and erodes confidence in the process.

- Right severity, right team, right root cause — three separate ways to be 'accurate'
- Fast-but-wrong is more expensive than it looks: rework + trust

☐ What's the difference between Impact, Urgency, Severity, and Priority? ▲

IF ASKED, SAY THIS
Impact = scope (how many users/functions affected). Urgency = time-sensitivity. Severity = technical criticality. Priority = the business call, typically Impact × Urgency — high/high is P1, high-impact-with-workaround is P2, low-impact-but-urgent is P3, low/low is P4.

Impact is scope — how many users or business functions are affected. Urgency is time-sensitivity — how fast it needs fixing given any workaround. Severity is usually technical criticality, how badly something is actually broken. Priority is the business call on queue order, typically Impact × Urgency: high impact plus high urgency is a P1, high impact with a workaround is P2, low impact but time-critical is P3, low impact and low urgency is P4. Keeping that matrix consistent matters, otherwise priority becomes whoever escalates loudest, which is its own operational risk.

- **Impact** = scope · **Urgency** = time pressure · **Severity** = technical criticality · **Priority** = the business decision
- P1 = high/high · P2 = high impact + workaround exists · P3 = low impact + urgent · P4 = low/low
- Consistency prevents 'loudest voice wins' prioritization

☐ When do you actually declare a Major Incident, a P1? ▲

IF ASKED, SAY THIS
Typical triggers: multiple business units affected, revenue/regulatory impact, no viable workaround, an SLA breach imminent, or clear executive visibility already. Declaring formally kicks off the major-incident process — bridge call, incident commander, a fixed comms cadence starting

immediately.

Typical triggers: multiple business units or customer segments affected, revenue or regulatory impact, no viable workaround exists, an SLA breach is imminent, or there's already clear executive visibility. Once any of those thresholds is crossed, you declare formally, which kicks off the major-incident process — a bridge call opens, an incident commander is assigned, and a fixed communication cadence starts immediately, for example updates every 15 to 30 minutes, instead of waiting for someone to ask.

- Multiple objective triggers, not just 'it feels bad'
- Declaring formally kicks off IC role + bridge + comms cadence
- Set the update cadence proactively — don't wait to be asked

- [ ] What ITIL processes do you work in day to day, and how do they differ?

IF ASKED, SAY THIS
Incident Management restores service fast — a workaround is a fine outcome here. Problem Management finds and permanently fixes the root cause so it doesn't recur. Change Management (ITIL v4: Change Enablement) governs how changes get introduced — standard/normal/emergency tiers. As a support engineer you live mostly in the first two.

Incident Management is about restoring service as fast as possible — a workaround is a perfectly good outcome here. Problem Management is about finding and permanently fixing the underlying root cause so the incident doesn't recur, usually worked after the fact via root cause analysis, often linked to a pattern of repeat incidents. Change Management, which ITIL v4 calls Change Enablement though most shops still say 'Change Management,' governs how changes get introduced in a controlled way — standard pre-approved low-risk changes skip heavy review, normal changes go through a CAB, and emergency changes get an expedited path because the risk of not fixing something outweighs the process overhead. As a support engineer you live mostly in the first two.

- **Incident** = restore fast, workaround is fine
- **Problem** = root cause, prevents recurrence
- **Change** = controlled introduction, standard/normal/emergency tiers
- Know both the v3 'Change Management' and v4 'Change Enablement' naming

- [ ] What's a Known Error Database, and how do you actually use it?

IF ASKED, SAY THIS
A Known Error Database stores the confirmed root cause and workaround for problems that are understood but not yet permanently fixed. L1/L2 search it first against a recurring symptom instead of re-diagnosing from zero — arguably the single biggest lever for cutting MTTR on a support desk.

It stores the root cause and workaround for problems that are understood but not yet permanently fixed. In practice, L1 and L2 search it first against a recurring symptom so they can apply the documented workaround immediately instead of re-diagnosing from zero — arguably the single biggest lever for cutting MTTR on a support desk, because it turns 'investigate from scratch' into 'look it up.' Every time a new root cause is confirmed, it should get written back into the KEDB, or the next engineer pays the same diagnosis cost again.

- KEDB = confirmed root cause + workaround for known problems
- Searching it first is a major MTTR lever
- Feeding new findings back in matters as much as using it

- [ ] What's a CAB, and when do you actually need one?

IF ASKED, SAY THIS
A Change Advisory Board reviews and approves higher-risk normal changes before production, weighing risk, impact, and rollback plan. Standard changes (pre-approved, low-risk) skip it by design; emergency changes get an expedited path. Routing everything through the heaviest process by default is itself a maturity red flag.

A Change Advisory Board reviews and approves higher-risk normal changes before they hit production, weighing risk, business impact, and the rollback plan. Standard changes — pre-approved, low-risk, done before — skip CAB entirely, which is the point of pre-approval. Emergency changes go through an expedited approval path, because the risk of leaving something broken outweighs the process overhead of a full board review. Knowing which tier a change falls into, and not routing everything through the heaviest process by default, is itself a sign of process maturity.

- CAB = normal, higher-risk changes only
- Standard changes are pre-approved — skip it by design
- Emergency = expedited, not skipped entirely
- Over-using CAB for everything is itself a maturity red flag

- [ ] How do you handle a frustrated stakeholder during an ongoing outage?

IF ASKED, SAY THIS
Acknowledge the impact honestly, give a factual status without promising a fix time you're not confident in, and commit to a specific next-update time — then actually hit it. Reliability of communication de-escalates people more than technical detail they can't parse mid-crisis anyway.

Acknowledge the impact honestly, give a factual status without promising a fix time you're not confident in, and commit to a specific next-update time — then actually hit it. Reliability of communication de-escalates people far more than technical detail they can't parse in the middle of a crisis anyway.

If you don't have an answer yet, 'I don't know yet, next update at X' is a completely legitimate thing to say, and it's far better than guessing just to make someone feel better in the moment.

- Facts over false reassurance
- Commit to a specific next-update time and hit it
- 'I don't know yet, here's when I'll know more' is a valid, professional answer

Chaos Engineering

- [ ] What is chaos engineering, and why does it specifically matter in banking/payments?

  IF ASKED, SAY THIS
  Chaos engineering is the discipline of deliberately injecting controlled failures into a system to proactively discover weaknesses before they cause a real outage, instead of finding out during an actual incident. You form a hypothesis about how the system should behave under a specific failure, then run the experiment

  Chaos engineering is the discipline of deliberately injecting **controlled** failures into a system to proactively discover weaknesses before they cause a real outage, instead of finding out during an actual incident. You form a hypothesis about how the system should behave under a specific failure, then run the experiment and see if reality matches.

  In banking/payments this matters more than most domains because outages have direct financial and regulatory consequences — you can't afford to discover that your failover doesn't actually work by having it fail during a real regional outage. It validates that redundancy and degraded-mode behavior work under real conditions, not just in an architecture diagram.

- [ ] How is chaos engineering different from traditional DR/resilience testing?

  IF ASKED, SAY THIS
  Traditional DR testing is usually a scheduled, scripted, full failover exercise (e.g. an annual DR drill failing over an entire data center) — disruptive and infrequent. Chaos engineering is typically smaller, targeted, more frequent experiments (kill one pod, add latency to one dependency) run continuously as part of

  Traditional DR testing is usually a scheduled, scripted, **full** failover exercise (e.g. an annual DR drill failing over an entire data center) — disruptive and infrequent. Chaos engineering is typically smaller, targeted, more frequent experiments (kill one pod, add latency to one dependency) run continuously as part of normal operations, often even against live production with a tightly controlled blast radius.

  They're complementary rather than competing: cheap, frequent chaos experiments surface and fix small weaknesses continuously, which raises your confidence going into the big, expensive, periodic DR test — and often directly informs what that DR test should even cover.

- [ ] Walk through the steady-state hypothesis method for designing a chaos experiment.

  IF ASKED, SAY THIS
  1) Define a measurable steady state using real business/system metrics — e.g. checkout success rate, p95 latency. 2) Form a hypothesis that steady state will hold even when a specific failure is introduced. 3) Inject the real-world failure (kill an instance, add latency, saturate a dependency). 4) Actively try to dispr

  1) Define a measurable **steady state** using real business/system metrics — e.g. checkout success rate, p95 latency. 2) Form a hypothesis that steady state will hold even when a specific failure is introduced. 3) Inject the real-world failure (kill an instance, add latency, saturate a dependency). 4) Actively try to **disprove** the hypothesis by watching for a deviation in steady state. 5) If it holds, your confidence in that resilience claim goes up; if it breaks, you've found a real weakness — cheaply, on your terms, instead of during an actual incident.

- [ ] What's 'blast radius' and how do you control it in a regulated environment like banking?

  IF ASKED, SAY THIS
  Blast radius is how much of the real system/traffic an experiment can affect if something goes wrong. Controls: start in non-prod, then a small percentage of production traffic via feature flags/traffic shifting; define hard, automated abort conditions tied to real business SLIs (e.g. auto-abort if transaction failure

  Blast radius is how much of the real system/traffic an experiment can affect if something goes wrong. Controls: start in non-prod, then a small percentage of production traffic via feature flags/traffic shifting; define hard, automated **abort conditions** tied to real business SLIs (e.g. auto-abort if transaction failure rate exceeds a threshold); run during low-traffic windows initially; get risk/compliance sign-off for production experiments given regulatory scrutiny; and always have an immediate manual kill-switch to halt the injected failure.

- [ ] Name the common chaos engineering tools and how they differ.

  IF ASKED, SAY THIS
  Chaos Monkey / Chaos Toolkit — open-source, pioneered by Netflix, instance/VM termination focus. Gremlin — commercial SaaS, broad attack library (CPU/memory/disk/network/state), strong safety controls and scheduling, common in enterprises including finance. LitmusChaos — CNCF, Kubernetes-native, experiments defined as

  Chaos Monkey / Chaos Toolkit — open-source, pioneered by Netflix, instance/VM termination focus. **Gremlin** — commercial SaaS, broad attack library (CPU/memory/disk/network/state), strong safety controls and scheduling, common in enterprises including finance. **LitmusChaos** — CNCF, Kubernetes-native, experiments defined as Kubernetes CRDs. **Chaos Mesh** — another CNCF Kubernetes-native option. **AWS Fault Injection Simulator (FIS)** — native AWS service for faults against EC2/ECS/EKS/RDS and simulated AZ failures.

  Choice usually comes down to: Kubernetes-first shop → Litmus/Chaos Mesh; want a managed tool with enterprise approval workflows → Gremlin; AWS-native → FIS.

- [ ] Give an example of a LitmusChaos pod-delete experiment definition.

IF ASKED, SAY THIS
Litmus experiments are defined as a ChaosEngine custom resource pointing at a target app and a named experiment:

Litmus experiments are defined as a `ChaosEngine` custom resource pointing at a target app and a named experiment:

```
apiVersion: litmuschaos.io/v1alpha1
kind: ChaosEngine
metadata:
  name: payment-svc-chaos
spec:
  appinfo:
    appns: payments
    applabel: app=payment-authorization
    appkind: deployment
  chaosServiceAccount: litmus-admin
  experiments:
    - name: pod-delete
      spec:
        components:
          env:
            - name: TOTAL_CHAOS_DURATION
              value: "60"
            - name: CHAOS_INTERVAL
              value: "10"
            - name: FORCE
              value: "false"
```

This deletes pods matching the label every 10 seconds for 60 seconds, using a graceful (non-forced) delete, while you watch whether readiness probes and traffic re-routing keep the steady-state SLI intact.

- [ ] What's a 'game day' and how would you plan one for a payments platform?

IF ASKED, SAY THIS
A planned, time-boxed exercise where a team deliberately simulates an incident and responders treat it exactly like a real one — real paging, real runbooks, real comms — to validate both the technical resilience and the human response process together. For payments, a game day might simulate 'primary payment gateway re

A planned, time-boxed exercise where a team deliberately simulates an incident and responders treat it exactly like a real one — real paging, real runbooks, real comms — to validate both the technical resilience *and* the human response process together. For payments, a game day might simulate 'primary payment gateway region is down' and check: does failover actually trigger, does the right person get paged, are the dashboards accurate, is the stakeholder communication clear — all without any real customer impact because it's scoped and controlled in advance.

- [ ] How do you get compliance and stakeholder buy-in to run chaos experiments in a highly regulated bank?

IF ASKED, SAY THIS
Build trust incrementally: run the first experiments in staging with full observability, then share concrete 'here's the weakness we found and fixed' results — wins are the best argument. Frame it in language compliance already understands: this is proactive risk management, and increasingly it's expected or even manda

Build trust incrementally: run the first experiments in staging with full observability, then share concrete 'here's the weakness we found and fixed' results — wins are the best argument. Frame it in language compliance already understands: this *is* proactive risk management, and increasingly it's expected or even mandated by operational-resilience regulation. Put every production experiment through the same change-review process as any other change, with a documented plan, rollback steps, and abort criteria. Only graduate to production experiments after several successful lower-risk ones have demonstrated the process is genuinely safe and controlled.

- [ ] What kinds of failures would you specifically inject for a capital markets trading platform?

IF ASKED, SAY THIS
Network: added latency/jitter on the market-data feed, packet loss between the matching engine and gateway. Resource: CPU/memory pressure on order-matching nodes. Dependency: simulated unavailability or slow response from a reference-data or pre-trade risk-check service. Infrastructure: AZ or primary-site failure, to v

Network: added latency/jitter on the market-data feed, packet loss between the matching engine and gateway. **Resource:** CPU/memory pressure on order-matching nodes. **Dependency:** simulated unavailability or slow response from a reference-data or pre-trade risk-check service. **Infrastructure:** AZ or primary-site failure, to validate failover to the DR site or a secondary matching engine. **State:** a stale or inconsistent order-book replica.

Each experiment should map to a specific claim you actually want to validate — e.g. 'if the risk-check service is slow, do we correctly queue or reject orders, rather than let them through unchecked or crash?'

How do you measure whether a chaos experiment succeeded or failed?

IF ASKED, SAY THIS
Success is judged against the steady-state hypothesis: did the real business/system SLIs stay within acceptable bounds during and after the injected failure? Track whether automated failover/alerting fired as designed, the actual MTTD for the injected fault, whether customer-facing metrics degraded beyond an agreed thr

Success is judged against the steady-state hypothesis: did the real business/system SLIs stay within acceptable bounds during and after the injected failure? Track whether automated failover/alerting fired as designed, the actual MTTD for the injected fault, whether customer-facing metrics degraded beyond an agreed threshold, and whether any manual intervention was needed that should have been automatic.

A 'failed' experiment — where steady state broke — isn't a bad outcome. That's the valuable finding. The real failure is not fixing the gap it exposed.

Is 'resilience testing' the same thing as 'chaos engineering'?

IF ASKED, SAY THIS
Related but not identical. Resilience testing is the broader umbrella — any testing that validates continued function under adverse conditions, including DR drills, manual failover tests, and combined load/failure tests. Chaos engineering is a specific methodology within that umbrella: controlled, hypothesis-driven, of

Related but not identical. **Resilience testing** is the broader umbrella — any testing that validates continued function under adverse conditions, including DR drills, manual failover tests, and combined load/failure tests. **Chaos engineering** is a specific methodology within that umbrella: controlled, hypothesis-driven, often continuous or automated experimentation, frequently run directly against production.

You can do resilience testing without doing chaos engineering (a manual, scripted annual DR test), but chaos engineering is always a form of resilience testing.

How would you chaos-test a payment gateway's dependency on a third-party card network you don't control?

IF ASKED, SAY THIS
You can't inject faults into their systems directly, so you simulate the effect at your integration boundary — a service-virtualization layer or a proxy like Toxiproxy injecting latency, timeouts, and error responses that mimic their documented failure modes.

You can't inject faults into their systems directly, so you simulate the effect at your integration boundary — a service-virtualization layer or a proxy like Toxiproxy injecting latency, timeouts, and error responses that mimic their documented failure modes.

What you're validating: does your system time out and retry correctly **with idempotency** (so a retry can never cause a duplicate charge), does it fail over to a backup processor if you have one, and does it degrade gracefully — queue for later, show a clear 'payment could not be processed' message — rather than hanging indefinitely.

What safety mechanisms do you build into every chaos experiment?

IF ASKED, SAY THIS
Automated abort conditions tied to real-time SLI monitoring (auto-halt if error rate or latency crosses a threshold), a manual kill-switch, hard time-boxing (a maximum experiment duration), starting with a small traffic percentage or a canary environment, running during business hours with the full team present rather

Automated abort conditions tied to real-time SLI monitoring (auto-halt if error rate or latency crosses a threshold), a manual kill-switch, hard time-boxing (a maximum experiment duration), starting with a small traffic percentage or a canary environment, running during business hours with the full team present rather than right before a weekend, and a rollback plan that's itself been tested in advance — a recovery path you've never exercised often doesn't actually work when you need it.

Walk through a realistic chaos engineering scenario and what it taught you.

IF ASKED, SAY THIS
Example: running a pod-delete experiment against a payment-authorization service that was assumed to be fully stateless and horizontally scaled. The infra-level resilience held — the load balancer re-routed traffic correctly. But the experiment surfaced that an in-memory idempotency cache wasn't shared across pods, so

Example: running a pod-delete experiment against a payment-authorization service that was assumed to be fully stateless and horizontally scaled. The infra-level resilience held — the load balancer re-routed traffic correctly. But the experiment surfaced that an in-memory idempotency cache wasn't shared across pods, so killing a pod mid-request created a small window where a client's automatic retry could get processed twice.

The lesson: the experiment validated the infrastructure assumption but exposed an **application-level** statefulness bug that no architecture diagram would have shown. The fix was moving idempotency keys to a shared store (Redis) with a TTL, closing the gap before it ever hit production traffic for real.

How do you integrate chaos experiments into CI/CD as continuous verification, rather than a one-off exercise?

IF ASKED, SAY THIS

Add automated, low-risk chaos experiments as a pipeline stage against staging after every deploy — e.g. a 'chaos smoke test' that kills a pod and verifies readiness probes and traffic re-routing before promoting to production. More mature setups run scheduled chaos experiments continuously in production (not just on de

Add automated, low-risk chaos experiments as a pipeline stage against staging after every deploy — e.g. a 'chaos smoke test' that kills a pod and verifies readiness probes and traffic re-routing before promoting to production. More mature setups run scheduled chaos experiments continuously in production (not just on deploy) via a scheduler — Gremlin scenarios or Litmus workflows on a cron — feeding a 'resilience score' dashboard.

The mindset shift this represents: resilience becomes a continuously tested, regressable property of the system, like test coverage, instead of a one-time architecture review that goes stale the moment the system changes.

- [ ] Walk through the structure of an AWS FIS experiment template.

IF ASKED, SAY THIS
Five core pieces, defined as JSON/YAML: targets (which AWS resources — selected by ID, tag, or filter, with a selection mode like a specific count or percentage), actions (what fault to inject — run in a specified order or simultaneously), stopConditions (CloudWatch alarms that auto-halt the experim

Five core pieces, defined as JSON/YAML: `targets` (which AWS resources — selected by ID, tag, or filter, with a selection mode like a specific count or percentage), `actions` (what fault to inject — run in a specified order or simultaneously), `stopConditions` (CloudWatch alarms that auto-halt the experiment), an experiment `roleArn` (the IAM role FIS assumes to actually perform the actions on your behalf), and optional experiment report configuration/tags. The template itself doesn't run anything — you create it once, then start an experiment from it, and FIS executes actions against the resolved targets while continuously watching the stop conditions.

- [ ] How does FIS actually inject a fault into an EC2 instance for CPU stress or network latency, given FIS itself doesn't run on the instance?

IF ASKED, SAY THIS
For instance-level faults like CPU stress or network latency, FIS doesn't reach into the instance directly — it uses AWS Systems Manager as the execution mechanism: the target instance needs the SSM Agent installed and an IAM instance profile permitting SSM to run commands, and the FIS action is eff

For instance-level faults like CPU stress or network latency, FIS doesn't reach into the instance directly — it uses AWS Systems Manager as the execution mechanism: the target instance needs the SSM Agent installed and an IAM instance profile permitting SSM to run commands, and the FIS action is effectively an `aws:ssm:send-command` action running an SSM document (e.g., a script using `stress-ng` for CPU/IO load, or a network-impairment tool for latency/packet loss) on the target. For faults that are pure AWS-API-level actions instead — stopping/terminating/rebooting an instance, modifying a security group — FIS calls the relevant AWS API directly and doesn't need SSM at all. Knowing which category a given fault falls into (API-level vs. needs-an-agent-inside-the-instance) is a good signal of real hands-on FIS experience.

- [ ] What are stop conditions in FIS, and why are they the single most important safety feature?

IF ASKED, SAY THIS
A stop condition is a CloudWatch alarm wired into the experiment template — if that alarm enters ALARM state while the experiment is running, FIS automatically halts it immediately, before it runs to completion. This is what makes it safe to run chaos experiments against production at all: without i

A stop condition is a CloudWatch alarm wired into the experiment template — if that alarm enters ALARM state while the experiment is running, FIS automatically halts it immediately, before it runs to completion. This is what makes it safe to run chaos experiments against production at all: without it, you're trusting a human to notice things are going badly wrong and manually cancel in time, which doesn't scale and isn't reliable at 2am. The practical setup: point the stop condition at the same steady-state-hypothesis metric you defined upfront (error rate, latency) so the experiment's own safety net is measuring the exact thing you're trying to protect, not a proxy metric that might lag the real damage.

- [ ] How would you use FIS to test EKS node failure specifically?

IF ASKED, SAY THIS
FIS has a purpose-built EKS action, aws:eks:terminate-nodegroup-instances , targeting a nodegroup resource and taking an instanceTerminationPercentage parameter — so instead of a blunt "kill one node" test, you can precisely control how much of the nodegroup gets terminated simultaneously (e.g., 30%

FIS has a purpose-built EKS action, `aws:eks:terminate-nodegroup-instances`, targeting a nodegroup resource and taking an `instanceTerminationPercentage` parameter — so instead of a blunt "kill one node" test, you can precisely control how much of the nodegroup gets terminated simultaneously (e.g., 30% for a partial, realistic degradation test vs. testing a full nodegroup wipeout). The point of running this isn't just "does Kubernetes reschedule the pods" — it's validating the whole chain: does the cluster autoscaler/Karpenter actually provision replacement capacity fast enough, do pod disruption budgets and readiness probes prevent a thundering-herd reschedule from causing its own outage, and does the service actually stay within its steady-state hypothesis (latency/error rate) throughout, not just eventually recover.

- [ ] What's the FIS Scenario Library, and what's an example of a pre-built AWS scenario?

IF ASKED, SAY THIS
AWS ships a library of pre-built, AWS-owned experiment templates for common, realistic failure patterns, so you don't have to hand-build every experiment from scratch — it covers both severe disruptions and partial/"gray failure" scenarios that are actually harder to detect than a clean outage. Exam

AWS ships a library of pre-built, AWS-owned experiment templates for common, realistic failure patterns, so you don't have to hand-build every experiment from scratch — it covers both severe disruptions and partial/"gray failure" scenarios that are actually harder to detect than a clean outage. Examples: "AZ: Power Interruption" simulates a full power loss in one Availability Zone, "Cross-Region: Connectivity" blocks network traffic between

regions, and gray-failure scenarios like "AZ: Application Slowdown" or "Cross-AZ: Traffic Slowdown" test whether your monitoring can even detect a degraded-but-not-down AZ — which is often the harder and more realistic case, since a clean AZ failure is exactly what multi-AZ architecture is designed to handle cleanly, while a slow, half-broken AZ is what actually causes confusing incidents.

- [ ] What IAM permissions does FIS need, and what's the least-privilege way to set that up?

IF ASKED, SAY THIS
FIS needs an experiment IAM role (referenced as roleArn in the template) that it assumes to perform the actions on your behalf — scoped to exactly the actions and resources that specific template uses, not a broad "FIS can do anything" role. For SSM-based actions, that role also needs SSM send-comma

FIS needs an experiment IAM role (referenced as `roleArn` in the template) that it assumes to perform the actions on your behalf — scoped to exactly the actions and resources that specific template uses, not a broad "FIS can do anything" role. For SSM-based actions, that role also needs SSM send-command permissions, and separately the target instances need their own IAM instance profile allowing SSM to execute on them — two different roles serving two different parts of the chain. The least-privilege discipline that matters most in an interview answer: one experiment role per experiment template (or per team/blast-radius scope), not one shared role reused across every chaos experiment in the account, since that shared role becoming compromised or misconfigured would then be able to disrupt anything any experiment ever touched.

- [ ] How would you integrate FIS experiments into a CI/CD pipeline for continuous resilience testing?

IF ASKED, SAY THIS
FIS is explicitly designed to support this — the same experiment template you'd run manually can be triggered via API/CLI as a pipeline stage, typically post-deploy against a staging environment (or a canary slice of production) as an automated resilience regression check, not just a manual periodic

FIS is explicitly designed to support this — the same experiment template you'd run manually can be triggered via API/CLI as a pipeline stage, typically post-deploy against a staging environment (or a canary slice of production) as an automated resilience regression check, not just a manual periodic exercise. The pattern: deploy → run a defined, low-blast-radius FIS experiment (e.g., terminate one instance in the new deployment, inject brief latency to a dependency) → assert the service stayed within its steady-state hypothesis throughout (query the same CloudWatch metrics the stop condition watches) → fail the pipeline if it didn't. This turns "did we regress our resilience" into the same kind of automated gate as a functional test suite, instead of something only checked in an occasional manual game day.

- [ ] How is AWS FIS different from third-party chaos tools like Gremlin or Chaos Mesh, and when would you still want one of those alongside FIS?

IF ASKED, SAY THIS
FIS is fully managed (no chaos-tooling infrastructure to run yourself), deeply integrated with AWS-native stop conditions/IAM/CloudWatch, and has purpose-built actions for AWS resources (EC2, ECS, EKS nodegroups, RDS) — the natural default if you're AWS-native end to end. Chaos Mesh is Kubernetes-na

FIS is fully managed (no chaos-tooling infrastructure to run yourself), deeply integrated with AWS-native stop conditions/IAM/CloudWatch, and has purpose-built actions for AWS resources (EC2, ECS, EKS nodegroups, RDS) — the natural default if you're AWS-native end to end. Chaos Mesh is Kubernetes-native and gives finer-grained, K8s-object-level fault injection (pod kill, network partition between specific pods, I/O faults inside a container) that's more precise than FIS's coarser EKS nodegroup-level actions when the fault you care about is really about Kubernetes scheduling/networking behavior specifically rather than the underlying AWS infrastructure. Gremlin adds a curated experiment library and cross-cloud/on-prem consistency if your environment isn't AWS-only. Realistic answer for an AWS-centric, FIS-focused role like this one: FIS as the primary tool since it's what's explicitly required, with awareness that Chaos Mesh is the natural complement specifically for pod/network-level Kubernetes fault injection FIS doesn't cover as precisely.

- [ ] Define "self-healing" at three different layers of the stack, from process to platform.

IF ASKED, SAY THIS
Process level : systemd Restart= , container restartPolicy — the simplest, most local form. Platform level : Kubernetes reconciliation, liveness/readiness probes, HPA/VPA, cloud auto-healing node groups replacing unhealthy instances. Application level : circuit breakers, retries with exponential bac

Process level: systemd `Restart=`, container `restartPolicy` — the simplest, most local form. **Platform level**: Kubernetes reconciliation, liveness/readiness probes, HPA/VPA, cloud auto-healing node groups replacing unhealthy instances. **Application level**: circuit breakers, retries with exponential backoff, graceful degradation/fallback responses when a dependency is down. A mature environment layers all three — platform self-healing catches what process-level restarts can't, and application-level resilience catches what neither infrastructure layer can see (a dependency that's technically "up" but returning bad data slowly).

- [ ] What's the difference between closed-loop auto-remediation and human-in-the-loop remediation, and how do you decide which failure modes get which?

IF ASKED, SAY THIS
Closed-loop: detect → diagnose → fix, fully automatically, for a well-understood failure mode with a bounded, reversible blast radius (restart a pod stuck in a known-bad state). Human-in-the-loop: the system detects and maybe diagnoses, but a person decides whether to act — reserved for anything nov

Closed-loop: detect → diagnose → fix, fully automatically, for a well-understood failure mode with a bounded, reversible blast radius (restart a pod stuck in a known-bad state). Human-in-the-loop: the system detects and maybe diagnoses, but a person decides whether to act — reserved for

anything novel or high-blast-radius (a database failover). The boundary between the two shouldn't be static — as confidence in a given automated remediation grows through track record, it can graduate from human-approved to fully closed-loop.

- [ ] Give an example of a runbook you'd automate end-to-end versus one you'd keep as documentation only, and why.

> **IF ASKED, SAY THIS**
> Automate : "pod in namespace X stuck in CrashLoopBackOff for >10 min with no recent deploy → restart it, log the action, alert if it recurs within an hour" — deterministic, low blast radius, and trivially reversible. Keep manual : "primary database is unreachable, consider failover to replica" — hig

Automate: "pod in namespace X stuck in CrashLoopBackOff for >10 min with no recent deploy → restart it, log the action, alert if it recurs within an hour" — deterministic, low blast radius, and trivially reversible. **Keep manual**: "primary database is unreachable, consider failover to replica" — high blast radius, potential data-consistency risk, and it needs a human judgment call about the specific situation. The dividing line is reversibility and how well-understood the failure mode is, not how technically complex the steps are.

- [ ] How would you design multi-AZ or multi-region HA so a single zone failure doesn't become a customer-facing incident?

> **IF ASKED, SAY THIS**
> Active-active where the data layer allows it, health-check-driven DNS or load-balancer failover so traffic routes away from a bad zone automatically, and a deliberate audit for cross-AZ single points of failure — a shared NAT gateway, a single RDS instance with no standby, a control-plane dependency

Active-active where the data layer allows it, health-check-driven DNS or load-balancer failover so traffic routes away from a bad zone automatically, and a deliberate audit for cross-AZ single points of failure — a shared NAT gateway, a single RDS instance with no standby, a control-plane dependency that isn't itself multi-AZ. None of this counts until you actually test the failover on a schedule; "it's configured for HA" and "it fails over correctly" are different claims.

- [ ] What's a bulkhead pattern, and how would you apply it to protect a critical service from a noisy, less-critical dependency?

> **IF ASKED, SAY THIS**
> Isolate the resource pools — connection pools, thread pools, rate limits — used per dependency, so a slow or failing dependency exhausts only its own allocation instead of resources shared with calls to healthy dependencies. Concretely: give each downstream API its own connection pool with its own t

Isolate the resource pools — connection pools, thread pools, rate limits — used per dependency, so a slow or failing dependency exhausts only its own allocation instead of resources shared with calls to healthy dependencies. Concretely: give each downstream API its own connection pool with its own timeout and its own circuit breaker, rather than one shared pool where a slow third-party integration can starve connections needed for a critical internal call.

- [ ] How does Kubernetes' HPA/VPA/Cluster Autoscaler stack function as a form of self-healing under load, and where does it fall short?

> **IF ASKED, SAY THIS**
> HPA scales pod replica count on a metric (CPU, custom metric); VPA adjusts a pod's resource requests/limits over time; Cluster Autoscaler adds/removes nodes when pods can't be scheduled or nodes sit underused. Together they absorb load spikes without a human paged to scale manually. Where it falls s

HPA scales pod replica count on a metric (CPU, custom metric); VPA adjusts a pod's resource requests/limits over time; Cluster Autoscaler adds/removes nodes when pods can't be scheduled or nodes sit underused. Together they absorb load spikes without a human paged to scale manually. Where it falls short: scaling reacts to a metric with lag (a sudden spike can outrun scale-up time), and none of it helps if the bottleneck is a downstream stateful dependency — a database — that doesn't scale horizontally as easily as stateless pods do.

- [ ] Explain the core principles of chaos engineering.

> **IF ASKED, SAY THIS**
> Define a steady-state hypothesis (what "normal" looks like, measurably), inject a real-world failure (kill a pod, add network latency, saturate a resource), and see whether the steady state holds. Critically: control the blast radius (start in staging, or a small % of production traffic), have an au

Define a steady-state hypothesis (what "normal" looks like, measurably), inject a real-world failure (kill a pod, add network latency, saturate a resource), and see whether the steady state holds. Critically: control the blast radius (start in staging, or a small % of production traffic), have an automatic abort condition, and treat every finding as an input to a fix, not just a demo. The goal isn't breaking things for its own sake — it's finding weaknesses before an uncontrolled failure finds them for you.

- [ ] Compare chaos tooling options briefly — Litmus, Gremlin, Chaos Mesh, AWS FIS.

> **IF ASKED, SAY THIS**
> Given TCG Digital's multi-cloud framing (AWS/Azure/GCP), I'd lean toward a Kubernetes-native option like LitmusChaos as the guild default, since it isn't tied to one cloud provider.

- **LitmusChaos** — CNCF, Kubernetes-native, free/OSS, good default for a Kubernetes-first guild standard
- **Chaos Mesh** — also Kubernetes-native CNCF project, strong for network-level fault injection

- **Gremlin** — commercial, polished UI, broader than just Kubernetes, easier for less chaos-experienced pods to adopt but a licensing cost across every pod
- **AWS FIS** — native to AWS, integrates cleanly if you're AWS-heavy, less useful for Azure/GCP pods

Given TCG Digital's multi-cloud framing (AWS/Azure/GCP), I'd lean toward a Kubernetes-native option like LitmusChaos as the guild default, since it isn't tied to one cloud provider.

- [ ] How would you run a guild-wide "game day" across multiple pods? ▲

IF ASKED, SAY THIS
Pick one realistic, well-scoped failure scenario (a shared dependency going down, a region failover), run it against a willing pilot pod first in staging with a clear abort button, document what actually happened versus what the runbook assumed, then fix the gaps before offering it as a repeatable g

Pick one realistic, well-scoped failure scenario (a shared dependency going down, a region failover), run it against a willing pilot pod first in staging with a clear abort button, document what actually happened versus what the runbook assumed, then fix the gaps before offering it as a repeatable guild service other pods can request. I would not mandate every pod run a game day on a fixed guild-wide schedule on day one — that's a fast way to get resentful, half-hearted participation instead of genuine learning.

- [ ] How do you standardize DR testing cadence across pods with a guild recommendation? ▲

IF ASKED, SAY THIS
Every production service needs a documented RTO/RPO and a DR test on a fixed cadence (I'd suggest at minimum annually, more often for anything client-critical) — the guild owns the standard and the reporting, pods own the actual test execution since they know their service's failover mechanics. I'd

Every production service needs a documented RTO/RPO and a DR test on a fixed cadence (I'd suggest at minimum annually, more often for anything client-critical) — the guild owns the standard and the reporting, pods own the actual test execution since they know their service's failover mechanics. I'd track "time since last successful DR test" as a guild health metric alongside SLO attainment, since an untested DR plan is really just an assumption.

- [ ] Active-active vs. active-passive multi-region — how do you help a pod choose? ▲

IF ASKED, SAY THIS
Active-passive is simpler and cheaper — one region serves traffic, the other stands ready — but costs you failover time and often means the standby region's readiness is untested until the day you actually need it. Active-active serves traffic from both regions simultaneously, giving faster failover

Active-passive is simpler and cheaper — one region serves traffic, the other stands ready — but costs you failover time and often means the standby region's readiness is untested until the day you actually need it. Active-active serves traffic from both regions simultaneously, giving faster failover and validating both regions continuously, at the cost of real complexity around data consistency and conflict resolution. For most pods I'd default to active-passive unless their SLO genuinely can't tolerate failover time — active-active is a meaningful engineering investment that should be justified by the SLO, not adopted by default.

Performance Testing

- [ ] What's the actual difference between load, stress, endurance/soak, and spike testing?

> **IF ASKED, SAY THIS**
> Load testing validates behavior at expected/target load — does it meet SLOs at normal or expected-peak traffic. Stress testing pushes beyond expected capacity to find the actual breaking point and, just as important, how it fails — gracefully or catastrophically. Endurance/soak testing runs a sustained moderate load fo

Load testing validates behavior at expected/target load — does it meet SLOs at normal or expected-peak traffic. **Stress testing** pushes beyond expected capacity to find the actual breaking point and, just as important, *how* it fails — gracefully or catastrophically. **Endurance/soak testing** runs a sustained moderate load for an extended period (hours or days) to surface slow-building issues like memory leaks. **Spike testing** applies a sudden, sharp jump in load — like a market-open surge — to validate autoscaling and queueing react fast enough.

- [ ] Compare JMeter, Gatling, k6, and Locust as performance-testing tools.

> **IF ASKED, SAY THIS**
> JMeter — mature, GUI-based, huge protocol support, Java-based, can get resource-heavy at very high concurrency. Gatling — Scala-based, code-as-config, strong reporting, efficient at high concurrency. k6 — modern, JavaScript-based scripting, great CI/CD integration and developer experience. Locust — Python-based, test s

JMeter — mature, GUI-based, huge protocol support, Java-based, can get resource-heavy at very high concurrency. **Gatling** — Scala-based, code-as-config, strong reporting, efficient at high concurrency. **k6** — modern, JavaScript-based scripting, great CI/CD integration and developer experience. **Locust** — Python-based, test scenarios written as Python code, distributed/scalable via worker nodes.

For performance testing embedded directly in CI/CD, k6 or Locust tend to fit best because the test definitions are code, live in git, and are easy to diff and review like any other code change.

- [ ] How would you design a load test for a payments API to simulate a major seasonal traffic surge?

> **IF ASKED, SAY THIS**
> Baseline current traffic from real production telemetry first. Define target load as a deliberate multiplier of historical peak (e.g. 3–5× last year's festive peak, with extra headroom on top). Ramp gradually rather than stepping instantly, so you can watch autoscaling react in real time rather than just measuring a fi

Baseline current traffic from real production telemetry first. Define target load as a deliberate multiplier of historical peak (e.g. 3–5× last year's festive peak, with extra headroom on top). **Ramp gradually** rather than stepping instantly, so you can watch autoscaling react in real time rather than just measuring a fixed steady load. Use a realistic transaction mix — not just one endpoint, but authorize, capture, refund, and status-check in roughly production proportion. Run against a production-like environment with realistic data volumes, for long enough to see true steady-state behavior, and judge results against your actual SLIs (p95/p99 latency, error rate) rather than just 'did it stay up'.

- [ ] What metrics do you actually watch during a load test?

> **IF ASKED, SAY THIS**
> Latency percentiles (p50/p95/p99, often p99.9 for payments), throughput achieved vs. offered, error rate broken down by type (timeouts vs. 5xx vs. application-level declines — these mean very different things), resource utilization across every tier (CPU, memory, DB connections, thread-pool saturation), queue depths, a

Latency percentiles (p50/p95/p99, often p99.9 for payments), throughput achieved vs. offered, error rate broken down by type (timeouts vs. 5xx vs. application-level declines — these mean very different things), resource utilization across every tier (CPU, memory, DB connections, thread-pool saturation), queue depths, and the latency of downstream dependencies — so you can tell whether a bottleneck is in your service or in something you call.

- [ ] How do you actually narrow down a performance bottleneck across app, middleware, DB, and infra layers, rather than just guessing?

> **IF ASKED, SAY THIS**
> Start from distributed traces — they show exactly where time was spent across each hop for a specific slow request, not just that the API overall was slow. Then drill per tier: app (thread-pool exhaustion, GC pauses, profile with async-profiler or py-spy), middleware (queue backlog, gateway rate limiting), DB (slow que

Start from distributed traces — they show exactly where time was spent across each hop for a specific slow request, not just that the API overall was slow. Then drill per tier: **app** (thread-pool exhaustion, GC pauses, profile with async-profiler or py-spy), **middleware** (queue backlog, gateway rate limiting), **DB** (slow queries via `EXPLAIN`, lock contention, connection-pool exhaustion), **infra** (CPU throttling, network saturation, noisy-neighbor effects on shared infra).

The actual skill is going from 'the API is slow' to '80% of that latency is one query's lock wait' using traces plus tier-specific metrics together — not intuition.

- [ ] How do you turn load test results into an actual capacity plan?

> **IF ASKED, SAY THIS**

Take the load test's SLO-breach point (or hard breaking point), apply a safety margin — commonly planning for 1.5–2× expected peak — and translate that into concrete resource numbers: pod replica counts, DB instance sizing, connection-pool limits. Factor in year-over-year growth trends and seasonal multipliers on top o

Take the load test's SLO-breach point (or hard breaking point), apply a safety margin — commonly planning for 1.5–2× expected peak — and translate that into concrete resource numbers: pod replica counts, DB instance sizing, connection-pool limits. Factor in year-over-year growth trends and seasonal multipliers on top of current peak. Treat this as a living model revisited quarterly or before any major event, not a one-time exercise you run once and file away.

- [ ] How would you embed performance testing into CI/CD — shift-left performance testing?

IF ASKED, SAY THIS
Run lightweight performance smoke tests automatically on every merge to staging — a short k6 or Gatling run with fixed, hard pass/fail thresholds on p95 latency and error rate, gating the pipeline (fail the build if p95 regresses more than, say, 15% vs. baseline). Reserve full-scale load/stress tests for a scheduled or

Run lightweight performance smoke tests automatically on every merge to staging — a short k6 or Gatling run with fixed, hard pass/fail thresholds on p95 latency and error rate, gating the pipeline (fail the build if p95 regresses more than, say, 15% vs. baseline). Reserve full-scale load/stress tests for a scheduled or pre-release pipeline stage since they're slower and more expensive to run on every commit. Store historical results to trend performance over time, so you catch a slow creeping regression across ten small PRs, not just a sudden one from a single bad commit.

- [ ] What kinds of issues does endurance/soak testing surface that a short load test never would?

IF ASKED, SAY THIS
Memory leaks (heap growing steadily without release), connection pool or file-descriptor exhaustion, log files filling disk over hours, unbounded cache growth, and token/credential-expiry bugs that only manifest after the token would have refreshed several times. These are all issues where the failure mode only appears

Memory leaks (heap growing steadily without release), connection pool or file-descriptor exhaustion, log files filling disk over hours, unbounded cache growth, and token/credential-expiry bugs that only manifest after the token would have refreshed several times. These are all issues where the failure mode only appears after sustained runtime — a 10-minute load test would show a perfectly healthy system right up until it isn't.

- [ ] How is performance testing for a strict-latency trading system different from typical API load testing?

IF ASKED, SAY THIS
It usually needs specialized tooling beyond a normal HTTP load tester — custom harnesses measuring at microsecond precision, sometimes using hardware timestamps or kernel-bypass networking, run against dedicated test hardware matching production exactly (shared/virtualized infra adds noisy jitter that would otherwise d

It usually needs specialized tooling beyond a normal HTTP load tester — custom harnesses measuring at microsecond precision, sometimes using hardware timestamps or kernel-bypass networking, run against dedicated test hardware matching production exactly (shared/virtualized infra adds noisy jitter that would otherwise dominate the measurement). You also care about the **full tail distribution** — p99.9, p99.99 — since in trading, a single slow outlier order can matter enormously for market fairness, not just user experience.

- [ ] Sketch a basic k6 script for load-testing a payment endpoint.

IF ASKED, SAY THIS
The thresholds block is what makes this CI/CD-friendly — k6 exits non-zero automatically if p95 latency or error rate breaches the defined bar, so it can gate a pipeline without a human reading a report.

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '2m', target: 100 },
    { duration: '5m', target: 100 },
    { duration: '2m', target: 0 },
  ],
  thresholds: {
    http_req_duration: ['p(95)<300'],
    http_req_failed: ['rate<0.01'],
  },
};

export default function () {
  const payload = JSON.stringify({ amount: 500, currency: 'INR', merchantId: 'm_123' });
  const res = http.post('https://api.example.com/v1/payments', payload, {
    headers: { 'Content-Type': 'application/json' },
  });
```

```
  check(res, { 'status is 200': (r) => r.status === 200 });
  sleep(1);
}
```

The `thresholds` block is what makes this CI/CD-friendly — k6 exits non-zero automatically if p95 latency or error rate breaches the defined bar, so it can gate a pipeline without a human reading a report.

- [ ] A load test shows good average latency but a bad p99. What does that actually tell you?

IF ASKED, SAY THIS
It points to a tail-latency problem affecting a minority of requests significantly, hidden by the average. Common causes: GC pauses, occasional lock contention, a subset of requests hitting a cold cache or a slower DB replica/shard, connection-pool queueing under bursty concurrency, or one under-performing instance in

It points to a tail-latency problem affecting a minority of requests significantly, hidden by the average. Common causes: GC pauses, occasional lock contention, a subset of requests hitting a cold cache or a slower DB replica/shard, connection-pool queueing under bursty concurrency, or one under-performing instance in the fleet that the fleet-wide average masks entirely.

For SLOs, p99/p99.9 usually matters more than the average, because it represents your worst-treated customers — in payments that could literally mean a customer whose transaction is stuck or delayed while everyone else's went through fine.

- [ ] How do you approach capacity validation ahead of a major launch or a regulatory reporting deadline?

IF ASKED, SAY THIS
Run a full-scale load test against a production-like environment with realistic data volume well ahead of the date, leaving real time for remediation. Include downstream dependencies in the test scope, not just your service in isolation. Confirm autoscaling policies actually trigger correctly under the test's specific

Run a full-scale load test against a production-like environment with realistic data volume well ahead of the date, leaving real time for remediation. Include downstream dependencies in the test scope, not just your service in isolation. Confirm autoscaling policies actually trigger correctly under the test's specific ramp profile. If the traffic pattern for that day is predictable, dry-run it directly. And have a rollback or circuit-breaker plan ready in case real traffic ends up exceeding even what you tested for.

- [ ] What's the relationship between performance testing and error budgets?

IF ASKED, SAY THIS
A performance regression can silently burn error budget without a single outright error, if your SLO includes a latency component — e.g. '95% of requests under 300ms'. A regression to 350ms median doesn't throw an error, but it breaches that SLO. Performance testing is the preventive gate that stops a budget-burning re

A performance regression can silently burn error budget without a single outright error, if your SLO includes a latency component — e.g. '95% of requests under 300ms'. A regression to 350ms median doesn't throw an error, but it breaches that SLO. Performance testing is the **preventive gate** that stops a budget-burning regression before it reaches production; the SLO/error budget is the after-the-fact production signal that performance testing exists to protect in the first place.

Kubernetes

What is Kubernetes and what problem does it solve?

IF ASKED, SAY THIS
Kubernetes is an open-source container orchestration platform that automates deployment, scaling, networking, and lifecycle management of containerized workloads. You describe the desired state declaratively, and Kubernetes' control loops continuously reconcile actual state to match it — handling self-healing, scaling, rollouts, and service discovery automatically.

- Containers solved packaging — but running hundreds or thousands of them across many machines by hand is impossible. You need something to schedule them, restart them when they crash, scale them, network them together, and keep them at the state you declared.
- Kubernetes is a container orchestration platform that does exactly that. You declare the desired state (e.g. '5 replicas of payment-service running this image'), and Kubernetes continuously works to make reality match that declaration.
- Core value: self-healing, declarative configuration, automated rollout/rollback, service discovery, load balancing, horizontal scaling, and bin-packing workloads onto nodes efficiently.

Likely follow-up: How is Kubernetes different from Docker? (Docker packages/runs a container; Kubernetes orchestrates many containers across many hosts.) · What does 'declarative' mean vs 'imperative'?

Explain declarative vs imperative management in Kubernetes.

IF ASKED, SAY THIS
Imperative commands tell Kubernetes exactly what to do step by step and are useful for quick fixes. Declarative management describes the desired end state in manifests applied with `kubectl apply`, and Kubernetes reconciles toward it. Production environments use declarative + GitOps because it's idempotent, version-controlled, and auditable.

- Imperative = you tell Kubernetes the exact steps to perform now: `kubectl run`, `kubectl create`, `kubectl scale`. Good for quick one-offs and debugging.
- Declarative = you describe the end state in YAML and apply it: `kubectl apply -f`. Kubernetes figures out the steps. This is GitOps-friendly, reviewable, repeatable, and idempotent.
- In production you almost always use declarative manifests stored in Git, because the desired state becomes auditable and reproducible.

```
# Imperative
kubectl create deployment web --image=nginx

# Declarative
kubectl apply -f deployment.yaml
```

Likely follow-up: Why is `kubectl apply` idempotent but `kubectl create` is not?

What is a Namespace and when would you use one?

IF ASKED, SAY THIS
A Namespace logically partitions a cluster so resources can be isolated and scoped per team or environment. It enables per-tenant resource quotas, RBAC boundaries, and avoids naming collisions. Cluster-scoped objects like Nodes and PersistentVolumes don't live in namespaces.

- A Namespace is a virtual cluster inside a physical cluster — a way to scope and isolate groups of resources (Pods, Services, etc.).
- Used for: separating teams/environments (dev/staging), applying ResourceQuotas and LimitRanges per tenant, scoping RBAC, and avoiding name collisions.
- Not everything is namespaced — Nodes, PersistentVolumes, and ClusterRoles are cluster-scoped. DNS for a Service is `..svc.cluster.local`.

```
kubectl get pods --namespace=payments
kubectl get ns
```

Likely follow-up: Name three resources that are NOT namespaced. · How does cross-namespace service DNS resolution work?

What are the main components of a Kubernetes cluster?

IF ASKED, SAY THIS
A cluster has a control plane and worker nodes. The control plane (API server, etcd, scheduler, controller manager) makes global decisions and stores state. Worker nodes run workloads via the kubelet, kube-proxy, and a container runtime. Add-ons like CoreDNS, a CNI plugin, and metrics-server provide DNS, networking, and metrics.

- Control Plane: kube-apiserver (front door), etcd (state store), kube-scheduler (placement), kube-controller-manager (reconciliation loops), cloud-controller-manager (cloud integration).
- Worker Nodes: kubelet (node agent), kube-proxy (service networking), container runtime (containerd/CRI-O).

- Add-ons: CoreDNS, CNI plugin, metrics-server, ingress controller.

Likely follow-up: Which component is the only one that talks to etcd?

- [ ] What is the Kubernetes Control Plane?

IF ASKED, SAY THIS
The control plane maintains the desired state of the cluster. It handles API requests, scheduling, controller reconciliation, and stores all cluster state in etcd. Worker nodes execute workloads while the control plane makes all management decisions.

- Think of Kubernetes as a company: workers don't decide what work to do — management decides. The control plane is the management team.
- It contains the API Server, Scheduler, Controller Manager, and etcd. Every decision starts here.
- Without the control plane there are no deployments, no scheduling, no scaling, and no enforcement of desired state — worker nodes keep running existing Pods but can't be managed.

Likely follow-up: Can Pods keep running if the control plane is down? (Yes — existing Pods run, but you can't manage the cluster.)

- [ ] Explain the API Server and why it's central.

IF ASKED, SAY THIS
The API Server is the central management component and the only component that reads/writes etcd. Every request flows through it, where it authenticates, authorizes, validates, runs admission control, and persists desired state. Because it's stateless it's run in multiple replicas for HA.

- The API Server is the front door of Kubernetes. Every request — from users, kubelets, the scheduler, and controllers — goes through it. Even the scheduler never talks to etcd directly.
- Responsibilities: accept requests, authenticate, authorize (RBAC), run admission control, validate objects, persist to etcd, and return status.
- It's stateless, so you run multiple replicas behind a load balancer for high availability.

```
kubectl get pods   # → API Server → etcd → API Server → kubectl
```

Likely follow-up: Why is the API Server highly available? (If it's down, you can't deploy, scale, or update — though running apps continue.)

- [ ] What is etcd and why is it critical?

IF ASKED, SAY THIS
etcd is a distributed, strongly consistent key-value store that is Kubernetes' single source of truth, holding all cluster state. It uses Raft consensus and runs as an odd-numbered quorum (3 or 5). Only the API Server interacts with it directly, so it must be backed up regularly.

- etcd is a distributed, strongly consistent key-value store. It holds the entire cluster state: Pods, Deployments, Secrets, ConfigMaps, Nodes, RBAC, leases, certificates — everything.
- It uses the Raft consensus algorithm, so you run an odd number of members (3 or 5) to tolerate failures while maintaining quorum.
- Only the API Server talks to etcd directly. Back it up regularly — a corrupted/lost etcd with no backup means rebuilding the cluster.

```
ETCDCTL_API=3 etcdctl snapshot save backup.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=ca.crt --cert=server.crt --key=server.key
```

Likely follow-up: Can the Scheduler read etcd directly? (No — only the API Server.) · Why an odd number of etcd members?

- [ ] Explain the Scheduler and how it decides where a Pod runs.

IF ASKED, SAY THIS
The Scheduler watches for unscheduled Pods and selects the best node in two phases: filtering removes nodes that can't satisfy the Pod's requirements, and scoring ranks the rest by fit. It accounts for resource requests, affinity, taints/tolerations, and topology, then binds the Pod to the chosen node.

- The Scheduler watches for Pods with no assigned node (`nodeName` empty) and decides WHERE — not HOW — they run.
- Two phases: Filtering (predicates) eliminates nodes that can't fit (insufficient CPU/memory, taints without tolerations, node selectors). Scoring (priorities) ranks remaining nodes (least-loaded, affinity, topology spread). Highest score wins, then it binds the Pod.
- Inputs: resource requests, node affinity/anti-affinity, pod affinity, taints/tolerations, topology spread constraints, and priority classes.

Likely follow-up: What happens if no node passes filtering? (Pod stays Pending with a scheduling event.)

- [ ] What is the Controller Manager and what is a control loop?

IF ASKED, SAY THIS
The Controller Manager runs the control loops that continuously compare actual cluster state against the desired state in etcd and take corrective action when they differ. This reconciliation model is what gives Kubernetes its self-healing behavior.

- The Controller Manager runs many controllers (Deployment, ReplicaSet, Node, Job, CronJob, Namespace, ServiceAccount...). Each runs a control loop: observe actual state → compare with desired state in etcd → act to close the gap.
- Example: desired replicas = 5, actual = 4 → the ReplicaSet controller creates 1 Pod.
- This continuous reconciliation is what makes Kubernetes self-healing.

Likely follow-up: How do controllers detect drift? (They watch the API Server for changes and re-reconcile.)

- [ ] Explain Authentication, Authorization, and Admission Control.

IF ASKED, SAY THIS
Every API request goes through authentication (verifies identity via certs/tokens/OIDC), authorization (checks permissions, usually RBAC), then admission control (mutating webhooks can modify the request; validating webhooks/policies can reject it). Only after all three does the object get persisted to etcd.

- Every API request passes through three stages in order:
- 1) Authentication — 'Who are you?' via client certs, bearer tokens, OIDC, or service account tokens.
- 2) Authorization — 'What are you allowed to do?' usually via RBAC (Role/RoleBinding, ClusterRole/ClusterRoleBinding).
- 3) Admission Control — 'Should this request be mutated or rejected?' Mutating webhooks change objects (e.g. inject sidecars); validating webhooks/policies enforce rules (Pod Security, ResourceQuota, LimitRange).

Likely follow-up: Difference between mutating and validating admission webhooks? · What runs first — mutating or validating? (Mutating, then validating.)

- [ ] What is the Kubelet and what does it do?

IF ASKED, SAY THIS
The kubelet is the per-node agent that watches the API Server for Pods assigned to its node and makes sure their containers run and stay healthy. It talks to the container runtime through the CRI, runs probes, mounts volumes, and reports node and Pod status back to the control plane.

- The kubelet runs on every worker node. It registers the node, watches the API Server for Pods bound to its node, and ensures those Pods' containers are running and healthy.
- It drives the container runtime via CRI to pull images and start/stop containers, runs liveness/readiness/startup probes, mounts volumes, and reports node + Pod status back to the API Server.
- It does NOT manage Pods that weren't scheduled to its node, and it doesn't create Pods on its own (except static Pods from a local manifest path).

Likely follow-up: What is a static Pod? (A Pod the kubelet runs directly from a local manifest, e.g. control plane components in kubeadm.)

- [ ] What is kube-proxy? (And what it is NOT.)

IF ASKED, SAY THIS
kube-proxy runs on every node and implements Service networking by programming iptables or IPVS rules that route and load-balance traffic from a Service's virtual IP to its healthy backend Pods. It manages network rules rather than proxying HTTP itself, and can be replaced by eBPF dataplanes like Cilium.

- kube-proxy runs on each node and implements the Service abstraction at the network level. It does NOT proxy application/HTTP traffic in the usual mode.
- It watches Services and EndpointSlices and programs iptables or IPVS rules so that traffic to a Service's ClusterIP is load-balanced across healthy backend Pods.
- Modern clusters often use IPVS (better at scale) or eBPF-based dataplanes (e.g. Cilium) that can replace kube-proxy entirely.

Likely follow-up: iptables vs IPVS mode — why does IPVS scale better? · How does kube-proxy differ from an Ingress controller?

- [ ] Explain the Container Runtime and CRI.

IF ASKED, SAY THIS
The container runtime downloads images and runs containers; the kubelet drives it via the CRI gRPC interface. Since 1.24, dockershim was removed and clusters use containerd or CRI-O. Docker-built images still work because they're OCI-compliant.

- The container runtime pulls images and creates/starts/stops containers. The kubelet talks to it through the Container Runtime Interface (CRI), a standard gRPC API.

- Docker was the original default but was removed in Kubernetes 1.24 (dockershim deprecation). Modern runtimes are containerd and CRI-O, both CRI-native.
- Removing Docker as the runtime doesn't affect images — Docker-built images are OCI-compliant and run fine on containerd.

Likely follow-up: Why was dockershim removed? (Maintenance burden; Docker wasn't CRI-native and required a shim.)

- [ ] What is a Pod and why not just run containers directly?

IF ASKED, SAY THIS
A Pod is the smallest schedulable unit in Kubernetes — a group of one or more containers that share a network namespace, IP, and optionally storage. Kubernetes manages Pods rather than raw containers so it can consistently handle networking, storage, scheduling, and health.

- A Pod is the smallest deployable unit — one or more tightly-coupled containers that share a network namespace (same IP and port space) and can share storage volumes.
- Kubernetes schedules Pods, not bare containers, so it can attach networking, storage, lifecycle, and probes uniformly.
- Containers in the same Pod talk over localhost. Most Pods have one main container; multi-container Pods use the sidecar pattern.

Likely follow-up: Do containers in a Pod share an IP? (Yes.)

- [ ] Explain liveness, readiness, and startup probes.

IF ASKED, SAY THIS
Liveness probes detect a hung container and trigger a restart; readiness probes decide whether a Pod receives Service traffic without restarting it; startup probes give slow-starting apps time before liveness/readiness apply. The classic mistake is using an aggressive liveness probe where readiness was needed, causing unnecessary restart loops.

- Liveness: 'is the container alive?' If it fails, the kubelet restarts the container. Use it to recover from deadlocks.
- Readiness: 'can the container serve traffic?' If it fails, the Pod is removed from Service endpoints (no restart). Use it during warm-up or when a dependency is down.
- Startup: 'has the app finished starting?' Disables liveness/readiness until it passes — protects slow-starting apps from being killed prematurely.
- Common pitfall: a too-aggressive liveness probe restarts healthy-but-busy Pods, creating cascading restarts. Readiness, not liveness, should gate traffic.

```
readinessProbe:
  httpGet: { path: /healthz, port: 8080 }
  initialDelaySeconds: 5
  periodSeconds: 10
livenessProbe:
  httpGet: { path: /livez, port: 8080 }
  periodSeconds: 15
  failureThreshold: 3
```

Likely follow-up: A Pod is restarting in a loop under load — which probe is likely misconfigured? (Liveness.)

- [ ] What are init containers and sidecar containers?

IF ASKED, SAY THIS
Init containers run sequentially to completion before the main containers start, used for setup tasks like migrations or waiting on dependencies. Sidecar containers run alongside the main container for the Pod's lifetime — proxies, log shippers, exporters. Kubernetes 1.28+ supports native sidecars as always-restarting init containers.

- Init containers run to completion, in order, before the main containers start. Used for setup: waiting on a dependency, running migrations, fetching config/secrets. If one fails, the Pod restarts it.
- Sidecar containers run alongside the main container for the Pod's lifetime — logging agents, service-mesh proxies (Envoy), metrics exporters. As of K8s 1.28+, native sidecars are init containers with `restartPolicy: Always`.
- Difference: init = run-and-exit before app; sidecar = run-with app.

Likely follow-up: How do native sidecars (1.28+) differ from a plain extra container?

- [ ] Explain Pod QoS classes.

IF ASKED, SAY THIS
Kubernetes assigns a QoS class from a Pod's requests/limits: Guaranteed (requests == limits for all containers), Burstable (some requests set), and BestEffort (nothing set). Under node memory pressure the kubelet evicts BestEffort first, then Burstable, then Guaranteed — so critical workloads should be Guaranteed.

- Guaranteed: every container has requests == limits for both CPU and memory. Evicted last.
- Burstable: at least one container has a request but not requests == limits. Evicted in the middle.
- BestEffort: no requests or limits set at all. Evicted first under memory pressure.
- QoS is derived automatically from your requests/limits — you don't set it directly. For critical workloads, aim for Guaranteed.

Likely follow-up: Which Pods get killed first under memory pressure? (BestEffort.)

What are restartPolicy values and how do they interact with controllers?

IF ASKED, SAY THIS
restartPolicy controls how the kubelet restarts containers within a Pod: Always (default, required by Deployments), OnFailure, or Never (used by Jobs). It restarts containers in place; replacing a whole failed Pod is the controller's job. Repeated failed restarts under Always show up as CrashLoopBackOff.

- restartPolicy (Always | OnFailure | Never) governs container restarts within a Pod by the kubelet.
- Deployments/ReplicaSets require Always. Jobs use OnFailure or Never. When a Pod is deleted, the controller (not restartPolicy) creates a replacement.
- CrashLoopBackOff is the kubelet backing off exponentially while repeatedly restarting a failing container under restartPolicy: Always.

Likely follow-up: What does CrashLoopBackOff actually mean?

What's the difference between labels and annotations?

IF ASKED, SAY THIS
Labels are identifying key/value metadata used by selectors to group and target objects (Services finding Pods, controllers managing replicas). Annotations are non-identifying metadata for tools and humans and can't be used in selectors. Use labels for selection, annotations for everything else.

- Labels are identifying metadata used for selection and grouping. Selectors (in Services, ReplicaSets, Deployments) match on labels. Keep them short and queryable.
- Annotations are non-identifying metadata for tools and humans — build info, change-cause, ingress configuration, checksums. You cannot select on annotations.
- Rule of thumb: if something needs to select/group it, it's a label; if it's arbitrary attached data, it's an annotation.

```
kubectl get pods -l app=payment,tier=backend
kubectl label pod mypod env=prod
```

Likely follow-up: Can a Service select Pods by annotation? (No — only labels.)

How does a Service know which Pods to route to?

IF ASKED, SAY THIS
A Service uses a label selector; the endpoints controller watches for ready Pods whose labels match and adds them to the Service's EndpointSlices, which kube-proxy uses to load-balance traffic. If the Pod labels don't match the selector, the Service ends up with no endpoints.

- A Service has a `selector` (e.g. app=payment). The endpoints controller continuously finds Pods matching that selector and ready (passing readiness), and populates the Service's EndpointSlices.
- kube-proxy then load-balances traffic to those endpoints. If labels don't match, the Service has no endpoints and traffic fails.
- A frequent bug: Pod template labels don't match the Service selector → 'no endpoints'.

Likely follow-up: A Service returns no response and `kubectl get endpoints` is empty — what's the first thing to check? (Selector vs Pod labels and readiness.)

What is a ReplicaSet and why do we rarely create one directly?

IF ASKED, SAY THIS
A ReplicaSet keeps a desired number of identical Pods running, recreating any that die. We rarely create one directly because Deployments manage ReplicaSets and layer rolling updates and rollbacks on top — each Deployment revision is backed by its own ReplicaSet.

- A ReplicaSet ensures a specified number of identical Pod replicas are running at all times — it's the self-healing replica controller.
- You rarely create it directly because a Deployment manages ReplicaSets for you and adds rollout/rollback on top.
- Each Deployment revision corresponds to a ReplicaSet; scaling and healing are delegated to the ReplicaSet.

Likely follow-up: How many ReplicaSets does a Deployment keep around? (Controlled by revisionHistoryLimit.)

- [ ] What is a Deployment and what does it add over a ReplicaSet?

IF ASKED, SAY THIS
A Deployment is the standard controller for stateless workloads. It manages ReplicaSets and provides declarative rolling updates, rollbacks, and scaling — creating a new ReplicaSet on each spec change and shifting traffic gradually while keeping revision history for rollback.

- A Deployment manages stateless apps declaratively. It creates and manages ReplicaSets and provides controlled rollouts, rollbacks, and scaling.
- On a spec change (e.g. new image), it creates a new ReplicaSet and gradually shifts Pods from old to new per the rollout strategy.
- It tracks revision history so you can roll back to a previous version.

```
kubectl rollout status deployment/web
kubectl scale deployment/web --replicas=5
```

Likely follow-up: What object does a Deployment actually create to run Pods? (A ReplicaSet.)

- [ ] Explain maxSurge and maxUnavailable in a rolling update.

IF ASKED, SAY THIS
In a RollingUpdate, maxSurge controls how many Pods can be created above the desired replica count and maxUnavailable controls how many can be unavailable below it. Setting maxUnavailable=0 with maxSurge>0 gives a fully available, zero-downtime rollout at the cost of temporary extra capacity.

- RollingUpdate is the default Deployment strategy. maxSurge = how many extra Pods above desired count can be created during the update. maxUnavailable = how many Pods can be unavailable below desired count.
- Both can be absolute numbers or percentages. maxSurge=25%, maxUnavailable=25% is the default.
- maxUnavailable=0, maxSurge=1 gives a zero-downtime, one-at-a-time rollout (needs spare capacity). maxSurge=0 forces it to remove before adding (no extra capacity, but reduces availability).

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
```

Likely follow-up: How do you guarantee zero downtime during a rollout? (maxUnavailable=0 + readiness probes + PodDisruptionBudget.)

- [ ] How do you roll back a bad deployment?

IF ASKED, SAY THIS
Use `kubectl rollout undo` to return to the previous revision (or `--to-revision=N` for a specific one). Deployments retain revision history backed by old ReplicaSets, so rollback is just a controlled rollout back to a prior template.

- Deployments keep revision history (revisionHistoryLimit). You can undo to the previous revision or a specific one.
- Always record the change-cause (via annotation or `--record`, now deprecated) so history is readable.
- Rollback creates a new rollout back to the old ReplicaSet — it's not magic time travel, it's another controlled rollout.

```
kubectl rollout history deployment/web
kubectl rollout undo deployment/web
kubectl rollout undo deployment/web --to-revision=3
```

Likely follow-up: What governs how many old revisions you can roll back to? (revisionHistoryLimit.)

- [ ] Compare RollingUpdate, Recreate, Blue-Green, and Canary strategies.

IF ASKED, SAY THIS
RollingUpdate gradually replaces Pods with zero downtime; Recreate tears down all old Pods first (downtime, for incompatible versions); Blue-Green runs both versions fully and flips traffic instantly for fast rollback at double cost; Canary shifts a small traffic percentage to the new version and ramps based on metrics, usually via a mesh or Argo Rollouts/Flagger.

- RollingUpdate (built-in): gradual replacement, zero downtime, mixed versions briefly.
- Recreate (built-in): kill all old Pods, then start new — causes downtime, used when versions can't coexist (e.g. incompatible DB schema).
- Blue-Green: run old (blue) and new (green) in full, switch traffic at once via Service/ingress; instant rollback but doubles resources.
- Canary: send a small % of traffic to the new version, watch metrics, then ramp up. Usually done with a service mesh, ingress weights, or tools like Argo Rollouts/Flagger.

Likely follow-up: Which strategy needs a service mesh or progressive-delivery tool? (Canary/Blue-Green typically.)

- [ ] What are the Service types and when do you use each?

IF ASKED, SAY THIS
ClusterIP gives an internal-only virtual IP for service-to-service traffic; NodePort opens a static port on every node; LoadBalancer provisions an external cloud load balancer; ExternalName maps the Service to an external DNS name via CNAME. Most internal traffic uses ClusterIP, and external cloud exposure uses LoadBalancer (often fronted by Ingress).

- ClusterIP (default): stable virtual IP reachable only inside the cluster. For internal service-to-service traffic.
- NodePort: exposes the Service on a static port on every node's IP. Mostly for dev/testing or behind an external LB.
- LoadBalancer: provisions a cloud load balancer with an external IP. The standard way to expose a service externally in cloud.
- ExternalName: maps a Service to an external DNS name via a CNAME — no proxying, just DNS.

Likely follow-up: How does NodePort relate to LoadBalancer? (LoadBalancer builds on NodePort under the hood in most cloud implementations.)

- [ ] How does a ClusterIP Service actually route traffic to Pods?

IF ASKED, SAY THIS
The Service has a stable ClusterIP and a set of EndpointSlices listing ready backend Pods. kube-proxy programs iptables/IPVS rules that load-balance and DNAT traffic from the ClusterIP to a backend Pod IP, while CoreDNS resolves the Service name to that ClusterIP so clients connect by name.

- The Service gets a stable virtual ClusterIP. The endpoints controller fills EndpointSlices with ready Pods matching the selector.
- kube-proxy programs iptables/IPVS so packets to the ClusterIP are DNAT'd and load-balanced to a backend Pod IP.
- CoreDNS resolves the Service name to the ClusterIP, so apps connect by name, not IP.

Likely follow-up: What provides DNS resolution for Services? (CoreDNS.)

- [ ] What is a headless Service and when do you need one?

IF ASKED, SAY THIS
A headless Service (clusterIP: None) returns the individual Pod IPs via DNS instead of a single virtual IP, so clients can address specific Pods. It's essential for StatefulSets and stateful systems that need stable per-Pod DNS identities or client-side load balancing.

- A headless Service sets clusterIP: None. Instead of a single virtual IP, DNS returns the individual Pod IPs (A records) directly.
- Used when clients need to address specific Pods — StatefulSets (stable per-Pod DNS), databases, peer-to-peer systems, or client-side load balancing.
- Combined with a StatefulSet you get `pod-0.svc.namespace.svc.cluster.local` stable identities.

```
spec:
  clusterIP: None
  selector:
    app: cassandra
```

Likely follow-up: Why do StatefulSets need a headless Service?

- [ ] What is Ingress and how does it differ from a Service?

IF ASKED, SAY THIS
A Service does L4 load balancing to Pods, while Ingress provides L7 HTTP/HTTPS routing — host/path rules, TLS termination, and a single entry point fronting many Services. Ingress is only a set of rules; an Ingress Controller (NGINX, Traefik, cloud ALB) must be deployed to enforce them.

- A Service is L3/L4 (IP/port) load balancing. Ingress is L7 (HTTP/HTTPS) routing: host- and path-based rules, TLS termination, and a single entry point for many services.
- Ingress is just an API object — it needs an Ingress Controller (NGINX, Traefik, HAProxy, cloud ALB controller) actually running to fulfill the rules.
- Typical setup: one external LoadBalancer → Ingress controller → routes by host/path to many ClusterIP Services.

Likely follow-up: Does creating an Ingress object do anything without a controller? (No.)

- [ ] What is the Gateway API and why is it replacing Ingress?

IF ASKED, SAY THIS

Gateway API is the successor to Ingress: a more expressive, portable, role-separated standard. GatewayClass and Gateway are owned by infra/platform teams while HTTPRoute and other route objects are owned by app teams. It natively supports traffic splitting, header routing, and cross-namespace routing without vendor-specific annotations.

- Ingress is limited: vendor-specific behavior lives in annotations, and it's HTTP-centric with weak role separation.
- Gateway API is a newer, more expressive, role-oriented standard. GatewayClass (infra provider) → Gateway (cluster operator) → HTTPRoute/TCPRoute/GRPCRoute (app teams). It supports header-based routing, traffic splitting, and cross-namespace routing natively.
- It separates concerns: platform teams own Gateways, app teams own Routes — better multi-team governance.

Likely follow-up: Which roles map to GatewayClass, Gateway, and HTTPRoute?

- [ ] What is a StatefulSet and how does it differ from a Deployment?

IF ASKED, SAY THIS
A StatefulSet runs stateful apps that need stable identity and storage. Each Pod gets a stable ordinal name, stable DNS via a headless Service, and a dedicated PersistentVolumeClaim that survives rescheduling. Pods are created, scaled, and updated in a deterministic order — unlike Deployments, where Pods are interchangeable.

- A StatefulSet manages stateful apps that need stable identity and storage. Unlike Deployments, it gives each Pod a stable, ordinal name (app-0, app-1...), stable network identity via a headless Service, and its own PersistentVolumeClaim that persists across rescheduling.
- Pods are created/scaled/deleted in order, and updates roll in reverse ordinal order.
- Deployments treat Pods as interchangeable cattle; StatefulSets treat them as distinct, ordered pets.

Likely follow-up: What gives each StatefulSet Pod a stable DNS name? (A headless Service.)

- [ ] What happens to storage when a StatefulSet Pod is rescheduled or deleted?

IF ASKED, SAY THIS
PVCs created from volumeClaimTemplates are bound to each Pod's stable identity, so a rescheduled Pod reattaches to the same volume and keeps its data. Deleting the StatefulSet or scaling down does not delete PVCs by default — they're retained to protect data unless you set a persistentVolumeClaimRetentionPolicy or remove them manually.

- Each Pod's PVC (from volumeClaimTemplates) is tied to its ordinal identity. If a Pod is rescheduled, it reattaches to the same PVC/PV — data persists.
- Deleting the StatefulSet does NOT delete the PVCs by default (to protect data); you must delete them explicitly. (Newer versions add persistentVolumeClaimRetentionPolicy to control this.)
- Scaling down leaves the PVCs behind so data isn't lost if you scale back up.

Likely follow-up: How do you intentionally clean up StatefulSet storage?

- [ ] What is a DaemonSet and what is it used for?

IF ASKED, SAY THIS
A DaemonSet runs exactly one Pod per node (or per matching subset), automatically adding Pods as nodes join and removing them as nodes leave. It's used for node-level agents like log shippers, node-exporter, CNI plugins, and security daemons, and typically tolerates node taints to run everywhere.

- A DaemonSet ensures one copy of a Pod runs on every node (or a subset matched by nodeSelector/affinity). As nodes join, the DaemonSet schedules a Pod there automatically; as nodes leave, the Pod is removed.
- Used for node-level agents: log collectors (Fluent Bit), monitoring (node-exporter), CNI plugins (Cilium/Calico), storage daemons, security agents.
- DaemonSet Pods often tolerate taints so they can run even on control-plane or specialized nodes.

Likely follow-up: How do you run a DaemonSet Pod on tainted control-plane nodes? (Add a matching toleration.)

- [ ] What's the difference between a Job and a CronJob?

IF ASKED, SAY THIS
A Job runs Pods to successful completion for batch/one-off work and retries failures up to backoffLimit. A CronJob schedules Jobs on a recurring cron schedule. Jobs support parallelism and completion counts; CronJobs add scheduling, concurrency policy, and history limits.

- A Job runs a Pod (or several) to completion — for batch tasks like migrations, backups, or one-off processing. It tracks successful completions and retries failures per backoffLimit.
- A CronJob creates Jobs on a schedule (cron syntax) — recurring tasks like nightly backups or report generation.
- Key Job knobs: completions, parallelism, backoffLimit, activeDeadlineSeconds, ttlSecondsAfterFinished.

```
spec:
  schedule: "0 2 * * *"
  concurrencyPolicy: Forbid
```

```
  jobTemplate:
    spec:
      backoffLimit: 3
```

Likely follow-up: What does concurrencyPolicy control on a CronJob? (Whether overlapping runs are Allowed, Forbidden, or Replaced.)

- [ ] What's the difference between a ConfigMap and a Secret?

IF ASKED, SAY THIS
ConfigMaps hold non-sensitive configuration; Secrets hold sensitive data. Both can be consumed as env vars or mounted files, but Secrets are only base64-encoded by default — not encrypted — so you should enable encryption at rest, tighten RBAC, and ideally source secrets from an external manager like Vault or AWS Secrets Manager.

- Both store key/value config and can be mounted as env vars or files. ConfigMaps are for non-sensitive config; Secrets are for sensitive data (passwords, tokens, keys).
- Secrets are only base64-encoded by default (NOT encrypted) — base64 is not security. Enable encryption at rest (EncryptionConfiguration) and restrict RBAC.
- Best practice: use an external secrets manager (Vault, AWS Secrets Manager via External Secrets Operator) rather than committing Secrets to Git.

Likely follow-up: Is a Kubernetes Secret encrypted by default? (No — base64-encoded; enable encryption at rest.)

- [ ] If you update a ConfigMap, do running Pods see the change?

IF ASKED, SAY THIS
ConfigMap values injected as environment variables are fixed at Pod start and don't change live — you must restart the Pods. Volume-mounted ConfigMaps update in the filesystem after a sync, but the app must re-read them. A common trick is annotating the Pod template with a ConfigMap checksum so updates trigger a rolling restart automatically.

- Env vars from a ConfigMap are injected at Pod start and do NOT update live — you must restart/roll the Pods.
- ConfigMaps mounted as volumes DO update in the Pod's filesystem (eventually, after the kubelet sync), but the app must re-read the file to pick up changes.
- Common pattern: add a checksum annotation of the ConfigMap to the Pod template so changing the ConfigMap triggers a rolling update (Helm does this).

```
kubectl rollout restart deployment/web
```

Likely follow-up: How do you force Pods to pick up a changed ConfigMap? (Roll the Deployment, e.g. via checksum annotation or `kubectl rollout restart`.)

- [ ] Explain PersistentVolume, PersistentVolumeClaim, and StorageClass.

IF ASKED, SAY THIS
A PersistentVolume is an actual storage resource in the cluster; a PersistentVolumeClaim is a user's request for storage that binds to a PV; and a StorageClass defines a storage type and provisioner enabling dynamic provisioning, so a PVC referencing it triggers automatic creation of a backing PV. Pods always reference PVCs, never PVs directly.

- PV: a piece of storage in the cluster (an EBS volume, NFS share, etc.), provisioned by an admin or dynamically. It's a cluster resource.
- PVC: a user's request for storage (size, access mode). It binds to a matching PV. Pods reference PVCs, not PVs directly.
- StorageClass: defines a 'type' of storage and a provisioner for dynamic provisioning — when a PVC requests a StorageClass, a PV is created on demand. This is how cloud volumes get auto-created.

Likely follow-up: What enables dynamic provisioning? (A StorageClass with a provisioner.)

- [ ] Explain access modes: ReadWriteOnce, ReadOnlyMany, ReadWriteMany, ReadWriteOncePod.

IF ASKED, SAY THIS
RWO mounts read-write to a single node, ROX read-only to many nodes, RWX read-write to many nodes (requires a shared filesystem like NFS/EFS), and ReadWriteOncePod restricts to exactly one Pod. Block storage like EBS is typically RWO, so spreading multiple Pods that need the same RWO volume across nodes causes multi-attach errors.

- ReadWriteOnce (RWO): mounted read-write by a single node (multiple Pods on that node can share it). Most block storage (EBS) is RWO.
- ReadOnlyMany (ROX): mounted read-only by many nodes.
- ReadWriteMany (RWX): mounted read-write by many nodes — needs a shared filesystem (NFS, EFS, CephFS).
- ReadWriteOncePod: read-write by exactly one Pod (stricter than RWO). Common error: scheduling two Pods needing RWO on different nodes → multi-attach error.

Likely follow-up: Why can't two Pods on different nodes share an EBS volume read-write? (EBS is RWO — single node.)

- [ ] Explain the Kubernetes networking model and the role of CNI.

IF ASKED, SAY THIS
Kubernetes requires that every Pod has a unique IP and can reach every other Pod without NAT, with containers in a Pod sharing one network namespace. Kubernetes delegates the implementation to a CNI plugin — Calico, Cilium, Flannel, or the AWS VPC CNI — which assigns Pod IPs and programs the routing or overlay. Cilium's eBPF dataplane can even replace kube-proxy.

- Kubernetes networking rules: every Pod gets its own IP; Pods communicate across nodes without NAT; containers in a Pod share the Pod's network namespace.
- Kubernetes itself doesn't implement Pod networking — a CNI plugin (Calico, Cilium, Flannel, AWS VPC CNI) does, assigning IPs and wiring routes/overlays.
- Cilium uses eBPF for high-performance networking and can replace kube-proxy. AWS VPC CNI gives Pods real VPC IPs.

Likely follow-up: What does the AWS VPC CNI do differently? (Assigns real VPC IPs to Pods.)

- [ ] What is CoreDNS and how does service discovery work?

IF ASKED, SAY THIS
CoreDNS provides in-cluster DNS, running in kube-system with every Pod configured to use it. Services resolve via `..svc.cluster.local` (or just the service name within the same namespace), enabling name-based service discovery. Headless Services and StatefulSets resolve to specific Pod IPs.

- CoreDNS runs as a Deployment in kube-system and serves cluster DNS. Every Pod's /etc/resolv.conf points at the CoreDNS Service.
- Services resolve as `..svc.cluster.local`. Within the same namespace, just `` works thanks to search domains.
- Headless Services resolve to individual Pod IPs; StatefulSet Pods get `pod-0...svc.cluster.local`.

```
kubectl -n kube-system get pods -l k8s-app=kube-dns
kubectl run -it --rm dnsutils --image=busybox -- nslookup my-svc
```

Likely follow-up: A Pod can reach a Service by IP but not by name — where do you look? (CoreDNS health and the Pod's DNS config.)

- [ ] What are taints and tolerations and how do they differ from node affinity?

IF ASKED, SAY THIS
Taints on nodes repel Pods unless those Pods carry a matching toleration — it's a node-side 'keep off' mechanism with NoSchedule/PreferNoSchedule/NoExecute effects. Node affinity is a Pod-side preference or requirement that attracts Pods to nodes with matching labels. They're complementary: taint specialized nodes to keep generic workloads off, and add affinity so the right workloads target them.

- Taints are applied to NODES and repel Pods that don't tolerate them ('keep off unless allowed'). Tolerations on Pods let them be scheduled onto tainted nodes. Effects: NoSchedule, PreferNoSchedule, NoExecute.
- Node affinity is applied to PODS and attracts them to nodes with matching labels ('I want to run here'). It's required (hard) or preferred (soft).
- Mental model: taints/tolerations = nodes pushing Pods away; affinity = Pods pulling themselves toward nodes. They're complementary — taint a GPU node AND add affinity so only GPU workloads land there.

```
kubectl taint nodes node1 gpu=true:NoSchedule
```

Likely follow-up: What does the NoExecute taint effect do to already-running Pods? (Evicts those without a matching toleration.)

- [ ] Explain pod affinity, anti-affinity, and topology spread constraints.

IF ASKED, SAY THIS
Pod affinity co-locates Pods with related Pods; anti-affinity keeps replicas apart to survive node/zone failures; and topology spread constraints provide a modern, declarative way to distribute Pods evenly across zones or nodes using maxSkew. Together they let you design for high availability and fault isolation.

- Pod affinity: schedule Pods near other Pods (same node/zone) — e.g. co-locate a cache with its app for latency.
- Pod anti-affinity: keep Pods apart — e.g. spread replicas across nodes so one node failure doesn't take all replicas. requiredDuringScheduling (hard) vs preferred (soft).
- Topology spread constraints: a cleaner, modern way to evenly distribute Pods across topology domains (zones, nodes) with maxSkew, balancing availability without the overhead of anti-affinity rules.

Likely follow-up: How do you ensure replicas land in different availability zones? (topologySpreadConstraints on topology.kubernetes.io/zone, or zone anti-affinity.)

- [ ] Explain HPA, VPA, and Cluster Autoscaler and how they relate.

IF ASKED, SAY THIS
HPA scales the number of Pod replicas based on CPU/memory/custom metrics; VPA right-sizes per-Pod CPU/memory requests; and the Cluster Autoscaler adds or removes nodes when Pods can't schedule or nodes are idle. They work together — HPA adds Pods and the Cluster Autoscaler adds nodes if there's no room — but HPA and VPA shouldn't target the same resource metric.

- HPA (Horizontal Pod Autoscaler): scales the NUMBER of Pod replicas based on metrics (CPU/memory via metrics-server, or custom/external metrics). Reacts to load.
- VPA (Vertical Pod Autoscaler): adjusts Pod CPU/memory REQUESTS to right-size them. Often recreates Pods to apply new requests.
- Cluster Autoscaler: adds/removes NODES when Pods can't be scheduled (pending due to insufficient capacity) or nodes are underutilized.
- They compose: HPA adds Pods → if no node has room, Cluster Autoscaler adds a node. Don't run HPA and VPA on the same metric (CPU) simultaneously — they conflict.

```
kubectl autoscale deployment web --cpu-percent=70 --min=3 --max=20
```

Likely follow-up: Why shouldn't HPA and VPA both act on CPU? (They fight each other.) · What provides CPU/memory metrics to HPA? (metrics-server.)

- [ ] What is Helm and what problem does it solve?

IF ASKED, SAY THIS
Helm is Kubernetes' package manager. A Chart packages templated manifests, values.yaml parameterizes them per environment, and a Release is an installed instance. It solves reuse, versioning, and lifecycle management — letting you install, upgrade, and roll back applications consistently across environments.

- Helm is the package manager for Kubernetes. A Chart bundles templated manifests; values.yaml parameterizes them; a Release is an installed instance of a chart with specific values.
- It solves templating/reuse (one chart, many environments via different values), versioning, and lifecycle (install/upgrade/rollback).
- Helm 3 is client-only (no Tiller). `helm upgrade --install` is the idempotent workhorse; `helm rollback` reverts a release.

```
helm upgrade --install web ./chart -f values-prod.yaml
helm rollback web 2
```

Likely follow-up: What changed between Helm 2 and Helm 3? (Tiller was removed; Helm 3 is client-side with RBAC via kubeconfig.)

- [ ] Explain RBAC: Role, ClusterRole, RoleBinding, ClusterRoleBinding.

IF ASKED, SAY THIS
Roles define namespaced permissions and ClusterRoles define cluster-wide or cluster-scoped permissions. RoleBindings grant those permissions to subjects (users, groups, ServiceAccounts) within a namespace, while ClusterRoleBindings grant them cluster-wide. A RoleBinding can reference a ClusterRole to reuse a permission set in one namespace, and the guiding principle is least privilege.

- Role: a set of permissions (verbs on resources) scoped to ONE namespace. ClusterRole: same but cluster-wide or for cluster-scoped resources (nodes, PVs).
- RoleBinding: grants a Role (or ClusterRole) to subjects within a namespace. ClusterRoleBinding: grants a ClusterRole cluster-wide.
- Subjects = users, groups, or ServiceAccounts. Follow least privilege; avoid binding cluster-admin broadly.
- Trick: a RoleBinding can reference a ClusterRole to reuse a common permission set within a single namespace.

Likely follow-up: Can a RoleBinding reference a ClusterRole? (Yes — to scope shared permissions to one namespace.)

- [ ] What are common Kubernetes security best practices for a Pod?

IF ASKED, SAY THIS
Harden Pods with a securityContext: run as non-root, disable privilege escalation, drop capabilities, and use a read-only root filesystem. Enforce restricted Pod Security Standards, apply default-deny NetworkPolicies, use minimal scanned images with pinned digests, set resource limits, disable unused ServiceAccount token automounting, and pull secrets from an external manager.

- Run as non-root, drop all Linux capabilities, set readOnlyRootFilesystem, and use a securityContext with runAsNonRoot and allowPrivilegeEscalation: false.
- Enforce Pod Security Standards (restricted) via Pod Security Admission. Apply NetworkPolicies for default-deny east-west traffic.
- Use minimal/distroless images, scan for CVEs, pin image digests, set resource limits, and source secrets from a manager (Vault). Restrict ServiceAccount tokens (automountServiceAccountToken: false where unused).

```
securityContext:
  runAsNonRoot: true
  allowPrivilegeEscalation: false
```

```
readOnlyRootFilesystem: true
capabilities: { drop: ["ALL"] }
```

Likely follow-up: How do you enforce 'no root containers' cluster-wide? (Pod Security Admission with the restricted profile, or a policy engine like Kyverno/OPA Gatekeeper.)

- [ ] What is a NetworkPolicy and what's the default behavior without one?

IF ASKED, SAY THIS
Without any NetworkPolicy, all Pod-to-Pod traffic is allowed by default. A NetworkPolicy uses label selectors to restrict ingress/egress; once a Pod is selected by a policy, everything in that direction is denied except what's explicitly allowed. It only works with a policy-enforcing CNI like Calico or Cilium, and the recommended baseline is default-deny plus explicit allows.

- By default, all Pods can talk to all other Pods — the network is fully open. A NetworkPolicy restricts ingress/egress at L3/L4 using Pod/namespace selectors.
- Once any NetworkPolicy selects a Pod, that Pod becomes default-deny for the selected direction except what the policy allows.
- Requires a CNI that enforces policies (Calico, Cilium); some CNIs ignore NetworkPolicies. Best practice: a default-deny baseline, then allow specific flows.

Likely follow-up: If you apply a NetworkPolicy but nothing is blocked, what's likely wrong? (CNI doesn't enforce NetworkPolicies.)

- [ ] How do you monitor a Kubernetes cluster?

IF ASKED, SAY THIS
Use Prometheus to scrape cluster and app metrics (with kube-state-metrics and node-exporter), Grafana for dashboards, and Alertmanager for alerts. Ship container stdout/stderr via Fluent Bit/Fluentd to Elasticsearch, Loki, or Splunk, and add distributed tracing with OpenTelemetry/Jaeger. Drive alerting from SLOs and error-budget burn rate rather than raw thresholds.

- Metrics: Prometheus scrapes cluster + app metrics; kube-state-metrics exposes object state; node-exporter exposes node metrics; metrics-server feeds HPA. Visualize in Grafana; alert via Alertmanager.
- Logging: containers log to stdout/stderr; a node-level agent (Fluent Bit/Fluentd) ships logs to Elasticsearch/Loki/Splunk.
- Tracing: OpenTelemetry → Jaeger/Tempo for distributed traces. Define SLIs/SLOs and alert on burn rate, not just raw thresholds.

Likely follow-up: What feeds the HPA its CPU/memory metrics? (metrics-server.) · Why alert on burn rate instead of static thresholds?

- [ ] What are the 'golden signals' and how do they map to Kubernetes?

IF ASKED, SAY THIS
The golden signals are latency, traffic, errors, and saturation. In Kubernetes, latency comes from app/mesh metrics, traffic from request rate, errors from 5xx rates and failing probes, and saturation from CPU/memory utilization versus limits — including CPU throttling and OOMKills. Tracking these gives a complete health picture.

- The four golden signals (Google SRE): Latency, Traffic, Errors, Saturation.
- Latency: request duration (app metrics / mesh). Traffic: requests/sec or throughput. Errors: HTTP 5xx rate, failed probes. Saturation: CPU/memory/disk utilization vs limits, Pod pending counts, throttling.
- In K8s, saturation also includes CPU throttling (container hitting its CPU limit) and OOMKills (memory limit exceeded).

Likely follow-up: What does an OOMKilled status indicate? (Container exceeded its memory limit.)

- [ ] A Pod is stuck in Pending. How do you troubleshoot?

IF ASKED, SAY THIS
Pending means the Pod hasn't been scheduled. Run `kubectl describe pod` and read the Events — they usually state the reason: insufficient resources, unsatisfiable node selectors/affinity, taints without tolerations, an unbound PVC, or a ResourceQuota limit. Cross-check node capacity and PVC status to confirm.

- Pending = not yet scheduled to a node. Start with `kubectl describe pod` and read the Events at the bottom.
- Common causes: insufficient CPU/memory on all nodes; node selectors/affinity that match nothing; taints without tolerations; unbound PVC (no matching PV/StorageClass); too-large resource requests; or hitting a ResourceQuota.
- Check `kubectl get nodes` for capacity, taints, and Ready status. If it's a PVC issue, `kubectl get pvc` will show Pending.

```
kubectl describe pod <pod>
kubectl get nodes
kubectl get events --sort-by=.lastTimestamp
```

Likely follow-up: Pod is Pending with 'Insufficient cpu' — your options? (Lower requests, add nodes / Cluster Autoscaler, or reschedule.)

- [ ] A Pod is in CrashLoopBackOff. Walk through your debugging.

IF ASKED, SAY THIS
CrashLoopBackOff means the container repeatedly starts and exits. Check `kubectl logs --previous` to see why the last instance died, and `kubectl describe pod` for the exit code and events. Typical causes are missing config/secrets, a failing startup dependency, a bad entrypoint, OOMKill (exit 137), or an over-aggressive liveness probe — fix the cause and redeploy.

- CrashLoopBackOff = the container keeps starting then exiting, and the kubelet is backing off between restarts.
- Steps: `kubectl logs ` for the current attempt, `kubectl logs --previous` for the crashed instance. Check exit code in `kubectl describe pod`.
- Common causes: app config/secret missing or wrong, failing dependency at startup, bad command/entrypoint, OOMKilled (exit 137), or a too-aggressive liveness probe. Fix the root cause, then redeploy.

```
kubectl logs <pod> --previous
kubectl describe pod <pod>
```

Likely follow-up: What does exit code 137 usually mean? (OOMKilled — memory limit exceeded / SIGKILL.)

- [ ] A Service returns connection errors even though Pods are running. Diagnose.

IF ASKED, SAY THIS
Check whether the Service has endpoints with `kubectl get endpoints`; empty endpoints usually mean a selector/label mismatch or no ready Pods. Verify readiness, that selector labels match, that targetPort matches the container port, and that the app binds 0.0.0.0. Then check CoreDNS resolution, kube-proxy, and any NetworkPolicy that could be blocking the path.

- Work outward: Are the Pods Ready (passing readiness)? `kubectl get endpoints ` — empty endpoints means selector/label mismatch or no ready Pods.
- Check selector vs Pod labels, the targetPort vs containerPort, and that the app listens on 0.0.0.0 not 127.0.0.1.
- Then test DNS (`nslookup`), then kube-proxy health, then NetworkPolicies that might block the traffic, then ingress if external.

```
kubectl get endpoints <svc>
kubectl get pods -l <selector> -o wide
```

Likely follow-up: Endpoints list is empty — top two causes? (Label/selector mismatch or readiness failing.)

- [ ] etcd is unhealthy or a control-plane node is down. What's the impact and response?

IF ASKED, SAY THIS
Losing etcd quorum stops all writes — no deploys, scaling, or updates — though existing Pods keep running because the kubelet maintains them. With a 3-member etcd you can lose one and keep quorum; restore the member, or if quorum is lost, restore from the latest snapshot. Prevent this with an HA control plane, frequent etcd backups, and monitoring of etcd disk/fsync latency.

- Impact: if etcd loses quorum, the API Server can't write — no deployments, scaling, or updates. Existing Pods keep running (the kubelet maintains them), so data-plane stays up but the cluster is unmanageable.
- Response: with an odd-member etcd, losing one of three is survivable (quorum 2/3). Restore the failed member; if quorum is lost, restore from the latest etcd snapshot.
- Prevention: HA control plane (3+ nodes), regular etcd snapshots, monitored etcd health and disk latency (etcd is very sensitive to disk fsync latency).

```
ETCDCTL_API=3 etcdctl snapshot status backup.db
ETCDCTL_API=3 etcdctl snapshot restore backup.db
```

Likely follow-up: Do running applications go down if the control plane fails? (No — the data plane keeps serving; you just can't manage the cluster.)

- [ ] A node shows NotReady. How do you investigate?

IF ASKED, SAY THIS
Run `kubectl describe node` and check its Conditions and last heartbeat. NotReady usually means the kubelet stopped, the container runtime failed, the CNI/network broke, the disk filled, or the node lost API-server connectivity. On the node, check kubelet and runtime status, disk usage, and kubelet logs. After the eviction timeout, the control plane reschedules the node's Pods elsewhere.

- `kubectl describe node ` — check Conditions (MemoryPressure, DiskPressure, PIDPressure) and the kubelet's last heartbeat.
- Common causes: kubelet crashed/stopped, container runtime down, network/CNI failure, disk full, or the node lost connectivity to the API Server.

- SSH to the node: check `systemctl status kubelet`, runtime status, disk space (`df -h`), and logs (`journalctl -u kubelet`). Pods on a NotReady node may be evicted after the eviction timeout.

```
kubectl describe node <node>
kubectl get nodes
# on node:
systemctl status kubelet && journalctl -u kubelet -n100
```

Likely follow-up: What happens to Pods on a node that stays NotReady? (They're eventually evicted/rescheduled after the timeout.)

- [ ] What is GitOps and what are its core principles?

IF ASKED, SAY THIS
GitOps uses Git as the single source of truth for declarative desired state, with an in-cluster agent that continuously pulls from Git and reconciles the live cluster to match it. Changes happen through reviewed PRs, giving a full audit trail, easy rollback via git revert, and automatic drift correction.

- GitOps means Git is the single source of truth for the desired state of your system. Everything — manifests, Helm values, config — lives in Git.
- A controller running IN the cluster continuously reconciles the live cluster to match Git (the pull model). You change infrastructure by merging a PR, not by running kubectl apply manually.
- Four principles: declarative desired state, versioned & immutable in Git, pulled automatically, and continuously reconciled.

Likely follow-up: Why is the pull model more secure than pushing with kubectl from CI? (No external system holds cluster credentials.) · How do you roll back in GitOps? (git revert the commit.)

- [ ] Describe Argo CD's architecture.

IF ASKED, SAY THIS
Argo CD has an API server (UI/API/RBAC/SSO), a repo server that clones Git and renders manifests, and an application controller that diffs desired vs live state and performs syncs, with Redis for caching. The controller is the reconciliation engine that keeps clusters matching Git and reports sync and health status.

- Argo CD runs three main components: the API/Server (UI, API, SSO, RBAC), the Repo Server (clones Git and renders manifests via Helm/Kustomize/plain YAML), and the Application Controller (compares desired vs live state and syncs). Redis caches state.
- The Application Controller is the reconciliation engine: it watches Applications, computes the diff between the rendered Git manifests and the live cluster, and reports Sync/Health status.
- It can manage the local cluster and many remote clusters registered to it.

Likely follow-up: Which component renders Helm/Kustomize? (The repo server.)

- [ ] Explain the difference between Argo CD Sync status and Health status.

IF ASKED, SAY THIS
Sync status tells you whether the cluster matches Git (Synced/OutOfSync); health status tells you whether the running resources are actually healthy (Healthy/Progressing/Degraded). They're independent — you can be Synced but Degraded, or OutOfSync but Healthy — so you watch both.

- Sync status answers 'does the live cluster match Git?' — Synced or OutOfSync.
- Health status answers 'are the deployed resources actually healthy?' — Healthy, Progressing, Degraded, Missing.
- They are orthogonal: an app can be Synced but Degraded (Git applied, pods crashing) or OutOfSync but Healthy (running fine, but a newer Git version isn't applied yet).

Likely follow-up: An app is Synced but Degraded — where do you look? (Pod logs/events, not Git.)

- [ ] What do automated sync, prune, and self-heal do in Argo CD?

IF ASKED, SAY THIS
Automated sync applies Git changes without manual triggering; prune deletes resources removed from Git so no orphans remain; and self-heal reverts manual cluster changes back to Git. Together they give true GitOps enforcement, though many teams auto-sync non-prod and keep manual sync plus approvals for prod.

- Automated sync applies Git changes automatically when they appear (vs manual sync you trigger).
- Prune deletes resources that were removed from Git (otherwise orphans linger).
- Self-heal reverts any manual/live drift back to the Git-declared state, enforcing Git as truth.

Likely follow-up: What's the risk of enabling self-heal? (It reverts emergency manual hotfixes unless you also change Git.)

- [ ] What are the App-of-Apps pattern and ApplicationSets?

IF ASKED, SAY THIS
App-of-Apps is a parent Application that manages child Applications, used to bootstrap an entire cluster declaratively. ApplicationSet is a controller that templates and generates many Applications from generators (clusters, Git directories, lists), which is the scalable way to manage fleets and multi-environment deployments.

- App-of-Apps: a parent Argo CD Application whose Git source contains the manifests for other Applications, so syncing the parent bootstraps and manages all children — great for bootstrapping a whole cluster's add-ons and workloads.
- ApplicationSet: a controller that generates many Applications from a template plus a generator (per cluster, per Git directory, per list) — the scalable, templated evolution of App-of-Apps.
- Use ApplicationSets to deploy the same app across many clusters/regions/environments from one definition.

Likely follow-up: How would you deploy one app to 20 clusters with minimal duplication? (ApplicationSet with a cluster generator.)

- [ ] What are Argo CD sync waves and resource hooks?

IF ASKED, SAY THIS
Sync waves order which resources apply first within a sync (lower numbers first), useful for dependencies like CRDs before workloads. Resource hooks run Jobs at PreSync/Sync/PostSync phases for tasks like database migrations and smoke tests, and a failed PreSync hook stops the deploy.

- Sync waves order resource application within a single sync (lower wave numbers apply first) — e.g. create a namespace and CRDs before the workloads that depend on them.
- Resource hooks run Jobs at sync phases: PreSync (e.g. a DB migration before deploy), Sync, and PostSync (e.g. smoke tests/notifications).
- A failed PreSync hook halts the sync, protecting you from deploying on top of a failed migration.

```
metadata:
  annotations:
    argocd.argoproj.io/sync-wave: "1"
    argocd.argoproj.io/hook: PreSync
```

Likely follow-up: How do you run a database migration before the app rolls out with Argo CD? (A PreSync hook Job.)

- [ ] How does Argo CD detect drift, and how do you roll back?

IF ASKED, SAY THIS
Argo CD continuously diffs live vs Git and marks drift as OutOfSync with a visible diff; self-heal can auto-revert it. To roll back, prefer git revert of the offending commit (true GitOps) or use argocd app rollback to a previous synced revision from the app's history.

- Argo CD continuously compares live state to the rendered Git manifests; any divergence (a manual kubectl edit, an operator change) shows the app OutOfSync with an exact diff you can view in the UI or CLI.
- With self-heal on, it auto-reverts drift; otherwise you sync to reconcile.
- Rollback is either git revert (the GitOps way — reverts the commit so the whole system converges back) or argocd app rollback to a previous synced revision from Argo's history.

```
argocd app history myapp
argocd app rollback myapp 42
```

Likely follow-up: Why is git revert preferable to argocd app rollback for a permanent fix? (Git stays the source of truth; otherwise the next sync re-applies the bad version.)

- [ ] In a GitOps world, what is the boundary between CI and CD?

IF ASKED, SAY THIS
CI builds/tests and produces an immutable image, then commits the new desired state (image tag) to a Git config repo. CD (Argo CD) pulls that change and reconciles the cluster. The clean boundary — CI writes to Git, CD reads from Git — means the pipeline holds no cluster credentials and every deploy is an auditable commit.

- CI (e.g. GitHub Actions) builds, tests, and produces an immutable artifact (a container image with an immutable tag), then updates the desired state in a Git config repo (bumps the image tag).
- CD (Argo CD) is the in-cluster controller that notices the Git change and reconciles the cluster to it. CI never touches the cluster directly.
- This separation means the pipeline needs no cluster credentials, and every deploy is a reviewable Git commit.

Likely follow-up: Why shouldn't the CI pipeline run kubectl apply against prod directly? (It needs cluster creds and bypasses the Git audit trail.)

Why must container image tags be immutable, and what's wrong with :latest?

IF ASKED, SAY THIS
Immutable tags (e.g. the Git SHA) ensure a tag always maps to exactly one image, making deploys reproducible and rollbacks precise. :latest is ambiguous — nodes may run different builds, rollback is impossible, and GitOps can't detect a change because the tag string is unchanged. Always deploy by immutable digest/SHA.

- Tag by the Git commit SHA (or a semver + build) so a given tag always refers to exactly one image. This makes deploys reproducible and rollbacks precise.
- Using :latest is an anti-pattern: the running version is ambiguous, different nodes may pull different images, rollbacks are impossible, and GitOps can't detect a change (the tag string never changes even though the image did).
- Set imagePullPolicy appropriately and never rely on :latest in production.

Likely follow-up: How does GitOps know a new version shipped if the tag never changes? (It doesn't — you must bump to a new immutable tag.)

How do you get a fully zero-downtime rollout on Kubernetes?

IF ASKED, SAY THIS
Combine RollingUpdate with maxUnavailable=0 and maxSurge>0, correct readiness probes so traffic only reaches ready Pods, a PodDisruptionBudget to bound voluntary disruptions, and graceful SIGTERM handling to drain in-flight requests. Missing any one of these can drop connections during a deploy.

- Use RollingUpdate with maxUnavailable: 0 and maxSurge > 0 so new Pods come up before old ones go away.
- Define correct readiness probes so traffic only shifts to Pods that are actually ready.
- Add a PodDisruptionBudget so voluntary disruptions (drains, upgrades) can't take down too many replicas at once.
- Handle SIGTERM gracefully (drain in-flight requests during terminationGracePeriodSeconds).

```
strategy:
  rollingUpdate: { maxSurge: 1, maxUnavailable: 0 }
```

Likely follow-up: Which single misconfiguration most often causes dropped requests during a deploy? (Missing/incorrect readiness probe.)

What is Argo Rollouts and how does it add canary/blue-green?

IF ASKED, SAY THIS
Argo Rollouts replaces a Deployment with a Rollout CRD that supports canary and blue-green with weighted traffic steps and automated metric analysis. It shifts a small traffic percentage, evaluates Prometheus-based AnalysisRuns, and promotes or automatically aborts — the progressive-delivery capability plain Deployments lack.

- A Deployment only does rolling updates. Argo Rollouts replaces the Deployment with a Rollout resource that supports canary and blue-green with fine-grained steps.
- Canary: shift a small percentage of traffic, pause, run automated AnalysisRuns against metrics (Prometheus error rate/latency), then promote or auto-abort.
- It integrates with ingress/service mesh for traffic shaping and gives a dashboard/CLI to watch the rollout.

```
kubectl argo rollouts get rollout web --watch
```

Likely follow-up: How does a canary automatically abort on a bad deploy? (An AnalysisRun on error-rate/latency fails its threshold.)

How do you handle secrets in a GitOps repo?

IF ASKED, SAY THIS
Use Sealed Secrets or SOPS to store only encrypted secrets in Git (decrypted in-cluster), or the External Secrets Operator to keep just a reference in Git and pull the real value from Vault/cloud secret managers at runtime. Never commit plaintext secrets, even to a private repo.

- Never commit plaintext secrets. Encrypt-before-commit with Sealed Secrets (a controller decrypts them in-cluster) or SOPS (age/KMS-encrypted values).
- Or reference-don't-store: the External Secrets Operator pulls secrets from Vault/AWS Secrets Manager/Azure Key Vault at runtime, so only a reference lives in Git.
- Even a private repo is not a safe place for raw secrets.

Likely follow-up: Sealed Secrets vs External Secrets — what's the trade-off? (Encrypted-in-Git vs referenced-from-an-external-store.)

Push-based vs pull-based continuous delivery — why does GitOps prefer pull?

IF ASKED, SAY THIS
In push CD the pipeline holds cluster credentials and applies changes externally, with no drift correction. In pull CD an in-cluster agent reconciles from Git, so no external system needs cluster credentials, drift is detected and healed automatically, and rollback is a git revert — which is why GitOps standardized on pull.

- Push: a CI pipeline holds cluster credentials and runs kubectl/helm against the cluster. Simple, but the external system has powerful standing access and there's no continuous reconciliation.
- Pull: an in-cluster agent (Argo CD/Flux) pulls from Git and reconciles. No external credentials, built-in drift detection and self-heal, and rollback is a Git revert.
- Pull scales cleanly to many clusters (each runs its own agent).

Likely follow-up: How does pull-based CD reduce credential blast radius? (No CI system holds kubeconfig/admin creds.)

- [ ] Walk through safely draining a node for maintenance.

IF ASKED, SAY THIS
cordon the node (stop new scheduling), drain it (gracefully evict Pods, honoring PodDisruptionBudgets), perform maintenance, then uncordon to allow scheduling again. Drain typically needs --ignore-daemonsets, and --delete-emptydir-data if any Pods use emptyDir.

- cordon marks the node unschedulable so no new Pods land there. drain evicts existing Pods gracefully (respecting PodDisruptionBudgets and graceful termination), rescheduling them elsewhere.
- Do maintenance, then uncordon to make the node schedulable again.
- Use --ignore-daemonsets (DaemonSet Pods are managed per-node) and be careful with --delete-emptydir-data (it discards emptyDir data).

```
kubectl cordon node1
kubectl drain node1 --ignore-daemonsets --delete-emptydir-data
# ...maintenance...
kubectl uncordon node1
```

Likely follow-up: What stops drain from evicting too many replicas at once? (PodDisruptionBudget.)

- [ ] How does a kubeadm cluster upgrade work, and in what order?

IF ASKED, SAY THIS
Upgrade the control plane before the workers, one minor version at a time. On the control plane: bump kubeadm, run kubeadm upgrade apply, then upgrade kubelet/kubectl. On each worker: drain, upgrade kubeadm and kubelet, run kubeadm upgrade node, then uncordon. The kubelet must never be newer than the API server.

- Order: upgrade the control plane first, then the worker nodes — never nodes ahead of the control plane (version skew rules forbid it).
- Control plane: upgrade kubeadm, run kubeadm upgrade apply vX, then upgrade kubelet/kubectl on that node. Repeat for other control-plane nodes (kubeadm upgrade node).
- Workers: drain the node, upgrade kubeadm + kubelet, kubeadm upgrade node, then uncordon. Upgrade one minor version at a time.

Likely follow-up: Can you skip a minor version during upgrade? (No — upgrade sequentially.)

- [ ] What is a PodDisruptionBudget and why does it matter?

IF ASKED, SAY THIS
A PodDisruptionBudget bounds how many Pods of an app can be voluntarily disrupted at once (via minAvailable/maxUnavailable), protecting availability during node drains, autoscaler scale-downs, and rollouts. It doesn't protect against involuntary disruptions like a node crash.

- A PDB limits how many Pods of an application can be voluntarily disrupted at once — via minAvailable or maxUnavailable.
- It applies to voluntary disruptions (node drains, cluster autoscaler scale-down, rollouts), NOT involuntary ones (a node crashing).
- Without a PDB, a drain could evict all replicas of a service simultaneously and cause an outage.

```
spec:
  minAvailable: 2
  selector: { matchLabels: { app: web } }
```

Likely follow-up: A node drain hangs forever — how can a PDB cause that? (minAvailable can't be satisfied, so eviction is blocked.)

- [ ] Explain PriorityClass and preemption.

IF ASKED, SAY THIS

A PriorityClass gives Pods an integer priority so higher-priority Pods schedule first under contention. If a high-priority Pod can't fit, preemption evicts lower-priority Pods to make room. It's used to guarantee capacity for critical services while letting low-priority batch work yield.

- A PriorityClass assigns an integer priority to Pods. Higher-priority Pods schedule ahead of lower ones when resources are scarce.
- Preemption: if a high-priority Pod can't schedule, the scheduler may evict (preempt) lower-priority Pods to make room.
- Use it to protect critical workloads (e.g. control-plane add-ons, payment services) and let batch/best-effort work be preempted.

Likely follow-up: What protects a Pod from being preempted despite low priority? (There's no absolute guarantee — but PDBs and priority tuning help.)

What do ResourceQuota and LimitRange do?

IF ASKED, SAY THIS
ResourceQuota caps a namespace's total resource usage and object counts for fair multi-tenant sharing; LimitRange sets per-container default and min/max requests/limits so workloads get sane defaults and can't over- or under-request. Together they enforce namespace-level governance.

- ResourceQuota caps total resource consumption per namespace (total CPU/memory requests/limits, object counts like Pods/Services). It enforces fair sharing across teams.
- LimitRange sets default and min/max requests/limits per Pod/container in a namespace — so Pods that omit requests get sensible defaults and can't request absurd amounts.
- Together they give per-namespace governance.

Likely follow-up: A Pod is rejected with 'must specify limits' — which object caused it? (A LimitRange or a ResourceQuota requiring limits.)

How do you back up and disaster-recover a Kubernetes cluster?

IF ASKED, SAY THIS
Back up cluster state with regular etcd snapshots stored off-cluster (restorable via etcdctl), and back up application objects and persistent-volume data with Velero (including CSI volume snapshots). With GitOps, workload manifests are already in Git, so recovery is re-syncing plus restoring etcd/PV data for state.

- Cluster state: take regular etcd snapshots (etcdctl snapshot save) and store them off-cluster; you can restore the entire control-plane state from a snapshot.
- Application/PV data: use a tool like Velero to back up namespaces, objects, and persistent volumes (via CSI snapshots) — and to migrate/restore across clusters.
- GitOps helps too: the desired state already lives in Git, so re-creating workloads on a fresh cluster is a re-sync; you still need etcd/PV backups for stateful data.

```
ETCDCTL_API=3 etcdctl snapshot save snap.db
velero backup create prod-backup --include-namespaces prod
```

Likely follow-up: If GitOps holds all manifests, why still back up etcd and PVs? (For stateful data and objects created outside Git.)

Explain Kubernetes admission control and mutating vs validating webhooks.

IF ASKED, SAY THIS
Admission controllers intercept requests after authz and before persistence. Mutating webhooks run first and can modify objects (e.g. inject sidecars, set defaults); validating webhooks run next and only accept or reject. Policy engines like Gatekeeper/Kyverno use these hooks to enforce custom cluster-wide policy.

- After authentication and authorization, admission controllers can modify or reject a request before it's persisted. Mutating webhooks run first and can change the object (inject a sidecar, add default labels); validating webhooks run second and can only accept/reject.
- Policy engines like OPA Gatekeeper and Kyverno plug in as admission webhooks to enforce custom rules (require labels, block :latest, deny privileged Pods).
- Built-in controllers include ResourceQuota, LimitRange, and Pod Security Admission.

Likely follow-up: Which runs first, mutating or validating? (Mutating.) · How would you block all :latest images cluster-wide? (A validating policy in Kyverno/Gatekeeper.)

What is Pod Security Admission and its three profiles?

IF ASKED, SAY THIS
Pod Security Admission enforces the Pod Security Standards per namespace using labels, replacing PodSecurityPolicy. Its three profiles are privileged (no restrictions), baseline (blocks common escalations), and restricted (fully hardened: non-root, no privilege escalation, dropped capabilities, seccomp). Each can run in enforce, audit, or warn mode.

- Pod Security Admission (PSA) enforces the Pod Security Standards at the namespace level via labels. It replaced the removed PodSecurityPolicy.

- Three profiles: privileged (unrestricted), baseline (blocks known privilege escalations), and restricted (hardened — non-root, no privilege escalation, drop capabilities, seccomp).
- Modes: enforce (reject), audit (log), warn (warn) — set per namespace via labels.

```
kubectl label ns prod pod-security.kubernetes.io/enforce=restricted
```

Likely follow-up: How do you enforce 'restricted' on a namespace? (Label it pod-security.kubernetes.io/enforce=restricted.)

- [ ] Kubernetes Secrets aren't encrypted by default — how do you fix that?

IF ASKED, SAY THIS
Secrets are only base64-encoded in etcd by default. Enable encryption at rest via an EncryptionConfiguration on the API server — ideally a KMS provider backed by a cloud KMS/HSM — then re-encrypt existing Secrets. Combine with least-privilege RBAC and, ideally, an external secrets manager.

- By default Secrets are only base64-encoded in etcd — anyone with etcd access or get-secret RBAC can read them.
- Enable encryption at rest with an EncryptionConfiguration on the API server (aescbc/secretbox, or better a KMS provider backed by a cloud KMS/HSM). Then rewrite existing Secrets so they're re-encrypted.
- Also tighten RBAC on Secrets and prefer an external secrets manager.

Likely follow-up: After enabling encryption at rest, how do you encrypt existing Secrets? (kubectl get secrets -A -o yaml | kubectl replace -f - to rewrite them.)

- [ ] How do you secure the software supply chain for container images?

IF ASKED, SAY THIS
Scan images for CVEs in CI and use minimal/distroless bases, sign images with cosign and verify signatures at admission so only trusted images deploy, generate an SBOM, and pin base images by digest from a trusted registry. This chains build-time and deploy-time controls.

- Scan images for CVEs in CI (Trivy/Grype) and fail the build on critical findings; use minimal/distroless bases to shrink attack surface.
- Sign images (cosign) and verify signatures at admission (a Kyverno/Gatekeeper policy or the sigstore policy controller) so only signed, trusted images run.
- Generate an SBOM, pin base images by digest, and pull from a trusted internal registry/proxy.

Likely follow-up: How do you guarantee only signed images run in the cluster? (Admission policy verifying cosign signatures.)

- [ ] What runtime-security controls harden a container (seccomp, AppArmor, capabilities)?

IF ASKED, SAY THIS
Apply a seccomp profile (RuntimeDefault) to limit syscalls, AppArmor/SELinux to confine file/process operations, drop all capabilities and add back only what's required, and set runAsNonRoot, no privilege escalation, and a read-only root filesystem. Add Falco for runtime threat detection. These layer kernel-level defenses under the app.

- seccomp restricts which syscalls a container can make (use the RuntimeDefault profile as a baseline).
- AppArmor/SELinux confine what files/operations a process may perform.
- Drop all Linux capabilities and add back only what's needed; set runAsNonRoot, allowPrivilegeEscalation: false, and readOnlyRootFilesystem in the securityContext. Add runtime threat detection (Falco) for anomalous behavior.

```
securityContext:
  seccompProfile: { type: RuntimeDefault }
  runAsNonRoot: true
  readOnlyRootFilesystem: true
  capabilities: { drop: ["ALL"] }
```

Likely follow-up: What's the quickest baseline seccomp setting? (seccompProfile type RuntimeDefault.)

- [ ] How should a Pod get cloud permissions (e.g. read an S3 bucket) securely?

IF ASKED, SAY THIS
Use workload identity — EKS IRSA or Azure Workload Identity — to federate a Kubernetes ServiceAccount to a scoped cloud IAM role via OIDC, so the Pod gets short-lived, least-privilege credentials with no static keys. Avoid granting the node role to all Pods, and disable ServiceAccount token automounting where unused.

- Don't bake cloud access keys into images or grant the whole node's IAM role to every Pod. Use workload identity: EKS IRSA (IAM Roles for Service Accounts) or Azure Workload Identity federate a Kubernetes ServiceAccount to a scoped cloud role via OIDC.
- Each workload's ServiceAccount gets short-lived, least-privilege credentials automatically — no secrets in the cluster.

- Also set automountServiceAccountToken: false where the API token isn't needed.

Likely follow-up: Why is IRSA better than giving the node role the S3 permission? (Every Pod on the node would inherit it — no least privilege.)

- [ ] How do you restrict east-west (Pod-to-Pod) traffic?

IF ASKED, SAY THIS
Apply NetworkPolicies: start with a default-deny per namespace, then explicitly allow only the required flows (remembering to allow DNS egress). Once a Pod is selected by a policy it's default-deny for that direction. This only works with a policy-enforcing CNI like Calico or Cilium.

- By default all Pods can talk to all Pods. A NetworkPolicy restricts ingress/egress using Pod/namespace selectors; once a policy selects a Pod, that direction becomes default-deny except what's allowed.
- Adopt a default-deny baseline per namespace, then allow only required flows (and don't forget to allow DNS egress to kube-dns on UDP/TCP 53).
- Requires an enforcing CNI (Calico, Cilium); some CNIs ignore NetworkPolicies.

Likely follow-up: You applied a default-deny and DNS broke — why? (You forgot to allow egress to kube-dns on port 53.)

- [ ] How do you generate a manifest fast without writing YAML by hand? (CKAD speed)

IF ASKED, SAY THIS
Generate manifests imperatively with --dry-run=client -o yaml, then tweak and apply — e.g. kubectl create deployment web --image=nginx --dry-run=client -o yaml > d.yaml. This avoids hand-writing YAML and is the key speed technique for CKAD/CKA.

- Use imperative commands with --dry-run=client -o yaml to generate a starting manifest, then edit and apply. This is the single biggest time-saver on the exams.
- kubectl create deployment, kubectl run, kubectl create service, kubectl create configmap/secret all support it.
- Combine with kubectl explain to recall field paths without docs-diving.

```
kubectl create deploy web --image=nginx --replicas=3 --dry-run=client -o yaml > web.yaml
kubectl run tmp --image=busybox --restart=Never --rm -it -- sh
```

Likely follow-up: How do you create a Pod YAML skeleton quickly? (kubectl run p --image=nginx --dry-run=client -o yaml.)

- [ ] How do you extract specific fields with JSONPath and custom-columns?

IF ASKED, SAY THIS
Use -o jsonpath to extract exact fields and -o custom-columns to build a focused table, optionally with --sort-by. For example, list all Pod images or a node's IP directly, avoiding fragile text parsing.

- -o jsonpath pulls exact values (e.g. all Pod names, a node's internal IP). -o custom-columns builds a table of chosen fields.
- Combine with --sort-by to order results. These avoid grep/awk gymnastics.
- kubectl get ... -o wide adds handy columns (node, IP) without JSONPath.

```
kubectl get pods -o jsonpath='{.items[*].spec.containers[*].image}'
kubectl get pods -o custom-columns=NAME:.metadata.name,STATUS:.status.phase
```

Likely follow-up: How would you list every container image running in a namespace? (jsonpath over .items[*].spec.containers[*].image.)

- [ ] How do you debug a running or distroless Pod that has no shell?

IF ASKED, SAY THIS
Use kubectl debug to attach an ephemeral container carrying your tools to the running Pod (sharing its process/network namespaces) — ideal for distroless images with no shell. It can also clone the Pod with a debug container or drop a debug Pod onto a node.

- kubectl debug attaches an ephemeral container (with your tools) to a running Pod, sharing its namespaces — perfect for distroless images with no shell.
- kubectl debug can also create a copy of a Pod with a debug container, or start a debugging Pod on a node.
- For quick checks: kubectl exec into the container, or kubectl logs --previous for a crashed instance.

```
kubectl debug -it mypod --image=busybox --target=app
```

Likely follow-up: Why can't you kubectl exec into a distroless container? (No shell/binaries — use an ephemeral debug container.)

- [ ] How do you switch contexts and namespaces quickly?

IF ASKED, SAY THIS
Use kubectl config use-context to switch clusters and kubectl config set-context --current --namespace=NS to change the default namespace so you don't repeat -n. kubectx/kubens and a k alias with completion make this instant — and always verify your current context before destructive actions.

- kubectl config use-context switches clusters; kubectl config set-context --current --namespace=X changes the default namespace so you stop typing -n.
- The kubectx/kubens tools (or aliases) make this instant. Set alias k=kubectl and enable completion.
- Always confirm your context before a destructive command (kubectl config current-context).

```
kubectl config set-context --current --namespace=payments
alias k=kubectl
```

Likely follow-up: How do you avoid typing -n on every command? (Set the namespace on the current context.)

- [ ] apply vs create vs replace vs patch — when do you use each?

IF ASKED, SAY THIS
apply is declarative and idempotent (three-way merge, safe to re-run); create makes a new object and errors if it exists; replace overwrites the entire object and can fail on immutable fields; patch edits specific fields. Production/GitOps uses apply; create/patch are for quick imperative changes.

- apply is declarative and idempotent — it merges your manifest with the live object (three-way merge) and is the default for GitOps.
- create fails if the object already exists (imperative, one-shot). replace overwrites the whole object (must include all fields; can fail on immutable fields).
- patch changes specific fields (strategic/JSON/merge patch) — handy for a quick targeted edit.

Likely follow-up: Why can replace fail where apply succeeds? (replace overwrites everything and may hit immutable fields.)

- [ ] Give a fast troubleshooting toolkit of one-liners.

IF ASKED, SAY THIS
Reach for kubectl describe and kubectl get events --sort-by=.lastTimestamp for the reason, kubectl logs --previous for crash output, kubectl get -o wide for placement, kubectl top for usage, and kubectl explain / api-resources to recall structure. This toolkit resolves most Pod/scheduling/networking issues fast.

- Start with describe + events for the 'why'; logs (and --previous) for the 'what'; get -o wide for placement.
- kubectl get events --sort-by=.lastTimestamp surfaces recent cluster activity; kubectl top shows resource usage (needs metrics-server).
- kubectl explain recalls field structure; kubectl api-resources lists object kinds and whether they're namespaced.

```
kubectl describe pod P
kubectl logs P --previous
kubectl top pods
kubectl get events --sort-by=.lastTimestamp
```

Likely follow-up: Command to see the most recent cluster events? (kubectl get events --sort-by=.lastTimestamp.)

- [ ] What are CRDs and the Operator pattern?

IF ASKED, SAY THIS
A CRD extends the Kubernetes API with a custom resource type, and an Operator is a controller that watches those resources and reconciles them — encoding operational logic like provisioning, backup, and failover. Together they let complex stateful software be managed declaratively, the same way built-in objects are.

- A CustomResourceDefinition (CRD) extends the Kubernetes API with your own resource type (e.g. a Kafka or Postgres object). It becomes a first-class object you can kubectl get.
- An Operator is a custom controller that watches those custom resources and runs a control loop encoding operational knowledge — provisioning, backups, failover, upgrades — so the software self-manages.
- It's the reconciliation pattern applied to app-specific operations.

Likely follow-up: How is an Operator different from a Helm chart? (Helm templates manifests once; an Operator continuously reconciles and automates day-2 operations.)

CPU throttling vs OOMKill — how do requests/limits cause each?

IF ASKED, SAY THIS
CPU is compressible, so exceeding the CPU limit throttles the container (slower, not killed) — a common hidden cause of latency. Memory is incompressible, so exceeding the memory limit causes an OOMKill (exit 137). Set requests for scheduling and limits to bound impact, and beware overly tight CPU limits masquerading as app slowness.

- CPU is compressible: exceeding the CPU limit throttles the container (it runs slower) — it is NOT killed. This often shows up as unexplained latency.
- Memory is not compressible: exceeding the memory limit triggers an OOMKill (exit 137) — the container is terminated.
- Set requests for scheduling and limits to bound blast radius; requests==limits gives Guaranteed QoS. Too-tight CPU limits create latency that looks like an app bug.

Likely follow-up: A service is mysteriously slow but not crashing — what limit do you suspect? (CPU limit throttling.)

What is the Downward API and when is it useful?

IF ASKED, SAY THIS
The Downward API injects Pod and container metadata (name, namespace, node, labels, resource limits) into the container as env vars or files, so an app can learn its own identity/placement without hardcoding it. It's common for telemetry, logging context, and self-aware configuration.

- The Downward API exposes Pod/container metadata to the container as environment variables or files — things like the Pod name, namespace, node name, labels, annotations, and resource limits.
- Useful for logging/telemetry that needs the Pod identity, or apps that must know their own namespace/node without hardcoding.
- It avoids baking environment-specific values into images.

Likely follow-up: How would a Pod know its own namespace at runtime? (Downward API exposing metadata.namespace.)

What does a service mesh add over plain Kubernetes networking?

IF ASKED, SAY THIS
A service mesh puts a proxy alongside each service to transparently provide mutual TLS, advanced traffic management (canary, retries, timeouts, circuit breaking), and per-service L7 observability — without changing app code. The cost is operational complexity and sidecar overhead, so adopt it only when those capabilities are needed.

- A service mesh (Istio, Linkerd) injects a sidecar proxy (or uses an eBPF/ambient dataplane) beside each Pod to control service-to-service traffic transparently.
- It adds automatic mutual TLS (encryption + identity), fine-grained traffic management (canary/retries/timeouts/circuit breaking), and rich L7 observability (golden signals per service) without app code changes.
- Trade-off: added complexity and sidecar overhead — only adopt when you need those features.

Likely follow-up: What does a mesh give you 'for free' security-wise? (Automatic mTLS between services.)

Name the multi-container Pod patterns and when to use them.

IF ASKED, SAY THIS
Sidecar augments the main container (logging, proxy, exporter); Ambassador proxies the app's outbound connections through a helper container to simplify external access; Adapter transforms the app's output into a standard format for other systems. All exploit the shared Pod network/volume namespace.

- Sidecar: augments the main container for the Pod's lifetime (log shipper, proxy, metrics exporter).
- Ambassador: a proxy container that brokers the main container's connections to the outside (e.g. a local proxy to a database/cache), simplifying the app.
- Adapter: transforms the main container's output into a standard format (e.g. normalizing logs/metrics for a monitoring system).

Likely follow-up: Which pattern would you use to normalize an app's metrics into Prometheus format without changing the app? (Adapter.)

How do you handle deprecated/removed Kubernetes APIs during upgrades?

IF ASKED, SAY THIS
Before upgrading, scan for deprecated/removed APIs (pluto/kubent, API-server deprecation warnings) and migrate manifests to the current apiVersion, since a removed version stops applying after the upgrade. In GitOps this is a config-repo PR bumping apiVersion, done before the cluster upgrade.

- Kubernetes periodically removes deprecated API versions (e.g. old Ingress/PSP APIs). Manifests using a removed version fail to apply after an upgrade.
- Detect ahead of time with tooling (kubectl deprecations plugins, pluto, kubent) and the API server's deprecation warnings; migrate manifests to the current apiVersion.

- In GitOps, this is a PR that updates apiVersion across the config repo before you upgrade the cluster.

Likely follow-up: How do you find manifests using a soon-to-be-removed API version? (Tools like pluto/kubent or API deprecation warnings.)

- [ ] What's operationally different about OpenShift versus vanilla Kubernetes that you'd need to know coming in with a K8s background?

IF ASKED, SAY THIS
OpenShift adds opinionated layers on top of K8s: Routes instead of (or alongside) Ingress for exposing services, Security Context Constraints (SCCs) instead of raw Pod Security Policies/Standards — considerably more restrictive by default (containers don't run as root by default, unlike vanilla K8s where that's often t

OpenShift adds opinionated layers on top of K8s: **Routes** instead of (or alongside) Ingress for exposing services, **Security Context Constraints (SCCs)** instead of raw Pod Security Policies/Standards — considerably more restrictive by default (containers don't run as root by default, unlike vanilla K8s where that's often the path of least resistance), built-in image streams and an integrated registry, and the `oc` CLI as a superset of `kubectl`.

Coming from pure K8s, the SCC model is usually the biggest adjustment — a lot of "works fine on vanilla K8s" images fail on OpenShift until built to not assume root.

- [ ] How would you troubleshoot a pod stuck in CrashLoopBackOff on an OpenShift cluster supporting a build/CI workload?

IF ASKED, SAY THIS
Same fundamentals as vanilla K8s — oc logs (including --previous for the crashed instance's logs), oc describe pod for events (OOMKilled, failed liveness probe, image pull failure), resource limits against actual usage. OpenShift-specific: check whether it's actually an SCC denial masquerading as a crash (the container

Same fundamentals as vanilla K8s — `oc logs` (including `--previous` for the crashed instance's logs), `oc describe pod` for events (OOMKilled, failed liveness probe, image pull failure), resource limits against actual usage.

OpenShift-specific: check whether it's actually an **SCC denial masquerading as a crash** (the container trying to write somewhere it can't due to running as a non-root, arbitrarily-assigned UID — a very common OpenShift-specific gotcha) via `oc get events` and the SCC assigned to the project.

- [ ] How would you size and manage a Kubernetes/OpenShift cluster running CI/build agents specifically — what's different from a typical application workload cluster?

IF ASKED, SAY THIS
Build workloads are typically bursty and resource-heavy per-pod (a compile job might need significant CPU/memory for a short window, then nothing) — cluster autoscaling needs to react fast to queue depth , not just steady-state utilization, with node pools sized for build-agent profiles (often larger, sometimes local S

Build workloads are typically bursty and resource-heavy per-pod (a compile job might need significant CPU/memory for a short window, then nothing) — cluster autoscaling needs to react fast to **queue depth**, not just steady-state utilization, with node pools sized for build-agent profiles (often larger, sometimes local SSD for scratch space) separate from general application node pools. Ephemeral storage management matters more here — a build filling a node's local disk can affect other pods scheduled there if not properly limited.

- [ ] What's your approach to image management — base images, scanning, registry — for containers running R&D build/CI workloads?

IF ASKED, SAY THIS
Standardized, centrally-maintained base images (so many teams aren't each maintaining a slightly-different base with its own patch cadence) rebuilt on a schedule to pick up security patches, integrated with the same scanning discipline as Black Duck/Coverity — scan images in the registry too, since a clean codebase on

Standardized, centrally-maintained base images (so many teams aren't each maintaining a slightly-different base with its own patch cadence) rebuilt on a schedule to pick up security patches, integrated with the same scanning discipline as Black Duck/Coverity — scan images in the registry too, since a clean codebase on a vulnerable base image is still a vulnerable deployment.

Image streams (OpenShift) or a well-organized tagging/retention policy so old, unpatched images don't linger available to accidentally pull.

- [ ] How would you handle persistent storage for a stateful engineering service (like Artifactory or a database backend) running on OpenShift/Kubernetes?

IF ASKED, SAY THIS
PersistentVolumeClaims backed by a storage class matching the workload's actual I/O profile (block storage via CSI for database-like access, not a shared-filesystem class meant for read-many). Backup strategy needs to exist at the application level too — a PVC snapshot alone often isn't application-consistent for a dat

PersistentVolumeClaims backed by a storage class matching the workload's actual I/O profile (block storage via CSI for database-like access, not a shared-filesystem class meant for read-many). Backup strategy needs to exist at the **application level** too — a PVC snapshot alone often isn't application-consistent for a database without coordinating a quiesce/flush first.

For anything mission-critical, test the *restore* path, not just the backup — an untested backup is a hypothesis, not a plan.

- [ ] A team wants to migrate an existing VM-based engineering service to run on OpenShift. What would you actually evaluate before agreeing?

IF ASKED, SAY THIS
Whether the app is genuinely container-friendly (stateless-enough, no weird licensing tied to physical/VM hardware identifiers — a real recurring issue with EDA/engineering tool licensing specifically), what its actual resource profile looks like under real load, storage/networking requirements against what the cluster

Whether the app is genuinely container-friendly (stateless-enough, no weird licensing tied to physical/VM hardware identifiers — a real recurring issue with EDA/engineering tool licensing specifically), what its actual resource profile looks like under real load, storage/networking requirements against what the cluster can provide, and — often the deciding factor — whether the operational win is worth the migration effort versus leaving a stable, low-churn VM-based service alone. Not everything needs to move just because the platform exists.

- [ ] What's your default systematic approach when a pod isn't working?

IF ASKED, SAY THIS
A layered narrowing, not a random poke: (1) kubectl get pods -n ns -o wide — status, restarts, node, IP at a glance; (2) kubectl describe pod — the Events section at the bottom almost always names the proximate cause (scheduling failure, image pull error, probe failure, OOM); (3) kubectl logs [-c container] [--previous] — appl

A layered narrowing, not a random poke: (1) `kubectl get pods -n ns -o wide` — status, restarts, node, IP at a glance; (2) `kubectl describe pod <pod>` — the Events section at the bottom almost always names the proximate cause (scheduling failure, image pull error, probe failure, OOM); (3) `kubectl logs <pod> [-c container] [--previous]` — application-level errors, `--previous` to see the crashed container's last logs before restart wiped them; (4) if the container is up but misbehaving, `kubectl exec -it <pod> -- sh` to inspect from inside; (5) if it can't even start a shell, `kubectl debug` with an ephemeral container. Events narrow the search space before logs do — jumping straight to logs on a pod that's not even scheduled wastes time.

- [ ] Describe the pod scheduling and startup sequence — where can it fail at each step?

IF ASKED, SAY THIS
Scheduled (API server accepts the spec → scheduler binds it to a node — can fail: insufficient resources, taints without matching tolerations, node selectors/affinity with no match, PodDisruptionBudget conflicts) → Pulling image (kubelet pulls the image — can fail: wrong image name/tag, missing imagePullSecrets , registry rate limits/auth

Scheduled (API server accepts the spec → scheduler binds it to a node — can fail: insufficient resources, taints without matching tolerations, node selectors/affinity with no match, PodDisruptionBudget conflicts) → **Pulling image** (kubelet pulls the image — can fail: wrong image name/tag, missing `imagePullSecrets`, registry rate limits/auth) → **Init containers run sequentially** (each must exit 0 before the next starts — a hanging or failing init container blocks the whole pod indefinitely) → **Main containers start** (can crash immediately — bad command, missing config/secret mount, application startup error) → **Readiness probe passes** (only then does the pod receive traffic via Service endpoints — a container can be "Running" but still not receiving traffic if readiness never passes).

- [ ] What's the difference between what you'd check for a liveness probe failure vs a readiness probe failure?

IF ASKED, SAY THIS
A failing readiness probe just removes the pod from Service endpoints — the pod keeps running, so you can still exec / logs into it live and see exactly what state it's in; check if it's still starting up (probe fired before the app was ready — tune initialDelaySeconds) or genuinely unhealthy (a dependency it needs, like a DB connection,

A **failing readiness probe** just removes the pod from Service endpoints — the pod keeps running, so you can still `exec` / `logs` into it live and see exactly what state it's in; check if it's still starting up (probe fired before the app was ready — tune `initialDelaySeconds`) or genuinely unhealthy (a dependency it needs, like a DB connection, is down). A **failing liveness probe** gets the container *killed and restarted* by kubelet — by the time you notice via `kubectl get pods`, the evidence may already be gone, so `kubectl logs --previous` and `kubectl describe`'s Events (which record the liveness failure reason before the restart) are your primary evidence.

- [ ] CrashLoopBackOff — what causes it, and what's the backoff behavior itself?

IF ASKED, SAY THIS
It means the container starts, exits (crashes or the main process just finishes), and kubelet keeps restarting it — with an exponential backoff between attempts (10s, 20s, 40s... capped at 5 minutes) which is why you'll see the status flip between CrashLoopBackOff and Running if you watch it. Root causes are almost always application-leve

It means the container starts, exits (crashes or the main process just finishes), and kubelet keeps restarting it — with an exponential backoff between attempts (10s, 20s, 40s... capped at 5 minutes) which is why you'll see the status flip between `CrashLoopBackOff` and `Running` if you watch it. Root causes are almost always application-level: an uncaught startup exception, a missing required env var/config/secret, a bad entrypoint command, the app failing to bind to the expected port, or a failed dependency check at boot (can't reach DB) with no retry logic. `kubectl logs <pod> --previous` is the first move — you need the crashed container's output, not the fresh one that just restarted.

- [ ] ImagePullBackOff / ErrImagePull — walk through the diagnosis.

IF ASKED, SAY THIS
kubectl describe pod 's Events will show the exact pull error string. Common causes: (1) typo in image name/tag or the tag genuinely doesn't exist; (2) private registry without a correctly referenced imagePullSecrets on the pod spec (or the ServiceAccount it uses); (3) expired registry credentials — very common with ECR, whose tokens expi

`kubectl describe pod` 's Events will show the exact pull error string. Common causes: (1) typo in image name/tag or the tag genuinely doesn't exist; (2) private registry without a correctly referenced `imagePullSecrets` on the pod spec (or the ServiceAccount it uses); (3) expired registry credentials — very common with ECR, whose tokens expire every 12 hours if not refreshed by an automated credential helper; (4) registry rate limiting (Docker Hub anonymous pull limits are a classic culprit); (5) network policy or egress firewall blocking the node from reaching the registry at all, which shows as a timeout rather than an auth error.

- [ ] Pod stuck in Pending — what are you checking, in order?

IF ASKED, SAY THIS
kubectl describe pod Events will say exactly why the scheduler can't place it, but the usual suspects in order of frequency: (1) insufficient CPU/memory on every node relative to the pod's requests — kubectl describe nodes or kubectl top nodes to confirm; (2) no node matches nodeSelector / affinity rules; (3) a taint on every candidate no

`kubectl describe pod` Events will say exactly why the scheduler can't place it, but the usual suspects in order of frequency: (1) insufficient CPU/memory on every node relative to the pod's `requests` — `kubectl describe nodes` or `kubectl top nodes` to confirm; (2) no node matches `nodeSelector` / `affinity` rules; (3) a taint on every candidate node with no matching `toleration` on the pod; (4) an unbound PVC the pod depends on (if using `volumeBindingMode: WaitForFirstConsumer`, the PV won't even provision until a pod needs it, and if provisioning fails the pod stays Pending); (5) a PodDisruptionBudget or ResourceQuota blocking scheduling in that namespace.

- [ ] A pod shows OOMKilled — what's actually happening at the kernel level, and how do you size the fix?

IF ASKED, SAY THIS
The container exceeded its memory limit , and the kernel cgroup OOM killer terminated it (exit code 137 = 128 + SIGKILL 9) — this is enforced by the kernel, not Kubernetes politely asking it to stop, so there's no graceful shutdown. Check kubectl describe pod for Last State: Terminated, Reason: OOMKilled , then look at actual usage trend

The container exceeded its memory `limit`, and the kernel cgroup OOM killer terminated it (exit code 137 = 128 + SIGKILL 9) — this is enforced by the kernel, not Kubernetes politely asking it to stop, so there's no graceful shutdown. Check `kubectl describe pod` for `Last State: Terminated, Reason: OOMKilled`, then look at actual usage trend via `kubectl top pod` or your metrics stack (Prometheus `container_memory_working_set_bytes`) over time — don't just bump the limit blindly. Distinguish a genuine memory leak (usage climbs monotonically until killed, points to an app bug) from a legitimately under-provisioned limit (usage is stable but above the configured ceiling, e.g. during a traffic spike or a batch job's peak).

GotchaUnlike CPU, memory limits are hard — a container can't "wait" for memory the way it can be throttled for CPU. There's no memory throttling, only OOM kill.

- [ ] Pod shows Evicted — how is this different from OOMKilled, and what triggers it?

IF ASKED, SAY THIS
Eviction is node-level kubelet action, not a container-level cgroup kill — it happens when the node itself is under resource pressure (memory, disk, PID) and kubelet proactively evicts entire pods (starting with the lowest-priority / most-over-their-requests ones) to protect node stability, rather than the kernel reactively killing one co

Eviction is **node-level** kubelet action, not a container-level cgroup kill — it happens when the node itself is under resource pressure (memory, disk, PID) and kubelet proactively evicts entire pods (starting with the lowest-priority / most-over-their-requests ones) to protect node stability, rather than the kernel reactively killing one container that individually exceeded its own limit. Check `kubectl describe node` for pressure conditions (`MemoryPressure`, `DiskPressure`, `PIDPressure`) and the eviction message in `kubectl get pod -o yaml` under `status.reason`, which names the specific resource that triggered it.

- [ ] An init container is stuck and the pod never starts the main containers — how do you debug it?

IF ASKED, SAY THIS
Init containers run strictly sequentially and each must exit 0 before the next (or the main containers) start — a hung or endlessly-retrying init container (commonly a "wait for dependency" script polling a DB or another service) blocks the pod indefinitely with status Init:0/2 style output. You must target logs at the specific init conta

```
kubectl logs <pod> -c <init-container-name>
kubectl describe pod <pod>   # shows which init container is currently running
```

Init containers run strictly sequentially and each must exit 0 before the next (or the main containers) start — a hung or endlessly-retrying init container (commonly a "wait for dependency" script polling a DB or another service) blocks the pod indefinitely with status `Init:0/2` style output. You must target logs at the specific init container by name with `-c`, since `kubectl logs` without it targets the (not-yet-running) main container by default.

- [ ] A node shows NotReady — what's your checklist?

IF ASKED, SAY THIS

kubectl describe node — check Conditions (Ready, MemoryPressure, DiskPressure, PIDPressure, NetworkUnavailable) and their reasons/messages. Is kubelet actually running on that node? (systemctl status kubelet if you have node access, or check for a heartbeat gap in kubectl get node -o yaml under status.conditions[].lastHeartbeatTim

- `kubectl describe node <node>` — check Conditions (Ready, MemoryPressure, DiskPressure, PIDPressure, NetworkUnavailable) and their reasons/messages.
- Is kubelet actually running on that node? (`systemctl status kubelet` if you have node access, or check for a heartbeat gap in `kubectl get node -o yaml` under `status.conditions[].lastHeartbeatTime`).
- Network connectivity between the node and control plane — security group/firewall changes, VPN/Direct Connect drop for hybrid setups, or a CNI plugin crash on that node.
- Disk pressure from log/image buildup — `df -h` on the node if reachable; unreclaimed container images are a very common silent cause.
- For managed node groups (EKS), check the underlying ASG/EC2 instance health and any recent node group update or spot interruption.

- [ ] Explain node resource pressure conditions and what kubelet does about each.

IF ASKED, SAY THIS
MemoryPressure , DiskPressure , and PIDPressure are set when a node crosses configured eviction thresholds. Kubelet responds by evicting pods to reclaim the resource, prioritizing eviction by how far a pod's actual usage exceeds its requests (a pod using far more than it requested is evicted before one at or under its request), and within

`MemoryPressure` , `DiskPressure` , and `PIDPressure` are set when a node crosses configured eviction thresholds. Kubelet responds by evicting pods to reclaim the resource, prioritizing eviction by how far a pod's actual usage exceeds its `requests` (a pod using far more than it requested is evicted before one at or under its request), and within that, by Pod Priority class. `DiskPressure` specifically can also trigger kubelet's image garbage collection (removing unused container images) before it resorts to evicting pods.

- [ ] How do taints and tolerations cause silent scheduling failures, and how do you debug them?

IF ASKED, SAY THIS
A taint on a node (kubectl taint nodes node1 key=value:NoSchedule) repels pods unless they declare a matching toleration . The classic silent failure: someone taints a node pool for a specific workload (e.g. GPU nodes tainted nvidia.com/gpu=true:NoSchedule), and an unrelated deployment's pods simply never schedule there — but since tai

```
kubectl describe node <node> | grep Taints
kubectl get pod <pod> -o yaml | grep -A5 tolerations
```

A taint on a node (`kubectl taint nodes node1 key=value:NoSchedule`) repels pods unless they declare a matching `toleration` . The classic silent failure: someone taints a node pool for a specific workload (e.g. GPU nodes tainted `nvidia.com/gpu=true:NoSchedule`), and an unrelated deployment's pods simply never schedule there — but since taints don't affect nodes without the taint, pods just pile up Pending only if *every* viable node is tainted or otherwise unschedulable, which can look like "random" scheduling failures if your cluster has a mix of tainted and untainted nodes and you don't immediately realize capacity on the untainted ones ran out.

- [ ] How would you investigate a node that's healthy per Kubernetes but the pods on it have degraded performance?

IF ASKED, SAY THIS
Kubernetes' node "Ready" condition is a fairly coarse kubelet heartbeat — it doesn't capture CPU steal time on a noisy-neighbor-affected instance, disk I/O saturation, network throughput degradation, or clock skew. Go outside kubectl: node-level metrics from your observability stack (node_exporter/Datadog agent) for CPU steal %, disk iowa

Kubernetes' node "Ready" condition is a fairly coarse kubelet heartbeat — it doesn't capture CPU steal time on a noisy-neighbor-affected instance, disk I/O saturation, network throughput degradation, or clock skew. Go outside kubectl: node-level metrics from your observability stack (node_exporter/Datadog agent) for CPU steal %, disk iowait, network errors/drops; for cloud instances, check the provider's instance-health/status-check dashboards (AWS EC2 status checks can flag a degraded underlying host that Kubernetes has no visibility into at all); and compare whether the issue is isolated to pods on this one node vs. cluster-wide, which tells you node-specific vs. application/dependency-wide.

- [ ] A pod can't resolve a Service's DNS name. Walk the diagnosis from the pod outward.

IF ASKED, SAY THIS
(1) Confirm the pod's /etc/resolv.conf points at the cluster DNS service IP (usually CoreDNS's ClusterIP) with the right search domains — a custom dnsPolicy can break this. (2) Check CoreDNS pods themselves are Running and not CrashLooping — a common cause is CoreDNS OOMKilled under load with too-low memory limits. (3) Check CoreDNS logs

```
kubectl exec -it <pod> -- nslookup my-service.my-namespace.svc.cluster.local
kubectl exec -it <pod> -- cat /etc/resolv.conf
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns
```

(1) Confirm the pod's `/etc/resolv.conf` points at the cluster DNS service IP (usually CoreDNS's ClusterIP) with the right search domains — a custom `dnsPolicy` can break this. (2) Check CoreDNS pods themselves are Running and not CrashLooping — a common cause is CoreDNS OOMKilled under load with too-low memory limits. (3) Check CoreDNS logs for `SERVFAIL` s pointing at upstream resolver issues if it's an external

name. (4) Confirm the target Service/Endpoints actually exist (`kubectl get endpoints my-service` — if empty, DNS will resolve the Service's ClusterIP fine but connections will fail, which is really an endpoints problem, not DNS).

- [ ] DNS resolves but the connection times out or is refused — what's next?

IF ASKED, SAY THIS
This isolates the problem away from DNS and toward Service/Endpoints/NetworkPolicy: kubectl get endpoints — if it's empty, the Service's label selector doesn't match any pod's labels (a classic typo/mismatch bug), so the Service exists and resolves but routes nowhere; if endpoints exist but connections still fail, check for a Ne

This isolates the problem away from DNS and toward Service/Endpoints/NetworkPolicy: `kubectl get endpoints <service>` — if it's empty, the Service's label selector doesn't match any pod's labels (a classic typo/mismatch bug), so the Service exists and resolves but routes nowhere; if endpoints exist but connections still fail, check for a `NetworkPolicy` denying traffic between the calling and target namespaces/pods (default-deny policies are a common source of "worked yesterday, broken today" after a security team rolls one out), and check the target pod's own readiness (unready pods are removed from Endpoints automatically).

- [ ] Explain how kube-proxy actually routes Service traffic — iptables mode vs IPVS mode.

IF ASKED, SAY THIS
kube-proxy watches Services/Endpoints and programs rules so traffic to a Service's ClusterIP gets DNAT'd to one of its backing pod IPs. In iptables mode (long-time default), it writes a chain of iptables rules per Service — this becomes a genuine performance/scale problem past a few thousand Services since rule evaluation is roughly linea

kube-proxy watches Services/Endpoints and programs rules so traffic to a Service's ClusterIP gets DNAT'd to one of its backing pod IPs. In **iptables mode** (long-time default), it writes a chain of iptables rules per Service — this becomes a genuine performance/scale problem past a few thousand Services since rule evaluation is roughly linear. **IPVS mode** uses the kernel's IP Virtual Server, backed by hash tables, giving O(1) lookup regardless of Service count and additional load-balancing algorithms (round-robin, least-connection) beyond iptables' random selection — the standard recommendation for larger clusters. When Service routing looks broken cluster-wide (not just one Service), checking kube-proxy's own pod health/logs on the affected node(s) is a step people skip.

- [ ] How do you debug a NetworkPolicy that's blocking traffic you expect to be allowed?

IF ASKED, SAY THIS
Remember NetworkPolicies are additive/allow-only and namespace-scoped — once any policy selects a pod, that pod's traffic (in whichever direction, ingress/egress, the policy covers) defaults to deny except what's explicitly allowed, and multiple policies on the same pod are OR'd together, not AND'd. Debugging: (1) kubectl get networkpolic

Remember NetworkPolicies are **additive/allow-only and namespace-scoped** — once any policy selects a pod, that pod's traffic (in whichever direction, ingress/egress, the policy covers) defaults to deny except what's explicitly allowed, and multiple policies on the same pod are OR'd together, not AND'd. Debugging: (1) `kubectl get networkpolicy -n <ns>` and read each one's `podSelector` to see which actually target the pod in question; (2) check whether the source pod's namespace/labels match what the policy's `from` / `ingress` rules expect — a missing `namespaceSelector` alongside a `podSelector` is a common miss, since by default an ingress rule with only a podSelector matches pods in the *same* namespace as the policy; (3) remember NetworkPolicy enforcement itself depends on the CNI plugin supporting it — some CNIs silently no-op policies they don't support, which is worth confirming for the specific CNI in play.

- [ ] Intermittent connection resets between pods, not constant failures — what do you suspect?

IF ASKED, SAY THIS
Intermittent (not total) failure usually points at something probabilistic rather than fully broken: conntrack table exhaustion on the node under high connection churn (check nf_conntrack metrics/sysctls — net.netfilter.nf_conntrack_max), a specific unhealthy pod behind a Service still receiving some traffic due to a readiness-probe/endp

Intermittent (not total) failure usually points at something probabilistic rather than fully broken: conntrack table exhaustion on the node under high connection churn (check `nf_conntrack` metrics/sysctls — `net.netfilter.nf_conntrack_max`), a specific unhealthy pod behind a Service still receiving some traffic due to a readiness-probe/endpoint-propagation lag, DNS caching a stale/removed pod IP briefly after a rolling update, or (for a service mesh) sidecar proxy connection pool/circuit-breaker settings tripping under load. Correlate the reset pattern against deploy/scaling events and node-level conntrack metrics before assuming it's an application bug.

- [ ] A PVC is stuck Pending — what's the diagnosis path?

IF ASKED, SAY THIS
kubectl describe pvc Events usually state it directly: no StorageClass matches the request, the StorageClass's provisioner is misconfigured or its controller pod is down (e.g. the EBS CSI driver pods crashed), the requested size/access mode isn't satisfiable by any available PV (for static provisioning), or — very common with volumeBindin

```
kubectl describe pvc <name> -n ns
kubectl get storageclass
kubectl describe storageclass <name>
```

`kubectl describe pvc` Events usually state it directly: no StorageClass matches the request, the StorageClass's provisioner is misconfigured or its controller pod is down (e.g. the EBS CSI driver pods crashed), the requested size/access mode isn't satisfiable by any available PV (for static

provisioning), or — very common with `volumeBindingMode: WaitForFirstConsumer` — the PVC is deliberately waiting for a pod to be scheduled first, so it looks "stuck" but is actually just waiting on the pod that references it to also be schedulable (a chicken-and-egg debugging trap if you're only looking at the PVC in isolation).

- [ ] A pod is stuck in ContainerCreating for a long time with a volume mount error — what's happening?

IF ASKED, SAY THIS

kubectl describe pod Events will show something like "Unable to attach or mount volumes" or a specific CSI error. Common causes: (1) the EBS/EFS/disk volume is already attached to a different node (common after a StatefulSet pod reschedules to a new node before the old attachment fully detaches — an AWS API-level "volume is in use" error)

`kubectl describe pod` Events will show something like "Unable to attach or mount volumes" or a specific CSI error. Common causes: (1) the EBS/EFS/disk volume is already attached to a different node (common after a StatefulSet pod reschedules to a new node before the old attachment fully detaches — an AWS API-level "volume is in use" error); (2) the node and volume are in different Availability Zones (EBS volumes are AZ-local — if the scheduler places the pod in a different AZ than a statically-provisioned EBS volume, it can never mount, and this specifically is why `WaitForFirstConsumer` binding mode exists, to make the scheduler AZ-aware at bind time); (3) CSI driver pods themselves unhealthy on that node; (4) IAM/IRSA permissions missing for the CSI driver to call the cloud API.

- [ ] What's the practical difference between ReadWriteOnce , ReadWriteMany , and ReadOnlyMany , and how does it bite people?

IF ASKED, SAY THIS

RWO — mountable read-write by pods on a single node at a time (note: not literally single-pod in newer Kubernetes with RWOP for that stricter guarantee, but effectively single-node in practice for most block storage like EBS). RWX — read-write from multiple nodes simultaneously, needed for genuinely shared storage but only supported by ne

`RWO` — mountable read-write by pods on a *single node* at a time (note: not literally single-pod in newer Kubernetes with RWOP for that stricter guarantee, but effectively single-node in practice for most block storage like EBS). `RWX` — read-write from multiple nodes simultaneously, needed for genuinely shared storage but only supported by network filesystems (EFS, NFS, Azure Files), not block storage like EBS. `ROX` — read-only from multiple nodes. The classic production bite: trying to scale a Deployment using an EBS-backed RWO PVC to multiple replicas — the second pod's volume mount hangs/fails because the volume is already attached read-write to the node running the first pod. If you need genuinely shared read-write storage across replicas, you need an RWX-capable backend (EFS), not just a bigger EBS volume.

- [ ] How would you tell if etcd is the bottleneck for a slow/unresponsive cluster?

IF ASKED, SAY THIS

Symptoms: kubectl commands hang or time out, API server latency spikes, leader elections thrashing. Check etcd metrics if exposed (commonly via the API server's own metrics or a dedicated etcd exporter): etcd_disk_wal_fsync_duration_seconds and etcd_disk_backend_commit_duration_seconds — etcd is extremely latency-sensitive to disk write s

Symptoms: `kubectl` commands hang or time out, API server latency spikes, leader elections thrashing. Check etcd metrics if exposed (commonly via the API server's own metrics or a dedicated etcd exporter): `etcd_disk_wal_fsync_duration_seconds` and `etcd_disk_backend_commit_duration_seconds` — etcd is extremely latency-sensitive to disk write speed since every write is fsync'd before ack, so slow/noisy-neighbor disks are the most common root cause; `etcd_server_leader_changes_seen_total` climbing indicates instability, often from network latency between etcd members exceeding election timeouts; and raw etcd DB size approaching its (typically 8GB) quota, which throws the whole cluster into a read-only "space quota exceeded" alarm state until compacted/defragmented. On managed EKS/AKS/GKE you don't operate etcd directly, but you should still know these are the levers, since it shapes what you can and can't do (e.g., asking the cloud provider for a control-plane scale-up vs. it being entirely opaque).

- [ ] The API server is returning 5xxs / timing out under load. What's your triage?

IF ASKED, SAY THIS

Check API server metrics: apiserver_request_duration_seconds by verb/resource to see if it's uniformly slow or concentrated on one resource type (a controller with a bad watch/list pattern hammering one resource is a classic cause — e.g. a misbehaving operator doing full LIST instead of using informers/watches efficiently); apiserver_curr

Check API server metrics: `apiserver_request_duration_seconds` by verb/resource to see if it's uniformly slow or concentrated on one resource type (a controller with a bad watch/list pattern hammering one resource is a classic cause — e.g. a misbehaving operator doing full LIST instead of using informers/watches efficiently); `apiserver_current_inflight_requests` against configured `--max-requests-inflight` / `--max-mutating-requests-inflight` limits, since the API server will start rejecting with 429s once saturated (a protective mechanism, not itself the root cause); and correlate with etcd health, since API server latency very often just reflects etcd latency underneath it. For managed control planes, this is largely the cloud provider's problem once you've ruled out "misbehaving client hammering the API" on your side.

- [ ] Pods are stuck Pending even though nodes clearly have capacity. Scheduler or controller-manager issue?

IF ASKED, SAY THIS

Check whether the scheduler is even running/healthy first (kubectl get pods -n kube-system -l component=kube-scheduler on self-managed control planes, or its equivalent leader-election lease) — a crashed or leader-election-stuck scheduler means nothing gets bound to nodes at all regardless of capacity, which looks identical to a scheduli

Check whether the **scheduler** is even running/healthy first (`kubectl get pods -n kube-system -l component=kube-scheduler` on self-managed control planes, or its equivalent leader-election lease) — a crashed or leader-election-stuck scheduler means nothing gets bound to nodes at all regardless of capacity, which looks identical to a scheduling-constraint problem from the outside until you check scheduler health directly. If the

scheduler is healthy, re-examine constraints (affinity/taints/quotas covered above) rather than assuming "capacity" is purely CPU/memory — PodDisruptionBudgets and admission webhooks can also block binding in ways that look like scheduling failures but aren't resource-related at all.

- [ ] How do ephemeral debug containers work, and when do you reach for them over kubectl exec ?

IF ASKED, SAY THIS
kubectl exec only works if the target container has a shell and is actually running. kubectl debug attaches a temporary ephemeral container to an already-running pod, sharing its process namespace (with --target) — useful for distroless/minimal images with no shell at all, or for inspecting a container's process/network namespace using a

```
kubectl debug -it <pod> --image=busybox --target=<container>
```

`kubectl exec` only works if the target container has a shell and is actually running. `kubectl debug` attaches a temporary **ephemeral container** to an already-running pod, sharing its process namespace (with `--target`) — useful for distroless/minimal images with no shell at all, or for inspecting a container's process/network namespace using a fully-featured debug image (like a toolbox with `curl` , `tcpdump` , `strace`) without rebuilding or redeploying the actual application image. `kubectl debug node/<node>` similarly gives you a privileged debugging pod with the host's root filesystem mounted, for node-level investigation without SSH access.

- [ ] When would you reach for crictl instead of kubectl , and why does it exist?

IF ASKED, SAY THIS
crictl talks directly to the container runtime (containerd/CRI-O) via the CRI socket on a node — it operates one level below Kubernetes, showing containers the runtime knows about regardless of whether the kubelet/API server can currently see or report them correctly. It's the right tool when the problem might be between kubelet and the r

`crictl` talks directly to the container runtime (containerd/CRI-O) via the CRI socket on a node — it operates one level below Kubernetes, showing containers the runtime knows about regardless of whether the kubelet/API server can currently see or report them correctly. It's the right tool when the problem might be *between* kubelet and the runtime (kubelet reports a pod as stuck/unknown state but you need to know what the runtime itself thinks is running), or when you're debugging kubelet/runtime issues directly on a node via SSH — `crictl ps` , `crictl logs` , `crictl inspect` mirror Docker CLI-style commands but at the CRI level.

- [ ] How do you correlate a Kubernetes event with what actually happened on the node's kubelet?

IF ASKED, SAY THIS
kubectl get events --sort-by=.lastTimestamp -n ns gives the cluster-level view, but events are also rate-limited/deduplicated and expire (default ~1h retention in etcd) — for deeper node-level detail, SSH to the node and check journalctl -u kubelet (or -u containerd), which has the kubelet's own detailed logs including things that never

`kubectl get events --sort-by=.lastTimestamp -n ns` gives the cluster-level view, but events are also rate-limited/deduplicated and expire (default ~1h retention in etcd) — for deeper node-level detail, SSH to the node and check `journalctl -u kubelet` (or `-u containerd`), which has the kubelet's own detailed logs including things that never surface as a Kubernetes Event at all (e.g. detailed CNI plugin errors, cgroup manager errors). This is the step people skip when a described Event is vague ("failed to create pod sandbox") and the real detail is only in kubelet's own logs.

- [ ] Explain requests vs limits precisely, including what happens at each boundary.

IF ASKED, SAY THIS
requests is what the scheduler reserves and guarantees — it's the basis for bin-packing decisions and Quality of Service class. limits is the hard ceiling: for CPU, exceeding it means the container is throttled (CFS bandwidth control — it doesn't get killed, just slowed, which shows as increased latency, not crashes); for memory, exceedin

`requests` is what the scheduler reserves and guarantees — it's the basis for bin-packing decisions and Quality of Service class. `limits` is the hard ceiling: for CPU, exceeding it means the container is **throttled** (CFS bandwidth control — it doesn't get killed, just slowed, which shows as increased latency, not crashes); for memory, exceeding it means **OOMKilled** immediately, since memory can't be throttled the way CPU can. QoS classes derive from the request/limit relationship: `Guaranteed` (requests == limits on every container, for both CPU and memory), `Burstable` (requests set but below limits, or only some resources set), `BestEffort` (neither set at all) — and QoS class directly determines eviction priority order under node pressure (BestEffort evicted first, Guaranteed last).

- [ ] An HPA isn't scaling even though CPU is clearly high. What do you check?

IF ASKED, SAY THIS
kubectl describe hpa 's Conditions/Events directly say why (this is usually enough to answer the question in one command): (1) metrics-server not running or not reachable, so HPA has no current metric value at all; (2) the target Deployment's pods don't have CPU requests set — HPA's percentage-based CPU target is calculated against reques

```
kubectl describe hpa <name>
kubectl get --raw /apis/metrics.k8s.io/v1beta1/pods
```

`kubectl describe hpa` 's Conditions/Events directly say why (this is usually enough to answer the question in one command): (1) metrics-server not running or not reachable, so HPA has no current metric value at all; (2) the target Deployment's pods don't have CPU `requests` set — HPA's percentage-based CPU target is calculated against requests, and without a request there's no baseline to compute a percentage from, so it silently

can't scale on CPU; (3) already at `maxReplicas`; (4) a stabilization window (`behavior.scaleUp.stabilizationWindowSeconds`) delaying the scale-up decision even though the metric has crossed threshold; (5) the HPA is targeting the wrong object/wrong metric name entirely (a copy-paste error from another manifest).

- [ ] What's the difference between HPA and VPA, and why is running both on the same workload dangerous by default?

> **IF ASKED, SAY THIS**
> HPA (Horizontal Pod Autoscaler) changes the number of pod replicas based on observed metrics. VPA (Vertical Pod Autoscaler) changes a pod's requests / limits — in its default "Auto"/"Recreate" update mode, it does this by evicting and recreating the pod with new resource values. Running both, targeting the same metric (e.g. both reacting

HPA (Horizontal Pod Autoscaler) changes the *number* of pod replicas based on observed metrics. **VPA** (Vertical Pod Autoscaler) changes a pod's `requests` / `limits` — in its default "Auto"/"Recreate" update mode, it does this by evicting and recreating the pod with new resource values. Running both, targeting the same metric (e.g. both reacting to CPU) on the same workload, can create a feedback fight: VPA resizes requests, which changes the CPU utilization percentage HPA is calculating from (since HPA's percentage is against requests), triggering an HPA scaling decision, which changes replica count, which changes per-pod load, which VPA reacts to again. The supported pattern is either HPA on CPU/memory + VPA in recommendation-only mode (no auto-apply) for visibility, or HPA on a custom/external metric while VPA governs a genuinely different resource dimension.

- [ ] A namespace's ResourceQuota is blocking new deployments, but usage looks like it's under the limit — why?

> **IF ASKED, SAY THIS**
> ResourceQuota commonly counts against requests sum, not actual usage — so even if real utilization is low, if the sum of every pod's requested CPU/memory in that namespace already hits the quota ceiling, new pods are rejected at admission regardless of how idle those existing pods actually are. Also check whether the quota tracks object c

ResourceQuota commonly counts against `requests` sum, not actual usage — so even if real utilization is low, if the sum of every pod's *requested* CPU/memory in that namespace already hits the quota ceiling, new pods are rejected at admission regardless of how idle those existing pods actually are. Also check whether the quota tracks object counts (`pods`, `services`, `persistentvolumeclaims`) separately from compute — a namespace can be well under its CPU/memory quota but blocked entirely because it hit a `count/pods` limit, which is easy to miss if you only checked the compute numbers. `kubectl describe resourcequota -n ns` shows both used and hard limits for every tracked dimension at once.

- [ ] SCENARIO Scenario: a rolling deploy is causing 502s for ~10 seconds on every pod replacement. Root-cause and fix.

> **IF ASKED, SAY THIS**
> Classic cause: the pod is removed from the Service's Endpoints only after it starts terminating, but the container process may stop accepting connections (or the load balancer/proxy hasn't yet noticed the endpoint removal) slightly before traffic fully stops being routed to it — a race between "endpoint deregistration propagation" and "co

Classic cause: the pod is removed from the Service's Endpoints only *after* it starts terminating, but the container process may stop accepting connections (or the load balancer/proxy hasn't yet noticed the endpoint removal) slightly before traffic fully stops being routed to it — a race between "endpoint deregistration propagation" and "container shutdown." Fix pattern: add a `preStop` hook that sleeps a few seconds (`sleep 5`) before the container actually receives SIGTERM, giving kube-proxy/Endpoints/load-balancer time to propagate the removal before the pod stops accepting connections, combined with `terminationGracePeriodSeconds` large enough to cover that sleep plus actual graceful shutdown of in-flight requests.

- [ ] SCENARIO Scenario: one specific pod is randomly killed and restarted every few hours with no OOM/crash evidence in its own logs. What do you suspect and how do you confirm?

> **IF ASKED, SAY THIS**
> Suspect the liveness probe itself is the killer, not an application crash — check kubectl describe pod Events for "Liveness probe failed" entries around the restart times, and check the probe's own configuration for a too-aggressive timeoutSeconds / periodSeconds / failureThreshold relative to the app's actual response time under periodic

Suspect the **liveness probe** itself is the killer, not an application crash — check `kubectl describe pod` Events for "Liveness probe failed" entries around the restart times, and check the probe's own configuration for a too-aggressive `timeoutSeconds` / `periodSeconds` / `failureThreshold` relative to the app's actual response time under periodic load (e.g. a GC pause, a cron-triggered batch job sharing the container, or a connection-pool exhaustion moment that makes the health endpoint respond slowly but the app is otherwise fine). Confirm by correlating restart timestamps against any periodic internal job schedule and against the probe's configured thresholds — not by staring at application logs, which won't show anything since the app itself wasn't actually broken.

- [ ] SCENARIO Scenario: cluster-wide, pods everywhere start failing to pull images at the same time. What's your first hypothesis and how do you confirm it fast?

> **IF ASKED, SAY THIS**
> A cluster-wide, simultaneous failure points away from any single app/image and toward something shared: registry outage or auth expiry (e.g. an ECR token/credential helper that expired cluster-wide, or a Docker Hub rate limit hit because every node pulls anonymously), a DNS resolution failure to the registry hostname, or an egress network

A cluster-wide, simultaneous failure points away from any single app/image and toward something shared: registry outage or auth expiry (e.g. an ECR token/credential helper that expired cluster-wide, or a Docker Hub rate limit hit because every node pulls anonymously), a DNS resolution failure to the registry hostname, or an egress network path change (firewall/proxy/NAT gateway) blocking registry access for every node at once. Confirm fast by picking any two unrelated pods' `describe` output — if the error string is identical across unrelated images/namespaces, that all but confirms

a shared infrastructure cause rather than coincidental unrelated app issues, and you pivot straight to registry/network/credential status instead of debugging individual workloads.

- [ ] How would you approach service mesh adoption (if at all) for a microservices architecture on ECS/EKS?

IF ASKED, SAY THIS
A service mesh (App Mesh, or Istio/Linkerd on EKS) adds real value for consistent mTLS between services, fine-grained traffic control (canary routing, circuit breaking) at the network layer, and unified cross-service observability — but it's genuine operational complexity that isn't justified for a handful of services. Reasonable default:

A service mesh (App Mesh, or Istio/Linkerd on EKS) adds real value for consistent mTLS between services, fine-grained traffic control (canary routing, circuit breaking) at the network layer, and unified cross-service observability — but it's genuine operational complexity that isn't justified for a handful of services.

Reasonable default: don't reach for one until the number of services and the specific cross-cutting need — not "microservices" as a buzzword — actually justifies the operational cost of running and understanding it.

- [ ] What's your approach to container image security for a platform running many microservices?

IF ASKED, SAY THIS
Standardized, minimal base images (distroless/slim variants reduce attack surface) maintained centrally rather than each team choosing its own, scanning integrated into CI/CD and the registry (ECR scanning), and running containers as non-root by default — worth enforcing as a standard even on ECS/EKS, where it isn't forced by the platform

Standardized, minimal base images (distroless/slim variants reduce attack surface) maintained centrally rather than each team choosing its own, scanning integrated into CI/CD and the registry (ECR scanning), and running containers as **non-root by default** — worth enforcing as a standard even on ECS/EKS, where it isn't forced by the platform the way it is on OpenShift.

- [ ] How would you design service-to-service communication and discovery for a microservices architecture on ECS/EKS?

IF ASKED, SAY THIS
ECS Service Connect or Cloud Map for service discovery on ECS, Kubernetes-native DNS-based discovery on EKS — either way, services discover each other by logical name, never hardcoded IPs, so scaling/rescheduling doesn't break connectivity. For synchronous calls, an internal ALB or the service mesh (if adopted) handles load balancing and

ECS Service Connect or Cloud Map for service discovery on ECS, Kubernetes-native DNS-based discovery on EKS — either way, services discover each other by logical name, never hardcoded IPs, so scaling/rescheduling doesn't break connectivity. For synchronous calls, an internal ALB or the service mesh (if adopted) handles load balancing and can add retry/timeout/circuit-breaking consistently rather than each service reinventing its own.

- [ ] How would you decide whether a given piece of functionality should be a new microservice versus added to an existing one?

IF ASKED, SAY THIS
Genuine independent scaling/deployment needs, a clear bounded-context boundary, or a distinct reliability/security profile are real reasons to split. "It felt like it should be its own service" without one of those concrete drivers usually just adds operational overhead (another thing to deploy, monitor, secure) without a corresponding be

Genuine independent scaling/deployment needs, a clear bounded-context boundary, or a distinct reliability/security profile are real reasons to split. "It felt like it should be its own service" without one of those concrete drivers usually just adds operational overhead (another thing to deploy, monitor, secure) without a corresponding benefit — premature microservice-ing is a common architecture mistake worth explicitly guarding against.

- [ ] What's Helm, and how would you use it to manage application deployments on EKS specifically?

IF ASKED, SAY THIS
Helm is Kubernetes' package manager — a chart packages a set of Kubernetes manifests (Deployments, Services, ConfigMaps, etc.) as a templated, versioned, reusable unit, with values overridden per environment rather than maintaining separate copies of raw YAML per environment. On EKS specifically: charts typically live in a chart repositor

Helm is Kubernetes' package manager — a **chart** packages a set of Kubernetes manifests (Deployments, Services, ConfigMaps, etc.) as a templated, versioned, reusable unit, with values overridden per environment rather than maintaining separate copies of raw YAML per environment.

On EKS specifically: charts typically live in a chart repository (or a Git repo with a defined structure), deployed via `helm upgrade --install` from CI/CD rather than manually, with `helm rollback` giving a fast, tested rollback path tied to release history — genuinely useful during an incident, not just a deployment convenience.

- [ ] Walk through Kubernetes Service types — ClusterIP, NodePort, LoadBalancer — when do you use each?

IF ASKED, SAY THIS
ClusterIP (the default) for internal-only service-to-service communication within the cluster — the vast majority of services in a typical setup. NodePort exposes a service on a static port on every node's IP — rarely used directly in production on EKS, mostly a building block. LoadBalancer provisions an actual cloud load balancer (an ALB

ClusterIP (the default) for internal-only service-to-service communication within the cluster — the vast majority of services in a typical setup. **NodePort** exposes a service on a static port on every node's IP — rarely used directly in production on EKS, mostly a building block. **LoadBalancer** provisions an actual cloud load balancer (an ALB/NLB via the AWS Load Balancer Controller) — the standard way to expose a service externally, usually fronted further by Ingress for HTTP-layer routing across multiple services sharing one load balancer.

- [ ] How does Ingress work on EKS, and how would you design it for multiple applications sharing a cluster?

IF ASKED, SAY THIS
Ingress defines HTTP(S) routing rules (host/path-based) to internal Services, implemented by an Ingress Controller — on EKS, typically the AWS Load Balancer Controller, which provisions and manages an ALB satisfying the Ingress resource's routing rules. For multiple applications sharing a cluster: one shared ALB with path/host-based routi

Ingress defines HTTP(S) routing rules (host/path-based) to internal Services, implemented by an Ingress Controller — on EKS, typically the AWS Load Balancer Controller, which provisions and manages an ALB satisfying the Ingress resource's routing rules.

For multiple applications sharing a cluster: one shared ALB with path/host-based routing to each application's Service (cost-efficient, one load balancer) versus one ALB per application (more isolation, more cost) — the choice depends on how much traffic-management/security isolation each application genuinely needs versus just minimizing load balancer sprawl.

- [ ] What's CNI, and why does it matter for network policy enforcement in Kubernetes?

IF ASKED, SAY THIS
The Container Network Interface is the plugin layer responsible for actually wiring up pod networking — assigning IPs, setting up routing so pods can reach each other and external services. On EKS, the default is the Amazon VPC CNI , which assigns pods real VPC IP addresses rather than an overlay network — which is why pod IP exhaustion i

The Container Network Interface is the plugin layer responsible for actually wiring up pod networking — assigning IPs, setting up routing so pods can reach each other and external services. On EKS, the default is the **Amazon VPC CNI**, which assigns pods real VPC IP addresses rather than an overlay network — which is why pod IP exhaustion in a subnet is a real, distinctive EKS capacity-planning concern most generic Kubernetes advice doesn't mention.

It matters for NetworkPolicy specifically because **not every CNI enforces NetworkPolicy** — the base VPC CNI historically needed an add-on (or a different CNI like Calico) to actually enforce the policies you define, versus accepting them silently and doing nothing. Worth knowing this distinction if asked whether NetworkPolicies "just work" on EKS.

- [ ] How do you manage configuration (ConfigMaps and Secrets) across environments in Kubernetes?

IF ASKED, SAY THIS
ConfigMaps for non-sensitive config (feature flags, environment-specific URLs, log levels), Secrets for sensitive values — though raw Kubernetes Secrets are only base64-encoded, not encrypted, by default, so for anything genuinely sensitive, integrating with a real secrets manager (AWS Secrets Manager or Parameter Store via the External S

ConfigMaps for non-sensitive config (feature flags, environment-specific URLs, log levels), Secrets for sensitive values — though raw Kubernetes Secrets are only base64-encoded, not encrypted, by default, so for anything genuinely sensitive, integrating with a real secrets manager (AWS Secrets Manager or Parameter Store via the External Secrets Operator, syncing into Kubernetes Secrets) is more defensible than storing sensitive values as raw K8s Secrets checked into a values file.

Per-environment values layered through Helm's values files (or Kustomize overlays) rather than maintaining fully separate manifest sets per environment, so the actual application definition stays single-sourced.

- [ ] Walk through troubleshooting a pod that can't reach another service in the same cluster — where do you look, in order?

IF ASKED, SAY THIS
Confirm the target Service and its Endpoints actually exist and have healthy pod IPs behind them (kubectl get endpoints — a Service with zero endpoints because its label selector doesn't match any running pod is one of the most common causes and easy to miss). Confirm DNS resolution of the service name from inside the calling pod. Check

Confirm the target Service and its Endpoints actually exist and have healthy pod IPs behind them (`kubectl get endpoints` — a Service with zero endpoints because its label selector doesn't match any running pod is one of the most common causes and easy to miss). Confirm DNS resolution of the service name from inside the calling pod. Check NetworkPolicy — if one exists and the CNI actually enforces it (see above) but doesn't explicitly allow this traffic, it's silently dropped rather than erroring clearly.

Then the target pod's own readiness — a pod that's Running but failing its readiness probe gets pulled out of the Service's endpoints entirely, which looks identical to a networking problem from the caller's side.

Helm

- [ ] Explain Helm chart structure and how you manage environment-specific configuration.

> **IF ASKED, SAY THIS**
> Chart.yaml (metadata) + values.yaml (defaults) + templates/ (Go-templated K8s manifests) + charts/ (subchart dependencies), with per-environment values-*.yaml files layered on top via --values at deploy time.

Helm is the standard way to package and deploy applications on Kubernetes:

Chart structure: `mychart/ Chart.yaml # metadata: name, version, description, dependencies values.yaml # default values (dev environment baseline) values-staging.yaml # staging overrides values-prod.yaml # production overrides templates/ # Kubernetes YAML templates with Go templating deployment.yaml service.yaml ingress.yaml hpa.yaml charts/ # subcharts (dependencies) .helmignore`

Environment-specific configuration: `# Deploy to dev with default values helm upgrade --install payments ./charts/payments -n dev # Deploy to prod with production overrides helm upgrade --install payments ./charts/payments --values ./charts/payments/values-prod.yaml -n production --set image.tag=1.2.3` values-prod.yaml contains: replica count, resource limits, HPA settings, ingress hosts, database endpoints — anything that differs between environments.

Helm release management: `helm list -n production` shows all deployed releases. `helm history payments -n production` shows revision history — every helm upgrade creates a new revision stored as a ConfigMap in the cluster.
`helm rollback payments 2 -n production` rolls back to revision 2. This is different from kubectl rollout undo — Helm rollback restores the entire Helm release (all resources), not just the Deployment.

Helm test: Built-in testing mechanism. Write a Job that tests the deployed application (hits /health, verifies DB connectivity). Run with `helm test payments` after deployment as a smoke test in the CI/CD pipeline.

Chart versioning: Chart.yaml has two versions: chart version (the chart itself) and appVersion (the application version). Always increment chart version when the chart changes. This enables rollback to a specific chart version.

- [ ] What problem does Helm actually solve that raw kubectl apply of YAML manifests doesn't?

> **IF ASKED, SAY THIS**
> Templating (one chart, many environments via values overrides), release versioning (every helm upgrade is a tracked revision you can roll back as a unit), and dependency management (subcharts) — raw manifests give you none of these for free.

The release-as-a-unit property is the one people underrate: `helm rollback` reverts the ENTIRE set of resources a release touched back to a prior revision in one command, whereas rolling back raw manifests means manually figuring out and reverting every file that changed.

- [ ] How does Helm's Go templating actually work, and what are the parts people get wrong first?

> **IF ASKED, SAY THIS**
> Templates are Go text/template syntax (`{{ .Values.x }}` , `{{ if }}` , `{{ range }}`) rendered against the merged values before being sent to the API server. Common trip-ups: whitespace control (`{{- }}` vs `{{ }}`), and forgetting `| quote` / `| toYaml` when injecting values into YAML that need specific typing.

`helm template` (renders locally without installing) and `helm install --dry-run --debug` are the two commands to actually SEE the rendered YAML before it hits the cluster — essential for debugging a templating bug rather than guessing.

```
{{- range .Values.env }}
- name: {{ .name }}
  value: {{ .value | quote }}
{{- end }}
```

- [ ] What are Helm hooks, and what's a real production use for one?

> **IF ASKED, SAY THIS**
> Hooks (annotated via helm.sh/hook: pre-install, post-upgrade, etc.) let a chart run a Job at a specific point in the release lifecycle — e.g. a pre-upgrade hook running a DB migration before the new app version's pods start, so the schema is ready before code that depends on it goes live.

A common gotcha: hook resources aren't tracked/cleaned up as part of the normal release the same way regular templates are, unless you also set a `hook-delete-policy` — forgetting this leaves stray completed Jobs accumulating in the cluster.

- [ ] How do you test and lint a Helm chart before it's trusted in a CI/CD pipeline?

> **IF ASKED, SAY THIS**
> helm lint catches structural/template issues; helm template piped into kubeval/kubeconform validates the rendered YAML against the K8s API schema; helm test runs an in-cluster smoke-test Job post-deploy (hits /health, checks DB connectivity) as the final gate.

Layering all three catches different failure classes: lint catches chart authoring mistakes, schema validation catches "this YAML isn't even valid Kubernetes," and helm test catches "it's valid YAML but the app doesn't actually come up healthy."

- [ ] Helm vs. Kustomize — how do you actually decide between them, or do you use both?

IF ASKED, SAY THIS
Helm templates (Go templating + packaging + release management); Kustomize overlays (patches a base set of plain YAML per environment, no templating language, ships built into kubectl). Many teams use both: Helm for third-party charts you install (ingress controllers, cert-manager), Kustomize for their own app manifests where a full templating language is overkill.

The practical tell: if you're fighting Go-template whitespace/escaping rules to express something Kustomize's strategic-merge-patch would do in three lines, that's a sign the tool doesn't fit the job.

- [ ] How do secrets get into a Helm-deployed application without ending up committed in values.yaml?

IF ASKED, SAY THIS
values.yaml should hold config, not secrets — actual secret values come from an external source (Vault, AWS Secrets Manager, Sealed Secrets, or SOPS-encrypted values files) and get injected either as a pre-existing Kubernetes Secret the chart references, or via the External Secrets Operator syncing from Vault/Secrets Manager into a K8s Secret automatically.

Plaintext secrets in a values file — even a "private" one — is one of the most common Helm anti-patterns; anyone with repo read access or chart history access can see them, and they persist in the Helm release's stored revision history in the cluster too.

- [ ] Managing many charts across many services and environments — what does that actually look like at scale?

IF ASKED, SAY THIS
A shared "library chart" (common templates for logging/monitoring/probes every service's chart depends on, so a platform-wide change happens in one place) plus Helmfile or ArgoCD's App-of-Apps pattern to declare which chart+values combination deploys to which environment as its own version-controlled manifest, rather than running helm commands by hand.

Without a library chart, the same boilerplate (liveness probes, standard labels, resource limits) gets copy-pasted into every service's chart and drifts over time — a library chart turns a platform standard into something enforced by inheritance instead of convention.

- [ ] What problem does Helm solve, and what exactly is it?

IF ASKED, SAY THIS
Helm is the package manager for Kubernetes. Raw manifests don't parameterize well across environments — you either hand-edit YAML per environment or build your own templating with sed/envsubst. Helm gives you three things together: a templating engine (Go templates over YAML), a packaging format (a chart — a versioned,

Helm is the package manager for Kubernetes. Raw manifests don't parameterize well across environments — you either hand-edit YAML per environment or build your own templating with sed/envsubst. Helm gives you three things together: a **templating engine** (Go templates over YAML), a **packaging format** (a chart — a versioned, shareable bundle of templates + defaults), and a **release/state tracker** (Helm knows what it installed, in what revision, and can diff/rollback).

Think of it as: Chart = source code, Release = a running instance of that chart in a specific namespace with a specific values override, Revision = a version of that release's history.

- [ ] Helm 2 vs Helm 3 — what changed architecturally, and why does it matter?

IF ASKED, SAY THIS
Helm 2 ran a server-side component called Tiller inside the cluster with broad (often cluster-admin) RBAC — a standing security liability and a single point of failure shared by every user of the cluster. Helm 3 removed Tiller entirely: Helm is now a client-only binary that talks straight to the Kubernetes API server u

Helm 2 ran a server-side component called **Tiller** inside the cluster with broad (often cluster-admin) RBAC — a standing security liability and a single point of failure shared by every user of the cluster. Helm 3 removed Tiller entirely: Helm is now a **client-only** binary that talks straight to the Kubernetes API server using whatever kubeconfig/RBAC context you already have. Permissions are now scoped to the actual user or CI service account, not a shared superuser.

Other load-bearing changes: release information moved from `ConfigMaps in kube-system` to **Secrets in the release's own namespace**; releases became namespace-scoped instead of a single global namespace; Helm 3 does 3-way strategic merge patches on upgrade (comparing old manifest, new manifest, and live state — so it detects and can warn about out-of-band changes); `helm install` no longer auto-generates a random release name silently in the same way and requires a name or `--generate-name`; and `requirements.yaml` was folded directly into `Chart.yaml`.

Interview flagIf someone asks "why was Tiller removed," the answer is security (RBAC blast radius) and multi-tenancy, not performance.

- [ ] Where and how does Helm store release state?

IF ASKED, SAY THIS

By default, in a Kubernetes Secret per revision, in the same namespace as the release, named like sh.helm.release.v1..v , type helm.sh/release.v1 . The payload is the rendered manifest plus chart metadata and values, gzip-compressed and base64-encoded. This is why helm history and helm rollback

By default, in a **Kubernetes Secret** per revision, in the same namespace as the release, named like `sh.helm.release.v1.<release-name>.v<revision>` , type `helm.sh/release.v1` . The payload is the rendered manifest plus chart metadata and values, gzip-compressed and base64-encoded. This is why `helm history` and `helm rollback` work without any external database — the history lives in the cluster itself. You can point the backend at ConfigMaps or an external SQL store instead via `HELM_DRIVER` , but Secrets is the default and most common in production (encrypted at rest if your cluster has etcd encryption enabled).

- [ ] Define: Chart, Release, Revision, Repository — precisely.

IF ASKED, SAY THIS
Chart — the packaged template bundle (Chart.yaml + values.yaml + templates/). Versioned independently via SemVer. Release — a named, deployed instance of a chart into a cluster/namespace, with a specific set of values applied. You can install the same chart multiple times under different release names. Revision — an in

- **Chart** — the packaged template bundle (Chart.yaml + values.yaml + templates/). Versioned independently via SemVer.
- **Release** — a named, deployed instance of a chart into a cluster/namespace, with a specific set of values applied. You can install the same chart multiple times under different release names.
- **Revision** — an integer that increments every time a release is installed, upgraded, or rolled back. `helm rollback myapp 3` reverts to revision 3's rendered manifest.
- **Repository** — an HTTP(S) endpoint (or OCI registry) serving an `index.yaml` that lists available charts and versions, e.g. Artifact Hub-listed repos or a private Nexus/Harbor/S3-backed repo.

- [ ] What does a minimal chart directory look like, file by file?

IF ASKED, SAY THIS
Anything under templates/ that renders to empty content is skipped. Filenames starting with _ (like _helpers.tpl) are treated as library files and never rendered directly into manifests.

```
mychart/
  Chart.yaml          # metadata: name, version, appVersion, apiVersion: v2
  values.yaml          # default configuration values
  values.schema.json    # optional JSON Schema to validate values
  charts/               # vendored subchart dependencies (.tgz or unpacked)
  templates/
    deployment.yaml
    service.yaml
    ingress.yaml
    _helpers.tpl        # named template snippets, no manifest of its own
    NOTES.txt            # printed to stdout after install/upgrade
    tests/
      test-connection.yaml
  .helmignore           # patterns excluded when packaging (like .gitignore)
  Chart.lock            # resolved dependency versions (auto-generated)
```

Anything under `templates/` that renders to empty content is skipped. Filenames starting with `_` (like `_helpers.tpl`) are treated as library files and never rendered directly into manifests.

- [ ] What fields does Chart.yaml need, and what's the difference between version and appVersion ?

IF ASKED, SAY THIS
version is what changes every time you publish a new chart release and is what repositories index on. appVersion is purely metadata shown in helm list — Helm never uses it to resolve anything. It's a very common interview trip-up to conflate the two.

```
apiVersion: v2
name: mychart
description: A Helm chart for my service
type: application       # or "library"
version: 1.4.2           # CHART version (SemVer) — bump when templates/values change
appVersion: "2.9.0"       # the VERSION OF THE APP being deployed (e.g. container image tag) — informational only
dependencies:
  - name: redis
    version: "18.x.x"
    repository: "https://charts.bitnami.com/bitnami"
    condition: redis.enabled
```

`version` is what changes every time you publish a new chart release and is what repositories index on. `appVersion` is purely metadata shown in `helm list` — Helm never uses it to resolve anything. It's a very common interview trip-up to conflate the two.

- [ ] What's the purpose of _helpers.tpl and NOTES.txt ?

IF ASKED, SAY THIS
_helpers.tpl holds reusable named templates (via define) for things like standardized label blocks, fully-qualified names, or image string construction — so every manifest in the chart references the same helper instead of duplicating logic. NOTES.txt is templated like any other file but its rendered output is printed

`_helpers.tpl` holds reusable named templates (via `define`) for things like standardized label blocks, fully-qualified names, or image string construction — so every manifest in the chart references the same helper instead of duplicating logic. `NOTES.txt` is templated like any other file but its rendered output is printed to the terminal after a successful `helm install` / `upgrade` — commonly used to show the user how to access the app (e.g., a `kubectl port-forward` command or the generated ingress URL).

- [ ] What is values.schema.json and why would a senior engineer add one?

IF ASKED, SAY THIS
It's a JSON Schema file that Helm validates values.yaml (and any -f / --set overrides) against before rendering templates. It converts "typo in a values key silently does nothing" or "wrong type causes a cryptic template error 40 lines deep" into a clear validation error at the CLI. For a platform team publishing chart

It's a JSON Schema file that Helm validates `values.yaml` (and any `-f` / `--set` overrides) against *before* rendering templates. It converts "typo in a values key silently does nothing" or "wrong type causes a cryptic template error 40 lines deep" into a clear validation error at the CLI. For a platform team publishing charts to many consuming teams, this is the difference between self-service that fails safely and self-service that pages you at 2am.

- [ ] What is Chart.lock and when is it generated?

IF ASKED, SAY THIS
It records the exact resolved versions of every dependency listed in Chart.yaml at the time helm dependency update was run — analogous to a lockfile in npm/pip. helm dependency build reads Chart.lock (if present) and re-downloads exactly those pinned versions into charts/ , giving reproducible builds instead of silentl

It records the exact resolved versions of every dependency listed in `Chart.yaml` at the time `helm dependency update` was run — analogous to a lockfile in npm/pip. `helm dependency build` reads `Chart.lock` (if present) and re-downloads exactly those pinned versions into `charts/`, giving reproducible builds instead of silently picking up a newer dependency version that happens to match the version range in Chart.yaml.

- [ ] How does the Helm templating engine actually work under the hood?

IF ASKED, SAY THIS
Helm templates are Go's text/template syntax layered with Sprig template functions, applied to YAML files as plain text (the engine doesn't understand YAML semantics — it's a string substitution pass that happens to usually produce valid YAML). Actions are wrapped in {{ }} . A "pipeline" chains values through functions

Helm templates are Go's `text/template` syntax layered with Sprig template functions, applied to YAML files as plain text (the engine doesn't understand YAML semantics — it's a string substitution pass that happens to usually produce valid YAML). Actions are wrapped in `{{ }}`. A "pipeline" chains values through functions with `|`, evaluated left to right, e.g. `{{ .Values.name | default "app" | quote }}` takes the value, substitutes a default if empty, then wraps it in quotes.

- [ ] Name the built-in objects available inside every template, and what each gives you.

IF ASKED, SAY THIS
.Values — merged values (chart defaults overridden by -f / --set). .Release — .Release.Name , .Release.Namespace , .Release.Revision , .Release.IsUpgrade / .IsInstall . .Chart — the contents of Chart.yaml (.Chart.Name , .Chart.Version ...). .Files — access to non-template files in the chart (e.g. .Files.Get "config/a

- `.Values` — merged values (chart defaults overridden by `-f` / `--set`).
- `.Release` — `.Release.Name`, `.Release.Namespace`, `.Release.Revision`, `.Release.IsUpgrade` / `.IsInstall`.
- `.Chart` — the contents of Chart.yaml (`.Chart.Name`, `.Chart.Version` ...).
- `.Files` — access to non-template files in the chart (e.g. `.Files.Get "config/app.conf"`).
- `.Capabilities` — cluster info like `.Capabilities.KubeVersion`, useful for conditionally rendering manifests based on API availability.
- `Template` — metadata about the current template file, e.g. `.Template.BasePath`.

- [ ] Give the everyday Sprig/template functions you'd actually use in production charts.

IF ASKED, SAY THIS
nindent N vs indent N : nindent adds a newline before indenting, which is what you want 95% of the time when injecting a multi-line block into YAML at a specific indentation level. Forgetting this is one of the most common "why is my YAML malformed" bugs in Helm charts.

```
{{ .Values.replicaCount | default 1 }}                # fallback if unset
{{ .Values.name | quote }}                            # wrap string in ""
{{ .Values.labels | toYaml | nindent 4 }}             # dump a map as YAML, indent by 4
{{ include "mychart.fullname" . | trunc 63 | trimSuffix "-" }}  # DNS-1123 safe name
{{ required "image.tag is required!" .Values.image.tag }}       # hard-fail if missing
{{ .Values.name | upper }} / {{ .Values.name | lower }}
{{- if .Values.debug }} ... {{- end }}
```

`nindent N` vs `indent N` : `nindent` adds a newline before indenting, which is what you want 95% of the time when injecting a multi-line block into YAML at a specific indentation level. Forgetting this is one of the most common "why is my YAML malformed" bugs in Helm charts.

- [ ] Explain if/else , range , and with — including how scope (the "dot") changes inside each.

IF ASKED, SAY THIS

range iterates a list/map and rebinds . to each element inside the loop. with doesn't loop — it just rebinds . to the given value for its block, mainly to avoid repeating a long path. The trap: once you're inside range or with , . no longer refers to the root context, so .Release.Name breaks inside those blocks — you m

```
{{- if .Values.ingress.enabled }}
kind: Ingress
{{- else }}
# no ingress block rendered
{{- end }}

{{- range .Values.env }}
- name: {{ .name }}
  value: {{ .value | quote }}
{{- end }}

{{- with .Values.resources }}
requests:
  cpu: {{ .requests.cpu }}
{{- end }}
```

`range` iterates a list/map and rebinds `.` to each element inside the loop. `with` doesn't loop — it just rebinds `.` to the given value for its block, mainly to avoid repeating a long path. The trap: once you're inside `range` or `with` , `.` no longer refers to the root context, so `.Release.Name` breaks inside those blocks — you must use `$.Release.Name` , since `$` always keeps a reference to the original root context regardless of scope.

Common trap question"Your template works until you wrap it in a `with` block, then `.Release.Name` throws a nil pointer error — why?" Answer: scope shift; fix with `$.Release.Name` .

- [ ] define / template / include — what's the difference, and which should you default to?

IF ASKED, SAY THIS

define declares a named template. template "name" . renders it inline but cannot be used inside a pipeline (you can't pipe its output into nindent). include "name" . does the same rendering but returns a string, so it can be piped — which is why include ... | nindent 4 is the standard idiom and include is almost alway

```
{{/* in _helpers.tpl */}}
{{- define "mychart.labels" -}}
app.kubernetes.io/name: {{ .Chart.Name }}
app.kubernetes.io/instance: {{ .Release.Name }}
{{- end }}

{{/* usage in a manifest */}}
metadata:
  labels:
{{ include "mychart.labels" . | nindent 4 }}
```

`define` declares a named template. `template "name" .` renders it inline but **cannot** be used inside a pipeline (you can't pipe its output into `nindent`). `include "name" .` does the same rendering but returns a string, so it **can** be piped — which is why `include ... | nindent 4` is the standard idiom and `include` is almost always preferred over `template` in real charts.

- [ ] Explain whitespace control: {{- and -}} .

IF ASKED, SAY THIS

IF ASKED, SAY THIS
Template actions leave the surrounding newline/whitespace in the output by default, which in YAML can produce blank lines or bad indentation that breaks parsing. The hyphen trims whitespace on that side: {{- trims preceding whitespace/newlines, -}} trims trailing. Chart authors almost always write control structures as

Template actions leave the surrounding newline/whitespace in the output by default, which in YAML can produce blank lines or bad indentation that breaks parsing. The hyphen trims whitespace on that side: `{{-` trims preceding whitespace/newlines, `-}}` trims trailing. Chart authors almost always write control structures as `{{- if ... }}` / `{{- end }}` so the conditional itself doesn't leave stray blank lines in rendered output — this is why "why does my rendered YAML have random blank lines" is usually a missing hyphen.

- [ ] How do you reference chart-relative files, and what's .Files useful for?

IF ASKED, SAY THIS
.Files.Get "path/relative/to/chart/root" reads a static file's contents as a string (e.g. embedding a config file into a ConfigMap), .Files.Glob "configs/*.conf" matches multiple, and .Files.AsConfig / .AsSecrets turn a glob directly into a ConfigMap/Secret data: block. Common use: shipping a Prometheus scrape config,

`.Files.Get "path/relative/to/chart/root"` reads a static file's contents as a string (e.g. embedding a config file into a ConfigMap), `.Files.Glob "configs/*.conf"` matches multiple, and `.Files.AsConfig` / `.AsSecrets` turn a glob directly into a ConfigMap/Secret `data:` block. Common use: shipping a Prometheus scrape config, an nginx.conf, or a set of Grafana dashboard JSON files inside a chart without templating each one individually.

- [ ] What's the precedence order when values come from multiple sources?

IF ASKED, SAY THIS
Highest to lowest: --set / --set-string / --set-json on the command line > -f / --values files, evaluated in the order given (later files win) > the chart's own values.yaml . For subcharts, a parent's values.yaml can override a subchart's defaults by nesting under the subchart's name as a key.

Highest to lowest: `--set` / `--set-string` / `--set-json` on the command line > `-f` / `--values` files, evaluated in the order given (later files win) > the chart's own `values.yaml`. For subcharts, a parent's `values.yaml` can override a subchart's defaults by nesting under the subchart's name as a key.

```
helm upgrade myapp ./mychart \
  -f values-common.yaml \
  -f values-prod.yaml \
  --set image.tag=$(git rev-parse --short HEAD) \
  --atomic --wait --timeout 5m
```

- [ ] What is .Values.global and why does it exist?

IF ASKED, SAY THIS
Any key placed under global: in the parent values.yaml is automatically visible to every subchart as .Values.global.x , in addition to that subchart's own values. It's the mechanism for sharing cross-cutting settings — image registry, environment name, common labels — across a chart with many dependencies without dupli

Any key placed under `global:` in the parent values.yaml is automatically visible to *every* subchart as `.Values.global.x`, in addition to that subchart's own values. It's the mechanism for sharing cross-cutting settings — image registry, environment name, common labels — across a chart with many dependencies without duplicating the setting into each subchart's values block.

- [ ] When you merge two values files, do lists merge or replace? What about maps?

IF ASKED, SAY THIS
Maps merge recursively (a key in the override that isn't present in the base is added; a key present in both is replaced at that level). Lists/arrays are replaced wholesale , never merged element-by-element — if your prod values file redefines env: as a 2-item list, it fully replaces the base chart's 5-item list, it do

Maps merge recursively (a key in the override that isn't present in the base is added; a key present in both is replaced at that level). **Lists/arrays are replaced wholesale**, never merged element-by-element — if your prod values file redefines `env:` as a 2-item list, it fully replaces the base chart's 5-item list, it doesn't append. This is a very common production surprise: someone adds one extra env var in an override file expecting it to add to the base list, and instead it silently drops everything else.

Real production trapOverriding a list-typed value (env vars, volumes, tolerations) always requires re-specifying the entire list, not just the delta.

- [ ] What are Helm hooks, and what hook points are available?

IF ASKED, SAY THIS
Hooks let a chart run Kubernetes resources (almost always Jobs) at specific points in the release lifecycle, outside the normal template rendering order — e.g. a DB migration Job that must finish before the new app pods start. Available hooks: pre-install , post-install , pre-upgrade , post-upgrade , pre-rollback , pos

Hooks let a chart run Kubernetes resources (almost always Jobs) at specific points in the release lifecycle, outside the normal template rendering order — e.g. a DB migration Job that must finish before the new app pods start. Available hooks: `pre-install`, `post-install`, `pre-upgrade`, `post-upgrade`, `pre-rollback`, `post-rollback`, `pre-delete`, `post-delete`, and `test` (run only on demand via `helm test`).

```
apiVersion: batch/v1
kind: Job
metadata:
  name: {{ .Release.Name }}-db-migrate
  annotations:
    "helm.sh/hook": pre-upgrade
    "helm.sh/hook-weight": "-5"
    "helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
spec:
  template:
    spec:
      containers:
        - name: migrate
          image: myapp-migrator:{{ .Values.image.tag }}
      restartPolicy: Never
```

- [ ] How is execution order determined when multiple hooks share the same hook type?

IF ASKED, SAY THIS
By helm.sh/hook-weight — a string-encoded integer, ascending order (lower/more negative runs first). Hooks with the same weight fall back to alphabetical order by resource name. Helm waits for each hook Job to reach a ready/completed state (using the same readiness logic as --wait) before moving to the next weight tie

By `helm.sh/hook-weight` — a string-encoded integer, **ascending** order (lower/more negative runs first). Hooks with the same weight fall back to alphabetical order by resource name. Helm waits for each hook Job to reach a ready/completed state (using the same readiness logic as `--wait`) before moving to the next weight tier, and before proceeding with the main install/upgrade.

- [ ] What does helm.sh/hook-delete-policy control, and what's the default if you omit it?

IF ASKED, SAY THIS
It controls when Helm deletes the hook resource: before-hook-creation (delete the previous hook resource right before creating a new one — the default if unspecified, which is why old completed Jobs from a prior release don't normally pile up), hook-succeeded (delete immediately after success), hook-failed (delete afte

It controls when Helm deletes the hook resource: `before-hook-creation` (delete the previous hook resource right before creating a new one — the default if unspecified, which is why old completed Jobs from a prior release don't normally pile up), `hook-succeeded` (delete immediately after success), `hook-failed` (delete after failure — rarely used, since you usually want to keep failed hook logs for debugging). You can combine multiple, comma-separated.

- [ ] What does helm test actually run, and how do you write a test hook?

IF ASKED, SAY THIS
Resources annotated "helm.sh/hook": test are skipped during install/upgrade and only run when you explicitly invoke helm test . It's typically a Pod that curls a health endpoint or runs a smoke-test script and exits 0/non-zero — Helm reports pass/fail based on the container exit code. Commonly wired into a CD

Resources annotated `"helm.sh/hook": test` are skipped during install/upgrade and only run when you explicitly invoke `helm test <release>`. It's typically a Pod that curls a health endpoint or runs a smoke-test script and exits 0/non-zero — Helm reports pass/fail based on the container exit code. Commonly wired into a CD pipeline as a post-deploy verification gate before marking a rollout successful.

- [ ] What's the most important gotcha about hooks and release ownership that catches people in production?

IF ASKED, SAY THIS
Hook resources are not tracked as part of the normal release manifest the way templates in templates/ are — Helm creates them out-of-band and (depending on delete policy) may or may not clean them up. If you don't set a delete policy and the default doesn't apply as expected, or if a hook Job fails and its policy doesn

Hook resources are **not tracked as part of the normal release manifest** the way templates in `templates/` are — Helm creates them out-of-band and (depending on delete policy) may or may not clean them up. If you don't set a delete policy and the default doesn't apply as expected, or if a hook Job fails and its policy doesn't delete on failure, you can accumulate orphaned Jobs/Pods across many upgrades that `helm uninstall` won't necessarily clean up, since `helm uninstall` only guarantees deletion of hooks associated with *delete*-phase hooks, not stale pre/post-install/upgrade ones from history.

- [ ] How do you declare and manage chart dependencies?

IF ASKED, SAY THIS
condition points at a boolean values path that enables/disables that dependency at render time. tags let you toggle groups of dependencies together via tags.: false . alias lets you include the same dependency chart twice under different names (e.g. two Redis instances for different purposes).

```
# Chart.yaml
dependencies:
  - name: redis
    version: "18.6.1"
    repository: "https://charts.bitnami.com/bitnami"
    condition: redis.enabled
    tags: ["caching"]
  - name: common-labels
    version: "1.0.0"
    repository: "file://../common-labels"
    alias: labels
```

`condition` points at a boolean values path that enables/disables that dependency at render time. `tags` let you toggle groups of dependencies together via `tags.<tag>: false`. `alias` lets you include the same dependency chart twice under different names (e.g. two Redis instances for different purposes).

- [ ] helm dependency update vs helm dependency build — when do you use each?

IF ASKED, SAY THIS
helm dependency update resolves the version constraints in Chart.yaml against the repositories, downloads the matching charts into charts/ , and (re)writes Chart.lock with the exact versions picked. helm dependency build is the reproducible variant — it reads the existing Chart.lock and downloads exactly those pinned v

`helm dependency update` resolves the version constraints in Chart.yaml against the repositories, downloads the matching charts into `charts/`, and (re)writes `Chart.lock` with the exact versions picked. `helm dependency build` is the reproducible variant — it reads the *existing* `Chart.lock` and downloads exactly those pinned versions without re-resolving, which is what you want in CI so a build doesn't silently pick up a newer dependency version between pipeline runs.

- [ ] What's a "library chart" and how is it different from an application chart?

IF ASKED, SAY THIS
A chart with type: library in Chart.yaml contains only reusable define blocks (named templates) and produces no Kubernetes resources of its own — helm install against a pure library chart deploys nothing. It exists purely to be listed as a dependency and provide shared helpers (standard label sets, common naming, secur

A chart with `type: library` in Chart.yaml contains only reusable `define` blocks (named templates) and produces **no Kubernetes resources of its own** — `helm install` against a pure library chart deploys nothing. It exists purely to be listed as a dependency and provide shared helpers (standard label sets, common naming, security-context defaults) to every chart that depends on it, so a platform team can enforce conventions across many application charts from one place instead of copy-pasting `_helpers.tpl` everywhere.

- [ ] Walk through how a traditional Helm HTTP repository works.

IF ASKED, SAY THIS
A repo is just a static file server hosting packaged .tgz charts plus an index.yaml that lists every chart name/version and its download URL and checksum. helm repo add registers the URL locally, helm repo update re-fetches index.yaml . No server-side logic is required — it can be S3/GCS with static hosting, GitHub Pag

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm search repo bitnami/redis --versions
helm pull bitnami/redis --version 18.6.1 --untar
```

A repo is just a static file server hosting packaged `.tgz` charts plus an `index.yaml` that lists every chart name/version and its download URL and checksum. `helm repo add` registers the URL locally, `helm repo update` re-fetches `index.yaml`. No server-side logic is required — it can be S3/GCS with static hosting, GitHub Pages, Nexus, Artifactory, or Harbor's chart repo mode.

- [ ] How do OCI registries work as Helm repositories, and why has the ecosystem moved toward them?

IF ASKED, SAY THIS
Since Helm 3.8, OCI registries (the same protocol container images use — ECR, GHCR, Harbor, ACR) are first-class chart storage. This means one registry and one auth/RBAC model for both your container images and your charts, instead of maintaining a separate chart-repo server, and it plugs directly into existing image-s

```
helm registry login registry.example.com -u user
helm package ./mychart
helm push mychart-1.4.2.tgz oci://registry.example.com/charts
helm install myapp oci://registry.example.com/charts/mychart --version 1.4.2
```

Since Helm 3.8, OCI registries (the same protocol container images use — ECR, GHCR, Harbor, ACR) are first-class chart storage. This means one registry and one auth/RBAC model for both your container images and your charts, instead of maintaining a separate chart-repo server, and it plugs directly into existing image-scanning/signing tooling. Most platform teams building new pipelines default to OCI over classic HTTP repos now.

- [ ] What is Artifact Hub, and what's the difference between it and a repository?

IF ASKED, SAY THIS
Artifact Hub is a public, CNCF-hosted catalog/search index across many independently-hosted chart repositories (and other artifact types) — it doesn't host the charts itself, it aggregates metadata so you can discover "which repo has a maintained Kafka chart" before running helm repo add against the actual repo it poin

Artifact Hub is a public, CNCF-hosted *catalog/search index* across many independently-hosted chart repositories (and other artifact types) — it doesn't host the charts itself, it aggregates metadata so you can discover "which repo has a maintained Kafka chart" before running `helm repo add` against the actual repo it points you to.

- [ ] How do you sign a chart and verify provenance before install?

IF ASKED, SAY THIS
Signing uses GPG: helm package --sign generates a .prov provenance file alongside the chart containing a signed hash of the chart contents. --verify on install checks the signature against a trusted public keyring before proceeding, so a tampered or wrong chart fails closed rather than silently deploying.

```
helm package --sign --key 'mykey' --keyring ~/.gnupg/secring.gpg ./mychart
# produces mychart-1.4.2.tgz AND mychart-1.4.2.tgz.prov

helm install myapp ./mychart-1.4.2.tgz --verify --keyring ~/.gnupg/pubring.gpg
```

Signing uses GPG: `helm package --sign` generates a `.prov` provenance file alongside the chart containing a signed hash of the chart contents. `--verify` on install checks the signature against a trusted public keyring before proceeding, so a tampered or wrong chart fails closed rather than silently deploying.

- [ ] What do --atomic , --wait , and --timeout actually do, and how do they interact?

IF ASKED, SAY THIS
--wait makes Helm block until all resources (Deployments, StatefulSets, Jobs, etc.) report ready, instead of returning as soon as the API server accepts the manifests. --timeout bounds how long --wait (and hooks) will wait before giving up (default 5m). --atomic implies --wait and additionally triggers an automatic hel

`--wait` makes Helm block until all resources (Deployments, StatefulSets, Jobs, etc.) report ready, instead of returning as soon as the API server accepts the manifests. `--timeout` bounds how long `--wait` (and hooks) will wait before giving up (default 5m). `--atomic` implies `--wait` and additionally triggers an automatic `helm rollback` to the previous good revision if the install/upgrade fails or times out — this is the single most important flag for safe CD pipelines, because without it a failed upgrade just leaves the cluster in a half-applied state and a human has to notice and roll back manually.

```
helm upgrade myapp ./mychart -f values-prod.yaml --atomic --wait --timeout 10m
```

- [ ] How does helm rollback actually work — does it re-apply the old manifest or something smarter?

IF ASKED, SAY THIS
helm rollback takes the stored manifest from that historical revision's Secret and applies it as a new revision (i.e., rolling back to revision 3 from revision 7 creates revision 8, whose content matches revision 3) — history is append-only, it never deletes forward revisions. This means you can "r

`helm rollback <release> <revision>` takes the stored manifest from that historical revision's Secret and applies it as a *new* revision (i.e., rolling back to revision 3 from revision 7 creates revision 8, whose content matches revision 3) — history is append-only, it never deletes forward revisions. This means you can "roll forward to a rollback" again later. Rollback goes through the same 3-way merge and (if hooks are defined) `pre-rollback` / `post-rollback` hooks as a normal upgrade.

- [ ] Why is helm upgrade --install the standard idiom in CD pipelines instead of separate install/upgrade calls?

IF ASKED, SAY THIS
It makes the deploy step idempotent: the same command works whether the release already exists (does an upgrade) or this is the very first deploy to that environment (does an install), so your pipeline doesn't need branching logic to check release existence first. Combined with --create-namespace it

also handles a bran

```
helm upgrade myapp ./mychart --install -n myns --create-namespace \
  -f values-prod.yaml --atomic --wait
```

It makes the deploy step idempotent: the same command works whether the release already exists (does an upgrade) or this is the very first deploy to that environment (does an install), so your pipeline doesn't need branching logic to check release existence first. Combined with `--create-namespace` it also handles a brand-new environment in one shot.

- [ ] How do you preview exactly what a release will change before applying it?

IF ASKED, SAY THIS
helm template ./mychart -f values-prod.yaml renders manifests locally with no cluster contact at all — good for static review/linting/CI. helm install --dry-run --debug goes further: it renders against the live cluster context (so it can resolve things like .Capabilities) and validates against the API server without p

`helm template ./mychart -f values-prod.yaml` renders manifests locally with no cluster contact at all — good for static review/linting/CI. `helm install --dry-run --debug` goes further: it renders against the live cluster context (so it can resolve things like `.Capabilities`) and validates against the API server without persisting anything. Neither of these shows a diff against what's *currently* deployed — for an actual diff against the live release, you need the community `helm-diff` plugin (`helm diff upgrade myapp ./mychart -f values-prod.yaml`), which is not built into core Helm and is a very common gap teams don't realize until they need it mid-incident.

- [ ] What does helm history show, and what's stored per revision?

IF ASKED, SAY THIS
Each row is a distinct Secret with its own full rendered manifest, values, and status (deployed , superseded , failed , pending-install , pending-upgrade , pending-rollback , uninstalled). Interviewers like asking you to read a history table like this and explain what happened and what you'd do next — here, revision

```
helm history myapp -n prod
REVISION  UPDATED       STATUS      CHART          APP VERSION  DESCRIPTION
1         Jul 20 09:11  superseded  mychart-1.3.0  2.8.0        Install complete
2         Jul 24 14:02  superseded  mychart-1.4.0  2.9.0        Upgrade complete
3         Jul 29 18:47  deployed    mychart-1.4.2  2.9.0        Upgrade complete
4         Jul 30 08:15  failed      mychart-1.5.0  3.0.0        Upgrade "myapp" failed: timed out waiting for condition
```

Each row is a distinct Secret with its own full rendered manifest, values, and status (`deployed` , `superseded` , `failed` , `pending-install` , `pending-upgrade` , `pending-rollback` , `uninstalled`). Interviewers like asking you to read a history table like this and explain what happened and what you'd do next — here, revision 4 failed, so you'd either fix the root cause and re-upgrade, or `helm rollback myapp 3` .

- [ ] How does helm uninstall handle history, and what's --keep-history for?

IF ASKED, SAY THIS
By default helm uninstall deletes all release records along with the live resources — the release name becomes fully available for reuse immediately. --keep-history deletes the live Kubernetes resources but retains the revision Secrets, so helm history still works and helm rollback can theoretically resurrect it later;

By default `helm uninstall` deletes all release records along with the live resources — the release name becomes fully available for reuse immediately. `--keep-history` deletes the live Kubernetes resources but retains the revision Secrets, so `helm history` still works and `helm rollback` can theoretically resurrect it later; without it, once uninstalled, that history is gone and reinstalling the same name starts fresh at revision 1.

- [ ] You run helm upgrade and get: "another operation (install/upgrade/rollback) is in progress" . What's happening and how do you fix it?

IF ASKED, SAY THIS
Helm marks a release's current revision as pending-install / pending-upgrade / pending-rollback the moment an operation starts, and only flips it to deployed or failed when the operation completes. If the Helm client is killed mid-operation (CI job cancelled, network drop, pod eviction of the CI runner) before it can u

Helm marks a release's current revision as `pending-install` / `pending-upgrade` / `pending-rollback` the moment an operation starts, and only flips it to `deployed` or `failed` when the operation completes. If the Helm client is killed mid-operation (CI job cancelled, network drop, pod eviction of the CI runner) before it can update that status, the release is left permanently "stuck" in a pending state, and Helm refuses to start a new operation on top of one it thinks is still running.

Fix, safest first: (1) confirm nothing is actually still running — check for a live Job/process from the original operation; (2) if it's truly stuck, `helm rollback <release> <last-good-revision>` — rollback is usually allowed even from a pending state and clears the lock by writing a fresh revision; (3) if there's no prior good revision to roll back to (stuck on the very first install), you generally have to `helm uninstall` and reinstall, or as a last resort directly edit/delete the stuck revision Secret in that namespace — done carefully, since hand-editing release Secrets is easy to get wrong.

Interview favoriteThis exact scenario ("CI pipeline got killed mid-deploy, next run fails with this error") is one of the most commonly asked Helm production questions — know the mechanism (pending status lock), not just "restart it."

- [ ] A release is stuck in pending-install with zero prior revisions — why can't you just rollback?

IF ASKED, SAY THIS
Rollback needs a previously deployed revision to roll back to . If the very first install never completed successfully, revision 1 itself is the broken one — there's nothing earlier. The only paths forward are helm uninstall (clears it, name is free again) then reinstall, or in newer Helm versions retrying the upgrade/

Rollback needs a previously `deployed` revision to roll back *to*. If the very first install never completed successfully, revision 1 itself is the broken one — there's nothing earlier. The only paths forward are `helm uninstall` (clears it, name is free again) then reinstall, or in newer Helm versions retrying the upgrade/install against the same pending release (Helm will detect and resume/replace it in some versions) — but uninstall-and-retry is the dependable, version-agnostic answer to give in an interview.

- [ ] What does helm lint catch, and — just as important — what does it NOT catch?

IF ASKED, SAY THIS
helm lint ./mychart checks chart structure, required Chart.yaml fields, that templates render without Go-template errors, and that the rendered output is valid YAML. It does not validate against the live Kubernetes API schema (a rendered manifest can be syntactically valid YAML but reference a CRD that doesn't exist in

`helm lint ./mychart` checks chart structure, required Chart.yaml fields, that templates render without Go-template errors, and that the rendered output is valid YAML. It does **not** validate against the live Kubernetes API schema (a rendered manifest can be syntactically valid YAML but reference a CRD that doesn't exist in the target cluster, or use a deprecated/removed API version) — for that you need `helm install --dry-run` against the real cluster, or a schema-validation tool like `kubeconform` in CI.

- [ ] Your chart renders fine with default values but throws a template error only with the prod values file. How do you isolate it fast?

IF ASKED, SAY THIS
--debug prints the fully computed values (after merge) alongside the error, so you can immediately see what prod values actually resolved to versus what you assumed. -s / --show-only narrows rendering to a single template file so the error output isn't buried across 15 manifests. This combination — reproduce locally wi

```
helm template ./mychart -f values-prod.yaml --debug
# or narrow to one template:
helm template ./mychart -f values-prod.yaml -s templates/deployment.yaml --debug
```

`--debug` prints the fully computed values (after merge) alongside the error, so you can immediately see what prod values actually resolved to versus what you assumed. `-s` / `--show-only` narrows rendering to a single template file so the error output isn't buried across 15 manifests. This combination — reproduce locally with the exact failing values file, narrow to one template, read the computed values — is the fastest path, and is exactly the workflow interviewers want to hear rather than "I'd just look at the error."

- [ ] An upgrade with --atomic still left broken pods running — how is that possible?

IF ASKED, SAY THIS
--atomic rolls back the Helm release (reverting the manifests it manages) on failure, but it can't undo side effects a hook already caused (e.g. a pre-upgrade DB migration Job that already ran destructive SQL before the main upgrade failed), and it won't help if the failure is a false negative — e.g. your readiness pro

`--atomic` rolls back the *Helm release* (reverting the manifests it manages) on failure, but it can't undo side effects a hook already caused (e.g. a `pre-upgrade` DB migration Job that already ran destructive SQL before the main upgrade failed), and it won't help if the failure is a false negative — e.g. your readiness probe is misconfigured so Helm thinks pods never became ready and rolls back a deployment that was actually fine. Also, if the timeout is too short for a legitimately slow rollout (large image pull, slow DB migration), `--atomic` will trigger a rollback of a deploy that would have eventually succeeded — tuning `--timeout` realistically matters as much as setting the flag.

- [ ] You get a Kubernetes RBAC "forbidden" error mid-install that wasn't there in Helm 2 workflows. Why, and how do you fix it?

IF ASKED, SAY THIS
Because Helm 3 has no Tiller — every resource it tries to create is authorized against your (or your CI service account's) RBAC directly, not a shared superuser identity. If the identity running helm install lacks permission to create a given resource kind (common with CRDs, ClusterRoles, or cross-namespace resources),

Because Helm 3 has no Tiller — every resource it tries to create is authorized against *your* (or your CI service account's) RBAC directly, not a shared superuser identity. If the identity running `helm install` lacks permission to create a given resource kind (common with CRDs, ClusterRoles, or cross-namespace resources), the install fails partway with a clear "forbidden" error tied to that identity. Fix is standard Kubernetes RBAC: grant the CI service account (or user) the specific Role/ClusterRole+RoleBinding it needs — there's no Helm-specific workaround, and that's the point of the Helm 3 security model.

- [ ] How does Argo CD deploy a Helm chart, and how do you override values through it?

IF ASKED, SAY THIS

Argo CD runs the equivalent of helm template internally (it doesn't call helm install / upgrade) and applies the rendered manifests itself, which is why Argo CD tracks Helm releases as regular Argo Applications rather than through helm history — helm history and helm rollback aren't meaningful for a chart deployed thi

```
apiVersion: argoproj.io/v1alpha1
kind: Application
spec:
  source:
    repoURL: https://github.com/org/charts
    path: mychart
    targetRevision: main
    helm:
      valueFiles:
        - values-prod.yaml
      parameters:
        - name: image.tag
          value: "1.5.2"
  destination:
    server: https://kubernetes.default.svc
    namespace: prod
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

Argo CD runs the equivalent of `helm template` internally (it doesn't call `helm install` / `upgrade`) and applies the rendered manifests itself, which is why Argo CD tracks Helm releases as regular Argo *Applications* rather than through `helm history` — `helm history` and `helm rollback` aren't meaningful for a chart deployed this way; rollback instead means reverting the Git commit or targetRevision.

- [ ] What is Flux's HelmRelease CRD, and how does it differ from Argo CD's approach?

IF ASKED, SAY THIS

Flux's HelmRelease is a CRD reconciled by the helm-controller , which — unlike Argo CD — actually drives the real Helm SDK under the hood, so genuine Helm releases, history, and rollback semantics exist and helm history / helm get work as expected against clusters managed this way. It pairs with a HelmRepository or Git

Flux's `HelmRelease` is a CRD reconciled by the `helm-controller`, which — unlike Argo CD — actually drives the real Helm SDK under the hood, so genuine Helm releases, history, and rollback semantics exist and `helm history` / `helm get` work as expected against clusters managed this way. It pairs with a `HelmRepository` or `GitRepository` source CRD to know where the chart comes from, and supports automatic rollback on failed upgrades natively via the CRD's `remediation` field.

- [ ] What problem does Helmfile solve that plain Helm doesn't?

IF ASKED, SAY THIS

Plain Helm operates on one release at a time. Helmfile is a declarative manifest (helmfile.yaml) that describes many releases — across charts, repos, and environments — with shared values templating, environment-specific overrides, and dependency ordering (needs:), then applies them all with one helmfile apply / he

Plain Helm operates on one release at a time. Helmfile is a declarative manifest (`helmfile.yaml`) that describes *many* releases — across charts, repos, and environments — with shared values templating, environment-specific overrides, and dependency ordering (`needs:`), then applies them all with one `helmfile apply` / `helmfile sync` . It's the common answer when a platform has 20-plus microservices each with their own chart and you want one command (or one CI job) to bring an entire environment up to the declared state, with diffing built in via the same helm-diff plugin under the hood.

- [ ] Where do secrets belong in a Helm workflow — and where do they NOT belong?

IF ASKED, SAY THIS

They don't belong plaintext in values.yaml committed to Git, full stop — even though it's technically possible to reference .Values.dbPassword in a Secret manifest, that means the plaintext secret lives in your Git history and in the Helm release Secret (which itself is only as protected as your etcd encryption-at-rest

They don't belong plaintext in `values.yaml` committed to Git, full stop — even though it's technically possible to reference `.Values.dbPassword` in a Secret manifest, that means the plaintext secret lives in your Git history and in the Helm release Secret (which itself is only as protected as your etcd encryption-at-rest and RBAC on Secrets). Production patterns: External Secrets Operator or Sealed Secrets to keep the actual secret material in a vault (AWS Secrets Manager, HashiCorp Vault, Azure Key Vault) and have Kubernetes pull/decrypt it at apply time, with the chart only referencing a reference/pointer, never the value itself. SOPS-encrypted values files (decrypted just-in-time by the CD tool) are another common pattern.

- [ ] How would you enforce that only signed, approved charts get deployed in a regulated environment?

IF ASKED, SAY THIS

Combine: (1) chart signing at publish time (helm package --sign) with provenance files, (2) an admission controller or CI gate that runs helm verify / checks OCI artifact signatures (e.g. Cosign for OCI-stored charts) before allowing deploy, (3) a private, access-controlled registry rather than pulling directly from

Combine: (1) chart signing at publish time (`helm package --sign`) with provenance files, (2) an admission controller or CI gate that runs `helm verify` / checks OCI artifact signatures (e.g. Cosign for OCI-stored charts) before allowing deploy, (3) a private, access-controlled registry rather than pulling directly from public repos in prod pipelines, and (4) pinning exact chart versions (never a floating `~1.2` range) in whatever GitOps manifest references the chart, so what's deployed is always traceable to a specific, signed artifact.

- [ ] Scenario: a Helm upgrade just went out and prod is throwing 500s. Walk through exactly what you'd run, in order.

IF ASKED, SAY THIS
The key judgment point interviewers are listening for: restore service via rollback before root-causing, not the other way around.

1. `helm history <release> -n prod` — confirm the current revision and find the last `deployed` revision before it.
2. `helm rollback <release> <previous-revision> -n prod --wait --timeout 3m` — restore service first, diagnose second; don't debug live in a customer-impacting state if a known-good revision exists.
3. Verify: `kubectl get pods -n prod` , error rate back to baseline in your dashboard.
4. Only then diff what changed: `helm get values <release> --revision <bad-rev>` vs the prior one, or `helm diff` if installed, to find the actual change that caused the regression before attempting a fixed re-upgrade.

The key judgment point interviewers are listening for: restore service via rollback *before* root-causing, not the other way around.

- [ ] Scenario: leadership wants to migrate 40 services from raw kubectl apply manifests to Helm with zero downtime. How do you sequence it?

IF ASKED, SAY THIS
Do it incrementally, one service at a time, never a big-bang cutover: (1) build the chart to render output that is byte-for-byte (or functionally identical — same names, labels, selectors) equivalent to the current live manifests, verified via helm template diffed against kubectl get -o yaml for every resource; (2) hel

Do it incrementally, one service at a time, never a big-bang cutover: (1) build the chart to render output that is byte-for-byte (or functionally identical — same names, labels, selectors) equivalent to the current live manifests, verified via `helm template` diffed against `kubectl get -o yaml` for every resource; (2) `helm install` using `--dry-run` first, then a real install is actually an "adopt" problem — Helm won't take ownership of resources it didn't create, so you either delete-and-recreate under Helm (acceptable for stateless services with a rolling strategy) or use `helm.sh/resource-policy: keep` -style annotation tricks and `kubectl annotate` / `label` the existing resources with the ownership metadata Helm expects (`meta.helm.sh/release-name` , `meta.helm.sh/release-namespace`) so Helm adopts them without recreating; (3) prove it with the lowest-risk service first, then roll the pattern out; (4) only decommission the old raw-manifest CI path once every service is confirmed on Helm and has gone through at least one real upgrade cycle successfully.

- [ ] Scenario: 20 microservice charts share 80% identical boilerplate (labels, probes, security context). How do you reduce duplication without losing per-service flexibility?

IF ASKED, SAY THIS
Pull the shared logic into a library chart (labels, standard Deployment/Service scaffolding as named templates) that every service chart depends on, so a platform-wide change to, say, the standard security context happens in one place and propagates on the next helm dependency update + version bump. For values that dif

Pull the shared logic into a **library chart** (labels, standard Deployment/Service scaffolding as named templates) that every service chart depends on, so a platform-wide change to, say, the standard security context happens in one place and propagates on the next `helm dependency update` + version bump. For values that differ only by environment (not by service), layer `values-common.yaml` + `values-<env>.yaml` passed via multiple `-f` flags, orchestrated through Helmfile so the whole fleet's environment promotion is one command, while each service chart's own `values.yaml` still owns what's genuinely service-specific (replica count, resource requests, image name).

- [ ] Scenario: a chart works perfectly in staging but the exact same chart+values fails only in prod. What's your systematic checklist?

IF ASKED, SAY THIS
Confirm it's really the exact same chart version and values (diff the actual helm get values --revision output, don't assume). Cluster version/API skew: .Capabilities.KubeVersion differences can cause an API version referenced in a template to not exist in prod (e.g. an older/newer cluster). RBAC: the prod deploy ident

- Confirm it's really the exact same chart version and values (diff the actual `helm get values --revision` output, don't assume).
- Cluster version/API skew: `.Capabilities.KubeVersion` differences can cause an API version referenced in a template to not exist in prod (e.g. an older/newer cluster).
- RBAC: the prod deploy identity may have narrower permissions than staging's.
- Resource constraints: prod ResourceQuotas/LimitRanges rejecting requests that staging doesn't enforce.
- Admission controllers/policy engines (OPA Gatekeeper, Kyverno) active in prod but not staging, blocking on a policy staging never tested against.
- External dependencies the chart assumes exist (a specific StorageClass name, an existing Secret, a CRD) that were manually created in staging once and never captured as part of the chart's actual requirements.

Docker

- [ ] What is Docker and how does a container differ from a VM?

IF ASKED, SAY THIS
Docker runs applications in containers — isolated processes that share the host kernel and bundle the app with its dependencies. Unlike a VM, which virtualizes hardware and runs a full guest OS per instance, a container virtualizes the OS layer, so it's far lighter, starts in milliseconds, and packs more densely — at t

- Docker is a platform to build, ship, and run applications in containers. A container packages an app with its dependencies into a single isolated unit that runs on a shared host kernel.
- A VM virtualizes hardware and runs a full guest OS (its own kernel) on a hypervisor — heavyweight, slow to boot, GBs in size. A container virtualizes the OS and shares the host kernel — lightweight, starts in milliseconds, MBs in size.
- Containers give density and speed; VMs give stronger isolation. They're often combined: containers running inside VMs in the cloud.

- [ ] Explain the Docker architecture: client, daemon, and registry.

IF ASKED, SAY THIS
The Docker client sends commands over a REST API to the Docker daemon (dockerd), which builds images and manages containers, networks, and volumes — delegating low-level execution to containerd and runc. Registries like Docker Hub, ECR, or ACR store and distribute images that the daemon pulls and pushes.

- Docker client (`docker` CLI) sends commands to the Docker daemon (`dockerd`) over a REST API/socket.
- The daemon builds images, runs containers, and manages networks/volumes. It delegates actual container execution to containerd and runc (low-level OCI runtime).
- Registries (Docker Hub, ECR, ACR, GHCR) store and distribute images. `docker pull/push` moves images between the daemon and a registry.

- [ ] What are namespaces and cgroups?

IF ASKED, SAY THIS
Namespaces isolate what a container can see — separate PID, network, mount, UTS, IPC, and user views — while cgroups limit and account for what it can use, such as CPU, memory, and I/O. Namespaces provide isolation; cgroups provide resource control. Together they're the kernel foundation of containers.

- Namespaces provide isolation — each container gets its own view of resources: PID (processes), NET (network stack), MNT (filesystem mounts), UTS (hostname), IPC, and USER. This is why a container's `ps` only shows its own processes.
- cgroups (control groups) provide resource limiting and accounting — capping CPU, memory, and I/O per container. This is how `--memory` and `--cpus` are enforced.
- Together: namespaces decide what a container can see; cgroups decide how much it can use.

- [ ] What is the difference between an image and a container?

IF ASKED, SAY THIS
An image is a read-only template made of stacked filesystem layers plus metadata; a container is a runnable instance of an image with a thin writable layer on top. One image can spawn many containers — image is the blueprint, container is the running instance.

- An image is a read-only template — a stack of filesystem layers plus metadata (entrypoint, env, ports). It's the blueprint.
- A container is a running (or stopped) instance of an image with a thin writable layer on top. Many containers can run from one image.
- Analogy: image is the class, container is the object/instance.

- [ ] Explain image layers and the build cache.

IF ASKED, SAY THIS
Every Dockerfile instruction creates a cached, shareable layer. On rebuild Docker reuses layers up to the first change, then rebuilds the rest — so ordering matters. You keep builds fast by placing stable steps (dependency installs) early and volatile steps (copying app code) late, so dependency layers stay cached acro

- Each Dockerfile instruction (FROM, RUN, COPY...) creates a layer. Layers are cached and shared between images; identical layers are stored once.
- On rebuild, Docker reuses cached layers until the first changed instruction, then rebuilds everything after it. So instruction ORDER matters.
- Best practice: put rarely-changing steps first (install dependencies), and copy frequently-changing app code last, so dependency layers stay cached. Copy package manifests and install before copying source.

```
COPY package.json package-lock.json ./
RUN npm ci          # cached unless deps change
```

```
COPY . .            # changes often, kept last
```

- [ ] What's the difference between an image tag and a digest?

IF ASKED, SAY THIS
A tag is a mutable, friendly label that can be repointed to a different image, while a digest is an immutable SHA-256 hash of the exact image content. `latest` is just a default tag, not necessarily the newest build, so production deployments should pin images by digest for reproducibility and supply-chain safety.

- A tag (e.g. `app:1.2` or `app:latest`) is a mutable, human-friendly pointer — it can be moved to a different image later.
- A digest (`app@sha256:...`) is an immutable content hash that always refers to the exact same image bytes.
- `latest` is just a default tag, not 'the newest' — relying on it is risky. For reproducible, secure deployments, pin by digest.

```
docker pull nginx@sha256:abc123...   # immutable digest pin
```

- [ ] What is the difference between CMD and ENTRYPOINT?

IF ASKED, SAY THIS
ENTRYPOINT sets the executable that always runs; CMD supplies default arguments (or a default command alone). With both set, CMD becomes default args to ENTRYPOINT, and `docker run` arguments override CMD but not ENTRYPOINT. Always use the exec form so termination signals reach PID 1 for graceful shutdown.

- ENTRYPOINT defines the executable that always runs; CMD provides default arguments (or a default command if no ENTRYPOINT).
- When both are set, CMD's values become default args to ENTRYPOINT. Args passed to `docker run` override CMD but not ENTRYPOINT.
- Use exec form (`["nginx","-g","daemon off;"]`) not shell form, so signals (SIGTERM) reach the process directly for clean shutdown.

```
ENTRYPOINT ["python","app.py"]
CMD ["--port=8080"]   # default arg, overridable
```

- [ ] What is a multi-stage build and why use it?

IF ASKED, SAY THIS
A multi-stage build uses several FROM stages: you compile in a heavy builder stage, then copy only the final artifacts into a lean runtime stage. The shipped image excludes build tools and source, so it's dramatically smaller and has a reduced attack surface — especially with a distroless or alpine runtime base.

- A multi-stage build uses multiple FROM statements. You build/compile in a 'builder' stage with full toolchains, then COPY only the final artifacts into a slim runtime stage.
- Result: the final image excludes compilers, build deps, and source — much smaller and a smaller attack surface.
- Pair with a minimal/distroless base (`gcr.io/distroless`, `alpine`) for the runtime stage.

```
FROM golang:1.22 AS build
WORKDIR /src
COPY . . && RUN go build -o /app

FROM gcr.io/distroless/static
COPY --from=build /app /app
ENTRYPOINT ["/app"]
```

- [ ] How do you reduce Docker image size?

IF ASKED, SAY THIS
Start from a minimal base (alpine/distroless), use multi-stage builds to exclude compilers and dev dependencies, combine RUN commands and clean caches in the same layer, and use a .dockerignore to keep junk out of the build context. The key insight is that deleting files in a later layer doesn't shrink earlier ones, so

- Use a minimal base (alpine/distroless/slim). Use multi-stage builds to drop build tooling.
- Combine RUN steps and clean package caches in the same layer (e.g. `apt-get install ... && rm -rf /var/lib/apt/lists/*`), since deleting in a later layer doesn't shrink earlier ones.
- Add a .dockerignore to keep build context (node_modules, .git, tests) out of the image. Avoid installing unneeded packages.

What does .dockerignore do and why does it matter?

IF ASKED, SAY THIS
`.dockerignore` excludes files from the build context the same way `.gitignore` excludes from a repo. It speeds up builds by shrinking the context, keeps images smaller by avoiding copying things like .git and node_modules, and prevents accidentally baking secrets or .env files into the image — particularly when using

- `.dockerignore` excludes files/dirs from the build context sent to the daemon — like .gitignore for builds.
- Benefits: faster builds (smaller context), smaller images (no node_modules/.git copied in), and security (prevents accidentally copying secrets, .env, or credentials).
- Especially important with a broad `COPY . .`.

Explain Docker network drivers: bridge, host, none, overlay.

IF ASKED, SAY THIS
bridge is the default — containers get private IPs and you publish ports to the host; user-defined bridges add container DNS. host shares the host's network stack directly with no isolation or port mapping. none disables networking entirely. overlay spans multiple hosts for clustered/Swarm communication. You pick based

- bridge (default): containers get a private IP on a virtual bridge; you publish ports (`-p`) to reach them from the host. User-defined bridges add automatic DNS between containers.
- host: container shares the host's network namespace directly — no isolation, no port mapping, best performance.
- none: no networking at all — fully isolated.
- overlay: multi-host networking for Swarm/clustered setups — containers on different hosts communicate as if on one network.

```
docker run -d --network my-net --name api app
docker run --network host nginx   # shares host net
```

What is the difference between volumes and bind mounts?

IF ASKED, SAY THIS
Volumes are Docker-managed storage (under Docker's data dir) that's portable and the recommended way to persist and share data. Bind mounts map an explicit host directory into the container — ideal for development live-reload but coupled to the host's filesystem. tmpfs mounts live in memory and aren't persisted. Produc

- Volumes are managed by Docker (stored under /var/lib/docker/volumes). Portable, backup-friendly, the preferred way to persist data and share it between containers.
- Bind mounts map a specific host path into the container. Great for development (live code reload) but tightly coupled to host layout and a potential security risk.
- tmpfs mounts keep data in host memory only — non-persistent, for sensitive/scratch data.

```
docker run -v mydata:/var/lib/postgresql/data postgres  # volume
docker run -v $(pwd):/app node                          # bind mount
```

What is Docker Compose and when do you use it?

IF ASKED, SAY THIS
Docker Compose describes a multi-container app — services, networks, and volumes — in one YAML file and starts them together with `docker compose up`. It's the standard way to run multi-service stacks locally for development and testing, handling service DNS and startup order. For production-scale orchestration you'd u

- Compose defines multi-container applications in a single YAML file (`compose.yaml`) — services, networks, volumes — and brings them up together with `docker compose up`.
- Used for local dev and testing of multi-service apps (app + db + cache). It handles inter-service DNS and dependency ordering (depends_on).
- It's not an orchestrator for production scale — that's Kubernetes/Swarm.

What are Docker security best practices?

IF ASKED, SAY THIS
Run containers as a non-root user, drop Linux capabilities, and use a read-only root filesystem. Build from minimal/distroless images, scan them for CVEs, and pin bases by digest. Keep secrets out of images and ENV — inject at runtime from a secrets manager — and never mount the Docker socket into a container, since th

- Run as a non-root user (USER directive); never run containers as root unnecessarily. Drop capabilities and use `--read-only` root filesystem.

- Use minimal/distroless images and scan for CVEs (Trivy, Grype). Pin base images by digest.
- Never bake secrets into images or ENV — use secret managers or runtime injection. Don't mount the Docker socket into containers (root-equivalent escape).
- Enable content trust / image signing for supply-chain integrity.

```
USER 1000:1000
# docker run --read-only --cap-drop ALL myimage
```

- [ ] A container exits immediately after starting. How do you debug?

IF ASKED, SAY THIS
Start with `docker logs ` and `docker ps -a` for the exit code. A container only lives as long as its PID 1 process runs, so common causes are the main process exiting, a startup crash from missing config/env, or a process that daemonized into the background. To investigate, override the entrypoint with `doc

- Check logs first: `docker logs ` shows stdout/stderr and usually the crash reason. Use `docker ps -a` to see the exit code.
- Common causes: the main process finished (container lives only as long as PID 1 runs), a crash on startup (missing config/env), wrong CMD/ENTRYPOINT, or a foreground vs background issue (process daemonized and exited).
- Reproduce interactively: `docker run -it --entrypoint sh image` to poke around the filesystem and run the command manually.

```
docker logs <container>
docker run -it --entrypoint sh <image>
```

- [ ] Your Docker image builds successfully locally but when pushed to production Kubernetes, pods fail with ImagePullBackOff. How do you debug this?

IF ASKED, SAY THIS
ImagePullBackOff means K8s cannot pull the image. Follow this sequence: Step 1 - Get the exact error: kubectl describe pod POD-NAME and look at Events. The error message tells you exactly what failed: auth error, network error, image not found, etc. Step 2 - Verify the image exists: aws ecr describe-images --repository

ImagePullBackOff means K8s cannot pull the image. Follow this sequence:

Step 1 - Get the exact error: `kubectl describe pod POD-NAME` and look at Events. The error message tells you exactly what failed: auth error, network error, image not found, etc.

Step 2 - Verify the image exists: `aws ecr describe-images --repository-name payments-service --image-ids imageTag=1a2b3c4d`. Is the tag you are deploying actually in the registry?

Step 3 - Check pull secret: K8s needs credentials to pull from a private registry. Verify the imagePullSecret exists: `kubectl get secret ecr-pull-secret -n production`. ECR tokens expire every 12 hours - you need a cron job to refresh them. Or better, use the ECR credential helper or IRSA with the EKS IAM permissions.

Step 4 - Check node network: The node must be able to reach the registry endpoint. `kubectl debug node/NODE-NAME -it --image=alpine` and try `wget https://123456.dkr.ecr.ap-south-1.amazonaws.com`.

Step 5 - Check imagePullPolicy: If set to `Never`, K8s will never pull and will fail if the image is not locally cached. If set to `Always` and network is slow, it may timeout.

Real cause I have seen in production: The CI/CD pipeline pushed to ECR in us-east-1 but the EKS cluster is in ap-south-1. Different registry URL. The image literally did not exist in the right region.

- [ ] How do you reduce a Docker image from 2GB to under 200MB? Walk through your approach.

IF ASKED, SAY THIS
Image size reduction is a multi-step process I follow in order from most to least impactful: Step 1 - Multi-stage builds (biggest win): The build stage needs compilers, package managers, SDKs. The runtime stage needs NONE of that. A Java app only needs the JRE, not the JDK. A Go binary is statically compiled and needs

Image size reduction is a multi-step process I follow in order from most to least impactful:

Step 1 - Multi-stage builds (biggest win): The build stage needs compilers, package managers, SDKs. The runtime stage needs NONE of that. A Java app only needs the JRE, not the JDK. A Go binary is statically compiled and needs nothing except possibly libc.

Step 2 - Use a minimal base image:
- ubuntu: 77MB (not bad but has apt, bash, coreutils you do not need)
- alpine: 7MB (much smaller, uses musl libc - watch for compatibility issues)
- distroless: 20MB (no shell, no package manager, minimal attack surface - BEST for production)
- scratch: 0MB (for Go statically compiled binaries)

Step 3 - Combine RUN commands: Each RUN creates a layer. Multiple RUNs = multiple layers = wasted space even if you delete files in a later layer.
Bad: `RUN apt-get update` then `RUN apt-get install curl` then `RUN rm -rf /var/lib/apt/lists`

Good: `RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*`

Step 4 - Use .dockerignore: Without it, every file in the build context is sent to the Docker daemon. node_modules alone can be 500MB. Add .dockerignore first.

Step 5 - Remove build artifacts in the same layer: `RUN mvn package && rm -rf ~/.m2`

Step 6 - Use dive to inspect layers: `dive IMAGE` shows you exactly what each layer contains and what is wasting space.

Real example: Java Spring Boot app went from 750MB (openjdk:17) to 85MB (distroless/java17:nonroot) using multi-stage build.

- [ ] A container is consuming 100% CPU on the host and other containers are suffering. How do you identify it and what do you do immediately?

IF ASKED, SAY THIS
Production emergency response for container CPU runaway: Step 1 - Identify the culprit immediately: docker stats --no-stream shows CPU% per container. Or on the host: top -c and look for the container process name, or ps aux --sort=-%cpu | head -5 . Step 2 - In Kubernetes, identify the pod: kubectl top pods -A --sort-b

Production emergency response for container CPU runaway:

Step 1 - Identify the culprit immediately:
`docker stats --no-stream` shows CPU% per container. Or on the host: `top -c` and look for the container process name, or `ps aux --sort=-%cpu | head -5`.

Step 2 - In Kubernetes, identify the pod:
`kubectl top pods -A --sort-by=cpu | head -10`. Or in Grafana, check the CPU per pod panel.

Step 3 - Immediate mitigation:
If K8s: The pod should already be CPU-throttled by the resource limit. If it has no limit (never acceptable in production), patch it immediately:
`kubectl patch deployment myapp --patch='{"spec":{"template":{"spec":{"containers":[{"name":"myapp","resources":{"limits":{"cpu":"1000m"}}}]}}}}'`

If this is killing other pods, cordon the node to prevent new pods being scheduled there:
`kubectl cordon NODE-NAME`
Then drain and investigate: `kubectl drain NODE-NAME --ignore-daemonsets`

Step 4 - Root cause investigation:
- Get a CPU profile: for Java use jstack or async-profiler. For Python use py-spy. For Go use pprof.
- Check application logs for any infinite loop indicators (same log line repeated millions of times).
- Check if a recent deploy introduced the runaway code.

Long-term fix: EVERY container must have CPU limits in production. This is non-negotiable. Use OPA to reject deployments without resource limits.

- [ ] What actually is Docker, in one sentence you can lead with?

IF ASKED, SAY THIS
Docker is a standardized shipping container for software — it packages an application, all its dependencies, the runtime, and config into one image that runs identically on a laptop, in test, and in production.

Before Docker, 'works on my machine' was a real failure mode: the app depended on specific library versions present on the dev laptop but not the server. Docker removes that class of bug entirely by making the environment part of the shipped artifact, not an assumption about the target host.

- [ ] Image vs container vs registry — give the clean definitions and a memorable analogy for each.

IF ASKED, SAY THIS
Image = read-only blueprint (a class definition). Container = a running instance of that image (an object instantiated from the class). Registry = where images are stored and distributed (npm registry, but for images).

- **Image** — a read-only blueprint, built from a Dockerfile, stored in layers, immutable once built. In production: always use immutable tags like `v1.2.3` or a git SHA — never `:latest`.
- **Container** — a running instance of an image, with its own filesystem, network, and process space, but sharing the host OS kernel. Containers are ephemeral — never store important state inside one.
- **Registry** — a server that stores and distributes images (Docker Hub is public, ECR is AWS, ACR is Azure, Artifactory is enterprise). In enterprise settings, a private registry with vulnerability scanning before images reach production is standard.

- [ ] Write a production-grade multi-stage Dockerfile and explain the 5 rules it follows.

IF ASKED, SAY THIS
Multi-stage build: a fat builder stage compiles the app, then only the compiled artifact is copied into a minimal distroless/Alpine runtime stage — so the shipped image carries none of the build toolchain.

5 Dockerfile rules to remember: 1) Multi-stage builds — build image is fat, runtime image is tiny. 2) Copy dependency manifests (pom.xml/package.json) before source code, so the dependency layer stays cached across rebuilds. 3) Run as a non-root user (`USER 1001`) before `ENTRYPOINT`. 4) Use

`.dockerignore` to exclude node_modules/.git/test files from the build context. 5) Use distroless or Alpine base images — smaller means less attack surface.

```
FROM maven:3.9-openjdk-17 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -q
COPY src ./src
RUN mvn package -DskipTests -q

FROM gcr.io/distroless/java17-debian11
WORKDIR /app
COPY --from=builder /app/target/myapp.jar .
USER 1001
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "myapp.jar"]
```

- [ ] Give the essential Docker commands for debugging a live container, plus what the common exit codes mean.

IF ASKED, SAY THIS
docker logs -f, docker exec -it ... sh, docker inspect, and docker stats cover 90% of live debugging. Exit code 137 = OOMKilled (raise the memory limit); 127 = binary not found (wrong ENTRYPOINT path); 1 = application-level error, check logs.

The exit-code check is the fastest triage step — before reading a single log line, `docker inspect` or `kubectl describe` tells you the exit code, and that alone often points straight at the fix.

```
docker logs --tail=100 -f CONTAINER      # live log stream
docker exec -it CONTAINER /bin/sh        # shell inside container
docker inspect CONTAINER                 # full config, env vars, mounts
docker stats CONTAINER                   # live CPU and memory
docker system df                         # disk usage by images and volumes
docker system prune -a                   # clean up everything unused

# Exit codes
# 137 = OOM killed - increase memory limit
# 127 = binary not found - wrong ENTRYPOINT path
# 1   = application error - check logs
```

- [ ] What Docker best practices matter most in a regulated environment?

IF ASKED, SAY THIS
Minimal, purpose-built base images (distroless or slim variants) to shrink attack surface; multi-stage builds so build tools and secrets never ship in the final image; running as a non-root user; pinning base image versions or digests — never latest — for reproducibility and to avoid unexpected drif

Minimal, purpose-built base images (distroless or slim variants) to shrink attack surface; multi-stage builds so build tools and secrets never ship in the final image; running as a non-root user; pinning base image versions or digests — never `latest` — for reproducibility and to avoid unexpected drift; automated vulnerability scanning (Trivy, Grype, or the registry's built-in scanner) gating the pipeline on critical CVEs; and signed images with provenance/SBOM generation, which is increasingly expected in financial-services supply-chain security audits.

Terraform

- [ ] What is Terraform and how does declarative IaC differ from imperative?

> **IF ASKED, SAY THIS**
> Terraform is a declarative Infrastructure-as-Code tool: you describe the desired end state in HCL, and Terraform figures out the actions to reach it. Imperative tools (scripts, much of Ansible) describe the steps . Declarative means Terraform reconciles current state to desired state — it computes the diff (plan) and a

Terraform is a declarative Infrastructure-as-Code tool: you describe the *desired end state* in HCL, and Terraform figures out the actions to reach it. **Imperative** tools (scripts, much of Ansible) describe the *steps*. Declarative means Terraform reconciles current state to desired state — it computes the diff (plan) and applies only what's needed.

> **PRACTICAL NOTE**
> Terraform uses
> **PROVIDERS**
> (AWS, Azure, Kubernetes, etc.) that translate HCL resources into API calls. It's cloud-agnostic with a huge provider ecosystem and an explicit
> **PLAN/APPLY**
> workflow. The declarative model gives idempotency and a reviewable diff before any change.

- [ ] Walk through the core workflow: init, plan, apply, destroy.

> **IF ASKED, SAY THIS**
> terraform init — downloads providers and configures the backend (state).

- `terraform init` — downloads providers and configures the backend (state).
- `terraform plan` — refreshes state, computes the diff between desired config and reality, and shows what it *would* change. No changes made.
- `terraform apply` — executes the plan to reach the desired state (prompts for approval, or takes a saved plan file).
- `terraform destroy` — tears down everything in the state.

> **PRACTICAL NOTE**
> In CI, run `plan` on the PR (review the diff) and `apply` on merge, ideally against a saved plan file so apply does exactly what was reviewed. `terraform validate` and `fmt` belong in the lint stage. Always read the plan — especially anything marked
> **DESTROY/REPLACE**
> .

- [ ] Resources vs data sources — what's the difference?

> **IF ASKED, SAY THIS**
> A resource is something Terraform creates and manages (it goes in state and Terraform owns its lifecycle). A data source reads existing information that Terraform does not manage — e.g. look up an existing VPC, the latest AMI, or another team's output — to use as input. Resources write; data sources read.

A **resource** is something Terraform *creates and manages* (it goes in state and Terraform owns its lifecycle). A **data source** *reads* existing information that Terraform does **not** manage — e.g. look up an existing VPC, the latest AMI, or another team's output — to use as input. Resources write; data sources read.

> **PRACTICAL NOTE**
> Use data sources to reference things managed elsewhere (another Terraform state via `terraform_remote_state`, or resources created outside Terraform) without taking ownership. A common pattern: a data source finds the latest AMI ID so you don't hardcode it.

- [ ] How do you configure providers, version constraints, and aliases?

> **IF ASKED, SAY THIS**
> Declare providers and pin versions in a required_providers block (and pin Terraform itself with required_version) so builds are reproducible. Provider aliases let you configure multiple instances of the same provider — e.g. two AWS Regions or two accounts — and target resources at a specific alias with provider = aws.

Declare providers and pin versions in a `required_providers` block (and pin Terraform itself with `required_version`) so builds are reproducible. **Provider aliases** let you configure multiple instances of the same provider — e.g. two AWS Regions or two accounts — and target resources at a specific alias with `provider = aws.useast1`.

> **PRACTICAL NOTE**
> Pin versions (use `~>` for safe minor upgrades) and commit the `.terraform.lock.hcl` lock file so everyone uses identical provider versions. Aliases are how you do multi-Region/multi-account deployments in one configuration.

Terraform vs CloudFormation vs Pulumi vs Ansible — when which?

IF ASKED, SAY THIS
Terraform — declarative, cloud-agnostic, huge provider ecosystem, explicit state. The de-facto multi-cloud IaC standard.

- **Terraform** — declarative, cloud-agnostic, huge provider ecosystem, explicit state. The de-facto multi-cloud IaC standard.
- **CloudFormation** — AWS-native, no state file to manage (AWS tracks it), day-one AWS feature support; AWS-only.
- **Pulumi / CDK** — define infra in a real programming language (loops, types); good for complex logic.
- **Ansible** — primarily configuration management/provisioning (imperative-ish); complements rather than replaces IaC.

PRACTICAL NOTE
Terraform for multi-cloud and a mature module ecosystem; CloudFormation/CDK if you're all-in on AWS and want native integration; Pulumi when you want full programming-language power; Ansible for config management on top of provisioned infra.

What does the lifecycle meta-argument do (create_before_destroy, prevent_destroy, ignore_changes)?

IF ASKED, SAY THIS
create_before_destroy — make the replacement before deleting the old one (avoids downtime when a resource must be replaced).

- **create_before_destroy** — make the replacement before deleting the old one (avoids downtime when a resource must be replaced).
- **prevent_destroy** — guard a critical resource so an accidental destroy fails fast.
- **ignore_changes** — stop Terraform from reverting specific attributes that something else legitimately modifies (e.g. an autoscaling-adjusted desired_count, or tags set by another system).

PRACTICAL NOTE
`create_before_destroy` is key for zero-downtime replacement but needs the resource to allow two to exist briefly (naming/uniqueness). `ignore_changes` prevents endless plan churn when an external system mutates a managed attribute. `prevent_destroy` protects databases/state buckets.

What is Terraform state and why does it exist?

IF ASKED, SAY THIS
State is Terraform's record (a JSON file) mapping your configuration to the real resources it manages — resource IDs, attributes, and dependencies. Terraform needs it to know what it already created (so it can compute diffs), to track metadata, and to map config names to actual cloud resource IDs. Without state, Terraf

State is Terraform's record (a JSON file) mapping your configuration to the real resources it manages — resource IDs, attributes, and dependencies. Terraform needs it to know what it already created (so it can compute diffs), to track metadata, and to map config names to actual cloud resource IDs. Without state, Terraform couldn't tell an update from a create.

PRACTICAL NOTE
State can drift from reality (manual changes) — `plan` /refresh reconciles. State contains
SENSITIVE VALUES IN PLAINTEXT
(passwords, keys), so it must be stored securely. Never edit state by hand; use `terraform state` subcommands.

How do you set up a remote backend with locking on AWS, and why?

IF ASKED, SAY THIS
Use the S3 backend for state storage plus a DynamoDB table for state locking , so a team shares one durable, locked state instead of racing on local files.

Use the **S3 backend** for state storage plus a **DynamoDB table for state locking**, so a team shares one durable, locked state instead of racing on local files.

```
terraform {
  backend "s3" {
    bucket         = "mycorp-tfstate"
    key            = "prod/network.tfstate"
    region         = "us-east-1"
    dynamodb_table = "tf-locks"
    encrypt        = true
  }
}
```

PRACTICAL NOTE
Harden it: enable
S3 VERSIONING

(recover a corrupted/last-good state), enable encryption (SSE-KMS), restrict the bucket with a policy + Block Public Access, and authenticate via an assumed role/OIDC (not static keys). Isolate state per environment (separate keys/buckets for dev/stage/prod). Locking prevents two concurrent applies from corrupting state. (Newer Terraform can also use native S3 lockfile-based locking.)

- [ ] What does state locking prevent, and what do you do about a stale lock?

IF ASKED, SAY THIS
Locking prevents concurrent writes — two engineers (or a person and a pipeline) running apply at once, which would corrupt state. With the S3/DynamoDB backend, the lock is a DynamoDB item held for the duration of the operation. If a process dies mid-run, a stale lock can remain; you clear it with terraform force-unlock

Locking prevents **concurrent writes** — two engineers (or a person and a pipeline) running `apply` at once, which would corrupt state. With the S3/DynamoDB backend, the lock is a DynamoDB item held for the duration of the operation. If a process dies mid-run, a **stale lock** can remain; you clear it with `terraform force-unlock <LOCK_ID>` — but only after confirming no apply is actually running.

PRACTICAL NOTE
Never `force-unlock` blindly — if an apply is genuinely in progress you'll corrupt state. Serialize applies in CI (one at a time per state) so locks rarely collide. The lock is why a remote backend is mandatory for teams.

- [ ] How do you bring an existing (manually-created) resource under Terraform management?

IF ASKED, SAY THIS
Write the resource configuration to match it, then import it into state: either the classic terraform import aws_instance.web i-0abc123 , or a declarative import block (Terraform 1.5+) that lets you plan the import and even generate config. After import, run plan and reconcile the config until the diff is empty (no cha

Write the resource configuration to match it, then **import** it into state: either the classic `terraform import aws_instance.web i-0abc123`, or a declarative **import block** (Terraform 1.5+) that lets you plan the import and even generate config. After import, run `plan` and reconcile the config until the diff is empty (no changes), proving config matches reality.

PRACTICAL NOTE
Import maps a real resource to a state address — it does
NOT
write your HCL for you (the import block can generate a starting config). The goal is a no-op plan after import. Importing complex resources (with many sub-resources) is tedious — do it carefully, one at a time, and verify the empty diff.

- [ ] What are the key state commands, and what are moved blocks for?

IF ASKED, SAY THIS
moved blocks (1.1+) declare a refactor (old address → new address) in code , so Terraform updates state automatically and reviewers see the intent — safer than manual state mv .

- `terraform state list` — list managed resources.
- `terraform state show ADDR` — inspect one resource's state.
- `terraform state mv` — rename/move a resource in state (e.g. after refactoring into a module) without destroy/recreate.
- `terraform state rm` — stop managing a resource without destroying it.
- `terraform state pull/push` — read/write raw state (use with care).

moved blocks (1.1+) declare a refactor (old address → new address) *in code*, so Terraform updates state automatically and reviewers see the intent — safer than manual `state mv`.

PRACTICAL NOTE
Prefer
MOVED BLOCKS
over imperative `state mv` for refactors — they're version-controlled, reviewable, and run automatically for everyone. `state rm` is how you hand a resource off to another config or stop managing it without deleting.

- [ ] What is configuration drift, and how does Terraform handle it?

IF ASKED, SAY THIS
Drift is when the real infrastructure diverges from what's in state/config — usually because someone changed it manually in the console. On plan , Terraform refreshes state against reality and shows the drift as a diff; apply reverts it back to the declared config. terraform plan -refresh-only lets you inspect/accept d

Drift is when the real infrastructure diverges from what's in state/config — usually because someone changed it manually in the console. On `plan`, Terraform **refreshes** state against reality and shows the drift as a diff; `apply` reverts it back to the declared config. `terraform plan -refresh-only` lets you inspect/accept drift into state without changing infra.

PRACTICAL NOTE

Prevent drift by making Terraform the sole owner (no manual changes), enforcing it via guardrails/SCPs and PR-only workflows, and running scheduled drift-detection plans in CI. `ignore_changes` is the escape valve for attributes another system legitimately manages.

- [ ] State contains secrets in plaintext. How do you secure it?

IF ASKED, SAY THIS
Treat state as sensitive: store it in an encrypted remote backend (S3 with SSE-KMS), lock down access with tight IAM/bucket policies + Block Public Access, enable versioning, and never commit state to git . Limit who can read the backend (state access ≈ secret access). Mark sensitive variables/outputs with sensitive =

Treat state as sensitive: store it in an **encrypted remote backend** (S3 with SSE-KMS), lock down access with tight IAM/bucket policies + Block Public Access, enable versioning, and **never commit state to git**. Limit who can read the backend (state access ≈ secret access). Mark sensitive variables/outputs with `sensitive = true` so they're redacted from CLI output (though still present in state).

PRACTICAL NOTE
Because resource attributes like DB passwords land in state, anyone with backend read access can extract them. Minimize secrets in Terraform-managed resources where possible (generate at runtime, pull from a vault), and gate state access as tightly as production credentials.

- [ ] What is a Terraform module, and how is it structured?

IF ASKED, SAY THIS
A module is a reusable, encapsulated group of resources — a folder of .tf files with input variables , resources , and outputs . The root module is your working directory; it calls child modules with a module block. Modules let you package a pattern (e.g. "a standard VPC" or "a tagged S3 bucket") once and reuse it ever

A **module** is a reusable, encapsulated group of resources — a folder of `.tf` files with **input variables**, **resources**, and **outputs**. The **root module** is your working directory; it calls **child modules** with a `module` block. Modules let you package a pattern (e.g. "a standard VPC" or "a tagged S3 bucket") once and reuse it everywhere.

PRACTICAL NOTE
Good modules are small, single-purpose, and parameterized — they expose what callers need via variables and return useful values via outputs, hiding internal detail. Conventional files: `main.tf` , `variables.tf` , `outputs.tf` , `versions.tf` .

- [ ] How do you pass data between modules and compose them?

IF ASKED, SAY THIS
A module exposes results via output blocks; the caller references them as module.NAME.OUTPUT and passes them as inputs to another module. You compose by wiring one module's outputs into another's variables (e.g. the network module outputs subnet IDs, which the compute module takes as input). Terraform builds the depend

A module exposes results via `output` blocks; the caller references them as `module.NAME.OUTPUT` and passes them as inputs to another module. You compose by wiring one module's outputs into another's variables (e.g. the network module outputs subnet IDs, which the compute module takes as input). Terraform builds the dependency graph from these references automatically.

PRACTICAL NOTE
Keep coupling explicit through inputs/outputs rather than reaching into a module's internals. Avoid deeply nested module trees — they get hard to reason about. For cross-*state* sharing (separate Terraform configs), use the `terraform_remote_state` data source or published outputs.

- [ ] How do you version and source modules safely?

IF ASKED, SAY THIS
Reference modules from a source (local path, Git, or a registry) and pin a version . For registry/Git modules use a version constraint or a tagged ref so an upstream change can't silently alter your infra:

Reference modules from a **source** (local path, Git, or a registry) and **pin a version**. For registry/Git modules use a version constraint or a tagged ref so an upstream change can't silently alter your infra:

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"
  # ...inputs
}
```

PRACTICAL NOTE

Never reference an unpinned `main` branch for production modules — pin a tag/version for reproducibility, then upgrade deliberately and review the plan. Local modules (`./modules/...`) are versioned with the repo. Publish shared internal modules to a private registry for reuse and governance.

- [ ] count vs for_each on modules and resources — which and why?

IF ASKED, SAY THIS
Both create multiple instances. count indexes by number ([0] , [1]) — simple, but removing a middle item renumbers the rest, causing Terraform to destroy/recreate unrelated instances. for_each keys by a stable string (from a map/set) — adding/removing one item only affects that key. Prefer for_each whenever items hav

Both create multiple instances. **count** indexes by number (`[0]` , `[1]`) — simple, but removing a middle item **renumbers** the rest, causing Terraform to destroy/recreate unrelated instances. **for_each** keys by a stable string (from a map/set) — adding/removing one item only affects that key. Prefer `for_each` whenever items have stable identities.

PRACTICAL NOTE
The `count` renumbering trap is a classic production gotcha — deleting the 2nd of 5 subnets shouldn't recreate subnets 3–5, but with `count` it does. Use `count` only for true on/off (0 or 1) or genuinely index-independent lists; use `for_each` for named collections.

- [ ] How do variables work, including types, validation, and precedence?

IF ASKED, SAY THIS
variable blocks declare typed inputs (string, number, bool, list, map, object) with optional defaults and validation rules. Values come from (highest to lowest precedence): CLI -var / -var-file , then *.auto.tfvars / terraform.tfvars , then environment variables (TF_VAR_name), then the declared default.

`variable` blocks declare typed inputs (string, number, bool, list, map, object) with optional defaults and **validation** rules. Values come from (highest to lowest precedence): CLI `-var` / `-var-file` , then `*.auto.tfvars` / `terraform.tfvars` , then environment variables (`TF_VAR_name`), then the declared default.

PRACTICAL NOTE
Use
TYPE CONSTRAINTS + VALIDATION
to fail fast on bad input (e.g. restrict an environment var to dev/stage/prod). Keep environment-specific values in per-env `.tfvars` files. Mark secrets `sensitive = true` . Don't put secrets in committed tfvars — inject via env vars or a vault.

- [ ] Locals vs variables — when do you use each?

IF ASKED, SAY THIS
Variables are inputs from outside the module (the caller sets them). Locals are internal named expressions computed within the config — derived values, repeated expressions, or merged tags — that callers don't set. Use a variable when someone should pass it in; use a local to DRY up a computed or repeated value.

Variables are *inputs* from outside the module (the caller sets them). **Locals** are *internal* named expressions computed within the config — derived values, repeated expressions, or merged tags — that callers don't set. Use a variable when someone should pass it in; use a local to DRY up a computed or repeated value.

PRACTICAL NOTE
Common pattern: a `local.common_tags` map merged into every resource's tags, or a `local.name_prefix` built from environment + project. Locals make configs readable and keep one source of truth for derived values.

- [ ] Explain for expressions, conditionals, and dynamic blocks.

IF ASKED, SAY THIS
for expressions — transform/filter collections: [for s in var.subnets : s.id if s.public] (list) or {for k,v in var.m : k => upper(v)} (map).

- **for expressions** — transform/filter collections: `[for s in var.subnets : s.id if s.public]` (list) or `{for k,v in var.m : k => upper(v)}` (map).
- **Conditionals** — ternary: `var.env == "prod" ? 3 : 1` to vary counts/sizes by environment.
- **dynamic blocks** — generate repeated nested blocks (e.g. multiple ingress rules in a security group) from a collection, instead of hand-writing each.

PRACTICAL NOTE
These keep configs DRY and data-driven.
DYNAMIC
blocks are powerful but overusing them hurts readability — sometimes explicit blocks are clearer. `for` + `for_each` together let you turn a list of inputs into a keyed set of resources.

- [ ] Outputs and sensitive outputs — what are they for?

IF ASKED, SAY THIS
output blocks expose values from a module/config — to display after apply, to feed another module (module.x.output), or to share across states via terraform_remote_state . Marking an output sensitive = true redacts it from CLI/plan output so secrets aren't printed.

`output` blocks expose values from a module/config — to display after apply, to feed another module (`module.x.output`), or to share across states via `terraform_remote_state` . Marking an output `sensitive = true` redacts it from CLI/plan output so secrets aren't printed.

PRACTICAL NOTE
Outputs are the public interface of a module — expose what callers legitimately need (IDs, ARNs, endpoints), not internals. Sensitive outputs are still stored in state in plaintext; the flag only suppresses display, so secure the state regardless.

- [ ] What are Terraform workspaces, and what's the caveat for managing environments?

IF ASKED, SAY THIS
Workspaces let one configuration hold multiple independent states (e.g. default , dev , prod) under the same backend, switched with terraform workspace select . They're handy for lightweight per-environment or per-feature state separation using the same code.

Workspaces let one configuration hold multiple independent states (e.g. `default` , `dev` , `prod`) under the same backend, switched with `terraform workspace select` . They're handy for lightweight per-environment or per-feature state separation using the same code.

PRACTICAL NOTE
CAVEAT:
workspaces share the same backend and code, so they're a weak isolation boundary — easy to apply to the wrong env, and they don't separate credentials/blast radius. Many teams prefer
SEPARATE DIRECTORIES/STATE PER ENVIRONMENT
(or Terragrunt) for prod isolation. Use workspaces for ephemeral/feature environments, not as the only separation between dev and prod.

- [ ] Design a CI/CD pipeline for Terraform.

IF ASKED, SAY THIS
On a pull request: fmt -check , validate , a security/policy scan (checkov/tfsec/OPA/Sentinel), then terraform plan posted to the PR for review. On merge to main: apply the saved plan (so apply == reviewed plan), serialized so only one apply runs per state. Authenticate to the cloud via OIDC (no stored keys), use a rem

On a pull request: `fmt -check` , `validate` , a security/policy scan (checkov/tfsec/OPA/Sentinel), then `terraform plan` posted to the PR for review. On merge to main: `apply` the **saved plan** (so apply == reviewed plan), serialized so only one apply runs per state. Authenticate to the cloud via **OIDC** (no stored keys), use a remote locked backend, and isolate state per environment with manual approval before prod.

PRACTICAL NOTE
Key practices: plan-on-PR / apply-on-merge, apply the exact reviewed plan file, policy-as-code gates before apply, OIDC auth, and one-apply-at-a-time per state to avoid lock contention. Tools like
ATLANTIS
or Terraform Cloud/Enterprise automate the plan-comment-apply workflow.

- [ ] What problem does Terragrunt solve?

IF ASKED, SAY THIS
Terragrunt is a thin wrapper that reduces repetition in large Terraform setups: it keeps your backend and provider config DRY (define once, generate per-module), makes it easy to run the same modules across many environments/accounts, manages dependencies between components, and can run commands across many modules at

Terragrunt is a thin wrapper that reduces repetition in large Terraform setups: it keeps your backend and provider config DRY (define once, generate per-module), makes it easy to run the same modules across many environments/accounts, manages dependencies between components, and can run commands across many modules at once.

PRACTICAL NOTE
It shines when you have many environments/accounts and don't want to copy backend blocks and provider config into every folder. Plain Terraform plus good module design covers many cases; reach for Terragrunt when the env/account matrix gets large and repetitive.

- [ ] How do you enforce policy and catch problems before apply?

IF ASKED, SAY THIS
Use policy-as-code and scanning in the pipeline: Sentinel (Terraform Cloud/Enterprise) or OPA/Conftest to enforce rules on the plan (e.g. "no public S3", "only approved instance types", "mandatory tags"), and scanners like checkov / tfsec / Terrascan for misconfig/security issues. Gate apply on these passing.

Use **policy-as-code** and scanning in the pipeline: **Sentinel** (Terraform Cloud/Enterprise) or **OPA/Conftest** to enforce rules on the plan (e.g. "no public S3", "only approved instance types", "mandatory tags"), and scanners like **checkov / tfsec / Terrascan** for misconfig/security issues. Gate `apply` on these passing.

PRACTICAL NOTE
This shifts governance left — catching a public bucket or an over-broad IAM policy in the plan, before it's created. Combine with required-tags validation and cost estimation (Infracost) so reviewers see security *and* cost impact on every PR.

- [ ] How does Terraform determine dependencies, and when do you need depends_on?

IF ASKED, SAY THIS
Terraform builds a dependency graph automatically from references — if resource B uses aws_x.a.id , B implicitly depends on A and is created after it. You only need explicit depends_on when there's a dependency Terraform can't see from references (e.g. an IAM policy must exist before an action works, but the resource d

Terraform builds a **dependency graph** automatically from references — if resource B uses `aws_x.a.id`, B implicitly depends on A and is created after it. You only need explicit `depends_on` when there's a dependency Terraform **can't see** from references (e.g. an IAM policy must exist before an action works, but the resource doesn't reference it directly).

PRACTICAL NOTE
Prefer implicit dependencies (references) over `depends_on` — they're more precise. Overusing `depends_on` serializes things unnecessarily and can hide design issues. Use it sparingly for hidden ordering requirements.

- [ ] Why are provisioners a last resort?

IF ASKED, SAY THIS
Provisioners (local-exec , remote-exec) run scripts during create/destroy. They're a last resort because they break Terraform's declarative model: they're not tracked in state, not idempotent, fail unpredictably, and a failed provisioner taints the resource. Prefer cloud-native mechanisms — user data/cloud-init, AMIs

Provisioners (`local-exec` , `remote-exec`) run scripts during create/destroy. They're a last resort because they break Terraform's declarative model: they're not tracked in state, not idempotent, fail unpredictably, and a failed provisioner taints the resource. Prefer cloud-native mechanisms — user data/cloud-init, AMIs baked with Packer, or config management — over provisioners.

PRACTICAL NOTE
If you must use one, understand it runs once at create (or with triggers) and isn't reconciled afterward. For bootstrapping instances, `user_data` is the declarative path; for app config, use a config-management tool. Provisioners are escape hatches, not architecture.

- [ ] How do you handle secrets in Terraform safely?

IF ASKED, SAY THIS
Never hardcode secrets in .tf or commit them in tfvars. Pull them at run time from a secret store (Vault provider, AWS Secrets Manager / SSM data sources) or inject via TF_VAR_ environment variables from the CI secret store. Mark variables/outputs sensitive . Critically, remember secrets still land in state in plaintex

Never hardcode secrets in `.tf` or commit them in tfvars. Pull them at run time from a secret store (Vault provider, AWS Secrets Manager / SSM data sources) or inject via `TF_VAR_` environment variables from the CI secret store. Mark variables/outputs `sensitive`. Critically, remember secrets still land in **state in plaintext** — so secure the remote backend like a vault.

PRACTICAL NOTE
Best is to let Terraform reference a secret rather than own it (e.g. create the secret container, let the app fetch the value at runtime), minimizing what lands in state. Generate random secrets with the `random` provider and store them in Secrets Manager rather than printing them.

- [ ] Walk through common Terraform errors and how you fix them.

IF ASKED, SAY THIS
State lock errors — another apply running, or a stale lock → wait, or force-unlock only if truly stale.

- **State lock errors** — another apply running, or a stale lock → wait, or `force-unlock` only if truly stale.
- **"Resource already exists"** — created outside Terraform or in another state → `import` it.
- **Cycle errors** — circular dependencies → break the cycle (refactor references, sometimes split resources).
- **for_each with unknown/computed values** — keys must be known at plan time → derive keys from static input, not computed attributes.
- **Provider drift / inconsistent plan** — refresh, or `ignore_changes` for externally-managed attributes.

PRACTICAL NOTE

Use `TF_LOG=DEBUG` for deep diagnosis, `terraform plan` to understand intent before applying, and `state` subcommands to repair mismatches. The "for_each can't use computed values for keys" error trips up many — restructure so keys come from known inputs.

- [ ] How do you refactor Terraform safely (rename/move resources) without downtime?

IF ASKED, SAY THIS

Use moved blocks to declare old-address → new-address in code, so Terraform updates state instead of destroy/recreate, and the change is reviewable in the PR. For replacements that must happen, use create_before_destroy so the new resource exists before the old is removed. For pulling a resource into a module, moved (o

Use **moved blocks** to declare old-address → new-address in code, so Terraform updates state instead of destroy/recreate, and the change is reviewable in the PR. For replacements that must happen, use `create_before_destroy` so the new resource exists before the old is removed. For pulling a resource into a module, `moved` (or `terraform state mv`) preserves the existing resource.

PRACTICAL NOTE

Always check the `plan` for unexpected
DESTROY/REPLACE
before applying a refactor — that's how you catch a rename that would have recreated a database. `moved` blocks are the modern, version-controlled, team-safe way to refactor (vs imperative `state mv` that only runs on one machine).

- [ ] How do you share outputs across separate Terraform states?

IF ASKED, SAY THIS

Use the terraform_remote_state data source to read another configuration's published outputs (e.g. the networking state exposes vpc_id and subnet IDs, which the application state consumes). The producing config must declare those values as output s; the consumer reads them from the remote backend.

Use the `terraform_remote_state` data source to read another configuration's published outputs (e.g. the networking state exposes `vpc_id` and subnet IDs, which the application state consumes). The producing config must declare those values as `output`s; the consumer reads them from the remote backend.

PRACTICAL NOTE

This decouples large systems into separately-applied states (network, data, app) while still wiring them together. Alternatives: publish outputs to SSM Parameter Store and read them via a data source — looser coupling that also works for non-Terraform consumers. Keep cross-state dependencies explicit and minimal.

- [ ] How does Terraform decide apply order, and what breaks when a dependency is missing?

IF ASKED, SAY THIS

Terraform builds a directed acyclic graph (DAG) from every resource's references. Any interpolation of one resource's attribute inside another (aws_instance.web.subnet_id = aws_subnet.a.id) creates an implicit dependency edge; the graph walker then creates/updates in topological order and, where nothing depends on an

Terraform builds a directed acyclic graph (DAG) from every resource's references. Any interpolation of one resource's attribute inside another (`aws_instance.web.subnet_id = aws_subnet.a.id`) creates an **implicit** dependency edge; the graph walker then creates/updates in topological order and, where nothing depends on anything else, in parallel (default `-parallelism=10`).

When two resources are related only operationally — no attribute reference exists, e.g. an IAM policy that must exist before a Lambda starts invoking it via an unrelated trigger — Terraform can't see the relationship and may create them in either order or in parallel. That's what `depends_on` is for: an explicit edge with no data flow.

⚠ Common trap: missing an implicit dependency doesn't error at plan time — it applies "successfully" and fails at runtime (e.g. a Lambda deployed before its execution role's policy attachment propagates). Interviewers use this to test whether you understand IAM eventual consistency, not just Terraform syntax.

- [ ] Walk through a production S3 remote-state backend — and how locking actually works.

IF ASKED, SAY THIS

Standard pattern: an S3 bucket with versioning + SSE-KMS enabled for the state file itself, and a lock mechanism so two engineers (or two CI runs) can't apply concurrently and corrupt state.

Standard pattern: an S3 bucket with versioning + SSE-KMS enabled for the state file itself, and a lock mechanism so two engineers (or two CI runs) can't `apply` concurrently and corrupt state.

```
terraform {
  backend "s3" {
    bucket         = "acme-terraform-state"
    key            = "platform/prod/network.tfstate"
    region         = "ap-south-1"
    encrypt        = true
```

```
    dynamodb_table = "terraform-locks"   # pre-1.11 pattern
    # Terraform 1.11+ can lock natively via S3 conditional writes
    # instead of a separate DynamoDB table.
  }
}
```

Historically locking used a DynamoDB table (one item per state key, written on lock-acquire and deleted on release). Newer Terraform versions added native S3-based locking using conditional PUTs, removing the DynamoDB dependency — worth mentioning you're aware of both, since a lot of production estates still run the DynamoDB pattern.

- [ ] Workspaces vs. directory-per-environment for dev/stage/prod — which do you default to, and why?

IF ASKED, SAY THIS

Native terraform workspace gives you one state file per workspace under the same config — fast to spin up, but every environment is forced through identical code, and it's easy to terraform apply against the wrong workspace since the indicator is just a CLI prompt, not a directory you can see in your editor tab.

Native `terraform workspace` gives you one state file per workspace under the same config — fast to spin up, but every environment is forced through identical code, and it's easy to `terraform apply` against the wrong workspace since the indicator is just a CLI prompt, not a directory you can see in your editor tab.

Directory-per-environment (or a thin wrapper like Terragrunt) duplicates the backend config per env but makes blast radius explicit — you physically `cd` into `envs/prod`, and prod can diverge slightly (bigger instance sizes, extra guardrails) without conditionals littering the module. For anything regulated or with real prod/non-prod asymmetry, directory-per-environment with shared modules is the safer default; workspaces are fine for ephemeral, structurally-identical environments (PR preview envs, feature branches).

- [ ] What makes a Terraform module reusable across teams instead of becoming a "god module"?

IF ASKED, SAY THIS

Pin versions (source = "git::...?ref=v2.3.0" or a private registry with semver) so consumers upgrade deliberately, not on every merge to main. Keep the interface small — expose the 5–8 inputs teams actually vary, give everything else sane defaults, and resist adding a boolean flag for every edge case a single team nee

Pin versions (`source = "git::...?ref=v2.3.0"` or a private registry with semver) so consumers upgrade deliberately, not on every merge to main. Keep the interface small — expose the 5–8 inputs teams actually vary, give everything else sane defaults, and resist adding a boolean flag for every edge case a single team needs; that's usually a sign the module is doing two jobs.

- Compose from smaller modules (network, cluster, IAM) rather than one monolith that provisions "the whole platform" — smaller blast radius per `apply`, easier to test in isolation.
- Output the identifiers other modules will need (VPC ID, subnet IDs, cluster OIDC issuer URL) so downstream modules consume outputs, not hardcoded values.
- Validate inputs with `validation` blocks so a bad value fails at plan time with a clear message, not three resources deep into apply.

- [ ] When is terraform apply -target appropriate — and why do some teams ban it outright?

IF ASKED, SAY THIS

-target is genuinely useful for emergency, surgical fixes — e.g. one security group is misconfigured and you need it fixed now without running the full plan for a 400-resource stack. Used routinely, though, it lets state drift silently: resources depending on the targeted one don't get reconciled, so the "official" ful

`-target` is genuinely useful for emergency, surgical fixes — e.g. one security group is misconfigured and you need it fixed now without running the full plan for a 400-resource stack. Used routinely, though, it lets state drift silently: resources depending on the targeted one don't get reconciled, so the "official" full plan later shows a pile of unexpected changes nobody remembers approving.

⚠ The honest answer interviewers want: "I've used it for incident response, never as a habit — and I always follow it with a full untargeted plan to confirm nothing drifted." Claiming you never use it can sound like you haven't handled a live incident under a broken full plan.

- [ ] How do you keep secrets out of Terraform state and code?

IF ASKED, SAY THIS

Never hardcode a secret in a .tf file or a .tfvars committed to git — that's the baseline. Beyond that, remember state itself is not encrypted at rest by Terraform ; anything Terraform reads or creates, including secret values passed as resource arguments, lands in plaintext inside the state file unless the backend enc

Never hardcode a secret in a `.tf` file or a `.tfvars` committed to git — that's the baseline. Beyond that, remember **state itself is not encrypted at rest by Terraform**; anything Terraform reads or creates, including secret values passed as resource arguments, lands in plaintext inside the state file unless the backend encrypts it (SSE-KMS on the S3 bucket) and access is locked down via bucket policy + IAM.

Pattern: reference secrets via a data source at apply time — `data "aws_secretsmanager_secret_version"` — rather than passing literal values as variables, and mark any variable that must carry a literal secret as `sensitive = true` so it's redacted from plan/apply console output (note: this hides it from the CLI output, it does not encrypt it in state).

How do you catch and reconcile drift — infra that changed outside Terraform?

IF ASKED, SAY THIS
Scheduled, read-only terraform plan (or plan -refresh-only) on a cron in CI, posting the diff to a channel rather than auto-applying — drift is a signal to investigate, not necessarily to immediately overwrite (someone may have made an emergency console change during an incident that Terraform should learn to represen

Scheduled, read-only `terraform plan` (or `plan -refresh-only`) on a cron in CI, posting the diff to a channel rather than auto-applying — drift is a signal to investigate, not necessarily to immediately overwrite (someone may have made an emergency console change during an incident that Terraform should learn to represent, not stomp on).

When the drift is legitimate, `terraform apply -refresh-only` updates state to match reality without changing infrastructure; when it's not, a normal apply reverts it. For resources created outside Terraform entirely, `terraform import` (or the newer declarative `import {}` block in recent versions) brings them under management without recreating them.

How do you enforce policy — "no public S3 buckets," "every resource must be tagged" — before apply, not after?

IF ASKED, SAY THIS
Policy-as-code gates the plan, not the running infrastructure. Three common tool shapes come up:

Policy-as-code gates the plan, not the running infrastructure. Three common tool shapes come up:

- **Sentinel** — HashiCorp's own, only available in Terraform Cloud/Enterprise, evaluates against the plan JSON with policy sets tied to workspaces (advisory / soft-mandatory / hard-mandatory enforcement levels).
- **OPA / Conftest** — open source, run against `terraform show -json plan.out` in any CI, policies written in Rego; works regardless of which Terraform tier you're on.
- **Checkov / tfsec / Trivy** — static analysis on the `.tf` source itself, catching known-bad patterns (open security groups, unencrypted volumes) before a plan is even run — fastest feedback loop, but can't see values only known at apply time.

Good pipelines layer these: static scan on every PR, policy-as-code gate on the plan before any prod apply, and drift detection after.

How does one Terraform run provision resources across multiple AWS accounts?

IF ASKED, SAY THIS
Provider aliasing — define one provider block per account/role, each assuming a cross-account IAM role, then point specific resources at the aliased provider:

Provider aliasing — define one provider block per account/role, each assuming a cross-account IAM role, then point specific resources at the aliased provider:

```
provider "aws" {
  alias  = "shared_services"
  region = "ap-south-1"
  assume_role { role_arn = "arn:aws:iam::111111111111:role/terraform-exec" }
}

resource "aws_route53_zone" "internal" {
  provider = aws.shared_services
  name     = "internal.acme.com"
}
```

At larger scale this becomes account-per-environment with a central "management" account holding state buckets and assuming into spoke accounts — which is also the natural place to run the org-wide Service Control Policies (SCPs) that back up whatever Terraform-level governance you've layered on top.

CI is stuck: "Error acquiring the state lock" — the run that held it was killed. Walk through your fix.

IF ASKED, SAY THIS
First confirm no other apply is genuinely in-flight — check the CI job history and, if using DynamoDB locking, read the lock item (it has the operation, "who," and a timestamp). If it's a stale lock from a crashed runner, terraform force-unlock clears it — deliberately, one lock ID at a time, never scripted a

First confirm no other apply is genuinely in-flight — check the CI job history and, if using DynamoDB locking, read the lock item (it has the operation, "who," and a timestamp). If it's a stale lock from a crashed runner, `terraform force-unlock <LOCK_ID>` clears it — deliberately, one lock ID at a time, never scripted as a blanket "retry with force-unlock on failure" step, because that would silently break real concurrent-apply protection.

After unlocking, run a plan (not apply) first — the killed run may have partially applied before dying, so state and real infrastructure can already disagree. If `plan` shows unexpected deletes, that's a signal to inspect further (possibly `state rm` / `import` surgery) rather than blindly applying.

Walk through the Terraform workflow — write, plan, apply — what does each step actually do?

IF ASKED, SAY THIS

Write — declare desired state in HCL (resources, data sources, variables). Plan — Terraform reads current real-world state (from the state file, refreshed against the actual provider APIs), diffs it against desired configuration, and produces a list of exactly what it will add/change/destroy, without touching anything

Write — declare desired state in HCL (resources, data sources, variables). **Plan** — Terraform reads current real-world state (from the state file, refreshed against the actual provider APIs), diffs it against desired configuration, and produces a list of exactly what it will add/change/destroy, without touching anything yet. **Apply** — executes that plan against the provider APIs in dependency order (built from the resource graph), and writes the new resulting state back to the state file.

The plan step is what makes Terraform safe to run in a pipeline — it's a dry-run by construction, so a human (or an automated gate) can review exactly what will change before anything real happens.

- [ ] How does Terraform state work, and why is remote state with locking essential for a team?

IF ASKED, SAY THIS
The state file is Terraform's record of what it believes exists — resource IDs, attributes, dependency metadata — and it's what plan diffs your config against. Without it, Terraform would have no way to know a resource it manages already exists.

The state file is Terraform's record of what it believes exists — resource IDs, attributes, dependency metadata — and it's what plan diffs your config against. Without it, Terraform would have no way to know a resource it manages already exists.

Local state breaks the moment more than one person runs Terraform: two people apply from stale local copies and either silently overwrite each other's changes or get conflicting plans against reality. Remote state (S3+DynamoDB for locking, Azure Storage with native blob leasing, or Terraform Cloud) fixes both problems — one shared source of truth, and a lock that makes a second concurrent apply wait or fail loudly instead of corrupting state.

- [ ] What's Terraform drift, and how do you detect and reconcile it?

IF ASKED, SAY THIS
Drift is when real infrastructure no longer matches what's in the state file — usually because someone made a manual change in the console, or an external process (auto-scaling, another tool) modified something Terraform manages.

Drift is when real infrastructure no longer matches what's in the state file — usually because someone made a manual change in the console, or an external process (auto-scaling, another tool) modified something Terraform manages.

Detect it with a scheduled `terraform plan` (CI job, nightly) against the real environment — any non-empty diff on a config nobody intentionally changed is drift, and I'd alert on that rather than let it silently accumulate. Reconcile it either by applying to force reality back to the declared config, or, if the manual change was actually correct and should stick, by updating the HCL to match reality and re-running plan to confirm zero diff — the fix depends on which side was “right,” and that's a judgment call, not something to automate blindly.

- [ ] Explain Terraform modules — how do you structure a reusable module vs. a root/environment configuration?

IF ASKED, SAY THIS
A module is a reusable unit — typically main.tf , variables.tf , outputs.tf — that encapsulates a piece of infrastructure (a VPC, an EKS cluster, an RDS instance) behind a clean input/output interface, hiding the implementation detail of exactly which resources it creates.

A module is a reusable unit — typically `main.tf` , `variables.tf` , `outputs.tf` — that encapsulates a piece of infrastructure (a VPC, an EKS cluster, an RDS instance) behind a clean input/output interface, hiding the implementation detail of exactly which resources it creates.

The root/environment config doesn't declare raw resources directly for anything the module already covers — it just calls the module with environment-specific variables (`source = "../modules/eks", cluster_name = "prod-eks", node_count = 6`). That separation is what lets dev/staging/prod share the exact same tested module code while differing only in the variable values — the single biggest lever against “it worked in staging but not prod” config drift.

- [ ] How do you manage multiple environments in Terraform — workspaces vs. separate state files/directories?

IF ASKED, SAY THIS
Workspaces are lightweight — same config, multiple named state files — fine when environments are genuinely near-identical and you want to move fast. But they share the same backend config and the same code path, which makes it easy to terraform apply against the wrong workspace by mistake, and they don't let environme

Workspaces are lightweight — same config, multiple named state files — fine when environments are genuinely near-identical and you want to move fast. But they share the same backend config and the same code path, which makes it easy to `terraform apply` against the wrong workspace by mistake, and they don't let environments diverge structurally.

My default at anything beyond a small setup is separate directories per environment (`envs/dev` , `envs/staging` , `envs/prod`), each with its own backend/state and its own `.tfvars` , all calling the same shared modules. It's more files, but the blast-radius isolation is real — you cannot `apply` into prod by forgetting to switch a workspace, because prod is a physically separate directory with its own explicit state.

- [ ] How do you handle secrets in Terraform, like a database password, without leaking them into state or version control?

IF ASKED, SAY THIS
Never a literal value in .tf / .tfvars committed to git. I pull secrets at apply time from Vault, AWS Secrets Manager, or Azure Key Vault via a data source (aws_secretsmanager_secret_version , the Vault provider, etc.), so the secret's value never lives in the repo — only a reference to where it lives.

Never a literal value in `.tf` / `.tfvars` committed to git. I pull secrets at apply time from Vault, AWS Secrets Manager, or Azure Key Vault via a data source (`aws_secretsmanager_secret_version`, the Vault provider, etc.), so the secret's *value* never lives in the repo — only a reference to where it lives.

The catch: Terraform state still stores the resolved value in plaintext by default, since state records real attribute values — so remote state encryption at rest (S3 with SSE, Azure Storage encryption) and tight access control on who can read the state file are non-negotiable, not optional hardening. For anything especially sensitive, generating the secret inside the target service itself (e.g. a managed DB's own random-password generation) and only referencing it, rather than ever setting it from a Terraform variable, avoids the problem entirely.

- [ ] Walk through importing an existing, manually-created cloud resource into Terraform management.

IF ASKED, SAY THIS
Write the resource block in HCL matching what you believe the real resource looks like — type, name, at least the required arguments.

- Write the resource block in HCL matching what you believe the real resource looks like — type, name, at least the required arguments.
- Run `terraform import aws_instance.web i-0123456789abcdef0` (or the newer declarative `import` block, which is plan-reviewable before you commit to it).
- Run `terraform plan` immediately — it will very likely show a diff, because your HCL guess rarely matches every real attribute exactly.
- Iterate on the HCL until `plan` shows zero changes — that's confirmation your config now accurately describes the real resource, not a guess.
- Only then is it safe to let this resource be managed going forward — importing without reaching a clean plan risks Terraform “correcting” the resource back toward your wrong guess on the next apply.

- [ ] How do you integrate Terraform into a CI/CD pipeline safely — the JD's ‘associated pipelines’ — and what gates go before apply?

IF ASKED, SAY THIS
PR opened → pipeline runs terraform fmt -check , validate , and plan , posting the plan output as a PR comment so reviewers see the actual infrastructure diff, not just the code diff.

- **PR opened** → pipeline runs `terraform fmt -check`, `validate`, and `plan`, posting the plan output as a PR comment so reviewers see the actual infrastructure diff, not just the code diff.
- **Policy-as-code gate** (OPA/Sentinel/Checkov) — blocks the plan automatically if it violates guardrails (public S3 bucket, overly-broad security group, missing required tags) before a human even reviews it.
- **Required human approval** for prod applies specifically — lower environments can auto-apply on merge, prod does not.
- **Merge to main** → apply runs using the exact plan that was reviewed (not a fresh plan — avoids a TOCTOU gap between what was approved and what actually runs), with the pipeline's own OIDC-federated role, never a static credential.
- State locking is on by default so a concurrent run can't race the pipeline's own apply.

- [ ] What's a Terraform state lock, and what do you do if a pipeline run dies and leaves the state locked?

IF ASKED, SAY THIS
The lock (a DynamoDB item for the S3 backend, a blob lease for Azure Storage) prevents two apply operations from running against the same state concurrently, which would otherwise corrupt it. Terraform acquires the lock before it starts and releases it when it finishes.

The lock (a DynamoDB item for the S3 backend, a blob lease for Azure Storage) prevents two `apply` operations from running against the same state concurrently, which would otherwise corrupt it. Terraform acquires the lock before it starts and releases it when it finishes.

If a pipeline run is killed mid-apply (job cancelled, runner crashed), the lock can be left held with nothing left to release it. Before doing anything else, I confirm no apply is actually still running anywhere — check the CI system, check for any lingering process — then run `terraform force-unlock <lock-id>`. I treat this as a deliberate, logged action, never a reflexive first response, because force-unlocking while something genuinely is still applying is exactly how you get real state corruption.

- [ ] Explain terraform plan exit codes and how you'd use them to gate an automated pipeline.

IF ASKED, SAY THIS
With -detailed-exitcode : 0 means succeeded, no changes (infrastructure already matches config); 1 means an error occurred; 2 means succeeded, and there are changes to apply.

With `-detailed-exitcode`: **0** means succeeded, no changes (infrastructure already matches config); **1** means an error occurred; **2** means succeeded, and there *are* changes to apply.

In a pipeline I branch on that: exit 0 → nothing to do, skip the apply step entirely and say so explicitly in the pipeline output; exit 2 → changes exist, proceed to the review/approval gate; exit 1 → hard fail the pipeline immediately, page/notify, and do not attempt an apply against a plan that didn't even complete successfully.

- [ ] How would you design a Terraform module for a managed Kubernetes cluster that's conceptually reusable across AWS (EKS) and Azure (AKS) — what would actually differ?

IF ASKED, SAY THIS
I wouldn't try to force one literal module across both providers — the underlying resources are too different (the AWS provider's aws_eks_cluster vs. the AzureRM provider's azurerm_kubernetes_cluster have very different required arguments and IAM/identity models). Instead I'd design a consistent

interface — the same in

I wouldn't try to force one literal module across both providers — the underlying resources are too different (the AWS provider's `aws_eks_cluster` vs. the AzureRM provider's `azurerm_kubernetes_cluster` have very different required arguments and IAM/identity models). Instead I'd design a consistent *interface* — the same input variables (cluster name, node count, instance/VM size, network CIDR, Kubernetes version) and the same output shape (cluster endpoint, kubeconfig data, OIDC issuer URL) — backed by two separate provider-specific module implementations.

What genuinely differs underneath: identity (IRSA/OIDC federation on EKS vs. Azure AD Workload Identity/Managed Identity on AKS), networking (VPC/subnets/security groups vs. VNet/subnets/NSGs), and node provisioning (managed node groups vs. AKS node pools) — different enough that a single shared resource block would fight both providers' idioms rather than genuinely simplify anything.

- [ ] A terraform apply partially failed midway — some resources created, some didn't. What's your recovery process?

IF ASKED, SAY THIS

First, don't panic-retry blindly. Terraform state already reflects the resources that did get created — that's the whole point of state being updated incrementally during apply, not just at the end — so the state file isn't corrupted by a partial failure, just incomplete relative to the full desired config.

First, don't panic-retry blindly. Terraform state already reflects the resources that *did* get created — that's the whole point of state being updated incrementally during apply, not just at the end — so the state file isn't corrupted by a partial failure, just incomplete relative to the full desired config.

Read the actual error — usually a specific resource hit a provider-side failure (quota limit, a dependency not ready yet, a naming conflict). Fix the root cause, then simply re-run `terraform plan`: it will correctly show only the remaining resources that still need creating, since it diffs against the accurate current state, not attempt to recreate what already succeeded. If something looks inconsistent, `terraform refresh` (or plan, which refreshes implicitly) re-syncs state against real infrastructure before you proceed.

- [ ] What rollback strategy would you build into a pipeline, and when should it be automatic vs. human-gated?

IF ASKED, SAY THIS

Automatic rollback tied to a CloudWatch alarm or an Argo Rollouts analysis step (error rate/latency threshold breach during a canary) — machines are faster and more consistent than a human noticing a dashboard spike at 2am. Reserve human-gated rollback for changes with effects that a metric can't ca

Automatic rollback tied to a CloudWatch alarm or an Argo Rollouts analysis step (error rate/latency threshold breach during a canary) — machines are faster and more consistent than a human noticing a dashboard spike at 2am. Reserve human-gated rollback for changes with effects that a metric can't capture quickly enough — a data migration, a config change with a delayed blast radius, or anything in a regulated environment where a change-control process requires sign-off either way. The general principle: automate the reversible, fast-signal cases; keep humans in the loop for the ambiguous or slow-signal ones.

- [ ] How would you build a pipeline gate that blocks a deploy on a failed security scan?

IF ASKED, SAY THIS

Run the scan (Trivy/Grype for images, Snyk/Checkov for IaC, or a SAST tool for code) as a dedicated pipeline stage that runs before the deploy stage, fail the job on any finding above your severity threshold (typically Critical, sometimes High), and make that stage a required check the deploy stage

Run the scan (Trivy/Grype for images, Snyk/Checkov for IaC, or a SAST tool for code) as a dedicated pipeline stage that runs before the deploy stage, fail the job on any finding above your severity threshold (typically Critical, sometimes High), and make that stage a required check the deploy stage depends on — not just an informational report. The nuance is having an explicit, auditable exception process for false positives or accepted risk (a suppression file with a ticket reference and expiry), otherwise teams route around the gate entirely the first time it blocks something they need to ship.

- [ ] Walk through your Terraform experience — were you mainly creating modules, or consuming/extending built-in and community ones?

IF ASKED, SAY THIS

Answer this precisely rather than defensively — both are legitimate, real Terraform experience, and a panel asking this is usually gauging depth, not trying to catch you out. If your real experience is mostly consuming and configuring existing modules (community modules like terraform-aws-modules/ek

Answer this precisely rather than defensively — both are legitimate, real Terraform experience, and a panel asking this is usually gauging depth, not trying to catch you out. If your real experience is mostly consuming and configuring existing modules (community modules like `terraform-aws-modules/eks/aws`, or an internal org module), say that plainly and go one level deeper: which inputs you were actually tuning, what you had to debug when a module's defaults didn't fit your case, any time you forked or patched a module to add missing functionality — that's genuine depth even without having authored a module from a blank file. If you have authored modules from scratch, walk through one concretely: what it provisioned, why it needed to be a module rather than inline resources, and how you versioned and tested it. The weak answer either way is a vague "yes I've used Terraform a lot" with no specifics.

- [ ] How would you design a Terraform EKS module from scratch — what would you implement, step by step?

IF ASKED, SAY THIS

Break it into the resources an EKS cluster actually needs, roughly in dependency order: the control plane itself (aws_eks_cluster , pointing at VPC/subnet inputs and an IAM role with the cluster policy attached); an OIDC identity provider (aws_iam_openid_connect_provider) tied to the cluster's ow

Break it into the resources an EKS cluster actually needs, roughly in dependency order: the control plane itself (`aws_eks_cluster` , pointing at VPC/subnet inputs and an IAM role with the cluster policy attached); an OIDC identity provider (`aws_iam_openid_connect_provider`) tied to the

cluster's own OIDC issuer — this is what makes IRSA possible for every workload that comes after; node compute, either managed node groups (`aws_eks_node_group`) or Fargate profiles, each with their own IAM role and worker policies; cluster security group rules controlling control-plane-to-node traffic; and the core add-ons — VPC CNI, CoreDNS, kube-proxy, typically the EBS/EFS CSI driver — as `aws_eks_addon` resources. The module should accept VPC/subnet IDs as inputs rather than creating networking itself (that's a separate module's job), and must expose the cluster endpoint, CA data, and OIDC provider ARN as outputs, since every downstream IRSA role a consuming team creates needs that exact OIDC ARN.

- [ ] How would you structure the actual Terraform code/file layout for an EKS module to keep it clean and reusable?

IF ASKED, SAY THIS
Standard module layout: main.tf for the core resources, variables.tf for inputs with sensible defaults and validation blocks where it matters (e.g., validating the Kubernetes version against a supported list), outputs.tf for what downstream consumers need, versions.tf pinning Terraform and AWS provi

Standard module layout: `main.tf` for the core resources, `variables.tf` for inputs with sensible defaults and validation blocks where it matters (e.g., validating the Kubernetes version against a supported list), `outputs.tf` for what downstream consumers need, `versions.tf` pinning Terraform and AWS provider version constraints. Split node groups into their own file if you're supporting multiple node group definitions via a `for_each` over a variable — the pattern that scales best when different teams need different instance types or scaling configs from the same module. Keep add-ons in their own file so core cluster-provisioning logic isn't tangled with what's really cluster configuration. A README with a real usage example belongs in the module too — reviewers and future consumers use it as the actual reference more than they read variable descriptions.

- [ ] Once the EKS module is defined, how would you actually use it to create a cluster?

IF ASKED, SAY THIS
The point worth making out loud: the EKS module shouldn't know or care how the VPC was created — it just takes subnet IDs as input — which is exactly what makes it reusable across environments where networking might be provisioned completely differently (a shared VPC in one account, a per-environmen

```
module "eks" {
  source          = "./modules/eks"
  cluster_name    = "prod-platform"
  cluster_version = "1.31"
  vpc_id          = module.networking.vpc_id
  subnet_ids      = module.networking.private_subnet_ids
  node_groups = {
    general = { instance_types = ["m6i.large"], min_size = 2, max_size = 6 }
  }
}
```

The point worth making out loud: the EKS module shouldn't know or care how the VPC was created — it just takes subnet IDs as input — which is exactly what makes it reusable across environments where networking might be provisioned completely differently (a shared VPC in one account, a per-environment VPC in another).

- [ ] What inputs/variables would you expose to make an EKS module genuinely reusable across environments, not parameterized in name only?

IF ASKED, SAY THIS
The test for "genuinely reusable" is whether dev/staging/prod can all consume the exact same module with just different variable values, no forking. At minimum: cluster name and Kubernetes version, VPC/subnet IDs (never created inside the module), a map/list for node groups so the number and shape o

The test for "genuinely reusable" is whether dev/staging/prod can all consume the exact same module with just different variable values, no forking. At minimum: cluster name and Kubernetes version, VPC/subnet IDs (never created inside the module), a map/list for node groups so the number and shape of node groups isn't hardcoded, tags as a map for consistent-but-environment-specific cost/ownership tagging, and toggles for optional add-ons. The anti-pattern to avoid: a module with forty rarely-changed inputs because everything was made "configurable" defensively — that's actually harder to consume safely than a smaller set of genuinely-necessary inputs with good defaults for the rest.

- [ ] What outputs does an EKS module need to expose, and why do downstream consumers actually need them?

IF ASKED, SAY THIS
Cluster endpoint and certificate authority data — needed by anything configuring a Kubernetes/Helm provider against the cluster, including a separate Terraform stack managing in-cluster resources. The OIDC provider ARN and issuer URL — the one people forget, and then can't create IRSA roles for work

Cluster endpoint and certificate authority data — needed by anything configuring a Kubernetes/Helm provider against the cluster, including a separate Terraform stack managing in-cluster resources. The OIDC provider ARN and issuer URL — the one people forget, and then can't create IRSA roles for workloads without circling back to add it, since every "assume role from this specific service account" trust policy needs that exact ARN. Cluster security group ID — needed if other Terraform-managed resources (an RDS instance, a Lambda in the same VPC) need a rule allowing traffic from the cluster. General principle: output anything a different Terraform stack would otherwise have to re-derive with a data source or, worse, hardcode.

- [ ] How would you test or validate a Terraform module before it's trusted to provision a real cluster?

> **IF ASKED, SAY THIS**
> terraform validate plus a linter (tflint with the AWS ruleset) catch syntax and obvious misconfigurations for free in CI on every PR. Beyond that, a real terraform plan / apply cycle against a disposable test environment is the validation that actually matters — apply it, confirm the cluster comes u

`terraform validate` plus a linter (tflint with the AWS ruleset) catch syntax and obvious misconfigurations for free in CI on every PR. Beyond that, a real `terraform plan` / `apply` cycle against a disposable test environment is the validation that actually matters — apply it, confirm the cluster comes up and nodes join, tear it down, ideally automated so it runs on every module change rather than being a manual step someone forgets. Terratest or the native Terraform test framework (`.tftest.hcl`) let you assert specific outcomes programmatically rather than just "did apply succeed." For a module other teams will consume, a working example directory that's itself part of the CI test matrix is worth more in practice than most unit-test-style assertions, since it's testing the thing people will actually copy from.

- [ ] terraform-aws-modules/eks/aws vs. building your own EKS module from scratch — how do you decide?

> **IF ASKED, SAY THIS**
> The community module is battle-tested across a huge number of real deployments, handles AWS API edge cases you'd otherwise discover the hard way, and gets updated quickly when EKS itself changes — genuinely hard to justify not starting there for most teams. The case for building your own: you need b

The community module is battle-tested across a huge number of real deployments, handles AWS API edge cases you'd otherwise discover the hard way, and gets updated quickly when EKS itself changes — genuinely hard to justify not starting there for most teams. The case for building your own: you need behavior the community module doesn't support well, you want a much smaller surface area than a general-purpose module carries (easier to audit for a regulated environment), or you want the module's defaults to encode your org's specific security baseline rather than generic ones. A common, worth-naming middle ground: wrap the community module inside a thin internal module that pins specific defaults and exposes a smaller, opinionated interface — the battle-tested internals without every team having to independently get all forty of its inputs right.

- [ ] Walk through resolving a merge conflict in a Terraform module that two people modified at once — what's your process to avoid breaking prod?

> **IF ASKED, SAY THIS**
> Resolve the HCL conflict like any code conflict — understand both people's intent, not just mechanically pick one side, since Terraform conflicts often mean both changes are individually valid but together create an inconsistent resource definition.

Resolve the HCL conflict like any code conflict — understand both people's intent, not just mechanically pick one side, since Terraform conflicts often mean both changes are individually valid but together create an inconsistent resource definition.

The part that's different from application code: after resolving, I don't trust the merge until `terraform plan` runs clean against it — a syntactically valid merge can still produce a plan neither author intended (e.g. one person's change to a variable default combined with another's new resource referencing it in an unexpected way). I'd treat a clean, reviewed plan output as the actual merge gate, not just “git says it merged.” State itself is never manually edited to resolve this — the HCL is the source of truth; state just needs to reconcile against it on the next apply.

- [ ] Explain Terraform's core execution model: plan, graph, apply.

> **IF ASKED, SAY THIS**
> Terraform reads your HCL config, builds a resource dependency graph (from explicit references and implicit ones like interpolating one resource's attribute into another), compares desired state (your config) against the last-known state file , and computes a diff — that's terraform plan . terraform apply walks that graph, executing provid

Terraform reads your HCL config, builds a **resource dependency graph** (from explicit references and implicit ones like interpolating one resource's attribute into another), compares desired state (your config) against the last-known **state file**, and computes a diff — that's `terraform plan`. `terraform apply` walks that graph, executing provider API calls in dependency order (parallelizing independent branches), and updates the state file to reflect what was actually created/changed/destroyed. Nothing is truly "declarative magic" — every resource maps to specific provider API calls, and the state file is what lets Terraform know what it's already responsible for versus what's new.

- [ ] Resource vs data source — what's the actual distinction?

> **IF ASKED, SAY THIS**
> A resource block is something Terraform creates, updates, and destroys — it owns the full lifecycle and it's tracked in state as something Terraform is responsible for. A data source is read-only — it queries the provider for information about something that already exists (an AMI ID, an existing VPC, account details) so you can reference

```
resource "aws_instance" "web" {          # Terraform CREATES and manages this
  ami           = data.aws_ami.latest.id
  instance_type = "t3.medium"
}

data "aws_ami" "latest" {                # Terraform only READS this, doesn't manage it
  most_recent = true
  owners      = ["amazon"]
```

```
  filter { name = "name"; values = ["amzn2-ami-hvm-*"] }
}
```

A `resource` block is something Terraform creates, updates, and destroys — it owns the full lifecycle and it's tracked in state as something Terraform is responsible for. A `data` source is read-only — it queries the provider for information about something that already exists (an AMI ID, an existing VPC, account details) so you can reference it, but Terraform never creates, modifies, or destroys it, and destroying your config never touches the underlying object a data source pointed at.

- [ ] How does Terraform decide the order to create/destroy resources?

IF ASKED, SAY THIS
Purely from the dependency graph, not the order written in the file. An explicit reference (subnet_id = aws_subnet.main.id) creates an implicit dependency edge. Resources with no dependency relationship between them are created in parallel (bounded by -parallelism , default 10). For destroys, the graph is walked in reverse — dependents

Purely from the dependency graph, not the order written in the file. An explicit reference (`subnet_id = aws_subnet.main.id`) creates an implicit dependency edge. Resources with no dependency relationship between them are created in parallel (bounded by `-parallelism` , default 10). For destroys, the graph is walked in reverse — dependents are destroyed before their dependencies. `depends_on` exists for the rare case where a dependency isn't expressible through an attribute reference (e.g. an IAM policy that must exist before a resource works, but nothing in the resource's arguments actually references the policy).

- [ ] Why does Terraform need a state file at all — why can't it just query the cloud provider directly every time?

IF ASKED, SAY THIS
Two reasons providers can't fully answer alone: (1) mapping — the state file is what ties your HCL resource address (aws_instance.web) to the actual provider-assigned ID (i-0abc123); without it, Terraform has no way to know which real-world object a config block corresponds to. (2) metadata the provider doesn't expose — dependency gra

Two reasons providers can't fully answer alone: (1) **mapping** — the state file is what ties your HCL resource address (`aws_instance.web`) to the actual provider-assigned ID (`i-0abc123`); without it, Terraform has no way to know which real-world object a config block corresponds to. (2) **metadata the provider doesn't expose** — dependency graph info, and values only known at apply time that aren't always re-queryable identically. State is also how Terraform detects resources it's no longer managing (removed from config) so it knows to destroy them, and it's the mechanism for storing sensitive computed outputs and resource attributes for fast plan calculations without re-querying every single resource on every run (though `refresh` still does query providers to detect drift).

- [ ] Walk through the classic S3 + DynamoDB remote backend and exactly what each piece does.

IF ASKED, SAY THIS
S3 stores the actual state file (versioned, so enable S3 bucket versioning as a safety net for accidental corruption/overwrite — you can restore a prior version). DynamoDB provides state locking : before any plan / apply , Terraform writes a lock item to the table; if another run tries to acquire the same lock, it fails fast instead of tw

```
terraform {
  backend "s3" {
    bucket         = "my-tfstate-bucket"
    key            = "prod/network/terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "terraform-locks"
    encrypt        = true
  }
}
```

S3 stores the actual state file (versioned, so enable S3 bucket versioning as a safety net for accidental corruption/overwrite — you can restore a prior version). **DynamoDB** provides **state locking**: before any `plan` / `apply` , Terraform writes a lock item to the table; if another run tries to acquire the same lock, it fails fast instead of two processes racing to write state simultaneously and corrupting it. `encrypt = true` enables SSE on the S3 object — important since state files routinely contain sensitive values in plaintext.

Note (recent HCP Terraform / OpenTofu direction)Terraform 1.10+ added native S3 state locking without a separate DynamoDB table for some workflows, but the S3+DynamoDB pattern is still the one you'll see in the overwhelming majority of existing production setups and interview questions.

- [ ] Name the terraform state subcommands you'd actually use, and what each is for.

IF ASKED, SAY THIS
state mv is the one that saves you from an unwanted destroy/recreate cycle when you refactor code (rename a resource, move it into a module) — since Terraform otherwise sees a renamed resource as "old one removed, new one added" and would plan to destroy and recreate it. state rm is the escape hatch when you want Terraform to forget about

```
terraform state list                        # inventory of everything in state
terraform state show aws_instance.web        # full attributes of one resource
terraform state mv aws_instance.old aws_instance.new   # rename/refactor without destroy+recreate
```

```
terraform state rm aws_instance.web           # stop managing it, WITHOUT destroying the real resource
terraform state pull > backup.tfstate         # raw state as JSON, e.g. for manual inspection/backup
```

`state mv` is the one that saves you from an unwanted destroy/recreate cycle when you refactor code (rename a resource, move it into a module) — since Terraform otherwise sees a renamed resource as "old one removed, new one added" and would plan to destroy and recreate it. `state rm` is the escape hatch when you want Terraform to forget about something without touching the real infrastructure — common when splitting a monolithic state into smaller ones.

- [ ] What makes a good module boundary? What goes in, what stays in the root?

IF ASKED, SAY THIS
A module should encapsulate a coherent, reusable unit of infrastructure with a clear interface (inputs via variables.tf , outputs via outputs.tf) — e.g. "a VPC with subnets and routing," "an EKS cluster with node groups," "an RDS instance with backups configured." Avoid two anti-patterns: modules so granular they just wrap a single resou

A module should encapsulate a coherent, reusable unit of infrastructure with a clear interface (inputs via `variables.tf`, outputs via `outputs.tf`) — e.g. "a VPC with subnets and routing," "an EKS cluster with node groups," "an RDS instance with backups configured." Avoid two anti-patterns: modules so granular they just wrap a single resource with no real abstraction (adds indirection without value), and modules so broad they encode environment-specific business logic (making them non-reusable in practice even though they're technically "a module"). Root/environment-level config should own environment-specific wiring — which modules to call, with what inputs, and cross-module references — not raw resource definitions duplicated per environment.

- [ ] How do you version and pin a module source, and why does it matter?

IF ASKED, SAY THIS
Without a version pin, a shared module changing upstream (even a "minor" fix) can silently alter every consumer's plan on their next run — dangerous at scale. Registry modules take a version constraint (exact pin is the safe default for production; a range like ~> 5.8 is more common for actively-maintained internal modules where you want

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "5.8.1"     # exact pin — recommended for registry modules
  # or for a private Git module:
  # source = "git::https://github.com/org/tf-modules.git//vpc?ref=v2.3.0"
}
```

Without a version pin, a shared module changing upstream (even a "minor" fix) can silently alter every consumer's plan on their next run — dangerous at scale. Registry modules take a `version` constraint (exact pin is the safe default for production; a range like `~> 5.8` is more common for actively-maintained internal modules where you want patch updates). Git-sourced modules pin via `?ref=<tag/commit>` in the source URL — always pin to a tag or commit SHA, never a branch name, or you get the exact same "silent upstream change" risk a branch ref doesn't protect against.

- [ ] How does state work across modules — is there a separate state file per module?

IF ASKED, SAY THIS
No — by default, a module call doesn't get its own state file; all resources, including everything inside every nested module, live in one state file per root configuration , just namespaced by module path (module.vpc.aws_subnet.private["a"]). Splitting into genuinely separate state files means separate root configurations entirely (sep

No — by default, a module call doesn't get its own state file; all resources, including everything inside every nested module, live in **one state file per root configuration**, just namespaced by module path (`module.vpc.aws_subnet.private["a"]`). Splitting into genuinely separate state files means separate root configurations entirely (separate directories each with their own backend config), commonly wired together via remote state data sources (`terraform_remote_state`) or, more robustly, by publishing key outputs to something like SSM Parameter Store / a data source lookup rather than a tight remote-state coupling between independently-deployed stacks.

- [ ] What do Terraform workspaces actually do, and what's the biggest misconception about them?

IF ASKED, SAY THIS
Workspaces give you multiple independent state files under one backend/configuration, selected via terraform.workspace in your HCL. The biggest misconception: people treat workspaces as a full environment-isolation mechanism (dev/staging/prod), but they only isolate state — the same backend config, same variable defaults unless you explic

```
terraform workspace new staging
terraform workspace select staging
terraform apply   # uses a state file scoped to "staging" automatically
```

Workspaces give you multiple independent state files under one backend/configuration, selected via `terraform.workspace` in your HCL. The biggest misconception: people treat workspaces as a full environment-isolation mechanism (dev/staging/prod), but they only isolate *state* — the same backend config, same variable defaults unless you explicitly branch on `terraform.workspace`, and critically the **same IAM credentials/provider config** unless you engineer around that. For environments with genuinely different accounts, regions, or blast-radius requirements (which prod almost

always should have), most teams prefer entirely separate root configurations/directories per environment over workspaces, reserving workspaces for things like ephemeral per-PR preview environments within a single account.

- [ ] Directory-per-environment vs workspace-per-environment — what would you actually recommend for prod/staging/dev, and why?

IF ASKED, SAY THIS
Directory-per-environment (separate root modules, each with their own backend key and often their own AWS account/credentials) for anything where you want a hard blast-radius boundary — a terraform destroy run against the wrong workspace by mistake is a much easier accident than running it in the wrong directory with visibly different bac

Directory-per-environment (separate root modules, each with their own backend key and often their own AWS account/credentials) for anything where you want a hard blast-radius boundary — a `terraform destroy` run against the wrong workspace by mistake is a much easier accident than running it in the wrong directory with visibly different backend config and credentials in front of you. Workspaces are better suited to same-account, same-blast-radius, high-cardinality use cases like per-feature-branch ephemeral environments, where directory-per-instance would be unmanageable sprawl. Most regulated/production-grade setups (which is the context an AVP-level interview usually cares about) default to directory-per-environment for prod specifically, even if workspaces are used elsewhere.

- [ ] Walk through terraform import — what does it do and not do?

IF ASKED, SAY THIS
It brings an already-existing, manually-created (or otherwise unmanaged) real-world resource under Terraform's management by writing a corresponding entry into the state file — but it does not generate the HCL configuration block for you (older Terraform versions require you to hand-write the matching resource block yourself before/after

```
terraform import aws_s3_bucket.logs my-existing-bucket-name
```

It brings an already-existing, manually-created (or otherwise unmanaged) real-world resource under Terraform's management by writing a corresponding entry into the state file — but it does **not** generate the HCL configuration block for you (older Terraform versions require you to hand-write the matching `resource` block yourself before/after import, or it errors that the address doesn't exist in config). Newer Terraform (1.5+) added the `import` block plus `terraform plan -generate-config-out`, which can scaffold starting HCL automatically — a meaningful improvement for large-scale "adopt existing infra into Terraform" projects, worth mentioning if asked about legacy-to-IaC migrations.

- [ ] terraform taint is deprecated — what replaced it and why is the replacement better?

IF ASKED, SAY THIS
taint mutated the state file immediately, outside of any plan review — a destructive action with no preview step. -replace is a plan-time flag: it shows up as a planned destroy+recreate in the normal plan / apply output, so you review exactly what will happen before committing, consistent with how every other Terraform change is reviewed

```
# old (deprecated):
terraform taint aws_instance.web
# current:
terraform apply -replace="aws_instance.web"
```

`taint` mutated the state file immediately, outside of any plan review — a destructive action with no preview step. `-replace` is a plan-time flag: it shows up as a planned destroy+recreate in the normal `plan` / `apply` output, so you review exactly what will happen before committing, consistent with how every other Terraform change is reviewed rather than being a special imperative side-channel.

- [ ] What does terraform plan -refresh-only do, and when would you reach for it specifically?

IF ASKED, SAY THIS
It re-queries providers for the real current state of every managed resource and shows you a diff between that live reality and what's recorded in the state file — without proposing any changes to bring infrastructure in line with your HCL config (a normal plan does both: detect drift AND propose fixing it). This is the tool for "I suspec

It re-queries providers for the real current state of every managed resource and shows you a diff between that live reality and what's recorded in the state file — **without** proposing any changes to bring infrastructure in line with your HCL config (a normal plan does both: detect drift AND propose fixing it). This is the tool for "I suspect someone made a manual change out-of-band — show me exactly what drifted" as a pure investigation step, followed by `terraform apply -refresh-only` to accept the drift into state as the new baseline if the manual change should be kept, or a normal `apply` to revert it back to what config says if it shouldn't.

- [ ] What's lifecycle { create_before_destroy = true } for, and give a real scenario where you'd need it.

IF ASKED, SAY THIS
By default Terraform destroys the old resource before creating its replacement. For anything that other resources reference by ID/name where a gap would cause an outage or an error (an Auto Scaling Group's launch template, a security group still attached to running instances, a Route53 record another resource depends on), that ordering ca

```
resource "aws_launch_template" "app" {
  name_prefix = "app-lt-"
  lifecycle { create_before_destroy = true }
}
```

By default Terraform destroys the old resource before creating its replacement. For anything that other resources reference by ID/name where a gap would cause an outage or an error (an Auto Scaling Group's launch template, a security group still attached to running instances, a Route53 record another resource depends on), that ordering causes a hard failure or downtime window. `create_before_destroy` flips the order: create the new one first, repoint dependents, then destroy the old one — commonly paired with `name_prefix` instead of a fixed `name`, since you can't have two resources with the identical name existing simultaneously during that overlap window.

- [ ] What is configuration drift, what causes it in practice, and how do you catch it before it causes an incident?

IF ASKED, SAY THIS
Drift is when real infrastructure no longer matches what's in the state file/config — caused by manual console/CLI changes, another automation tool touching the same resources, or a cloud provider auto-adjusting something (e.g. AWS auto-assigning a default value Terraform didn't set explicitly). Left uncaught, the next apply either silent

Drift is when real infrastructure no longer matches what's in the state file/config — caused by manual console/CLI changes, another automation tool touching the same resources, or a cloud provider auto-adjusting something (e.g. AWS auto-assigning a default value Terraform didn't set explicitly). Left uncaught, the next `apply` either silently reverts a manual emergency fix someone made during an incident, or a plan shows unexpected changes that spook whoever's running it. Standard mitigation: a scheduled CI job running `terraform plan -detailed-exitcode` (exit code 2 means changes/drift detected, distinct from 0=no changes and 1=error) against every stack on a cadence, alerting the team rather than waiting to discover drift at the next real deploy.

- [ ] What's the difference between Sentinel and OPA/Conftest for policy-as-code with Terraform?

IF ASKED, SAY THIS
Sentinel is HashiCorp's own policy language, natively integrated with Terraform Cloud/Enterprise — it runs as part of the managed run pipeline with direct access to the plan, and can enforce/soft-mandatory/advisory levels of policy directly in that workflow. OPA (Open Policy Agent) with Conftest is the open-source, tool-agnostic route: yo

Sentinel is HashiCorp's own policy language, natively integrated with Terraform Cloud/Enterprise — it runs as part of the managed run pipeline with direct access to the plan, and can enforce/soft-mandatory/advisory levels of policy directly in that workflow. **OPA** (Open Policy Agent) with **Conftest** is the open-source, tool-agnostic route: you export the plan as JSON (`terraform show -json plan.out`) and run Conftest/OPA's Rego policies against that JSON, typically as a standalone CI step independent of which Terraform runner you use — the tradeoff being it's not natively wired into HCP Terraform's run pipeline the way Sentinel is, but it works with any CI system and any policy engine you might already standardize on elsewhere.

- [ ] Give a concrete policy-as-code rule you'd actually enforce for a production AWS environment.

IF ASKED, SAY THIS
Practical, high-value examples: deny any S3 bucket resource without block_public_access fully enabled; deny security group rules with 0.0.0.0/0 ingress on ports other than 80/443; require every taggable resource to carry a mandatory tag set (cost-center, owner, environment) for chargeback/ownership tracing; deny RDS instances without stor

Practical, high-value examples: deny any S3 bucket resource without `block_public_access` fully enabled; deny security group rules with `0.0.0.0/0` ingress on ports other than 80/443; require every taggable resource to carry a mandatory tag set (cost-center, owner, environment) for chargeback/ownership tracing; deny RDS instances without `storage_encrypted = true` ; cap instance types/sizes in non-prod accounts to control cost. These are exactly the kind of "what would you actually enforce" questions used to check whether policy-as-code experience is real or just terminology-deep.

- [ ] The state file appears corrupted or a resource's state is out of sync with reality in a way refresh can't fix. What's your recovery path?

IF ASKED, SAY THIS
(1) Restore from S3 versioning first if the bucket has it enabled — the fastest, safest recovery is a previous good version of the object, not manual surgery. (2) If a specific resource's state entry is the problem (not the whole file), terraform state rm the bad entry then terraform import it back cleanly. (3) As a last resort, terraform

(1) Restore from S3 versioning first if the bucket has it enabled — the fastest, safest recovery is a previous good version of the object, not manual surgery. (2) If a specific resource's state entry is the problem (not the whole file), `terraform state rm` the bad entry then `terraform import` it back cleanly. (3) As a last resort, `terraform state pull` to get the raw JSON, hand-edit carefully, then `terraform state push` — high-risk, only after backing up the current file, and generally something you'd narrate in an interview as "the option of last resort," since it's easy to make things worse.

- [ ] A provider version bump in CI suddenly changes plan output for resources you didn't touch. How do you investigate and prevent recurrence?

IF ASKED, SAY THIS
Check the provider's changelog for the version range between what was pinned and what CI just picked up — provider authors do sometimes change default values, attribute computation, or deprecate/rename arguments between versions in ways that produce a diff even though your HCL didn't change. Prevention: pin provider versions with a requir

Check the provider's changelog for the version range between what was pinned and what CI just picked up — provider authors do sometimes change default values, attribute computation, or deprecate/rename arguments between versions in ways that produce a diff even though your HCL didn't change. Prevention: pin provider versions with a `required_providers` block using a conservative constraint (or exact pin for critical stacks) and commit a `.terraform.lock.hcl` file to version control — that lock file is exactly analogous to Chart.lock in Helm or a package-lock.json, and its absence (or being gitignored by mistake) is the actual root cause behind "why did CI suddenly behave differently with no code change."

- [ ] SCENARIO Scenario: you need to split one large, slow-to-plan monolithic state file into smaller per-service states without downtime. Outline the approach.

  IF ASKED, SAY THIS
  Identify natural boundaries (network/shared vs per-service) and confirm no circular dependencies would be created by splitting. For each resource moving to a new state: terraform state rm from the old state, terraform import into the new state's config — this touches only Terraform's bookkeeping, not the real infrastructure, so there's no

  1. Identify natural boundaries (network/shared vs per-service) and confirm no circular dependencies would be created by splitting.
  2. For each resource moving to a new state: `terraform state rm` from the old state, `terraform import` into the new state's config — this touches only Terraform's bookkeeping, not the real infrastructure, so there's no actual downtime to the resources themselves.
  3. Wire any genuine cross-stack references via `terraform_remote_state` data sources or, more decoupled, via values published to SSM Parameter Store/Secrets Manager that the dependent stack reads instead of a tight remote-state coupling.
  4. Do it stack-by-stack, verifying `terraform plan` shows zero changes immediately after each split (proving the split didn't accidentally alter anything) before moving to the next.

- [ ] SCENARIO Scenario: leadership asks "how do we stop engineers from making manual console changes that Terraform then fights with?" What do you actually propose?

  IF ASKED, SAY THIS
  Technical and process layers together, since policy alone doesn't stick: (1) IAM/SCP guardrails removing broad console write access to Terraform-managed resource types for most engineers, with break-glass access that's audited/alerted on when used; (2) scheduled drift-detection CI (covered above) so any manual change is surfaced within ho

  Technical and process layers together, since policy alone doesn't stick: (1) IAM/SCP guardrails removing broad console write access to Terraform-managed resource types for most engineers, with break-glass access that's audited/alerted on when used; (2) scheduled drift-detection CI (covered above) so any manual change is surfaced within hours, not discovered at the next deploy; (3) fast, reliable Terraform pipelines (so there's no incentive to "just fix it in the console because the PR pipeline takes 40 minutes") — a slow IaC workflow is often the actual root cause of console-change habits, not a discipline problem; (4) a clear break-glass runbook for genuine incidents (manual fix now, immediately reconcile into Terraform after) rather than pretending emergencies never justify a console change.

- [ ] Terraform vs. CloudFormation vs. CDK — how would you decide which to standardize on for this kind of platform, and could you justify supporting more than one?

  IF ASKED, SAY THIS
  Terraform for multi-cloud consistency and its large module ecosystem. CloudFormation for pure-AWS shops wanting native, no-extra-tooling integration (sometimes ahead of Terraform provider support for newer services) and native StackSets for multi-account deployment. CDK when you want infrastructure defined in a real language (Python/TypeS

  Terraform for multi-cloud consistency and its large module ecosystem. **CloudFormation** for pure-AWS shops wanting native, no-extra-tooling integration (sometimes ahead of Terraform provider support for newer services) and native StackSets for multi-account deployment. **CDK** when you want infrastructure defined in a real language (Python/TypeScript) — genuinely useful for complex, parameterized, reusable platform modules where CFN YAML/JSON gets unwieldy, since CDK synthesizes to CloudFormation underneath.

  For a consulting/platform role serving multiple enterprise clients, fluency in more than one is realistic — client environments vary, and this JD lists all three explicitly.

- [ ] How would you design reusable platform modules/reference architectures for IaC that multiple teams or client engagements can consume?

  IF ASKED, SAY THIS
  Modules scoped to a coherent unit of infrastructure ("secure VPC", "standard ECS service") with sensible defaults but genuinely configurable inputs for real variance — not so rigid every consumer forks it, not so configurable it's just raw resources with extra steps. Versioned (semantic versioning on the module) so consumers pin to a know

  Modules scoped to a coherent unit of infrastructure ("secure VPC", "standard ECS service") with sensible defaults but genuinely configurable inputs for real variance — not so rigid every consumer forks it, not so configurable it's just raw resources with extra steps.

  Versioned (semantic versioning on the module) so consumers pin to a known-good version and upgrade deliberately, via a private module registry or tagged Git releases rather than everyone copy-pasting module code around — which is exactly how drift and unpatched issues accumulate silently.

- [ ] What's your approach to managing Terraform state safely in a team/enterprise environment?

  IF ASKED, SAY THIS
  Remote state (S3 backend with DynamoDB for locking is the classic AWS-native pattern, or Terraform Cloud) — never local state for anything beyond solo experimentation, since local state can't be safely shared and is a single point of data loss. State locking is non-negotiable for team use — without

it, concurrent applies corrupt state. Se

Remote state (S3 backend with DynamoDB for locking is the classic AWS-native pattern, or Terraform Cloud) — never local state for anything beyond solo experimentation, since local state can't be safely shared and is a single point of data loss.

State locking is non-negotiable for team use — without it, concurrent applies corrupt state. Separate state files per logical environment/component (not one monolithic state) limits blast radius and speeds up plan/apply for teams working on one piece.

- [ ] How would you structure a CI/CD pipeline specifically for Terraform/IaC changes, including the governance/approval piece?

IF ASKED, SAY THIS
Plan-then-apply as separate stages, plan output posted somewhere reviewable (a PR comment, a pipeline artifact) before a human approves the apply — nobody should be surprised by what an apply actually changes. Policy-as-code (Sentinel for TFC/E, or OPA/Checkov as tool-agnostic options) scanning the plan for violations (unencrypted resourc

Plan-then-apply as separate stages, plan output posted somewhere reviewable (a PR comment, a pipeline artifact) before a human approves the apply — nobody should be surprised by what an apply actually changes.

Policy-as-code (Sentinel for TFC/E, or OPA/Checkov as tool-agnostic options) scanning the plan for violations (unencrypted resources, overly permissive security groups, missing tags) automatically before human review. Separate pipelines/permissions per environment so a dev change doesn't share prod's apply credentials.

- [ ] How do you handle drift between what's declared in IaC and what actually exists in the AWS environment?

IF ASKED, SAY THIS
Regular drift detection (a scheduled terraform plan with no intended changes, checked for unexpected diffs — or CloudFormation's native drift detection) rather than discovering it only when the next real plan looks surprising. When found, the investigation matters as much as the fix — an emergency manual change during an incident should b

Regular drift detection (a scheduled `terraform plan` with no intended changes, checked for unexpected diffs — or CloudFormation's native drift detection) rather than discovering it only when the next real plan looks surprising.

When found, the investigation matters as much as the fix — an emergency manual change during an incident should be reconciled back into IaC promptly; recurring drift reveals the module doesn't actually control everything it should. Worth a hard rule: no manual console changes to IaC-managed resources outside a genuine break-glass emergency.

- [ ] How would you approach IaC for a multi-account/multi-region enterprise deployment — is it one big configuration, or something more modular?

IF ASKED, SAY THIS
Modular by account/environment/region, composed from shared reusable modules, rather than one giant configuration expressing the whole enterprise topology at once — operationally risky (a single apply touching everything) and slow to plan. A higher-level orchestration layer (Terraform workspaces, a CI/CD pipeline matrix, or CDK Pipelines/

Modular by account/environment/region, composed from shared reusable modules, rather than one giant configuration expressing the whole enterprise topology at once — operationally risky (a single apply touching everything) and slow to plan.

A higher-level orchestration layer (Terraform workspaces, a CI/CD pipeline matrix, or CDK Pipelines/StackSets) coordinates deploying the modular pieces to each target account/region in order, with account/region-specific variables layered on shared module defaults.

- [ ] Walk through Terraform state file management in detail — where it's stored, how locking works, what you'd do if it got corrupted or out of sync with reality.

IF ASKED, SAY THIS
State lives in the configured backend (S3 with DynamoDB locking is the standard AWS-native pattern) as a JSON representation of every resource Terraform manages and its last-known attributes. Locking (via the DynamoDB table) prevents two concurrent applies from corrupting it simultaneously — a lock left stuck after a crashed run is a real

State lives in the configured backend (S3 with DynamoDB locking is the standard AWS-native pattern) as a JSON representation of every resource Terraform manages and its last-known attributes. Locking (via the DynamoDB table) prevents two concurrent applies from corrupting it simultaneously — a lock left stuck after a crashed run is a real, if rare, operational annoyance (`terraform force-unlock` after confirming no run is actually in progress).

If state gets out of sync with reality (a manual change, or state lost/corrupted): `terraform plan` surfaces the drift; for lost state specifically, `terraform import` rebuilds state entries for existing resources one at a time — tedious but recoverable, which is exactly why remote state with S3 versioning enabled matters, so you can roll back to a last-known-good state file as a last resort.

- [ ] How would you design a Terraform backend and workspace strategy for a multi-account AWS Organization specifically?

IF ASKED, SAY THIS

Separate state files per account/environment — not Terraform workspaces sharing one backend config for genuinely different environments, since workspaces suit smaller variations within one environment better than isolating production from dev. A dedicated shared-services account often hosts the S3 backend bucket and DynamoDB lock table th

Separate state files per account/environment — not Terraform workspaces sharing one backend config for genuinely different environments, since workspaces suit smaller variations within one environment better than isolating production from dev. A dedicated shared-services account often hosts the S3 backend bucket and DynamoDB lock table that all other accounts' pipelines write to, with cross-account bucket policies granting each account's CI/CD role write access only to its own state path/prefix.

- [ ] How does Terraform assume cross-account IAM roles to provision resources across multiple AWS accounts — walk through the mechanics?

IF ASKED, SAY THIS

A provider block (or multiple aliased provider blocks, one per target account) configured with assume_role , pointing at a role ARN in the target account. The identity actually running Terraform (typically a CI/CD pipeline's role) needs sts:AssumeRole permission on that target role, and the target role's trust policy needs to explicitly a

A provider block (or multiple aliased provider blocks, one per target account) configured with `assume_role`, pointing at a role ARN in the target account. The identity actually running Terraform (typically a CI/CD pipeline's role) needs `sts:AssumeRole` permission on that target role, and the target role's trust policy needs to explicitly allow being assumed by that specific source role/account.

For many-account setups, this is usually generated/templated rather than hand-written per account, so onboarding a new account is a config addition, not a bespoke Terraform change.

- [ ] If you've mostly worked with existing Terraform rather than designing it from scratch, how do you talk about that experience convincingly rather than defensively?

IF ASKED, SAY THIS

Be specific about what "working with existing Terraform" actually involved — reading and understanding module structure well enough to safely extend it, resolving a real state or plan-diff issue, adding a new resource within an established pattern, reviewing colleagues' changes. That's genuine, demonstrable skill even without having autho

Be specific about what "working with existing Terraform" actually involved — reading and understanding module structure well enough to safely extend it, resolving a real state or plan-diff issue, adding a new resource within an established pattern, reviewing colleagues' changes. That's genuine, demonstrable skill even without having authored the original module design.

The honest framing is "I've operated within and extended an existing Terraform codebase" — not overclaiming greenfield design experience, and not underselling real hands-on skill as "I just ran commands" either.

Ansible

- [ ] What is Ansible, and how is its core model different from Terraform's?

IF ASKED, SAY THIS
Ansible is agentless configuration management — it SSHes into a host and runs tasks to bring it to a desired state (install packages, edit configs, restart services). Terraform is declarative infrastructure provisioning — it creates/destroys cloud resources via provider APIs. In practice: Terraform stands up the VM/cluster, Ansible configures what runs inside it.

The agentless part matters operationally — no daemon to install, patch, or have die on managed hosts; Ansible just needs SSH access and Python on the target. That's also why it scales less smoothly to thousands of hosts than agent-based tools, since every run is a fresh SSH connection per host (mitigated with connection pooling and `--forks`).

- [ ] Walk through the core pieces of Ansible: inventory, playbook, task, module, role.

IF ASKED, SAY THIS
Inventory = the list of hosts (static file or dynamic, e.g. pulled live from AWS). Playbook = a YAML file of plays, each targeting a host group and listing tasks. Task = one action, backed by a Module (the actual unit of work — apt, copy, template, systemd, etc.). Role = a reusable, structured bundle of tasks/handlers/templates/vars, the unit you actually share across projects.

Roles are what separate a toy playbook from production Ansible — a role has a fixed directory structure (tasks/, handlers/, templates/, defaults/, vars/) so it can be dropped into any playbook and just work, and shared via Ansible Galaxy or a private role repo.

```
---
- name: Configure web servers
  hosts: webservers
  become: true
  roles:
    - common
    - nginx
  vars:
    nginx_port: 8080
```

- [ ] Why does idempotency matter in Ansible, and how do modules actually implement it?

IF ASKED, SAY THIS
Running a playbook twice should produce the exact same end state and report the second run as "no changes" — this is what makes it safe to re-run against drifted or partially-applied hosts without manually figuring out what already happened. Modules implement it by checking current state before acting (e.g. the apt module checks if a package is already installed before trying to install it) rather than blindly executing shell commands.

This is exactly why Ansible strongly prefers proper modules (apt, copy, template, systemd) over the raw shell/command modules — a shell command has no concept of "already done," so it re-runs and reports "changed" every single time, which defeats idempotency and makes drift detection via Ansible reports useless.

- [ ] What are handlers, and why are they triggered differently from normal tasks?

IF ASKED, SAY THIS
A handler is a task that only runs if notified by another task that reported a change (e.g. "restart nginx" only fires if the nginx config file task actually changed something), and handlers run once at the END of the play even if multiple tasks notify the same handler — so editing 5 nginx config files still triggers exactly one restart, not five.

This coalescing behavior is what makes handlers the right tool for "restart this service if its config changed" — using a normal task instead would restart the service after every single config-file task that ran, causing unnecessary flapping.

- [ ] How do you keep secrets out of an Ansible playbook that's stored in Git?

IF ASKED, SAY THIS
Ansible Vault — encrypts a variables file (or specific values inline) with a password/key, so the encrypted file is safe to commit; Ansible transparently decrypts it at run time given the vault password (supplied via prompt, a password file, or an external secrets lookup like the AWS Secrets Manager or HashiCorp Vault lookup plugins for a fully automated pipeline).

The practical pattern: `ansible-vault encrypt group_vars/prod/secrets.yml`, then reference those variables normally in playbooks — Ansible handles decryption transparently as long as the vault password is available to the run, typically injected via a CI/CD secret rather than typed by hand.

```
ansible-vault encrypt group_vars/prod/secrets.yml
ansible-playbook site.yml --vault-password-file=.vault_pass
```

- [ ] How do you use dynamic inventory instead of a static hosts file, and why does it matter in a cloud environment?

IF ASKED, SAY THIS
A dynamic inventory script/plugin (e.g. the aws_ec2 plugin) queries the cloud provider live at run time and builds the host list from actual running instances — filtered/grouped by tags — instead of a hand-maintained static file that goes stale the moment autoscaling adds or removes an instance.

This is close to a hard requirement once autoscaling groups are involved: a static inventory simply can't track instances that come and go on their own, so any team running Ansible against cloud infrastructure at scale is using dynamic inventory almost by necessity.

- [ ] What does "become" do in Ansible, and what's the least-privilege way to use it?

IF ASKED, SAY THIS
become escalates privilege for a task (typically to root via sudo) — set at the play level (become: true) or per-task. Least-privilege practice: don't run the whole play as root by default; scope become to only the specific tasks that actually need it (installing packages, editing system files), and use a dedicated ansible service account with narrowly-scoped, audited sudo rules rather than full passwordless root.

Interviewers listen for whether you default to "just make everything become: true at the top" (works, but is a blanket-privilege anti-pattern) versus scoping it per-task — the latter is the answer that signals actual production security awareness.

- [ ] A playbook run partially fails halfway through a fleet of 200 servers. How do you handle it?

IF ASKED, SAY THIS
Ansible tracks which hosts failed in a .retry file you can replay against with --limit @sitename.retry — but the real production practice is running with serial: (a rolling batch size, e.g. serial: 10) plus max_fail_percentage so a bad change stops propagating after the first few hosts fail instead of hitting all 200 before anyone notices.

`serial` combined with a health-check task between batches (waiting for a service to report healthy before moving to the next batch) turns a playbook run into something closer to a rolling deployment with a built-in circuit breaker, rather than an all-or-nothing blast to every host at once.

```
- hosts: webservers
  serial: "20%"
  max_fail_percentage: 10
  tasks: ...
```

- [ ] Chef vs. Terraform — where does each fit, and how do teams typically use them together?

IF ASKED, SAY THIS
Terraform provisions infrastructure (creates the VM/cluster/network); Chef (like Ansible) configures what runs on top of it. Teams commonly use both together: Terraform for the cloud resources, then a config-management tool to configure the OS/application layer — the same division of labor Ansible fills, just with an agent-based, Ruby-DSL approach instead of Ansible's agentless YAML.

Terraform is for **provisioning** — creating the EC2 instance, the VPC, the load balancer, the RDS instance; it manages cloud resources through the provider's API. Chef is for **configuration management** — once a machine exists, Chef installs packages, manages config files, ensures services run, and keeps the OS-level state converged over the machine's lifetime. The common pattern: Terraform creates the instance (and often triggers the bootstrap via `user_data` or a `remote-exec`-free approach like baking Chef Client into the AMI), and Chef then owns everything that happens on that box afterward, on its own recurring convergence schedule, independent of whether Terraform ever runs again. They're complementary, not competing — trying to make Terraform do Chef's job (managing in-OS config drift over time) or vice versa (having Chef create cloud resources) works against what each tool is actually built for.

- [ ] Ansible vs. Terraform for managing Kubernetes or cloud resources — where does each actually fit, and where do people misuse Ansible?

IF ASKED, SAY THIS
Terraform owns declarative resource lifecycle (create/update/destroy cloud resources, K8s objects via its provider) with real state tracking and drift detection. Ansible is procedural — it runs a sequence of tasks and doesn't natively track "desired state" the way Terraform's state file does. Using Ansible to manage cloud infrastructure lifecycle (instead of Terraform) is a common misuse — it can technically create resources via modules, but you lose Terraform's state-diffing and plan/apply safety net.

The clean split most senior engineers land on: Terraform for anything with a lifecycle (create/modify/destroy an actual resource), Ansible for anything that's a sequence of configuration steps on an existing target — and when both touch the same system, Terraform usually calls out to Ansible (via a provisioner or a separate pipeline step) rather than the other way around.

- [ ] Why would a shop choose Ansible specifically for configuration management of a Linux fleet, versus Terraform or a competing CM tool?

> **IF ASKED, SAY THIS**
> Ansible is agentless (SSH-based, no daemon to manage/patch on every target — a real win at fleet scale) and procedural-with-idempotency, which maps naturally onto "configure this Linux server to a desired state" — OS hardening, package management, service config. Terraform is the better tool for provisioning the infras

Ansible is **agentless** (SSH-based, no daemon to manage/patch on every target — a real win at fleet scale) and procedural-with-idempotency, which maps naturally onto "configure this Linux server to a desired state" — OS hardening, package management, service config.

Terraform is the better tool for provisioning the infrastructure itself (the VM/network/storage resources existing in the first place). Common pattern: Terraform provisions the box, Ansible configures what's on it — complementary, not competing for the same job.

- [ ] How do you structure Ansible playbooks/roles for a large environment so they stay maintainable, not a pile of one-off scripts?

> **IF ASKED, SAY THIS**
> Roles as the unit of reuse (a linux-hardening role, a jenkins-agent role, each with its own defaults/vars/handlers/tasks), inventory organized by group so host- and group-specific variables aren't duplicated across playbooks, and ansible-vault for secrets rather than plaintext credentials in version control. Testing wi

Roles as the unit of reuse (a `linux-hardening` role, a `jenkins-agent` role, each with its own defaults/vars/handlers/tasks), inventory organized by group so host- and group-specific variables aren't duplicated across playbooks, and `ansible-vault` for secrets rather than plaintext credentials in version control.

Testing with **Molecule** (spins up a container/VM, applies the role, asserts the result) catches breakage before it hits real fleet hosts; an internal role registry keeps teams from reinventing the same role differently.

- [ ] How would you use Ansible for OS patching/hardening specifically, tying back to the earlier hardening discussion?

> **IF ASKED, SAY THIS**
> A hardening role applying the CIS/STIG baseline as idempotent tasks (safe to re-run, shows drift if someone's manually changed something), a separate patching playbook targeting a specific ring/group with --limit , and ansible-lint plus CI for the playbooks themselves — yes, the automation code deserves the same CI dis

A hardening role applying the CIS/STIG baseline as idempotent tasks (safe to re-run, shows drift if someone's manually changed something), a separate patching playbook targeting a specific ring/group with `--limit`, and `ansible-lint` plus CI for the playbooks themselves — yes, the automation code deserves the same CI discipline as application code — so a bad playbook change gets caught in test, not by breaking production hosts.

- [ ] What's GitOps, and how would you apply it specifically to infrastructure automation rather than just application deployment?

> **IF ASKED, SAY THIS**
> Git as the single source of truth for desired infrastructure state, with an automated reconciliation loop (rather than someone running ansible-playbook manually from a laptop) continuously ensuring the real environment matches what's declared — drift gets corrected or flagged automatically. Applied to infra automation:

Git as the single source of truth for desired infrastructure state, with an **automated reconciliation loop** (rather than someone running `ansible-playbook` manually from a laptop) continuously ensuring the real environment matches what's declared — drift gets corrected or flagged automatically.

Applied to infra automation: Ansible playbooks/roles live in Git, changes go through the same PR/review process as application code, and a controller (a scheduled CI job re-running playbooks, or a dedicated GitOps tool) applies changes rather than relying on a local run.

- [ ] How would you use Python for infrastructure automation tasks that Ansible doesn't fit well?

> **IF ASKED, SAY THIS**
> Ansible is great for declarative, idempotent state management; Python fits better for things needing real logic — custom monitoring/alerting integrations, one-off migration/data-transformation scripts, internal tooling/APIs (a self-service portal for developers to request CI agents), or custom Ansible modules when a bu

Ansible is great for declarative, idempotent state management; Python fits better for things needing real logic — custom monitoring/alerting integrations, one-off migration/data-transformation scripts, internal tooling/APIs (a self-service portal for developers to request CI agents), or custom Ansible modules when a built-in module doesn't cover a need.

Not competing — Python often builds the tooling/glue around an Ansible-driven core, or gets called from within Ansible via the `script`/custom-module mechanism.

- [ ] What's your approach to testing infrastructure-as-code changes before they hit production?

> **IF ASKED, SAY THIS**
> Same discipline as application code, adapted: lint (ansible-lint , terraform validate / tflint), a plan/dry-run step (terraform plan , Ansible --check) a human reviews before apply, a genuinely representative staging environment, and Molecule-style role testing for Ansible specifically. The habit that

matters most:

Same discipline as application code, adapted: lint (`ansible-lint` , `terraform validate` / `tflint`), a plan/dry-run step (`terraform plan` , Ansible `--check`) a human reviews before apply, a genuinely representative staging environment, and Molecule-style role testing for Ansible specifically.

The habit that matters most: nobody applies untested IaC changes directly to production "because it's just a small change" — that's exactly the kind of change that causes the memorable outages.

- [ ] How do you handle secrets management across Ansible, Terraform, and general automation scripts? ▲

IF ASKED, SAY THIS
Centralize in a real secrets manager (HashiCorp Vault, Azure Key Vault, AWS Secrets Manager) rather than scattering ansible-vault files, .tfvars with embedded secrets, and hardcoded script credentials across tools inconsistently. Where a tool-specific mechanism is still needed (ansible-vault for playbook-embedded secre

Centralize in a real secrets manager (HashiCorp Vault, Azure Key Vault, AWS Secrets Manager) rather than scattering `ansible-vault` files, `.tfvars` with embedded secrets, and hardcoded script credentials across tools inconsistently.

Where a tool-specific mechanism is still needed (ansible-vault for playbook-embedded secrets), the vault password itself should come from the central secrets manager — not be a shared file sitting in a team's home directory, a surprisingly common real-world anti-pattern worth explicitly naming.

Git

- [ ] Trunk-based development vs. GitFlow vs. GitHub Flow — what's the actual tradeoff, and which do you default to?

IF ASKED, SAY THIS
GitFlow (develop/release/feature/hotfix branches) suits slower, versioned-release software but creates long-lived branches that drift and merge painfully. Trunk-based (short-lived branches, merge to main constantly, feature flags hide unfinished work) fits continuous deployment and is what most teams doing multiple-deploys-a-day actually run. GitHub Flow sits in between — one main branch, short feature branches, PR + deploy on merge, no separate develop/release branches.

The real signal in this question is whether you can articulate WHY trunk-based fits continuous deployment specifically: long-lived branches mean long-lived merge conflicts and a big-bang integration at the end, which is the opposite of what a team shipping multiple times a day wants — feature flags exist specifically to let trunk-based teams merge unfinished work safely.

- [ ] Merge vs. rebase — what's actually different about the resulting history, and when do you use each?

IF ASKED, SAY THIS
Merge preserves true history (a merge commit shows exactly when and how branches combined) but creates a non-linear, sometimes noisy graph. Rebase rewrites your branch's commits to sit on top of the latest main, producing a clean linear history — but it REWRITES commit hashes, which is unsafe on any branch someone else has already pulled.

The operational rule that matters more than the philosophical debate: never rebase a shared/public branch. Rebase your own local feature branch before opening a PR (clean history for review); once it's shared or merged, only merge or use a non-destructive approach from there.

```
git checkout feature-x
git rebase main          # replay feature-x's commits onto latest main
git push --force-with-lease   # only safe because feature-x is still yours alone
```

- [ ] What actually IS a Git commit under the hood, and how does that explain why Git operations are so fast?

IF ASKED, SAY THIS
A commit is a content-addressed object (SHA-1/SHA-256 hash of its contents) pointing to a tree object (a snapshot of the whole file structure at that point) and a parent commit. Because objects are hashed by content, identical file content across commits is stored once — Git isn't storing per-file diffs, it's storing full snapshots deduplicated by content hash, which is why branching/checkout is near-instant (no diff computation, just pointer changes).

This is the detail that separates "knows git commands" from "understands git" in an interview: branches and tags are just lightweight pointers (a 40-character hash in a small file) to a commit — creating a branch is one file write, not a copy of anything, which is why Git branching is so cheap compared to older VCS tools.

- [ ] You just did something destructive — a bad reset --hard, or deleted a branch with unmerged work. How do you recover it?

IF ASKED, SAY THIS
git reflog — it's a local log of every place HEAD has pointed, including commits that are no longer reachable from any branch. Find the commit hash from before the disaster, then git checkout or git branch recovery-branch to get it back. The reflog typically retains entries for ~90 days by default, so this works even well after the mistake.

This is one of the single most valuable things to have memorized cold for an interview — a huge fraction of "oh no I think I just lost work" panic is solved in under a minute by someone who knows the reflog exists, versus someone who doesn't and assumes the work is gone.

```
git reflog
# find the hash from right before the mistake
git branch recovered-work <hash>
```

- [ ] git reset vs. git revert — what's the actual difference, and which is safe to use on a shared branch?

IF ASKED, SAY THIS
reset moves the branch pointer (and optionally the working directory/index) backward, rewriting history — unsafe once pushed/shared. revert creates a NEW commit that undoes a prior commit's changes, leaving history intact and forward-only — always safe on a shared branch, including main/production branches.

This is exactly why "how do you undo a bad commit that's already in production's main branch" has one correct answer in a team setting: revert, never reset — resetting a shared branch rewrites history everyone else's clone has already diverged from, causing exactly the kind of mess a production infra repo can't afford.

- [ ] What Git practices would you enforce specifically for a production infrastructure repo (Terraform/K8s manifests), as opposed to an application repo?

IF ASKED, SAY THIS
Mandatory PR review with at least one approval before merge to main, branch protection preventing direct pushes to main, a required CI check (terraform plan / kubeval) that must pass before merge is even allowed, signed commits or verified authorship for audit trail, and — specifically for infra — treating a revert of a bad infra change as equally urgent as an application rollback, since a bad Terraform merge can be actively destroying resources.

The audit-trail angle is worth naming explicitly in a regulated environment: infra repos often need to answer "who approved this change and when" for compliance, which application repos may not carry the same weight for — this is a good place to mention signed commits or a required-reviewers CODEOWNERS file if the role is at a bank or similarly regulated shop.

- [ ] Two engineers modified the same Terraform module in parallel and now there's a merge conflict. Walk through resolving it safely, not just mechanically.

IF ASKED, SAY THIS
Mechanically: git status shows the conflicted file, resolve the markers by hand, git add, continue the merge/rebase. Safely, for infra code specifically: after resolving, always run terraform plan again before merging — a syntactically-clean conflict resolution can still produce a plan that destroys/recreates resources neither engineer intended, which a plain merge tool can't catch.

This is the detail that shows infra-specific judgment rather than generic git knowledge: application code conflicts are usually safe once they compile and tests pass; Terraform conflicts need an EXTRA verification step (plan) because the danger isn't a syntax error, it's a semantically-valid-but-destructive change.

- [ ] What's git bisect, and when would you actually reach for it over just reading the log?

IF ASKED, SAY THIS
git bisect binary-searches your commit history to find the exact commit that introduced a bug — you mark a known-good and known-bad commit, Git checks out the midpoint, you test and mark it good/bad, and it narrows down in O(log n) steps instead of you eyeballing hundreds of commits.

This earns its place specifically when "something broke sometime in the last N commits/weeks" and there's no obvious suspect — for a slow, gradually-introduced regression (a memory leak, a perf regression) across dozens of commits, bisect finds the culprit commit far faster than manual log archaeology.

```
git bisect start
git bisect bad                 # current commit is broken
git bisect good v1.4.0         # this old tag was fine
# git checks out the midpoint - you test, then:
git bisect good   # or: git bisect bad
# repeat until git names the exact breaking commit
```

- [ ] What's git cherry-pick, and give a real production scenario where you'd use it.

IF ASKED, SAY THIS
cherry-pick applies the changes from one specific commit onto your current branch, without merging the whole branch it came from. Real scenario: a critical bug fix landed on main, but production is running an older release branch — cherry-pick just that fix commit onto the release branch for a hotfix, instead of merging all of main's unrelated in-progress work into production.

This comes up constantly in any team that maintains a release branch alongside active main-branch development — it's the mechanism for "take just this one fix, nothing else" across diverging branches.

- [ ] Monorepo vs. many small repos for a platform with dozens of microservices — what's the actual tradeoff?

IF ASKED, SAY THIS
Monorepo: atomic cross-service commits (change a shared library and all its consumers in one PR), simpler dependency management, one CI/CD config to maintain — but needs real tooling investment (path-based CI triggers, sparse checkout) to stay fast as it grows. Many repos: natural service boundaries and independent versioning/deploy cadence per team — but shared-library changes become a multi-repo, multi-PR coordination problem.

The tell for seniority here is mentioning that monorepos need explicit tooling (Bazel/Nx-style build graphs, or CI that only runs affected services' pipelines) to avoid every commit triggering a full-platform CI run — without that investment, a monorepo's CI time explodes as the platform grows.

Jenkins

What is Jenkins and how does the controller/agent model work?

IF ASKED, SAY THIS
Jenkins is an automation server for CI/CD. The controller schedules jobs, serves the UI, and holds configuration, while agents are separate machines that actually run the builds — distributing load and enabling multi-platform builds via labels. Agents can be static or dynamically provisioned, for ex

- Jenkins is an open-source automation server for CI/CD. The controller (formerly 'master') schedules jobs, serves the UI, and stores config; it should not run heavy builds itself.
- Agents (nodes) are machines that execute the build work, connected to the controller. This distributes load and lets builds run on different platforms/labels.
- Agents can be static VMs or dynamic (Kubernetes pods, cloud instances spun up per build).

What is the difference between Declarative and Scripted pipelines?

IF ASKED, SAY THIS
Declarative pipelines use a structured `pipeline { }` block with predefined sections like agent, stages, and post — readable, validated, and the recommended default. Scripted pipelines are full Groovy in a `node { }` block, offering maximum flexibility for complex logic at the cost of maintainabilit

- Declarative pipeline: a structured, opinionated syntax inside a `pipeline { }` block with predefined sections (agent, stages, steps, post). Easier to read, validate, and the recommended default.
- Scripted pipeline: full Groovy with `node { }` — more flexible and powerful for complex logic, but harder to maintain.
- Both live in a `Jenkinsfile` checked into source control (pipeline-as-code).

```
pipeline {
  agent any
  stages {
    stage('Build'){ steps { sh 'make build' } }
    stage('Test'){ steps { sh 'make test' } }
  }
  post { failure { mail to: 'team@x.com', subject: 'Build failed' } }
}
```

What is a Jenkinsfile and why is pipeline-as-code valuable?

IF ASKED, SAY THIS
A Jenkinsfile is the pipeline defined as code and committed to the repository. That makes CI logic version-controlled, reviewable, auditable, and reproducible, and it lets pipeline changes travel with the code change in the same branch or PR. Multibranch pipelines build on this by auto-creating jobs

- A Jenkinsfile defines the pipeline as code, stored in the repository alongside the application.
- Benefits: version control and code review of CI logic, reproducibility, history/audit, and the ability to branch pipeline changes with the code they affect.
- Multibranch pipelines auto-discover branches/PRs with a Jenkinsfile and create jobs for them.

How do you handle credentials and secrets in Jenkins?

IF ASKED, SAY THIS
Jenkins stores secrets encrypted via the Credentials plugin, scoped globally or per folder, and you consume them with `withCredentials` or the `credentials()` helper, which masks them in logs. Ideally you integrate an external secrets manager like Vault or AWS Secrets Manager so secrets never live i

- Use the Credentials plugin to store secrets (tokens, SSH keys, username/password) encrypted, scoped globally or per-folder.
- Reference them in pipelines via the `credentials()` helper or `withCredentials { }`, which masks them in logs.
- Better still, integrate an external manager (HashiCorp Vault, AWS Secrets Manager) so secrets aren't stored in Jenkins at all. Lock down Jenkins RBAC since it's a high-value target.

```
withCredentials([string(credentialsId:'api-token', variable:'TOKEN')]) {
  sh 'deploy --token=$TOKEN'
}
```

- [ ] How do you scale Jenkins and run builds on Kubernetes?

> **IF ASKED, SAY THIS**
> At scale you use the Kubernetes plugin to provision an ephemeral agent pod per build that's destroyed afterward — giving clean, elastic, cost-bounded capacity instead of long-lived snowflake agents. Pod templates define the build containers and resources, and shared libraries centralize common pipel

- The Kubernetes plugin spins up a pod per build as an ephemeral agent, then tears it down — clean environment, elastic capacity, no idle agents.
- You define agent pod templates (containers/resources) in the pipeline. This avoids snowflake static agents and bounds cost.
- Combine with shared libraries to DRY up pipeline logic across many repos.

- [ ] A pipeline stage fails intermittently ('flaky'). How do you approach it?

> **IF ASKED, SAY THIS**
> I'd first characterize the flakiness — which stage, agent, or condition — using console logs to separate environment issues (network, resource limits, dirty workspace) from test flakiness. Short-term mitigations include pinning versions, ensuring clean isolated workspaces, adding resource headroom,

- First, gather data: is it a specific stage, agent, or time? Check console logs and whether it's environment (network, resource limits) or test flakiness.
- Mitigations: pin tool/dependency versions, isolate state between builds (clean workspace), add resource headroom, and use `retry()` for genuinely transient external calls — sparingly.
- Long-term: fix root cause (non-deterministic tests, race conditions) rather than masking with retries.

- [ ] Jenkins is running fine but suddenly builds are queuing and agents are not picking up jobs. How do you debug this?

> **IF ASKED, SAY THIS**
> This is a real production problem with Jenkins at scale. I follow this sequence: Step 1 - Check agent status: Go to Manage Jenkins > Nodes or run: curl -s http://jenkins/computer/api/json | jq '.computer[].displayName + " " + .computer[].offline' . Are agents showing as offline? Disconnected? Busy?

This is a real production problem with Jenkins at scale. I follow this sequence:

Step 1 - Check agent status: Go to Manage Jenkins > Nodes or run: `curl -s http://jenkins/computer/api/json | jq '.computer[].displayName + " " + .computer[].offline'`. Are agents showing as offline? Disconnected? Busy?

Step 2 - Check agent logs: For Kubernetes agents, `kubectl get pods -n jenkins` and `kubectl logs AGENT-POD`. Common causes: pod evicted due to node memory pressure, JNLP connection refused, PVC full.

Step 3 - Check master disk: Jenkins master needs disk for workspace, artifacts, logs. `df -h` on the Jenkins master node. A full disk silently prevents agents from connecting.

Step 4 - Check JNLP port: Kubernetes agents connect to the Jenkins master on port 50000 (JNLP). Check that the Jenkins service exposes this port and firewall rules allow it.

Step 5 - Check resource quotas: In K8s, if the jenkins namespace has a ResourceQuota that is full, new agent pods cannot be scheduled. `kubectl describe quota -n jenkins`.

Root fix: In production I set up Prometheus monitoring on Jenkins itself - number of queued builds, agent count, build duration. Alert when queue depth exceeds 10 for more than 5 minutes. Catch it before engineers notice.

- [ ] A Jenkins pipeline is passing locally but failing in CI with 'permission denied' errors on Docker. What is happening and how do you fix it?

> **IF ASKED, SAY THIS**
> Classic Docker-in-Docker permission problem. Multiple possible causes: Cause 1 - Socket permission: The Jenkins agent container is trying to use the host Docker socket at /var/run/docker.sock. If the socket is owned by root and the container runs as a non-root user, you get permission denied. Fix: A

Classic Docker-in-Docker permission problem. Multiple possible causes:

Cause 1 - Socket permission: The Jenkins agent container is trying to use the host Docker socket at /var/run/docker.sock. If the socket is owned by root and the container runs as a non-root user, you get permission denied.

Fix: Add the jenkins user to the docker group in the agent image: `usermod -aG docker jenkins`. Or mount the socket with the right permissions. Or use DinD (Docker-in-Docker) with privileged mode.

Cause 2 - DinD container not ready: If using a separate DinD sidecar, the build container may start running Docker commands before the DinD daemon is ready. Fix: Add a wait-for-docker step: `until docker ps; do sleep 1; done`.

Cause 3 - Security context too restrictive: The K8s pod security policy or OPA admission controller may block privileged containers needed for DinD. Fix: Create a specific PodSecurityPolicy that allows privileged mode only for the docker container in jenkins namespace.

Production recommendation: Avoid Docker-in-Docker entirely. Use Kaniko for building images inside Kubernetes - it builds Docker images without needing Docker daemon or privileged access. Much more secure for regulated environments.

- [ ] How do you manage secrets in Jenkins pipelines across 50 microservices without repeating credentials everywhere?

> **IF ASKED, SAY THIS**
> At scale, you need a centralised secret strategy. Here is what I implement: 1. Vault integration as the source of truth: HashiCorp Vault stores all secrets. Jenkins has a Vault plugin. Pipelines authenticate to Vault using the Jenkins service account token (Kubernetes auth method). Vault returns sho

At scale, you need a centralised secret strategy. Here is what I implement:

1. Vault integration as the source of truth: HashiCorp Vault stores all secrets. Jenkins has a Vault plugin. Pipelines authenticate to Vault using the Jenkins service account token (Kubernetes auth method). Vault returns short-lived dynamic secrets.

2. Shared Library for Vault access: Write a Jenkins Shared Library function once that handles Vault authentication and secret retrieval. All 50 pipelines call this function. One place to update when Vault path changes.

```
// vars/getSecret.groovy in shared library
def call(String secretPath, String key) {
    def vaultSecrets = [
        [path: secretPath, secretValues: [
            [envVar: 'SECRET_VALUE', vaultKey: key]
        ]]
    ]
    withVault([vaultSecrets: vaultSecrets]) {
        return env.SECRET_VALUE
    }
}
```

3. Never print secrets: Jenkins masks credentials automatically if you use the credentials() binding. Never echo or print secret values. Add `set +x` before commands that use secrets in shell scripts.

4. Audit trail: Vault logs every secret access. You can see which pipeline accessed which secret and when. Critical for SOX compliance.

5. Secret rotation: Dynamic secrets from Vault rotate automatically. Jenkins gets a fresh token per build. Even if a build log is compromised, the token is already expired.

- [ ] Sketch a production-grade declarative Jenkinsfile and call out the pieces that actually matter operationally.

> **IF ASKED, SAY THIS**
> agent + stages(Checkout, Build, Test, Security Scan, Docker Build/Push, Deploy) + post block for notifications, plus options like timeout() and disableConcurrentBuilds() — the post block and timeout are the parts people forget and then get paged for.

- **options { timeout(time: 30, unit: 'MINUTES') }** — without this, a hung build can block the agent/executor indefinitely.
- **post { failure { slackSend(...) } }** — a pipeline with no failure notification just silently stops deploying and nobody notices until someone asks why the last deploy is 3 days old.
- **Credentials via the Jenkins credentials store** (`withCredentials`), never hardcoded in the Jenkinsfile — the Jenkinsfile lives in source control.
- **Parallel stages** for independent steps (e.g. unit tests + lint) to cut pipeline time — a pipeline that's slow enough that people start skipping it defeats its own purpose.

```
pipeline {
  agent any
  options { timeout(time: 30, unit: 'MINUTES'); disableConcurrentBuilds() }
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Build')    { steps { sh 'mvn -B package -DskipTests' } }
    stage('Test')     { steps { sh 'mvn test' } }
    stage('Docker')   { steps { sh 'docker build -t : .' } }
    stage('Deploy')   { steps { sh 'kubectl set image deploy/app app=:' } }
  }
  post {
    failure { slackSend(color: 'danger', message: "Build failed: ${env.JOB_NAME} #${env.BUILD_NUMBER}") }
  }
}
```

- [ ] Explain Jenkins architecture — controller vs agents — and how it scales.

> **IF ASKED, SAY THIS**
> The **controller** (historically 'master') schedules jobs, serves the UI, and stores configuration and job history, but shouldn't run heavy builds itself in a production setup. **Agents** (nodes) are the actual build executors — static, permanent VMs, or increasingly dynamic ones spun up on demand (

The **controller** (historically 'master') schedules jobs, serves the UI, and stores configuration and job history, but shouldn't run heavy builds itself in a production setup. **Agents** (nodes) are the actual build executors — static, permanent VMs, or increasingly dynamic ones spun up on demand (via

the Kubernetes plugin, EC2 plugin, or Docker) and torn down after the build finishes. Scaling is mostly about the agent layer: labels route specific jobs to agents with the right toolchain, and dynamic agents (especially Kubernetes-based, where each build gets a fresh pod) avoid the classic 'works on this one Jenkins node but not that one' drift problem and let capacity scale to zero when there's no work.

- [ ] Declarative vs scripted Jenkins pipelines — what's the real difference, and which do you default to?

IF ASKED, SAY THIS

Declarative pipelines use a structured, opinionated syntax (`pipeline { agent {} stages {} }`) with built-in support for things like `post` blocks, `options`, and `parameters` — easier to read, lint, and standardize across a team, and it's the default for the vast majority of pipelines. **Script

Declarative pipelines use a structured, opinionated syntax (`pipeline { agent {} stages {} }`) with built-in support for things like `post` blocks, `options`, and `parameters` — easier to read, lint, and standardize across a team, and it's the default for the vast majority of pipelines. **Scripted** pipelines are raw Groovy with full programmatic control — loops, complex conditionals, dynamic stage generation — useful for genuinely complex orchestration logic that declarative syntax can't express cleanly. In practice, teams default to declarative and drop into a scripted block (`script { ... }`) inside a stage only for the specific piece of logic that needs it, rather than writing the whole pipeline as scripted.

- [ ] What are Jenkins shared libraries, and why would a team invest in them?

IF ASKED, SAY THIS

A shared library is a Groovy codebase, versioned in its own repo, that pipelines across the org import (`@Library('my-lib') _`) to reuse common logic instead of copy-pasting the same 40-line deployment stage into every Jenkinsfile. Typical contents: standardized build/test/deploy steps, notification

A shared library is a Groovy codebase, versioned in its own repo, that pipelines across the org import (`@Library('my-lib') _`) to reuse common logic instead of copy-pasting the same 40-line deployment stage into every Jenkinsfile. Typical contents: standardized build/test/deploy steps, notification helpers, and org-wide guardrails like enforcing a security scan stage. The payoff is consistency and maintainability at scale — fixing a bug or adding a required stage happens once in the library instead of once per repo, and it becomes the natural place to encode 'how this org actually deploys' as reusable code rather than tribal knowledge in each team's Jenkinsfile.

- [ ] How do you handle credentials securely in a Jenkins pipeline?

IF ASKED, SAY THIS

Everything goes through the **Credentials plugin** — never a hardcoded secret in a Jenkinsfile or a shell step, since that would leak in build logs and source control. Credentials are stored encrypted in Jenkins and referenced by ID, then either bound to environment variables with `withCredentials([

Everything goes through the **Credentials plugin** — never a hardcoded secret in a Jenkinsfile or a shell step, since that would leak in build logs and source control. Credentials are stored encrypted in Jenkins and referenced by ID, then either bound to environment variables with `withCredentials([usernamePassword(...)])`/`string(...)` for the scope of a single step, or pulled dynamically from a real secrets backend (Vault, AWS Secrets Manager) via a plugin so Jenkins itself isn't the long-term source of truth. Jenkins also masks known credential values in console output automatically, but that's a safety net, not a substitute for scoping credentials tightly (folder- or job-level rather than globally available to every pipeline) and rotating them.

- [ ] How do you add and manage build agents/nodes at scale?

IF ASKED, SAY THIS

Static agents are registered manually or via JNLP/SSH and tagged with **labels** describing their capabilities (e.g. `linux-docker`, `windows-dotnet`), and pipelines target a label rather than a specific node name so jobs aren't tied to one machine's availability. At scale, the better pattern is dyn

Static agents are registered manually or via JNLP/SSH and tagged with **labels** describing their capabilities (e.g. `linux-docker`, `windows-dotnet`), and pipelines target a label rather than a specific node name so jobs aren't tied to one machine's availability. At scale, the better pattern is dynamic provisioning — the Kubernetes plugin spins up a pod per build defined by a pod template (with the exact tool image the job needs) and tears it down after, so there's no persistent fleet to patch and no cross-build state leakage. Executors-per-node controls concurrency per agent; getting that wrong (too high) is a common cause of flaky builds from resource contention on a shared box.

- [ ] What is TeamCity, and how do its core concepts map to what you already know from Jenkins?

IF ASKED, SAY THIS

TeamCity is JetBrains' CI/CD server, built around **Build Configurations** (roughly Jenkins' 'job'/pipeline), **VCS Roots** (a reusable, decoupled definition of a source repo and branch spec — one VCS root can be shared by several configurations, which is cleaner than Jenkins' more ad hoc per-job ch

TeamCity is JetBrains' CI/CD server, built around **Build Configurations** (roughly Jenkins' 'job'/pipeline), **VCS Roots** (a reusable, decoupled definition of a source repo and branch spec — one VCS root can be shared by several configurations, which is cleaner than Jenkins' more ad hoc per-job checkout config), **Build Chains** (dependencies between configurations — build A triggers build B, similar in spirit to Jenkins' pipeline stages but modeled as a graph across separate jobs rather than steps in one job), and **Templates** (shared configuration, TeamCity's rough equivalent to a Jenkins shared library, but as structured settings inheritance rather than Groovy code). TeamCity also has first-class support for configuration as Kotlin DSL, similar in intent to Jenkinsfile-as-code.

GitHub Actions

Explain the GitHub Actions hierarchy: workflow, job, step, action.

IF ASKED, SAY THIS
A workflow is an event-triggered YAML file containing jobs; each job runs its steps on one runner, with jobs running in parallel unless linked by `needs`. A step is either a shell command (`run`) or a reusable action (`uses`), and an action is a packaged reusable unit like `actions/checkout`. So the

- Workflow: a YAML file in `.github/workflows/` triggered by events. Contains one or more jobs.
- Job: a set of steps that run on a single runner; jobs run in parallel by default and can declare dependencies with `needs`.
- Step: an individual task — either a shell command (`run`) or a reusable action (`uses`).
- Action: a packaged, reusable unit (e.g. `actions/checkout`), referenced by a step.

```
jobs:
  test: { runs-on: ubuntu-latest, steps: [...] }
  deploy:
    needs: test
    runs-on: ubuntu-latest
```

What triggers a workflow? Explain events.

IF ASKED, SAY THIS
The `on:` key defines events that start a workflow — push, pull_request, schedule (cron), workflow_dispatch for manual runs, release, and others. You filter with branch, path, and tag filters to avoid wasteful runs, and `workflow_call` lets a workflow be invoked as a reusable building block by other

- The `on:` key defines triggers: push, pull_request, schedule (cron), workflow_dispatch (manual), release, and many more.
- You can filter by branches, paths, and tags to avoid unnecessary runs.
- workflow_call lets one workflow be reused by another (reusable workflows).

```
on:
  push: { branches: [main], paths: ['src/**'] }
  workflow_dispatch:
```

What are GitHub-hosted vs self-hosted runners?

IF ASKED, SAY THIS
GitHub-hosted runners are clean, ephemeral VMs provisioned per job with no maintenance, billed per minute. Self-hosted runners are machines you own and manage, needed for private network access, special hardware, or cost control. A key caution is to avoid self-hosted runners on public repositories,

- GitHub-hosted runners are ephemeral VMs GitHub provisions per job — clean environment, no maintenance, billed by minute. Good default.
- Self-hosted runners are machines you manage — needed for private network access, specialized hardware (GPUs), custom tooling, or cost control at scale.
- Security caution: don't use self-hosted runners on public repos (untrusted PRs can run code on your infra).

How do you manage secrets in GitHub Actions?

IF ASKED, SAY THIS
Secrets are stored at repo, environment, or org scope and referenced with `${{ secrets.NAME }}`, automatically masked in logs. Production deploys are gated using Environments with required reviewers and environment-scoped secrets. The modern best practice for cloud access is OIDC — the workflow assu

- Store secrets at repo, environment, or org level. Reference them via `${{ secrets.NAME }}`. They're masked in logs.
- Use Environments with required reviewers and environment secrets to gate production deploys.
- Best practice for cloud: use OIDC to assume a cloud IAM role short-term instead of storing long-lived cloud keys as secrets.

```
steps:
  - uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: arn:aws:iam::123:role/gha
      aws-region: us-east-1
```

What is a matrix build?

IF ASKED, SAY THIS
A matrix runs the same job across combinations of variables — like multiple language versions and operating systems — generating one parallel job per combination. `fail-fast: false` lets the rest continue when one combination fails. It eliminates duplicated job definitions and surfaces version- or p

- A matrix runs a job multiple times across combinations of variables — e.g. test on Node 18/20/22 across ubuntu/windows/macos.
- It auto-generates one job per combination, running in parallel. `fail-fast: false` keeps others going if one fails.
- Reduces duplication and catches version/platform-specific bugs.

```
strategy:
  fail-fast: false
  matrix:
    node: [18, 20, 22]
    os: [ubuntu-latest, windows-latest]
```

How do you cache dependencies and share data between jobs?

IF ASKED, SAY THIS
Caching with `actions/cache` restores dependency directories keyed by a lockfile hash to speed builds, but a cache may be missing so you can't depend on it. Artifacts (`upload-artifact`/`download-artifact`) reliably pass build outputs between jobs or persist them after the run. Since each job runs o

- Caching (`actions/cache`) speeds up runs by restoring dependency directories keyed by a lockfile hash. It's a performance optimization, not guaranteed to exist.
- Artifacts (`upload-artifact`/`download-artifact`) pass build outputs between jobs or persist them after the run — guaranteed within the run.
- Jobs run on separate runners with no shared filesystem, so artifacts (or outputs) are how you hand data between them.

Walk through GitHub Actions authenticating to AWS via OIDC — why is this the right answer instead of storing an access key as a repo secret?

IF ASKED, SAY THIS
Long-lived IAM access keys stored as repo secrets are a standing liability — they don't expire on their own, they're exfiltratable if a workflow or dependency is compromised, and rotating them is a manual process someone has to remember to do. OIDC federation removes the credential entirely: GitHub'

Long-lived IAM access keys stored as repo secrets are a standing liability — they don't expire on their own, they're exfiltratable if a workflow or dependency is compromised, and rotating them is a manual process someone has to remember to do. OIDC federation removes the credential entirely: GitHub's Actions runner requests a short-lived OIDC token scoped to that specific workflow run, AWS's IAM OIDC identity provider validates it against a trust policy, and issues temporary STS credentials that expire automatically — nothing long-lived ever exists to leak.

```
# One-time setup: IAM OIDC provider trusting token.actions.githubusercontent.com

# Workflow step:
- uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::123456789012:role/gha-deploy-role
    aws-region: ap-south-1
```

The trust policy on `gha-deploy-role` should be scoped tightly on the `sub` claim — e.g. restricted to a specific repo and branch (`repo:acme/platform:ref:refs/heads/main`) — so a workflow on a feature branch or a fork can't assume a production deploy role just because the OIDC provider trusts the GitHub org generally.

Reusable workflows vs. composite actions — both let you avoid copy-pasting YAML. What's the actual difference?

IF ASKED, SAY THIS
A composite action bundles a sequence of steps into something you call as a single step inside a job — it runs in the context of the calling job, sharing that job's runner and environment. A reusable workflow is invoked with uses: at the job level (jobs.build.uses: ./.github/workflows/build.yml) —

A **composite action** bundles a sequence of steps into something you call as a single step inside a job — it runs in the context of the calling job, sharing that job's runner and environment. A **reusable workflow** is invoked with `uses:` at the *job* level (`jobs.build.uses: ./.github/workflows/build.yml`) — it runs as its own job (or set of jobs), can define its own permissions, secrets, and even matrix strategy, and can itself call other reusable workflows.

Rule of thumb: composite action for "these 4 steps always go together" (checkout, setup language, install deps, run linter); reusable workflow for "this entire build-test-push pipeline should be identical across 15 microservice repos" — the latter is what actually enforces platform-wide standardization, since teams call the central workflow rather than maintaining their own copy that quietly drifts.

- [ ] How do GitHub Environments give you deployment governance, not just secret scoping?

> **IF ASKED, SAY THIS**
> An Environment (e.g. production) attached to a job via environment: production gates that job behind rules you configure once: required reviewers (a human must approve before the job runs — the closest GitHub-native equivalent to a change-approval-board gate), a wait timer, and branch restrictions

An Environment (e.g. `production`) attached to a job via `environment: production` gates that job behind rules you configure once: required reviewers (a human must approve before the job runs — the closest GitHub-native equivalent to a change-approval-board gate), a wait timer, and branch restrictions (only `main` can deploy to this environment). Secrets and variables scoped to that environment are only exposed to jobs that reference it — meaning a production database credential simply doesn't exist in the context of a job targeting staging, rather than relying on workflow logic to "remember" not to use it there.

For this JD's governance emphasis specifically: environments are the natural place to enforce "prod deploys require a second approver" as a platform-level control rather than a per-repo convention someone can forget to add.

- [ ] When do you need self-hosted runners instead of GitHub-hosted ones, and what security implications come with that choice?

> **IF ASKED, SAY THIS**
> Self-hosted runners earn their operational overhead when a job needs to reach something with no public path — an internal EKS API server, an on-prem system, a private VPC resource without exposing it publicly just for CI — or when you need specific hardware/licensing GitHub-hosted runners don't offe

Self-hosted runners earn their operational overhead when a job needs to reach something with no public path — an internal EKS API server, an on-prem system, a private VPC resource without exposing it publicly just for CI — or when you need specific hardware/licensing GitHub-hosted runners don't offer.

⚠ The security angle interviewers actually probe: a *persistent* self-hosted runner that also picks up workflow runs triggered by pull requests from forks is a real, well-documented attack path — a malicious PR can get arbitrary code executed on your runner, in your network, with whatever credentials that runner has cached. Mitigations: run `pull_request_target` workflows (which have access to secrets) with extreme care and minimal checkout of untrusted code, prefer **ephemeral** self-hosted runners (spun up per-job, destroyed after, so nothing persists to be compromised across runs), and never let fork PRs trigger workflows with write access to secrets without a required-approval gate first.

- [ ] A pipeline's dependency install step is the slowest part of every run. How do you speed it up?

> **IF ASKED, SAY THIS**
> actions/cache , keyed off a hash of the lockfile so the cache invalidates exactly when dependencies actually change, not on every run:

`actions/cache`, keyed off a hash of the lockfile so the cache invalidates exactly when dependencies actually change, not on every run:

```
- uses: actions/cache@v4
  with:
    path: ~/.cache/pip
    key: pip-${{ runner.os }}-${{ hashFiles('requirements.txt') }}
    restore-keys: pip-${{ runner.os }}-
```

The `restore-keys` fallback matters — even if the exact lockfile hash changed (one new dependency added), a partial-match cache from the previous key still saves most of the download/build time versus a fully cold cache. For container-based builds, layer caching via `docker/build-push-action` 's built-in cache-from/cache-to (backed by GitHub Actions cache or a registry) does the equivalent for image builds — skip rebuilding layers whose inputs haven't changed.

- [ ] Why do security-conscious pipelines pin third-party actions to a commit SHA instead of a version tag like @v4 ?

> **IF ASKED, SAY THIS**
> A tag is mutable — the maintainer (or an attacker who compromises the maintainer's account) can repoint v4 at different code at any time, and your workflow silently starts running whatever that tag now points to on its next trigger, with whatever permissions and secrets that workflow has. This is a

A tag is mutable — the maintainer (or an attacker who compromises the maintainer's account) can repoint `v4` at different code at any time, and your workflow silently starts running whatever that tag now points to on its next trigger, with whatever permissions and secrets that workflow has. This is a real, materialized supply-chain attack class, not a theoretical one. Pinning to a full commit SHA — `uses: aws-actions/configure-aws-credentials@e3dd6a...` — means the exact code that runs can't change without you deliberately bumping the pin (ideally with Dependabot opening a PR for each update, so it's reviewed rather than silent).

⚠ Pair this with least-privilege `permissions:` blocks at the workflow or job level (`permissions: {contents: read}` as a default, only elevating specific jobs that need `id-token: write` for OIDC or `packages: write` to publish) — the default `GITHUB_TOKEN` permissions are broader than most jobs actually need.

- [ ] Where's the line between what GitHub Actions should do and what Argo CD/Flux should do in a GitOps pipeline?

> **IF ASKED, SAY THIS**
> GitHub Actions owns CI — build, test, scan, and produce an artifact (a container image pushed to a registry, tagged with the commit SHA), then update a manifest or Helm values file in a separate GitOps config repo (bump the image tag). It should stop there for anything running on Kubernetes.

GitHub Actions owns **CI** — build, test, scan, and produce an artifact (a container image pushed to a registry, tagged with the commit SHA), then update a manifest or Helm values file in a separate GitOps config repo (bump the image tag). It should stop there for anything running on Kubernetes.

Argo CD/Flux owns **CD** — it continuously reconciles the cluster's actual state against that GitOps repo, pulling the change rather than Actions pushing it. This split matters for two real reasons: the cluster's deploy credentials never need to exist in CI at all (Actions never talks to the cluster directly, only to git and a registry), and the cluster is always reconcilable to a known-good state from git even if CI is down or compromised. For non-Kubernetes targets (Lambda, ECS) this separation is less rigid and Actions commonly does drive the deploy directly via the AWS CLI/SDK — worth being explicit in an interview that the GitOps pull model is a Kubernetes-specific pattern, not a universal rule.

- [ ] Someone pushes 4 commits to a branch in quick succession and 4 full pipeline runs kick off, racing each other. How do you prevent that?

IF ASKED, SAY THIS
Concurrency groups — scope a group key to the branch/ref, and cancel superseded runs automatically:

Concurrency groups — scope a group key to the branch/ref, and cancel superseded runs automatically:

```
concurrency:
  group: deploy-${{ github.ref }}
  cancel-in-progress: true
```

Only the latest push's run survives; earlier in-flight runs for the same ref are cancelled rather than racing to deploy. Worth a caveat in an interview answer: `cancel-in-progress: true` is right for build/test stages, but you generally do *not* want it on a job that's mid-deploy to production — cancelling a deploy halfway through can leave infrastructure in a partially-applied state, so that specific job is often better left to run to completion (or given a separate, non-cancelling concurrency group) even while earlier build stages get cancelled freely.

- [ ] A workflow deploys fine when you push directly, but fails with "credentials not found" on pull requests from forks. Why?

IF ASKED, SAY THIS
By design — GitHub does not expose repository secrets (or grant write-scoped GITHUB_TOKEN permissions, or allow OIDC id-token: write) to workflows triggered by a pull_request event from a fork, precisely because a fork's workflow file could be attacker-controlled. This is a security feature, not a

By design — GitHub does not expose repository secrets (or grant write-scoped `GITHUB_TOKEN` permissions, or allow OIDC `id-token: write`) to workflows triggered by a `pull_request` event from a fork, precisely because a fork's workflow file could be attacker-controlled. This is a security feature, not a bug to route around casually.

If forked-PR contributions genuinely need CI to run against real credentials (e.g. deploying a preview environment), the sanctioned pattern is splitting the workflow: a `pull_request` -triggered job with no secrets does build/lint/unit-test only, and a separate, more carefully gated `pull_request_target` or `workflow_run` -triggered workflow — which runs in the base repo's context with secrets, but should checkout and execute as little of the untrusted fork's code as possible — handles anything that needs real credentials, often behind a required-reviewer approval first.

- [ ] How do you store Kubernetes and AWS credentials securely in GitHub Actions?

IF ASKED, SAY THIS
Best practice — OIDC with IAM Role (no static credentials): GitHub Actions supports OIDC (OpenID Connect). GitHub acts as an identity provider. When the workflow runs, GitHub provides a short-lived OIDC token. AWS STS exchanges this token for temporary credentials to assume a specific IAM role. Setu

Best practice — OIDC with IAM Role (no static credentials):

GitHub Actions supports OIDC (OpenID Connect). GitHub acts as an identity provider. When the workflow runs, GitHub provides a short-lived OIDC token. AWS STS exchanges this token for temporary credentials to assume a specific IAM role.

Setup:
1. Create an IAM OIDC identity provider in AWS pointing to token.actions.githubusercontent.com
2. Create an IAM role with trust policy restricting to your specific repo and branch
3. In the workflow: permissions: id-token: write and use configure-aws-credentials action with role-to-assume

Trust policy condition:
`"Condition": {"StringLike": {"token.actions.githubusercontent.com:sub": "repo:myorg/myrepo:ref:refs/heads/main"}}`

Why this is better than GitHub Secrets with access keys:
No static credentials that can be rotated incorrectly or leaked. Credentials last 15 minutes (OIDC token lifetime). Each workflow run gets fresh credentials. Full audit trail in CloudTrail.

- [ ] How do you deploy to multiple environments (dev, staging, prod) from one GitHub Actions workflow?

IF ASKED, SAY THIS
Environment-based deployment with approvals: GitHub Actions supports Environments with required reviewers. Define dev, staging, prod environments in GitHub repo settings. Add protection rules: prod requires 2 approvers. Workflow structure: jobs: deploy-dev: environment: dev ... deploy-st

Environment-based deployment with approvals:

GitHub Actions supports Environments with required reviewers. Define dev, staging, prod environments in GitHub repo settings. Add protection rules: prod requires 2 approvers.

Workflow structure:
`jobs: deploy-dev: environment: dev ... deploy-staging: environment: staging needs: [deploy-dev] ... deploy-prod: environment: prod # Requires manual approval needs: [deploy-staging] # Only runs if staging succeeded`

Environment-specific values via GitHub Secrets and Variables:
Each environment (dev, staging, prod) has its own secrets: DB_HOST, AWS_ROLE_ARN, EKS_CLUSTER_NAME.
The same workflow file works for all environments — the secrets and variables differ.

Helm values approach:
`helm upgrade --install user-service ./charts \ --values ./charts/values-${{ matrix.environment }}.yaml`

- [ ] What happens if the Kubernetes rollout fails mid-way through? How does the pipeline handle it?

IF ASKED, SAY THIS
kubectl rollout status is the gate: kubectl rollout status deployment/user-service -n production --timeout=5m This command blocks until: all pods are Running and Ready (exit 0) or timeout is reached (exit 1). On timeout/failure the pipeline step exits non-zero → subsequent steps do not run → GitHub

kubectl rollout status is the gate:
`kubectl rollout status deployment/user-service -n production --timeout=5m`
This command blocks until: all pods are Running and Ready (exit 0) or timeout is reached (exit 1).

On timeout/failure the pipeline step exits non-zero → subsequent steps do not run → GitHub Actions marks the job as failed → Slack notification fires with failure.

But K8s still has half-deployed pods. How do you handle this?
Add an automatic rollback step in the pipeline:
`- name: Rollback on Failure if: failure() run: | kubectl rollout undo deployment/user-service -n production echo "Rolled back to previous version"`

What K8s does internally during a failed rollout:
If maxUnavailable=0 is set, old pods keep serving traffic while new pods fail to become Ready. The old deployment is still functional. K8s does NOT auto-rollback — it waits. Your pipeline's rollback step triggers the explicit undo.

Kubernetes rollback vs pipeline rollback:
kubectl rollout undo: reverts to the previous ReplicaSet. Fast (seconds). Uses the stored previous manifest.
Pipeline re-run with old image tag: re-executes the full pipeline with a previous commit SHA. Slower but provides a full audit trail in CI.

- [ ] Explain Readiness Probe vs Liveness Probe. What happens if you get them wrong?

IF ASKED, SAY THIS
Readiness Probe — controls TRAFFIC: If readiness probe fails: pod is removed from Service endpoints. Load balancer stops sending traffic to it. Pod keeps running. Container is NOT restarted. Use for: checking if the app is fully initialised and ready to serve requests (DB connections established, ca

Readiness Probe — controls TRAFFIC:
If readiness probe fails: pod is removed from Service endpoints. Load balancer stops sending traffic to it. Pod keeps running. Container is NOT restarted.
Use for: checking if the app is fully initialised and ready to serve requests (DB connections established, caches warmed).

Liveness Probe — controls CONTAINER RESTART:
If liveness probe fails N times (failureThreshold): kubelet kills and restarts the container. Pod stays but container is restarted (restart count increments).
Use for: detecting deadlocks or states where the app is running but cannot recover without a restart.

What happens if you get them wrong:
Wrong 1 — Liveness probe too aggressive (short initialDelaySeconds): App starts, JVM takes 45 seconds to load. Liveness probe fires after 10 seconds, sees the app not responding, kills the container. CrashLoopBackOff. App never starts. Fix: use startupProbe for slow-starting apps.

Wrong 2 — Readiness probe on same path as liveness: If a dependency (downstream service) is slow, readiness probe fails → pod removed from load balancer. But pod is healthy. Now you have zero available pods even though all containers are running. Fix: readiness probe should only check local health, not downstream dependencies.

Wrong 3 — No probes at all: Pod becomes Ready immediately after container starts, before app is initialised. Load balancer sends traffic to a pod that cannot handle it yet. Users get 503s during every deployment. Fix: always set readiness probe with initialDelaySeconds appropriate for your app startup time.

CI/CD

- [ ] What's the actual difference between a CI tool like Jenkins and a CD/deployment governance tool like Harness or ArgoCD?

> **IF ASKED, SAY THIS**
> Jenkins (or GitHub Actions) builds and tests your image — that's Continuous Integration. Harness/ArgoCD take over from there and govern the deployment itself: progressive rollout, automated verification against metrics, change-management integration, and automatic rollback — no human has to babysit a dashboard after every deploy.

Conflating the two is a common junior mistake: a green Jenkins build only proves the code compiles and unit tests pass, it says nothing about whether the deployment to production actually succeeded or is healthy. That's the job of the CD layer.

- [ ] Explain Harness Continuous Verification (CV). How is it different from just watching Grafana after a deploy?

> **IF ASKED, SAY THIS**
> Continuous Verification is what makes Harness worth the cost in regulated environments: What CV actually does: 1. Before the deployment, CV establishes a baseline by analysing the last N deployments and their metrics (error rate, P99 latency, log patterns) 2. During the canary phase, CV queries your

Continuous Verification is what makes Harness worth the cost in regulated environments:

What CV actually does:
1. Before the deployment, CV establishes a baseline by analysing the last N deployments and their metrics (error rate, P99 latency, log patterns)
2. During the canary phase, CV queries your observability tools (Splunk, Prometheus, New Relic, Datadog) every minute
3. It uses ML-based anomaly detection to compare canary metrics against the baseline
4. If metrics are within acceptable deviation, CV marks the step as PASSED. If anomaly detected, it fails.
5. On failure, CV automatically triggers the rollback and closes the ServiceNow CHG as Cancelled

Why it is different from watching Grafana manually:
- Manual watching requires an engineer to be available and alert during every deployment - not scalable
- Manual watching is subjective - "is this error spike normal or concerning?" requires experience
- CV is automated, consistent, and documented. Every verification result is stored as evidence
- CV fires the rollback in 2 minutes. A human checking Grafana might take 10 minutes to notice and act
- In financial services: CV output becomes part of the change record evidence for SOX audit

CV data sources Harness supports:
Splunk, Prometheus, New Relic, Datadog, AppDynamics, Dynatrace, Stackdriver, CloudWatch

Interview tip: Mention that CV uses both metrics AND log analysis. A deployment that does not increase error rate but starts logging a new exception pattern is also caught by CV log anomaly detection.

- [ ] A Harness deployment is stuck waiting for ServiceNow approval for 3 hours. How do you investigate and handle this?

> **IF ASKED, SAY THIS**
> This is a real production scenario that happens when ITSM and CI/CD are tightly integrated: Step 1 - Check Harness pipeline execution status: Go to the Harness pipeline execution, click on the ServiceNow Approval step. It shows the CHG number, current status, and which approver it is waiting on. Ste

This is a real production scenario that happens when ITSM and CI/CD are tightly integrated:

Step 1 - Check Harness pipeline execution status:
Go to the Harness pipeline execution, click on the ServiceNow Approval step. It shows the CHG number, current status, and which approver it is waiting on.

Step 2 - Check ServiceNow directly:
Find the CHG in ServiceNow. Common reasons for delay:
- CHG is in the wrong approval group's queue (routing rule misconfigured)
- The approver is on leave and there is no backup configured
- CHG was auto-assigned to a generic group and nobody picked it up
- The risk score calculation flagged it as high risk requiring full CAB (next CAB meeting is tomorrow)
- Change freeze is in effect (quarter-end, major event)

Step 3 - Escalation options:
If business-critical: raise an Emergency Change in ServiceNow. ECAB can approve in 15 minutes. The Harness pipeline will detect the CHG status change and proceed.

If not urgent: override the Harness gate manually with leadership approval. Harness logs who overrode and when (audit trail preserved).

Step 4 - Prevent recurrence:
- Configure backup approvers in ServiceNow approval rules
- Set auto-escalation: if not approved in 2 hours, escalate to the manager
- Classify low-risk deployments as Standard Changes (pre-approved, no manual gate)
- Build a ServiceNow report showing approval cycle time - expose the bottleneck to leadership

- [ ] How do you manage secrets in Harness pipelines across 50 microservices in a financial environment?

> **IF ASKED, SAY THIS**

Centralised secret management is a governance requirement in financial services: Harness Secrets Manager architecture: 1. Harness delegates (agents running in K8s) authenticate to your secret store using instance IAM roles - no static credentials 2. Supported backends: AWS Secrets Manager, HashiCorp

Centralised secret management is a governance requirement in financial services:

Harness Secrets Manager architecture:
1. Harness delegates (agents running in K8s) authenticate to your secret store using instance IAM roles - no static credentials
2. Supported backends: AWS Secrets Manager, HashiCorp Vault, Azure Key Vault, GCP Secret Manager
3. Secrets are referenced by path in the pipeline YAML - the actual value is never stored in Harness
4. Harness masks secret values in all execution logs automatically

For 50 microservices - hierarchy strategy:
- Global secrets: ECR credentials, SonarQube token, Slack webhook → org-level secret in Harness
- Service-specific secrets: DB passwords, API keys → project-level secrets scoped to each service
- Environment-specific: dev secrets vs prod secrets → separate secret stores or paths per environment

Secret rotation in production:
- AWS Secrets Manager rotates DB credentials automatically every 30 days
- Harness picks up the new value on the next pipeline run without any manual change
- Rotation events trigger a Harness webhook that runs a pipeline to restart pods with new credentials

Audit trail for financial compliance:
- Every time Harness accesses a secret, AWS CloudTrail records it
- Harness logs which pipeline step accessed which secret reference
- Together: full trail of who deployed what with which credentials at what time
- This satisfies SOX control for privileged access to production systems

- [ ] What is Harness and how does it compare to Jenkins and GitHub Actions?

IF ASKED, SAY THIS
Harness is an enterprise CI/CD platform with built-in deployment governance. Key differentiators: Continuous Verification (CV) - Queries Splunk or Grafana after every deploy - Auto-rolls back if metrics degrade vs baseline Built-in deployment strategies - Canary, Blue-Green, Rolling with automatic r

Harness is an enterprise CI/CD platform with built-in deployment governance.

Key differentiators:
Continuous Verification (CV)
- Queries Splunk or Grafana after every deploy
- Auto-rolls back if metrics degrade vs baseline

Built-in deployment strategies
- Canary, Blue-Green, Rolling with automatic rollback

ServiceNow integration
- Auto-creates and closes CHG records per deployment

Enterprise RBAC and audit trail

vs Jenkins
Jenkins needs heavy plugin management and infra ops overhead.
Harness is enterprise-ready out of the box.

vs GitHub Actions
Actions suits open-source and small teams.
Harness adds deployment governance, CV, and financial-grade approval gates.

Real-world benefit:
In financial services, Harness + ServiceNow means every prod deployment
auto-creates a CHG, gets approved, deploys, and closes the CHG.
Full compliance evidence with zero manual effort.

- [ ] How do you implement a canary deployment in Harness?

IF ASKED, SAY THIS
Pipeline stages: Stage 1 K8sCanaryDeploy Deploy new version to 10% of pods (1 replica). Stage 2 Verify (Continuous Verification) Harness queries Splunk and Prometheus for 15 minutes. Compares error rate and P99 latency vs the baseline window. If anomaly detected -> automatic rollback. Stage 3 K8

Pipeline stages:

Stage 1 K8sCanaryDeploy
Deploy new version to 10% of pods (1 replica).

Stage 2 Verify (Continuous Verification)
Harness queries Splunk and Prometheus for 15 minutes.
Compares error rate and P99 latency vs the baseline window.
If anomaly detected -> automatic rollback.

Stage 3 K8sCanaryDelete + K8sRollingDeploy
Remove canary pods and roll out to 100%.

Harness YAML:
execution:
steps:
- step:
type: K8sCanaryDeploy
spec:
instanceSelection:
type: Count
spec:
count: 1
- step:
type: Verify
- step:
type: K8sCanaryDelete
- step:
type: K8sRollingDeploy

Real-world tip:
Set the CV baseline window to match your traffic pattern.
For a payments service: compare against the same 15 min window
from the previous day to avoid false alerts on traffic spikes.

- [ ] A deployment went out and Harness CV flagged it. What happens next?

IF ASKED, SAY THIS
Real-time scenario - this is what the CV workflow looks like in practice: Step 1 CV fires Harness detects metric anomaly during the canary phase. Example: error rate jumped from 0.1% to 2.3% on the canary pods. Harness marks the Verify step as FAILED. Step 2 Automatic rollback triggers Harness exe

Real-time scenario - this is what the CV workflow looks like in practice:

Step 1 CV fires
Harness detects metric anomaly during the canary phase.
Example: error rate jumped from 0.1% to 2.3% on the canary pods.
Harness marks the Verify step as FAILED.

Step 2 Automatic rollback triggers
Harness executes K8sCanaryDelete to remove canary pods.
Traffic returns fully to stable version.
PagerDuty alert fires: 'Deployment rollback triggered by CV'.

Step 3 ServiceNow CHG auto-updated
CHG status set to CANCELLED.
Rollback details and CV metric data attached as evidence.

Step 4 SRE investigates
Compare metrics between canary pods and baseline pods in Grafana.
Check Splunk for error logs only on the canary pod IPs.
Review the artifact diff: what changed in this build vs the previous one?

Step 5 Fix and redeploy
Developer fixes the root cause.
New build goes through full pipeline: build -> scan -> dev -> staging -> CV canary -> prod.
New CHG created for the fix deployment.

Key value: rollback happened in under 2 minutes with zero manual intervention.

- [ ] How do you integrate Harness with ServiceNow for change management?

IF ASKED, SAY THIS
Integration flow: 1. Pipeline reaches the prod stage. 2. Harness creates a ServiceNow CHG record automatically. Contains: artifact version, deployment details, approver list, rollback plan. 3. ServiceNow routes the CHG to the CAB or change approver. 4. On approval, the Harness pipeline gate opens au

Integration flow:

1. Pipeline reaches the prod stage.
2. Harness creates a ServiceNow CHG record automatically.
Contains: artifact version, deployment details, approver list, rollback plan.
3. ServiceNow routes the CHG to the CAB or change approver.
4. On approval, the Harness pipeline gate opens automatically.
5. Post-deploy: Harness updates CHG status.
Success -> Implemented
Failure -> Cancelled with rollback notes attached

Real-world benefit:
- Every prod change has a linked CHG (full audit trail)
- Zero manual ticket creation means zero human error
- Satisfies SOX and PCI change evidence requirements automatically
- During a CAB audit, the CHG history in ServiceNow IS the deployment history

- [ ] How do you handle a hotfix deployment at 2am during a P1?

> **IF ASKED, SAY THIS**
> Real-time scenario: 1. P1 declared, root cause identified, hotfix code ready. 2. Developer raises an Emergency CHG in ServiceNow. ECAB approval via phone: 2 approvers in 5 minutes. 3. Harness emergency pipeline triggered: - Skips standard UAT gate (time critical) - Runs unit tests and Trivy scan onl

Real-time scenario:

1. P1 declared, root cause identified, hotfix code ready.
2. Developer raises an Emergency CHG in ServiceNow.
ECAB approval via phone: 2 approvers in 5 minutes.
3. Harness emergency pipeline triggered:
- Skips standard UAT gate (time critical)
- Runs unit tests and Trivy scan only (non-negotiable minimums)
- Deploys via rolling update (no canary at 2am - traffic is low)
4. SRE monitors Grafana in real time during the deploy.
Eyes on error rate and latency for 10 minutes post-deploy.
5. Error rate returns to baseline -> incident resolved.
6. ServiceNow INC and CHG both closed with evidence attached.
7. Next business day:
- Blameless RCA posted to team
- Proper fix with tests merged and deployed via normal pipeline
- PRB raised to address the root cause permanently

Key principle:
Even at 2am the audit trail must be complete.
Harness logs the artifact version, deploy time, and approver identity.
This is what financial regulators look for.

- [ ] Explain GitOps and how ArgoCD implements it. What is the pull model and why is it more secure than a push model?

> **IF ASKED, SAY THIS**
> GitOps is the practice of using Git as the single source of truth for both application code AND infrastructure state: The core principle: The desired state of your production cluster is fully described in Git. Any change to production is a Git commit. The cluster continuously reconciles itself to ma

GitOps is the practice of using Git as the single source of truth for both application code AND infrastructure state:

The core principle: The desired state of your production cluster is fully described in Git. Any change to production is a Git commit. The cluster continuously reconciles itself to match what is in Git.

Push model (traditional CI/CD, Jenkins/Harness):
Pipeline runs → pipeline has credentials to access the cluster → pipeline pushes changes to cluster
Security risk: Your CI system needs cluster credentials stored as secrets. If the CI system is compromised, an attacker can deploy anything to production.

Pull model (ArgoCD/Flux):
ArgoCD agent runs INSIDE the cluster → agent watches the Git repo → agent pulls and applies changes
Security advantage: No cluster credentials stored in CI. The CI pipeline only has permission to write to Git. ArgoCD running inside the cluster has cluster-admin, but that is normal for a workload running in the cluster. CI never touches the cluster directly.

ArgoCD reconciliation loop:
Every 3 minutes (default): ArgoCD compares the live cluster state to the desired state in Git
If different (drift): ArgoCD either auto-syncs (if configured) or shows OutOfSync status in UI
If auto-sync: ArgoCD applies the change, records who made the Git commit, and stores the diff in the application history

Full audit trail: Every production change is a Git commit with author, timestamp, and commit message. This is perfect for SOX audit: "who changed production and when?" - just check Git log.

- [ ] What is the App of Apps pattern in ArgoCD and when do you use it?

> **IF ASKED, SAY THIS**
> The App of Apps pattern solves multi-cluster and multi-service management at scale: The problem without App of Apps: You have 30 microservices across 3 clusters. That is 90 ArgoCD Application CRDs to manage manually. When you add a new environment (staging-asia), you have to create 30 more Applicati

The App of Apps pattern solves multi-cluster and multi-service management at scale:

The problem without App of Apps:
You have 30 microservices across 3 clusters. That is 90 ArgoCD Application CRDs to manage manually. When you add a new environment (staging-asia), you have to create 30 more Application CRDs by hand. Error-prone and does not scale.

App of Apps solution:
Create one root ArgoCD Application that points to a Git directory containing other Application manifests. When ArgoCD syncs the root app, it creates and manages all the child Applications automatically.

Directory structure:
`apps/ root-app.yaml # ArgoCD watches this dir payments-service.yaml # ArgoCD Application for payments auth-service.yaml # ArgoCD Application for auth fraud-service.yaml # etc. services/ payments/ # actual Helm chart values auth/`

Adding a new service: Just add a new Application YAML to apps/ directory. ArgoCD picks it up automatically on the next sync. No manual ArgoCD

UI work needed.

Adding a new cluster: Point a new ArgoCD instance at the root apps/ directory with a different values file. All 30 services are deployed to the new cluster automatically.

Ordering with sync waves: Use `argocd.argoproj.io/sync-wave: "1"` annotations to control deployment order. Deploy the database before the application. Deploy the ConfigMap before the Deployment.

- [ ] How do you handle secrets in GitOps? You cannot commit secrets to Git but ArgoCD needs them in the cluster.

IF ASKED, SAY THIS
This is the fundamental GitOps security challenge. Three approaches used in production: Approach 1 - Sealed Secrets (Bitnami): - SealedSecrets controller runs in the cluster. It has a private key. - You encrypt the secret with the public key: kubeseal --format yaml secret.yaml > sealed-secret.yaml -

This is the fundamental GitOps security challenge. Three approaches used in production:

Approach 1 - Sealed Secrets (Bitnami):
- SealedSecrets controller runs in the cluster. It has a private key.
- You encrypt the secret with the public key: `kubeseal --format yaml secret.yaml > sealed-secret.yaml`
- Commit the SealedSecret (encrypted blob) to Git safely
- ArgoCD deploys the SealedSecret. The controller decrypts it inside the cluster.
- Risk: if you lose the private key, all sealed secrets are unrecoverable. Backup the key.

Approach 2 - External Secrets Operator (recommended):
- Secrets stay in AWS Secrets Manager / Azure Key Vault / HashiCorp Vault
- You commit an ExternalSecret CRD to Git: "create a K8s Secret from this path in Secrets Manager"
- External Secrets Operator reads from the secret store and creates the K8s Secret in the cluster
- Rotation: when the secret rotates in Secrets Manager, ESO automatically updates the K8s Secret
- Nothing sensitive in Git. The ExternalSecret CRD is just a pointer, not the value.

Approach 3 - Vault Agent Injector:
- HashiCorp Vault runs separately
- Vault Agent sidecar is injected into pods
- The sidecar authenticates to Vault using the pod's service account token
- Vault injects secrets directly into the pod's filesystem (not as K8s Secrets)

Production recommendation: External Secrets Operator with AWS Secrets Manager or Azure Key Vault. It is the cleanest approach: secrets stay in a managed secret store, rotation is automatic, and nothing sensitive ever touches Git.

- [ ] How do you store Kubernetes and AWS credentials securely in GitHub Actions?

IF ASKED, SAY THIS
Best practice — OIDC with IAM Role (no static credentials): GitHub Actions supports OIDC (OpenID Connect). GitHub acts as an identity provider. When the workflow runs, GitHub provides a short-lived OIDC token. AWS STS exchanges this token for temporary credentials to assume a specific IAM role. Setu

Best practice — OIDC with IAM Role (no static credentials):

GitHub Actions supports OIDC (OpenID Connect). GitHub acts as an identity provider. When the workflow runs, GitHub provides a short-lived OIDC token. AWS STS exchanges this token for temporary credentials to assume a specific IAM role.

Setup:
1. Create an IAM OIDC identity provider in AWS pointing to token.actions.githubusercontent.com
2. Create an IAM role with trust policy restricting to your specific repo and branch
3. In the workflow: permissions: id-token: write and use configure-aws-credentials action with role-to-assume

Trust policy condition:
`"Condition": {"StringLike": {"token.actions.githubusercontent.com:sub": "repo:myorg/myrepo:ref:refs/heads/main"}}`

Why this is better than GitHub Secrets with access keys:
No static credentials that can be rotated incorrectly or leaked. Credentials last 15 minutes (OIDC token lifetime). Each workflow run gets fresh credentials. Full audit trail in CloudTrail.

- [ ] How do you deploy to multiple environments (dev, staging, prod) from one GitHub Actions workflow?

IF ASKED, SAY THIS
Environment-based deployment with approvals: GitHub Actions supports Environments with required reviewers. Define dev, staging, prod environments in GitHub repo settings. Add protection rules: prod requires 2 approvers. Workflow structure: jobs: deploy-dev: environment: dev ... deploy-st

Environment-based deployment with approvals:

GitHub Actions supports Environments with required reviewers. Define dev, staging, prod environments in GitHub repo settings. Add protection rules: prod requires 2 approvers.

Workflow structure:
`jobs: deploy-dev: environment: dev ... deploy-staging: environment: staging needs: [deploy-dev] ... deploy-prod: environment: prod # Requires manual approval needs: [deploy-staging] # Only runs if staging succeeded`

Environment-specific values via GitHub Secrets and Variables:
Each environment (dev, staging, prod) has its own secrets: DB_HOST, AWS_ROLE_ARN, EKS_CLUSTER_NAME.
The same workflow file works for all environments — the secrets and variables differ.

Helm values approach:
`helm upgrade --install user-service ./charts \ --values ./charts/values-${{ matrix.environment }}.yaml`

- [ ] What happens if the Kubernetes rollout fails mid-way through? How does the pipeline handle it?

> **IF ASKED, SAY THIS**
> kubectl rollout status is the gate: kubectl rollout status deployment/user-service -n production --timeout=5m This command blocks until: all pods are Running and Ready (exit 0) or timeout is reached (exit 1). On timeout/failure the pipeline step exits non-zero → subsequent steps do not run → GitHub

kubectl rollout status is the gate:
`kubectl rollout status deployment/user-service -n production --timeout=5m`
This command blocks until: all pods are Running and Ready (exit 0) or timeout is reached (exit 1).

On timeout/failure the pipeline step exits non-zero → subsequent steps do not run → GitHub Actions marks the job as failed → Slack notification fires with failure.

But K8s still has half-deployed pods. How do you handle this?
Add an automatic rollback step in the pipeline:
`- name: Rollback on Failure if: failure() run: | kubectl rollout undo deployment/user-service -n production echo "Rolled back to previous version"`

What K8s does internally during a failed rollout:
If maxUnavailable=0 is set, old pods keep serving traffic while new pods fail to become Ready. The old deployment is still functional. K8s does NOT auto-rollback — it waits. Your pipeline's rollback step triggers the explicit undo.

Kubernetes rollback vs pipeline rollback:
kubectl rollout undo: reverts to the previous ReplicaSet. Fast (seconds). Uses the stored previous manifest.
Pipeline re-run with old image tag: re-executes the full pipeline with a previous commit SHA. Slower but provides a full audit trail in CI.

- [ ] Explain Readiness Probe vs Liveness Probe. What happens if you get them wrong?

> **IF ASKED, SAY THIS**
> Readiness Probe — controls TRAFFIC: If readiness probe fails: pod is removed from Service endpoints. Load balancer stops sending traffic to it. Pod keeps running. Container is NOT restarted. Use for: checking if the app is fully initialised and ready to serve requests (DB connections established, ca

Readiness Probe — controls TRAFFIC:
If readiness probe fails: pod is removed from Service endpoints. Load balancer stops sending traffic to it. Pod keeps running. Container is NOT restarted.
Use for: checking if the app is fully initialised and ready to serve requests (DB connections established, caches warmed).

Liveness Probe — controls CONTAINER RESTART:
If liveness probe fails N times (failureThreshold): kubelet kills and restarts the container. Pod stays but container is restarted (restart count increments).
Use for: detecting deadlocks or states where the app is running but cannot recover without a restart.

What happens if you get them wrong:
Wrong 1 — Liveness probe too aggressive (short initialDelaySeconds): App starts, JVM takes 45 seconds to load. Liveness probe fires after 10 seconds, sees the app not responding, kills the container. CrashLoopBackOff. App never starts. Fix: use startupProbe for slow-starting apps.

Wrong 2 — Readiness probe on same path as liveness: If a dependency (downstream service) is slow, readiness probe fails → pod removed from load balancer. But pod is healthy. Now you have zero available pods even though all containers are running. Fix: readiness probe should only check local health, not downstream dependencies.

Wrong 3 — No probes at all: Pod becomes Ready immediately after container starts, before app is initialised. Load balancer sends traffic to a pod that cannot handle it yet. Users get 503s during every deployment. Fix: always set readiness probe with initialDelaySeconds appropriate for your app startup time.

- [ ] What does 'build once, deploy many' mean, and why does it matter?

> **IF ASKED, SAY THIS**
> It means the artifact that gets tested is the exact same binary/image/package that gets promoted through every environment — build in CI once, push it to an artifact repository (Artifactory, in this JD), then each environment's deployment stage pulls that same immutable artifact by version/tag rathe

It means the artifact that gets tested is the exact same binary/image/package that gets promoted through every environment — build in CI once, push it to an artifact repository (Artifactory, in this JD), then each environment's deployment stage pulls that same immutable artifact by version/tag rather than rebuilding from source per environment. Rebuilding per environment risks subtle drift (a different dependency version resolves, a different base image gets pulled) so what passed staging isn't provably what's running in prod. The pattern also makes rollback trivial — redeploy the previous artifact version instead of trying to rebuild an old commit.

- [ ] How do you keep a CI/CD pipeline fast without sacrificing safety?

> **IF ASKED, SAY THIS**

Parallelize what's independent — unit test suites split across executors, lint/security-scan/unit-test stages run concurrently rather than serially. Fail fast — cheap, fast checks (lint, unit tests) run before expensive ones (integration tests, full build), so a broken build is caught in seconds, no

Parallelize what's independent — unit test suites split across executors, lint/security-scan/unit-test stages run concurrently rather than serially. Fail fast — cheap, fast checks (lint, unit tests) run before expensive ones (integration tests, full build), so a broken build is caught in seconds, not after a 20-minute integration suite. Cache aggressively — dependency caches (npm/pip/maven) and Docker layer caching cut redundant work on every run. And gate strategically rather than gate everything — not every check needs to block merge; some can run post-merge or nightly if they're slow and lower-signal, keeping the PR feedback loop tight while still running the full suite before a production deploy.

- [ ] Where does security fit into a CI/CD pipeline you own?

IF ASKED, SAY THIS
At minimum: secrets never live in the pipeline definition or environment in plaintext (pulled from a vault at runtime), the pipeline's own service account/IAM role is scoped to only what that pipeline needs to deploy — not broad account access, and dependency/container scanning runs as an actual gat

At minimum: secrets never live in the pipeline definition or environment in plaintext (pulled from a vault at runtime), the pipeline's own service account/IAM role is scoped to only what that pipeline needs to deploy — not broad account access, and dependency/container scanning runs as an actual gate, not just a report nobody reads. For supply-chain concerns, that extends to knowing what went into a build (an SBOM) and, in more mature setups, signing artifacts so a deployment target can verify what it's running actually came from your pipeline and hasn't been tampered with in transit through the artifact repo. The goal is treating the pipeline itself as production infrastructure worth securing, not just the thing that deploys production infrastructure.

- [ ] Explain Blue-Green, Canary, and Rolling deployments, and how you'd implement one in a pipeline.

IF ASKED, SAY THIS
Rolling replaces instances/tasks in small batches, so at any moment old and new versions are both serving traffic — no extra infrastructure needed, but rollback means rolling forward again, and a bad version isn't fully isolated during the rollout. **Blue-Green** runs two complete environments;

Rolling replaces instances/tasks in small batches, so at any moment old and new versions are both serving traffic — no extra infrastructure needed, but rollback means rolling forward again, and a bad version isn't fully isolated during the rollout. **Blue-Green** runs two complete environments; traffic cuts over all at once (e.g. swapping which target group an ALB listener points to), which gives an instant rollback (flip back) at the cost of running double the infrastructure briefly. **Canary** shifts a small percentage of traffic to the new version first, watches error rate/latency, then ramps up — the best blast-radius control of the three, at the cost of needing weighted routing (ALB weighted target groups, or a service mesh) and good enough monitoring to actually trust the canary signal before promoting. In a pipeline, the deploy stage typically drives this through the platform's native primitive — ECS's own deployment configuration for rolling, or a scripted target-group swap for blue-green — gated on a health check or a manual approval step before full cutover.

- [ ] Walk through a CI/CD pipeline you'd design for deploying to EKS/ECS from a git push to production.

IF ASKED, SAY THIS
Push → CI stage: lint, unit tests, build image, tag with commit SHA (never :latest), push to ECR, run a vulnerability scan gate. CD stage: deploy to a dev/staging environment automatically, run smoke/integration tests, then a gated promotion to production — either an approval step or automated if y

Push → CI stage: lint, unit tests, build image, tag with commit SHA (never `:latest`), push to ECR, run a vulnerability scan gate. CD stage: deploy to a dev/staging environment automatically, run smoke/integration tests, then a gated promotion to production — either an approval step or automated if you trust your test suite and are running progressive delivery. For EKS, that's typically GitOps (Argo CD reconciling a manifest repo) rather than the pipeline pushing directly; for ECS, the pipeline itself usually calls `update-service` with the new task definition revision, ideally via CodeDeploy for a blue/green cutover with automatic rollback tied to CloudWatch alarms.

- [ ] Continuous delivery vs continuous deployment — how do you decide which to run?

IF ASKED, SAY THIS
Continuous delivery means every change is proven deployable and sits ready behind a manual trigger; continuous deployment means it ships automatically once it passes the pipeline, no human gate. The decision comes down to test-suite confidence and blast radius: a well-covered, well-observed service

Continuous delivery means every change is proven deployable and sits ready behind a manual trigger; continuous deployment means it ships automatically once it passes the pipeline, no human gate. The decision comes down to test-suite confidence and blast radius: a well-covered, well-observed service with fast rollback and good canary/error-budget signals can run full continuous deployment safely. A regulated, high-blast-radius, or under-tested service should keep a manual gate — the cost of a bad manual approval delay is much lower than the cost of an unreviewed change hitting a regulated production system.

- [ ] What's infrastructure drift, and how do you detect and remediate it?

IF ASKED, SAY THIS
Drift is when real infrastructure state diverges from what your IaC says it should be — someone made a manual console change, or an out-of-band process modified a resource. Detection: scheduled terraform plan runs (or drift-detection features in TFC/Terraform Cloud, or tools like driftctl) that diff

Drift is when real infrastructure state diverges from what your IaC says it should be — someone made a manual console change, or an out-of-band process modified a resource. Detection: scheduled `terraform plan` runs (or drift-detection features in TFC/Terraform Cloud, or tools like driftctl) that diff actual vs. desired state and alert on any non-empty plan outside of a real deploy window. Remediation is a policy decision, not just a technical

one — either re-apply Terraform to force state back (risky if the manual change was a legitimate emergency fix), or import the drifted state into Terraform and treat it as the new baseline. The real fix is process: lock down console write access so drift can't happen in the first place.

- [ ] Explain GitOps and how it differs from a traditional push-based pipeline.

IF ASKED, SAY THIS
In push-based CI/CD, the pipeline has credentials to the cluster and actively pushes changes out. In GitOps, git is the single source of truth for desired state, and an in-cluster agent (Argo CD, Flux) continuously pulls and reconciles actual state to match it — the cluster has no inbound deploy cre

In push-based CI/CD, the pipeline has credentials to the cluster and actively pushes changes out. In GitOps, git is the single source of truth for desired state, and an in-cluster agent (Argo CD, Flux) continuously pulls and reconciles actual state to match it — the cluster has no inbound deploy credentials to protect, and drift gets auto-corrected instead of silently accumulating. It also gives you a full audit trail for free (every change is a git commit) and trivial rollback (revert the commit).

- [ ] Trunk-based development vs. GitFlow — what's the tradeoff, and which do you prefer for a team practicing continuous deployment?

IF ASKED, SAY THIS
GitFlow — long-lived develop / release / feature branches, formal release branches cut periodically. Gives structure for scheduled, versioned releases, but branches diverge for a while and merges get bigger and riskier the longer they live.

GitFlow — long-lived `develop` / `release` / `feature` branches, formal release branches cut periodically. Gives structure for scheduled, versioned releases, but branches diverge for a while and merges get bigger and riskier the longer they live.

Trunk-based — everyone merges small, frequent changes into `main`, behind feature flags if the change isn't ready for users yet. It's what continuous deployment actually requires, because it keeps main always deployable and avoids the integration pain of long-lived branches. For a CD-practicing team I prefer trunk-based, with feature flags doing the job that release branches used to do — decoupling “code is merged” from “feature is visible to users.”

- [ ] Explain semantic versioning and how you'd automate version bumps and changelog generation in a pipeline.

IF ASKED, SAY THIS
SemVer is MAJOR.MINOR.PATCH — major for breaking changes, minor for backward-compatible new features, patch for backward-compatible fixes. The value is that a version number alone tells a consumer whether upgrading is safe to do blindly.

SemVer is `MAJOR.MINOR.PATCH` — major for breaking changes, minor for backward-compatible new features, patch for backward-compatible fixes. The value is that a version number alone tells a consumer whether upgrading is safe to do blindly.

To automate it, I'd adopt Conventional Commits (`feat:` , `fix:` , `BREAKING CHANGE:` prefixes) and run a tool like semantic-release in the pipeline — it parses commit messages since the last release, computes the correct version bump automatically, generates the changelog from those same commit messages, tags the release, and publishes, with zero manual “what version should this be” decision left to a human.

- [ ] What's the difference between a build artifact and a container image in a CI/CD context, and how do you manage versioning/retention?

IF ASKED, SAY THIS
A build artifact is the general output of a build step — a compiled binary, a jar, a zip of static assets — that may or may not end up inside a container. A container image packages that artifact plus its entire runtime environment (OS layer, dependencies, runtime) into one portable, immutable unit

A build artifact is the general output of a build step — a compiled binary, a jar, a zip of static assets — that may or may not end up inside a container. A container image packages that artifact plus its entire runtime environment (OS layer, dependencies, runtime) into one portable, immutable unit — which is why containers deploy so much more predictably across environments than a bare artifact plus a separately-configured host.

Both get versioned by immutable tag — commit SHA or a semver tag, never `latest` in anything that gets deployed — and pushed to a registry (ECR, ACR, Artifactory) with a retention policy: keep everything tagged/released, prune untagged and old feature-branch images on a schedule (e.g. 30/90 days) so the registry doesn't grow unbounded and cost/noise creeps up silently.

- [ ] How would you design a CI/CD pipeline that deploys the same application to both AWS and Azure?

IF ASKED, SAY THIS
Build once, deploy twice — the same container image, built and tested a single time, gets pushed to both ECR and ACR (or a registry both clouds can pull from) so there's no risk of the two clouds ever running genuinely different code. Deployment then branches into two provider-specific jobs: one app

Build once, deploy twice — the same container image, built and tested a single time, gets pushed to both ECR and ACR (or a registry both clouds can pull from) so there's no risk of the two clouds ever running genuinely different code. Deployment then branches into two provider-specific jobs: one applying the EKS/AWS Terraform and doing the AWS-side deploy, one applying the AKS/Azure Terraform and doing the Azure-side deploy — same pipeline, same trigger, same approval gate, parallel provider-specific execution.

The parts that genuinely need to differ — authentication (OIDC to an AWS role vs. Azure workload identity), and any cloud-specific config (load balancer annotations, storage class names) — stay isolated in each provider's job/module rather than leaking conditional logic into the shared application deployment manifest.

- [ ] What Git practices would you enforce for a production infrastructure repo, and why?

IF ASKED, SAY THIS
The underlying reason for all of it: an infra repo isn't just code, it's a direct, automated path to changing production — the review/gate discipline needs to match that, not be looser than application code review just because “it's just config.”

- **Branch protection on main** — no direct pushes, PR required, even for the person who “knows it's fine.”
- **Required reviews** (at least one, ideally from someone other than the author) — catches the mistakes a person can't see in their own change.
- **Required status checks** — `terraform plan` /lint/tests must pass before merge is even allowed, not just recommended.
- **Signed commits** for a genuinely sensitive infra repo — provenance matters when the repo controls production.
- **No force-push to main** — history integrity matters for audit trail, especially in a regulated environment.

The underlying reason for all of it: an infra repo isn't just code, it's a direct, automated path to changing production — the review/gate discipline needs to match that, not be looser than application code review just because “it's just config.”

- [ ] How would you structure a GitLab branching strategy for a platform with multiple Java microservices and frequent releases?

IF ASKED, SAY THIS
Trunk-based development with short-lived feature branches merged via MR into main, protected by required approvals and a passing pipeline — this keeps merge conflicts small and supports continuous deployment. For services needing longer release trains, such as regulated components with staged approv

Trunk-based development with short-lived feature branches merged via MR into main, protected by required approvals and a passing pipeline — this keeps merge conflicts small and supports continuous deployment. For services needing longer release trains, such as regulated components with staged approvals, I'd use release branches cut from main at a tag, with hotfixes cherry-picked back. I avoid long-lived GitFlow-style develop branches on high-velocity services; they accumulate drift and make CI signal stale.

- [ ] Walk me through how you'd design a GitLab CI/CD pipeline template that every Java service reuses.

IF ASKED, SAY THIS
An include-based template — .gitlab-ci.yml using an include directive against a shared project — with standard stages: build with Maven, unit test, static analysis, dependency and container scanning, package, deploy to dev, integration test, deploy to staging with a manual gate, deploy to prod with

An include-based template — .gitlab-ci.yml using an include directive against a shared project — with standard stages: build with Maven, unit test, static analysis, dependency and container scanning, package, deploy to dev, integration test, deploy to staging with a manual gate, deploy to prod with approval. Each service's own file extends the template and supplies service-specific variables, so a platform-wide pipeline improvement rolls out everywhere without every team touching their file.

- [ ] How do you enforce merge request quality gates in GitLab without slowing teams down too much?

IF ASKED, SAY THIS
Required approvals scaled to risk — one approver for low-risk services, two for shared or critical components — branch protection requiring a green pipeline before merge, and automated quality gates like coverage thresholds and scan pass status that block rather than just warn. I keep gates objectiv

Required approvals scaled to risk — one approver for low-risk services, two for shared or critical components — branch protection requiring a green pipeline before merge, and automated quality gates like coverage thresholds and scan pass status that block rather than just warn. I keep gates objective and automated wherever possible; subjective "someone needs to feel good about this" reviews are where MRs get stuck.

- [ ] What GitLab CI/CD features would you use to integrate security scanning into the pipeline?

IF ASKED, SAY THIS
Built-in SAST and Dependency Scanning for code and library vulnerabilities, Container Scanning for the built image, Secret Detection to catch committed credentials before merge, and DAST against a running staging environment for dynamic checks. I'd set severity-based gates — block on critical and hi

Built-in SAST and Dependency Scanning for code and library vulnerabilities, Container Scanning for the built image, Secret Detection to catch committed credentials before merge, and DAST against a running staging environment for dynamic checks. I'd set severity-based gates — block on critical and high, warn on medium — rather than all-or-nothing, since blocking every medium finding usually just trains teams to ignore the pipeline.

- [ ] How do GitLab runners factor into your pipeline architecture, and how would you plan runner capacity for multiple teams?

IF ASKED, SAY THIS
A mix of shared runners for lightweight, common jobs and dedicated project or group runners — often Kubernetes-executor runners for autoscaling — for build-heavy jobs, tagged so pipelines request the right runner class. I'd plan capacity around peak concurrent pipeline load, not average, since CI qu

A mix of shared runners for lightweight, common jobs and dedicated project or group runners — often Kubernetes-executor runners for autoscaling — for build-heavy jobs, tagged so pipelines request the right runner class. I'd plan capacity around peak concurrent pipeline load, not average, since CI queuing right before a release is exactly when teams are least tolerant of delay, and I'd isolate runners handling production credentials from general-purpose shared runners.

- [ ] How would you design rollback in a GitLab CI/CD pipeline?

IF ASKED, SAY THIS
Rollback should be a pipeline job, not a manual scramble — either re-running the last known-good deployment job, or triggering a dedicated rollback stage that redeploys the previous release tag. Every deploy ties to an immutable, addressable artifact such as a container image digest rather than "lat

Rollback should be a pipeline job, not a manual scramble — either re-running the last known-good deployment job, or triggering a dedicated rollback stage that redeploys the previous release tag. Every deploy ties to an immutable, addressable artifact such as a container image digest rather than "latest," so rollback always means redeploying exactly what was running before, never a rebuild.

- [ ] What's your view on manual approval gates in a pipeline — when do you use them versus full automation?

IF ASKED, SAY THIS
Manual gates for production deploys to genuinely high-risk or regulated services, and for anything crossing an environment boundary with real business impact if wrong, such as a schema migration. Elsewhere I push toward automated promotion gated by objective signals — passing tests, canary health ch

Manual gates for production deploys to genuinely high-risk or regulated services, and for anything crossing an environment boundary with real business impact if wrong, such as a schema migration. Elsewhere I push toward automated promotion gated by objective signals — passing tests, canary health checks, SLO adherence — because manual gates that exist "just in case" tend to become rubber-stamp approvals that add latency without adding real safety.

- [ ] How do you handle Maven dependency management and artifact versioning in a GitLab CI/CD pipeline for microservices?

IF ASKED, SAY THIS
A private GitLab Package Registry, or Nexus/Artifactory, as the artifact source of truth, semantic versioning for shared internal libraries, and a policy against snapshot or floating dependencies reaching production builds. Dependency scanning as a pipeline gate catches vulnerable transitive depende

A private GitLab Package Registry, or Nexus/Artifactory, as the artifact source of truth, semantic versioning for shared internal libraries, and a policy against snapshot or floating dependencies reaching production builds. Dependency scanning as a pipeline gate catches vulnerable transitive dependencies before they're baked into an image, and I pin build-tool and base-image versions explicitly rather than "latest" to keep builds reproducible.

- [ ] Compare GitLab CI/CD and Jenkins from an architecture standpoint — when would you recommend one over the other?

IF ASKED, SAY THIS
GitLab CI/CD gives pipeline-as-code tightly integrated with source control, MRs, and built-in security scanning, with less plugin sprawl — I'd favor it for a platform being built fresh or consolidating tooling. Jenkins is more flexible and often deeply embedded in legacy pipelines with custom shared

GitLab CI/CD gives pipeline-as-code tightly integrated with source control, MRs, and built-in security scanning, with less plugin sprawl — I'd favor it for a platform being built fresh or consolidating tooling. Jenkins is more flexible and often deeply embedded in legacy pipelines with custom shared libraries; I'd keep it where migration cost outweighs benefit or a specific plugin ecosystem is load-bearing. For a Java/Spring Boot platform standardizing tooling, I'd lean GitLab unless there's a strong existing Jenkins investment.

- [ ] How would you define release governance across dev, test, staging, pre-prod, and production in GitLab?

IF ASKED, SAY THIS
Environments modeled explicitly in GitLab, each with its own protection rules — who can deploy, whether approval is required, which branches or tags may deploy there. Promotion between environments carries the same artifact forward, never rebuilt per environment, with environment-specific config inj

Environments modeled explicitly in GitLab, each with its own protection rules — who can deploy, whether approval is required, which branches or tags may deploy there. Promotion between environments carries the same artifact forward, never rebuilt per environment, with environment-specific config injected at deploy time. Pre-prod and production gates require the full test and scan suite plus, for production, an explicit approval tied to a change record for auditability.

- [ ] How do you prevent "pipeline as a bottleneck" complaints while still enforcing quality gates?

IF ASKED, SAY THIS
Parallelize what can run in parallel — unit tests, linting, and scans don't need to be sequential — cache dependencies aggressively such as the Maven .m2 cache between runs, and fail fast by putting the cheapest, most likely-to-fail checks first. I also track pipeline duration as a metric with an ow

Parallelize what can run in parallel — unit tests, linting, and scans don't need to be sequential — cache dependencies aggressively such as the Maven .m2 cache between runs, and fail fast by putting the cheapest, most likely-to-fail checks first. I also track pipeline duration as a metric with an owner, the same way I'd track an SLO, so degradation gets caught and fixed instead of silently tolerated.

- [ ] How would you design GitLab CI/CD to support canary or blue-green deployments to Kubernetes?

IF ASKED, SAY THIS
The pipeline builds one immutable image, then a deploy stage applies it via Helm or Argo Rollouts, or a GitLab-native Kubernetes Agent integration, with a canary or blue-green strategy defined declaratively — for example an Argo Rollouts canary step sequence with automated analysis on error rate and

The pipeline builds one immutable image, then a deploy stage applies it via Helm or Argo Rollouts, or a GitLab-native Kubernetes Agent integration, with a canary or blue-green strategy defined declaratively — for example an Argo Rollouts canary step sequence with automated analysis on error rate and latency gating promotion. The pipeline triggers and monitors rollout status; automated rollback fires on a failed analysis step, with a manual override for the on-call engineer.

- [ ] What quality gates would you consider non-negotiable for a Java/Spring Boot service pipeline, and which would you make configurable per team?

IF ASKED, SAY THIS
Non-negotiable: passing unit tests, no critical or high vulnerabilities from dependency and container scanning, and secret detection. Configurable per team or risk tier: code coverage threshold, quality-gate strictness, and whether integration or contract tests are required pre-merge versus pre-depl

Non-negotiable: passing unit tests, no critical or high vulnerabilities from dependency and container scanning, and secret detection. Configurable per team or risk tier: code coverage threshold, quality-gate strictness, and whether integration or contract tests are required pre-merge versus pre-deploy. I keep the configurable list small and documented so it doesn't become a quiet excuse to weaken the pipeline.

- [ ] How do you handle secrets such as DB credentials or API keys in a GitLab CI/CD pipeline securely?

IF ASKED, SAY THIS
Never in the repo or in plain pipeline variables — GitLab CI/CD variables marked protected and masked at minimum, but for anything production-facing I'd integrate an external secrets manager and have the pipeline and runtime fetch secrets dynamically via short-lived tokens. OIDC federation between G

Never in the repo or in plain pipeline variables — GitLab CI/CD variables marked protected and masked at minimum, but for anything production-facing I'd integrate an external secrets manager and have the pipeline and runtime fetch secrets dynamically via short-lived tokens. OIDC federation between GitLab and the cloud provider is ideal, since no long-lived cloud credentials are stored in GitLab at all. Secret detection scanning on every commit catches accidental leaks before they merge.

- [ ] What is JFrog Artifactory, and where does it sit in the release process?

IF ASKED, SAY THIS
Artifactory is a **universal artifact repository manager** — the single place build outputs land after CI produces them, and the single place deployments pull from, for essentially any package type: Maven/Gradle jars, npm packages, PyPI wheels, Docker images, generic binaries, Helm charts. It sits b

Artifactory is a **universal artifact repository manager** — the single place build outputs land after CI produces them, and the single place deployments pull from, for essentially any package type: Maven/Gradle jars, npm packages, PyPI wheels, Docker images, generic binaries, Helm charts. It sits between 'CI just finished building this' and 'this is now deployable' — the artifact repository is what makes 'build once, deploy many' actually possible, since every environment pulls the identical, versioned artifact from one trusted source rather than each environment rebuilding from source or pulling from wherever a developer happens to have it.

- [ ] Explain local, remote, and virtual repositories in Artifactory, and how they combine.

IF ASKED, SAY THIS
A **local repository** holds artifacts your org actually builds and owns — your Docker images, your internal libraries. A **remote repository** is a caching proxy in front of an external repository (Docker Hub, npmjs.org, Maven Central) — the first request for a package fetches and caches it, and ev

A **local repository** holds artifacts your org actually builds and owns — your Docker images, your internal libraries. A **remote repository** is a caching proxy in front of an external repository (Docker Hub, npmjs.org, Maven Central) — the first request for a package fetches and caches it, and every subsequent request (from any build, anywhere) is served from that local cache, which is both faster and a resilience win if the upstream public registry has an outage or, worse, someone yanks a package version out from under you. A **virtual repository** is an aggregation layer presenting several local and remote repos under a single URL/endpoint, so a build tool only needs one repository configured instead of separately pointing at 'our internal repo' and 'the public proxy' — Artifactory resolves the request against whichever underlying repo actually has the artifact.

- [ ] Describe an artifact promotion workflow — moving a build from dev through to a production-ready release.

IF ASKED, SAY THIS
The pattern is: CI builds and publishes the artifact once into a non-production repository (say, `libs-snapshot-local` or a `dev` Docker repo), tagged with a build number or commit SHA. As that specific artifact passes each gate — staging deployment, QA sign-off, security scan — it gets **promoted**

The pattern is: CI builds and publishes the artifact once into a non-production repository (say, `libs-snapshot-local` or a `dev` Docker repo), tagged with a build number or commit SHA. As that specific artifact passes each gate — staging deployment, QA sign-off, security scan — it gets **promoted**, not rebuilt: either moved/copied to the next repository (`libs-release-local`, a `prod` Docker repo) or re-tagged, via the JFrog CLI, REST API, or a CI plugin step. The artifact's binary content never changes across this process — only its location/tag changes to reflect what's been validated — which is the actual mechanism behind 'the thing in staging is provably the same thing you're about to put in production.' Release

repositories are typically configured **immutable** — once published, that version can't be overwritten, so a version number always means exactly one specific build, forever.

- [ ] How does Artifactory work as a Docker registry, and how does that integrate with ECS/CI?

> **IF ASKED, SAY THIS**
> Artifactory exposes Docker repositories that behave like any standard Docker registry — `docker login`, `docker push`/`pull` against the Artifactory URL, with local repos for images you build and remote repos proxy-caching base images from Docker Hub so builds aren't dependent on an external registr

Artifactory exposes Docker repositories that behave like any standard Docker registry — `docker login`, `docker push`/`pull` against the Artifactory URL, with local repos for images you build and remote repos proxy-caching base images from Docker Hub so builds aren't dependent on an external registry's availability or rate limits. In a pipeline, the build stage tags and pushes the image to Artifactory after tests pass, and the deploy stage (ECS task definition's image field, or the CI step that registers a new task definition revision) references that exact image tag from Artifactory — often digest-pinned rather than just a mutable tag like `latest`, so a task definition always points at a specific, known-good image rather than whatever `latest` happens to resolve to at deploy time.

- [ ] How do you manage storage growth and retention in Artifactory?

> **IF ASKED, SAY THIS**
> Left unmanaged, an artifact repository grows forever — every CI build publishing a new artifact, snapshot builds accumulating especially fast since they're often published on every commit. The standard approach is retention policies scoped by repository type: aggressive cleanup on snapshot/dev repos

Left unmanaged, an artifact repository grows forever — every CI build publishing a new artifact, snapshot builds accumulating especially fast since they're often published on every commit. The standard approach is retention policies scoped by repository type: aggressive cleanup on snapshot/dev repositories (keep the last N builds, or anything older than X days, since those are disposable by nature), and much more conservative-to-none on release repositories (a production release artifact should generally be kept indefinitely, or per whatever compliance/audit retention the org requires, since you may need to redeploy or investigate an old version months later). This is usually automated — a scheduled cleanup job using **AQL** (Artifactory Query Language) to find and delete artifacts matching the retention criteria, rather than a manual periodic purge.

- [ ] How would you integrate Artifactory into a Jenkins or TeamCity pipeline — what does that look like concretely?

> **IF ASKED, SAY THIS**
> The **JFrog CLI** (`jf`) is the common thread across both — authenticate once, then `jf rt upload`/`jf docker push` to publish and `jf rt download` to pull, with the same commands working whether they're invoked from a Jenkinsfile shell step or a TeamCity build step, which keeps the actual publish/p

The **JFrog CLI** (`jf`) is the common thread across both — authenticate once, then `jf rt upload`/`jf docker push` to publish and `jf rt download` to pull, with the same commands working whether they're invoked from a Jenkinsfile shell step or a TeamCity build step, which keeps the actual publish/promote logic portable if the CI tool ever changes. Both Jenkins (the Artifactory plugin) and TeamCity also have native integrations that add build-info capture — associating exactly which artifacts a given CI build produced, which makes promotion and later traceability (which commit, which pipeline run, produced this exact artifact in prod) straightforward instead of inferred from a version number alone.

- [ ] Define blue-green deployment precisely — what makes it different from a rolling update?

> **IF ASKED, SAY THIS**
> Blue-green runs two complete, independent environments — "blue" (currently live) and "green" (the new version) — fully deployed and healthy side by side, with traffic cut over from blue to green in one atomic switch (a load balancer/DNS/router change), rather than gradually replacing instances in place. A rolling updat

Blue-green runs two **complete, independent environments** — "blue" (currently live) and "green" (the new version) — fully deployed and healthy side by side, with traffic cut over from blue to green in one atomic switch (a load balancer/DNS/router change), rather than gradually replacing instances in place. A rolling update, by contrast, incrementally replaces old pods/instances with new ones within a *single* environment, meaning both versions briefly coexist and share the same traffic pool during the rollout — blue-green never mixes versions in production traffic at all; it's all-old or all-new, switched instantly.

- [ ] What's the actual mechanism that performs "the switch," and what makes it fast?

> **IF ASKED, SAY THIS**
> The switch is whatever routing layer sits in front of both environments being repointed in one operation: a load balancer's target group swap, a Kubernetes Service's label selector being updated to match the green Deployment's pods instead of blue's, or a DNS record weight/target change. It's fast because nothing is be

The switch is whatever routing layer sits in front of both environments being repointed in one operation: a load balancer's target group swap, a Kubernetes Service's label selector being updated to match the green Deployment's pods instead of blue's, or a DNS record weight/target change. It's fast because nothing is being *created* at switch time — green is already fully running and health-checked before the switch happens; the switch is purely a routing change, which is why blue-green rollback is also close to instant: just flip the routing back, no need to redeploy or wait for old pods to come back up.

- [ ] What has to be true about the new (green) environment before you're actually safe to cut over?

IF ASKED, SAY THIS
It must be fully deployed at production scale (not a scaled-down "canary-sized" version — if it can't handle full production load the instant it receives 100% of traffic, the switch itself will cause an outage), pass real health checks, and — this is the part people skip — it must be validated with production-represent

It must be fully deployed at production scale (not a scaled-down "canary-sized" version — if it can't handle full production load the instant it receives 100% of traffic, the switch itself will cause an outage), pass real health checks, and — this is the part people skip — it must be validated with production-representative traffic or smoke tests *before* the switch, since blue-green's all-or-nothing traffic model means the first real production traffic green sees is 100% of it, with no gradual exposure to catch a subtle bug the way canary would. This is the core tradeoff versus canary, covered in the next chapter.

- [ ] Implement blue-green on Kubernetes using the Service selector pattern.

IF ASKED, SAY THIS
Deploy green fully alongside blue (both running, green receiving zero production traffic since the Service selector still points at blue). Validate green directly by port-forwarding or hitting its pods' IPs/a separate internal Service. Cut over by patching the Service's selector.version from blue to green — this is nea

```
# Two full Deployments, distinguished by a version label
# deployment-blue: labels: {app: myapp, version: blue}
# deployment-green: labels: {app: myapp, version: green}

apiVersion: v1
kind: Service
metadata:
  name: myapp
spec:
  selector:
    app: myapp
    version: blue      # <- flip this to "green" to cut over
  ports:
    - port: 80
```

Deploy green fully alongside blue (both running, green receiving zero production traffic since the Service selector still points at blue). Validate green directly by port-forwarding or hitting its pods' IPs/a separate internal Service. Cut over by patching the Service's `selector.version` from `blue` to `green` — this is near-instant since it's just an Endpoints recalculation, not a rollout. Keep the blue Deployment scaled up (not deleted) for a defined soak period so rollback is a selector flip back, not a redeploy.

- [ ] Implement blue-green on AWS using Route 53 weighted/failover routing.

IF ASKED, SAY THIS
Two separate target environments (ASGs/target groups) each behind their own ALB or target group, with a Route 53 record using weighted routing policy pointing at both — set blue to 100/green to 0 initially, and cut over by updating the weights to 0/100 (or use a failover routing policy with health checks for an even mo

Two separate target environments (ASGs/target groups) each behind their own ALB or target group, with a Route 53 record using weighted routing policy pointing at both — set blue to 100/green to 0 initially, and cut over by updating the weights to 0/100 (or use a failover routing policy with health checks for an even more automatic pattern). The tradeoff versus an ALB-level switch: DNS changes are subject to client-side and resolver TTL caching, so the cutover isn't instantaneous for every client the way an ALB target-group swap is — worth knowing this distinction explicitly, since "how fast is the cutover really" is a common follow-up.

- [ ] How does AWS CodeDeploy's native blue-green support work for ECS/Lambda?

IF ASKED, SAY THIS
For ECS, CodeDeploy provisions a second (green) task set alongside the running (blue) one, runs configured validation/health checks, then shifts ALB listener rules from the blue target group to the green target group — either all at once or on a linear/canary-style time schedule if configured (CodeDeploy actually suppo

For ECS, CodeDeploy provisions a second (green) task set alongside the running (blue) one, runs configured validation/health checks, then shifts ALB listener rules from the blue target group to the green target group — either all at once or on a linear/canary-style time schedule if configured (CodeDeploy actually supports both a true instant blue-green cutover and a gradual traffic-shifting variant, which is a useful nuance to mention: the tooling can do canary-style shifting even while conceptually you're using a blue-green environment strategy underneath). It keeps the original task set running for a configurable termination wait time, during which a CodeDeploy rollback is just re-pointing the listener back — the same "environment already exists, rollback is just routing" property as the general pattern.

- [ ] What infrastructure cost does blue-green fundamentally require, and how do teams justify/manage it?

IF ASKED, SAY THIS

By definition you're running two full production-capacity environments simultaneously during the deployment window — roughly double the compute cost for that period (not permanently, if you scale blue down/out after a successful cutover and soak period). Teams manage this by keeping the overlap window as short as reaso

By definition you're running two full production-capacity environments simultaneously during the deployment window — roughly double the compute cost for that period (not permanently, if you scale blue down/out after a successful cutover and soak period). Teams manage this by keeping the overlap window as short as reasonably safe (scale blue down once green is confirmed stable, not indefinitely), or by using ephemeral infrastructure (spin blue back down to zero after cutover rather than keeping full-scale standby capacity as a permanent cost) versus keeping a warm standby if instant rollback capability justifies the ongoing cost for a particularly critical service.

- [ ] Why is the database the hardest part of blue-green, and what's the core problem?

IF ASKED, SAY THIS
Application instances are stateless and easy to duplicate — the database usually isn't (you're not running two full copies of a production database and picking one). If green needs a schema change, both blue (old code) and green (new code) are typically pointed at the same database during the overlap/rollback window, m

Application instances are stateless and easy to duplicate — the database usually isn't (you're not running two full copies of a production database and picking one). If green needs a schema change, both blue (old code) and green (new code) are typically pointed at the *same* database during the overlap/rollback window, meaning the schema must simultaneously satisfy old code's expectations and new code's expectations — a schema change that breaks blue's queries makes blue-green's core promise (instant, safe rollback) false, since rolling back the app without also being able to roll back the schema leaves you stuck.

- [ ] What's the "expand/contract" pattern, and why is it the standard answer for blue-green-safe schema migrations?

IF ASKED, SAY THIS
Break a schema change into backward-compatible phases across separate deploys, never one atomic breaking change: Expand — add the new column/table without removing or repurposing the old one; deploy app code that can write to both old and new, read from old (still fully backward compatible with the still-running old ve

Break a schema change into backward-compatible phases across separate deploys, never one atomic breaking change: **Expand** — add the new column/table without removing or repurposing the old one; deploy app code that can write to both old and new, read from old (still fully backward compatible with the still-running old version). **Migrate** — backfill data, switch app code to read/write the new structure, while the old structure is still present and valid (so a rollback to the previous app version still works against the current schema). **Contract** — only once the new version has been running successfully for a safe period (old version is definitely not coming back) do you actually drop the old column/table. This turns "one risky atomic schema change" into several individually low-risk, individually rollback-safe deploys — exactly the property blue-green needs from its data layer to keep its instant-rollback promise honest.

- [ ] When would you deliberately choose blue-green over canary, and when is it the wrong choice?

IF ASKED, SAY THIS
Choose blue-green when instant, guaranteed-clean rollback matters more than gradual risk exposure — e.g. a major version bump, a change too risky/complex to reason about "partially rolled out," or a compliance/regulated context wanting a clean, auditable "before" and "after" state rather than a mixed-version window. Wr

Choose blue-green when instant, guaranteed-clean rollback matters more than gradual risk exposure — e.g. a major version bump, a change too risky/complex to reason about "partially rolled out," or a compliance/regulated context wanting a clean, auditable "before" and "after" state rather than a mixed-version window. **Wrong choice** when you genuinely need real production traffic to validate a change gradually and safely before full exposure (blue-green gives you zero real-traffic signal before 100% cutover, unlike canary's incremental exposure), when the double-capacity cost is prohibitive for that service's scale, or when the change is small/routine enough that canary's lower cost and equally fast automated rollback is simply the better fit.

- [ ] SCENARIO Scenario: right after a blue-green cutover, error rates spike. Walk through your exact response.

IF ASKED, SAY THIS
Flip the routing back to blue immediately — that's the entire point of having kept blue running and untouched; don't try to hot-fix green in place first. Verify recovery (error rate back to baseline) before doing anything else. Only then investigate green's failure using whatever monitoring/logs it generated during its

Flip the routing back to blue immediately — that's the entire point of having kept blue running and untouched; don't try to hot-fix green in place first. Verify recovery (error rate back to baseline) before doing anything else. Only then investigate green's failure using whatever monitoring/logs it generated during its brief live window, and critically check whether the database (section 6.3) was part of the problem — if green's deploy included a schema change that blue's rollback is now incompatible with, the "instant rollback" promise just broke, which is exactly why expand/contract discipline matters and is worth explicitly naming as the safeguard that should have prevented this.

- [ ] SCENARIO Scenario: leadership wants blue-green for a stateful service (a message queue consumer with in-flight processing state). What do you flag as the real risk?

IF ASKED, SAY THIS
In-flight processing state doesn't cleanly migrate with an instant routing switch the way stateless HTTP requests do — messages currently being processed by blue when the cutover happens can be lost, double-processed, or left in an inconsistent state depending on the consumer's ack/commit semantics at the exact moment

In-flight processing state doesn't cleanly migrate with an instant routing switch the way stateless HTTP requests do — messages currently being processed by blue when the cutover happens can be lost, double-processed, or left in an inconsistent state depending on the consumer's ack/commit semantics at the exact moment of cutover. The honest answer is that blue-green's clean "instant switch" model fits stateless request/response services best; for a stateful consumer you'd want to flag: draining blue's in-flight work to completion before fully stopping it (rather than an instant cutover), idempotent message processing so any edge-case reprocessing during the transition is safe rather than harmful, and possibly a different pattern entirely (rolling deploy with careful consumer-group rebalancing) if true instant cutover isn't achievable safely for this specific workload.

- [ ] Define canary deployment, and explain why it's specifically valuable for catching issues blue-green structurally can't.

IF ASKED, SAY THIS
Canary releases the new version to a small percentage of real production traffic (e.g. 5%) while the rest continues on the stable version, then progressively increases that percentage as confidence builds — unlike blue-green's all-or-nothing cutover. Because it's exposed to a genuine (if small) slice of real production

Canary releases the new version to a small percentage of real production traffic (e.g. 5%) while the rest continues on the stable version, then progressively increases that percentage as confidence builds — unlike blue-green's all-or-nothing cutover. Because it's exposed to a genuine (if small) slice of real production traffic and real user behavior before full rollout, it catches classes of bugs that only manifest under real traffic patterns, real data shapes, or real concurrent load — the exact blind spot blue-green has, since blue-green's green environment sees zero real traffic until the instant it sees 100% of it.

- [ ] What's the difference between canary and a plain rolling update — aren't both "gradual"?

IF ASKED, SAY THIS
A rolling update gradually replaces old instances with new ones based purely on deployment progress (X% of pods updated), with no feedback loop checking whether the new version is actually healthy under real traffic before continuing — it will happily roll all the way through even if the new version is subtly broken, a

A rolling update gradually replaces old instances with new ones based purely on deployment progress (X% of pods updated), with no feedback loop checking whether the new version is actually healthy under real traffic before continuing — it will happily roll all the way through even if the new version is subtly broken, as long as basic readiness probes pass. Canary is explicitly metric/analysis-gated: traffic percentage to the new version only increases if defined success criteria (error rate, latency, custom business metrics) are actually being met at each stage — it's a rollout strategy with a built-in automated judgment call at each step, not just a mechanical progress bar.

- [ ] What is "traffic shifting" at a technical level — how does a fraction of requests actually get routed to the new version?

IF ASKED, SAY THIS
Depends on the layer doing the splitting: at the Kubernetes/replica level , weighted routing can be approximated crudely by the ratio of old-version to new-version pod replica counts behind one Service (a coarse approximation — 1 canary pod out of 10 total gives roughly 10%, but it's replica-count-quantized, not precis

Depends on the layer doing the splitting: at the **Kubernetes/replica level**, weighted routing can be approximated crudely by the ratio of old-version to new-version pod replica counts behind one Service (a coarse approximation — 1 canary pod out of 10 total gives roughly 10%, but it's replica-count-quantized, not precisely controllable); at the **service mesh/ingress level** (Istio VirtualService, NGINX ingress canary annotations, Argo Rollouts + a mesh/ingress controller), traffic splitting is done precisely by percentage regardless of replica count, often with additional routing rules like header-based or cookie-based canary targeting (route specific internal users to canary regardless of the general percentage) for more controlled testing.

- [ ] What does Argo Rollouts add on top of a standard Kubernetes Deployment?

IF ASKED, SAY THIS
Rollout is a CRD that replaces (or wraps) a Deployment, adding native canary and blue-green strategies with defined steps, pauses, and — critically — integration with AnalysisRun / AnalysisTemplate resources that query a metrics provider (Prometheus, Datadog, CloudWatch) at each step and automatically abort/rollback th

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
spec:
  strategy:
    canary:
      steps:
        - setWeight: 10
        - pause: { duration: 5m }
        - setWeight: 50
        - pause: { duration: 10m }
        - setWeight: 100
      analysis:
        templates:
          - templateName: success-rate
```

`Rollout` is a CRD that replaces (or wraps) a Deployment, adding native canary and blue-green strategies with defined steps, pauses, and — critically — integration with `AnalysisRun` / `AnalysisTemplate` resources that query a metrics provider (Prometheus, Datadog, CloudWatch) at

each step and automatically abort/rollback the rollout if the defined success criteria aren't met, rather than a human having to watch dashboards and manually decide whether to proceed or roll back at each stage.

- [ ] How does Flagger's approach differ from Argo Rollouts?

IF ASKED, SAY THIS
Flagger is a Flux-ecosystem operator that automates canary analysis on top of your existing Deployment/service mesh setup (Istio, Linkerd, App Mesh, or NGINX/Gateway API) rather than replacing the Deployment resource with a new CRD the way Argo Rollouts' Rollout object does — you define a Canary custom resource pointin

Flagger is a Flux-ecosystem operator that automates canary analysis on top of your *existing* Deployment/service mesh setup (Istio, Linkerd, App Mesh, or NGINX/Gateway API) rather than replacing the Deployment resource with a new CRD the way Argo Rollouts' `Rollout` object does — you define a `Canary` custom resource pointing at your existing Deployment, and Flagger orchestrates creating the canary version, shifting traffic incrementally via the mesh, running metric checks (and optionally webhook-based load/smoke tests) between steps, and promoting or rolling back automatically. The practical choice between the two often comes down to whether you're already standardized on Argo CD/Rollouts specifically, or on Flux and want canary logic that layers onto standard Deployments without adopting a new primary workload resource type.

- [ ] How does Istio implement precise percentage-based traffic splitting?

IF ASKED, SAY THIS
A DestinationRule defines subsets (stable/canary) based on pod labels, and a VirtualService splits traffic between those subsets by explicit weight — enforced by Envoy sidecars at the request level, giving precise percentage control completely decoupled from how many actual pod replicas exist in each subset (unlike the

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
spec:
  hosts: [myapp]
  http:
    - route:
        - destination: { host: myapp, subset: stable }
          weight: 90
        - destination: { host: myapp, subset: canary }
          weight: 10
```

A `DestinationRule` defines subsets (stable/canary) based on pod labels, and a `VirtualService` splits traffic between those subsets by explicit weight — enforced by Envoy sidecars at the request level, giving precise percentage control completely decoupled from how many actual pod replicas exist in each subset (unlike the crude replica-ratio approximation mentioned above). This is also what enables more advanced routing (header/cookie-based canary targeting, mirroring traffic to canary without affecting the response) beyond simple weighted splitting.

- [ ] What is traffic mirroring/shadowing, and how is it different from canary?

IF ASKED, SAY THIS
Mirroring sends a copy of real production requests to the new version in parallel, while the response actually served to the user still comes entirely from the stable version — the new version's responses are discarded (or compared/logged) but never actually returned to a real user. This gives you real-traffic validati

Mirroring sends a **copy** of real production requests to the new version in parallel, while the response actually served to the user still comes entirely from the stable version — the new version's responses are discarded (or compared/logged) but never actually returned to a real user. This gives you real-traffic validation with genuinely **zero** user-facing risk, even lower risk than a 1% canary, at the cost of not being a true validation of end-to-end behavior for anything with side effects (a mirrored write request would double-write unless specifically guarded against) — so mirroring is most valuable for read-heavy/idempotent paths, while canary remains necessary for validating genuinely user-facing behavior and side-effecting operations.

- [ ] Design an AnalysisTemplate-style check for a canary — what metrics, and what makes a good threshold?

IF ASKED, SAY THIS
Good canary metrics are ones that actually reflect user impact and have enough traffic volume in the canary slice to be statistically meaningful, not noisy — error rate and p99 latency are the standard baseline; business-specific metrics (checkout completion rate, for an e-commerce example) are the more sophisticated a

```
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata: { name: success-rate }
spec:
  metrics:
    - name: error-rate
      interval: 1m
      successCondition: result[0] <= 0.01
      provider:
```

```
        prometheus:
          address: http://prometheus:9090
          query: |
            sum(rate(http_requests_total{version="canary",status=~"5.."}[1m]))
            /
            sum(rate(http_requests_total{version="canary"}[1m]))
```

Good canary metrics are ones that actually reflect user impact and have enough traffic volume in the canary slice to be statistically meaningful, not noisy — error rate and p99 latency are the standard baseline; business-specific metrics (checkout completion rate, for an e-commerce example) are the more sophisticated addition senior candidates should mention. A good threshold accounts for normal variance (comparing canary's rate against the stable version's *concurrent* rate, not a fixed historical number, controls for anything else happening at the same time — a traffic spike affecting both versions shouldn't falsely fail the canary). Low-traffic services need a longer analysis window or a higher canary percentage just to get a statistically meaningful sample size before a threshold check means anything.

- [ ] What happens mechanically when an automated canary analysis fails mid-rollout?

IF ASKED, SAY THIS
The rollout controller (Argo Rollouts, Flagger) immediately halts progression — no further weight increase — and typically automatically shifts weight back to 0% for the canary/new version, scaling it down and leaving 100% of traffic on the still-fully-running stable version, which is why this is fast: like blue-green,

The rollout controller (Argo Rollouts, Flagger) immediately halts progression — no further weight increase — and typically automatically shifts weight back to 0% for the canary/new version, scaling it down and leaving 100% of traffic on the still-fully-running stable version, which is why this is fast: like blue-green, the stable version was never scaled down during the canary process, so "rollback" here is also a routing change, not a redeploy. The failed AnalysisRun and its metric values are recorded for post-incident review, and (properly configured) triggers an alert to the team so the failure gets investigated even though users were largely protected by the automated rollback.

- [ ] Lay out Rolling vs Blue-Green vs Canary side by side: risk exposure, rollback speed, infra cost, and complexity.

IF ASKED, SAY THIS
The honest senior-level answer to "which is best" is that they're not strictly ordered — the right choice depends on the change's risk profile, the service's traffic volume, and how much automated analysis tooling the org has actually invested in; many mature platforms use canary as the default for routine releases and

- **Rolling** — lowest infra cost (no doubled capacity), but versions mix in production during rollout and there's no automated analysis gate by default; rollback means rolling forward to the previous version again (not instant, and old pods may already be gone).
- **Blue-Green** — near-instant rollback (routing flip), zero version mixing, but ~2x infra cost during the switch window and zero real-traffic validation before 100% cutover; complexity concentrates in state/database handling (expand/contract).
- **Canary** — lowest real-user risk exposure (small blast radius, validated with genuine production traffic before full rollout), fast automated rollback if tooling is in place, but the highest tooling/setup complexity (mesh or ingress-level traffic splitting, metrics pipeline, analysis templates) and requires enough traffic volume to make staged percentages statistically meaningful.

The honest senior-level answer to "which is best" is that they're not strictly ordered — the right choice depends on the change's risk profile, the service's traffic volume, and how much automated analysis tooling the org has actually invested in; many mature platforms use canary as the default for routine releases and reserve blue-green for major/structural changes where instant clean rollback matters more than gradual exposure.

- [ ] SCENARIO Scenario: a canary rollout passes all automated checks and reaches 100%, but a serious bug surfaces 2 hours later that the analysis window didn't catch. What does this tell you about the canary config, and what do you change?

IF ASKED, SAY THIS
It tells you the analysis window/duration didn't match the bug's actual manifestation time — some issues (a slow memory leak, a rare code path only hit by a specific downstream batch job, a cache-related bug that only shows once the cache fills) genuinely don't show up in a 5-10 minute canary window no matter how good

It tells you the analysis window/duration didn't match the bug's actual manifestation time — some issues (a slow memory leak, a rare code path only hit by a specific downstream batch job, a cache-related bug that only shows once the cache fills) genuinely don't show up in a 5-10 minute canary window no matter how good the metrics are. Fix isn't necessarily "make canary windows longer for everything" (that slows every release) — it's tiering: extend analysis duration and hold-at-partial-weight time specifically for higher-risk changes, keep short windows for routine low-risk ones, and separately invest in post-deploy monitoring/alerting that isn't purely canary-gated, since canary is a risk-reduction tool, not a complete substitute for ongoing production monitoring after full rollout.

- [ ] SCENARIO Scenario: you're introducing canary deployments to a team that's only ever done manual rolling deploys. What's the actual rollout plan for the practice itself, not just the tooling?

IF ASKED, SAY THIS
Start with manually-gated canary (fixed low percentage, human decides to proceed/rollback after watching dashboards) before automating the analysis — build trust in the mechanism and get the team comfortable reading the right signals first. Pick metrics carefully with the team's input (what actually indicates "this dep

1. Start with manually-gated canary (fixed low percentage, human decides to proceed/rollback after watching dashboards) before automating the analysis — build trust in the mechanism and get the team comfortable reading the right signals first.
2. Pick metrics carefully with the team's input (what actually indicates "this deploy is bad" for their specific service) rather than a generic template, since a canary gate the team doesn't trust or understand will just get bypassed under deadline pressure.

3. Once the manual process is trusted and the metrics have proven reliable (few false positives/negatives over several real releases), automate the analysis gate so it's not a manual step people can skip when rushed.
4. Tie it into existing CI/CD so canary becomes the default path for every deploy, not an optional extra step — an opt-in safety mechanism gets skipped exactly when it's needed most (under time pressure).

- [ ] How would you design an enterprise CI/CD pipeline incorporating automated testing, security scanning, and deployment governance — walk through the stages. ▲

> **IF ASKED, SAY THIS**
> Source → build/unit test → static/security scanning (SAST, dependency/SCA, IaC scanning if infra changes are included) → artifact publish (ECR/Artifactory) → deploy to a lower environment → automated integration/smoke tests → a governance gate (automated policy checks plus human approval for production) → deploy to production, ideally pro

Source → build/unit test → static/security scanning (SAST, dependency/SCA, IaC scanning if infra changes are included) → artifact publish (ECR/Artifactory) → deploy to a lower environment → automated integration/smoke tests → a governance gate (automated policy checks plus human approval for production) → deploy to production, ideally progressive (canary/blue-green) rather than a single big-bang cutover.

Each stage should fail fast with actionable feedback — a security scan that just says "failed" with no specifics gets ignored or routed around.

- [ ] GitHub Actions vs. Jenkins vs. GitLab CI vs. AWS CodePipeline — how would you decide which fits a given client engagement? ▲

> **IF ASKED, SAY THIS**
> GitHub Actions/GitLab CI when source control is already there and native integration matters most, without standing up separate CI infra. Jenkins for existing investment/expertise or genuinely complex custom pipeline needs, at the cost of Jenkins itself now being infrastructure you own. CodePipeline when deep native AWS integration matter

GitHub Actions/GitLab CI when source control is already there and native integration matters most, without standing up separate CI infra. **Jenkins** for existing investment/expertise or genuinely complex custom pipeline needs, at the cost of Jenkins itself now being infrastructure you own. **CodePipeline** when deep native AWS integration matters most (CodeBuild/CodeDeploy/CloudFormation, IAM-based permissions instead of separate CI credentials) and the org is comfortable being AWS-native for pipeline tooling too.

As with IaC, a consulting/platform role realistically needs fluency across more than one given varying client environments.

- [ ] How would you integrate security scanning into a CI/CD pipeline without it becoming a bottleneck developers route around? ▲

> **IF ASKED, SAY THIS**
> Gate on new/high-severity findings introduced by the current change rather than blocking every build on the entire legacy backlog, run scans in parallel with other stages where possible, and make findings actionable — specific file/line, clear remediation guidance, not a wall of undifferentiated output. A defined exception/waiver process

Gate on new/high-severity findings introduced by the current change rather than blocking every build on the entire legacy backlog, run scans in parallel with other stages where possible, and make findings actionable — specific file/line, clear remediation guidance, not a wall of undifferentiated output.

A defined exception/waiver process for genuine false positives or accepted risk, reviewed and time-boxed, so legitimate edge cases don't quietly train people to distrust or bypass the scanner.

- [ ] What does "deployment governance" mean concretely in a pipeline, and how would you implement it? ▲

> **IF ASKED, SAY THIS**
> Explicit approval gates for production (who can approve, logged/auditable), environment-specific credentials so a pipeline against dev genuinely cannot deploy to production without the proper stage, deployment windows/freeze periods enforced where the business needs them, and a complete audit trail — what was deployed, by whom/what proces

Explicit approval gates for production (who can approve, logged/auditable), environment-specific credentials so a pipeline against dev genuinely cannot deploy to production without the proper stage, deployment windows/freeze periods enforced where the business needs them, and a complete audit trail — what was deployed, by whom/what process, when, and what the previous version was for rollback.

- [ ] How would you design a rollback strategy as part of the CI/CD pipeline itself, not as an afterthought? ▲

> **IF ASKED, SAY THIS**
> A deployment strategy with rollback built in from the start — blue-green (instant traffic cutover back) or canary (halt and revert before full rollout on bad early signals) rather than in-place rolling updates that make "what does rollback even mean here" a harder question mid-incident. Automated rollback triggers tied to concrete health

A deployment strategy with rollback built in from the start — blue-green (instant traffic cutover back) or canary (halt and revert before full rollout on bad early signals) rather than in-place rolling updates that make "what does rollback even mean here" a harder question mid-incident.

Automated rollback triggers tied to concrete health signals (error rate, latency) post-deployment, not solely relying on a human noticing — the first few minutes after a deploy are exactly when automated safety nets matter most.

- [ ] How would you handle CI/CD pipeline design differently for infrastructure changes (Terraform/CloudFormation) versus application code changes?

> **IF ASKED, SAY THIS**
> Infrastructure changes warrant a more conservative gate — a human-reviewed plan/preview step is closer to mandatory, given the blast radius of a bad infra change (potentially the networking/security posture underneath many services at once, versus one service for a bad app deploy). Application pipelines can lean more toward progressive au

Infrastructure changes warrant a more conservative gate — a human-reviewed plan/preview step is closer to mandatory, given the blast radius of a bad infra change (potentially the networking/security posture underneath many services at once, versus one service for a bad app deploy).

Application pipelines can lean more toward progressive automated rollout with fast automated rollback, since the blast radius per bad deploy is more contained and rolling back a container image is faster and lower-risk than reversing an infrastructure change.

Observability

- [ ] CloudWatch Logs, Metrics, and Alarms — how do the three pieces actually fit together?

IF ASKED, SAY THIS
Logs is the raw event store — application/service log lines shipped in via agents, Lambda's automatic integration, or the CloudWatch agent on EC2. Metrics are numeric time series — either published directly by AWS services automatically, or derived from log data via metric filters (a pattern match o

Logs is the raw event store — application/service log lines shipped in via agents, Lambda's automatic integration, or the CloudWatch agent on EC2. Metrics are numeric time series — either published directly by AWS services automatically, or derived from log data via metric filters (a pattern match on log lines that increments a metric, e.g., counting ERROR lines). Alarms watch a metric against a threshold over an evaluation period and change state (OK/ALARM/INSUFFICIENT_DATA), triggering an action — an SNS notification, an Auto Scaling action, or, relevantly for this role, an FIS stop condition. The chain that actually matters operationally: logs generate metrics, metrics drive alarms, alarms drive automated response.

- [ ] What's CloudWatch Logs Insights, and how do you use it to debug across many log groups at once?

IF ASKED, SAY THIS
It's a purpose-built query language for CloudWatch Logs — you write a query (parse, filter, stats/aggregate, sort) against one or more log groups without needing to export data anywhere first, and it's genuinely fast even against large volumes because it only scans the time range you specify. Practi

It's a purpose-built query language for CloudWatch Logs — you write a query (parse, filter, stats/aggregate, sort) against one or more log groups without needing to export data anywhere first, and it's genuinely fast even against large volumes because it only scans the time range you specify. Practical use: query across every service's log group filtered to a specific request ID or user ID during an incident, or aggregate error counts by error type across a whole fleet of Lambda functions in one query — the alternative of grepping through the console log-group-by-log-group doesn't scale past a handful of services.

- [ ] What's a composite alarm in CloudWatch, and why use one instead of just more single-metric alarms?

IF ASKED, SAY THIS
A composite alarm combines the state of multiple other alarms with AND/OR logic into one higher-level alarm — e.g., only page if both "high latency" AND "high error rate" are simultaneously in ALARM, suppressing the page if just one is noisy on its own. This is a direct tool for the same alert-fatig

A composite alarm combines the state of multiple other alarms with AND/OR logic into one higher-level alarm — e.g., only page if both "high latency" AND "high error rate" are simultaneously in ALARM, suppressing the page if just one is noisy on its own. This is a direct tool for the same alert-fatigue problem you'd solve with routing rules elsewhere: instead of getting three separate pages for three related symptoms of the same underlying issue, you get one alarm that actually represents "this is a real problem," with the individual alarms still available for the detailed diagnosis once someone's paged.

- [ ] What's Container Insights, and what does it add over vanilla CloudWatch metrics for ECS/EKS?

IF ASKED, SAY THIS
Vanilla CloudWatch gives you the basics AWS publishes automatically (ECS service CPU/memory at the aggregate level). Container Insights adds container- and pod-level granularity — per-task/per-pod resource usage, cluster-level dashboards, and for EKS specifically, integration with the Kubernetes API

Vanilla CloudWatch gives you the basics AWS publishes automatically (ECS service CPU/memory at the aggregate level). Container Insights adds container- and pod-level granularity — per-task/per-pod resource usage, cluster-level dashboards, and for EKS specifically, integration with the Kubernetes API to surface node/pod/namespace metrics natively in CloudWatch rather than requiring a separate Prometheus stack just to see that level of detail. It costs extra (it's an opt-in, additional-charge feature) and adds some agent overhead, so the real tradeoff in an interview answer is "when is this worth it over just running kube-state-metrics + Prometheus" — usually when you want everything in one AWS-native pane rather than maintaining a separate OSS observability stack.

- [ ] Explain AWS X-Ray — how does it capture a trace, and what's a segment vs. a subsegment?

IF ASKED, SAY THIS
A segment is the top-level unit of work for one service in a trace — typically one request's full handling within that service, including start/end time and metadata. A subsegment is a nested breakdown within that segment — a specific downstream call (a DynamoDB query, an outbound HTTP call) — givin

A segment is the top-level unit of work for one service in a trace — typically one request's full handling within that service, including start/end time and metadata. A subsegment is a nested breakdown within that segment — a specific downstream call (a DynamoDB query, an outbound HTTP call) — giving you a mini-waterfall within the service's own segment, not just a single opaque block. Capture happens via the X-Ray SDK instrumenting your code (or auto-instrumentation for supported AWS services like Lambda/API Gateway), which sends trace data to the X-Ray daemon (a local UDP listener that batches and forwards to the X-Ray API), then X-Ray stitches segments from different services together into one trace using a shared trace ID, and renders the service map showing how everything connects.

- [ ] How does X-Ray sampling work, and why can't you trace 100% of requests at real scale?

IF ASKED, SAY THIS

Tracing every single request adds real overhead (every trace is data you're generating, storing, and paying for) and most requests are "boring" — identical to a thousand others, so full tracing gives diminishing diagnostic value past a certain volume. X-Ray's default sampling rule captures the first

Tracing every single request adds real overhead (every trace is data you're generating, storing, and paying for) and most requests are "boring" — identical to a thousand others, so full tracing gives diminishing diagnostic value past a certain volume. X-Ray's default sampling rule captures the first request each second plus 5% of any additional requests, but you can define custom sampling rules — e.g., always trace requests with an error status or above a latency threshold (the interesting ones), while sampling routine successful requests much more sparsely. The general principle, same as with metrics cardinality: sample the boring traffic, always capture the anomalous traffic.

- [ ] X-Ray vs. OpenTelemetry — when would you pick native X-Ray vs. an OTel-based approach on AWS?

IF ASKED, SAY THIS
X-Ray is deeply integrated with AWS-native services (Lambda, API Gateway, and others get automatic instrumentation with almost no setup) and is the simplest path if your stack is AWS-native end to end and you don't need portability. OTel is the better choice when you want backend portability (not lo

X-Ray is deeply integrated with AWS-native services (Lambda, API Gateway, and others get automatic instrumentation with almost no setup) and is the simplest path if your stack is AWS-native end to end and you don't need portability. OTel is the better choice when you want backend portability (not locked into X-Ray specifically), you're running a genuinely multi-cloud or hybrid environment, or you want one consistent instrumentation approach across services that don't all live on AWS. In practice a lot of AWS shops run both — X-Ray for the AWS-native pieces where it's nearly free to set up, OTel Collector configured with an AWS X-Ray exporter so OTel-instrumented services still land in the same X-Ray service map and traces stay unified rather than split across two disconnected systems.

- [ ] What is CloudTrail, and how is it different from CloudWatch — people often conflate them?

IF ASKED, SAY THIS
CloudTrail answers "who did what, when, from where" — it's an audit log of API calls made against your AWS account (console, CLI, SDK, or another service acting on your behalf), recording the identity, the action, the resource, and the source IP/time, whether or not that action succeeded. CloudWatch

CloudTrail answers "who did what, when, from where" — it's an audit log of API calls made against your AWS account (console, CLI, SDK, or another service acting on your behalf), recording the identity, the action, the resource, and the source IP/time, whether or not that action succeeded. CloudWatch answers "what is the system doing right now/over time" — operational metrics, logs, and alarms about resource behavior and performance. The distinction that matters: CloudTrail is fundamentally about accountability and compliance (who changed this security group, who deleted this bucket), CloudWatch is fundamentally about operational health — and GuardDuty's threat detection is actually built on top of CloudTrail data, which is worth naming as the bridge between the two.

- [ ] How would you use CloudTrail with CloudWatch Alarms/EventBridge to react to a risky API call in near-real-time?

IF ASKED, SAY THIS
CloudTrail alone is a log, not an alert — the pattern is to ship CloudTrail events into CloudWatch Logs, define a metric filter matching the specific risky API call (e.g., DeleteTrail , a root account login, a security group ingress rule opened to 0.0.0.0/0), and put a CloudWatch Alarm on that metr

CloudTrail alone is a log, not an alert — the pattern is to ship CloudTrail events into CloudWatch Logs, define a metric filter matching the specific risky API call (e.g., `DeleteTrail`, a root account login, a security group ingress rule opened to `0.0.0.0/0`), and put a CloudWatch Alarm on that metric that fires the moment the pattern occurs — not on a schedule. EventBridge is the more modern, more flexible alternative for the same job: it can match CloudTrail events directly by rule (no metric-filter indirection needed) and fan out to multiple targets — a Lambda for automated remediation, an SNS topic for paging, a Step Functions workflow for a multi-step response — which is generally the better fit when the response needs to be more than "just send a notification."

- [ ] ELK vs OpenSearch — why does OpenSearch exist, and when do you pick one over the other?

IF ASKED, SAY THIS
OpenSearch is the AWS/Linux-Foundation-stewarded fork created after Elastic changed Elasticsearch and Kibana's license away from a fully open-source one — OpenSearch stayed on a permissive Apache-2.0 license. Functionally they started identical and have been diverging since, though the core (index/s

OpenSearch is the AWS/Linux-Foundation-stewarded fork created after Elastic changed Elasticsearch and Kibana's license away from a fully open-source one — OpenSearch stayed on a permissive Apache-2.0 license. Functionally they started identical and have been diverging since, though the core (index/shard model, query DSL) remains close. Pick Elastic if you want the latest Elastic-only features and are fine with their license terms; pick OpenSearch if you're already AWS-centric (managed OpenSearch Service integrates cleanly with IAM/VPC), want the permissive license, or are avoiding vendor lock to Elastic's roadmap.

- [ ] What does each component in the ELK stack actually do?

IF ASKED, SAY THIS
Elasticsearch — the distributed search/storage engine, indexes and serves queries. Logstash — a heavier-weight ingest pipeline that can parse, transform, and enrich data before indexing (grok patterns, field extraction). Kibana — the visualization/query UI on top. Beats (Filebeat, Metricbeat) — ligh

Elasticsearch — the distributed search/storage engine, indexes and serves queries. Logstash — a heavier-weight ingest pipeline that can parse, transform, and enrich data before indexing (grok patterns, field extraction). Kibana — the visualization/query UI on top. Beats (Filebeat, Metricbeat) — lightweight shippers that read logs/metrics at the source and forward them, typically to Logstash or directly to Elasticsearch. In a Kubernetes context,

Logstash is often skipped entirely in favor of Fluent Bit doing lightweight parsing at the DaemonSet level, sending straight to Elasticsearch/OpenSearch to cut a hop.

- [ ] How would you design log ingestion from EKS pods into ELK/OpenSearch at scale?

IF ASKED, SAY THIS
Fluent Bit as a DaemonSet (one per node, low resource footprint) tailing container log files from the node's filesystem, enriching each record with Kubernetes metadata (namespace, pod, labels) via the kubernetes filter, then shipping to an output — either straight to OpenSearch/Elasticsearch, or thr

Fluent Bit as a DaemonSet (one per node, low resource footprint) tailing container log files from the node's filesystem, enriching each record with Kubernetes metadata (namespace, pod, labels) via the kubernetes filter, then shipping to an output — either straight to OpenSearch/Elasticsearch, or through a buffering layer (Kafka, or Fluent Bit's own backpressure handling) if ingestion volume spikes faster than the cluster can absorb. At real scale, also plan for multiline log handling (stack traces spanning multiple lines need to be stitched back into one event) and index-per-day-or-namespace so retention/deletion is a cheap index-drop instead of a slow document-level delete.

- [ ] What's an index lifecycle policy, and why do you need one in production?

IF ASKED, SAY THIS
Without one, indices grow forever and eventually the cluster runs out of disk or query performance degrades on huge indices. ILM/ISM policies automate the lifecycle: hot (actively written, fast storage), warm (read-mostly, can move to cheaper nodes, force-merge segments down), cold (rarely queried,

Without one, indices grow forever and eventually the cluster runs out of disk or query performance degrades on huge indices. ILM/ISM policies automate the lifecycle: hot (actively written, fast storage), warm (read-mostly, can move to cheaper nodes, force-merge segments down), cold (rarely queried, can even move to object storage), and delete (past your retention requirement — which in a regulated environment might be a compliance-driven minimum, not just a cost decision). Setting this up before you need it is the difference between a scheduled, boring rollover and a 2am "cluster is red because disk is full" incident.

- [ ] How do you troubleshoot a slow or failing Elasticsearch/OpenSearch query?

IF ASKED, SAY THIS
_profile API to see where time is actually going inside the query itself — often it's an unbounded wildcard/regex query, a deep pagination (from + size instead of search_after), or aggregating on a high-cardinality field without a filter narrowing the dataset first. At the cluster level, check for

`_profile` API to see where time is actually going inside the query itself — often it's an unbounded wildcard/regex query, a deep pagination (`from` + `size` instead of `search_after`), or aggregating on a high-cardinality field without a filter narrowing the dataset first. At the cluster level, check for hot-spotting — one node handling disproportionate shard traffic because shard allocation isn't balanced — and check JVM heap/GC pressure, since a cluster spending most of its time in GC will make every query slow regardless of how well-written it is.

- [ ] Shard vs replica — how does shard count affect performance?

IF ASKED, SAY THIS
A shard is a horizontal slice of an index's data — how the index is distributed across nodes for parallelism and to let it scale past a single node's capacity. A replica is a full copy of a shard for redundancy and read throughput, not a distinct partition of data. Too few shards for the data volume

A shard is a horizontal slice of an index's data — how the index is distributed across nodes for parallelism and to let it scale past a single node's capacity. A replica is a full copy of a shard for redundancy and read throughput, not a distinct partition of data. Too few shards for the data volume caps parallelism and eventually hits per-shard document/size limits; too many shards for the cluster size wastes overhead (every shard costs memory/file handles) and can actually slow queries down by fanning out to more nodes than necessary — right-sizing (a common rule of thumb is keeping individual shards in the tens-of-GB range) matters more than "more shards = faster."

- [ ] How do you control storage costs and cardinality in a high-volume logging pipeline?

IF ASKED, SAY THIS
Sample or drop at the source for high-volume, low-value logs (debug-level in production, health-check request logs) rather than indexing everything and filtering later — you've already paid the ingestion cost by then. Use index templates that explicitly map field types instead of relying on dynamic

Sample or drop at the source for high-volume, low-value logs (debug-level in production, health-check request logs) rather than indexing everything and filtering later — you've already paid the ingestion cost by then. Use index templates that explicitly map field types instead of relying on dynamic mapping, which silently creates a new field (and mapping explosion risk) for every slightly different log shape. Tiered storage (hot/warm/cold, or even a cold tier backed by S3) moves cost down as data ages without deleting it outright, which matters when compliance requires longer retention than you'd want to keep on fast, expensive storage.

- [ ] Synchronous vs asynchronous communication between microservices — when do you pick each?

IF ASKED, SAY THIS
Synchronous (REST/gRPC) when the caller genuinely needs an immediate answer to proceed — a read, a validation check, anything in the user-facing request path where "try again later" isn't acceptable. Asynchronous (queue/event) when the caller doesn't need to wait for the work to finish, when you wan

Synchronous (REST/gRPC) when the caller genuinely needs an immediate answer to proceed — a read, a validation check, anything in the user-facing request path where "try again later" isn't acceptable. Asynchronous (queue/event) when the caller doesn't need to wait for the work to finish, when you want to decouple the producer's uptime from the consumer's (producer can keep accepting work even if a consumer is temporarily down), or when one event needs to fan out to multiple independent consumers. The tradeoff is complexity: async buys resilience and decoupling but costs you eventual consistency and harder debugging (a request's effects are no longer traceable in one synchronous call stack).

- [ ] Explain the outbox pattern — why is it needed for reliable event publishing?

IF ASKED, SAY THIS
The problem: if a service writes to its database and then separately publishes an event, those two operations aren't atomic — a crash between them either loses the event or double-processes it. The outbox pattern fixes this by writing the event into an "outbox" table in the same local database trans

The problem: if a service writes to its database and then separately publishes an event, those two operations aren't atomic — a crash between them either loses the event or double-processes it. The outbox pattern fixes this by writing the event into an "outbox" table in the same local database transaction as the business data change, so both succeed or fail together. A separate process (a polling job, or change-data-capture like Debezium reading the DB's write-ahead log) then reads the outbox table and actually publishes to Kafka/SNS/SQS, with at-least-once delivery — the atomicity guarantee you actually need moves to where a transaction can enforce it.

- [ ] SQS vs Kafka/SNS/EventBridge — what's the actual difference?

IF ASKED, SAY THIS
SQS is a queue: a message is consumed once and gone, good for point-to-point work distribution (a pool of workers pulling jobs). Kafka is a durable, ordered log that multiple independent consumer groups can each read at their own pace, replaying from any offset — the right fit when multiple systems

SQS is a queue: a message is consumed once and gone, good for point-to-point work distribution (a pool of workers pulling jobs). Kafka is a durable, ordered log that multiple independent consumer groups can each read at their own pace, replaying from any offset — the right fit when multiple systems need the same event stream independently, or you need strict ordering per partition key. SNS/EventBridge are pub/sub fan-out — one event, many subscribers, each typically feeding its own SQS queue — good for "notify everyone who cares" without the producer knowing who's listening.

- [ ] How do you handle poison messages in an event-driven pipeline?

IF ASKED, SAY THIS
A dead-letter queue (DLQ) is the baseline — after N failed processing attempts, the message moves off the main queue so it can't block or endlessly retry-loop the rest of the pipeline, and gets surfaced for investigation instead of silently dropped. The harder part is what happens next: alerting on

A dead-letter queue (DLQ) is the baseline — after N failed processing attempts, the message moves off the main queue so it can't block or endlessly retry-loop the rest of the pipeline, and gets surfaced for investigation instead of silently dropped. The harder part is what happens next: alerting on DLQ depth (a growing DLQ is itself an incident signal), and having an actual runbook for reprocessing — is it safe to just replay the message once the bug's fixed, or did partial processing already happen and a replay would duplicate side effects (which loops back to needing idempotent consumers in the first place).

- [ ] What's the saga pattern, and why do distributed transactions need it?

IF ASKED, SAY THIS
A normal ACID transaction can't span multiple services' independent databases — there's no single transaction coordinator across service boundaries in most architectures. A saga breaks a multi-step business transaction into a sequence of local transactions, each with a corresponding compensating act

A normal ACID transaction can't span multiple services' independent databases — there's no single transaction coordinator across service boundaries in most architectures. A saga breaks a multi-step business transaction into a sequence of local transactions, each with a corresponding compensating action if a later step fails — e.g., "reserve inventory → charge payment → confirm order," and if payment fails, a compensating "release inventory" step runs instead of a rollback. Choreography (each service reacts to events and knows its own compensation) works for simple flows; orchestration (a central saga coordinator explicitly sequencing steps) is usually easier to reason about and debug once the flow has more than a few steps.

- [ ] How do you debug a "lost event" where a downstream service never received something it should have?

IF ASKED, SAY THIS
Distributed tracing with context propagated into the message (trace ID as a message attribute, not just in the body) turns this from a guessing game into a lookup — find the producer's span, follow the trace, see exactly where it stops. Absent that, work backward through each hop: did the producer's

Distributed tracing with context propagated into the message (trace ID as a message attribute, not just in the body) turns this from a guessing game into a lookup — find the producer's span, follow the trace, see exactly where it stops. Absent that, work backward through each hop: did the producer's write actually commit and publish (check the outbox/producer logs), did the message actually land in the queue/topic (broker-side metrics), did the consumer group have a lag spike or a rebalance that could've delayed or skipped it, and is there a DLQ entry that would explain a "processed but failed" case being mistaken for "never arrived." Most "lost" events turn out to be delayed, filtered by a routing rule, or sitting in a DLQ no one's watching.

- [ ] Metrics vs logs vs traces — and where does each of Prometheus, Grafana, and Loki actually fit?

IF ASKED, SAY THIS
The three pillars solve different questions. Metrics are cheap numeric time series, best for 'is it healthy right now,' dashboards and alerts. Logs are discrete detailed events, best for 'what exactly happened.' Traces follow one request across every service it touched, best for 'where is the latenc

The three pillars solve different questions. **Metrics** are cheap numeric time series, best for 'is it healthy right now,' dashboards and alerts. **Logs** are discrete detailed events, best for 'what exactly happened.' **Traces** follow one request across every service it touched, best for 'where is the latency, which service actually failed.' Mapped to tools: **Prometheus stores and queries metrics**. **Grafana doesn't store anything, it visualizes**, querying Prometheus for metrics, Loki for logs, and Tempo/Jaeger for traces, often on the same dashboard. **Loki stores logs**, and deliberately indexes only labels, not full text, so it stays cheap at scale, the opposite tradeoff from something like Elasticsearch.

Remember: Say it as a clean equation: Prometheus = metrics store, Loki = logs store, Grafana = visualizes *both* (and traces) · Grafana is a rendering layer, not a data store — this is exactly the distinction that got marked down · Loki's 'index labels only, not full text' design choice is worth naming — it's why it's cheap

- [ ] A Grafana dashboard suddenly shows 'No Data' on every panel — how do you fix it?

IF ASKED, SAY THIS
Check the data source connection first, Settings → Data Sources → Test, a URL, auth, or network change breaks every panel at once, the signature of a data-source problem rather than a query problem. If that's healthy, confirm the underlying source actually has data for the selected time range: a mig

Check the data source connection first, Settings → Data Sources → Test, a URL, auth, or network change breaks every panel at once, the signature of a data-source problem rather than a query problem. If that's healthy, confirm the underlying source actually has data for the selected time range: a migrated Prometheus won't have backfilled history, so a dashboard defaulting to 'last 6 hours' can look empty even though everything's technically working. Only after both check out do you look at the panel queries themselves for a broken PromQL/LogQL expression, often from a label that got renamed upstream.

Remember: Every panel dead at once → data source problem, not a query problem — check that first, not last · Time range mismatch after a migration is a very common false alarm · Individual panel(s) dead while others work → that's a query problem, a different branch of investigation

- [ ] CPU crosses 95% but no alert fires — why, and what do you check?

IF ASKED, SAY THIS
Work backwards through the alerting pipeline instead of guessing: does the alerting rule actually exist and is the PromQL expression correct, test it directly in the Prometheus UI, don't assume. If it evaluates correctly, check the for: duration; a spike that didn't sustain long enough never leaves

Work backwards through the alerting pipeline instead of guessing: does the alerting rule actually exist and is the PromQL expression correct, test it directly in the Prometheus UI, don't assume. If it evaluates correctly, check the `for:` duration; a spike that didn't sustain long enough never leaves Pending state. If it fired in Prometheus, check Alertmanager received it, a label mismatch between the alert and the route's matchers means it arrives but never gets routed anywhere. Last, check the notification channel itself, an expired PagerDuty/Slack integration token is a boring but common dead end.

Remember: Work the pipeline backwards: rule exists? → evaluates true? → for: duration met? → reached Alertmanager? → routed? → delivered? · A route label mismatch is a silent failure — the alert fires but nobody ever sees it, and nothing errors · This investigation order is reusable for almost any 'alert should have fired but didn't' question

- [ ] What does New Relic (or a similar APM) actually add on top of Prometheus + Grafana?

IF ASKED, SAY THIS
Prometheus/Grafana are metrics-first — you instrument and query time-series yourself. An APM like New Relic auto-instruments application code to give distributed tracing and code-level (line/function) visibility out of the box, which is much harder to get from a pure metrics stack without also standing up a separate tracing system (Jaeger/Tempo).

The practical trade-off: APM tools are faster to get deep code-level visibility from (agent-based auto-instrumentation), but Prometheus/Grafana is usually cheaper at scale and gives you more control over exactly what's collected. Many orgs run both — Prometheus/Grafana for infra + SLO dashboards, APM for deep application tracing when something needs code-level investigation.

- [ ] What are MTTD and MTTR, and how do you actually reduce each?

IF ASKED, SAY THIS
MTTD (mean time to detect) is reduced by SLO-based alerting that fires faster than slow threshold-only alerts, synthetic monitoring that catches issues before customer reports do, and closing genuine coverage blind spots. MTTR (mean time to resolve) is reduced by good runbooks, clear on-call ownersh

MTTD (mean time to detect) is reduced by SLO-based alerting that fires faster than slow threshold-only alerts, synthetic monitoring that catches issues before customer reports do, and closing genuine coverage blind spots. **MTTR** (mean time to resolve) is reduced by good runbooks, clear on-call ownership and routing, alerts that link straight to the relevant dashboard/trace instead of just saying 'CPU high', practiced incident response (game days build this muscle), and automated remediation for known patterns — auto-restart, auto-scale, circuit breakers — that resolve without ever paging a human.

- [ ] How do you design alerting to actually reduce noise instead of adding to it?

IF ASKED, SAY THIS

Alert on symptoms — SLO burn rate, user-facing error rate — rather than every possible cause; don't page on every CPU spike. Use multi-window multi-burn-rate alerts to filter transient blips while still catching real sustained problems. Deduplicate and group related alerts so one incident produces o

Alert on symptoms — SLO burn rate, user-facing error rate — rather than every possible cause; don't page on every CPU spike. Use multi-window multi-burn-rate alerts to filter transient blips while still catching real sustained problems. Deduplicate and group related alerts so one incident produces one page, not fifty pod-level alerts for a single node failure. Use severity tiers — page for user-impacting, ticket for informational. And regularly review and prune alerts that fire often without ever being actionable — that's the actual definition of alert fatigue.

- [ ] How do you use logs, metrics, and traces together during root cause analysis?

IF ASKED, SAY THIS
Metrics tell you something is wrong and roughly where/when — e.g. latency spiked on service X at 14:32. Traces show the exact path of a specific request and where time was actually spent across that transaction. Logs give you the detailed 'why' — the specific error message or business-logic detail f

Metrics tell you *something* is wrong and roughly where/when — e.g. latency spiked on service X at 14:32. Traces show the exact path of a specific request and where time was actually spent across that transaction. Logs give you the detailed 'why' — the specific error message or business-logic detail for that failing request.

Effective RCA moves top-down: dashboard → drill into a slow/failed trace → jump to the correlated logs for that exact trace ID → find root cause. Without correlation IDs linking all three together, this becomes manual, slow guesswork instead of a guided drill-down.

- [ ] How would you build a service health dashboard meant for non-engineering stakeholders?

IF ASKED, SAY THIS
Lead with business-relevant framing, not raw infra graphs: SLO compliance / error-budget-remaining as the headline, ideally traffic-light style; key business transaction volumes and success rates; current or recent incidents; and the trend over time — is reliability improving or degrading quarter ov

Lead with business-relevant framing, not raw infra graphs: SLO compliance / error-budget-remaining as the headline, ideally traffic-light style; key business transaction volumes and success rates; current or recent incidents; and the trend over time — is reliability improving or degrading quarter over quarter. Keep it audience-appropriate: an exec dashboard shows SLO trends and incident counts, an on-call engineer's dashboard shows detailed per-service golden signals. A wall of forty undifferentiated graphs isn't a dashboard, it's noise — curate to what actually drives a decision.

- [ ] How would you set up distributed tracing with OpenTelemetry across microservices?

IF ASKED, SAY THIS
Instrument each service with the OTel SDK — auto-instrumentation for common frameworks plus manual spans around critical business logic — and propagate trace context (the W3C traceparent header) across every service-to-service call, including through message queues where it's easy to forget. Export

Instrument each service with the OTel SDK — auto-instrumentation for common frameworks plus manual spans around critical business logic — and propagate trace context (the W3C `traceparent` header) across every service-to-service call, including through message queues where it's easy to forget. Export spans to an OTel Collector, which forwards to a backend like Jaeger, Tempo, or a vendor APM.

Get the sampling strategy right: **tail-based sampling** that keeps 100% of error and slow traces but samples normal traffic keeps you from losing exactly the traces that matter most, while still controlling data volume and cost.

- [ ] How do you correlate an alert with a specific deployment or config change?

IF ASKED, SAY THIS
Tag dashboards and metrics with deployment events — most APM tools (Datadog APM, for instance) support deployment markers overlaid directly on graphs, fed automatically by a CI/CD webhook. Add a 'recent changes' panel to the incident dashboard, pulling from your CI/CD or change-management system. An

Tag dashboards and metrics with deployment events — most APM tools (Datadog APM, for instance) support deployment markers overlaid directly on graphs, fed automatically by a CI/CD webhook. Add a 'recent changes' panel to the incident dashboard, pulling from your CI/CD or change-management system. And make it the automatic *first* triage step in any incident to check what deployed or changed in the recent window — correlation isn't causation, but it's the fastest hypothesis to check or rule out before going deeper.

- [ ] How would observability design differ between a batch settlement pipeline and a real-time payment API?

IF ASKED, SAY THIS
Real-time API: continuous per-request metrics via the RED method, tight alerting windows measured in minutes. Batch settlement: job-level metrics instead — start/end time, records processed, records failed or reconciled, duration vs. the SLA deadline — plus completeness checks like row counts in vs.

Real-time API: continuous per-request metrics via the RED method, tight alerting windows measured in minutes. Batch settlement: job-level metrics instead — start/end time, records processed, records failed or reconciled, duration vs. the SLA deadline — plus completeness checks like row counts in vs. out and checksum/reconciliation totals matching the source system.

You'd alert on 'job didn't start', 'job still running past expected completion', or 'reconciliation mismatch' rather than per-transaction latency, because the unit of concern is the whole batch run, not any individual record.

- [ ] How does the Datadog Agent collect metrics, traces, and logs, and how do integrations fit in?

IF ASKED, SAY THIS
A single unified Agent runs per host (or as a DaemonSet in Kubernetes/a sidecar in ECS) and handles all three signal types: it collects host-level metrics (CPU, memory, disk) natively, ships application traces via APM instrumentation (auto-instrumentation libraries or OpenTelemetry), and tails/forwa

A single unified Agent runs per host (or as a DaemonSet in Kubernetes/a sidecar in ECS) and handles all three signal types: it collects host-level metrics (CPU, memory, disk) natively, ships application traces via APM instrumentation (auto-instrumentation libraries or OpenTelemetry), and tails/forwards logs when log collection is enabled for a given source. **Integrations** are pre-built checks for specific technologies (PostgreSQL, Nginx, AWS services via the API-based integration rather than the Agent) that the Agent runs on a schedule to pull technology-specific metrics without writing custom collection code — enabling one is usually just a config file drop and an Agent restart, which is a big part of why Datadog's time-to-first-dashboard is fast compared to building the same coverage in a pure OSS Prometheus stack.

- [ ] What are the main Datadog monitor types, and how do you route alerts sensibly?

IF ASKED, SAY THIS
Metric monitors alert on a threshold or anomaly against a metric (CPU above X for Y minutes). **APM monitors** watch service-level indicators like error rate or latency percentiles directly from trace data. **Log monitors** alert on the rate of logs matching a query (a spike in ERROR-level logs)

Metric monitors alert on a threshold or anomaly against a metric (CPU above X for Y minutes). **APM monitors** watch service-level indicators like error rate or latency percentiles directly from trace data. **Log monitors** alert on the rate of logs matching a query (a spike in ERROR-level logs). **Composite monitors** combine several underlying monitors with boolean logic — useful for reducing noise by only paging when multiple related signals fire together, rather than getting three separate pages for one underlying incident. Routing goes through notification channels tied to the monitor — Slack for lower-urgency, PagerDuty/Opsgenie for anything that should wake someone up — and tagging monitors by service/team means routing (and later, filtering dashboards) follows ownership automatically instead of being hardcoded per monitor.

- [ ] What would you actually put on a core SRE dashboard for a production service?

IF ASKED, SAY THIS
The RED method covers request-driven services well: **Rate** (requests per second), **Errors** (error rate/count), **Duration** (latency, as percentiles — p50/p95/p99, not just an average, since an average hides the tail that's actually hurting users). For the infrastructure underneath, the USE meth

The RED method covers request-driven services well: **Rate** (requests per second), **Errors** (error rate/count), **Duration** (latency, as percentiles — p50/p95/p99, not just an average, since an average hides the tail that's actually hurting users). For the infrastructure underneath, the USE method fills the gap: **Utilization**, **Saturation**, **Errors** per resource (CPU, memory, disk, network, connection pools). Together those answer 'is the service serving traffic well' and 'is the infrastructure underneath it about to become the bottleneck' — which is a more actionable pairing than a dashboard that's just a wall of every metric the Agent happens to emit.

- [ ] Splunk and Datadog can overlap — practically, where does each tend to be the stronger tool?

IF ASKED, SAY THIS
Splunk's roots and strongest use case are log-centric — deep, flexible search across huge volumes of raw log/event data, and it's very commonly the tool of record for security/SIEM use cases and compliance-driven log retention, which is a big part of why it shows up in regulated environments. Datado

Splunk's roots and strongest use case are log-centric — deep, flexible search across huge volumes of raw log/event data, and it's very commonly the tool of record for security/SIEM use cases and compliance-driven log retention, which is a big part of why it shows up in regulated environments. Datadog's strength is full-stack observability that ties infrastructure metrics, APM traces, and logs together with less setup effort, and its dashboarding/alerting UX is generally faster to get real-time operational visibility from. In practice, a lot of orgs run both for different jobs rather than picking one — Datadog for day-to-day operational monitoring and incident response, Splunk for security/audit-log search and long-term compliance retention — rather than treating it as a strict either/or.

- [ ] How would you set up synthetic monitoring and tie it back to an SLO?

IF ASKED, SAY THIS
Synthetic checks (Datadog Synthetics, or an equivalent) simulate real user paths on a schedule from multiple regions — a login flow, a critical API endpoint — catching outages even when there's zero real traffic to generate a metric-based alert, which matters for low-traffic periods or services with

Synthetic checks (Datadog Synthetics, or an equivalent) simulate real user paths on a schedule from multiple regions — a login flow, a critical API endpoint — catching outages even when there's zero real traffic to generate a metric-based alert, which matters for low-traffic periods or services with no organic load at 3 AM. Tied to an SLO, the synthetic check's success rate becomes an input to (or a proxy for) availability — if the SLO is '99.9% successful checkout requests,' the synthetic can be the canary that fires *before* the SLO's error budget actually burns meaningfully, giving an earlier warning than waiting for the aggregate real-traffic metric to cross a threshold. The practical setup: run the check frequently enough (every 1-5 minutes) to catch a real outage quickly, from more than one region so a single region's network blip doesn't false-positive, and alert on consecutive failures rather than a single miss.

- [ ] How do you design a severity/priority tier system for alerts, and why does having one matter?

IF ASKED, SAY THIS

A clear tier system (P1: page immediately, wakes someone up / P2: notify during business hours / P3: log/dashboard only, no notification) prevents the single biggest cause of alert fatigue -- everything treated as equally urgent. Without explicit tiers, teams either page for everything (burnout) or tune out alerts entirely (missed real incidents) -- there's no stable middle ground without a deliberate system.

The framing worth leading with: alert severity should be defined by USER/BUSINESS IMPACT, not by which metric crossed a threshold -- "disk at 85%" and "checkout is down" should never share a severity tier just because both are technically alerts.

- [ ] How does alert ROUTING actually work end to end -- from a metric crossing a threshold to a human's phone buzzing?

IF ASKED, SAY THIS

Monitoring tool (Prometheus Alertmanager, Grafana, Splunk, CloudWatch) evaluates a rule and fires an alert -> routes to an incident-management/on-call platform (PagerDuty, Opsgenie, or a native equivalent) which owns escalation policy and schedule -> that platform pages the current on-call person via their configured channel (push, SMS, phone call) -> if unacknowledged within a set window, auto-escalates to the next person/tier. The monitoring tool detects; the on-call platform owns WHO gets notified and WHEN it escalates.

Keeping these two responsibilities conceptually separate (detection vs. routing/escalation) is worth stating explicitly -- a common design mistake is trying to make the monitoring tool itself own escalation logic, when a dedicated on-call platform's escalation policies (multi-tier, schedule-aware, override-friendly) do it far better.

- [ ] What's a systematic (not ad-hoc) process for reducing alert fatigue on a noisy team?

IF ASKED, SAY THIS

Audit first, don't guess: pull actual alert volume by rule over a real window (a month) and rank by frequency -- the noisiest 10% of rules usually account for most of the pain. For each noisy rule: is it actionable (does firing require a human to actually DO something)? If not, downgrade it to a dashboard/log-only signal, not a page. Tune thresholds against real historical data rather than a guessed number. Add deduplication/grouping so 50 related alerts from one root cause become one notification, not 50.

The "is it actionable" filter is the single highest-leverage question in this whole process -- a genuinely huge fraction of noisy alerts fire for conditions where the response is always "nothing, it resolves itself" or "wait and see," which means it should never have been a page-worthy alert in the first place.

- [ ] What's a composite/multi-condition alert, and why is it usually better than a single-metric threshold?

IF ASKED, SAY THIS

A composite alert fires only when MULTIPLE conditions are true together (e.g. error rate above X AND request volume above a minimum, so a single slow request during near-zero traffic doesn't trigger a false page) -- versus a single-metric alert that fires on one number crossing one line regardless of context. Composite alerts are harder to author but dramatically reduce false positives from edge cases a single metric can't distinguish.

The volume-gate example (error rate alone vs. error rate AND minimum traffic) is a concrete, commonly-asked pattern -- a 100% error rate on 1 request during a quiet 3am window is noise, not a P1, and a well-designed composite alert knows the difference where a naive threshold doesn't.

- [ ] How do you actually test that a new alert works correctly before trusting it in production?

IF ASKED, SAY THIS

Deliberately trigger the condition in a non-production environment first if possible (or a controlled test in prod for something safe to simulate) and confirm it fires, routes to the right person, and the runbook/link attached to it is accurate and current. Also deliberately verify the INVERSE -- that it does NOT fire on normal/expected variation, by watching it silently for a period before it's allowed to actually page anyone.

The silent-observation-period practice is worth naming specifically -- shipping a new alert straight into paging mode without first confirming it doesn't false-positive on normal noise is exactly how a new "safety" alert becomes next week's fatigue complaint instead.

- [ ] How would you design a CloudWatch-centric monitoring strategy for an enterprise integration platform spanning API Gateway, Lambda, ECS, and third-party connectivity?

IF ASKED, SAY THIS

Per-service dashboards from the relevant golden signals (API Gateway: 4xx/5xx, latency; Lambda: invocations, errors, duration, throttles, concurrency; ECS: task health, CPU/memory), plus a higher-level "integration health" dashboard rolling up end-to-end flow status — stuck transactions, ERP connectivity status — since a stakeholder or on

Per-service dashboards from the relevant golden signals (API Gateway: 4xx/5xx, latency; Lambda: invocations, errors, duration, throttles, concurrency; ECS: task health, CPU/memory), plus a higher-level **"integration health"** dashboard rolling up end-to-end flow status — stuck transactions, ERP connectivity status — since a stakeholder or on-call engineer needs the business-level view, not twelve dashboards to mentally correlate themselves.

- [ ] What would you actually use CloudTrail for operationally, beyond just "it's required for compliance"?

IF ASKED, SAY THIS

Security investigation (who did what, when, from where — essential for any incident involving unexpected changes or suspicious access), operational troubleshooting (a resource was deleted or a config changed and nobody remembers — CloudTrail answers definitively rather than relying on tribal memory), and as an input to automated detection

Security investigation (who did what, when, from where — essential for any incident involving unexpected changes or suspicious access), operational troubleshooting (a resource was deleted or a config changed and nobody remembers — CloudTrail answers definitively rather than relying on tribal memory), and as an input to automated detection — CloudTrail events feeding EventBridge rules that alert on sensitive actions (an SCP modified, logging disabled, a new admin IAM user created) near-real-time rather than discovered days later in a log review.

- [ ] How would you design centralized logging across a multi-account, multi-service architecture?

IF ASKED, SAY THIS

A dedicated logging/security account receiving CloudTrail and CloudWatch Logs from all member accounts (cross-account log delivery, or a centralized S3 bucket with appropriate policies) rather than logs scattered per-account, where a compromised or accidentally-deleted account could destroy its own audit trail. Structured logging (JSON, n

A dedicated logging/security account receiving CloudTrail and CloudWatch Logs from all member accounts (cross-account log delivery, or a centralized S3 bucket with appropriate policies) rather than logs scattered per-account, where a compromised or accidentally-deleted account could destroy its own audit trail.

Structured logging (JSON, not free-text) from applications specifically, so downstream tooling — CloudWatch Logs Insights, or an external platform like Splunk/OpenSearch/Datadog — can actually query and correlate rather than grep-and-hope.

- [ ] When would you bring in a third-party observability platform (Grafana, Prometheus, Datadog, Splunk, OpenSearch) on top of native CloudWatch, rather than staying CloudWatch-native?

IF ASKED, SAY THIS

When you need one pane of glass across multiple clouds or cloud + on-prem (relevant again given hybrid ERP connectivity), when CloudWatch's dashboarding/query capability genuinely falls short (Grafana's dashboarding and PromQL are considerably more flexible), or when the org already has organizational investment in one of these and consis

When you need one pane of glass across multiple clouds or cloud + on-prem (relevant again given hybrid ERP connectivity), when CloudWatch's dashboarding/query capability genuinely falls short (Grafana's dashboarding and PromQL are considerably more flexible), or when the org already has organizational investment in one of these and consistency across their broader estate matters more than being purely AWS-native.

The tradeoff is real operational cost — running Prometheus, or Datadog/Splunk licensing — that needs to be justified by an actual gap CloudWatch doesn't close.

- [ ] How would you design alerting so the right people get paged for the right things across a platform this broad?

IF ASKED, SAY THIS

Alert routing tied to actual ownership (the team/rotation that can act on a given alert, not a single catch-all channel everyone half-monitors), severity-tiered (page for business-impacting, ticket for degraded-but-functioning, dashboard-only for informational), and — specifically for an integration-heavy platform — alerts that distinguis

Alert routing tied to actual ownership (the team/rotation that can act on a given alert, not a single catch-all channel everyone half-monitors), severity-tiered (page for business-impacting, ticket for degraded-but-functioning, dashboard-only for informational), and — specifically for an integration-heavy platform — alerts that distinguish **"our infrastructure is unhealthy"** from **"a third-party/ERP dependency is unhealthy,"** since the response and owning team likely differ even though both look like "the integration is failing" naively.

- [ ] What's your approach to using observability data to actually drive architectural decisions, not just react to incidents?

IF ASKED, SAY THIS

Treat recurring alert patterns and dashboard trends as design-review input, not just noise to triage — a Lambda consistently near its concurrency limit is a signal to revisit the architecture, not just something to keep silencing. Regular review of what's actually generating cost, latency, or error-budget consumption surfaces the pieces d

Treat recurring alert patterns and dashboard trends as design-review input, not just noise to triage — a Lambda consistently near its concurrency limit is a signal to revisit the architecture, not just something to keep silencing. Regular review of what's actually generating cost, latency, or error-budget consumption surfaces the pieces due for rework before they cause a real incident — the proactive complement to incident-driven improvement.

Prometheus

- [ ] What is Prometheus and how does its pull model work?

IF ASKED, SAY THIS
Prometheus is a monitoring system and time-series database that pulls metrics by scraping targets' `/metrics` HTTP endpoints on an interval, discovering targets statically or via service discovery like Kubernetes. The pull model lets Prometheus control timing and detect down targets naturally via fa

- Prometheus is an open-source monitoring system and time-series database. It scrapes (pulls) metrics over HTTP from targets' `/metrics` endpoints at a set interval.
- Targets are discovered statically or via service discovery (Kubernetes, Consul, EC2). Each scrape stores samples in the local TSDB.
- Pull benefits: Prometheus controls scrape timing, can detect a target being down (scrape fails), and targets don't need to know about Prometheus. Short-lived jobs that can't be scraped push to a Pushgateway instead.

- [ ] Explain the Prometheus metric types: counter, gauge, histogram, summary.

IF ASKED, SAY THIS
A counter only increases and is queried via rate() — used for totals like requests or errors. A gauge moves up and down for values like memory or queue depth. A histogram buckets observations (e.g. latencies) so you can compute quantiles server-side with histogram_quantile(), while a summary compute

- Counter: monotonically increasing value (requests_total, errors_total). You query its rate, never the raw value. Resets to 0 on restart.
- Gauge: a value that goes up and down (memory_usage, queue_depth, temperature).
- Histogram: samples observations into configurable buckets (request durations), enabling quantile estimation server-side via histogram_quantile().
- Summary: like histogram but computes quantiles client-side; can't be aggregated across instances. Prefer histograms for aggregatable latency.

- [ ] What are labels and how do they affect cardinality?

IF ASKED, SAY THIS
Labels are key/value dimensions, and every unique combination of label values is its own time series. Cardinality is the count of unique series, and it's the main scaling risk: putting unbounded values like user IDs, request IDs, or full URLs into labels explodes series count and exhausts memory. Th

- Labels are key/value dimensions on a metric (method, status, path). Each unique label-set is a separate time series.
- Cardinality = number of unique series. Putting unbounded values in labels (user IDs, request IDs, full URLs, timestamps) explodes series count, exhausting memory and slowing queries.
- Keep labels low-cardinality and bounded; never label by something with millions of values.

- [ ] Why do you use rate() on counters, and what's the difference between rate and irate?

IF ASKED, SAY THIS
Counters only increase, so you query their per-second growth rather than the raw value. rate() computes the average per-second increase over a range and handles resets, giving smooth values ideal for alerts and dashboards. irate() uses only the last two samples for a responsive but spiky result suit

- Counters only increase, so the raw value is meaningless; rate() computes the per-second average increase over a time window, handling counter resets automatically.
- rate() averages over the whole range (smooth, good for alerting/graphs). irate() uses only the last two samples (responsive, spiky, good for fast-moving graphs).
- Rule of thumb: rate() for alerts/dashboards, irate() for high-resolution short-term graphs. Always use a range at least 4x the scrape interval.

```
rate(http_requests_total{status=~"5.."}[5m])
/ rate(http_requests_total[5m])   # error ratio
```

- [ ] How do recording rules and alerting rules work?

IF ASKED, SAY THIS
Recording rules precompute frequent or expensive PromQL expressions into new time series, speeding dashboards and reusing logic. Alerting rules evaluate an expression and fire an alert once it stays true for the configured `for:` duration, sending it to Alertmanager, which then groups, deduplicates,

- Recording rules precompute expensive/frequent PromQL expressions and store them as new series — speeding up dashboards and reusing complex queries.
- Alerting rules evaluate a PromQL expression; when it's true for a `for:` duration, an alert fires and is sent to Alertmanager.

- Alertmanager handles routing, grouping, deduplication, silencing, and inhibition before notifying (Slack, PagerDuty, email).

```
groups:
- name: slo
  rules:
  - alert: HighErrorRate
    expr: job:errors:ratio > 0.05
    for: 10m
    labels: { severity: page }
```

- [ ] How do you scale Prometheus and achieve long-term storage / HA?

IF ASKED, SAY THIS
A single Prometheus isn't clustered and keeps data locally with bounded retention, so for HA you run two identical instances scraping the same targets. For long-term storage and a global view you add Thanos or Cortex/Mimir, which provide object-storage backends, deduplication across HA pairs, downsa

- A single Prometheus is not clustered and stores data locally with limited retention. For HA, run two identical Prometheus servers scraping the same targets.
- For long-term storage and global query, use Thanos or Cortex/Mimir: they add object-storage backends, deduplication across HA pairs, downsampling, and a global query view.
- Federation lets a higher-level Prometheus scrape aggregated metrics from others, but Thanos/Mimir is the modern approach at scale.

- [ ] Prometheus pull vs push — why pull, and when do you still need a push gateway?

IF ASKED, SAY THIS
Pull means Prometheus scrapes targets on its own schedule, which gives you a free up/down signal for free (a target that can't be scraped is visibly down) and keeps configuration centralized in Prometheus rather than scattered across every app. The pushgateway exists for the one case pull doesn't fi

Pull means Prometheus scrapes targets on its own schedule, which gives you a free up/down signal for free (a target that can't be scraped is visibly down) and keeps configuration centralized in Prometheus rather than scattered across every app. The pushgateway exists for the one case pull doesn't fit: short-lived batch jobs that finish before a scrape would ever catch them — a nightly cron job pushes its result metric before exiting, and Prometheus scrapes the pushgateway itself. It's explicitly not meant as a general push-metrics pattern; overusing it turns the pushgateway into a single point of failure and a metrics dumping ground.

- [ ] Write a PromQL query to alert on error rate above 5% over 5 minutes.

IF ASKED, SAY THIS
rate() needs a counter and a range vector — it computes per-second average increase over the window, which is why 5xx count alone isn't enough; you need it as a ratio of total requests so the threshold means the same thing at low and high traffic. Worth pairing with for: 5m in the alert rule itself

```
(
  sum(rate(http_requests_total{status=~"5.."}[5m]))
  /
  sum(rate(http_requests_total[5m]))
) > 0.05
```

`rate()` needs a counter and a range vector — it computes per-second average increase over the window, which is why 5xx count alone isn't enough; you need it as a ratio of total requests so the threshold means the same thing at low and high traffic. Worth pairing with `for: 5m` in the alert rule itself so a single noisy minute doesn't page anyone.

- [ ] Counter vs gauge vs histogram vs summary — what's each for?

IF ASKED, SAY THIS
Counter only goes up (requests served, errors) — always paired with rate() , never read raw. Gauge goes up and down (current memory usage, queue depth, active connections). Histogram buckets observations (request duration) into configurable buckets server-side, letting you compute quantiles later, a

Counter only goes up (requests served, errors) — always paired with `rate()`, never read raw. Gauge goes up and down (current memory usage, queue depth, active connections). Histogram buckets observations (request duration) into configurable buckets server-side, letting you compute quantiles later, aggregate across instances, and it's what powers `histogram_quantile()`. Summary computes quantiles client-side at scrape time — cheaper to query but can't be aggregated across instances (you can't average two different services' p99s meaningfully). Default to histogram for latency; it's the one that actually composes.

- [ ] How does Prometheus service discovery work in Kubernetes?

IF ASKED, SAY THIS
Native Kubernetes SD watches the API server for pods/services/endpoints matching label selectors and auto-generates scrape targets — no static target list to maintain. In practice, most clusters run the Prometheus Operator, where you define ServiceMonitor / PodMonitor CRDs per workload (port, path,

Native Kubernetes SD watches the API server for pods/services/endpoints matching label selectors and auto-generates scrape targets — no static target list to maintain. In practice, most clusters run the Prometheus Operator, where you define `ServiceMonitor` / `PodMonitor` CRDs per workload (port, path, interval), and the operator translates those into Prometheus's scrape config automatically. `kube-state-metrics` is the separate piece that exposes cluster object state itself (deployment replica counts, pod phase) as metrics — distinct from node/cAdvisor metrics, which cover resource usage.

- [ ] How would you design an on-call Grafana dashboard — what goes on it and why?

IF ASKED, SAY THIS
Lead with the RED method for request-driven services (Rate, Errors, Duration) or USE for resources (Utilization, Saturation, Errors) — whichever question the on-call engineer needs answered in the first 30 seconds: "is it serving traffic, is it erroring, is it slow." Top row: the 2-3 SLI panels tied

Lead with the RED method for request-driven services (Rate, Errors, Duration) or USE for resources (Utilization, Saturation, Errors) — whichever question the on-call engineer needs answered in the first 30 seconds: "is it serving traffic, is it erroring, is it slow." Top row: the 2-3 SLI panels tied directly to the service's SLO with the error-budget burn rate visible. Below that: dependency health (DB connections, downstream call latency) so you can tell "is it us or a dependency" fast. Deliberately exclude anything that isn't actionable during an incident — a dashboard with 40 panels is worse than one with 6 the on-call engineer has memorized.

- [ ] How do you avoid alert fatigue when writing Grafana/Prometheus alerting rules?

IF ASKED, SAY THIS
Alert on symptoms (user-facing SLO burn) not causes (a single node's CPU) — causes generate noise, symptoms generate signal. Always pair a threshold with a for duration so single-sample blips don't page. Route by severity properly — page for things that need a human right now, route everything else

Alert on symptoms (user-facing SLO burn) not causes (a single node's CPU) — causes generate noise, symptoms generate signal. Always pair a threshold with a `for` duration so single-sample blips don't page. Route by severity properly — page for things that need a human right now, route everything else to a ticket/Slack channel for daytime triage. And periodically audit: any alert that's been acknowledged-and-ignored more than a couple times without action is either mistuned or shouldn't page — that's the practical definition of noise.

- [ ] What is OpenTelemetry, and what are its three core signal types?

IF ASKED, SAY THIS
A CNCF standard (graduated in 2026) for generating and exporting telemetry in a vendor-neutral way — instrument your code once with OTel SDKs/auto-instrumentation, and send data to whatever backend you want without re-instrumenting. The three signals: traces (a request's path across services, as a t

A CNCF standard (graduated in 2026) for generating and exporting telemetry in a vendor-neutral way — instrument your code once with OTel SDKs/auto-instrumentation, and send data to whatever backend you want without re-instrumenting. The three signals: traces (a request's path across services, as a tree of spans), metrics (numeric measurements over time), and logs (the newest of the three to get first-class OTel treatment, with trace-context correlation baked in).

- [ ] Explain the OpenTelemetry Collector — why run it as a separate component instead of exporting straight from your app?

IF ASKED, SAY THIS
The Collector sits between your instrumented apps and your backend(s), with three stages: receivers (accept data — OTLP, Prometheus scrape, etc.), processors (batch, filter, redact PII, add resource attributes, sample), and exporters (send to one or many backends — Prometheus, Tempo/Jaeger, an ELK/O

The Collector sits between your instrumented apps and your backend(s), with three stages: receivers (accept data — OTLP, Prometheus scrape, etc.), processors (batch, filter, redact PII, add resource attributes, sample), and exporters (send to one or many backends — Prometheus, Tempo/Jaeger, an ELK/OpenSearch stack, a commercial vendor). Running it separately decouples apps from backend choice (swap backends by changing collector config, not redeploying every service), centralizes sampling and PII scrubbing in one place instead of duplicating that logic per-app, and buffers/retries so a backend outage doesn't back-pressure your application's request path.

- [ ] What are OpenTelemetry's GenAI semantic conventions, and why do LLM calls need their own convention separate from standard HTTP conventions?

IF ASKED, SAY THIS
The GenAI SIG extended OTel with a gen_ai.* attribute namespace specifically because LLM calls don't fit the old APM assumptions: the same input can produce a different output (non-determinism breaks "reproduce the bug with the same request"), cost tracks token count rather than request count, and a

The GenAI SIG extended OTel with a `gen_ai.*` attribute namespace specifically because LLM calls don't fit the old APM assumptions: the same input can produce a different output (non-determinism breaks "reproduce the bug with the same request"), cost tracks token count rather than request count, and a single user request can fan out into a whole tree of nested spans — LLM calls, tool/function calls, retrieval steps — that needs to render as one coherent trace, not a flat list. The conventions standardize span/attribute names for model, token counts (input/output/reasoning), and now

extend into agent orchestration and MCP tool-calling instrumentation, so telemetry stays portable across OpenAI/Anthropic/Bedrock and across observability backends instead of every team inventing its own attribute names.

- [ ] How would you instrument an LLM or agent call so you can debug a slow, expensive request in production?

> **IF ASKED, SAY THIS**
> Each LLM invocation and each tool/function call becomes its own child span under the request's trace, carrying gen_ai.* attributes — model name, input/output token counts, and critically time-to-first-token (TTFT) as a separate measurement from total latency, since a slow TTFT points at model/provid

Each LLM invocation and each tool/function call becomes its own child span under the request's trace, carrying `gen_ai.*` attributes — model name, input/output token counts, and critically time-to-first-token (TTFT) as a separate measurement from total latency, since a slow TTFT points at model/provider queuing while a slow total-with-fast-TTFT points at a long generation or a chained tool call. One practical trap: don't put the raw prompt/completion text in span attributes — they're indexed, size-limited, and a PII/compliance risk; the convention puts that content in span events instead, which the Collector can filter or drop before export without touching application code. For multi-agent flows, the piece that actually breaks by default is trace context propagation across the agent-to-subagent boundary — most frameworks don't do it automatically, so you end up with disconnected trace fragments instead of one tree unless you wire it explicitly.

- [ ] Trace vs span, and how does context propagation work across services?

> **IF ASKED, SAY THIS**
> A trace is the whole request's journey; a span is one unit of work within it (one HTTP call, one DB query, one function). Spans form a tree via parent-child relationships, and propagation is how that tree survives crossing a network boundary — the caller injects a trace context (trace ID + parent sp

A trace is the whole request's journey; a span is one unit of work within it (one HTTP call, one DB query, one function). Spans form a tree via parent-child relationships, and propagation is how that tree survives crossing a network boundary — the caller injects a trace context (trace ID + parent span ID, standardized as the W3C `traceparent` header) into the outgoing request headers, and the callee's instrumentation reads it back out to continue the same trace instead of starting a new, disconnected one. Message queues need the same treatment (context stuffed into message attributes) or async work loses trace continuity — a common gap in event-driven architectures.

- [ ] How do you correlate logs, metrics, and traces for a single request?

> **IF ASKED, SAY THIS**
> Trace ID is the join key. Structured logging that includes the active trace ID/span ID on every log line (most OTel logging integrations inject this automatically) lets you jump from a trace in Tempo/Jaeger straight to the exact log lines for that request in your log backend, and exemplars link a me

Trace ID is the join key. Structured logging that includes the active trace ID/span ID on every log line (most OTel logging integrations inject this automatically) lets you jump from a trace in Tempo/Jaeger straight to the exact log lines for that request in your log backend, and exemplars link a metric data point (e.g. one slow bucket in a latency histogram) back to a specific trace that fell in it — that's the practical bridge from "the p99 spiked" to "here's the actual slow request that caused it."

- [ ] What's cardinality explosion in a metrics system, and how do you prevent it?

> **IF ASKED, SAY THIS**
> Every unique combination of label values creates a new time series — putting something high-cardinality in a label (user ID, raw request path with IDs in it, a UUID) multiplies series count and can take down Prometheus's memory or blow up your metrics bill on a hosted backend. Prevention: never labe

Every unique combination of label values creates a new time series — putting something high-cardinality in a label (user ID, raw request path with IDs in it, a UUID) multiplies series count and can take down Prometheus's memory or blow up your metrics bill on a hosted backend. Prevention: never label with unbounded values, normalize paths (`/users/:id` not `/users/12345`) before they hit a metric label, keep high-cardinality data in traces/logs where it belongs instead, and set recording rules to pre-aggregate away dimensions you don't actually query on.

- [ ] Describe Prometheus architecture — why is it pull-based?

> **IF ASKED, SAY THIS**
> Prometheus scrapes HTTP /metrics endpoints on a schedule, instrumented apps directly, or exporters like node_exporter / postgres_exporter for things that can't instrument themselves, and stores samples in a local time-series database. You query with PromQL; alerting rules evaluate PromQL expressions

Prometheus scrapes HTTP `/metrics` endpoints on a schedule, instrumented apps directly, or exporters like `node_exporter` / `postgres_exporter` for things that can't instrument themselves, and stores samples in a local time-series database. You query with PromQL; alerting rules evaluate PromQL expressions and fire to Alertmanager, which handles dedup, grouping, and routing. Service discovery, Kubernetes or Consul, finds targets dynamically as they come and go. Pull matters because Prometheus controls scrape timing and immediately knows a target is down when a scrape fails — `up == 0` is a built-in signal you get for free, which a push model doesn't give you as cleanly.

> **Remember:** Pull → Prometheus knows a target is down the instant a scrape fails (`up==0`) · For short-lived/batch jobs that can't be scraped, the answer is Pushgateway — name it as the exception · Prometheus is single-node by design — that's the setup for the HA question

- [ ] How do you run Prometheus highly available and at scale?

IF ASKED, SAY THIS
Start with the simplest layer: an HA pair — two identical Prometheus instances scrape the same targets independently, and Alertmanager's clustering dedupes their alerts. That solves availability but gives you two separate databases with no shared long-term view. Federation , a global Prometheus scra

Start with the simplest layer: an **HA pair** — two identical Prometheus instances scrape the same targets independently, and Alertmanager's clustering dedupes their alerts. That solves availability but gives you two separate databases with no shared long-term view. **Federation**, a global Prometheus scraping aggregated series from leaf instances, works for a small rollup but doesn't scale as a real data plane. The modern pattern is **remote write**: each Prometheus streams samples to central long-term storage like Thanos, Mimir, or Cortex. The **Thanos sidecar model** specifically ships TSDB blocks to object storage (S3/Azure Blob); a Query component fans out across sidecars and a store gateway, and a Compactor downsamples old data for cheap long retention.

Remember: Layer the answer: HA pair (availability) → federation (limited rollup) → remote write / Thanos (the real answer) · This is the exact question you needed help with before — lead with 'HA pair' then go straight to Thanos sidecar + object storage · Scaling hygiene either way: control cardinality, shard scrape jobs, keep local retention short (~15d)

- [ ] How does Alertmanager actually prevent alert fatigue?

IF ASKED, SAY THIS
Four mechanisms, and naming all four signals depth: grouping batches related alerts into one notification instead of fifty — fifty pods on a dead node becomes one page, not fifty. Routing sends alerts to the right place by label, severity=page to PagerDuty, severity=ticket to Jira. Inhibition suppre

Four mechanisms, and naming all four signals depth: **grouping** batches related alerts into one notification instead of fifty — fifty pods on a dead node becomes one page, not fifty. **Routing** sends alerts to the right place by label, severity=page to PagerDuty, severity=ticket to Jira. **Inhibition** suppresses downstream alerts when a higher-level one already fired — a node-down alert suppresses its pods' alerts automatically. **Silences** mute known noise during planned maintenance. Run Alertmanager clustered so dedup survives an instance loss, and keep a dead-man's-switch alert, one that's always firing, so its absence tells you the alerting pipeline itself died.

Remember: Grouping / Routing / Inhibition / Silences — four distinct mechanisms, don't collapse them into one answer · Dead-man's-switch is the detail almost nobody mentions unprompted · Prometheus decides *what's* wrong; Alertmanager decides *who hears about it and how* — say that division of labor explicitly

- [ ] Recording rules vs alerting rules — what's the difference?

IF ASKED, SAY THIS
Recording rules precompute an expensive or frequently-used PromQL query into a new stored time series, for example job:http_error_ratio:5m — this makes dashboards fast and gives everyone the same SLI definition to alert on instead of five slightly different ad hoc queries. Alerting rules evaluate an

Recording rules precompute an expensive or frequently-used PromQL query into a new stored time series, for example `job:http_error_ratio:5m` — this makes dashboards fast and gives everyone the same SLI definition to alert on instead of five slightly different ad hoc queries. Alerting rules evaluate an expression and, once it's true for at least the `for:` duration, fire an alert with labels and annotations attached. That `for:` clause is your first line of defense against flapping — a five-second spike shouldn't page anyone.

Remember: Recording = precompute for speed + consistency. Alerting = evaluate + fire. · `for:` duration is the flap-suppressor — always mention it · A shared recording rule for your SLI stops five teams alerting on five slightly different definitions of 'error rate'

- [ ] What PromQL functions come up constantly, and what's the difference between rate() and irate()?

IF ASKED, SAY THIS
rate() averages the per-second increase over the whole time range you give it, smooth, and the right choice for alerting since you don't want one noisy sample to trigger a page. irate() uses only the last two data points, spiky and reactive, better for a dashboard where you want to see a real-time b

`rate()` averages the per-second increase over the whole time range you give it, smooth, and the right choice for alerting since you don't want one noisy sample to trigger a page. `irate()` uses only the last two data points, spiky and reactive, better for a dashboard where you want to see a real-time blip immediately. `histogram_quantile()` turns a histogram metric into a percentile, p95/p99 latency being the classic use — always watch the tail, not the average, since averages hide exactly the minority of users who are actually suffering.

Remember: `rate()` for alerts (smooth), `irate()` for dashboards (reactive) · `histogram_quantile()` + watch p95/p99, not the mean · This doubles as your answer to 'why do averages lie' if asked separately

- [ ] Prometheus memory keeps growing and performance is degrading — what's happening?

IF ASKED, SAY THIS
Almost always cardinality — a new code path started emitting a label with unbounded values, a user ID or a request ID baked directly into a metric label, and each unique combination creates a brand-new time series. Confirm with topk(10, count by (__name__)({__name__=~".+"})) to find which metric exp

Almost always **cardinality** — a new code path started emitting a label with unbounded values, a user ID or a request ID baked directly into a metric label, and each unique combination creates a brand-new time series. Confirm with `topk(10, count by (__name__)({__name__=~".+"}))` to find which metric exploded, then fix it with `metric_relabel_configs` to drop the offending label at scrape time, or move that dimension into a log/trace instead of a metric label — metrics should stay low-cardinality by design.

Remember: Cardinality explosion is the default answer to 'Prometheus memory is growing' · The topk-by-name query is the actual diagnostic, not just the theory · High-cardinality data belongs in logs/traces, not metric labels — that's the underlying principle

- [ ] Your Grafana dashboard is showing gaps in metrics every 2-3 hours. Prometheus data is there but Grafana is not displaying it. What is wrong?

IF ASKED, SAY THIS
Metric gaps in Grafana while Prometheus has the data usually come from a few specific causes: Cause 1 - Grafana session/token expiry: If the Prometheus data source uses a token for authentication, and that token expires every few hours, Grafana will fail to query. Check the data source health in Gra

Metric gaps in Grafana while Prometheus has the data usually come from a few specific causes:

Cause 1 - Grafana session/token expiry: If the Prometheus data source uses a token for authentication, and that token expires every few hours, Grafana will fail to query. Check the data source health in Grafana > Configuration > Data Sources. Look for auth errors.

Cause 2 - Prometheus retention or compaction: Prometheus compacts data in the background. During compaction, recent data may not be queryable for a brief window. The gap pattern (every 2-3 hours) matches the default compaction interval. Check Prometheus logs for compaction events.

Cause 3 - Prometheus scrape target down: The application pod restarted every 2-3 hours (OOM kill, liveness probe failure). Check: `kubectl get pod --field-selector=status.phase=Running` and compare restart count over time. If the pod restarts, the counter resets and there is a gap in the metric.
Fix: check WHY the pod is restarting. Do not fix Grafana - fix the underlying restart.

Cause 4 - Staleness handling: Prometheus marks a metric as stale if the target has not been scraped for 5 minutes. After that, Grafana shows a gap. This happens if the scrape interval is 30s but the target becomes unavailable for more than 5 minutes.

Cause 5 - Clock skew between Prometheus and K8s nodes: If node time is drifting, Prometheus timestamps are inconsistent. Grafana queries by time range - if the timestamps do not match, you see gaps. Fix: ensure NTP is synchronised across all nodes.

Debugging approach: Query the metric directly in Prometheus UI for the time range where the gap appears. If data exists in Prometheus but not Grafana, it is a data source connection issue. If data does not exist in Prometheus, the scrape target was down.

- [ ] You need to set up monitoring for a new microservice from scratch. Walk me through everything you would configure.

IF ASKED, SAY THIS
Complete observability setup for a new microservice: Step 1 - Instrument the application: Add Micrometer (Java) or prometheus-client (Python/Go) to the app. Expose /actuator/prometheus endpoint. Ensure these metrics are exposed at minimum: HTTP request count by status and path, HTTP request duration

Complete observability setup for a new microservice:

Step 1 - Instrument the application: Add Micrometer (Java) or prometheus-client (Python/Go) to the app. Expose /actuator/prometheus endpoint. Ensure these metrics are exposed at minimum: HTTP request count by status and path, HTTP request duration histogram (enables P50/P95/P99), JVM metrics (heap, GC, threads), active database connections, custom business metrics (payment count, transaction amount).

Step 2 - Configure Prometheus scraping: Add ServiceMonitor or PodMonitor CR (if using Prometheus Operator):
`apiVersion: monitoring.coreos.com/v1 kind: ServiceMonitor metadata: name: payments-service spec: selector: matchLabels: app: payments-service endpoints: - port: http path: /actuator/prometheus interval: 15s`

Step 3 - Create SLO recording rules: Write PrometheusRule with recording rules for request rate, error rate, and latency quantiles. Pre-compute these so dashboards load fast.

Step 4 - Build the Grafana dashboard in this order:
Row 1: RED metrics (Rate, Error rate, Duration/P99) with SLO threshold lines
Row 2: Infrastructure (pod count, restarts, CPU, memory)
Row 3: JVM internals (heap, GC pause, thread pools)
Row 4: Dependencies (DB query time, cache hit rate, downstream error rates)
Row 5: Business metrics (payment volume, transaction value, failure by reason)

Step 5 - Set up alerts: Fast burn rate alert (PagerDuty), slow burn rate alert (Slack), pod restart alert (Slack), latency SLO alert (Slack).

Step 6 - Write the runbook: Every alert links to a runbook. The runbook includes: what the alert means, common causes, diagnostic commands, remediation steps, escalation path.

Step 7 - Synthetic monitoring: Add a Blackbox Exporter probe that hits the health endpoint from outside the cluster every 30 seconds. This is the outside-in check that catches issues the internal metrics miss.

Grafana

What is Grafana and how does it relate to Prometheus?

IF ASKED, SAY THIS
Grafana is a visualization and dashboarding platform that queries data sources and renders panels — it doesn't store metrics itself. Prometheus is a typical data source: Grafana sends it PromQL and charts the results, while also supporting Loki, Tempo, CloudWatch, Elasticsearch, and SQL. So Promethe

- Grafana is an open-source visualization and dashboarding platform. It does NOT store metrics — it queries data sources and renders panels.
- Prometheus is a common data source (storage + query engine); Grafana sends PromQL to it and visualizes results. Grafana supports many sources: Loki (logs), Tempo (traces), CloudWatch, Elasticsearch, SQL.
- Division of labor: Prometheus collects/stores/alerts on metrics; Grafana visualizes and can also alert.

What are Grafana data sources and dashboard variables?

IF ASKED, SAY THIS
A data source is a configured query backend like Prometheus or Loki that panels read from. Template variables such as $namespace or $pod make dashboards dynamic — dropdowns that filter every panel at once, often populated from label values via queries. They let a single dashboard serve many services

- A data source is a configured backend (Prometheus, Loki, CloudWatch...) that panels query.
- Template variables (e.g. $namespace, $pod) turn static dashboards into dynamic ones — dropdowns that filter all panels at once. They can be populated from label values via queries (label_values()).
- This lets one dashboard serve many environments/services instead of duplicating dashboards.

```
label_values(kube_pod_info{namespace="$namespace"}, pod)
```

How does Grafana alerting work and when use it vs Prometheus Alertmanager?

IF ASKED, SAY THIS
Grafana unified alerting evaluates rules across any data source and routes through contact points and notification policies, which is ideal when alerts span metrics, logs, and SQL together. Prometheus with Alertmanager keeps evaluation next to the metrics and stays robust even if Grafana is down, ma

- Grafana unified alerting evaluates rules against any data source and routes via contact points / notification policies (its own Alertmanager-compatible pipeline). Good when you alert across mixed sources (metrics + logs + SQL) in one place.
- Prometheus + Alertmanager keeps alerting close to the metrics, evaluated by Prometheus itself — robust even if Grafana is down, and the standard for pure-Prometheus stacks.
- Many orgs centralize on one to avoid duplicate/competing alerts. Keep alert evaluation independent of your dashboard layer for reliability.

What makes a good dashboard, and how do you provision them as code?

IF ASKED, SAY THIS
A good dashboard leads with the golden signals — latency, traffic, errors, saturation — uses template variables for reuse, flows top-down from overview to detail, and avoids vanity-metric clutter. You provision dashboards as code by storing their JSON in Git and deploying via provisioning files, the

- Good dashboards lead with the golden signals (latency, traffic, errors, saturation), use variables for reuse, order panels top-down from overview to detail, and avoid clutter and vanity metrics.
- Provision as code: dashboards are JSON; store them in Git and deploy via provisioning configs, Terraform (grafana provider), or the API — so dashboards are versioned and reproducible, not click-ops.
- Use consistent units, thresholds, and links to runbooks.

Metrics vs logs vs traces — Prometheus/Grafana/Loki roles?

IF ASKED, SAY THIS
Metrics = is it healthy now (Prometheus stores them). Logs = what exactly happened (Loki stores them, labels only). Traces = where's the latency. **Grafana stores nothing** — it visualizes all three.

Metrics = is it healthy now (Prometheus stores them). Logs = what exactly happened (Loki stores them, labels only). Traces = where's the latency. **Grafana stores nothing** — it visualizes all three.

- [ ] Grafana dashboard shows 'No Data' on every panel.

IF ASKED, SAY THIS
Check the data source connection first — everything dying at once is a data-source problem, not a query problem. Then check the time range wasn't left too narrow post-migration. Panel queries last.

Check the data source connection first — everything dying at once is a data-source problem, not a query problem. Then check the time range wasn't left too narrow post-migration. Panel queries last.

- [ ] CPU crosses 95% but no alert fires.

IF ASKED, SAY THIS
Walk the pipeline backwards: rule exists and evaluates true? Cleared the `for:` duration? Reached Alertmanager? Route labels actually match? Check the notification channel last.

Walk the pipeline backwards: rule exists and evaluates true? Cleared the `for:` duration? Reached Alertmanager? Route labels actually match? Check the notification channel last.

- [ ] Grafana dashboard takes over a minute to load.

IF ASKED, SAY THIS
Usually an expensive, un-aggregated query over raw high-cardinality data across a long range. Fix with a recording rule to precompute it, or shrink the default time range/step.

Usually an expensive, un-aggregated query over raw high-cardinality data across a long range. Fix with a recording rule to precompute it, or shrink the default time range/step.

- [ ] A new Kubernetes cluster is added — integrate it into Grafana?

IF ASKED, SAY THIS
Deploy exporters (node_exporter, kube-state-metrics) on it, federate or remote_write into the existing central store, then add a cluster template variable to existing dashboards instead of cloning new ones.

Deploy exporters (node_exporter, kube-state-metrics) on it, federate or remote_write into the existing central store, then add a cluster template variable to existing dashboards instead of cloning new ones.

- [ ] Grafana server itself becomes unavailable in business hours.

IF ASKED, SAY THIS
Dashboards down ≠ monitoring down — Prometheus/Alertmanager still evaluate and page independently. Check Grafana's own logs/resources, restart if stuck, and have a CLI fallback (`promtool query`) for the outage window.

Dashboards down ≠ monitoring down — Prometheus/Alertmanager still evaluate and page independently. Check Grafana's own logs/resources, restart if stuck, and have a CLI fallback (`promtool query`) for the outage window.

- [ ] Production logs stop appearing after a server restart.

IF ASKED, SAY THIS
Check the forwarder/agent actually restarted and reconnected. Confirm the monitored file path or app still exists post-restart, and check for a stale PID/lock file blocking startup.

Check the forwarder/agent actually restarted and reconnected. Confirm the monitored file path or app still exists post-restart, and check for a stale PID/lock file blocking startup.

- [ ] Splunk dashboards take several minutes to load.

IF ASKED, SAY THIS
Usually a raw, expensive search over too wide a range. Convert it to a scheduled, pre-computed summary/accelerated data model instead of running the live search on every load.

Usually a raw, expensive search over too wide a range. Convert it to a scheduled, pre-computed summary/accelerated data model instead of running the live search on every load.

- [ ] Millions of logs generated in one hour.

IF ASKED, SAY THIS
First confirm it's real traffic, not a logging bug — a retry loop or debug-level logging left on in prod. Filter/sample genuinely low-value volume, and check ingest/licensing limits aren't about to throttle.

First confirm it's real traffic, not a logging bug — a retry loop or debug-level logging left on in prod. Filter/sample genuinely low-value volume, and check ingest/licensing limits aren't about to throttle.

- [ ] Alert after 5 failed logins within 2 minutes.

IF ASKED, SAY THIS
A scheduled SPL search: count failed-login events in a 2-minute window, threshold ≥5, trigger via a webhook/email/SOAR playbook — auto-block the IP if it routes through SOAR.

A scheduled SPL search: count failed-login events in a 2-minute window, threshold ≥5, trigger via a webhook/email/SOAR playbook — auto-block the IP if it routes through SOAR.

- [ ] Symptom-based alerting vs SLO-burn-rate alerting — why do senior SREs alert differently than junior ones?

IF ASKED, SAY THIS
Junior approach: alert on raw resource thresholds (CPU>80%, mem>75%) — noisy, ~50 alerts/day, real problems get lost. Senior approach: alert on SLO error-budget burn rate — a fast-burn page (will exhaust the budget in ~1hr) and a slow-burn Slack notice (exhausts in ~5 days) replace 50 noisy alerts with 2 meaningful ones, because every alert now maps to actual user impact.

The core problem with threshold alerting is that a resource metric crossing a number doesn't reliably mean users are affected — high CPU alone might be totally fine if latency and error rate are still within SLO. Burn-rate alerting inverts the logic: it starts from the thing that actually matters (are we spending our error budget faster than sustainable) and only pages when that's true.

- [ ] What should every production SRE dashboard have, at minimum, for a 3am incident?

IF ASKED, SAY THIS
Three rows: (1) Service health — request rate, error rate, and p99 latency, each with the SLO threshold drawn as a line; (2) Infrastructure — pod/restart counts, CPU/memory per node, DB connection pool utilization; (3) Dependencies — downstream API error rates, cache hit rate, queue depth/consumer lag, and a deployment-annotation timeline so you can instantly see if a recent deploy lines up with the incident.

The deployment-annotation timeline is the detail people most often skip and most often need — at 3am, 'did this start right after a deploy' is usually the fastest first question to answer, and it's useless if deploys aren't marked on the same timeline as the metrics.

- [ ] How do you manage Grafana dashboards as code rather than clicking through the UI?

IF ASKED, SAY THIS
Dashboards are exported as JSON and either provisioned via config files (Grafana's provisioning system reads a directory of dashboard JSON on startup) or managed through Terraform (the grafana provider) / the Grafana API — so dashboards live in git, get code review, and roll out consistently across environments instead of drifting per-cluster.

This matters operationally the same way Infrastructure-as-Code matters generally: without it, the "prod" and "staging" versions of a dashboard quietly diverge over time as different engineers make ad-hoc UI edits, and nobody can tell which version is authoritative during an incident.

- [ ] What are Grafana template variables/variables, and why do they matter for a dashboard used across many services?

IF ASKED, SAY THIS
Template variables (e.g. $service, $environment, $namespace) let one dashboard definition serve every service/environment instead of needing a hand-built copy per service — the variable populates a dropdown at the top of the dashboard and every panel's query substitutes the selected value.

Without variables, teams end up copy-pasting dashboards per service, and any improvement to the dashboard has to be manually re-applied to every copy — variables turn one well-designed dashboard into the shared standard for an entire platform.

- [ ] What are Grafana annotations, and how do they help during an incident review?

IF ASKED, SAY THIS
Annotations are markers overlaid directly on a graph's timeline — typically auto-added on every deployment — so anyone looking at a metrics spike can instantly see whether it lines up with a release, without cross-referencing a separate deploy log.

This is the single highest-leverage addition to a dashboard for incident response: "did this start right after a deploy" is usually the first question asked in an incident, and annotations answer it visually in seconds instead of requiring someone to go dig through CI/CD history.

Datadog

- [ ] How does the Datadog Agent get data into Datadog, at a high level?

> **IF ASKED, SAY THIS**
> The Agent is a process (or DaemonSet in Kubernetes) that runs on every host/node, collecting system metrics, running configured integration checks (Postgres, Redis, nginx, etc.), tailing/forwarding logs, and receiving APM traces via a local endpoint (StatsD-style on port 8125 for custom metrics, or the trace-agent list

The Agent is a process (or DaemonSet in Kubernetes) that runs on every host/node, collecting system metrics, running configured integration checks (Postgres, Redis, nginx, etc.), tailing/forwarding logs, and receiving APM traces via a local endpoint (StatsD-style on port 8125 for custom metrics, or the trace-agent listening on 8126) — then batches and forwards everything to Datadog's SaaS backend over HTTPS. In Kubernetes specifically, you typically run it as a DaemonSet (one Agent pod per node, node-level + container metrics) plus optionally the Cluster Agent (a single deployment that reduces per-node API server load by centralizing cluster-level metadata collection and orchestrating cluster checks).

- [ ] What's the Cluster Agent for, specifically, and why not just run the regular Agent on every node?

> **IF ASKED, SAY THIS**
> Without it, every node-level Agent would independently hit the Kubernetes API server for cluster-wide metadata (events, cluster-level checks, HPA custom-metrics data) — multiplying API server load by node count. The Cluster Agent centralizes that: it's a single deployment that talks to the API server once, then the per

Without it, every node-level Agent would independently hit the Kubernetes API server for cluster-wide metadata (events, cluster-level checks, HPA custom-metrics data) — multiplying API server load by node count. The Cluster Agent centralizes that: it's a single deployment that talks to the API server once, then the per-node Agents talk to the Cluster Agent instead, which also enables features like the Datadog Cluster Agent acting as an external metrics provider for HPA (scaling on a Datadog metric, not just CPU/memory).

- [ ] How do integrations work — how does Datadog know how to monitor, say, Postgres or nginx specifically?

> **IF ASKED, SAY THIS**
> Integrations are Agent-side check plugins (Python-based historically) that know how to talk to a specific technology's native metrics interface (Postgres system views, nginx's stub_status endpoint, a Redis INFO command) and translate that into Datadog metrics on a schedule, configured via a YAML file per integration (

Integrations are Agent-side check plugins (Python-based historically) that know how to talk to a specific technology's native metrics interface (Postgres system views, nginx's `stub_status` endpoint, a Redis `INFO` command) and translate that into Datadog metrics on a schedule, configured via a YAML file per integration (`conf.d/postgres.d/conf.yaml`) specifying connection details. In Kubernetes, Autodiscovery lets the Agent detect a running container matching a known integration (via pod annotations or Docker labels) and auto-configure the check without manual per-pod setup — important at scale where you're not hand-configuring monitoring per pod.

- [ ] What monitor types does Datadog offer, and when would you reach for each?

> **IF ASKED, SAY THIS**
> Metric monitor — threshold on a metric query (avg CPU > 80% for 5 minutes). The everyday default. Anomaly monitor — alerts on deviation from a learned historical baseline rather than a fixed threshold; good for metrics with strong daily/weekly seasonality where a static threshold would false-positive constantly (e.g. t

- **Metric monitor** — threshold on a metric query (avg CPU > 80% for 5 minutes). The everyday default.
- **Anomaly monitor** — alerts on deviation from a learned historical baseline rather than a fixed threshold; good for metrics with strong daily/weekly seasonality where a static threshold would false-positive constantly (e.g. traffic volume).
- **Outlier monitor** — flags one host/container behaving differently from its peer group (e.g. one pod in a 20-pod deployment with wildly different latency) — a fixed threshold wouldn't catch this since the group average looks fine.
- **Forecast monitor** — projects a metric's trend forward and alerts before a hard threshold is actually crossed (disk filling up in 6 hours at current rate).
- **Composite monitor** — boolean combination of other monitors (alert only if A AND B, or A but NOT B) — used to reduce noise from correlated alerts firing independently.
- **APM/Log monitors** — thresholds on trace-derived metrics (error rate, p99 latency) or log-based facets/counts directly.

- [ ] How would you reduce alert noise from a monitor that flaps constantly around its threshold?

> **IF ASKED, SAY THIS**
> Add an evaluation window ("alert only if the condition is true for N out of M evaluations," not a single sample crossing), tune Recovery threshold separately and lower than the alert threshold to create hysteresis (avoids re-triggering immediately after briefly dipping back under), consider whether an anomaly or foreca

Add an evaluation window ("alert only if the condition is true for N out of M evaluations," not a single sample crossing), tune `Recovery threshold` separately and lower than the alert threshold to create hysteresis (avoids re-triggering immediately after briefly dipping back under), consider whether an anomaly or forecast monitor type fits the metric's actual behavior better than a fixed threshold, and use a composite monitor to require

corroborating signals (e.g. high latency AND high error rate, not either alone) before paging — this is the same "reduce alert fatigue" judgment PagerDuty questions probe from the routing side; Datadog is where you fix it at the source.

- [ ] What are monitor notification/escalation options inside Datadog itself, before it ever reaches PagerDuty?

IF ASKED, SAY THIS
Monitor notification messages support conditional templating ({{#is_alert}} / {{#is_warning}} / {{#is_recovery}}) so different severities route to different channels/integrations right from the message body, plus @notify-target handles for Slack, email, or a specific PagerDuty service integration. You can also scope

```
{{#is_alert}}
@pagerduty-critical-services  @slack-platform-oncall
{{/is_alert}}
{{#is_recovery}}
@slack-platform-oncall
{{/is_recovery}}
```

Monitor notification messages support conditional templating (`{{#is_alert}}` / `{{#is warning}}` / `{{#is_recovery}}`) so different severities route to different channels/integrations right from the message body, plus `@notify-target` handles for Slack, email, or a specific PagerDuty service integration. You can also scope notifications per-tag dynamically (e.g. route to the owning team's channel based on a `team:` tag on the affected resource) using multi-alert monitors that group by tag.

- [ ] What's a "multi-alert" (grouped) monitor, and why does it matter for a fleet of many hosts/pods?

IF ASKED, SAY THIS
A monitor query with a by {tag} grouping (e.g. avg:system.cpu.user{*} by {host}) evaluates the threshold independently per group value and can fire/resolve/notify for each host separately, rather than one monitor covering the entire fleet's average — which would mask a single bad host inside a healthy fleet average. T

A monitor query with a `by {tag}` grouping (e.g. `avg:system.cpu.user{*} by {host}`) evaluates the threshold *independently per group value* and can fire/resolve/notify for each host separately, rather than one monitor covering the entire fleet's average — which would mask a single bad host inside a healthy fleet average. This is the standard pattern for fleet-wide monitors: one monitor definition, many independent alert instances, each carrying its own tag context in the notification so responders know exactly which host/pod/service is affected without digging.

- [ ] Dashboard JSON as code — how would you manage Datadog dashboards the same way you manage infrastructure?

IF ASKED, SAY THIS
Dashboards (and monitors) are managed through Terraform's Datadog provider (datadog_dashboard , datadog_monitor resources) or the Datadog API directly with JSON definitions checked into Git — the same "no manual console changes for anything that matters" principle from the Terraform/drift chapter applies here. This is

Dashboards (and monitors) are managed through Terraform's Datadog provider (`datadog_dashboard` , `datadog_monitor` resources) or the Datadog API directly with JSON definitions checked into Git — the same "no manual console changes for anything that matters" principle from the Terraform/drift chapter applies here. This is the answer that separates "used Datadog" from "operated Datadog as part of a platform" in an interview: dashboards-as-code means a new service onboarding automatically gets its standard dashboard/monitor set via the same CI pipeline that provisions its infrastructure, not a manual click-through someone forgets to do.

- [ ] What's a Notebook in Datadog, and how is it different from a Dashboard?

IF ASKED, SAY THIS
A Dashboard is a persistent, structured grid of widgets meant for ongoing monitoring (built once, glanced at repeatedly). A Notebook is more like an incident/investigation document — free-form text interleaved with live graphs, built during or after an investigation to narrate "here's what we looked at and what we foun

A Dashboard is a persistent, structured grid of widgets meant for ongoing monitoring (built once, glanced at repeatedly). A Notebook is more like an incident/investigation document — free-form text interleaved with live graphs, built during or after an investigation to narrate "here's what we looked at and what we found," commonly used for postmortem writeups or live incident investigation notes where the graphs need to sit alongside human-written analysis, not just a static grid.

- [ ] Explain the trace/span model Datadog APM uses.

IF ASKED, SAY THIS
A trace represents one end-to-end request as it moves through your system; it's composed of spans , each representing one unit of work (an HTTP handler, a DB query, a downstream service call), with parent-child relationships forming a tree. Auto-instrumentation libraries (dd-trace for various languages) hook into comm

A **trace** represents one end-to-end request as it moves through your system; it's composed of **spans**, each representing one unit of work (an HTTP handler, a DB query, a downstream service call), with parent-child relationships forming a tree. Auto-instrumentation libraries (`dd-trace` for various languages) hook into common frameworks/DB drivers to generate spans automatically; a **trace ID** is propagated across service boundaries via HTTP

headers so spans generated in different services/processes still stitch together into one coherent trace, which is exactly what lets you see "this slow user request spent 400ms in service A, then 1.2s waiting on a downstream call to service B's database."

- [ ] What is trace sampling, and why does it matter operationally?

IF ASKED, SAY THIS

Capturing 100% of traces at high request volume is expensive (ingestion cost and storage) — sampling controls what fraction of traces are actually kept for detailed inspection. Datadog supports head-based sampling (decide at trace start, e.g. keep 20% at random) and more importantly error and latency-biased sampling (a

Capturing 100% of traces at high request volume is expensive (ingestion cost and storage) — sampling controls what fraction of traces are actually kept for detailed inspection. Datadog supports head-based sampling (decide at trace start, e.g. keep 20% at random) and more importantly **error and latency-biased sampling** (always keep traces that error out or exceed a latency threshold, sample normally otherwise) — since the traces you actually need for debugging are disproportionately the slow/failed ones, not a random 20% that's mostly healthy fast requests. Trace metrics (request counts, error rates, latency percentiles) are computed from 100% of traffic regardless of sampling rate — only the detailed trace payload for inspection is sampled, not the aggregate stats used for monitors.

- [ ] How would you use APM to find the root cause of a sudden latency regression across a microservices fleet?

IF ASKED, SAY THIS

Start at the Service Map / Service Catalog view to see which service's latency actually changed vs. which are just downstream victims of one slow dependency — a single slow service can make everything upstream of it look slow in isolated dashboards. Drill into that service's trace flame graphs for the affected window,

Start at the Service Map / Service Catalog view to see which service's latency actually changed vs. which are just downstream victims of one slow dependency — a single slow service can make everything upstream of it look slow in isolated dashboards. Drill into that service's trace flame graphs for the affected window, comparing a slow trace against a normal one span-by-span to see exactly which span (a specific DB query, an external API call) grew. Cross-reference with the deploy timeline (did this service or one of its dependencies deploy right before the regression started) and correlate against infrastructure metrics for that service (CPU throttling, connection pool exhaustion) rather than assuming it's purely application-code-level.

- [ ] Walk through the log pipeline: ingestion → processing → indexing.

IF ASKED, SAY THIS

Logs arrive via the Agent's log collection (tailing files, container stdout/stderr, or direct API/forwarder submission), pass through configurable Log Pipelines which apply Grok parsing rules to extract structured attributes from raw text (timestamp, log level, request ID, custom fields), can be enriched/remapped, then

Logs arrive via the Agent's log collection (tailing files, container stdout/stderr, or direct API/forwarder submission), pass through configurable **Log Pipelines** which apply Grok parsing rules to extract structured attributes from raw text (timestamp, log level, request ID, custom fields), can be enriched/remapped, then are routed to one or more **indexes** based on inclusion filters. Only indexed logs are searchable/retained for the index's configured retention and count toward index-based billing — this index-routing step is where cost control actually happens.

- [ ] Log ingestion costs are climbing fast for a high-volume service. What levers do you actually have, beyond "log less"?

IF ASKED, SAY THIS

(1) Exclusion filters at the index level — drop known-noisy, low-value logs (health-check 200s, debug-level chatter) before they count toward the indexed volume, while still optionally retaining them briefly in a cheaper "logs without limits" ingestion-only tier if you need brief forensic access without full indexing c

(1) **Exclusion filters** at the index level — drop known-noisy, low-value logs (health-check 200s, debug-level chatter) before they count toward the indexed volume, while still optionally retaining them briefly in a cheaper "logs without limits" ingestion-only tier if you need brief forensic access without full indexing cost. (2) Sampling on specific high-cardinality/high-volume log sources rather than dropping the source entirely. (3) Route different log sources to indexes with different retention (short retention for verbose debug logs, longer for audit/security-relevant ones) instead of one blanket retention policy. (4) Reduce log verbosity at the application level for genuinely low-value log lines rather than treating this purely as a Datadog configuration problem.

- [ ] How do logs, metrics, and traces get correlated in the Datadog UI — what makes that possible?

IF ASKED, SAY THIS

Correlation depends on shared identifiers being present across signal types: injecting the active trace_id / span_id into your application's log output (most dd-trace libraries do this automatically for supported logging frameworks) lets Datadog show "logs for this specific trace" directly from the APM trace view. Cons

Correlation depends on shared identifiers being present across signal types: injecting the active `trace_id` / `span_id` into your application's log output (most `dd-trace` libraries do this automatically for supported logging frameworks) lets Datadog show "logs for this specific trace" directly from the APM trace view. Consistent tagging (same `service`, `env`, `version` tags applied uniformly across metrics, logs, and traces — usually via Unified Service Tagging conventions) is what lets you pivot from a dashboard graph straight to the matching logs/traces for that exact service+environment+time window without manually re-filtering each signal type separately.

- [ ] What's the difference between a synthetic API test and a synthetic browser test, and when do you use each?

IF ASKED, SAY THIS
An API test makes a direct HTTP(S) request (or multi-step chain of requests) from external locations on a schedule, checking status code/response body/latency — good for uptime/latency SLO verification of an endpoint from the outside-in, independent of your own infrastructure's view of itself. A browser test drives a r

An **API test** makes a direct HTTP(S) request (or multi-step chain of requests) from external locations on a schedule, checking status code/response body/latency — good for uptime/latency SLO verification of an endpoint from the outside-in, independent of your own infrastructure's view of itself. A **browser test** drives a real headless browser through a recorded user flow (login, add to cart, checkout) — catches frontend/JS issues and multi-step user journey breakage that a pure API test can't, at the cost of being slower and more brittle to UI changes. Both run from Datadog's global network of test locations, which is valuable specifically because it validates availability from *outside* your own network, catching issues (DNS, CDN, regional connectivity) your internal monitoring literally cannot see since it's measuring from inside the perimeter.

- [ ] What does RUM (Real User Monitoring) capture that synthetic tests can't?

IF ASKED, SAY THIS
RUM instruments the actual application running in real users' real browsers/devices — capturing genuine performance across the full diversity of real network conditions, device types, and geographies your synthetic test locations don't represent, plus real user interaction data (actual click paths, real session replay,

RUM instruments the actual application running in real users' real browsers/devices — capturing genuine performance across the full diversity of real network conditions, device types, and geographies your synthetic test locations don't represent, plus real user interaction data (actual click paths, real session replay, genuine Core Web Vitals as experienced) rather than one scripted flow run on a schedule. Synthetic tests answer "is the service up and how fast is a specific critical path from a controlled location" — RUM answers "what are real users actually experiencing right now, everywhere." They're complementary, not substitutes for each other.

- [ ] What is "Unified Service Tagging" and why does Datadog push it so hard?

IF ASKED, SAY THIS
It's the convention of consistently applying env , service , and version tags identically across every signal type (infra metrics, logs, traces, RUM) for a given deployable unit. Without this consistency, you can't reliably pivot between signal types for the same logical thing — e.g. a trace tagged service:checkout won

```
DD_ENV=prod
DD_SERVICE=checkout-api
DD_VERSION=1.24.0
```

It's the convention of consistently applying `env` , `service` , and `version` tags identically across every signal type (infra metrics, logs, traces, RUM) for a given deployable unit. Without this consistency, you can't reliably pivot between signal types for the same logical thing — e.g. a trace tagged `service:checkout` won't correlate with logs tagged `app:checkout-api` even though a human knows they're the same service, because the platform doesn't. It's the tagging discipline that makes cross-signal correlation (section 4.5) actually work in practice rather than in theory.

- [ ] What's a common tagging mistake at scale, and what does it cost you?

IF ASKED, SAY THIS
High-cardinality tags on metrics — tagging a metric with something like a raw user ID, request ID, or pod name (which changes constantly) rather than a bounded dimension (service, region, status code) causes metric cardinality explosion, driving up cost and, in extreme cases, hitting custom-metric limits that cause the

High-cardinality tags on metrics — tagging a metric with something like a raw user ID, request ID, or pod name (which changes constantly) rather than a bounded dimension (service, region, status code) causes metric cardinality explosion, driving up cost and, in extreme cases, hitting custom-metric limits that cause the metric to stop being reliably queryable at all. The fix is keeping high-cardinality identifiers in logs/traces (which are built to handle that) while keeping metric tags to genuinely low-cardinality, meaningful-for-aggregation dimensions.

- [ ] How are SLOs defined and tracked natively in Datadog?

IF ASKED, SAY THIS
A Datadog SLO is built from either a metric-based definition (a good-events/total-events ratio query against existing metrics) or a monitor-based definition (the SLO is "this monitor is in OK state X% of the time" over a rolling window). Datadog then tracks error budget consumption automatically, shows burn-rate, and c

A Datadog SLO is built from either a **metric-based** definition (a good-events/total-events ratio query against existing metrics) or a **monitor-based** definition (the SLO is "this monitor is in OK state X% of the time" over a rolling window). Datadog then tracks error budget consumption automatically, shows burn-rate, and can alert specifically on fast error-budget burn (not just the underlying metric threshold) — the same SLO/error-budget concepts from your Prometheus/Grafana background, just computed and visualized natively rather than hand-built with recording rules.

- [ ] Map the concepts: what's the Prometheus/Grafana equivalent of a Datadog Monitor, Dashboard, and the Agent?

IF ASKED, SAY THIS

Datadog Agent ≈ Prometheus exporters + node_exporter (collection), though architecturally different: Datadog Agents push to Datadog's backend; Prometheus pulls/scrapes exporters on an interval. This push-vs-pull distinction is a genuinely common interview question. Datadog Monitor ≈ Prometheus Alerting rule evaluated b

- Datadog **Agent** ≈ Prometheus **exporters** + **node_exporter** (collection), though architecturally different: Datadog Agents *push* to Datadog's backend; Prometheus *pulls/scrapes* exporters on an interval. This push-vs-pull distinction is a genuinely common interview question.
- Datadog **Monitor** ≈ Prometheus **Alerting rule** evaluated by Alertmanager.
- Datadog **Dashboard** ≈ **Grafana Dashboard**.
- Datadog metric query language ≈ **PromQL**, though Datadog's is arguably less expressive for complex joins but comes with a managed backend requiring no self-hosted storage/scaling concerns (no "sizing your Prometheus TSDB" problem).

☐ Given you know Prometheus/Grafana well, what's the honest tradeoff argument for Datadog (SaaS) vs self-hosted Prometheus/Grafana? ▲

IF ASKED, SAY THIS
For Datadog: no operational burden of running/scaling/upgrading your own TSDB and Alertmanager, unified platform across metrics/logs/traces/RUM/synthetics with correlation built-in rather than stitched together across separate open-source tools, faster time-to-value for a growing platform team without dedicated observa

For Datadog: no operational burden of running/scaling/upgrading your own TSDB and Alertmanager, unified platform across metrics/logs/traces/RUM/synthetics with correlation built-in rather than stitched together across separate open-source tools, faster time-to-value for a growing platform team without dedicated observability headcount. **For Prometheus/Grafana:** no per-host/per-metric SaaS billing at scale (cost can grow very fast with Datadog as fleet size and cardinality grow), full data ownership/control (relevant for data residency/compliance constraints), no vendor lock-in on query language/dashboards, and often lower cost at very large scale if you have the platform engineering capacity to actually run it well. Most senior-level answers acknowledge it's a real cost/control-vs-operational-burden tradeoff, not a strictly-better-tool answer either direction.

☐ Scenario: Datadog monthly bill has doubled in a quarter with no corresponding infra growth. How do you investigate? ▲

IF ASKED, SAY THIS
Datadog's own Usage/Cost pages break down spend by product (infra hosts, APM host-based or ingested-span-based billing, custom metrics, indexed logs, synthetics runs) — start there rather than guessing. Custom metric cardinality explosion (section 4.7's tagging mistake) and log index growth (a new noisy log source, or

Datadog's own Usage/Cost pages break down spend by product (infra hosts, APM host-based or ingested-span-based billing, custom metrics, indexed logs, synthetics runs) — start there rather than guessing. Custom metric cardinality explosion (section 4.7's tagging mistake) and log index growth (a new noisy log source, or a team accidentally routing everything to a long-retention index) are the two most common silent-bill-doubling culprits in practice. Cross-reference the cost timeline against recent onboarding of new services/teams onto the platform — often it's not misuse, it's simply un-budgeted organic growth that needs governance (index routing policy, custom metric review in code review) rather than a single bug to fix.

☐ Scenario: you're asked to design the Datadog rollout for a newly-onboarded service from scratch. What's the minimum viable setup? ▲

IF ASKED, SAY THIS
Unified Service Tagging (env / service / version) applied consistently at deploy time — get this right from day one since retrofitting tags across historical data is impossible. APM auto-instrumentation enabled for baseline trace/latency/error-rate visibility with minimal code changes. A small set of monitors on the

1. Unified Service Tagging (`env` / `service` / `version`) applied consistently at deploy time — get this right from day one since retrofitting tags across historical data is impossible.
2. APM auto-instrumentation enabled for baseline trace/latency/error-rate visibility with minimal code changes.
3. A small set of monitors on the metrics that actually predict user-facing impact (error rate, p99 latency, saturation) rather than a large noisy set of infrastructure-threshold monitors nobody reads.
4. Log pipeline configured with a sensible index (not the default catch-all) and an exclusion filter for known-noisy sources from day one, not retrofitted after a cost surprise.
5. An SLO defined against the service's actual availability/latency target, tied to the monitors above, as the basis for the team's own error-budget conversations — not built ad hoc after the first incident.
6. All of the above defined as code (Terraform) and part of the service's onboarding template, so the next service gets this by default, not by someone remembering to click through the UI.

Splunk

- [ ] What are index, sourcetype, and source in Splunk, and how do they relate?

IF ASKED, SAY THIS
source = where the data physically came from (a file path, a script, a network port). sourcetype = the format/classification Splunk uses to parse it (access_combined, json, syslog). index = the named bucket of storage the event actually lands in — you choose the index at ingest time, usually one per data domain or retention policy.

A single sourcetype (e.g. `json`) might exist across many sources, and a single source might be routed to different indexes based on retention or access-control needs. Getting index design right matters operationally: too few indexes and you can't apply different retention/permissions per team; too many and search performance and admin overhead both suffer.

- [ ] Walk through the forwarder → indexer → search head architecture.

IF ASKED, SAY THIS
A Universal Forwarder on each source host ships raw data to Indexers, which parse and store it (and build the searchable index). A Search Head is where users/dashboards run searches — it distributes the search across indexers (in a clustered deployment) and merges results back.

In production, Indexers are typically clustered for replication/search-availability (index cluster with a cluster master), and Search Heads can also be clustered (search head cluster) for HA and to share saved searches/dashboards. Forwarders are deliberately lightweight — they do minimal parsing, so they don't burden the source host.

- [ ] Explain the basic shape of an SPL search — what does the pipe actually do?

IF ASKED, SAY THIS
SPL is a pipeline: each stage after `|` takes the result set from the previous stage and transforms it — exactly like a Unix shell pipeline. `search terms | stats count by field | sort -count` reads as: find events, then aggregate, then sort.

This mental model — "it's just piped Unix commands over structured events" — is the fastest way to reason about an unfamiliar SPL query: read left to right, each stage narrows or reshapes what flows into the next.

```
index=web_prod sourcetype=access_combined status>=500
| stats count by uri_path
| sort -count
| head 10
```

- [ ] What do stats, eval, and where each do, and when do you reach for each?

IF ASKED, SAY THIS
stats aggregates (count, avg, sum, p95, dc — distinct count) into summary rows. eval computes a new field from an expression (math, string ops, conditionals) without aggregating. where filters rows using an expression (works on calculated/eval'd fields too, unlike the initial search terms).

A common pattern: `eval` to derive a field (like a response-time bucket), then `stats` to aggregate by it, then `where` to filter the aggregated result (e.g. only buckets with count > 100).

```
... | eval duration_bucket=case(response_time<100,"fast",response_time<500,"ok","slow")
| stats count by duration_bucket
| where count > 100
```

- [ ] What does rex do and when do you need it?

IF ASKED, SAY THIS
rex extracts a new field from existing event text using a regular expression, on the fly, without needing a pre-configured field extraction. It's the go-to when a value you need to search/aggregate on isn't already a parsed field.

For anything you'll query repeatedly, a permanent field extraction (configured in props.conf/transforms.conf, or via the UI) is better than `rex` in every search — it's indexed and reusable. `rex` is the right tool for one-off or exploratory searches.

```
... | rex field=_raw "user_id=(?<user_id>\d+)"
| stats count by user_id
```

- [ ] stats vs timechart vs chart — what's the actual difference?

IF ASKED, SAY THIS
stats gives one summary row (or one row per group-by value) with no time axis. timechart buckets results over time (a time series, perfect for a trend graph). chart is like stats but shaped for a 2D visualization (one field on rows, another on columns).

Rule of thumb: if the question is "what happened over time" → `timechart`. If it's "what's the total/breakdown right now" → `stats`. If you need a cross-tab (e.g. error count by host AND by status code) → `chart`.

```
index=web_prod status>=500
| timechart span=5m count by uri_path limit=5
```

- [ ] Write an SPL query for a p95/p99 latency dashboard panel, production-style.

IF ASKED, SAY THIS
stats with the perc95()/perc99() functions, bucketed by time via timechart, gives you exactly the percentile-over-time panel an SRE dashboard needs — average latency alone hides the tail that actually causes complaints.

Percentiles matter more than averages for latency specifically because a small percentage of very slow requests (the tail) can be invisible in an average but is exactly what causes user-visible pain and SLO burn.

```
index=web_prod sourcetype=access_combined
| timechart span=1m perc50(response_time) as p50, perc95(response_time) as p95, perc99(response_time) as p99
```

- [ ] A dashboard panel that used to load in 2 seconds now takes 40. What do you check first?

IF ASKED, SAY THIS
Check the search itself before anything else: is it using an unbounded/very wide time range, a leading wildcard (index=* or a wildcard search term, which forces a full scan instead of using the index), or a raw-text search without field extraction where a scoped one would do. Then check indexer load/health, and whether the underlying data volume genuinely grew.

Leading wildcards (`*error*`) are one of the most common self-inflicted Splunk performance problems — they defeat Splunk's index-time optimizations and force it to scan much more raw data than a properly scoped search would.

- [ ] How do you set up a production alert in Splunk, and what are the operational pitfalls?

IF ASKED, SAY THIS
A saved search with a schedule and a trigger condition (e.g. result count > threshold), wired to an alert action (email, webhook to PagerDuty/Slack). Pitfalls: setting the search interval shorter than the search actually takes to run (alerts pile up/overlap), and alerting on raw counts instead of a rate (so alert sensitivity silently changes with traffic volume).

Throttling (suppressing repeat alerts for the same field-value combination within a window) is the other thing people forget — without it, one ongoing issue can page the same person every 5 minutes for hours.

- [ ] What's the difference between a summary index and a regular search over raw data, and why would you use one?

IF ASKED, SAY THIS
A summary index stores pre-computed, aggregated results from a scheduled search — so a dashboard reading from it queries a small pre-aggregated dataset instead of re-scanning huge raw volumes every time it loads.

This trades some flexibility (you can only slice the data the way it was pre-aggregated) for a large performance win on expensive, frequently-viewed panels — a classic "compute once, read many times" pattern.

- [ ] Explain the transaction and join commands — when would you actually reach for each?

IF ASKED, SAY THIS
transaction groups events into a single "transaction" based on a shared field and a time constraint (e.g. all events for one request_id within 30s) — good for reconstructing a multi-step flow. join combines results from two different searches on a common field, similar to a SQL join, but is expensive and best avoided at scale in favor of stats-based approaches when possible.

In practice, senior engineers avoid `join` where they can — it doesn't scale well and is one of the more common causes of a slow, timing-out search. A `stats` -based combination (aggregating both searches on the shared field, or using `append` + `stats`) is usually the more production-safe alternative.

- [ ] How does Splunk licensing typically constrain what you can do, and what do SREs need to watch for?

IF ASKED, SAY THIS
Splunk is commonly licensed by daily indexed data volume (GB/day) — exceeding it repeatedly triggers license violations that can eventually block search. SREs need to actively manage sourcetype/index volume (drop noisy debug-level logs at the forwarder, use index-time filtering) rather than let ingestion grow unchecked.

This is a very real production concern that's easy to miss coming from a pure-engineering background — a well-intentioned "log everything at DEBUG" change can blow through a license ceiling within days.

- [ ] Compare Prometheus, Grafana, Datadog, Splunk, and ELK — what's each actually best for?

IF ASKED, SAY THIS
Prometheus — open-source metrics/TSDB, pull-based, PromQL, the de facto standard for Kubernetes-native metrics; no built-in logs or long-term storage. Grafana — the visualization layer, source-agnostic (Prometheus, Loki, Datadog, Splunk as data sources) — almost always paired with something else rat

Prometheus — open-source metrics/TSDB, pull-based, PromQL, the de facto standard for Kubernetes-native metrics; no built-in logs or long-term storage. **Grafana** — the visualization layer, source-agnostic (Prometheus, Loki, Datadog, Splunk as data sources) — almost always paired with something else rather than standing alone. **Datadog** — commercial, all-in-one SaaS (metrics + logs + traces + APM), strong integrations out of the box, cost scales with volume. **Splunk** — primarily log-centric, an extremely powerful search language (SPL), historically strong in security/SIEM use cases relevant to banking compliance. **ELK/Elastic** — open-source log aggregation and search, a self-hosted alternative to Splunk.

A bank running all five in one JD isn't unusual — typically Splunk/ELK for logs and SIEM/compliance, Prometheus + Grafana for Kubernetes-native metrics, and Datadog (or similar) for unified APM.

- [ ] Simple XML vs. Dashboard Studio -- what's the actual difference, and which do you reach for?

IF ASKED, SAY THIS
Simple XML is the older, code-first dashboard format -- you write XML directly (or via the classic editor), more portable across Splunk versions, better for complex conditional logic. Dashboard Studio is the newer, visual drag-and-drop builder with a more polished/customizable look and easier collaboration for non-SPL-fluent stakeholders, but historically had some feature gaps versus what raw Simple XML could do. For an SRE-facing operational dashboard, Simple XML's flexibility often wins; for an executive-facing status dashboard, Dashboard Studio's polish often wins.

Naming the actual audience-driven reason for the choice (operational precision vs. presentation polish) reads better than just picking one as universally superior -- both are legitimate tools for different dashboard purposes.

- [ ] What's a base search / post-process search pattern, and why does it matter for a dashboard with many panels?

IF ASKED, SAY THIS
Without it, every panel on a dashboard runs its OWN full search against raw data independently -- a dashboard with 8 panels over the same time range and index can mean 8x the search load. A base search runs once, and each panel then runs a lightweight POST-PROCESS search (filtering/aggregating) against that single result set instead of re-scanning raw data 8 times -- dramatically cheaper for the indexers and faster to load.

This is one of the highest-leverage Splunk performance techniques for anyone actually building dashboards rather than just writing one-off searches -- a slow, expensive dashboard is very often just missing this pattern, running N independent full searches where 1 base search plus N cheap post-process searches would do.

- [ ] How do you make a Splunk dashboard interactive -- letting a viewer filter by service or time range without editing the search itself?

IF ASKED, SAY THIS
Dashboard inputs (dropdown, time-range picker, text box, multiselect) bound to TOKENS -- a token is a variable that gets substituted into every panel's underlying search. A service-selector dropdown bound to a $service$ token means every panel's search references $service$ instead of a hardcoded value, so changing the dropdown re-runs every panel filtered to the new selection, without anyone touching SPL.

Tokens are the mechanism that turns a fixed report into a genuinely reusable operational tool -- a dashboard hardcoded to one service is only useful for that one service; a token-driven one serves the whole team across every service with the same panels.

- [ ] How do you get a dashboard delivered as a scheduled report (e.g. a daily PDF to stakeholders) instead of requiring someone to log in and check it?

IF ASKED, SAY THIS
Schedule a PDF delivery (or a scheduled report/alert with a PDF attachment) directly from the dashboard's settings -- set the cadence (daily/weekly), recipients, and it renders and emails automatically. Useful for stakeholders who want a daily status snapshot without needing Splunk access or SPL knowledge at all.

This is worth mentioning specifically for a support/ops-facing role, since it directly addresses "how do you keep non-technical stakeholders informed without extra manual work every day" -- automating the delivery, not just building the dashboard, is the complete answer.

- [ ] What panels would you actually put on an SRE operational dashboard, and how would you lay them out?

IF ASKED, SAY THIS

Top row: the RED metrics (Rate, Errors, Duration) as the immediate health snapshot. Second row: infrastructure signals (CPU/memory/disk trends, restart counts). Third row: dependency health (downstream service error rates, queue depth). A deployment-annotation overlay on the time-series panels, so a metrics shift can be instantly checked against "did this line up with a recent deploy." Same structural principle as a good Grafana dashboard, just built in SPL/Splunk instead.

This deliberately mirrors the "3am dashboard" design principles covered in the Grafana tab -- the underlying design logic (RED metrics first, deployment context visible, dependency health included) is tool-agnostic, and demonstrating that you apply the SAME good dashboard-design thinking regardless of which tool you're using is a strong signal of transferable seniority.

KQL

- [ ] What is KQL and where does it actually show up in the Azure ecosystem?

IF ASKED, SAY THIS
Kusto Query Language — a read-only query language for large, timestamped datasets. It powers Azure Monitor / Log Analytics workspaces, Application Insights, Microsoft Sentinel (security), and Azure Data Explorer directly. If you're querying logs or security events anywhere in Azure, it's almost certainly KQL underneath.

Given his AZ-104/SC-300/SC-100 background, the most likely real-world entry points are Sentinel (security incident/threat hunting queries) and Azure Monitor Log Analytics (infra/app log queries) — both use the identical language, just different tables.

- [ ] Explain the basic shape of a KQL query — how does it compare to SQL for someone coming from a SQL background?

IF ASKED, SAY THIS
KQL is also pipe-based, like SPL: start with a table name, then pipe through operators that filter/shape/aggregate. The biggest SQL-habit trap: column order is table | where | project | summarize, not SELECT-then-WHERE — you name the table FIRST, not last.

Reading left to right: `TableName | where TimeGenerated > ago(1h) | where Level == "Error" | summarize count() by Computer` — each stage narrows/reshapes what flows to the next, same mental model as SPL's pipeline.

```
SigninLogs
| where TimeGenerated > ago(24h)
| where ResultType != "0"
| summarize FailedAttempts=count() by UserPrincipalName
| sort by FailedAttempts desc
```

- [ ] where, project, extend, summarize — what does each actually do?

IF ASKED, SAY THIS
where filters rows. project selects/renames specific columns (and drops the rest — like SQL SELECT). extend adds a new computed column WITHOUT dropping the existing ones (project would drop them unless re-listed). summarize aggregates rows into groups (KQL's GROUP BY).

The extend-vs-project distinction is the one people trip on coming from SQL: `project` gives you exactly the columns you list; `extend` gives you everything you already had PLUS the new one. Use `extend` when you're building up computed fields step by step and want to keep the originals for later stages.

```
AppRequests
| where TimeGenerated > ago(1h)
| extend DurationBucket = case(DurationMs < 100, "fast", DurationMs < 1000, "ok", "slow")
| summarize count() by DurationBucket
```

- [ ] How do you work with time windows in KQL — ago(), bin(), and between?

IF ASKED, SAY THIS
ago(1h) means "1 hour before now" — used in where TimeGenerated > ago(1h) to scope a time range. bin(TimeGenerated, 5m) rounds each timestamp down into 5-minute buckets, the standard way to build a time-series chart. between is for an explicit start/end range: TimeGenerated between (start .. end).

`bin()` combined with `summarize` is exactly how you build a Grafana/Prometheus-style time-series panel in KQL — bucket the timestamp, then aggregate within each bucket.

```
AppRequests
| where TimeGenerated > ago(6h)
| summarize RequestCount=count(), AvgDuration=avg(DurationMs) by bin(TimeGenerated, 5m)
| order by TimeGenerated asc
```

- [ ] Write a KQL query for a p95/p99 latency panel — the KQL equivalent of the Splunk perc95() query.

IF ASKED, SAY THIS
summarize with percentile() (or percentiles() for multiple at once), bucketed by bin() for the time axis — same shape as the Splunk timechart+perc95 pattern, different function name.

percentiles() computing several at once (e.g. 50th/95th/99th) in a single pass is more efficient than three separate percentile() calls if you need all three for a dashboard.

```
AppRequests
| where TimeGenerated > ago(1h)
| summarize percentiles(DurationMs, 50, 95, 99) by bin(TimeGenerated, 1m)
```

- [ ] How do joins work in KQL, and which join kind do you actually reach for most?

IF ASKED, SAY THIS
join (kind=inner|leftouter|rightouter|fullouter|leftsemi|leftanti) combines two tables on a shared key, same concept as SQL. leftouter is the most commonly reached-for in practice — keep everything from the left/primary table and enrich with matches from the right, without dropping rows that had no match.

`leftanti` is a specifically useful, less-SQL-familiar kind: it returns rows from the left table that have NO match in the right — e.g. "find all VMs in an inventory table that have NO corresponding heartbeat in the last hour," a genuinely common ops query shape.

```
Heartbeat
| where TimeGenerated > ago(1h)
| summarize LastHeartbeat=max(TimeGenerated) by Computer
| join kind=rightouter (VMInventory) on Computer
| where isnull(LastHeartbeat)
```

- [ ] What does a basic Sentinel threat-hunting KQL query look like, conceptually?

IF ASKED, SAY THIS
Same query pipeline pattern applied to security tables (SigninLogs, SecurityEvent, AuditLogs, etc.) — typically: scope a time range, filter to a suspicious pattern (repeated failed logins, impossible-travel sign-ins, an unusual process), then summarize/count to surface outliers rather than eyeballing raw events.

A very common real pattern: detect brute-force sign-in attempts by counting failed SigninLogs per user within a short window and flagging any user over a threshold — structurally identical to the failed-login query shown earlier, which is exactly the kind of query his SC-300/SC-100 background would be expected to write.

```
SigninLogs
| where TimeGenerated > ago(1h)
| where ResultType == "50126"
| summarize FailedCount=count() by UserPrincipalName, bin(TimeGenerated, 10m)
| where FailedCount > 5
```

- [ ] What's the difference between a KQL query used for an Azure Monitor alert rule vs one used for a Sentinel analytics rule?

IF ASKED, SAY THIS
Mechanically the query language is identical, but the intent differs: a Monitor alert rule usually checks operational health (error rate, resource metrics) and fires an ops alert; a Sentinel analytics rule checks for a security-relevant pattern and creates a security incident, often with entity mapping (linking the result to a specific user/host/IP for the investigation timeline).

Sentinel analytics rules typically need explicit entity mapping configured so the resulting incident is actually actionable (an analyst can pivot straight to "this user" or "this IP") rather than just a raw count.

- [ ] How do you make a KQL query reusable — is there an equivalent of a SQL view or a saved function?

IF ASKED, SAY THIS
Yes — a KQL function (let name = (params) { ... query ... };) lets you define a reusable query block, parameterized, callable like a table. In Log Analytics/Sentinel this is commonly saved as a "Function" so a team shares one canonical definition of, say, "what counts as a failed sign-in" instead of every dashboard redefining it slightly differently.

This matters operationally the same way a shared SQL view does — without it, subtly different copies of "the same" query drift apart across dashboards and nobody's quite sure which one is authoritative.

```
let FailedSignins = (lookback:timespan) {
  SigninLogs
  | where TimeGenerated > ago(lookback)
  | where ResultType != "0"
```

```
};
FailedSignins(1h) | summarize count() by UserPrincipalName
```

- [ ] A KQL query that used to run in a couple seconds is now timing out. What do you check?

IF ASKED, SAY THIS
Same instinct as the Splunk equivalent: check the time range first (did it silently widen), check for an expensive join or a regex-heavy parse operator scanning far more data than needed, and check whether summarize is running before enough filtering has narrowed the row count — filter early, aggregate late.

Ordering matters in KQL for performance even though it doesn't change correctness in simple cases — putting `where` clauses as early as possible in the pipe chain reduces the row count flowing into every subsequent (more expensive) operator like `join` or `summarize`.

- [ ] How do you query across multiple tables or even multiple Log Analytics workspaces at once in KQL?

IF ASKED, SAY THIS
union combines rows from multiple tables (or the same table across workspaces) into one result set — e.g. union AzureActivity, SecurityEvent — and the cross-workspace() / workspace() functions let a single query reach into other Log Analytics workspaces by ID, useful when a large org keeps workspaces split per team or subscription.

This comes up constantly in larger orgs: security or platform teams often need one query that spans workspaces owned by different application teams, and needing to log into each workspace separately doesn't scale operationally.

```
union AzureActivity, AzureDiagnostics
| where TimeGenerated > ago(1h)
| where Level == "Error"
| summarize count() by ResourceProvider, bin(TimeGenerated, 5m)
```

- [ ] What's an Azure Monitor Workbook, and how does it relate to a raw KQL query?

IF ASKED, SAY THIS
A Workbook is an interactive report built from one or more KQL queries, combined with parameters, charts, and text — think of it as the Azure-native equivalent of a Grafana dashboard, but purpose-built around Log Analytics/Application Insights data and shareable as a single artifact within Azure.

Workbooks are the natural place to operationalize a KQL query you've validated ad-hoc — rather than re-pasting the same query into the portal every incident, it becomes a parameterized, reusable report the whole team can open.

PagerDuty

- [ ] Define, precisely: Service, Escalation Policy, Schedule, and how they relate.

> **IF ASKED, SAY THIS**
> The relationship: an event comes into a Service → the Service's assigned Escalation Policy determines who's notified first → that policy usually points at a Schedule to resolve "who" dynamically rather than a fixed person.

- **Service** — represents one monitorable thing (an application, a component) with its own integration key(s) receiving events, its own alert grouping settings, and its own assigned escalation policy.
- **Escalation Policy** — an ordered set of rules ("notify X, wait N minutes, if unacknowledged notify Y") defining who gets paged and in what order if the first responder doesn't respond in time. One policy can be reused across multiple services.
- **Schedule** — defines who is "on-call" at any given time (rotation layers, overrides) — an escalation policy rule typically targets a Schedule (so "whoever is currently on-call" is paged) rather than a hardcoded individual.

The relationship: an event comes into a **Service** → the Service's assigned **Escalation Policy** determines who's notified first → that policy usually points at a **Schedule** to resolve "who" dynamically rather than a fixed person.

- [ ] What's a rotation "layer" in a PagerDuty schedule, and why would you need more than one?

> **IF ASKED, SAY THIS**
> A layer is one rotation pattern (e.g. "weekly rotation among these 5 engineers, handoff Tuesdays 10am"). Multiple layers stack, with higher layers taking priority over lower ones for overlapping time — the classic use is a primary layer (the normal weekly rotation) plus a higher-priority override layer used for one-off

A layer is one rotation pattern (e.g. "weekly rotation among these 5 engineers, handoff Tuesdays 10am"). Multiple layers stack, with higher layers taking priority over lower ones for overlapping time — the classic use is a primary layer (the normal weekly rotation) plus a higher-priority override layer used for one-off vacation swaps or holiday coverage without disturbing the underlying rotation pattern everyone else still follows. Follow-the-sun coverage across time zones is also commonly modeled as multiple layers, each restricted to different time-of-day windows per region.

- [ ] What does "urgency" control on a PagerDuty incident, and how can it be dynamic?

> **IF ASKED, SAY THIS**
> Urgency (high or low) controls the actual notification behavior — high-urgency triggers the full escalation policy notification chain (push/SMS/phone call per the policy), while low-urgency typically only notifies without the aggressive escalation, useful for genuinely non-critical alerts that still deserve a ticket-

Urgency (`high` or `low`) controls the actual notification behavior — high-urgency triggers the full escalation policy notification chain (push/SMS/phone call per the policy), while low-urgency typically only notifies without the aggressive escalation, useful for genuinely non-critical alerts that still deserve a ticket-like record without waking someone up. Support Hours-based urgency lets a service page loudly during business hours but downgrade automatically to low-urgency overnight for lower-severity issues — while critical/high-severity alerts should generally still page at full urgency regardless of time, which is a deliberate configuration choice, not a default.

- [ ] How do you avoid a single point of failure in an escalation policy itself (what if the on-call person's phone is dead)?

> **IF ASKED, SAY THIS**
> Multiple escalation rule levels with real time-bound handoffs (level 1: primary on-call, wait 5-10 min; level 2: secondary on-call or the whole team; level 3: the EM/manager) so an unresponsive first responder isn't a dead end. Multiple contact methods per person (push notification, SMS, phone call) configured with the

Multiple escalation rule levels with real time-bound handoffs (level 1: primary on-call, wait 5-10 min; level 2: secondary on-call or the whole team; level 3: the EM/manager) so an unresponsive first responder isn't a dead end. Multiple contact methods per person (push notification, SMS, phone call) configured with their own per-method delay, so a single failed channel (app not opening, no signal) doesn't silently swallow the page. And genuinely testing the policy periodically (not just trusting it was configured correctly once) — an escalation policy that's never actually fired end-to-end is an unverified assumption, which is exactly the kind of thing that fails silently until the one incident where it matters.

- [ ] Walk through the incident states: triggered → acknowledged → resolved. What does each actually mean operationally?

> **IF ASKED, SAY THIS**
> Triggered — a new event created an incident (or matched an existing open one) and the escalation policy is actively notifying; no one has responded yet. Acknowledged — a responder has claimed it, which pauses further escalation for a configurable window (so the rest of the team isn't paged while someone's actively work

Triggered — a new event created an incident (or matched an existing open one) and the escalation policy is actively notifying; no one has responded yet. **Acknowledged** — a responder has claimed it, which pauses further escalation for a configurable window (so the rest of the team isn't paged while someone's actively working it) — but critically, acknowledging is a commitment to work it, and an acknowledged-but-abandoned incident will typically re-escalate after the ack timeout if not resolved, precisely to prevent someone silently swallowing a page and going back to sleep. **Resolved** — the underlying issue is confirmed fixed; the incident is closed. Responders should never resolve prematurely just to stop notifications — that's what silencing/snoozing tools or a deliberate "false alarm, resolving" note are for, versus resolving something still actually broken.

- [ ] What's the difference between reassigning an incident and adding a responder?

IF ASKED, SAY THIS
Reassigning transfers ownership entirely — the new assignee is now responsible, the original person is no longer on the hook for it. Adding a responder (or looping in another team/escalation policy via "Add Responders") brings additional people into the incident without removing the original owner's responsibility — th

Reassigning transfers ownership entirely — the new assignee is now responsible, the original person is no longer on the hook for it. Adding a responder (or looping in another team/escalation policy via "Add Responders") brings additional people into the incident without removing the original owner's responsibility — the right move when you need another team's expertise for one piece (e.g. a database team to look at a specific query) but you're still driving the overall incident, versus a genuine handoff where responsibility for the whole incident moves to someone else (shift change, or it turns out to be entirely a different team's system).

- [ ] How do Status Updates / incident notes factor into a well-run incident?

IF ASKED, SAY THIS
Status updates posted directly on the PagerDuty incident (or synced from a connected Slack incident channel) create a timestamped, structured record of what was tried, what was found, and current status — visible to anyone who joins the incident midway (stakeholders, additional responders) without needing to be caught

Status updates posted directly on the PagerDuty incident (or synced from a connected Slack incident channel) create a timestamped, structured record of what was tried, what was found, and current status — visible to anyone who joins the incident midway (stakeholders, additional responders) without needing to be caught up verbally, and this same timeline becomes the raw material for the postmortem afterward. Skipping this during the heat of an incident is a common mistake that makes the postmortem much harder to write accurately afterward, since memory of exact timing/sequence degrades fast.

- [ ] What problem does Event Orchestration solve that a plain Service integration doesn't?

IF ASKED, SAY THIS
A basic Service integration just takes every incoming event and creates/updates an incident against a single fixed escalation policy. Event Orchestration adds a rules engine in front of that: conditional routing based on event payload content (route to a different escalation policy or team's service based on a severity

A basic Service integration just takes every incoming event and creates/updates an incident against a single fixed escalation policy. Event Orchestration adds a rules engine in front of that: conditional routing based on event payload content (route to a different escalation policy or team's service based on a `severity` or `component` field in the payload), suppression rules (drop known-noisy events before they ever page anyone), and dynamic enrichment/transformation of the event before it becomes an incident — letting one integration endpoint intelligently fan events out to the right team/urgency instead of needing a separate hand-configured integration per routing scenario.

- [ ] How does alert deduplication actually work, and what's the dedup_key for?

IF ASKED, SAY THIS
If a monitoring tool sends the same underlying condition repeatedly (a threshold check firing every minute while still breached), you don't want a new incident per evaluation — the dedup_key lets subsequent events with the same key update the existing open incident (or resolve it, via event_action: "resolve" with the m

```
{
  "routing_key": "...",
  "event_action": "trigger",
  "dedup_key": "high-cpu-web-server-42",
  "payload": { "summary": "CPU > 90%", "severity": "critical", "source": "web-42" }
}
```

If a monitoring tool sends the same underlying condition repeatedly (a threshold check firing every minute while still breached), you don't want a new incident per evaluation — the `dedup_key` lets subsequent events with the same key update the existing open incident (or resolve it, via `event_action: "resolve"` with the matching key, e.g. when the source monitoring tool's own condition clears) instead of spawning duplicates. Most modern integrations (Datadog, Prometheus Alertmanager) generate this automatically from the underlying alert's identity, but understanding the raw mechanism matters for building/debugging custom integrations.

- [ ] What's "alert grouping" (intelligent grouping) and why does it matter during a real large-scale incident?

IF ASKED, SAY THIS
When many independent alert sources fire in a short window because of one shared root cause (a single AZ outage triggering separate alerts from 15 different services all hosted there), alert grouping combines related alerts into a single incident instead of paging the whole org with 15 separate near-simultaneous incide

When many independent alert sources fire in a short window because of one shared root cause (a single AZ outage triggering separate alerts from 15 different services all hosted there), alert grouping combines related alerts into a single incident instead of paging the whole org with 15 separate near-simultaneous incidents. PagerDuty offers time-based grouping (bundle alerts to the same service within a configurable window), content-based grouping (bundle by matching payload fields), and an intelligent/ML-based grouping option — the operational payoff is a much saner incident

response experience (one incident, one bridge call, one timeline) versus 15 people independently investigating what's actually the same root cause without realizing it.

- [ ] How does a Datadog monitor actually trigger a PagerDuty incident, end to end?

IF ASKED, SAY THIS
You configure the Datadog-PagerDuty integration (an API-key-based connection), which creates named "service" targets in Datadog corresponding to PagerDuty services/integration keys; a monitor's notification message includes an @pagerduty- handle (as shown in section 4.2), and when the monitor transitions

You configure the Datadog-PagerDuty integration (an API-key-based connection), which creates named "service" targets in Datadog corresponding to PagerDuty services/integration keys; a monitor's notification message includes an `@pagerduty-<service-name>` handle (as shown in section 4.2), and when the monitor transitions to Alert state, Datadog posts an event to PagerDuty's Events API v2 using that service's integration key, with the monitor's own alert/recovery transitions naturally mapping to PagerDuty's trigger/resolve event actions via a shared dedup key derived from the monitor+group.

- [ ] How does Prometheus Alertmanager integrate with PagerDuty, and where does routing logic actually live?

IF ASKED, SAY THIS
Alertmanager itself does the routing/grouping/silencing (its route tree matches on alert labels to decide which receiver, and therefore which PagerDuty service, an alert goes to) — PagerDuty's own Event Orchestration rules are a second, complementary layer on top of whatever Alertmanager already decided, not a replacem

```
receivers:
  - name: pagerduty-critical
    pagerduty_configs:
      - service_key: '<PD_INTEGRATION_KEY>'
        severity: '{{ .CommonLabels.severity }}'

route:
  routes:
    - match: { severity: critical }
      receiver: pagerduty-critical
```

Alertmanager itself does the routing/grouping/silencing (its `route` tree matches on alert labels to decide which receiver, and therefore which PagerDuty service, an alert goes to) — PagerDuty's own Event Orchestration rules are a second, complementary layer on top of whatever Alertmanager already decided, not a replacement for it. A common interview distinction: Alertmanager owns "which alerts get grouped/routed to which destination," PagerDuty owns "who gets notified, in what order, with what escalation" once it receives that routed event.

- [ ] How do you actually measure and drive down alert fatigue on a team, beyond "reduce noise" as a slogan?

IF ASKED, SAY THIS
Track it as real metrics, not a vibe: pages-per-on-call-shift over time, percentage of pages that were acknowledged-but-required-no-action (a strong noise signal), after-hours page rate specifically, and mean-time-to-acknowledge trend (creeping up is often an early fatigue symptom before people start explicitly complai

Track it as real metrics, not a vibe: pages-per-on-call-shift over time, percentage of pages that were acknowledged-but-required-no-action (a strong noise signal), after-hours page rate specifically, and mean-time-to-acknowledge trend (creeping up is often an early fatigue symptom before people start explicitly complaining). Then act on the data: retire or downgrade-to-low-urgency any monitor with a high false-positive rate, push genuinely actionable-but-not-urgent alerts to a ticket/low-urgency queue instead of a page, and treat "this paged and there was nothing to do" as a bug to fix in the monitor, not a tolerable cost of doing business — every unnecessary page is real damage to trust in the on-call system, which eventually causes people to under-react to real pages too.

- [ ] As an AVP/lead, how would you structure on-call rotation to be sustainable, not just technically functional?

IF ASKED, SAY THIS
Rotation length and cadence that respects real recovery time (weekly is common; nightly/very-short rotations tend to prevent anyone from ever fully decompressing between shifts), a genuinely enforced secondary/backup layer so primary on-call isn't a true single point of failure for their entire week, explicit compensat

Rotation length and cadence that respects real recovery time (weekly is common; nightly/very-short rotations tend to prevent anyone from ever fully decompressing between shifts), a genuinely enforced secondary/backup layer so primary on-call isn't a true single point of failure for their entire week, explicit compensation or time-off-in-lieu policy for after-hours pages (a concrete signal that leadership treats on-call burden as real work, not free overtime), a blameless review of pages that shouldn't have happened, and — the leadership-specific piece — pushing back on feature roadmap pressure that would grow on-call burden without a corresponding investment in the reliability work that would reduce it. This is exactly the kind of judgment question an AVP panel is listening for, distinct from the mechanics questions above.

- [ ] What makes a postmortem actually useful versus a box-checking exercise, and how does PagerDuty's incident data feed into it?

IF ASKED, SAY THIS

A useful postmortem is blameless (focused on system/process gaps, not "who made the mistake"), has an accurate timeline (this is where PagerDuty's own timestamped incident log — trigger time, ack time, status updates, resolve time — becomes the objective backbone rather than relying on reconstructed memory), states imp

A useful postmortem is blameless (focused on system/process gaps, not "who made the mistake"), has an accurate timeline (this is where PagerDuty's own timestamped incident log — trigger time, ack time, status updates, resolve time — becomes the objective backbone rather than relying on reconstructed memory), states impact in concrete terms (duration, affected users/requests, revenue if applicable), and produces specific, owned, tracked action items — not vague ones like "improve monitoring" with no owner or deadline that quietly never get done. PagerDuty's Postmortem feature (or an external doc pulling from the incident's timeline export) exists specifically to reduce the friction of assembling that accurate timeline, since reconstructing it from memory hours or days later is where inaccuracy creeps in.

- [ ] Scenario: on-call engineers are complaining of being paged 15+ times a night, mostly for things that self-resolve in under 2 minutes. What do you actually change?

IF ASKED, SAY THIS
Don't start by asking people to "just ignore the noisy ones" — fix it structurally: add an evaluation window to the source monitors so a 90-second blip doesn't cross the alert threshold (section 4.2's monitor-tuning), introduce auto-resolve logic so self-healing conditions close out without ever paging a human in the f

Don't start by asking people to "just ignore the noisy ones" — fix it structurally: add an evaluation window to the source monitors so a 90-second blip doesn't cross the alert threshold (section 4.2's monitor-tuning), introduce auto-resolve logic so self-healing conditions close out without ever paging a human in the first place if they resolve within a defined grace period, and audit which specific monitors account for the bulk of the 15 nightly pages — it's very often a small number of consistently-noisy sources, not a uniformly noisy system, so fixing the worst 3-4 offenders often solves most of the volume. Track the pages-per-night metric before/after as evidence the fix actually worked, not just anecdotal relief.

- [ ] Scenario: a major incident spans three teams, each with their own PagerDuty service and escalation policy. How do you avoid three uncoordinated incident responses?

IF ASKED, SAY THIS
This is exactly what alert grouping/Event Orchestration (section 5.3) and "Add Responders"/multi-team incident linking are for: if the three teams' alerts are recognizably related (same time window, related services, or a shared upstream cause), grouping rules should combine them into one parent incident rather than th

This is exactly what alert grouping/Event Orchestration (section 5.3) and "Add Responders"/multi-team incident linking are for: if the three teams' alerts are recognizably related (same time window, related services, or a shared upstream cause), grouping rules should combine them into one parent incident rather than three isolated ones firing independently. If they weren't automatically grouped, the incident commander role matters here — whoever notices the overlap should manually merge/link the incidents and designate a single incident commander coordinating across all three teams' responders in one shared channel/bridge, rather than three teams independently investigating what's actually one root cause without realizing they're looking at the same problem from different angles.

Linux & Networking

- [ ] How do Linux cgroups and namespaces underpin container isolation, and why does that matter when you're troubleshooting a "noisy neighbor" problem?

> **IF ASKED, SAY THIS**
> Namespaces (pid, net, mnt, uts, ipc, user) give a process its own isolated view of the system; cgroups (v1 hierarchies, or the now-default unified v2 hierarchy) account for and cap the resources — cpu, memory, io, pids — a group of processes can use. Containers are really just processes wrapped in b

Namespaces (pid, net, mnt, uts, ipc, user) give a process its own isolated view of the system; cgroups (v1 hierarchies, or the now-default unified v2 hierarchy) account for and cap the resources — cpu, memory, io, pids — a group of processes can use. Containers are really just processes wrapped in both.

For a noisy-neighbor problem, isolation only helps if limits are actually set — a container with no cpu/memory requests set can still starve others sharing the node. Diagnose with:

```
systemd-cgtop
cat /sys/fs/cgroup/<path>/cpu.stat
cat /sys/fs/cgroup/<path>/memory.current
```

then check whether the offending workload actually has limits enforced (in Kubernetes: `requests` / `limits` on the pod spec).

- [ ] A production host reports high load average but low CPU utilization. What's your diagnostic path?

> **IF ASKED, SAY THIS**
> Load average counts processes waiting on the CPU and processes in uninterruptible sleep (D state) — usually blocked on disk or network I/O, not CPU. High load with low CPU% almost always means I/O wait.

Load average counts processes waiting on the CPU **and** processes in uninterruptible sleep (D state) — usually blocked on disk or network I/O, not CPU. High load with low CPU% almost always means I/O wait.

```
vmstat 1 5        # watch the 'wa' column
iostat -x 1 5      # per-device %util, await
ps aux | awk '$8=="D"'   # who's stuck in D state
free -m            # rule out swap thrash
```

Common culprits: a hung NFS mount, a failing disk with retry storms, or heavy swapping. Fix the root cause, don't just restart the box.

- [ ] Walk through what happens when a systemd unit with Restart=on-failure keeps crash-looping — how do you stop the churn safely?

> **IF ASKED, SAY THIS**
> systemd applies its restart-rate limiting (StartLimitIntervalSec / StartLimitBurst) — once a unit exceeds the burst count inside the interval, systemd stops trying and marks it failed, so it won't spin forever by default.

systemd applies its restart-rate limiting (`StartLimitIntervalSec` / `StartLimitBurst`) — once a unit exceeds the burst count inside the interval, systemd stops trying and marks it failed, so it won't spin forever by default.

```
systemctl status myapp
journalctl -u myapp -f --since "10 min ago"
systemctl reset-failed myapp   # after fixing root cause
```

If you need to stop the churn immediately while you investigate, `systemctl stop myapp` (or mask it) rather than letting it keep flapping. This is the same underlying pattern as Kubernetes CrashLoopBackOff — a bounded, backing-off retry, not infinite hammering.

- [ ] Explain the CAP theorem in practical infrastructure terms — what does it actually force you to decide?

> **IF ASKED, SAY THIS**
> A distributed system can only fully guarantee two of Consistency, Availability, and Partition tolerance at once. In the real world, network partitions will happen, so partition tolerance isn't optional — the real decision is what you do during a partition: stay consistent and refuse some requests (C

A distributed system can only fully guarantee two of Consistency, Availability, and Partition tolerance at once. In the real world, network partitions *will* happen, so partition tolerance isn't optional — the real decision is what you do **during** a partition: stay consistent and refuse some requests (CP), or stay available and risk serving stale/divergent data (AP).

PACELC extends this: even with no partition, you're trading latency against consistency. Concretely — etcd is CP (it will reject writes rather than risk split-brain); DynamoDB/Cassandra lean AP with tunable consistency levels per query.

- [ ] Why does etcd use Raft consensus, and what does that mean for how many nodes you run?

IF ASKED, SAY THIS
Raft achieves consensus through leader election plus log replication, and requires a majority quorum to commit a write. Quorum = floor(n/2)+1, which is why clusters run odd numbers of members:

Raft achieves consensus through leader election plus log replication, and requires a **majority quorum** to commit a write. Quorum = floor(n/2)+1, which is why clusters run odd numbers of members:

- 3 nodes → quorum 2 → tolerates 1 failure
- 5 nodes → quorum 3 → tolerates 2 failures
- 4 nodes tolerates only 1 failure too, but wastes a node — even numbers add cost without added fault tolerance

This matters directly for you because etcd backs the Kubernetes control plane's state — an etcd quorum loss means the API server can't accept writes, even if kubelets and workloads are still running fine.

- [ ] What's a split-brain scenario in a highly available cluster, and how do you prevent it?

IF ASKED, SAY THIS
A network partition leaves two sides of a cluster each believing they are the primary/leader. Both start accepting writes independently, and the data diverges — by the time the partition heals, you have two conflicting histories to reconcile, sometimes unrecoverably.

A network partition leaves two sides of a cluster each believing *they* are the primary/leader. Both start accepting writes independently, and the data diverges — by the time the partition heals, you have two conflicting histories to reconcile, sometimes unrecoverably.

Prevention: quorum-based consensus (a minority partition simply can't elect a leader, per the Raft math above), fencing/STONITH ("shoot the other node in the head") in classic Pacemaker/Corosync-style HA clusters, and witness or tiebreaker nodes when you have an even split risk (e.g., exactly two data centers).

- [ ] You're wiring a batch job into an automated retry loop. What does making it idempotent actually require in practice?

IF ASKED, SAY THIS
Idempotent means running it twice has the same effect as running it once. In practice that means: an idempotency key on the operation (so a retried request is recognized as a duplicate), upsert semantics instead of blind insert/increment, and checking current state before acting rather than assuming

Idempotent means running it twice has the same effect as running it once. In practice that means: an idempotency key on the operation (so a retried request is recognized as a duplicate), upsert semantics instead of blind insert/increment, and checking current state before acting rather than assuming a clean starting point.

Without this, "just retry on failure" is dangerous — a payment job retried after a timeout (when the first attempt actually succeeded server-side) can double-charge; a provisioning job can double-create resources. Automation without idempotency just automates the damage faster.

- [ ] How does clock skew quietly break distributed systems, and what's your first move if you suspect it?

IF ASKED, SAY THIS
Skewed clocks break TLS certificate validation, Kerberos ticket auth, distributed-tracing timelines (spans appearing to start before their parent), log correlation across hosts, and last-write-wins conflict resolution silently picking the wrong "winner."

Skewed clocks break TLS certificate validation, Kerberos ticket auth, distributed-tracing timelines (spans appearing to start before their parent), log correlation across hosts, and last-write-wins conflict resolution silently picking the wrong "winner."

```
chronyc tracking
timedatectl status
chronyc sources -v
```

Check NTP source health and drift first — it's a common, easy-to-miss root cause behind symptoms that look like application bugs.

- [ ] Explain at-least-once, at-most-once, and exactly-once delivery semantics, and where each is an acceptable default for a system you'd operate.

IF ASKED, SAY THIS
At-most-once : fire and forget, message can be lost, never duplicated — fine for best-effort metrics/telemetry. At-least-once : retried until acknowledged, so duplicates are possible — the common default, acceptable when the consumer is idempotent. Exactly-once : in real systems this is usually "eff

At-most-once: fire and forget, message can be lost, never duplicated — fine for best-effort metrics/telemetry. **At-least-once**: retried until acknowledged, so duplicates are possible — the common default, acceptable when the consumer is idempotent. **Exactly-once**: in real systems this is usually "effectively-once," built from at-least-once delivery plus idempotent processing (e.g., Kafka's transactional producer/consumer semantics), not a delivery guarantee that exists on the wire by itself.

- [ ] A downstream dependency starts timing out intermittently. How do circuit breakers and bulkheads limit the blast radius?

> **IF ASKED, SAY THIS**
> A circuit breaker tracks failure rate to a dependency and trips from closed → open once it crosses a threshold, failing fast instead of piling up slow/hanging calls; it periodically half-opens to test recovery. A bulkhead isolates the resource pool (threads, connections) used for that one dependency

A **circuit breaker** tracks failure rate to a dependency and trips from closed → open once it crosses a threshold, failing fast instead of piling up slow/hanging calls; it periodically half-opens to test recovery. A **bulkhead** isolates the resource pool (threads, connections) used for that one dependency so its slowness can't exhaust resources shared with calls to healthy dependencies.

Without either, one flaky dependency causes thread/connection exhaustion that cascades into an outage of everything, not just the calls that actually depend on it — the classic thundering-herd failure. Envoy/Istio implement this at the mesh layer; resilience4j is a common in-app option.

- [ ] A Linux server has load average of 24 but only 4 CPUs and CPU usage shows only 15%. What is happening and how do you investigate?

> **IF ASKED, SAY THIS**
> This is the classic IO-wait vs CPU confusion question: What this means: Load average counts ALL processes that are running OR waiting to run OR waiting for IO. CPU percentage only counts processes actively computing. 24 load with 15% CPU = 20+ processes are blocked waiting for something, not computi

This is the classic IO-wait vs CPU confusion question:

What this means: Load average counts ALL processes that are running OR waiting to run OR waiting for IO. CPU percentage only counts processes actively computing. 24 load with 15% CPU = 20+ processes are blocked waiting for something, not computing.

Step 1 - Confirm with vmstat:
`vmstat 1 5`
Look at the "wa" column (IO wait). If wa is above 30%, the server is stuck waiting for disk or network IO.

Step 2 - Identify the IO culprit:
`iostat -x 1 5`
Look for a disk with await above 100ms or %util at 100%. A disk at 100% util is the bottleneck.
`iotop -o` shows which processes are generating the IO.

Step 3 - Find D-state processes:
`ps aux | awk '$8 ~ /D/ {print}'`
D state = uninterruptible sleep = process is waiting for IO and cannot be killed. If many processes are in D state, you have severe IO bottleneck.

Common causes:
- Application writing too many logs to a slow disk (fix: reduce log verbosity, move to faster storage)
- NFS mount that has gone stale (fix: unmount and remount, or the NFS server is down)
- Failing disk causing retries: `dmesg | grep -i error` will show SCSI/ATA errors
- Elasticsearch or database doing heavy indexing

On a Kubernetes node: This exact scenario causes node pressure → K8s starts evicting pods. You will see eviction events in `kubectl describe node NODE-NAME` before the node becomes NotReady.

- [ ] How do you find which process is holding a port and how do you handle zombie processes?

> **IF ASKED, SAY THIS**
> Two common SRE operational questions: Finding which process holds a port: ss -tulnp | grep :8080 - modern way, fast netstat -tulnp | grep :8080 - older systems lsof -i :8080 - also shows process name fuser 8080/tcp - just shows the PID The output shows you the PID and process name. Then: ps aux | gr

Two common SRE operational questions:

Finding which process holds a port:
`ss -tulnp | grep :8080` - modern way, fast
`netstat -tulnp | grep :8080` - older systems
`lsof -i :8080` - also shows process name
`fuser 8080/tcp` - just shows the PID

The output shows you the PID and process name. Then:
`ps aux | grep PID` - see what the process is
`kill -15 PID` - send graceful shutdown signal
`kill -9 PID` - force kill if -15 does not work

Real scenario: Application fails to start with "Address already in use". The previous instance crashed but did not release the port. This port conflict is resolved by killing the zombie process.

Zombie processes:
A zombie is a process that has finished executing but its parent has not read its exit status. It stays in the process table as "Z" (defunct).
`ps aux | grep "Z"` or `ps aux | awk '$8 == "Z"'`

Why zombies are usually not a problem: they consume no CPU, no memory, just one row in the process table. One or two zombies is harmless.

Why zombies become a problem: if the application creates thousands of zombie children (a bug in the parent process not reaping children), the process table fills up. New processes cannot be created. The system becomes effectively frozen.

Fix: you cannot kill a zombie directly - it is already dead. You must kill or fix the PARENT process so it reaps its children. Send SIGCHLD to the

parent. If the parent is the application, fix the code to call wait()/waitpid().

In Kubernetes: this is why PID 1 in containers should use an init process (like tini). Tini reaps zombie child processes automatically.

- [ ] How do you use tcpdump and what have you used it for in production SRE work?

> **IF ASKED, SAY THIS**
> tcpdump is the SRE's last resort when all other debugging fails: Common production use cases: 1. Verify traffic is reaching the server: tcpdump -i eth0 port 8080 -n If you see SYN packets arriving but no SYN-ACK, the server is not listening or the connection is being dropped by iptables. 2. Diagnose

tcpdump is the SRE's last resort when all other debugging fails:

Common production use cases:

1. Verify traffic is reaching the server:
`tcpdump -i eth0 port 8080 -n`
If you see SYN packets arriving but no SYN-ACK, the server is not listening or the connection is being dropped by iptables.

2. Diagnose slow database connections:
`tcpdump -i eth0 host DB-IP port 5432 -n -w /tmp/db.pcap`
Open in Wireshark. Look at the time delta between the query being sent and the response arriving. If the delta is 2 seconds, the DB is slow, not the app.

3. Prove that traffic is being dropped by a firewall:
`tcpdump -i eth0 'tcp[tcpflags] & (tcp-rst) != 0'`
RST packets mean the connection is being actively rejected. If you see RST from the server, the service is not listening. If you see RST from a firewall, the traffic is being blocked.

4. Diagnose DNS resolution failures:
`tcpdump -i eth0 port 53 -n`
Is the DNS query being sent? Is the response coming back? What is the response? (NXDOMAIN = name does not exist, SERVFAIL = DNS server issue)

5. Verify TLS handshake:
`tcpdump -i eth0 'tcp port 443 and (tcp[((tcp[12] & 0xf0) >> 2)] = 22)'`
Filters TLS handshake packets. If you see ClientHello but no ServerHello, the server is not responding to TLS.

Important: On a production server with high traffic, add filters immediately. Unfiltered tcpdump can generate gigabytes of data per minute and fill the disk.

- [ ] What are the most important Linux commands an SRE must know?

> **IF ASKED, SAY THIS**
> Process and CPU: top / htop - real-time process view ps aux - snapshot of all processes pidstat -u 1 5 - CPU per process over time strace -p PID - system calls a process is making lsof -p PID - files and sockets open by a process Memory: free -h - total, used, free

Process and CPU:
top / htop - real-time process view
ps aux - snapshot of all processes
pidstat -u 1 5 - CPU per process over time
strace -p PID - system calls a process is making
lsof -p PID - files and sockets open by a process

Memory:
free -h - total, used, free, cached memory
vmstat 1 5 - memory, swap, CPU stats over time
/proc/meminfo - detailed memory breakdown

Disk and IO:
df -h - disk space per filesystem
du -sh /var/log - size of a directory
iostat -x 1 5 - disk IO stats (await, util)
iotop - IO per process (like top for disk)

Network:
ss -tulnp - listening sockets and which process owns them
netstat -s - network statistics
tcpdump -i eth0 port 8080 - capture packets on port 8080
curl -v - verbose HTTP request for debugging

Logs:
tail -f /var/log/app.log - follow a log file
journalctl -u myapp -f - systemd service logs
grep -i error /var/log/app.log - search for errors
awk '{print $9}' access.log | sort | uniq -c - HTTP status code count

- [ ] How do you investigate a server that has run out of disk space?

> **IF ASKED, SAY THIS**

Disk full is a common P2 or P1 event depending on the service affected. Step 1 - Confirm and identify which filesystem is full df -h Look for 100% or Use% above 95. Step 2 - Find what is consuming the space du -sh /* 2>/dev/null | sort -rh | head -20 Drill down into the largest directory: du -sh /va

Disk full is a common P2 or P1 event depending on the service affected.

Step 1 - Confirm and identify which filesystem is full
df -h
Look for 100% or Use% above 95.

Step 2 - Find what is consuming the space
du -sh /* 2>/dev/null | sort -rh | head -20
Drill down into the largest directory:
du -sh /var/log/* | sort -rh | head -10

Step 3 - Common culprits
- Application logs not rotated: /var/log/myapp/*.log
- Core dump files: find / -name 'core.*' -type f
- Docker layers: docker system df then docker system prune
- Old journal logs: journalctl --disk-usage
Clear with: journalctl --vacuum-size=500M
- Temp files: ls -lh /tmp

Step 4 - Immediate relief
Compress and move old logs:
gzip /var/log/myapp/app.log.2024-01-14
mv *.gz /mnt/archive/

Delete confirmed unnecessary files.
Do NOT blindly delete files you are unsure about.

Step 5 - Root cause fix
Add logrotate config if missing.
Add CloudWatch disk alert at 80% (before it hits 100%).
In K8s: set resource requests on PVCs and add StorageClass limits.
Long-term: evaluate whether the volume needs to be resized (AWS: extend EBS volume).

- [ ] Give the essential Linux command cheat sheet an SRE should have cold before any interview. ▲

IF ASKED, SAY THIS
Process/CPU: top, ps aux --sort=-%cpu, pidstat. Memory: free -h, vmstat. Disk: df -h, du -sh, iostat -x. Network: ss -tulnp, tcpdump, dig. Logs: journalctl, tail -f, grep.

Grouped by what you're diagnosing rather than alphabetically -- this is how to actually recall them under interview pressure, by symptom category rather than command name.

```
# --- PROCESS / CPU ---
top -bn1                          # snapshot of CPU/mem, all processes
ps aux --sort=-%cpu | head -10    # top CPU consumers
pidstat -u 1 5                    # per-process CPU, sampled
strace -p PID -c -f               # syscall summary for a running process

# --- MEMORY ---
free -h                           # overall memory + swap
vmstat 1 5                        # memory/CPU/IO snapshot over time; watch 'wa' column
ps aux --sort=-%mem | head -10    # top memory consumers

# --- DISK / IO ---
df -h                              # filesystem usage
du -sh /var/log/*                  # what's actually eating space
iostat -x 1 5                      # per-disk IO, watch %util and await
iotop -o                           # which process is generating IO

# --- NETWORK ---
ss -tulnp | grep :8080              # what's listening on a port (modern)
lsof -i :8080                       # same, shows process name too
tcpdump -i eth0 port 443 -w cap.pcap  # packet capture
dig +short example.com              # DNS lookup

# --- LOGS ---
journalctl -u myservice -f          # follow systemd unit's logs live
tail -f /var/log/app.log | grep ERROR
```

- [ ] Walk through what actually happens, step by step, when you type a URL and hit enter -- the classic systems-fundamentals question. ▲

IF ASKED, SAY THIS
DNS resolution (name -> IP, checking local cache/hosts file/resolver chain first) -> TCP three-way handshake to that IP on port 443 -> TLS handshake (cert validation, key exchange) -> HTTP request sent over the now-encrypted connection -> server processes and responds -> browser renders.

Interviewers use this to probe how deep your networking model actually goes -- a strong answer names where each step could fail (DNS misconfiguration, TCP connection refused vs. timeout, a cert validation failure, a 5xx from the server) rather than just reciting the happy path.

- [ ] TCP vs. UDP -- what's the actual tradeoff, and what's a real infrastructure example of each?

IF ASKED, SAY THIS
TCP: connection-oriented, guarantees ordered delivery and retransmits lost packets -- used wherever correctness matters more than latency (HTTP/HTTPS, database connections). UDP: connectionless, no delivery guarantee, lower overhead -- used where speed matters more than occasional loss (DNS queries, video/voice streaming, some metrics protocols like StatsD).

A concrete infra example worth having ready: DNS primarily uses UDP for its speed (a lost query just gets retried by the resolver), but falls back to TCP for responses too large for a single UDP packet (like zone transfers) -- a real example of a protocol using both deliberately.

- [ ] How does a Layer 4 load balancer differ from a Layer 7 load balancer, and when do you actually need L7?

IF ASKED, SAY THIS
L4 (like a classic Network Load Balancer) routes based on IP/port only -- fast, protocol-agnostic, can't see HTTP content. L7 (like an Application Load Balancer) terminates and reads the actual HTTP request -- enabling path-based routing, header-based routing, and things like sticky sessions or WAF rules that require understanding the request itself.

The decision rule to state plainly: if routing decisions need to look at anything inside the request (URL path, host header, cookies), you need L7; if you just need to spread raw TCP/UDP connections across targets as fast as possible, L4 is simpler and lower-latency.

- [ ] How does CIDR notation and subnetting actually work -- can you do the math live if asked?

IF ASKED, SAY THIS
The /N number is how many bits are fixed for the network portion -- fewer host bits means fewer usable addresses. A /24 has 8 host bits = 256 addresses (254 usable after network+broadcast). Each bit you add to the prefix (/24 to /25) halves the address count. To subnet a /24 into 4 equal pieces, you move to /26 (4 host bits fewer... actually 2 bits borrowed = 4 subnets of 64 addresses each): 10.0.0.0/26, 10.0.0.64/26, 10.0.0.128/26, 10.0.0.192/26.

The fast mental-math trick worth having ready: number of addresses in a /N subnet = 2^(32-N). /24 = 2^8 = 256. /26 = 2^6 = 64. /28 = 2^4 = 16. Being able to produce this instantly, rather than needing a calculator, is exactly what this question is testing.

- [ ] What are the main DNS record types, and when do you use each?

IF ASKED, SAY THIS
A -- hostname to IPv4 address. AAAA -- hostname to IPv6. CNAME -- alias to another hostname (can't coexist with other records on the same name). MX -- mail server priority list. TXT -- arbitrary text, commonly used for domain verification and SPF/DKIM email authentication. NS -- delegates a subdomain to other nameservers. A CNAME at the zone apex (the bare domain, no subdomain) is technically disallowed by the DNS spec -- that's exactly why cloud providers offer an ALIAS/ANAME record type as a CNAME-like workaround at the apex.

The apex-CNAME restriction is a good one to have ready -- it directly explains why you can't just CNAME your bare domain to a load balancer the same way you would a subdomain, and why ALIAS records (Route53) or ANAME (some other DNS providers) exist as the fix.

- [ ] How does a router actually decide where to send a packet -- what's a routing table doing?

IF ASKED, SAY THIS
A routing table is a list of destination-network-to-next-hop mappings, matched by LONGEST PREFIX MATCH -- if a packet's destination matches both a broad route (0.0.0.0/0, the default route) and a more specific one (10.0.1.0/24), the more specific match always wins. This is exactly the mechanism behind a VPC route table sending most traffic to a NAT gateway (default route) while sending traffic destined for a peered VPC's CIDR down a more specific route to the peering connection instead.

Connecting this directly to a VPC route table is what makes the answer land as applied knowledge rather than textbook networking -- the same longest-prefix-match principle is exactly what you're configuring every time you add a route in AWS/Azure.

- [ ] What's a CDN actually doing, and why does it help even for dynamic, non-cacheable content?

IF ASKED, SAY THIS
A CDN caches content at edge locations geographically close to users, cutting latency for cacheable content (static assets, images) by serving from nearby rather than round-tripping to origin. For dynamic/non-cacheable content, it still helps two ways: TCP/TLS connection setup terminates at the nearby edge (fewer expensive round-trips before the request even reaches your origin), and many CDNs offer a persistent, already-warm connection from edge back to origin, avoiding a fresh handshake per user request.

The connection-termination benefit for non-cacheable content is the detail people miss -- it's easy to think of a CDN as "only helps static files," but the TLS/TCP handshake savings apply to every request that touches the edge, cacheable or not.

- [ ] Given a full outage report, how do you reason through which network layer is actually broken -- physical, network, transport, or application?

IF ASKED, SAY THIS
Work bottom-up with concrete tests: can you reach the host at all (ping -- network layer, ICMP)? Can you open the specific port (telnet/nc to the port -- transport layer, TCP)? Does the service respond correctly at the port (curl/application-level request -- application layer)? A failure at each stage points at a different fix: ping fails = routing/firewall/host-down; port unreachable but ping works = service down or a security-group/firewall rule blocking that port specifically; port open but app errors = the application itself, not networking at all.

This bottom-up, layer-by-layer diagnostic order is a genuinely useful structure to state explicitly in an interview -- it demonstrates a repeatable troubleshooting METHOD rather than jumping straight to guessing based on the symptom's surface appearance.

- [ ] Give a real production one-liner using grep, awk, and sed each, and what each is actually for.

IF ASKED, SAY THIS
grep finds lines matching a pattern. awk processes structured/columnar text (splits by whitespace or a delimiter, operates per-field). sed does stream find-and-replace transformation. In one pipeline: grep 'ERROR' app.log | awk '{print $1, $2, $NF}' | sed 's/,/./g' -- filter to error lines, print timestamp/level/last-field, then normalize commas to periods.

The distinction worth stating precisely: grep is for FINDING lines, awk is for RESHAPING columnar data, sed is for TRANSFORMING text in place -- knowing which of the three actually fits a given task (rather than reaching for whichever one you remember syntax for) is the real skill being tested.

```
# Top 10 IPs hitting an endpoint from an access log
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -10

# Replace all occurrences of an old hostname in a batch of config files
sed -i 's/old-host.internal/new-host.internal/g' *.conf

# Extract just the error messages from a mixed log
grep -oP 'ERROR: \K.*' app.log
```

- [ ] Explain Linux file permissions and ownership -- chmod, chown, and what umask actually controls.

IF ASKED, SAY THIS
Permissions are three sets of read/write/execute -- owner, group, everyone else -- shown as e.g. rwxr-xr-- (owner: all, group: read+execute, others: read only), or numerically 754 (7=rwx, 5=r-x, 4=r--). chown changes WHO owns a file (user and/or group); chmod changes WHAT they can do with it. umask sets the DEFAULT permissions new files get, by subtracting from the system default (typically 666 for files, 777 for directories) -- a umask of 022 results in new files created as 644.

The umask-as-subtraction framing is the part people get backwards -- umask doesn't directly set permissions, it masks OUT bits from the default, which is why a HIGHER umask value means MORE restrictive resulting permissions, the opposite of how chmod numbers work.

```
chmod 750 script.sh        # owner: rwx, group: r-x, other: none
chmod u+x,g-w deploy.sh     # symbolic form: add execute for owner, remove write for group
chown app:app /var/www/app  # set both user and group owner
umask 027                   # new files: 640, new dirs: 750
```

- [ ] How do you manage services with systemd -- the commands that actually matter day to day?

IF ASKED, SAY THIS
systemctl status/start/stop/restart for basic lifecycle; systemctl enable to make a service start on boot (separate from starting it now); journalctl -u -f to tail its logs live. systemctl daemon-reload after editing a unit file, or the change won't take effect. systemctl is-active / is-enabled for quick scripted health checks.

The enable-vs-start distinction is a classic gotcha worth stating explicitly -- starting a service doesn't make it survive a reboot, and enabling it doesn't start it right now; production deploys typically need both, and forgetting daemon-reload after a unit-file edit is one of the most common "why isn't my change taking effect" moments.

```
systemctl restart nginx
systemctl enable --now nginx    # enable AND start in one command
journalctl -u nginx -f --since "10 min ago"
systemctl daemon-reload         # required after editing a .service file
```

- [ ] How do you find files matching specific criteria and act on all of them at once -- find and xargs?

IF ASKED, SAY THIS
find searches by name, type, size, age, permissions, and more; xargs takes a list of items (often piped from find) and runs a command against each, batching multiple items per invocation for efficiency rather than spawning one process per file. The classic safe pattern uses -print0 and -0 to correctly handle filenames with spaces or special characters, which a naive pipe would break on.

The -print0/-0 pairing is worth including specifically -- a plain find | xargs pipeline silently breaks on filenames containing spaces (a genuinely common real-world gotcha), and knowing the null-delimited fix is a real signal of hands-on shell scripting experience.

```
# Delete log files older than 30 days
find /var/log/app -name "*.log" -mtime +30 -delete

# Safely handle filenames with spaces
find . -name "*.tmp" -print0 | xargs -0 rm -f

# Find files over 100MB and list them with sizes
find /data -type f -size +100M -exec ls -lh {} \;
```

- [ ] Walk through your approach to hardening a new RHEL/Rocky Linux server before it goes into production.

IF ASKED, SAY THIS
Start from a minimal install (reduce attack surface), apply a CIS Benchmark or DISA STIG baseline as the hardening standard, disable/remove unused services, enforce SELinux in enforcing mode (not permissive — a common shortcut that quietly defeats the control), configure firewalld with explicit allow rules rather than

Start from a minimal install (reduce attack surface), apply a CIS Benchmark or DISA STIG baseline as the hardening standard, disable/remove unused services, enforce **SELinux in enforcing mode** (not permissive — a common shortcut that quietly defeats the control), configure `firewalld` with explicit allow rules rather than a blanket-open posture, centralize auth (LDAP/AD via SSSD) instead of local accounts, enable `auditd` for compliance logging, and bake all of this into a golden image / Ansible role / Kickstart profile so it's repeatable — not a manual checklist run once per box.

- [ ] How do you approach OS patching across a large fleet of mission-critical R&D Linux servers without breaking things?

IF ASKED, SAY THIS
Staged rollout, not a big-bang window: canary/test ring first, validate critical services and app compatibility, then roll through pre-defined rings with bake time between them. Maintain a patch/reboot maintenance calendar coordinated with R&D teams — patching an EDA compute farm mid-simulation is a very different conv

Staged rollout, not a big-bang window: canary/test ring first, validate critical services and app compatibility, then roll through pre-defined rings with bake time between them. Maintain a patch/reboot maintenance calendar coordinated with R&D teams — patching an EDA compute farm mid-simulation is a very different conversation than patching an idle dev box. Automate via Ansible or Satellite/Foreman for RHEL fleets rather than manual per-host patching, and always have a rollback/snapshot plan for anything that can't tolerate a failed patch.

- [ ] Explain systemd's role in modern Linux administration — what would you actually be troubleshooting with it day to day?

IF ASKED, SAY THIS
systemd is the init system and service manager — units (service, socket, mount, timer) define how processes start, their dependencies, and restart behavior. Day to day: systemctl status for current state and a recent log tail, journalctl -u for full service logs, systemctl list-dependencies when a service

systemd is the init system and service manager — units (service, socket, mount, timer) define how processes start, their dependencies, and restart behavior. Day to day: `systemctl status <unit>` for current state and a recent log tail, `journalctl -u <unit>` for full service logs, `systemctl list-dependencies` when a service won't start because something it depends on failed, and understanding target units when a box comes up in an unexpected state after reboot. Timers (systemd's cron-replacement) come up often for scheduled maintenance on modern RHEL/Ubuntu.

- [ ] What's your troubleshooting process when a Linux server's disk I/O is degraded and an R&D team is complaining about slow builds?

IF ASKED, SAY THIS
iostat -x for per-device utilization/await/queue depth to confirm it's actually I/O-bound and identify the hot device, iotop to find the driving process, check for swap thrashing (vmstat , free) since memory pressure often masquerades as I/O, check for near-full disks (behave differently than expected, especially on

`iostat -x` for per-device utilization/await/queue depth to confirm it's actually I/O-bound and identify the hot device, `iotop` to find the driving process, check for swap thrashing (`vmstat` , `free`) since memory pressure often masquerades as I/O, check for near-full disks (behave differently than expected, especially on copy-on-write filesystems), and check underlying storage health (RAID rebuild, a failing disk, or a noisy-neighbor/throttling issue on virtualized storage). Also confirm it's actually I/O and not CPU/network (dependency fetches) — don't assume the reported symptom is the root cause.

- [ ] How would you approach performance tuning for a Linux host running compute-intensive engineering workloads (builds, simulations)?

IF ASKED, SAY THIS
Profile before tuning — top / htop for a quick read, perf for CPU-level profiling if genuinely compute-bound — and understand the actual bottleneck before reaching for kernel tunables. Common levers: CPU governor/frequency scaling (matters a lot for burst compute), NUMA awareness for multi-socket servers (a process pin

Profile before tuning — `top` / `htop` for a quick read, `perf` for CPU-level profiling if genuinely compute-bound — and understand the actual bottleneck before reaching for kernel tunables.

Common levers: CPU governor/frequency scaling (matters a lot for burst compute), **NUMA awareness** for multi-socket servers (a process pinned across NUMA nodes pays a real latency cost), transparent huge pages (can help or hurt — test, don't assume), `ulimits` (file descriptors, process counts — a classic silent failure for large parallel builds), and mount options (`noatime` for build trees).

- [ ] What's SELinux actually doing, and how do you troubleshoot an SELinux-related access denial without just disabling it?

IF ASKED, SAY THIS
SELinux enforces mandatory access control via security contexts/labels, independent of standard Unix permissions — even root can be denied by policy. Troubleshooting: ausearch -m avc -ts recent or /var/log/audit/audit.log for AVC denials, then audit2allow to see what a policy exception would look like (review before ap

SELinux enforces mandatory access control via security contexts/labels, independent of standard Unix permissions — even root can be denied by policy.

Troubleshooting: `ausearch -m avc -ts recent` or `/var/log/audit/audit.log` for AVC denials, then `audit2allow` to see what a policy exception would look like (review before applying) — or more often, fix the actual mislabeled file with `restorecon` / `chcon` . "Just set it to permissive" is the instinct to resist in an interview answer — it's a real control, especially relevant given this role touches audit/compliance.

- [ ] How would you design storage for a Linux server supporting high-throughput build/simulation workloads — LVM, filesystem choice, mount strategy?

IF ASKED, SAY THIS
LVM for flexibility (resize without downtime, snapshot before risky changes) at a small overhead cost. Filesystem: XFS as a strong general default for large files/high throughput (common RHEL default), ext4 for general-purpose simplicity. Separate mount points for volatile/high-churn data (build scratch, logs) from OS

LVM for flexibility (resize without downtime, snapshot before risky changes) at a small overhead cost. Filesystem: **XFS** as a strong general default for large files/high throughput (common RHEL default), **ext4** for general-purpose simplicity. Separate mount points for volatile/high-churn data (build scratch, logs) from OS and app binaries so a runaway build filling disk doesn't take down the host. For shared engineering storage, NFS tuning (`rsize` / `wsize` , async vs sync) matters a lot — a naive NFS mount is often the real bottleneck people blame on "slow compute."

- [ ] How do you handle virtualization in this kind of environment — when would you virtualize vs. go bare-metal for engineering compute?

IF ASKED, SAY THIS
Virtualize for flexibility, isolation, and fast provisioning — most dev/build infrastructure, CI runners, and general engineering services are well-served by VMs (KVM/oVirt, VMware, or cloud instances), since the operational win (snapshot, clone, migrate, right-size) usually outweighs overhead. Bare-metal still wins wh

Virtualize for flexibility, isolation, and fast provisioning — most dev/build infrastructure, CI runners, and general engineering services are well-served by VMs (KVM/oVirt, VMware, or cloud instances), since the operational win (snapshot, clone, migrate, right-size) usually outweighs overhead.

Bare-metal still wins where virtualization overhead genuinely matters — heavy HPC/simulation farms where every % of throughput counts, or specialized hardware-passthrough needs (GPU-heavy workloads, hardware-in-the-loop setups common in embedded/semiconductor R&D).

- [ ] Walk through your troubleshooting process when a service is unreachable — where do you start?

IF ASKED, SAY THIS
Work outward from the host: is the process actually running and listening on the expected port (ss -tlnp)? Is the local firewall blocking it? Reachable from localhost but not remotely points to the network path, not the service. Then the path: routing, security groups/NSGs if cloud-hosted, any intermediate load balan

Work outward from the host: is the process actually running and listening on the expected port (`ss -tlnp`)? Is the local firewall blocking it? Reachable from localhost but not remotely points to the network path, not the service.

Then the path: routing, security groups/NSGs if cloud-hosted, any intermediate load balancer/proxy, and **DNS resolution** (a huge share of "the service is down" tickets are actually DNS). `tcpdump` or `traceroute` when the simple checks don't explain it.

- [ ] Explain how you'd troubleshoot intermittent DNS resolution failures affecting build servers.

IF ASKED, SAY THIS

Confirm which resolver is actually in use (/etc/resolv.conf , or resolvectl status on modern distros) and whether it's consistent across the fleet — a common cause is resolver config drift after a config-management change. Check for resolver timeout/retry exhaustion under load (build farms doing lots of parallel looku

Confirm which resolver is actually in use (`/etc/resolv.conf` , or `resolvectl status` on modern distros) and whether it's consistent across the fleet — a common cause is resolver config drift after a config-management change. Check for resolver timeout/retry exhaustion under load (build farms doing lots of parallel lookups can genuinely overwhelm an under-provisioned internal DNS server). Rule out split-horizon DNS issues and a flaky upstream forwarder before assuming local misconfiguration.

- [ ] How would you design load balancing for a set of internal engineering services (artifact repository, CI controllers)?

IF ASKED, SAY THIS
Layer 7 (HAProxy/NGINX/F5, or a cloud LB) when you need content-based routing, SSL termination, or health-check-driven traffic shifting — the common case for HTTP services like Artifactory or a Jenkins controller cluster. Layer 4 when you just need fast TCP-level distribution. Health checks need to reflect actual servi

Layer 7 (HAProxy/NGINX/F5, or a cloud LB) when you need content-based routing, SSL termination, or health-check-driven traffic shifting — the common case for HTTP services like Artifactory or a Jenkins controller cluster. **Layer 4** when you just need fast TCP-level distribution.

Health checks need to reflect actual service health, not just "port is open" — an Artifactory node that's up but can't reach its backend storage should fail its health check, or the LB keeps sending it traffic.

- [ ] What storage systems have you worked with, and how would you choose between NFS, block storage, and object storage for different engineering use cases?

IF ASKED, SAY THIS
NFS for shared, POSIX-compliant access engineering tools expect (home directories, shared build trees, tool installations). Block storage (SAN, or cloud EBS/managed disks) for database backends and anything needing low-latency, single-host-attached performance. Object storage (S3/Blob) for large, infrequently-modified

NFS for shared, POSIX-compliant access engineering tools expect (home directories, shared build trees, tool installations). **Block storage** (SAN, or cloud EBS/managed disks) for database backends and anything needing low-latency, single-host-attached performance. **Object storage** (S3/Blob) for large, infrequently-modified artifacts — build outputs, container images, archived logs — where POSIX semantics aren't needed and cost/scale matter more than latency.

- [ ] How do you approach capacity planning for network bandwidth on an engineering platform supporting large build/artifact transfers?

IF ASKED, SAY THIS
Baseline current usage against peak (not average) build-farm-to-artifact-repository transfer volume, understand burst patterns (a full CI pipeline kicking off simultaneously looks very different from steady state), and plan headroom against the busiest realistic scenario. For hybrid/cloud-connected environments, egress

Baseline current usage against peak (not average) build-farm-to-artifact-repository transfer volume, understand burst patterns (a full CI pipeline kicking off simultaneously looks very different from steady state), and plan headroom against the busiest realistic scenario. For hybrid/cloud-connected environments, egress cost and cross-region/cross-AZ bandwidth specifically need attention — shipping large build artifacts across regions gets expensive and slow fast.

- [ ] What's your experience with virtualization platforms, and how does that intersect with the networking you'd be responsible for?

IF ASKED, SAY THIS
Virtual networking (virtual switches/bridges, VLANs tied to port groups, overlay networks for container platforms) adds a layer pure physical-network troubleshooting doesn't prepare you for — a connectivity issue can be in the physical fabric, the hypervisor's virtual switch config, or the VM's own network stack. Diagn

Virtual networking (virtual switches/bridges, VLANs tied to port groups, overlay networks for container platforms) adds a layer pure physical-network troubleshooting doesn't prepare you for — a connectivity issue can be in the physical fabric, the hypervisor's virtual switch config, or the VM's own network stack. Diagnosing which layer requires working from both directions (`ip a` / `ip route` inside the guest, and the corresponding hypervisor/vSwitch view) — worth stating explicitly, since it's a common gap for people coming from a purely physical or purely cloud-networking background.

Security / DevSecOps

- [ ] Walk through your end-to-end process for identifying, assessing, and prioritizing a newly disclosed vulnerability across a large environment.

  IF ASKED, SAY THIS
  Identify : continuous scanning (SCA, container image, IaC) plus direct vendor advisories and the CISA Known Exploited Vulnerabilities (KEV) catalog, so you're not solely dependent on your own scan cadence catching something new. Assess : is it network-reachable in your environment, is the vulnerable

  Identify: continuous scanning (SCA, container image, IaC) plus direct vendor advisories and the CISA Known Exploited Vulnerabilities (KEV) catalog, so you're not solely dependent on your own scan cadence catching something new. **Assess**: is it network-reachable in your environment, is the vulnerable code path actually exercised, what compensating controls already reduce effective risk. **Prioritize**: CVSS as a starting point, but weighted heavily by EPSS score (likelihood of real-world exploitation in the next 30 days) and KEV membership (already being exploited), not CVSS severity alone — a "critical" CVSS score with no realistic exploit path is a very different priority than a "high" one that's actively being used in attacks.

- [ ] CVSS gives you a severity score, but two "critical" CVEs can warrant very different urgency. What else do you factor in?

  IF ASKED, SAY THIS
  Exploitability in practice — does a public or weaponized exploit exist. Exposure — is the affected component internet-facing or air-gapped internal. Asset criticality and blast radius — a critical CVE on a disposable dev sandbox is not the same urgency as one on a payment-processing service. Compens

  Exploitability in practice — does a public or weaponized exploit exist. Exposure — is the affected component internet-facing or air-gapped internal. Asset criticality and blast radius — a critical CVE on a disposable dev sandbox is not the same urgency as one on a payment-processing service. Compensating controls already in place — a WAF rule or network segmentation that meaningfully reduces the effective risk even though the CVSS number itself doesn't move. Treating CVSS as the whole answer is how teams either burn out patching low-risk criticals or miss a genuinely urgent high.

- [ ] You don't own the code in a vulnerable service — how do you actually drive remediation with a team that doesn't report to you?

  IF ASKED, SAY THIS
  An org-wide SLA policy tied to severity, agreed in advance rather than invented ad hoc when you need it — so you're enforcing a shared standard, not your personal opinion. Ticketing with visibility up to their leadership, not just the individual engineer. Regular vulnerability review forums where op

  An org-wide SLA policy tied to severity, agreed in advance rather than invented ad hoc when you need it — so you're enforcing a shared standard, not your personal opinion. Ticketing with visibility up to their leadership, not just the individual engineer. Regular vulnerability review forums where open items get surfaced across teams. A defined escalation path for missed SLAs. And framing the conversation as shared risk ownership — "this is exposure the org carries" — rather than a compliance chore you're imposing on them from outside.

- [ ] Where in a CI/CD pipeline do SAST, DAST, SCA, container image scanning, and IaC scanning each belong, and why does placement matter?

  IF ASKED, SAY THIS
  SAST, SCA, and IaC scanning run early — at commit/PR time — for fast feedback that blocks a bad pattern before it merges. Container image scanning runs at build, right after the image is produced. DAST runs later, against a running staging environment, because it needs a live app to actually attack.

  SAST, SCA, and IaC scanning run early — at commit/PR time — for fast feedback that blocks a bad pattern before it merges. Container image scanning runs at build, right after the image is produced. DAST runs later, against a running staging environment, because it needs a live app to actually attack. Placement matters because the cost of catching an issue scales with how far it traveled: minutes at commit time versus a full incident once it's in production.

- [ ] A CI/CD security gate is flooding teams with findings, many of them low real-risk. How do you fix scanner fatigue without turning the gate off?

  IF ASKED, SAY THIS
  Tune the ruleset to your actual stack — disable rule categories that don't apply and are just generating noise. Gate on severity, not on every finding — block critical/high, warn on medium/low rather than treating all findings as equally deploy-blocking. Build a documented risk-acceptance/suppressio

  Tune the ruleset to your actual stack — disable rule categories that don't apply and are just generating noise. Gate on severity, not on every finding — block critical/high, warn on medium/low rather than treating all findings as equally deploy-blocking. Build a documented risk-acceptance/suppression workflow with a named approver and an expiry date, so exceptions are visible and time-boxed instead of silently ignored. And re-tune periodically as false-positive patterns emerge — a gate nobody trusts gets routed around, which is worse than no gate at all.

- [ ] How do you approach base image and container hardening as vulnerability reduction, not just patching after the fact?

  IF ASKED, SAY THIS
  Minimal or distroless base images to shrink the attack surface before a single CVE is even scanned for. Non-root user, read-only root filesystem where the app allows it. Multi-stage builds so build tooling never ships in the final image. Image signing (cosign/Sigstore) and SBOM generation (Syft) so

Minimal or distroless base images to shrink the attack surface before a single CVE is even scanned for. Non-root user, read-only root filesystem where the app allows it. Multi-stage builds so build tooling never ships in the final image. Image signing (cosign/Sigstore) and SBOM generation (Syft) so that the next time a widely-used library has a CVE, you can answer "are we affected, and where" in minutes by querying your SBOM inventory instead of re-scanning everything from scratch.

- [ ] A critical, actively-exploited CVE — a "Log4Shell-style" event — is announced. Walk through your first two hours.

> **IF ASKED, SAY THIS**
> First 30 minutes : confirm exposure — do you actually run the affected component, and where; this is exactly where a maintained SBOM turns a frantic manual audit into a quick query. Next hour : apply an immediate compensating control if a real patch will take longer — a WAF rule, disabling the affec

First 30 minutes: confirm exposure — do you actually run the affected component, and where; this is exactly where a maintained SBOM turns a frantic manual audit into a quick query. **Next hour**: apply an immediate compensating control if a real patch will take longer — a WAF rule, disabling the affected feature, network restriction — then prioritize patching internet-facing and highest-criticality assets first, not alphabetically or by convenience. Communicate exposure status to leadership/security early, even before remediation is complete, rather than going quiet until it's "done." **Beyond that**: track remediation to full completion and verify with a re-scan — a deployed patch isn't confirmed fixed until you've checked.

- [ ] How do you operationalize "maintaining active security watch" without it becoming an unstructured, exhausting feed of noise?

> **IF ASKED, SAY THIS**
> Subscribe to a small number of high-signal sources — CISA KEV, vendor advisories for the specific stack you actually run, an NVD feed filtered against your SBOM's components — instead of generic security news. Automate correlation against your own asset/SBOM inventory so what reaches you is pre-filt

Subscribe to a small number of high-signal sources — CISA KEV, vendor advisories for the specific stack you actually run, an NVD feed filtered against your SBOM's components — instead of generic security news. Automate correlation against your own asset/SBOM inventory so what reaches you is pre-filtered to "does this affect us," not everything published that week. And run it on a defined weekly cadence rather than constant context-switching, with a clear escalation path for anything KEV-listed that jumps the queue immediately.

- [ ] What's your patch management process for OS-level patching across a large fleet, balancing timely patching against not breaking production?

> **IF ASKED, SAY THIS**
> Tiered cadence — a genuine zero-day/emergency gets an out-of-band expedited path, everything else follows a regular monthly (or similar) cycle. Staged rollout — canary group, then a wave, then the full fleet — with automated health-check gating between stages so a bad patch is caught on a handful of

Tiered cadence — a genuine zero-day/emergency gets an out-of-band expedited path, everything else follows a regular monthly (or similar) cycle. Staged rollout — canary group, then a wave, then the full fleet — with automated health-check gating between stages so a bad patch is caught on a handful of hosts, not the whole fleet. Test in non-prod first, unless the emergency itself makes that infeasible. And a documented rollback plan agreed *before* starting the rollout, not improvised if something breaks.

- [ ] How does secrets management fit into your DevSecOps practice, especially across CI/CD?

> **IF ASKED, SAY THIS**
> Centralized secrets manager (Vault, AWS Secrets Manager/SSM) instead of secrets living in environment variables or repos. Short-lived, dynamically issued credentials over static long-lived keys wherever the target system supports it. OIDC federation for CI/CD-to-cloud authentication instead of stori

Centralized secrets manager (Vault, AWS Secrets Manager/SSM) instead of secrets living in environment variables or repos. Short-lived, dynamically issued credentials over static long-lived keys wherever the target system supports it. OIDC federation for CI/CD-to-cloud authentication instead of storing cloud credentials in the CI system at all. And secret-scanning (gitleaks/trufflehog) as a pipeline gate, catching the accidental commit that policy alone won't prevent.

- [ ] How would you map your vulnerability and patch management practice to a compliance framework like PCI-DSS or SOC2 in a regulated organization?

> **IF ASKED, SAY THIS**
> PCI-DSS specifically requires defined patch timelines by severity and regular ASV (Approved Scanning Vendor) scans for in-scope systems. The practical translation: your internal SLA policy (severity → remediation deadline) needs to be defensible with evidence on demand — scan history, ticket closure

PCI-DSS specifically requires defined patch timelines by severity and regular ASV (Approved Scanning Vendor) scans for in-scope systems. The practical translation: your internal SLA policy (severity → remediation deadline) needs to be defensible with evidence on demand — scan history, ticket closure times against SLA, documented exception approvals with expiry. Built this way from the start, an audit becomes "here's the evidence," rather than a scramble to reconstruct a compliance story after the fact.

- [ ] What Kubernetes-specific controls do you put in place as part of a DevSecOps posture, beyond general vulnerability scanning?

> **IF ASKED, SAY THIS**

Pod Security Admission or Kyverno policies blocking privileged/root containers at admission time. NetworkPolicies enforcing default-deny east-west traffic so a compromised pod can't freely reach everything else in the cluster. RBAC least privilege audited regularly, not just set once. Runtime threat

Pod Security Admission or Kyverno policies blocking privileged/root containers at admission time. NetworkPolicies enforcing default-deny east-west traffic so a compromised pod can't freely reach everything else in the cluster. RBAC least privilege audited regularly, not just set once. Runtime threat detection (Falco) — because scanning only catches *known*-vulnerable code, not malicious behavior happening live in a running container. And admission-time image provenance checks, only permitting signed/verified images to run at all.

- [ ] What's the difference between EPSS and CVSS, and why do serious vulnerability-management programs use both?

IF ASKED, SAY THIS

CVSS scores severity — how bad it would be if exploited — as a mostly static property of the vulnerability itself. EPSS (Exploit Prediction Scoring System, maintained by FIRST.org) scores probability of exploitation in the near term, updated continuously against real-world exploitation activity. A h

CVSS scores *severity* — how bad it would be if exploited — as a mostly static property of the vulnerability itself. EPSS (Exploit Prediction Scoring System, maintained by FIRST.org) scores *probability of exploitation* in the near term, updated continuously against real-world exploitation activity. A high-CVSS, low-EPSS vulnerability may be theoretically severe but not realistically being attacked yet; a moderate-CVSS, high-EPSS one may need faster action. Using both, alongside CISA's KEV catalog for confirmed active exploitation, is what turns a raw scan list into an actually risk-ordered backlog instead of "patch everything labeled critical, in whatever order the ticket queue landed."

- [ ] Why does dedicated secrets management matter — what's actually wrong with an env var or a config file?

IF ASKED, SAY THIS

A secret sitting in a plain environment variable or a checked-in config file is effectively unmanaged: no audit trail of who accessed it or when, no rotation without a manual hunt across every place it's referenced, and it tends to leak sideways — into CI logs when a debug step echoes environment va

A secret sitting in a plain environment variable or a checked-in config file is effectively unmanaged: no audit trail of who accessed it or when, no rotation without a manual hunt across every place it's referenced, and it tends to leak sideways — into CI logs when a debug step echoes environment variables, into a Docker image layer if it was set at build time instead of runtime, into a Git history if a config file with a real value was ever committed even once and later 'removed.' A dedicated secrets manager centralizes all of that: access is scoped and logged, rotation can be automated centrally, and the actual secret value never has to be copy-pasted between five different places by five different engineers over the service's lifetime, each copy being one more place it can leak from.

- [ ] Explain HashiCorp Vault's basic model — secrets engines, dynamic vs. static secrets, auth methods.

IF ASKED, SAY THIS

A **secrets engine** is a pluggable backend for a category of secret — the KV engine for simple static key/value secrets, the AWS engine for generating IAM credentials on demand, the database engine for generating short-lived database credentials, and so on. **Static secrets** are stored values Vaul

A **secrets engine** is a pluggable backend for a category of secret — the KV engine for simple static key/value secrets, the AWS engine for generating IAM credentials on demand, the database engine for generating short-lived database credentials, and so on. **Static secrets** are stored values Vault hands back on request (like a KV entry) — better than a config file, but still a long-lived credential someone has to remember to rotate. **Dynamic secrets** are generated by Vault on demand, scoped to a short TTL, and automatically revoked/expired — so instead of a database password that's been the same for two years, an application requests a credential, gets one valid for an hour, and it simply stops working after that without anyone having to manually rotate it. **Auth methods** control how a client proves who it is to Vault in the first place (AppRole for machine-to-machine, Kubernetes auth for pods authenticating via their service account token, and others) before it's allowed to request any secret at all.

- [ ] AWS-native options — Secrets Manager and Parameter Store SecureString — vs. a 3rd-party Vault. When would you use which?

IF ASKED, SAY THIS

Parameter Store (SecureString) is the lightweight option — KMS-encrypted key/value storage, free up to a generous request rate, good for straightforward app config and secrets without built-in rotation automation. **Secrets Manager** adds native automatic rotation (with built-in Lambda rotation

Parameter Store (SecureString) is the lightweight option — KMS-encrypted key/value storage, free up to a generous request rate, good for straightforward app config and secrets without built-in rotation automation. **Secrets Manager** adds native automatic rotation (with built-in Lambda rotation functions for RDS and a few other AWS services out of the box) and tighter cross-service integration (ECS's `secrets` field in a task definition pulls from either directly), at a per-secret monthly cost Parameter Store doesn't have. **Vault** earns its place over both when there's a genuine need it covers that AWS-native tools don't: **dynamic, short-lived credentials** as the default posture rather than the exception, a **multi-cloud** environment where one secrets system needs to serve AWS and something else consistently, or an org-wide secrets strategy that predates or extends beyond any one cloud. For an AWS-first shop without multi-cloud pressure, Secrets Manager/Parameter Store usually covers the need with meaningfully less operational overhead than running Vault yourself.

- [ ] How does secret rotation actually work, and why does it matter beyond just 'best practice'?

IF ASKED, SAY THIS

With Secrets Manager, rotation is typically a scheduled Lambda function that knows the specific credential type's rotation protocol — for RDS, it creates a new database user/password, updates the secret, and (for the built-in RDS rotation function) handles the two-step process so old and new credent

With Secrets Manager, rotation is typically a scheduled Lambda function that knows the specific credential type's rotation protocol — for RDS, it creates a new database user/password, updates the secret, and (for the built-in RDS rotation function) handles the two-step process so old and new credentials both work briefly during cutover, avoiding a window where nothing can connect. With Vault, dynamic secrets sidestep the problem differently — instead of rotating a long-lived credential, Vault just issues new short-lived ones continuously, so there's no 'rotation event' to orchestrate at all. The reason this matters beyond compliance checkbox: a credential that's never rotated is a credential where a leak from six months ago (a log line, an old backup, a departed contractor's laptop) is still valid today — rotation bounds how long a leaked-but-undetected credential stays useful to whoever has it.

- [ ] How should secrets actually flow into a running container securely — what do you avoid?

IF ASKED, SAY THIS
Avoid baking a secret into a Docker image at build time (`ARG`/`ENV` with a real value, or a config file copied in with real credentials) — it's permanently in that image's layer history, retrievable by anyone who can pull the image, even if a later layer 'removes' the file. The safer pattern is inj

Avoid baking a secret into a Docker image at build time (`ARG`/`ENV` with a real value, or a config file copied in with real credentials) — it's permanently in that image's layer history, retrievable by anyone who can pull the image, even if a later layer 'removes' the file. The safer pattern is injecting the secret at **runtime**, not build time — ECS's `secrets` field pulling from Secrets Manager/Parameter Store directly into the running task's environment, or a Kubernetes pod mounting a Vault-injected secret as a file via a sidecar/CSI driver, so the secret exists only in the running container's memory or a tmpfs mount, never in the image itself. Where possible, prefer an **IAM role** (a task role, IRSA) over a static credential entirely — if the workload's access to, say, S3 is via an assumed role rather than an access key, there's no long-lived secret for that particular access path to leak in the first place.

- [ ] Where does vulnerability scanning fit across a CI/CD pipeline — what are the main categories of tooling?

IF ASKED, SAY THIS
Shift-left is the operating principle — catch a known vulnerability before it ships, not after. **SCA (Software Composition Analysis)** scans your application's dependencies against known-CVE databases (tools like Snyk or Dependabot are common examples) — the majority of real-world vulnerabilities l

Shift-left is the operating principle — catch a known vulnerability before it ships, not after. **SCA (Software Composition Analysis)** scans your application's dependencies against known-CVE databases (tools like Snyk or Dependabot are common examples) — the majority of real-world vulnerabilities land here, in a transitive dependency nobody explicitly chose. **Container image scanning** (Trivy, Grype, or a registry's built-in scanner like ECR's) checks the base image and installed packages inside a built image for known CVEs — different from SCA since it covers OS-level packages, not just application dependencies. **IaC scanning** (tfsec, Checkov) checks Terraform/CloudFormation for misconfigurations *before* apply — an S3 bucket about to be created with public access, a security group about to open 0.0.0.0/0 on a sensitive port — catching the mistake in a PR review instead of after it's live. Each category catches a different class of problem, which is why a mature pipeline runs more than one, not just whichever's easiest to bolt on first.

- [ ] What does container image scanning actually catch, and where in the lifecycle should it run?

IF ASKED, SAY THIS
It catches known CVEs in the base OS image and every package layered on top — an outdated base image is a very common, very avoidable source of vulnerabilities that has nothing to do with the application code itself, which is exactly why it's worth scanning separately from application dependency sca

It catches known CVEs in the base OS image and every package layered on top — an outdated base image is a very common, very avoidable source of vulnerabilities that has nothing to do with the application code itself, which is exactly why it's worth scanning separately from application dependency scanning. It should run at more than one point: **at build time** in CI, as a gate that can fail the build on a critical/high finding before the image is ever published; **at the registry** (many registries, including ECR, can scan on push automatically) as a backstop; and ideally **periodically against already-deployed images**, since a scan at build time only reflects CVEs known *as of that build* — a new CVE disclosed against a package already sitting in a running production image won't be caught by a one-time build-time scan, only by rescanning.

- [ ] How does IaC scanning work for Terraform, and what's a concrete example of what it would catch?

IF ASKED, SAY THIS
Tools in this category (tfsec, Checkov, and similar) parse Terraform code statically — without ever running `plan` or touching real AWS — against a rule set of known misconfiguration patterns, and can run as a fast pre-commit hook or a PR-gating CI step. A concrete catch: an `aws_s3_bucket` resource

Tools in this category (tfsec, Checkov, and similar) parse Terraform code statically — without ever running `plan` or touching real AWS — against a rule set of known misconfiguration patterns, and can run as a fast pre-commit hook or a PR-gating CI step. A concrete catch: an `aws_s3_bucket` resource with no explicit block on public access, or a security group rule with `cidr_blocks = ["0.0.0.0/0"]` on port 22/3389 — both are the kind of mistake that's easy to make in a rushed PR and expensive to discover after the fact once it's actually live and possibly already been found by something scanning the internet for exactly that. The value proposition is the same as any shift-left scanning: fixing a flagged line in a PR review costs a few minutes; finding the same misconfiguration in a live production security incident costs a great deal more.

- [ ] You've got a scan report with 40 findings — how do you actually triage it instead of trying to fix everything?

IF ASKED, SAY THIS
Severity (CVSS score) is the starting filter, but not sufficient alone — a critical-severity CVE in a library function your code never actually calls is genuinely lower real-world risk than a medium-severity one in a function that's directly exposed to untrusted input. **Exploitability and context**

Severity (CVSS score) is the starting filter, but not sufficient alone — a critical-severity CVE in a library function your code never actually calls is genuinely lower real-world risk than a medium-severity one in a function that's directly exposed to untrusted input. **Exploitability and context** matter

as much as the raw score: is there a known public exploit (this is what tools referencing EPSS/CVE-KEV data help surface), is the affected component actually reachable from outside the trust boundary, is there a compensating control already in place (a WAF rule, network isolation) that reduces real exposure even if the underlying finding is technically still true. In practice, triage means separating 'fix this now, it's genuinely exploitable and reachable' from 'track this and fix on the next regular patch cycle' from 'accepted risk, documented, with a reason' — treating every finding as equally urgent is how a scanning program burns credibility and stops getting taken seriously.

- [ ] What's baseline security hardening you'd expect on any AWS infrastructure you own, regardless of the specific JD?

IF ASKED, SAY THIS
Least-privilege IAM everywhere — scoped roles instead of broad managed policies, and no long-lived access keys where a role would do instead. **Encryption** at rest (EBS, S3, RDS all support this essentially for free) and in transit (TLS on load balancers, enforced rather than optional). **S3 Bl

Least-privilege IAM everywhere — scoped roles instead of broad managed policies, and no long-lived access keys where a role would do instead. **Encryption** at rest (EBS, S3, RDS all support this essentially for free) and in transit (TLS on load balancers, enforced rather than optional). **S3 Block Public Access** enabled at the account level as a default-deny backstop, with any genuinely-intentional public bucket as an explicit, reviewed exception rather than the default state. **Security group hygiene** — no broad `0.0.0.0/0` on anything beyond a public-facing load balancer's 443, and a regular review catching rules nobody remembers the reason for anymore, since those tend to accumulate silently over a service's lifetime. None of this is exotic or specific to any one JD's tooling — it's the baseline that should exist before layering on whatever specific scanning/monitoring tools a given org has standardized on.

- [ ] How would you design a secrets management standard across the platform?

IF ASKED, SAY THIS
A centralized secrets manager — Vault, AWS Secrets Manager, or Azure Key Vault — as the single source of truth, never secrets in source control, plaintext CI variables, or baked into images. Applications fetch secrets at runtime via short-lived, scoped credentials, ideally workload-identity federati

A centralized secrets manager — Vault, AWS Secrets Manager, or Azure Key Vault — as the single source of truth, never secrets in source control, plaintext CI variables, or baked into images. Applications fetch secrets at runtime via short-lived, scoped credentials, ideally workload-identity federation rather than long-lived static keys, and secret rotation is automated rather than a manual calendar reminder.

- [ ] What's your approach to certificate management for TLS across a large service mesh or platform?

IF ASKED, SAY THIS
Automated issuance and rotation — cert-manager in Kubernetes integrated with an internal or public CA — rather than manually tracked expiry dates, since expired-certificate outages are one of the most preventable, embarrassing incident categories and entirely solvable with automation. I'd still moni

Automated issuance and rotation — cert-manager in Kubernetes integrated with an internal or public CA — rather than manually tracked expiry dates, since expired-certificate outages are one of the most preventable, embarrassing incident categories and entirely solvable with automation. I'd still monitor certificate expiry as a proactive alert, at thirty, fourteen, and seven days out, as a backstop even with automated rotation in place.

- [ ] How would you design least-privilege access for engineers to production systems and GitLab or deployment tooling?

IF ASKED, SAY THIS
Role-based access scoped tightly to actual job requirements, standing production access minimized or eliminated in favor of time-boxed, logged, just-in-time elevated access for genuine incident response, and GitLab protected environments and branches so only authorized roles can trigger production d

Role-based access scoped tightly to actual job requirements, standing production access minimized or eliminated in favor of time-boxed, logged, just-in-time elevated access for genuine incident response, and GitLab protected environments and branches so only authorized roles can trigger production deploys. I'd audit access grants on a regular cadence, since privilege accumulates if nobody actively prunes it.

- [ ] How do you ensure auditability of changes and deployments for compliance purposes?

IF ASKED, SAY THIS
Every production change traceable to a specific commit, MR with reviewer identity, pipeline run, and deploy — GitLab's audit log plus environment deployment history gives most of this natively if the pipeline is the only path to production, with no manual apply to prod ever. I'd tie deploys to chang

Every production change traceable to a specific commit, MR with reviewer identity, pipeline run, and deploy — GitLab's audit log plus environment deployment history gives most of this natively if the pipeline is the only path to production, with no manual apply to prod ever. I'd tie deploys to change records where the compliance framework requires it, so evidence generation is a byproduct of the normal process, not a separate manual task.

- [ ] What's your approach to API security across the platform — authentication, authorization, rate limiting?

IF ASKED, SAY THIS
OAuth2/OIDC with short-lived tokens for authentication, fine-grained authorization enforced consistently, ideally centralized at the gateway plus defense-in-depth at the service level, and rate limiting or quota enforcement at the gateway to protect against both abuse and accidental self-inflicted o

OAuth2/OIDC with short-lived tokens for authentication, fine-grained authorization enforced consistently, ideally centralized at the gateway plus defense-in-depth at the service level, and rate limiting or quota enforcement at the gateway to protect against both abuse and accidental self-inflicted

overload. I'd standardize this at the platform level via the gateway and a shared auth library, since letting each team implement API auth independently reliably produces inconsistent, weaker-than-intended security.

- [ ] What does "shifting security left" mean concretely — name the specific pipeline gates.

IF ASKED, SAY THIS
Concretely, not as a slogan: pre-commit/IDE-level secret scanning, a SAST scan on every PR (not just before release), dependency/SCA scanning on every build (fails on known-CVE dependencies above your severity bar), IaC scanning (Checkov/tfsec) on every Terraform PR so a misconfigured security group

Concretely, not as a slogan: pre-commit/IDE-level secret scanning, a SAST scan on every PR (not just before release), dependency/SCA scanning on every build (fails on known-CVE dependencies above your severity bar), IaC scanning (Checkov/tfsec) on every Terraform PR so a misconfigured security group is caught before merge, and a container image scan before the image is allowed to be pushed to a "deployable" registry tag. The "left" part is specifically about moving these from a pre-release audit (expensive, late, blocks a release) to every-commit feedback (cheap, immediate, blocks nothing but that one bad commit).

- [ ] SAST vs DAST vs SCA — what's each and give a tool example.

IF ASKED, SAY THIS
SAST (static) analyzes source code without running it, catching things like injection-prone patterns or hardcoded secrets — e.g. Semgrep, SonarQube. DAST (dynamic) attacks a running application from the outside, the way a real attacker would — e.g. OWASP ZAP — catching runtime issues static analysis

SAST (static) analyzes source code without running it, catching things like injection-prone patterns or hardcoded secrets — e.g. Semgrep, SonarQube. DAST (dynamic) attacks a running application from the outside, the way a real attacker would — e.g. OWASP ZAP — catching runtime issues static analysis can't see (actual auth bypass, real injection through a live endpoint). SCA (software composition analysis) scans your dependency tree for known-vulnerable third-party libraries — e.g. Snyk, Trivy, Dependabot — which matters because most real-world vulnerabilities live in dependencies, not code you wrote yourself.

- [ ] How do you scan container images for vulnerabilities, and what do you do when a base image has a critical CVE with no fix available?

IF ASKED, SAY THIS
Trivy or Grype in the pipeline, scanning both OS packages and application dependencies in the image layers, with the build failing above your severity threshold. When there's genuinely no patched version yet: check whether the vulnerable component is actually reachable/exploitable in your specific u

Trivy or Grype in the pipeline, scanning both OS packages and application dependencies in the image layers, with the build failing above your severity threshold. When there's genuinely no patched version yet: check whether the vulnerable component is actually reachable/exploitable in your specific usage (not every CVE in an unused code path is a real risk), apply a compensating control if one exists (network policy blocking the exposed surface, WAF rule), and file an explicit, time-boxed exception with a ticket to re-scan once a fix ships — rather than either blocking every deploy indefinitely or silently suppressing the finding.

- [ ] Least privilege for EKS workload IAM specifically — IRSA vs Pod Identity.

IF ASKED, SAY THIS
Both scope IAM permissions to a specific ServiceAccount instead of the whole node, which is the core least-privilege win over node-level roles. IRSA does this via an OIDC trust relationship per cluster that has to be explicitly configured and kept in sync; EKS Pod Identity (the newer mechanism) simp

Both scope IAM permissions to a specific ServiceAccount instead of the whole node, which is the core least-privilege win over node-level roles. IRSA does this via an OIDC trust relationship per cluster that has to be explicitly configured and kept in sync; EKS Pod Identity (the newer mechanism) simplifies this with a cluster-level association managed through an EKS API instead of hand-wired OIDC trust policies, and doesn't require touching the IAM role's trust policy every time you add a new cluster. Either way, the actual least-privilege discipline is the same: one role per workload scoped to exactly the actions/resources it needs, never a shared "apps-can-do-everything" role reused across unrelated services.

- [ ] How do you detect and respond to anomalous activity in a cloud account?

IF ASKED, SAY THIS
GuardDuty (AWS-native threat detection — unusual API call patterns, known-bad IPs, compromised credential usage signatures) feeding into Security Hub for a consolidated view across accounts, with high-severity findings routed to an alerting/paging path, not just a dashboard nobody watches. CloudTrai

GuardDuty (AWS-native threat detection — unusual API call patterns, known-bad IPs, compromised credential usage signatures) feeding into Security Hub for a consolidated view across accounts, with high-severity findings routed to an alerting/paging path, not just a dashboard nobody watches. CloudTrail is the raw audit log underneath all of this — GuardDuty's ML-based detection is built on top of it, but CloudTrail itself is also directly queryable for "who did X" during an investigation. Response side needs a defined runbook per finding type — rotate credentials automatically for a confirmed key-compromise finding, isolate an instance's security group for a compromise-indicator finding — because during an actual incident is the worst time to be improvising the response steps.

- [ ] Explain secrets scanning in a CI pipeline — what failure mode is it actually preventing?

IF ASKED, SAY THIS

It's catching the case where a developer accidentally commits a real credential (an AWS key, a DB password, an API token) into source, which — even if immediately reverted — remains in git history forever unless you rewrite history, and is often already scraped by bots within minutes on a public rep

It's catching the case where a developer accidentally commits a real credential (an AWS key, a DB password, an API token) into source, which — even if immediately reverted — remains in git history forever unless you rewrite history, and is often already scraped by bots within minutes on a public repo. Tools like gitleaks/truffleHog scan every commit (ideally pre-commit locally, and again as a CI gate as the backstop) against known credential patterns. The response runbook matters as much as the detection: a caught secret needs to be treated as compromised and rotated immediately, not just removed from the file — removing it from the current commit doesn't un-expose it from history.

- [ ] What's your approach to container runtime security — detecting a compromised container after it's already running?

IF ASKED, SAY THIS

Static scanning (image vulnerabilities) only covers the "before it runs" risk; runtime security covers "it's running and now doing something it shouldn't." Tools like Falco watch system calls and flag anomalous behavior — a container spawning an unexpected shell, writing to a location it never write

Static scanning (image vulnerabilities) only covers the "before it runs" risk; runtime security covers "it's running and now doing something it shouldn't." Tools like Falco watch system calls and flag anomalous behavior — a container spawning an unexpected shell, writing to a location it never writes to, making an outbound connection to an unusual destination — based on rules tied to expected behavior for that workload. The response side needs to be fast and automatable: isolate (network policy, or kill the pod) first, preserve evidence (don't just delete it — you lose the ability to investigate), then root-cause.

- [ ] How do you handle compliance evidence collection in a regulated environment without it becoming pure toil?

IF ASKED, SAY THIS

Automate the evidence generation at the point the control is actually enforced, rather than manually assembling proof after the fact — CloudTrail/audit logs as the evidence for "who approved what," pipeline gate pass/fail records as evidence for "was the security scan actually required and did it ru

Automate the evidence generation at the point the control is actually enforced, rather than manually assembling proof after the fact — CloudTrail/audit logs as the evidence for "who approved what," pipeline gate pass/fail records as evidence for "was the security scan actually required and did it run," and Terraform plan/apply logs tied to change tickets as evidence for change management. The anti-pattern to avoid is a separate manual "compliance spreadsheet" process bolted on after the real work happens — that's both pure toil and less trustworthy than evidence generated automatically by the systems doing the actual enforcement.

- [ ] What does "shifting security left" mean concretely — name the specific pipeline gates.

IF ASKED, SAY THIS

Concretely, not as a slogan: pre-commit/IDE-level secret scanning, a SAST scan on every PR (not just before release), dependency/SCA scanning on every build (fails on known-CVE dependencies above your severity bar), IaC scanning (Checkov/tfsec) on every Terraform PR so a misconfigured security group

Concretely, not as a slogan: pre-commit/IDE-level secret scanning, a SAST scan on every PR (not just before release), dependency/SCA scanning on every build (fails on known-CVE dependencies above your severity bar), IaC scanning (Checkov/tfsec) on every Terraform PR so a misconfigured security group is caught before merge, and a container image scan before the image is allowed to be pushed to a "deployable" registry tag. The "left" part is specifically about moving these from a pre-release audit (expensive, late, blocks a release) to every-commit feedback (cheap, immediate, blocks nothing but that one bad commit).

- [ ] Least privilege for EKS workload IAM specifically — IRSA vs Pod Identity.

IF ASKED, SAY THIS

Both scope IAM permissions to a specific ServiceAccount instead of the whole node, which is the core least-privilege win over node-level roles. IRSA does this via an OIDC trust relationship per cluster that has to be explicitly configured and kept in sync; EKS Pod Identity (the newer mechanism) simp

Both scope IAM permissions to a specific ServiceAccount instead of the whole node, which is the core least-privilege win over node-level roles. IRSA does this via an OIDC trust relationship per cluster that has to be explicitly configured and kept in sync; EKS Pod Identity (the newer mechanism) simplifies this with a cluster-level association managed through an EKS API instead of hand-wired OIDC trust policies, and doesn't require touching the IAM role's trust policy every time you add a new cluster. Either way, the actual least-privilege discipline is the same: one role per workload scoped to exactly the actions/resources it needs, never a shared "apps-can-do-everything" role reused across unrelated services.

- [ ] How do you detect and respond to anomalous activity in a cloud account?

IF ASKED, SAY THIS

GuardDuty (AWS-native threat detection — unusual API call patterns, known-bad IPs, compromised credential usage signatures) feeding into Security Hub for a consolidated view across accounts, with high-severity findings routed to an alerting/paging path, not just a dashboard nobody watches. CloudTrai

GuardDuty (AWS-native threat detection — unusual API call patterns, known-bad IPs, compromised credential usage signatures) feeding into Security Hub for a consolidated view across accounts, with high-severity findings routed to an alerting/paging path, not just a dashboard nobody watches. CloudTrail is the raw audit log underneath all of this — GuardDuty's ML-based detection is built on top of it, but CloudTrail itself is also directly queryable for "who did X" during an investigation. Response side needs a defined runbook per finding type — rotate credentials automatically for a confirmed key-compromise finding, isolate an instance's security group for a compromise-indicator finding — because during an actual incident is the worst time to be improvising the response steps.

- [ ] Walk through the difference between IAM Roles, Policies, and Permission Boundaries, and how they compose in a real architecture.

> **IF ASKED, SAY THIS**
> A Policy is the actual permission document (JSON allow/deny statements). A Role is an identity that can be assumed, with policies attached — the mechanism for temporary, scoped credentials rather than long-lived access keys. A Permission Boundary sets the maximum permissions a role/user can ever have regardless of what's directly attached

A **Policy** is the actual permission document (JSON allow/deny statements). A **Role** is an identity that can be assumed, with policies attached — the mechanism for temporary, scoped credentials rather than long-lived access keys. A **Permission Boundary** sets the maximum permissions a role/user can ever have regardless of what's directly attached — lets a team create its own roles within a safe ceiling.

They compose as layers: effective permissions are the *intersection* of what the identity policy grants AND what the permission boundary allows.

- [ ] How would you design cross-account access for an enterprise client with multiple AWS accounts (dev/staging/prod, or per-business-unit)?

> **IF ASKED, SAY THIS**
> Role-based cross-account access — a role in the target account with a trust policy naming the specific source account/role allowed to assume it, never shared long-lived credentials. For human access, federated through IAM Identity Center against a central identity source, no per-account local IAM users. For service-to-service cross-accoun

Role-based cross-account access — a role in the target account with a trust policy naming the specific source account/role allowed to assume it, never shared long-lived credentials. For human access, federated through **IAM Identity Center** against a central identity source, no per-account local IAM users.

For service-to-service cross-account access, scope the trust policy as tightly as possible (specific role ARN, not a whole account) — **external ID conditions** add protection for third-party cross-account access specifically, mitigating the "confused deputy" problem.

- [ ] What are Service Control Policies, and how do they differ from IAM policies?

> **IF ASKED, SAY THIS**
> SCPs apply at the AWS Organizations level (an OU or account) and set the maximum permissions available within that account — they don't grant permissions, only restrict what IAM policies can ever allow, no matter how permissive. Classic use: an SCP denying the ability to disable CloudTrail or leave an approved region, applied org-wide — a

SCPs apply at the AWS Organizations level (an OU or account) and set the **maximum** permissions available within that account — they don't grant permissions, only restrict what IAM policies can ever allow, no matter how permissive.

Classic use: an SCP denying the ability to disable CloudTrail or leave an approved region, applied org-wide — a guardrail that survives individual account misconfiguration, which is the point of them living at the organization layer rather than relying on every account's IAM being perfect.

- [ ] How would you design an enterprise IAM strategy from scratch for a new AWS Organization supporting multiple business units?

> **IF ASKED, SAY THIS**
> Multi-account structure (Organizations/Control Tower) separating environments and business units rather than one account with everything — blast-radius containment is the core reasoning. Centralized identity via IAM Identity Center federated to the enterprise's existing IdP. SCPs at the OU level for org-wide guardrails, standardized permi

Multi-account structure (Organizations/Control Tower) separating environments and business units rather than one account with everything — blast-radius containment is the core reasoning. Centralized identity via IAM Identity Center federated to the enterprise's existing IdP.

SCPs at the OU level for org-wide guardrails, standardized permission-set/role naming conventions for consistent access requests and audits, and a clear review process for new roles/policies — least-privilege as policy, not aspiration.

- [ ] How do you actually design least-privilege IAM policies in practice, rather than starting broad and "tightening later" (which rarely happens)?

> **IF ASKED, SAY THIS**
> Start from the specific actions the workload actually needs (CloudTrail logs of what a similar existing workload calls, or IAM Access Analyzer's policy generation from observed activity) rather than a broad managed policy with a promise to narrow later. Scope resource ARNs specifically rather than * where avoidable, add condition keys (so

Start from the specific actions the workload actually needs (CloudTrail logs of what a similar existing workload calls, or IAM Access Analyzer's policy generation from observed activity) rather than a broad managed policy with a promise to narrow later.

Scope resource ARNs specifically rather than `*` where avoidable, add condition keys (source IP, MFA presence, tags) where they add real protection, and run **IAM Access Analyzer continuously** — not just at initial design — since environments drift.

- [ ] What's your approach to managing secrets and encryption keys — Secrets Manager vs. SSM Parameter Store vs. KMS — when do you use which?

> **IF ASKED, SAY THIS**

KMS is the encryption key layer underneath the other two (and used directly for S3/EBS/RDS encryption at rest). Secrets Manager for secrets needing automatic rotation (DB credentials, API keys to third-party ERP/RPA systems) — rotation Lambda integration is the differentiator justifying its cost over Parameter Store. SSM Parameter Store f

KMS is the encryption key layer underneath the other two (and used directly for S3/EBS/RDS encryption at rest). **Secrets Manager** for secrets needing automatic rotation (DB credentials, API keys to third-party ERP/RPA systems) — rotation Lambda integration is the differentiator justifying its cost over Parameter Store.

SSM Parameter Store for config values and secrets that don't need rotation — a very common cost pattern is Parameter Store for the bulk of config, Secrets Manager reserved specifically for rotating credentials.

- [ ] How would you architect KMS key usage and key policies for a multi-account, multi-team environment?

IF ASKED, SAY THIS

Customer-managed keys (not AWS-managed defaults) for anything needing control over key policy, rotation schedule, and cross-account grants — a default key can't be shared cleanly cross-account, which matters here. Key policies scoped per use case. For cross-account access to encrypted resources, both the resource's IAM policy AND the KMS

Customer-managed keys (not AWS-managed defaults) for anything needing control over key policy, rotation schedule, and cross-account grants — a default key can't be shared cleanly cross-account, which matters here. Key policies scoped per use case.

For cross-account access to encrypted resources, **both the resource's IAM policy AND the KMS key policy** need to grant access — a very common real-world gotcha where someone grants S3 access cross-account but forgets the key policy, and access silently fails.

- [ ] Walk through how you'd design secure connectivity and authentication for integrating with third-party systems and ERP platforms (SAP, Oracle, BaaN) specifically.

IF ASKED, SAY THIS

Credentials in Secrets Manager (never hardcoded or plaintext config), retrieved at runtime by a role narrowly scoped to just that secret. Network path secured via PrivateLink, Direct Connect, or VPN rather than traversing the open internet. For the AWS-side identity the third party might authenticate back to, a dedicated IAM role/API Gate

Credentials in Secrets Manager (never hardcoded or plaintext config), retrieved at runtime by a role narrowly scoped to just that secret. Network path secured via PrivateLink, Direct Connect, or VPN rather than traversing the open internet.

For the AWS-side identity the third party might authenticate back to, a dedicated IAM role/API Gateway authorizer scoped to exactly what that integration needs — **never reuse a broad internal service role** for an external-facing integration point, so a compromised third-party credential's blast radius stays as small as possible.

- [ ] What's Amazon Cognito, and how would you use it for authentication in an architecture like this JD describes (API Gateway plus third-party integration)?

IF ASKED, SAY THIS

Cognito provides two related pieces: User Pools (a managed user directory with sign-up/sign-in, MFA, and OAuth/OIDC token issuance — for actual end-user authentication) and Identity Pools (federate an authenticated identity, from a User Pool or an external IdP, into temporary AWS IAM credentials — for authorizing direct access to AWS reso

Cognito provides two related pieces: **User Pools** (a managed user directory with sign-up/sign-in, MFA, and OAuth/OIDC token issuance — for actual end-user authentication) and **Identity Pools** (federate an authenticated identity, from a User Pool or an external IdP, into temporary AWS IAM credentials — for authorizing direct access to AWS resources).

For an API Gateway-fronted architecture, a Cognito User Pool authorizer is a common pattern: users authenticate against the pool, get a JWT, API Gateway validates it natively without custom authorizer code for the common case. For the ERP/third-party integration side of this JD specifically, Cognito is less likely to be the mechanism — that's more OAuth-client-credentials/API-key territory — Cognito's fit here is actual end-user-facing authentication, if this platform has one.

- [ ] What does GuardDuty actually detect, and how would you respond operationally to a finding?

IF ASKED, SAY THIS

A managed threat-detection service analyzing VPC Flow Logs, CloudTrail, and DNS logs for suspicious patterns — compromised credentials making unusual API calls, an EC2 instance communicating with a known-malicious IP, reconnaissance-style behavior — without you having to build or maintain the detection logic yourself. Operationally: findi

A managed threat-detection service analyzing VPC Flow Logs, CloudTrail, and DNS logs for suspicious patterns — compromised credentials making unusual API calls, an EC2 instance communicating with a known-malicious IP, reconnaissance-style behavior — without you having to build or maintain the detection logic yourself.

Operationally: findings should route into the same centralized alerting/ticketing path as other security signals, triaged by severity, with a defined response runbook per finding type — isolate the resource, rotate the credential, investigate further — rather than each finding handled ad hoc the first time someone notices it.

- [ ] How does Security Hub aggregate findings, and how would you use it as part of ongoing security governance?

IF ASKED, SAY THIS

Security Hub aggregates findings from GuardDuty, Config, Inspector, and other AWS/partner security tools into one normalized view, and runs automated checks against standards (CIS AWS Foundations, PCI DSS, AWS Foundational Security Best Practices) to give a continuous compliance score rather than a point-in-time audit. Operationally: a re

Security Hub aggregates findings from GuardDuty, Config, Inspector, and other AWS/partner security tools into one normalized view, and runs automated checks against standards (CIS AWS Foundations, PCI DSS, AWS Foundational Security Best Practices) to give a continuous compliance score rather than a point-in-time audit.

Operationally: a regular review cadence (weekly, or event-driven via EventBridge for high-severity findings), findings routed to the owning team rather than one person triaging everything, and tracking the compliance score trend over time as a genuine governance metric — not a dashboard nobody looks at until an audit is imminent.

SQL / Databases

- [ ] Write a basic SQL query you might actually run as a DevOps engineer — say, checking deployment metadata.

IF ASKED, SAY THIS
Something like this comes up often — checking which service versions are currently marked active across environments, joining a deployments table to a services table.

Something like this comes up often — checking which service versions are currently marked active across environments, joining a deployments table to a services table.

- [ ] As a DevOps/SRE engineer (not a DBA), what PostgreSQL checks do you actually run when something looks wrong?

IF ASKED, SAY THIS
`pg_stat_activity` is the first stop — shows every current connection, what query it's running, and how long it's been running, which immediately surfaces a long-running or stuck query that might be the reason things are slow. Connection count against `max_connections` catches a connection-pool-exha

`pg_stat_activity` is the first stop — shows every current connection, what query it's running, and how long it's been running, which immediately surfaces a long-running or stuck query that might be the reason things are slow. Connection count against `max_connections` catches a connection-pool-exhaustion situation before it fully takes the app down. `pg_database_size()` and checking the underlying EBS volume's free space cover the 'are we about to run out of disk' failure mode, which on RDS specifically can quietly put the instance into a read-only or unavailable state if it's not caught ahead of time. None of this requires deep DBA expertise — it's the same triage instinct as checking `top`/`df -h` on a host, just pointed at the database.

- [ ] What's the practical difference between a JOIN and a subquery?

IF ASKED, SAY THIS
A JOIN combines rows from two (or more) tables side by side based on a related column, producing one result set — the standard way to pull related data together in one pass, and generally what the query planner optimizes best. A subquery is a query nested inside another, typically used to filter or

A JOIN combines rows from two (or more) tables side by side based on a related column, producing one result set — the standard way to pull related data together in one pass, and generally what the query planner optimizes best. A subquery is a query nested inside another, typically used to filter or compute a value the outer query then uses — for example, `WHERE service_id IN (SELECT id FROM services WHERE team = 'platform')`. They can often express the same logical question, but a JOIN is usually preferred when you need columns from both tables in the output, while a subquery reads more naturally when you only need to filter or check existence against the other table without needing its columns back.

- [ ] Why do indexes matter, and when would you add one?

IF ASKED, SAY THIS
Without an index, PostgreSQL scans every row of a table to satisfy a query (a sequential scan) — fine on a small table, expensive on a large one. An index on the column(s) a query filters or joins on lets it jump straight to matching rows instead. The trade-off: an index speeds up reads on that colu

Without an index, PostgreSQL scans every row of a table to satisfy a query (a sequential scan) — fine on a small table, expensive on a large one. An index on the column(s) a query filters or joins on lets it jump straight to matching rows instead. The trade-off: an index speeds up reads on that column but adds overhead to every write (insert/update/delete has to maintain the index too), so it's not free to add everywhere. The practical signal for adding one is a genuinely slow query (caught via `pg_stat_activity`, slow query logs, or `EXPLAIN ANALYZE`) that's doing a sequential scan on a large table for a filter/join column that gets queried often — not indexing preemptively on every column 'just in case.'

- [ ] How would you quickly check whether a PostgreSQL RDS instance is healthy from the command line?

IF ASKED, SAY THIS
`psql -h -U -d -c 'SELECT 1;'` is the simplest possible liveness check — if that returns, the instance is accepting connections and executing queries. From there, `\l` lists databases, `\dt` lists tables in the current one, confirming the schema you expect is actually present. Beyond basic connectiv

`psql -h -U -d -c 'SELECT 1;'` is the simplest possible liveness check — if that returns, the instance is accepting connections and executing queries. From there, `\l` lists databases, `\dt` lists tables in the current one, confirming the schema you expect is actually present. Beyond basic connectivity, the CloudWatch metrics RDS exposes (CPUUtilization, FreeStorageSpace, DatabaseConnections, ReadLatency/WriteLatency) usually answer 'is it healthy' faster than digging through `psql` directly, since they're already being collected and can be alarmed on — `psql` checks are more for the moment you're actively investigating something CloudWatch already told you is wrong.

- [ ] A report that ran in 10 seconds now takes 5 minutes.

IF ASKED, SAY THIS
Check what changed first — row count growth, then `EXPLAIN ANALYZE` vs the expected plan. Seq Scan where an index is expected + stale `last_analyze` is the classic pairing. Fix: `ANALYZE` / `VACUUM ANALYZE`, not a rewrite.

Check what changed first — row count growth, then `EXPLAIN ANALYZE` vs the expected plan. Seq Scan where an index is expected + stale `last_analyze` is the classic pairing. Fix: `ANALYZE` / `VACUUM ANALYZE`, not a rewrite.

- [ ] Users report duplicate records after multiple transactions.

IF ASKED, SAY THIS

It's a concurrency bug, not performance — a check-then-insert race under READ COMMITTED, or a client retry with no idempotency key. Fix: unique constraint + upsert at the DB level, never an app-only check.

It's a concurrency bug, not performance — a check-then-insert race under READ COMMITTED, or a client retry with no idempotency key. Fix: unique constraint + upsert at the DB level, never an app-only check.

- [ ] A query is consuming 90% of database CPU.

IF ASKED, SAY THIS

`pg_stat_activity` / `pg_stat_statements` to identify it, `EXPLAIN ANALYZE` to classify: seq scan, sort spilling from low `work_mem`, or a weak index. `pg_terminate_backend` buys time; the index/rewrite is the real fix.

`pg_stat_activity` / `pg_stat_statements` to identify it, `EXPLAIN ANALYZE` to classify: seq scan, sort spilling from low `work_mem`, or a weak index. `pg_terminate_backend` buys time; the index/rewrite is the real fix.

- [ ] Frequent deadlocks during peak hours.

IF ASKED, SAY THIS

Pull the deadlock graph — Postgres logs both queries involved. Root cause is almost always inconsistent lock order across code paths touching the same tables. Fix: one consistent lock order everywhere, not just retries.

Pull the deadlock graph — Postgres logs both queries involved. Root cause is almost always inconsistent lock order across code paths touching the same tables. Fix: one consistent lock order everywhere, not just retries.

- [ ] Blocking vs deadlock — what's the difference?

IF ASKED, SAY THIS

Blocking = one transaction waiting on a lock, normal, resolves itself. Deadlock = a cycle of transactions waiting on each other — never resolves alone, so the DB kills a victim to break it.

Blocking = one transaction waiting on a lock, normal, resolves itself. Deadlock = a cycle of transactions waiting on each other — never resolves alone, so the DB kills a victim to break it.

- [ ] How do you read an execution plan?

IF ASKED, SAY THIS

Read bottom-up. Seq Scan = no index used, fine small / bad large. Index Only Scan = fastest. Nested Loop fine at small scale, Hash/Merge Join at bigger scale. Estimated vs actual rows gap = stale stats.

Read bottom-up. Seq Scan = no index used, fine small / bad large. Index Only Scan = fastest. Nested Loop fine at small scale, Hash/Merge Join at bigger scale. Estimated vs actual rows gap = stale stats.

- [ ] How do you think about indexing strategy?

IF ASKED, SAY THIS

B-tree covers equality/range; composite indexes only help if the query matches the **leftmost columns**. Covering index → Index Only Scan. Every index costs writes + disk — check for unused ones.

B-tree covers equality/range; composite indexes only help if the query matches the **leftmost columns**. Covering index → Index Only Scan. Every index costs writes + disk — check for unused ones.

- [ ] A report that normally runs in 10 seconds now takes 5 minutes — what's your investigation?

IF ASKED, SAY THIS

Don't guess — check what changed first. Has the row count grown significantly, a bulk load, a new data source? Run EXPLAIN ANALYZE and compare the plan to what you'd expect: a Seq Scan on a table that should be using an index is the single most common cause, and it usually means table statistics are

Don't guess — check what changed first. Has the row count grown significantly, a bulk load, a new data source? Run `EXPLAIN ANALYZE` and compare the plan to what you'd expect: a **Seq Scan** on a table that should be using an index is the single most common cause, and it usually means

table statistics are stale after a bulk change. Check `pg_stat_user_tables` for `last_analyze` / `last_autoanalyze` — if it hasn't run since the data changed, the planner is working off old assumptions. Also rule out a blocking session sitting on the same table before you touch anything.

Remember: Sequence: what changed? → EXPLAIN ANALYZE → stale stats check → blocking check · 'Seq Scan where you expect an index' + 'stale last_analyze' is the classic pairing · Fix is usually `ANALYZE` or `VACUUM ANALYZE`, not a query rewrite

- [ ] Users report duplicate records after multiple transactions — how do you approach this?

IF ASKED, SAY THIS

This one's a trap if you treat it as a performance question — it's a concurrency and data-integrity problem. The usual cause is a check-then-insert race: two transactions both check 'does this exist?', both get 'no,' both insert. Under READ COMMITTED, the common default, that race is entirely possib

This one's a trap if you treat it as a performance question — it's a **concurrency and data-integrity** problem. The usual cause is a check-then-insert race: two transactions both check 'does this exist?', both get 'no,' both insert. Under READ COMMITTED, the common default, that race is entirely possible. The other frequent cause is a client retry: a request times out after the write actually succeeded server-side, the client retries, and a second row gets created because there was no idempotency key to catch the repeat. The durable fix is a unique constraint at the database level plus an `ON CONFLICT DO NOTHING` upsert pattern — never rely on an application-level check alone.

Remember: Say the word 'race condition' — this is what separates a data question from a performance one · Unique constraint + upsert at the DB level is the fix, not 'add a check before insert' · Client retries without an idempotency key are a very common real-world cause

- [ ] A query is consuming 90% of the database CPU — what do you do?

IF ASKED, SAY THIS

Identify it first — pg_stat_activity for what's running right now, or pg_stat_statements ordered by total_exec_time for the worst offender over time. CPU-heavy queries are usually doing one of three things: a sequential scan on a large table, a hash join or sort spilling because work_mem is too smal

Identify it first — `pg_stat_activity` for what's running right now, or `pg_stat_statements` ordered by `total_exec_time` for the worst offender over time. CPU-heavy queries are usually doing one of three things: a sequential scan on a large table, a hash join or sort spilling because `work_mem` is too small, or a poorly-selective index that isn't actually narrowing the result set much. `EXPLAIN ANALYZE` tells you which. If it's a runaway ad hoc query actively hurting production, `pg_terminate_backend(pid)` buys you time immediately; the real fix — an index, a rewrite, or a work_mem tune — comes after.

Remember: Identify → EXPLAIN ANALYZE → classify (scan / spill / bad index) → fix · `pg_terminate_backend` is a legitimate immediate mitigation, not a last resort to feel bad about · Naming `work_mem` as a lever most people forget is a senior-level detail

- [ ] Your application experiences frequent deadlocks during peak hours — walk me through it.

IF ASKED, SAY THIS

Pull the deadlock graph — Postgres logs deadlocks by default with both queries involved. It'll show two transactions each holding a lock the other is waiting for. The near-universal root cause: two different code paths update the same tables but acquire locks in a different order — transaction A loc

Pull the deadlock graph — Postgres logs deadlocks by default with both queries involved. It'll show two transactions each holding a lock the other is waiting for. The near-universal root cause: two different code paths update the same tables but acquire locks in a different order — transaction A locks table1 then table2, transaction B locks table2 then table1, and under enough concurrency they eventually cross. It only shows up under peak load because that's when the timing windows actually overlap. The durable fix is enforcing one consistent lock acquisition order across every code path that touches those tables — retries alone just mask it until load goes up again.

Remember: The log/graph names the exact two queries — read it before theorizing · Root cause is almost always **inconsistent lock order** across code paths · 'Retries mask it, consistent ordering fixes it' is the line that shows understanding, not just recognition

- [ ] What's the difference between blocking and a deadlock?

IF ASKED, SAY THIS

Blocking is one transaction waiting for a lock held by another — completely normal, and it resolves on its own the moment the holder commits or rolls back. A deadlock is two or more transactions each waiting on a lock the other one holds — a cycle that can never resolve by itself, so the database's

Blocking is one transaction waiting for a lock held by another — completely normal, and it resolves on its own the moment the holder commits or rolls back. A deadlock is two or more transactions each waiting on a lock the other one holds — a cycle that can never resolve by itself, so the database's deadlock detector picks a victim and kills it to break the cycle. Blocking becomes a problem when it's *long* — a transaction held open, an idle-in-transaction session, usually an app bug, blocking others for minutes. A deadlock is a problem the instant it happens, by definition.

Remember: Blocking = normal + self-resolving; Deadlock = cyclic + needs the DB to intervene · Long blocking is usually the same idle-in-transaction app bug as connection-pool saturation · Interviewers listen for you knowing these are *different* problems with different fixes

- [ ] How do you actually read an execution plan?

IF ASKED, SAY THIS

Start from the innermost/lowest lines — that's where execution begins — and work outward. Check the scan type on your big tables: Seq Scan means no usable index, fine on a small table, a red flag on a large one. Index Scan means it found rows via an index then fetched them. Index Only Scan is fastes

Start from the innermost/lowest lines — that's where execution begins — and work outward. Check the scan type on your big tables: **Seq Scan** means no usable index, fine on a small table, a red flag on a large one. **Index Scan** means it found rows via an index then fetched them. **Index Only Scan** is fastest since it never touches the table at all. Check the join type: **Nested Loop** is fine for small row counts but disastrous at scale; **Hash Join** and **Merge Join** show up as data grows. Then compare estimated vs actual rows — a big gap means the planner's statistics are wrong, your stale-stats signal again. A high 'Rows Removed by Filter' means an index isn't as selective as it looks.

Remember: Read bottom-up — the innermost node executes first · Seq Scan isn't inherently bad — it's bad specifically on a *large* table · Estimated vs actual rows mismatch = stats problem, not a query problem

- [ ] How do you think about indexing strategy? ▲

IF ASKED, SAY THIS

Default B-tree indexes cover equality and range lookups on a column. For queries filtering on multiple columns together, a composite index helps — but only if the query's filter matches the index's leftmost columns ; an index on (a, b) helps a query filtering on a, or on a and b, but not one filteri

Default B-tree indexes cover equality and range lookups on a column. For queries filtering on multiple columns together, a composite index helps — but only if the query's filter matches the index's **leftmost columns**; an index on (a, b) helps a query filtering on a, or on a and b, but not one filtering on b alone. A covering index that includes every column a query needs lets Postgres do an Index Only Scan and skip the table entirely. The tradeoff people forget: every index slows down writes and costs disk, so a table with a dozen unused indexes is its own performance problem — `pg_stat_user_indexes` with `idx_scan = 0` finds the ones nobody's using.

Remember: Leftmost-prefix rule for composite indexes — the detail that separates knowing from reciting · Covering index → Index Only Scan → fastest possible read · Indexes aren't free — mention the write-cost tradeoff unprompted, it reads as senior

Kafka

- [ ] Why would a high-throughput platform use Kafka in the first place?

> **IF ASKED, SAY THIS**
> The core problem Kafka solves here: an ad-serving system generates a huge volume of events — impressions, clicks, bid requests/responses — and multiple different downstream systems need to consume those events independently: billing, analytics, fraud detection, reporting, ML feature pipelines. Kafka decouples the produ

The core problem Kafka solves here: an ad-serving system generates a huge volume of events — impressions, clicks, bid requests/responses — and multiple different downstream systems need to consume those events independently: billing, analytics, fraud detection, reporting, ML feature pipelines. Kafka decouples the producer (the ad server) from all of those consumers, so the ad server just publishes the event once and moves on, instead of calling five different downstream systems synchronously and taking on all their latency and failure modes.

Three specific reasons it fits this domain well:
• **Durable replay** — if a downstream billing consumer had a bug for two hours, you can replay that window from the log instead of losing the data.
• **Buffering against bursty traffic** — live sports creates massive, synchronized event bursts at ad breaks; Kafka absorbs that burst so downstream systems can process at their own sustainable pace instead of being overwhelmed in real time.
• **Independent consumer scaling** — the fraud-detection consumer and the billing consumer can scale completely independently of each other and of the ad server itself.

- [ ] Explain Kafka partitions and how you'd decide how many to create for a high-volume topic like ad impressions.

> **IF ASKED, SAY THIS**
> What a partition is: the unit of parallelism in Kafka. Order is only guaranteed within a partition, not across partitions. In a consumer group, each partition is consumed by exactly one consumer at a time — so partition count is your ceiling on consumer parallelism. How messages land in a partition: if the producer spe

What a partition is: the unit of parallelism in Kafka. Order is only guaranteed within a partition, not across partitions. In a consumer group, each partition is consumed by exactly one consumer at a time — so partition count is your ceiling on consumer parallelism.

How messages land in a partition: if the producer specifies a key, all messages with that key hash to the same partition — e.g. keying impression events by user_id or ad_request_id guarantees per-user ordering. No key means round-robin distribution for maximum spread.

Sizing the partition count: partition count ≈ max(target throughput ÷ per-consumer throughput, target throughput ÷ per-producer throughput). As a rule of thumb I'd start with 3-6 partitions for a low-volume topic and 20-50+ for a genuinely high-volume one like impression events at Fubo's scale.

The gotcha worth mentioning: you can increase partition count later, but it breaks existing key-based ordering — a key that always hashed to partition 3 may now hash to partition 7, silently breaking per-user ordering guarantees for anything relying on it. For anything order-sensitive, get the partition count right up front rather than changing it later.

- [ ] What is consumer lag, how do you monitor it, and how do you fix it?

> **IF ASKED, SAY THIS**
> Definition: the gap between the latest offset produced to a partition and the offset the consumer has actually processed. If the producer is at message 1000 and the consumer has read to 800, lag is 200 — the consumer can't keep up with production rate. Monitoring: Prometheus + kafka-exporter exposing kafka_consumer_gro

Definition: the gap between the latest offset produced to a partition and the offset the consumer has actually processed. If the producer is at message 1000 and the consumer has read to 800, lag is 200 — the consumer can't keep up with production rate.

Monitoring: Prometheus + kafka-exporter exposing `kafka_consumer_group_lag`, alerted when it exceeds a threshold for a sustained period (not an instant blip). Or directly: `kafka-consumer-groups.sh --bootstrap-server kafka:9092 --describe --group my-group`.

Common causes, in the order I'd check them:
• **Slow downstream call** in the consumer (e.g. a synchronous API call per message) — fix by making it async or batching.
• **Too few consumer instances** for the partition count — max parallelism equals partition count, so scale consumers up to match.
• **A poison-pill message** stuck in an infinite retry loop — fix with a dead-letter topic after N retries.
• **A genuine producer burst** (a live sports ad break is a perfect example) — scale consumers automatically using KEDA with consumer lag as the scaling trigger, rather than a fixed replica count.

- [ ] A Kafka broker goes down. What's the impact, and what do you check?

> **IF ASKED, SAY THIS**
> Impact depends entirely on replication factor. With the standard replication factor of 3, every partition has one leader and two followers. If the broker that was a leader for some partitions goes down, the controller elects a new leader from the in-sync replica (ISR) set — this typically takes seconds to low minutes.

Impact depends entirely on replication factor. With the standard replication factor of 3, every partition has one leader and two followers. If the broker that was a leader for some partitions goes down, the controller elects a new leader from the in-sync replica (ISR) set — this typically takes seconds to low minutes. Producers writing to affected partitions see transient "leader not available" errors during that window; consumers see a brief interruption.

What I'd check immediately:
`kafka-topics.sh --bootstrap-server kafka:9092 --describe --under-replicated-partitions`
Under-replicated partitions is the critical signal — if replication factor is 3 and one broker is down, any partition where that broker was a follower now

has ISR=2 instead of 3. If a second broker fails before the first is replaced, you could hit ISR=1 — real data-loss risk. That's exactly why under-replicated partitions should be a paging alert, not something reviewed the next morning.

On Kubernetes specifically: brokers run as a StatefulSet, so a dead broker pod comes back with the same stable identity and the same PVC, and the cluster rebalances automatically once it rejoins.

- [ ] How would you run Kafka reliably on Kubernetes?

IF ASKED, SAY THIS
StatefulSet, not Deployment — each broker needs a stable identity (broker.id tied to pod ordinal) and its own persistent volume that survives rescheduling. Anti-affinity rules so broker replicas are spread across nodes and AZs — you never want two replicas of the same partition's ISR set to be one bad node away from bo

StatefulSet, not Deployment — each broker needs a stable identity (broker.id tied to pod ordinal) and its own persistent volume that survives rescheduling.

Anti-affinity rules so broker replicas are spread across nodes and AZs — you never want two replicas of the same partition's ISR set to be one bad node away from both disappearing.

PodDisruptionBudgets so a routine node drain during a cluster upgrade can't take out enough brokers at once to threaten quorum/ISR.

Disk-aware scheduling — Kafka is genuinely disk I/O and disk-space heavy in a way a typical stateless web pod isn't, so node sizing and storage class (fast SSD-backed) matter more than for most workloads.

A senior-level judgment call worth stating out loud: self-managing Kafka on K8s (with an operator like Strimzi) gives full control but real operational burden — broker upgrades, storage expansion, and tuning are genuinely hard to get right. A managed option (MSK, Confluent Cloud) trades some control for offloading that burden. Knowing when each is the right call is itself a senior-level decision, not just knowing how to run it self-managed.

- [ ] How do you handle a "poison pill" message that gets a consumer stuck retrying forever?

IF ASKED, SAY THIS
The failure mode: a consumer processes a message, hits an error, and retries the same message indefinitely — meanwhile consumer lag climbs unbounded because the consumer is stuck on one message and never advances its offset. The fix: a dead-letter topic (DLT). After N retries (with backoff), the consumer publishes the

The failure mode: a consumer processes a message, hits an error, and retries the same message indefinitely — meanwhile consumer lag climbs unbounded because the consumer is stuck on one message and never advances its offset.

The fix: a dead-letter topic (DLT). After N retries (with backoff), the consumer publishes the failed message to a separate DLT instead of retrying forever, commits its offset, and moves on to the next message. The DLT then gets reviewed and reprocessed separately — by an engineer or an automated reprocessing job — without holding up the live pipeline.

Why this matters for monitoring too: DLT depth becomes its own alertable metric — a sudden spike in DLT volume is often the earliest signal of a bad deploy or a schema change breaking downstream parsing.

- [ ] How would you design monitoring and alerting for a Kafka-based event pipeline?

IF ASKED, SAY THIS
Broker health: under-replicated partitions (page-worthy), ISR shrink/expand rate, request latency percentiles, disk usage per broker (Kafka fills disk fast at high volume). Consumer health: consumer lag per group (via kafka-exporter/JMX exporter into Prometheus), DLT depth, consumer group rebalance frequency (frequent

Broker health: under-replicated partitions (page-worthy), ISR shrink/expand rate, request latency percentiles, disk usage per broker (Kafka fills disk fast at high volume).

Consumer health: consumer lag per group (via kafka-exporter/JMX exporter into Prometheus), DLT depth, consumer group rebalance frequency (frequent rebalancing usually means something's unstable).

Business-relevant view: I'd build a Grafana dashboard specifically correlating consumer lag against live-event windows — because a small amount of lag during normal traffic is a non-event, but the exact same lag during a live sports ad break is a real, revenue-affecting problem. The alert thresholds might even reasonably differ by time-of-day/event schedule for that reason.

Alerting philosophy: alert on symptoms that matter (lag trending toward a threshold that risks stale bidding data or delayed billing), not on every internal metric that moves — same symptom-based philosophy behind the 40% alert-noise reduction from earlier roles.

- [ ] When would you choose Kafka over RabbitMQ for a new event-driven use case?

IF ASKED, SAY THIS
Kafka for high-throughput event streaming needing replay or retention — an event log multiple consumers read independently, or reprocessing history — and ordered, partitioned processing at scale. RabbitMQ for more traditional message-queue patterns, task distribution, request-reply, complex routing, where volume is low

Kafka for high-throughput event streaming needing replay or retention — an event log multiple consumers read independently, or reprocessing history — and ordered, partitioned processing at scale. RabbitMQ for more traditional message-queue patterns, task distribution, request-reply, complex

routing, where volume is lower and simpler operational overhead with built-in per-message acknowledgment and dead-lettering matters more than Kafka's added complexity.

- [ ] What causes consumer lag, and what's your actual troubleshooting path when it starts climbing?

IF ASKED, SAY THIS
Lag means producers are writing faster than a consumer group can process. First check whether it's a slow-consumer problem (processing logic itself is slow — check consumer-side latency/errors) vs. an under-provisioned problem (too few consumer instances relative to partition count — a consumer group can never have more active consumers than partitions, so partition count caps your parallelism). Fix: profile/speed up the consumer logic, or add more consumer instances up to the partition count, or repartition the topic if you're already maxed out.

The partition-count ceiling is the detail that trips people up — if a topic has 12 partitions, adding a 13th consumer to the group does nothing, since Kafka can only assign one consumer per partition within a group. Scaling consumption further requires increasing partition count (a topic-level change, not just a deployment change), which is why partition count needs to be planned ahead of expected peak throughput.

- [ ] Walk through what happens when a Kafka broker fails — how does the cluster keep serving that broker's partitions?

IF ASKED, SAY THIS
Every partition has a leader (handles all reads/writes) and follower replicas on other brokers that continuously copy from the leader. When a broker holding a partition leader fails, the controller detects it (via ZooKeeper historically, or KRaft in newer versions) and promotes an in-sync replica (ISR) on another broker to leader — producers/consumers reconnect to the new leader automatically via updated metadata.

"In-sync replica" is the key term to use precisely: only replicas that are fully caught up (ISR) are eligible for promotion, which is why `min.insync.replicas` combined with `acks=all` on the producer is what actually guarantees no data loss on a leader failure — without that config, a lagging replica could be promoted and silently lose the most recent messages.

- [ ] What's the actual difference between at-least-once, at-most-once, and exactly-once delivery in Kafka, and which do you default to?

IF ASKED, SAY THIS
At-most-once: consumer commits offset before processing — a crash mid-processing loses the message. At-least-once (the common default): consumer processes then commits offset — a crash before commit means reprocessing, so consumers must be idempotent. Exactly-once (via Kafka transactions/idempotent producer) is available but adds real overhead and complexity — most teams use at-least-once plus idempotent consumer logic instead of paying for true exactly-once semantics.

The idempotent-consumer point is what a senior answer leads with: rather than fighting for exactly-once delivery, design the consumer so processing the same message twice is harmless (e.g. an upsert keyed by message ID instead of an insert) — this sidesteps the whole problem more cheaply than Kafka transactions.

- [ ] How do you decide on partition count and key strategy for a new topic?

IF ASKED, SAY THIS
Partition count sets your max consumer parallelism and should be sized for peak expected throughput with headroom (increasing later is possible but doesn't rebalance existing data cleanly). Key strategy determines ordering guarantees: messages with the same key always land on the same partition and are strictly ordered relative to each other — pick a key that groups things needing relative order (e.g. all events for one user/order) while still spreading load evenly across partitions (a low-cardinality key like a single "type" field can create hot partitions).

The hot-partition failure mode is a good concrete example to have ready: keying purely by a coarse field like "event_type" when 90% of events are one type sends 90% of traffic to one partition, defeating the whole point of partitioning — cardinality of the key matters as much as its semantic correctness.

- [ ] A Kafka consumer group keeps rebalancing constantly, causing processing pauses across every consumer in the group. What's going on?

IF ASKED, SAY THIS
A rebalance storm is almost always caused by consumers being kicked out and rejoining repeatedly — most commonly because a consumer's processing takes longer than max.poll.interval.ms (Kafka assumes it died and reassigns its partitions), or session.timeout.ms is too aggressive relative to actual GC pauses/network blips. Every rebalance pauses the ENTIRE group, not just the slow consumer, which is why one badly-tuned consumer can degrade a whole pipeline's throughput.

The fix is almost always tuning max.poll.records down (process smaller batches per poll, so each poll cycle finishes well within the interval) combined with raising max.poll.interval.ms modestly — rather than just raising timeouts blindly, which masks the problem instead of fixing the actual processing-time mismatch.

- [ ] SNS vs. SQS vs. EventBridge — walk through when you'd use each.

IF ASKED, SAY THIS
SQS for point-to-point queuing where one consumer (or group) processes each message — decoupling with built-in visibility timeout and DLQ support. SNS for pub/sub fan-out where multiple independent consumers all need to react to the same event. EventBridge for sophisticated content-based event routing (rules matching o

SQS for point-to-point queuing where one consumer (or group) processes each message — decoupling with built-in visibility timeout and DLQ support. **SNS** for pub/sub fan-out where multiple independent consumers all need to react to the same event.

EventBridge for sophisticated content-based event routing (rules matching on event patterns, not just topic subscription) — the natural choice integrating with SaaS/third-party event sources, or when you want schema registry and more structured event governance than raw SNS/SQS provides.

- [ ] How would you design a fan-out architecture where one event needs to trigger multiple independent downstream processes? ▲

IF ASKED, SAY THIS
SNS topic as the fan-out point, each downstream process subscribing via its own SQS queue (the "fan-out to SQS" pattern) rather than subscribing Lambda functions directly — the intermediate queue gives buffering, retry, and DLQ handling per-consumer that direct Lambda subscription doesn't provide as cleanly. Each consu

SNS topic as the fan-out point, each downstream process subscribing via its **own SQS queue** (the "fan-out to SQS" pattern) rather than subscribing Lambda functions directly — the intermediate queue gives buffering, retry, and DLQ handling per-consumer that direct Lambda subscription doesn't provide as cleanly.

Each consumer processes independently; if one is temporarily down, its queue simply buffers rather than the fan-out event being lost or blocking others.

- [ ] When would you reach for Kafka/MSK instead of native AWS messaging (SNS/SQS/EventBridge)? ▲

IF ASKED, SAY THIS
When you need features native AWS messaging doesn't provide well — very high-throughput streaming with consumer replay (re-reading historical events, which SQS's delete-on-consume model doesn't support natively), strict ordering at scale across partitions, or existing significant Kafka investment/expertise and multi-cl

When you need features native AWS messaging doesn't provide well — very high-throughput streaming with consumer **replay** (re-reading historical events, which SQS's delete-on-consume model doesn't support natively), strict ordering at scale across partitions, or existing significant Kafka investment/expertise and multi-cloud portability needs.

MSK removes the operational burden of running Kafka yourself while keeping its specific capabilities — the tradeoff being a heavier, more operationally complex piece than SNS/SQS for teams that don't genuinely need Kafka-specific features.

- [ ] How do you handle message ordering and exactly-once processing requirements in an SQS-based architecture? ▲

IF ASKED, SAY THIS
SQS standard queues don't guarantee order and can deliver more than once — for strict ordering needs, FIFO queues provide ordering within a message group and exactly-once processing (dedup within a 5-minute window). Tradeoff: FIFO has lower throughput limits, so the use case needs genuine ordering/dedup requirements to

SQS standard queues don't guarantee order and can deliver more than once — for strict ordering needs, **FIFO queues** provide ordering within a message group and exactly-once processing (dedup within a 5-minute window).

Tradeoff: FIFO has lower throughput limits, so the use case needs genuine ordering/dedup requirements to justify it — many assumed-FIFO cases actually just need idempotent consumers with standard queues, simpler and higher-throughput.

- [ ] What's your approach to dead-letter queue strategy across a messaging-heavy architecture? ▲

IF ASKED, SAY THIS
Every queue that matters gets a DLQ with a defined max-receive-count before routing there, rather than indefinite retries or silent disappearance after exhaustion. Critically, a DLQ needs actual monitoring/alerting — a message landing there should generate a signal someone sees — plus a defined next step (automated rep

Every queue that matters gets a DLQ with a defined max-receive-count before routing there, rather than indefinite retries or silent disappearance after exhaustion. Critically, a DLQ needs **actual monitoring/alerting** — a message landing there should generate a signal someone sees — plus a defined next step (automated reprocessing after a fix, or manual investigation for anything financial/sensitive). A DLQ nobody looks at is just where failed messages go to be forgotten.

Shell Script

Script 1 - Health Check and Auto-Restart: Monitor a service and restart if unresponsive

IF ASKED, SAY THIS
Deployed as: Cron job every 5 minutes

Uses curl to check HTTP status code. Sends Slack alert before and after restart. Logs everything with timestamps. set -euo pipefail ensures script fails fast on any error. In production we also push a custom metric to CloudWatch on every run so we can alert if the script itself stops running.

```
#!/bin/bash
# Health check and auto-restart script
# Cron: */5 * * * * /opt/scripts/health_check.sh

set -euo pipefail

SERVICE_NAME="payments-api"
HEALTH_URL="http://localhost:8080/health"
SLACK_WEBHOOK="${SLACK_WEBHOOK_URL}"
LOG_FILE="/var/log/health_check.log"

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOG_FILE"
}

alert_slack() {
    curl -s -X POST -H "Content-type: application/json" \
        --data "{\"text\":\":rotating_light: $1\"}" \
        "$SLACK_WEBHOOK" > /dev/null
}

check_health() {
    curl -s -o /dev/null -w "%{http_code}" --max-time 10 "$HEALTH_URL"
}

restart_service() {
    log "Restarting $SERVICE_NAME"
    systemctl restart "$SERVICE_NAME"
    sleep 10
    local status
    status=$(check_health)
    if [ "$status" -eq 200 ]; then
        log "Restart successful"
        alert_slack "$SERVICE_NAME restarted successfully on $(hostname)"
    else
        log "ERROR: Restart FAILED. HTTP: $status"
        alert_slack "CRITICAL: $SERVICE_NAME restart FAILED on $(hostname)!"
        exit 1
    fi
}

main() {
    local status
    status=$(check_health)
    if [ "$status" -eq 200 ]; then
        log "OK: $SERVICE_NAME healthy (HTTP $status)"
    else
        log "WARN: HTTP $status - restarting"
        alert_slack "WARNING: $SERVICE_NAME unhealthy on $(hostname). HTTP: $status"
        restart_service
    fi
}

main
```

Script 2 - Disk Usage Monitor and Auto-Cleanup: Two-tier alerts and automatic log compression

IF ASKED, SAY THIS
Deployed as: Cron job every hour

```
#!/bin/bash
# Disk usage monitor and auto-cleanup
# Cron: 0 * * * * /opt/scripts/disk_monitor.sh

set -euo pipefail

THRESHOLD_WARN=80
THRESHOLD_CRIT=90
LOG_DIR="/var/log/myapp"
ARCHIVE_DIR="/var/log/myapp/archive"
RETENTION_DAYS=30
SLACK_WEBHOOK="${SLACK_WEBHOOK_URL}"

log() { echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1"; }

alert() {
    local level="$1" msg="$2"
    local icon=":warning:"
    [ "$level" = "CRITICAL" ] && icon=":rotating_light:"
    curl -s -X POST -H "Content-type: application/json" \
        --data "{\"text\":\"$icon *$level* $msg\"}" \
        "$SLACK_WEBHOOK" > /dev/null
}

clean_old_logs() {
    log "Cleaning logs older than $RETENTION_DAYS days"
    find "$LOG_DIR" -name "*.log" -mtime +"$RETENTION_DAYS" | while read -r f; do
        gzip -c "$f" > "$ARCHIVE_DIR/$(basename $f).gz"
        > "$f"
        log "Compressed: $f"
    done
    find "$ARCHIVE_DIR" -name "*.gz" -mtime +90 -delete
}

df -H | grep -vE 'Filesystem|tmpfs|udev' | while read -r line; do
    usage=$(echo "$line" | awk '{print $5}' | tr -d '%')
    mount=$(echo "$line" | awk '{print $6}')

    if [ "$usage" -ge "$THRESHOLD_CRIT" ]; then
        log "CRITICAL: $mount at $usage%"
        alert "CRITICAL" "Disk $mount at $usage% on $(hostname). Auto-cleaning."
        clean_old_logs
    elif [ "$usage" -ge "$THRESHOLD_WARN" ]; then
        log "WARNING: $mount at $usage%"
        alert "WARNING" "Disk $mount at $usage% on $(hostname). Investigate."
    else
        log "OK: $mount at $usage%"
    fi
done
```

- [ ] Script 3 - Kubernetes Pod Restart Monitor: Alert when pods are restarting frequently

IF ASKED, SAY THIS
Deployed as: Kubernetes CronJob every 15 minutes

```
#!/bin/bash
# Kubernetes pod restart monitor
# Run as K8s CronJob every 15 minutes

set -euo pipefail

NAMESPACE="${NAMESPACE:-production}"
RESTART_THRESHOLD=5
SLACK_WEBHOOK="${SLACK_WEBHOOK_URL}"
```

```
log() { echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1"; }

alert_slack() {
    curl -s -X POST -H "Content-type: application/json" \
        --data "{\"text\":\"$1\"}" "$SLACK_WEBHOOK" > /dev/null
}

check_pod_restarts() {
    log "Checking pod restarts in namespace: $NAMESPACE"

    high_restart_pods=$(kubectl get pods -n "$NAMESPACE" --no-headers \
        -o custom-columns=NAME:.metadata.name,RESTARTS:.status.containerStatuses[0].restartCount \
        2>/dev/null | awk -v t="$RESTART_THRESHOLD" '$2 > t {print $1 " restarts:" $2}')

    if [ -n "$high_restart_pods" ]; then
        log "High restart pods: $high_restart_pods"
        alert_slack ":warning: *High Pod Restarts in $NAMESPACE*\n$high_restart_pods"

        # Capture logs from each crashing pod for context
        echo "$high_restart_pods" | awk '{print $1}' | while read -r pod; do
            log "Previous logs from $pod:"
            kubectl logs "$pod" -n "$NAMESPACE" --previous --tail=20 2>/dev/null || true
        done
    else
        log "All pods within normal restart threshold"
    fi
}

check_pod_restarts
```

- [ ] Script 4 - Log Error Analysis and Daily Report: Parse application logs and generate error summary

IF ASKED, SAY THIS
Deployed as: Cron job at 8am daily

```
#!/bin/bash
# Log error analysis and daily report
# Cron: 0 8 * * * /opt/scripts/log_analysis.sh

set -euo pipefail

LOG_FILE="/var/log/myapp/app.log"
REPORT_FILE="/tmp/error_report_$(date +%Y%m%d).txt"
THRESHOLD=50
DATE_FILTER=$(date '+%Y-%m-%d' --date='yesterday')

generate_report() {
    {
        echo "=== Error Report for $DATE_FILTER ==="
        echo "Generated: $(date)"
        echo ""
        echo "--- Error Count by Hour ---"
        grep "$DATE_FILTER" "$LOG_FILE" | grep " ERROR " \
            | awk '{print substr($2,1,2)":00"}' | sort | uniq -c

        echo ""
        echo "--- Top 10 Error Messages ---"
        grep "$DATE_FILTER" "$LOG_FILE" | grep " ERROR " \
            | sed 's/.*ERROR //' | sort | uniq -c | sort -rn | head -10

        echo ""
        echo "--- Error Rate ---"
        total=$(grep -c "$DATE_FILTER" "$LOG_FILE" || echo 0)
        errors=$(grep "$DATE_FILTER" "$LOG_FILE" | grep -c " ERROR " || echo 0)
        echo "Total lines : $total"
        echo "Error lines : $errors"
        if [ "$total" -gt 0 ]; then
```

```
            echo "Error rate  : $(echo "scale=2; $errors * 100 / $total" | bc)%"
        fi
    } > "$REPORT_FILE"

    error_count=$(grep "$DATE_FILTER" "$LOG_FILE" | grep -c " ERROR " || echo 0)
    if [ "$error_count" -gt "$THRESHOLD" ]; then
        mail -s "High Error Rate: $error_count errors yesterday" \
            sre-team@company.com < "$REPORT_FILE"
    fi
}

[ -f "$LOG_FILE" ] || { echo "Log file not found: $LOG_FILE"; exit 1; }
generate_report
echo "Log analysis complete"
```

- [ ] Top CPU & memory processes (Linux Fundamentals)

IF ASKED, SAY THIS
Fastest way to see what's actually hammering a box.

```
ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%cpu | head -11
ps -eo pid,ppid,cmd,%mem,%cpu --sort=-%mem | head -11
```

- [ ] Find and kill whatever is holding a port (Linux Fundamentals)

IF ASKED, SAY THIS
For the classic “address already in use” situation.

```
lsof -i :8080 -t | xargs -r kill -9
# or, if lsof isn't installed:
fuser -k 8080/tcp
```

- [ ] Find what's eating disk space (Linux Fundamentals)

IF ASKED, SAY THIS
Disk-full triage in two commands.

```
df -h
du -ah /var/log 2>/dev/null | sort -rh | head -20
```

- [ ] Tail logs and grep errors from the last N minutes (Linux Fundamentals)

IF ASKED, SAY THIS
Works for journald services and plain log files.

```
# systemd service:
journalctl -u myapp --since "10 minutes ago" | grep -iE "error|exception|fail"

# plain file, timestamped lines:
awk -v d="$(date -d '10 minutes ago' '+%Y-%m-%dT%H:%M:%S')" '$0 > d' app.log | grep -i error
```

- [ ] Check a certificate's expiry from the shell (Linux Fundamentals)

IF ASKED, SAY THIS
No browser needed — pure openssl.

```
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null \
  | openssl x509 -noout -enddate
```

- [ ] Generic wait-for-healthy loop with timeout (Automation Patterns)

IF ASKED, SAY THIS
The pattern behind almost every deploy script's readiness check.

```
TIMEOUT=120
ELAPSED=0
until curl -sf https://service.internal/healthz >/dev/null; do
  if [ "$ELAPSED" -ge "$TIMEOUT" ]; then
    echo "Timed out waiting for health after ${TIMEOUT}s"
    exit 1
  fi
  sleep 5
  ELAPSED=$((ELAPSED + 5))
done
echo "Service healthy after ${ELAPSED}s"
```

- [ ] Find every pod that isn't Running, across all namespaces (Kubernetes)

IF ASKED, SAY THIS
First command in any cluster-wide triage.

```
kubectl get pods --all-namespaces \
  --field-selector=status.phase!=Running,status.phase!=Succeeded
```

- [ ] Pods sorted by restart count (Kubernetes)

IF ASKED, SAY THIS
Fastest way to spot a flapping pod cluster-wide.

```
kubectl get pods --all-namespaces \
  --sort-by='.status.containerStatuses[0].restartCount' | tail -20
```

- [ ] Rolling-restart every deployment matching a label (Kubernetes)

IF ASKED, SAY THIS
Bulk bounce without touching each deployment by hand.

```
kubectl get deploy -n prod -l tier=backend -o name \
  | xargs -r -I{} kubectl rollout restart {} -n prod
```

- [ ] EC2 instances missing a required tag (AWS CLI)

IF ASKED, SAY THIS
Compliance check, one line, server-side JMESPath filter.

```
aws ec2 describe-instances \
  --query "Reservations[].Instances[?!not_null(Tags[?Key=='Owner'].Value)].InstanceId" \
  --output text
```

- [ ] Every S3 bucket with its region (AWS CLI)

IF ASKED, SAY THIS
list-buckets doesn't return region — you need a second call per bucket, and us-east-1 comes back as None.

```
for b in $(aws s3api list-buckets --query 'Buckets[].Name' --output text); do
  region=$(aws s3api get-bucket-location --bucket "$b" --query 'LocationConstraint' --output text)
  echo "$b: ${region:-us-east-1}"
done
```

- [ ] Security groups with SSH open to the world (jq version) (AWS CLI)

IF ASKED, SAY THIS
Same audit as the boto3 one, but pure CLI + jq.

```
aws ec2 describe-security-groups --output json \
  | jq -r '.SecurityGroups[]
      | select(.IpPermissions[]?.FromPort==22
          and (.IpPermissions[]?.IpRanges[]?.CidrIp=="0.0.0.0/0"))
      | .GroupId'
```

- [ ] CI-safe terraform apply using detailed-exitcode (Terraform)

IF ASKED, SAY THIS
0 = no changes, 2 = changes to apply, 1 = real failure. This is the actual pipeline gate.

```
terraform plan -detailed-exitcode -out=tfplan
code=$?

if [ $code -eq 0 ]; then
  echo "No changes."
  exit 0
elif [ $code -eq 2 ]; then
  terraform apply tfplan
else
  echo "Plan failed."
  exit 1
fi
```

- [ ] Confirm nothing is running before force-unlocking state (Terraform)

IF ASKED, SAY THIS
Never force-unlock reflexively — check first.

```
if pgrep -f "terraform apply" >/dev/null; then
  echo "An apply is still running — do not unlock."
  exit 1
fi

read -p "Confirm force-unlock ID $1? [y/N] " ans
[ "$ans" = "y" ] && terraform force-unlock -force "$1"
```

- [ ] Azure VMs with power state (Azure CLI)

IF ASKED, SAY THIS
az equivalent of the EC2 instance-listing script.

```
az vm list -d -o table \
  --query "[].{Name:name, PowerState:powerState, ResourceGroup:resourceGroup}"
```

- [ ] Pre-commit hook blocking obvious secrets (Git / CI)

IF ASKED, SAY THIS
Drop in .git/hooks/pre-commit. Simple grep, not a real secret scanner, but catches the obvious cases before they leave your laptop.

```
#!/bin/bash
if git diff --cached | grep -EiI "aws_secret_access_key|BEGIN (RSA|EC) PRIVATE KEY|api[_-]?key\s*="; then
  echo "Possible secret detected in staged changes — commit blocked."
  exit 1
fi
```

- [ ] Write a Python script to monitor an API and trigger a ServiceNow alert if it is down.

IF ASKED, SAY THIS
import requests import time import logging import os logging.basicConfig(level=logging.INFO, format='%(asctime)s %(levelname)s %(message)s') ENDPOINT = 'https://api.example.com/health' THRESHOLD_MS = 2000 CHECK_INTERVAL = 60 def check_endpoint(url): try: start = time.time() resp = requests.g

```
import requests
import time
import logging
import os

logging.basicConfig(level=logging.INFO,
format='%(asctime)s %(levelname)s %(message)s')

ENDPOINT = 'https://api.example.com/health'
THRESHOLD_MS = 2000
CHECK_INTERVAL = 60

def check_endpoint(url):
try:
start = time.time()
resp = requests.get(url, timeout=10)
latency_ms = (time.time() - start) * 1000
return {
'status' : resp.status_code,
'latency_ms': round(latency_ms, 2),
'healthy' : resp.status_code == 200 and latency_ms < THRESHOLD_MS
}
except requests.exceptions.RequestException as e:
return {'status': None, 'latency_ms': None,
'healthy': False, 'error': str(e)}

def create_snow_incident(details):
url = os.environ['SNOW_INSTANCE'] + '/api/now/table/incident'
auth = (os.environ['SNOW_USER'], os.environ['SNOW_PASS'])
data = {
'short_description': 'API health check FAILED: ' + ENDPOINT,
'description' : str(details),
'urgency' : '1',
'impact' : '1',
'assignment_group' : 'SRE-Platform'
}
resp = requests.post(url, json=data, auth=auth,
headers={'Content-Type':'application/json'}, timeout=10)
resp.raise_for_status()
logging.info('INC created: ' + resp.json()['result']['number'])

if __name__ == '__main__':
while True:
result = check_endpoint(ENDPOINT)
if not result['healthy']:
logging.error('UNHEALTHY: ' + str(result))
create_snow_incident(result)
else:
```

```
logging.info('OK | ' + str(result['latency_ms']) + ' ms')
time.sleep(CHECK_INTERVAL)
```

- [ ] Write a Python script to find and report untagged AWS EC2 instances using boto3.

IF ASKED, SAY THIS

import boto3 import json from datetime import datetime REQUIRED_TAGS = ['Environment', 'Owner', 'Cost-Center', 'Data-Classification'] def get_missing_tags(instance): existing = {t['Key'] for t in instance.get('Tags', [])} return [tag for tag in REQUIRED_TAGS if tag not in existing] def find_untagged

```
import boto3
import json
from datetime import datetime

REQUIRED_TAGS = ['Environment', 'Owner', 'Cost-Center', 'Data-Classification']

def get_missing_tags(instance):
existing = {t['Key'] for t in instance.get('Tags', [])}
return [tag for tag in REQUIRED_TAGS if tag not in existing]

def find_untagged_instances(region='ap-south-1'):
ec2 = boto3.client('ec2', region_name=region)
report = []

paginator = ec2.get_paginator('describe_instances')
for page in paginator.paginate():
for reservation in page['Reservations']:
for inst in reservation['Instances']:
if inst['State']['Name'] == 'terminated':
continue
missing = get_missing_tags(inst)
if missing:
report.append({
'instance_id' : inst['InstanceId'],
'instance_type': inst['InstanceType'],
'state' : inst['State']['Name'],
'launch_time' : str(inst['LaunchTime']),
'missing_tags' : missing
})
return report

if __name__ == '__main__':
results = find_untagged_instances()
print('Untagged instances found: ' + str(len(results)))
for r in results:
print(json.dumps(r, indent=2))
# In production: send this report to a Slack webhook or S3 bucket
# and trigger a ServiceNow task if count is above threshold
```

- [ ] Write a shell script to check disk usage and alert if above a threshold.

IF ASKED, SAY THIS

#!/bin/bash set -euo pipefail THRESHOLD=80 LOG_FILE='/var/log/disk_check.log' SLACK_WEBHOOK='https://hooks.slack.com/services/YOUR/WEBHOOK/URL' log() { echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOG_FILE" } send_slack_alert() { local message="$1" curl -s -X POST -H 'Content-type: applicatio

```
#!/bin/bash
set -euo pipefail

THRESHOLD=80
LOG_FILE='/var/log/disk_check.log'
SLACK_WEBHOOK='https://hooks.slack.com/services/YOUR/WEBHOOK/URL'

log() {
echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOG_FILE"
}

send_slack_alert() {
local message="$1"
curl -s -X POST -H 'Content-type: application/json' \
--data "{\"text\":\"$message\"}" \
"$SLACK_WEBHOOK"
}

log 'Starting disk usage check'

# Check each mounted filesystem
df -H | grep -vE '^Filesystem|tmpfs|udev|cdrom' | while read -r line; do
usage=$(echo "$line" | awk '{print $5}' | tr -d '%')
mount=$(echo "$line" | awk '{print $6}')

if [ "$usage" -ge "$THRESHOLD" ]; then
msg="ALERT: Disk usage ${usage}% on ${HOSTNAME}:${mount} (threshold: ${THRESHOLD}%)"
```

```
log "$msg"
send_slack_alert ":warning: $msg"
else
log "OK: ${mount} is at ${usage}%"
fi
done

log 'Disk check complete'

# Schedule via cron: */15 * * * * /opt/scripts/disk_check.sh
```

- [ ] Write a Bash snippet to rotate and archive logs older than a given number of days.

IF ASKED, SAY THIS
Worth mentioning out loud in an interview: in a regulated environment you wouldn't just delete — audit/compliance retention requirements often mean the 'delete' step is actually 'move to cold storage' with its own retention policy, not outright removal.

```
#!/usr/bin/env bash
LOG_DIR="/var/log/app"
DAYS_TO_KEEP=7
ARCHIVE_DAYS=30

# compress logs older than DAYS_TO_KEEP
find "$LOG_DIR" -name "*.log" -mtime +$DAYS_TO_KEEP -exec gzip {} ;

# delete compressed archives older than ARCHIVE_DAYS
find "$LOG_DIR" -name "*.log.gz" -mtime +$ARCHIVE_DAYS -delete
```

Worth mentioning out loud in an interview: in a regulated environment you wouldn't just delete — audit/compliance retention requirements often mean the 'delete' step is actually 'move to cold storage' with its own retention policy, not outright removal.

- [ ] How would you script a canary analysis check comparing error rates between the old and new version?

IF ASKED, SAY THIS
Query the metrics backend for the error rate of version=canary versus version=stable over a rolling window, compute the delta or ratio between them, and compare against a pre-configured threshold — e.g. canary error rate must not exceed stable by more than X percentage points. Output a clear pass/fa

Query the metrics backend for the error rate of `version=canary` versus `version=stable` over a rolling window, compute the delta or ratio between them, and compare against a pre-configured threshold — e.g. canary error rate must not exceed stable by more than X percentage points. Output a clear pass/fail plus the actual numbers used, so the decision is auditable. Ideally this feeds an automated pipeline gate (Flagger or Argo Rollouts can do exactly this natively) rather than a human eyeballing two dashboards side by side under release pressure.

- [ ] Write a shell one-liner to find and delete files older than N days.

IF ASKED, SAY THIS
find with -mtime and -delete is the safe, standard pattern — answer the literal requirement (all matching files) rather than over-filtering by an assumed extension unless the question specifies one.

find with -mtime and -delete is the safe, standard pattern — answer the literal requirement (all matching files) rather than over-filtering by an assumed extension unless the question specifies one.

```
find /var/log -type f -mtime +10 -delete
```

- [ ] Write a shell script to check if a service is running and restart it if not.

IF ASKED, SAY THIS
Use the init system's own status check (systemctl is-active) rather than grepping ps output, which is fragile.

Use the init system's own status check (systemctl is-active) rather than grepping ps output, which is fragile.

```
#!/bin/bash
SERVICE="nginx"
if ! systemctl is-active --quiet "$SERVICE"; then
  echo "$SERVICE is down, restarting..."
```

```
  systemctl restart "$SERVICE"
fi
```

- [ ] What's your rule of thumb for when a manual step should become a script, and when a script should become a full automation/pipeline step?

> **IF ASKED, SAY THIS**
> Second time you do a manual step, write it down as a documented step; second time you follow that documented step, script it; once a script is being run regularly by multiple people or triggered by an event, wire it into the actual pipeline/automation rather than leaving it as a script someone has to remember to run.

Second time you do a manual step, write it down as a documented step; second time you follow that documented step, script it; once a script is being run regularly by multiple people or triggered by an event, wire it into the actual pipeline/automation rather than leaving it as a script someone has to remember to run.

- [ ] How do you make an automation script safe to run against production?

> **IF ASKED, SAY THIS**
> Dry-run mode by default with an explicit flag required for the real/destructive action, idempotency (safe to re-run without duplicating effects), and logging every action taken so there's an audit trail if something needs to be manually reversed.

Dry-run mode by default with an explicit flag required for the real/destructive action, idempotency (safe to re-run without duplicating effects), and logging every action taken so there's an audit trail if something needs to be manually reversed.

Python Script

Retry decorator with exponential backoff (Reusable Patterns)

IF ASKED, SAY THIS
The single most commonly asked ‘write a function’ question in these interviews.

```
import time, functools

def retry(times=3, delay=1, backoff=2):
    def deco(fn):
        @functools.wraps(fn)
        def wrapper(*args, **kwargs):
            d = delay
            for attempt in range(1, times + 1):
                try:
                    return fn(*args, **kwargs)
                except Exception as e:
                    if attempt == times:
                        raise
                    print(f"attempt {attempt} failed: {e}, retrying in {d}s")
                    time.sleep(d)
                    d *= backoff
        return wrapper
    return deco

# usage:
# @retry(times=3, delay=2)
# def call_flaky_api(): ...
```

Minimal circuit breaker (Reusable Patterns)

IF ASKED, SAY THIS
Stops hammering a dependency that's already failing.

```
import time

class CircuitBreaker:
    def __init__(self, fail_threshold=5, reset_after=30):
        self.fail_threshold = fail_threshold
        self.reset_after = reset_after
        self.failures = 0
        self.opened_at = None

    def call(self, fn, *args, **kwargs):
        if self.opened_at and time.time() - self.opened_at < self.reset_after:
            raise RuntimeError("circuit open")
        try:
            result = fn(*args, **kwargs)
            self.failures = 0
            self.opened_at = None
            return result
        except Exception:
            self.failures += 1
            if self.failures >= self.fail_threshold:
                self.opened_at = time.time()
            raise
```

Structured JSON logging setup (Reusable Patterns)

IF ASKED, SAY THIS
Ship-ready — every log line is one grep-able JSON object.

```
import logging, json, sys

class JsonFormatter(logging.Formatter):
    def format(self, record):
        return json.dumps({
            "ts": self.formatTime(record),
            "level": record.levelname,
            "msg": record.getMessage(),
            "logger": record.name,
        })

handler = logging.StreamHandler(sys.stdout)
handler.setFormatter(JsonFormatter())
logging.basicConfig(level=logging.INFO, handlers=[handler])
```

- [ ] argparse CLI skeleton with --dry-run (Reusable Patterns) ▲

IF ASKED, SAY THIS
The standard shape for any automation tool teammates will run.

```
import argparse

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--dry-run", action="store_true")
    p.add_argument("--env", required=True, choices=["dev", "staging", "prod"])
    args = p.parse_args()

    if args.dry_run:
        print(f"[dry-run] would act on {args.env}")
        return 0

    # real action here
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

- [ ] EC2 instances missing a required tag (AWS — boto3) ▲

IF ASKED, SAY THIS
Paginated properly — won't silently miss instances past page one.

```
import boto3

def untagged_instances(tag="Owner", region="ap-south-1"):
    ec2 = boto3.client("ec2", region_name=region)
    offenders = []
    paginator = ec2.get_paginator("describe_instances")
    for page in paginator.paginate():
        for r in page["Reservations"]:
            for i in r["Instances"]:
                tags = {t["Key"]: t["Value"] for t in i.get("Tags", [])}
                if tag not in tags:
                    offenders.append(i["InstanceId"])
    return offenders
```

- [ ] Unencrypted EBS volumes across every region (AWS — boto3) ▲

IF ASKED, SAY THIS
EC2/EBS is region-scoped — no single global call covers this.

```python
import boto3

def unencrypted_volumes_all_regions():
    ec2 = boto3.client("ec2", region_name="us-east-1")
    regions = [r["RegionName"] for r in ec2.describe_regions()["Regions"]]
    found = {}
    for region in regions:
        c = boto3.client("ec2", region_name=region)
        vols = c.describe_volumes(
            Filters=[{"Name": "encryption", "Values": ["false"]}]
        )["Volumes"]
        if vols:
            found[region] = [v["VolumeId"] for v in vols]
    return found
```

- [ ] Total size of everything under an S3 prefix (AWS — boto3) ▲

IF ASKED, SAY THIS
Paginator-based aggregation — the shape you'd reuse for any S3 rollup.

```python
import boto3

def bucket_prefix_size(bucket, prefix=""):
    s3 = boto3.client("s3")
    paginator = s3.get_paginator("list_objects_v2")
    total = 0
    for page in paginator.paginate(Bucket=bucket, Prefix=prefix):
        for obj in page.get("Contents", []):
            total += obj["Size"]
    return total
```

- [ ] Pods with a high restart count, across namespaces (Kubernetes) ▲

IF ASKED, SAY THIS
The real toil-reduction script pattern — replaces a manual kubectl sweep.

```python
from kubernetes import client, config

config.load_kube_config()
v1 = client.CoreV1Api()

def flapping_pods(threshold=5):
    out = []
    for ns in v1.list_namespace().items:
        pods = v1.list_namespaced_pod(ns.metadata.name).items
        for pod in pods:
            for cs in (pod.status.container_statuses or []):
                if cs.restart_count >= threshold:
                    out.append((ns.metadata.name, pod.metadata.name, cs.restart_count))
    return out
```

- [ ] Rollout-restart a deployment via annotation patch (Kubernetes) ▲

IF ASKED, SAY THIS
The safe way — lets the existing rolling-update strategy pace it, unlike deleting pods directly.

```python
from kubernetes import client, config
from datetime import datetime, timezone

config.load_kube_config()
```

```
apps = client.AppsV1Api()

def rollout_restart(name, namespace="default"):
    now = datetime.now(timezone.utc).isoformat()
    body = {"spec": {"template": {"metadata": {"annotations": {
        "kubectl.kubernetes.io/restartedAt": now
    }}}}}
    apps.patch_namespaced_deployment(name, namespace, body)
```

- [ ] Azure VMs missing a required tag (Azure — SDK)

IF ASKED, SAY THIS
Mirrors the boto3 tag-audit script — same pattern, Azure SDK.

```
from azure.identity import DefaultAzureCredential
from azure.mgmt.compute import ComputeManagementClient

SUBSCRIPTION_ID = "YOUR_SUBSCRIPTION_ID"

def untagged_vms(required_tag="Owner"):
    cred = DefaultAzureCredential()
    compute = ComputeManagementClient(cred, SUBSCRIPTION_ID)
    offenders = []
    for vm in compute.virtual_machines.list_all():
        tags = vm.tags or {}
        if required_tag not in tags:
            offenders.append(vm.name)
    return offenders
```

- [ ] AKS clusters and their Kubernetes version (Azure — SDK)

IF ASKED, SAY THIS
Upgrade-planning script — spot clusters running an old version at a glance.

```
from azure.identity import DefaultAzureCredential
from azure.mgmt.containerservice import ContainerServiceClient

SUBSCRIPTION_ID = "YOUR_SUBSCRIPTION_ID"
cred = DefaultAzureCredential()
aks = ContainerServiceClient(cred, SUBSCRIPTION_ID)

for cluster in aks.managed_clusters.list():
    print(cluster.name, cluster.kubernetes_version)
```

- [ ] Minimal Slack / webhook notifier (Monitoring & Alerting)

IF ASKED, SAY THIS
The five lines every alerting script eventually needs.

```
import requests

def notify(webhook_url, text):
    resp = requests.post(webhook_url, json={"text": text}, timeout=5)
    resp.raise_for_status()
```

- [ ] Check multiple endpoints concurrently (Monitoring & Alerting)

IF ASKED, SAY THIS

Threading is right here — I/O-bound, releases the GIL while waiting.

```
import requests
from concurrent.futures import ThreadPoolExecutor, as_completed

def check(url):
    try:
        r = requests.get(url, timeout=5)
        return url, r.status_code
    except Exception as e:
        return url, f"ERROR: {e}"

def check_all(urls):
    with ThreadPoolExecutor(max_workers=10) as ex:
        futures = {ex.submit(check, u): u for u in urls}
        return dict(f.result() for f in as_completed(futures))
```

- [ ] Count error lines in a log by type (Log Processing) ▲

IF ASKED, SAY THIS
Classic 'parse this log' live-coding ask.

```
import re
from collections import Counter

def count_errors(path):
    pattern = re.compile(r"ERROR\s+(\w+Exception|\w+Error)")
    counts = Counter()
    with open(path) as f:
        for line in f:
            m = pattern.search(line)
            if m:
                counts[m.group(1)] += 1
    return counts
```

- [ ] Script 1 - API Health Monitor with ServiceNow Integration: Monitor endpoints and auto-create ServiceNow incidents on failure ▲

IF ASKED, SAY THIS
Deployed as: AWS Lambda triggered by EventBridge every 1 minute

Deployed as AWS Lambda triggered by EventBridge every minute. Uses dataclass for type safety. All credentials from AWS Secrets Manager - never hardcoded. Creates ServiceNow INC automatically so no manual ticket creation. Structured logging so every run is searchable in CloudWatch. The Lambda emits a custom CloudWatch metric on each run so we can alert if the automation itself stops working.

```
"""
API Health Monitor - auto-creates ServiceNow incidents on failure
Deploy as AWS Lambda triggered by EventBridge every 1 minute
"""
import requests, time, logging, os, json
from dataclasses import dataclass
from typing import Optional

logging.basicConfig(level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s")
logger = logging.getLogger(__name__)

ENDPOINTS = [
    {"name": "payments-api",  "url": "https://api.example.com/health",   "timeout_ms": 2000},
    {"name": "auth-service",  "url": "https://auth.example.com/health",  "timeout_ms": 1000},
    {"name": "fraud-service", "url": "https://fraud.example.com/health", "timeout_ms": 3000},
]

@dataclass
class HealthResult:
    name:       str
```

```
    url:        str
    status:     Optional[int]
    latency_ms: Optional[float]
    healthy:    bool
    error:      Optional[str] = None


def check_endpoint(name, url, timeout_ms) -> HealthResult:
    try:
        start = time.time()
        resp = requests.get(url, timeout=timeout_ms / 1000)
        latency_ms = (time.time() - start) * 1000
        return HealthResult(
            name=name, url=url,
            status=resp.status_code,
            latency_ms=round(latency_ms, 2),
            healthy=resp.status_code == 200 and latency_ms < timeout_ms
        )
    except requests.exceptions.RequestException as exc:
        return HealthResult(name=name, url=url,
            status=None, latency_ms=None,
            healthy=False, error=str(exc))


def create_snow_incident(result: HealthResult) -> str:
    snow_url = os.environ["SNOW_INSTANCE"] + "/api/now/table/incident"
    payload = {
        "short_description": f"Health check FAILED: {result.name}",
        "description": (
            f"URL: {result.url}\n"
            f"Status: {result.status}\n"
            f"Latency: {result.latency_ms}ms\n"
            f"Error: {result.error}"
        ),
        "urgency": "1", "impact": "1",
        "assignment_group": "SRE-Platform"
    }
    resp = requests.post(snow_url, json=payload,
        auth=(os.environ["SNOW_USER"], os.environ["SNOW_PASS"]),
        headers={"Content-Type": "application/json"}, timeout=30)
    resp.raise_for_status()
    number = resp.json()["result"]["number"]
    logger.info("Created incident %s for %s", number, result.name)
    return number


def lambda_handler(event, context):
    failed = []
    for ep in ENDPOINTS:
        r = check_endpoint(ep["name"], ep["url"], ep["timeout_ms"])
        if r.healthy:
            logger.info("OK  | %s | %sms", r.name, r.latency_ms)
        else:
            logger.error("FAIL| %s | %s", r.name, r.error or r.status)
            failed.append(r)
            create_snow_incident(r)
    return {"statusCode": 200,
            "body": json.dumps({"checked": len(ENDPOINTS), "failed": len(failed)})}
```

Script 2 - AWS Tagging Compliance Checker: Find untagged EC2 and RDS resources and upload report to S3

IF ASKED, SAY THIS
Deployed as: AWS Lambda triggered daily by EventBridge

Runs daily and uploads a JSON compliance report to S3 with the date in the key path. Uses paginators so it handles accounts with thousands of instances. In financial services this report feeds directly into the compliance dashboard. We added an alert if violation count exceeds 10 so the team fixes tags before the monthly audit.

```
"""
AWS Resource Tagging Compliance Checker
Finds EC2 and RDS instances missing required tags
Run as Lambda daily or as part of CI/CD compliance gate
"""
import boto3, json, logging
from datetime import datetime
from typing import List, Dict

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

REQUIRED_TAGS = ["Environment", "Owner", "Cost-Center", "Data-Classification"]
REGION = "ap-south-1"


def get_missing_tags(tags: list) -> List[str]:
    existing = {t["Key"] for t in (tags or [])}
    return [tag for tag in REQUIRED_TAGS if tag not in existing]


def check_ec2_instances() -> List[Dict]:
    ec2 = boto3.client("ec2", region_name=REGION)
    violations = []
    paginator = ec2.get_paginator("describe_instances")
    for page in paginator.paginate():
        for reservation in page["Reservations"]:
            for inst in reservation["Instances"]:
                if inst["State"]["Name"] == "terminated":
                    continue
                missing = get_missing_tags(inst.get("Tags", []))
                if missing:
                    violations.append({
                        "resource_type": "EC2",
                        "resource_id":   inst["InstanceId"],
                        "instance_type": inst["InstanceType"],
                        "state":         inst["State"]["Name"],
                        "missing_tags":  missing
                    })
    return violations


def check_rds_instances() -> List[Dict]:
    rds = boto3.client("rds", region_name=REGION)
    violations = []
    paginator = rds.get_paginator("describe_db_instances")
    for page in paginator.paginate():
        for db in page["DBInstances"]:
            tags = rds.list_tags_for_resource(
                ResourceName=db["DBInstanceArn"])["TagList"]
            missing = get_missing_tags(tags)
            if missing:
                violations.append({
                    "resource_type": "RDS",
                    "resource_id":   db["DBInstanceIdentifier"],
                    "engine":        db["Engine"],
                    "missing_tags":  missing
                })
    return violations


def lambda_handler(event, context):
    all_v = check_ec2_instances() + check_rds_instances()
    logger.info("Total violations: %s", len(all_v))

    s3 = boto3.client("s3", region_name=REGION)
    key = f"tagging/{datetime.now().strftime('%Y/%m/%d')}/report.json"
    s3.put_object(Bucket="company-compliance-reports", Key=key,
        Body=json.dumps({"date": datetime.now().isoformat(),
                         "total": len(all_v), "violations": all_v}, indent=2),
```

```
        ContentType="application/json")
    return {"violations": len(all_v), "report_key": key}
```

Script 3 - Kubernetes Deployment Health Checker: Post-deployment validation that triggers pipeline rollback on failure

IF ASKED, SAY THIS
Deployed as: Jenkins or Harness post-deploy step

Called from the Jenkins or Harness pipeline as a post-deployment step. If it exits with code 1 the pipeline triggers a rollback automatically. Checks both replica readiness and CrashLoopBackOff. Captures previous container logs automatically for fast debugging. We also set a hard timeout so it never blocks the pipeline indefinitely if something unexpected happens.

```
"""
Kubernetes Deployment Health Checker
Post-deployment validation - exits code 1 to trigger pipeline rollback
Called from Jenkins or Harness as post-deploy step
"""
import subprocess, json, time, sys, logging, argparse

logging.basicConfig(level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s")
logger = logging.getLogger(__name__)


def kubectl(args: list) -> dict:
    cmd = ["kubectl"] + args + ["-o", "json"]
    r = subprocess.run(cmd, capture_output=True, text=True, timeout=30)
    if r.returncode != 0:
        raise RuntimeError(f"kubectl failed: {r.stderr}")
    return json.loads(r.stdout)


def wait_for_ready(deployment: str, namespace: str,
                   expected: int, timeout: int = 300) -> bool:
    deadline = time.time() + timeout
    logger.info("Waiting for %s/%s to have %s ready replicas",
                namespace, deployment, expected)
    while time.time() < deadline:
        try:
            data  = kubectl(["get", "deployment", deployment, "-n", namespace])
            ready = data["status"].get("readyReplicas", 0)
            total = data["status"].get("replicas", 0)
            logger.info("Ready: %s/%s", ready, total)
            if ready >= expected:
                logger.info("Deployment %s is healthy", deployment)
                return True
        except Exception as exc:
            logger.warning("Check error: %s", exc)
        time.sleep(10)
    logger.error("Timed out after %ss", timeout)
    return False


def check_no_crash_loop(namespace: str) -> bool:
    r = subprocess.run(
        ["kubectl", "get", "pods", "-n", namespace, "--no-headers"],
        capture_output=True, text=True, timeout=30)
    crash_pods = [l.split()[0] for l in r.stdout.splitlines()
                  if "CrashLoopBackOff" in l or "Error" in l]
    if crash_pods:
        logger.error("CrashLoopBackOff pods: %s", crash_pods)
        for pod in crash_pods:
            subprocess.run(["kubectl", "logs", pod, "-n", namespace,
                            "--previous", "--tail=20"], timeout=30)
        return False
    return True


def main():
```

```
    p = argparse.ArgumentParser()
    p.add_argument("--deployment", required=True)
    p.add_argument("--namespace",  required=True)
    p.add_argument("--replicas",   type=int, default=3)
    p.add_argument("--timeout",    type=int, default=300)
    args = p.parse_args()

    ok = (wait_for_ready(args.deployment, args.namespace,
                         args.replicas, args.timeout)
          and check_no_crash_loop(args.namespace))

    if ok:
        logger.info("All health checks PASSED. Deployment is healthy.")
        sys.exit(0)
    else:
        logger.error("Health checks FAILED. Pipeline will trigger rollback.")
        sys.exit(1)


if __name__ == "__main__":
    main()
```

Script 4 - SLO Error Budget Tracker: Query Prometheus, calculate budget consumed, post colour-coded Slack report

IF ASKED, SAY THIS
Deployed as: Kubernetes CronJob at 8am daily

This script directly supports the SRE error budget conversation. It queries Prometheus daily, calculates how much of the monthly error budget each service has consumed, and posts a colour-coded report to Slack. RED means budget nearly exhausted and we freeze feature releases. GREEN means we have headroom to take risks. Leadership loves this because it translates reliability into business language they understand.

```
"""
SLO Error Budget Tracker
Queries Prometheus, calculates budget consumed per service, posts Slack report
Run as K8s CronJob daily at 8am
"""
import requests, os, json, logging
from datetime import datetime

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

PROMETHEUS_URL = os.environ.get("PROMETHEUS_URL", "http://prometheus:9090")
SLACK_WEBHOOK  = os.environ["SLACK_WEBHOOK_URL"]

SERVICES = [
    {"name": "payments-service", "slo": 0.999},
    {"name": "auth-service",     "slo": 0.9995},
    {"name": "fraud-service",    "slo": 0.998},
]


def promql(query: str) -> float:
    resp = requests.get(f"{PROMETHEUS_URL}/api/v1/query",
                        params={"query": query}, timeout=30)
    resp.raise_for_status()
    results = resp.json()["data"]["result"]
    return float(results[0]["value"][1]) if results else 0.0


def calc_budget(name: str, slo: float) -> dict:
    total   = promql(f'increase(http_requests_total{{service="{name}"}}[30d])')
    success = promql(
        f'increase(http_requests_total{{service="{name}",status!~"5.."}}[30d])')
    if total == 0:
        return {"name": name, "error": "No data"}

    actual_reliability  = success / total
    budget_total        = 1 - slo
```

```
    errors_allowed      = total * budget_total
    errors_actual       = total - success
    consumed_pct        = round(errors_actual / errors_allowed * 100, 1) if errors_allowed else 0
    remaining_pct       = round(100 - consumed_pct, 1)
    status = ("CRITICAL" if remaining_pct < 10
              else "WARNING" if remaining_pct < 25
              else "OK")

    return {
        "name":          name,
        "slo":           f"{slo * 100}%",
        "actual":        f"{actual_reliability * 100:.4f}%",
        "total_requests": int(total),
        "total_errors":   int(errors_actual),
        "consumed_pct":   consumed_pct,
        "remaining_pct":  remaining_pct,
        "status":         status
    }


def send_slack_report(budgets: list):
    lines = [f"*SLO Error Budget Report - {datetime.now().strftime('%Y-%m-%d')}*\n"]
    for b in budgets:
        if "error" in b:
            lines.append(f"  {b['name']}: {b['error']}")
            continue
        icon = (":red_circle:"    if b["status"] == "CRITICAL"
                else ":yellow_circle:" if b["status"] == "WARNING"
                else ":green_circle:")
        lines.append(
            f"{icon} *{b['name']}*\n"
            f"    SLO: {b['slo']} | Actual: {b['actual']}\n"
            f"    Budget consumed: {b['consumed_pct']}%"
            f" | Remaining: {b['remaining_pct']}%"
        )
    requests.post(SLACK_WEBHOOK, json={"text": "\n".join(lines)}, timeout=10)
    logger.info("Slack report sent")


def main():
    budgets = [calc_budget(s["name"], s["slo"]) for s in SERVICES]
    for b in budgets:
        logger.info("%s", json.dumps(b))
    send_slack_report(budgets)


if __name__ == "__main__":
    main()
```

- [ ] Show a Python snippet for auto-remediating pods that keep failing readiness.

IF ASKED, SAY THIS
The shape matters more than the exact code — read state, decide, act, log the action for audit:

The shape matters more than the exact code — read state, decide, act, log the action for audit:

```
from kubernetes import client, config

config.load_incluster_config()
v1 = client.CoreV1Api()

def restart_stuck_pods(namespace, max_restarts=5):
    pods = v1.list_namespaced_pod(namespace)
    for pod in pods.items:
        for cs in pod.status.container_statuses or []:
            if cs.restart_count > max_restarts and not cs.ready:
                print(f"Deleting stuck pod {pod.metadata.name} "
                      f"(restarts={cs.restart_count})")
```

```
                v1.delete_namespaced_pod(pod.metadata.name, namespace)
                break
```

In an interview, flag the guardrails you'd add: a dry-run mode, a max-actions-per-run cap so a bad rule can't wipe out a whole namespace, and an audit log/Slack notification for every action taken.

- [ ] How do you handle retries and backoff for a flaky external API call (e.g. AWS throttling)?

IF ASKED, SAY THIS
Exponential backoff with jitter, not a fixed delay — fixed delays cause thundering-herd retries that make throttling worse. For AWS specifically, boto3's built-in retry config (adaptive mode) already does this, so the honest answer is often "configure it, don't hand-roll it":

Exponential backoff with jitter, not a fixed delay — fixed delays cause thundering-herd retries that make throttling worse. For AWS specifically, boto3's built-in retry config (adaptive mode) already does this, so the honest answer is often "configure it, don't hand-roll it":

```
import boto3
from botocore.config import Config

config = Config(retries={"max_attempts": 5, "mode": "adaptive"})
client = boto3.client("ec2", config=config)
```

When you do need custom retry logic (a non-AWS API), the pattern is: catch the specific transient exception (not a bare `except`), backoff = base * 2^attempt + random jitter, cap the max delay, and give up after N attempts with a clear error rather than retrying forever.

- [ ] multiprocessing vs threading vs asyncio in Python — when do you use each for an ops script?

IF ASKED, SAY THIS
CPU-bound work (parsing/transforming lots of data) → multiprocessing, because the GIL means threads don't actually parallelize CPU work. I/O-bound work with blocking libraries (most boto3/requests-based scripts) → threading is fine and simpler than async. I/O-bound work at high concurrency (hundreds

CPU-bound work (parsing/transforming lots of data) → multiprocessing, because the GIL means threads don't actually parallelize CPU work. I/O-bound work with blocking libraries (most boto3/requests-based scripts) → threading is fine and simpler than async. I/O-bound work at high concurrency (hundreds of simultaneous API calls, e.g. querying every resource across 50 AWS accounts) → asyncio with an async HTTP client, since it avoids the memory/context-switch overhead of hundreds of OS threads. For a typical "check every EC2 instance across accounts" ops script, threading or a small asyncio + aioboto3 setup are both reasonable; multiprocessing would be overkill and slower to start up.

- [ ] Write a boto3 script to find unattached EBS volumes.

IF ASKED, SAY THIS
Talking point: "available" status means unattached, and pagination matters because describe_volumes silently truncates at 1000 results without it — a real bug source in cost-audit scripts.

```
import boto3

ec2 = boto3.client("ec2")
paginator = ec2.get_paginator("describe_volumes")

orphaned = []
for page in paginator.paginate(
    Filters=[{"Name": "status", "Values": ["available"]}]
):
    for vol in page["Volumes"]:
        orphaned.append({
            "id": vol["VolumeId"],
            "size_gb": vol["Size"],
            "az": vol["AvailabilityZone"],
        })

print(f"Found {len(orphaned)} unattached volumes")
for v in orphaned:
    print(v)
```

Talking point: "available" status means unattached, and pagination matters because `describe_volumes` silently truncates at 1000 results without it — a real bug source in cost-audit scripts.

- [ ] How do you make an automation script safe to run repeatedly (idempotency)?

IF ASKED, SAY THIS
Check current state before acting, and make the action a no-op if the desired state already holds — "create if not exists" rather than "create," "set replica count to 3" rather than "add a replica." For anything destructive, add a dry-run flag by default and require an explicit --yes / --apply to ac

Check current state before acting, and make the action a no-op if the desired state already holds — "create if not exists" rather than "create," "set replica count to 3" rather than "add a replica." For anything destructive, add a dry-run flag by default and require an explicit `--yes` / `--apply` to actually mutate state. Log every action taken with enough context to reconstruct what happened, and design so a script that crashes halfway through and gets re-run doesn't double-apply or corrupt state — this is the same discipline Terraform gives you for free, which is worth mentioning as the "why not just use Terraform for this" answer when it applies.

- [ ] What's actually worth unit testing in infrastructure automation code?

IF ASKED, SAY THIS
The decision logic, not the AWS calls — mock the boto3/kubernetes client and test that your script makes the right call given a given input state (e.g., "given these 3 pods with these restart counts, exactly pod-2 gets deleted"). Testing against real AWS in a unit test is slow, flaky, and costs mone

The decision logic, not the AWS calls — mock the boto3/kubernetes client and test that your script makes the right call given a given input state (e.g., "given these 3 pods with these restart counts, exactly pod-2 gets deleted"). Testing against real AWS in a unit test is slow, flaky, and costs money; that's what a separate, smaller integration test suite against a sandbox account is for. Also worth testing: edge cases like empty results, pagination boundaries, and partial failures (API call 2 of 5 throws — does your script handle that gracefully or leave things half-done?).

- [ ] Bash vs Python — how do you decide?

IF ASKED, SAY THIS
Bash for genuinely simple, linear command-chaining — a handful of CLI calls piped together, run once, throwaway. Python once there's any real logic: conditionals beyond a simple if, error handling you actually care about, data structures (parsing JSON/YAML output rather than grepping text), or anyth

Bash for genuinely simple, linear command-chaining — a handful of CLI calls piped together, run once, throwaway. Python once there's any real logic: conditionals beyond a simple if, error handling you actually care about, data structures (parsing JSON/YAML output rather than grepping text), or anything a teammate will need to read and modify later. The practical tell: if you're about to write a bash script with more than ~30 lines or your third nested if, it should probably be Python.

- [ ] How would you structure a Python CLI tool multiple teammates will use?

IF ASKED, SAY THIS
argparse or click for argument parsing with real --help text (click if you want subcommands, which most real ops tools grow into). Config via a layered approach — CLI flags override environment variables override a config file — so it works both interactively and in CI. Package it properly (a pyproj

`argparse` or `click` for argument parsing with real `--help` text (click if you want subcommands, which most real ops tools grow into). Config via a layered approach — CLI flags override environment variables override a config file — so it works both interactively and in CI. Package it properly (a `pyproject.toml`, installable via pip/pipx) rather than "clone the repo and run the script," so version drift across teammates' laptops isn't a source of bugs. Structured logging (not just print statements) so output is greppable and can feed a pipeline log aggregator later.

- [ ] Give an example of Lambda-based automation for an operational problem, and how it's different from a script you'd just run manually.

IF ASKED, SAY THIS
The shape that matters is event-driven, not scheduled-and-forgotten: a Lambda triggered by an EventBridge rule reacting to a specific event (a CloudTrail-sourced security-group change, an FIS experiment's alarm firing, an S3 upload) rather than a cron job polling on a schedule, since reacting to the

The shape that matters is event-driven, not scheduled-and-forgotten: a Lambda triggered by an EventBridge rule reacting to a specific event (a CloudTrail-sourced security-group change, an FIS experiment's alarm firing, an S3 upload) rather than a cron job polling on a schedule, since reacting to the actual event is both faster and cheaper. A concrete example: a Lambda subscribed to an EventBridge rule matching an EC2 instance-unhealthy state change, which automatically tags the instance for investigation, posts context to Slack, and — if it matches a known-safe pattern — triggers remediation, all within seconds rather than waiting for a human to notice a dashboard. The genuinely different engineering concern versus a manual script: Lambda executions are stateless and can be triggered concurrently, so idempotency isn't optional — EventBridge's at-least-once delivery makes "the same event arrives twice" a real, not theoretical, scenario the automation has to handle safely.

- [ ] How do you honestly frame the productivity impact of automation you've built, without just throwing out an unverifiable percentage?

IF ASKED, SAY THIS
The credible version ties the claim to something concrete and checkable rather than a bare number: "this used to be a 45-minute manual runbook step done by whoever was on-call, now it's a single Lambda invocation that runs in under a minute and doesn't depend on someone remembering the exact steps"

The credible version ties the claim to something concrete and checkable rather than a bare number: "this used to be a 45-minute manual runbook step done by whoever was on-call, now it's a single Lambda invocation that runs in under a minute and doesn't depend on someone remembering the exact steps" is a specific, believable claim an interviewer can follow up on naturally. A bare "this saved 80% of the time" with nothing underneath it invites exactly the follow-up question that's hardest to answer well if the number wasn't something actually tracked. If there genuinely isn't a hard measured number for a piece of automation, it's stronger to describe the qualitative change — eliminated a category of human error, freed the on-call person from a repetitive task during an incident — than to reach for a number that isn't real.

- [ ] What actually causes a Python Lambda cold start, and what's in your control to reduce it?

> **IF ASKED, SAY THIS**
> A cold start is the time to download/mount the code package, initialize the execution environment, and run everything at module scope before your handler is invoked — imports, SDK client construction, config loading. In your control: keep the deployment package small (unused dependencies directly ad

A cold start is the time to download/mount the code package, initialize the execution environment, and run everything at module scope before your handler is invoked — imports, SDK client construction, config loading. In your control: keep the deployment package small (unused dependencies directly add init time), move expensive imports and client construction to module scope so they're reused across warm invocations rather than per-call, and avoid pulling in heavyweight libraries you only use for one code path.

Out of your control but worth knowing: memory allocation also scales CPU, so a function set to 128MB can have a visibly slower cold start than the same code at 1024MB purely from less CPU during init — bumping memory is a legitimate cold-start fix, not just a throughput one. For latency-critical synchronous paths, **provisioned concurrency** keeps a pool of pre-initialized environments warm, sidestepping cold starts entirely at a fixed cost.

- [ ] Zip deployment package, Lambda layers, and container image Lambdas — when do you reach for each?

> **IF ASKED, SAY THIS**
> Plain zip is the default for most functions — simple, fast to deploy, 50MB direct upload / 250MB unzipped limit. Layers pull shared dependencies (a common requests / boto3 -extras/internal-utils package) out of every function's own package, deployed and versioned once, referenced by ARN from many fu

Plain **zip** is the default for most functions — simple, fast to deploy, 50MB direct upload / 250MB unzipped limit. **Layers** pull shared dependencies (a common `requests` / `boto3` -extras/internal-utils package) out of every function's own package, deployed and versioned once, referenced by ARN from many functions — keeps individual function packages small and avoids redundant uploads, but layers are still bound by the same combined 250MB unzipped ceiling.

Container image Lambdas (up to 10GB) matter when dependencies genuinely don't fit zip limits — heavy ML libraries, native binaries, or when the team wants one consistent Docker-based build/test/deploy pipeline across both Lambda and ECS/EKS workloads rather than maintaining two different packaging mechanisms.

- [ ] How does Lambda's error-handling behavior differ between an API Gateway trigger, an S3 event, and an SQS trigger?

> **IF ASKED, SAY THIS**
> This is really three different invocation models wearing the same "trigger" name:

This is really three different invocation models wearing the same "trigger" name:

- **Synchronous** (API Gateway, ALB, direct `Invoke`): caller waits for a response; on error, the caller sees the failure directly — Lambda itself doesn't retry.
- **Asynchronous** (S3, SNS, EventBridge): Lambda queues the event internally and retries automatically (twice, by default, with a delay) before sending to a configured Dead Letter Queue or on-failure destination — the invoking service already got an immediate "accepted" response and moved on.
- **Poll-based** (SQS, Kinesis, DynamoDB Streams): Lambda's own polling infrastructure reads batches and calls your function; for SQS specifically, an unhandled exception fails the *whole batch* by default, meaning even the messages that would've succeeded get retried too — unless you implement **partial batch response** (report specific failed message IDs back) so only the actual failures are retried and re-visible on the queue.

⚠ "Partial batch failure" for SQS is a very commonly asked follow-up — know that it requires both enabling `ReportBatchItemFailures` on the event source mapping *and* returning the right shape (`{"batchItemFailures": [{"itemIdentifier": "..."}]}`) from your handler, or it silently does nothing.

- [ ] Reserved concurrency vs. provisioned concurrency — these get confused constantly. What does each actually control?

> **IF ASKED, SAY THIS**
> Reserved concurrency is a cap, not a warm pool — it sets both the maximum number of concurrent executions a function can use and guarantees that capacity is carved out of the account's shared pool so noisy neighbor functions can't starve it. Set it too low and you get throttling (429s) under load; i

Reserved concurrency is a cap, not a warm pool — it sets both the maximum number of concurrent executions a function can use *and* guarantees that capacity is carved out of the account's shared pool so noisy neighbor functions can't starve it. Set it too low and you get throttling (429s) under load; it's a governance/isolation tool, not a performance one.

Provisioned concurrency is the performance tool — it pre-initializes a set number of execution environments so invocations hit warm code immediately, eliminating cold starts for that slice of traffic, at the cost of paying for that capacity whether it's used or not. The two compose: you might reserve 50 concurrent executions for a function and provision 10 of them to always be warm, letting the other 40 scale normally with cold starts under burst.

- [ ] Why does idempotency matter so much more for Lambda than for a typical always-on service?

> **IF ASKED, SAY THIS**
> Most Lambda event sources (SQS, SNS, S3, Kinesis, EventBridge) offer at-least-once delivery, not exactly-once — a message can and will occasionally be delivered twice (a consumer times out after successfully processing but before deleting from the queue, a retry races a slow success, etc). A handler

Most Lambda event sources (SQS, SNS, S3, Kinesis, EventBridge) offer **at-least-once** delivery, not exactly-once — a message can and will occasionally be delivered twice (a consumer times out after successfully processing but before deleting from the queue, a retry races a slow success, etc). A handler that isn't idempotent — e.g. "charge the customer's card," "insert a row" without a uniqueness constraint — will double-charge or double-insert on that inevitable redelivery.

Standard pattern: derive or accept an idempotency key (a message ID, a client-supplied request ID) and record it — DynamoDB with a conditional write is the common building block, or the **AWS Lambda Powertools idempotency utility** for Python, which wraps a handler and automatically short-circuits repeat invocations with the same key, returning the cached prior result instead of re-executing.

- [ ] What does good observability look like inside a Python Lambda handler, specifically?

IF ASKED, SAY THIS

AWS Lambda Powertools for Python is the practical default rather than hand-rolling this — it gives you structured JSON logging with automatic correlation IDs tied to the request, a @tracer.capture_lambda_handler decorator that wires up X-Ray tracing (including automatic sub-segments around boto3 cal

AWS Lambda Powertools for Python is the practical default rather than hand-rolling this — it gives you structured JSON logging with automatic correlation IDs tied to the request, a `@tracer.capture_lambda_handler` decorator that wires up X-Ray tracing (including automatic sub-segments around `boto3` calls), and custom EMF (embedded metric format) metrics that show up in CloudWatch without a separate PutMetricData call per point.

```
from aws_lambda_powertools import Logger, Tracer, Metrics
from aws_lambda_powertools.metrics import MetricUnit

logger = Logger()
tracer = Tracer()
metrics = Metrics()

@logger.inject_lambda_context
@tracer.capture_lambda_handler
@metrics.log_metrics
def handler(event, context):
    logger.info("processing order", order_id=event["id"])
    metrics.add_metric(name="OrdersProcessed", unit=MetricUnit.Count, value=1)
```

The payoff in a real incident: structured logs let you filter CloudWatch Logs Insights by `order_id` across every function in the trace, and X-Ray shows you exactly which downstream call (a specific DynamoDB table, a specific HTTP dependency) is where the latency actually lives — instead of guessing from wall-clock duration alone.

- [ ] When does a Lambda function actually need to be attached to a VPC — and what's the cost of that?

IF ASKED, SAY THIS

You need VPC attachment when the function must reach something with no public endpoint — an RDS instance in private subnets, an internal ElastiCache cluster, an internal-only service. If everything a function talks to is a public AWS service API (S3, DynamoDB, SQS) or reachable over the internet, ke

You need VPC attachment when the function must reach something with no public endpoint — an RDS instance in private subnets, an internal ElastiCache cluster, an internal-only service. If everything a function talks to is a public AWS service API (S3, DynamoDB, SQS) or reachable over the internet, keep it out of the VPC — it's genuinely simpler and was historically also faster to cold-start.

Cost: VPC-attached Lambdas need ENIs in your subnets, and NAT gateway (or VPC endpoints) if they also need to reach public AWS APIs from inside the VPC — a function that's VPC-attached only for RDS access but also calls S3 needs an S3 VPC endpoint, or its S3 calls silently fail/timeout with no NAT path out. AWS's Hyperplane ENI improvement years ago made the historical "VPC Lambda = terrible cold start" problem mostly a non-issue for new deployments, but it's still one extra layer of networking to get right — a common real interview trap is a Lambda that times out only when VPC-attached, which is almost always a missing NAT gateway or missing VPC endpoint for whatever public API it also needs.

- [ ] When do you reach for Step Functions instead of chaining Lambdas by having one invoke the next?

IF ASKED, SAY THIS

Direct Lambda-invokes-Lambda chaining hides the workflow inside code — no visual state, no built-in retry/backoff per step, and a failure partway through leaves you reconstructing what happened from logs. Step Functions makes the workflow itself the artifact: each state's retry/catch policy is decla

Direct Lambda-invokes-Lambda chaining hides the workflow inside code — no visual state, no built-in retry/backoff per step, and a failure partway through leaves you reconstructing what happened from logs. Step Functions makes the workflow itself the artifact: each state's retry/catch policy is declarative, the execution history shows exactly which step failed and with what input/output, and long-running or human-in-the-loop steps (wait for an approval, wait for a callback token) are natively supported without a Lambda sitting there polling or timing out. Reach for it once you have more than two or three steps with real branching/retry logic, or whenever "what happened during this specific failed run at 3am" needs to be answerable without log spelunking.

- [ ] One malformed message in an SQS queue is causing an entire Lambda-backed pipeline to stall. What's happening and how do you fix it?

IF ASKED, SAY THIS

Classic "poison pill" scenario. Without partial batch response enabled, a single unprocessable message inside a batch fails the whole batch, so it goes back to the queue, gets redelivered, fails again identically — and because it keeps getting re-batched with otherwise-healthy messages, it can drag

Classic "poison pill" scenario. Without partial batch response enabled, a single unprocessable message inside a batch fails the whole batch, so it goes back to the queue, gets redelivered, fails again identically — and because it keeps getting re-batched with otherwise-healthy messages, it can drag good messages down with it on every retry cycle until `maxReceiveCount` is hit.

Fix, in order: (1) enable `ReportBatchItemFailures` so only the actually-failing message ID is retried, not the whole batch; (2) configure a **dead-letter queue** on the source SQS queue with a sane `maxReceiveCount` (e.g. 3–5) so a message that will never succeed stops consuming pipeline capacity and lands somewhere for inspection instead of looping indefinitely; (3) make the handler defensive — validate/parse the message in a try/except and explicitly route unparseable payloads to the DLQ path rather than letting an uncaught exception be the only signal.

- [ ] How do you test Lambda functions before they ever hit AWS?

IF ASKED, SAY THIS
Three layers, and a mature pipeline uses all three rather than picking one: unit tests against the handler function directly — construct the event payload as a plain dict, call the handler, assert on the return value, mocking boto3 calls with moto or unittest.mock so tests run in milliseconds with n

Three layers, and a mature pipeline uses all three rather than picking one: **unit tests** against the handler function directly — construct the event payload as a plain dict, call the handler, assert on the return value, mocking `boto3` calls with `moto` or `unittest.mock` so tests run in milliseconds with no AWS credentials needed at all. **SAM CLI** (`sam local invoke`, `sam local start-api`) runs your actual packaged function inside a Docker container that mimics the real Lambda runtime — closer to production behavior, catches packaging/dependency issues unit tests can't. **LocalStack** emulates a wider slice of AWS (SQS, DynamoDB, S3) locally for integration-style tests of the whole event flow, not just the handler in isolation. CI typically runs the fast unit tests on every commit and reserves SAM/LocalStack integration tests for pre-merge or nightly, since they're slower.

- [ ] When do you reach for Python vs. Bash for an SRE automation task?

IF ASKED, SAY THIS
Bash for quick glue — piping existing CLI tools together, a one-off step inside a CI job, anything under ~20 lines where I'm mostly orchestrating other programs rather than writing logic.

Bash for quick glue — piping existing CLI tools together, a one-off step inside a CI job, anything under ~20 lines where I'm mostly orchestrating other programs rather than writing logic.

Python once I need structured error handling, an SDK (boto3, azure-identity, the Kubernetes client), anything testable, or a script that other people will maintain after me. Past roughly 30–40 lines, Bash's error handling and string manipulation get painful enough that Python is the better investment even for a “simple” task.

- [ ] Write a short Python snippet that checks an HTTP endpoint's health and exits non-zero if it's unhealthy, for use as a CI/CD gate or a cron check.

IF ASKED, SAY THIS
Fails fast, exits non-zero on any problem so CI/cron can key off the exit code, and prints a diagnosable message on failure:

Fails fast, exits non-zero on any problem so CI/cron can key off the exit code, and prints a diagnosable message on failure:

```
import sys, requests

URL = "https://service.internal/healthz"
try:
    r = requests.get(URL, timeout=5)
    r.raise_for_status()
    if r.json().get("status") != "ok":
        raise ValueError(f"unexpected body: {r.text}")
    print("healthy")
except Exception as e:
    print(f"UNHEALTHY: {e}", file=sys.stderr)
    sys.exit(1)
```

- [ ] How do you handle secrets and config in a Python automation script that runs inside CI/CD?

IF ASKED, SAY THIS
Never hardcoded. In CI, secrets come in as environment variables injected by the pipeline's own secret store, or the script fetches them at runtime via the cloud SDK using the pipeline's OIDC-federated role (no static keys sitting in the pipeline config either).

Never hardcoded. In CI, secrets come in as environment variables injected by the pipeline's own secret store, or the script fetches them at runtime via the cloud SDK using the pipeline's OIDC-federated role (no static keys sitting in the pipeline config either).

`python-dotenv` is fine for local development only, and `.env` is always in `.gitignore`, never committed. At startup I validate every required variable is present and fail immediately with a clear message naming what's missing, rather than failing confusingly three steps later.

- [ ] Explain Python virtual environments and dependency pinning — why do they matter for reproducible automation?

IF ASKED, SAY THIS
A venv (or Poetry environment) isolates a project's dependencies from the system Python and from every other project, so upgrading one script's libraries doesn't silently break another. Pinning versions — a requirements.txt with exact versions, or a lockfile — means the script behaves identically on

A venv (or Poetry environment) isolates a project's dependencies from the system Python and from every other project, so upgrading one script's libraries doesn't silently break another. Pinning versions — a `requirements.txt` with exact versions, or a lockfile — means the script behaves identically on my laptop, the CI runner, and the production cron box, instead of drifting whenever someone's environment happens to pick up a newer library.

For anything that runs unattended in production, I bake the pinned dependencies into the container image itself rather than installing at runtime, so the artifact that gets tested is exactly the artifact that runs.

- [ ] Write a Python function using boto3 to list EC2 instances missing a required tag, for a compliance-automation script.

IF ASKED, SAY THIS
The Azure equivalent uses the azure-mgmt-compute SDK's virtual_machines.list_all() and checks vm.tags the same way — same pattern, different client.

```
import boto3

def find_untagged_instances(required_tag="Owner", region="ap-south-1"):
    ec2 = boto3.client("ec2", region_name=region)
    paginator = ec2.get_paginator("describe_instances")
    offenders = []
    for page in paginator.paginate():
        for reservation in page["Reservations"]:
            for instance in reservation["Instances"]:
                tags = {t["Key"]: t["Value"] for t in instance.get("Tags", [])}
                if required_tag not in tags:
                    offenders.append(instance["InstanceId"])
    return offenders
```

The Azure equivalent uses the `azure-mgmt-compute` SDK's `virtual_machines.list_all()` and checks `vm.tags` the same way — same pattern, different client.

- [ ] How would you write a Python script to safely restart a Kubernetes deployment's pods one at a time, using the kubernetes client library?

IF ASKED, SAY THIS
The cleanest way is to patch the deployment's pod template with a rollout-restart annotation — the same thing kubectl rollout restart does — and let the existing rolling-update strategy (maxSurge/maxUnavailable) handle the one-at-a-time part safely:

The cleanest way is to patch the deployment's pod template with a rollout-restart annotation — the same thing `kubectl rollout restart` does — and let the existing rolling-update strategy (maxSurge/maxUnavailable) handle the one-at-a-time part safely:

```
from kubernetes import client, config
from datetime import datetime, timezone

config.load_kube_config()
apps = client.AppsV1Api()

def rollout_restart(name, namespace="default"):
    now = datetime.now(timezone.utc).isoformat()
    body = {"spec": {"template": {"metadata": {"annotations": {
        "kubectl.kubernetes.io/restartedAt": now
    }}}}}
    apps.patch_namespaced_deployment(name, namespace, body)
```

I'd avoid deleting pods directly in a loop — it bypasses the Deployment controller's own pacing and readiness checks, which is exactly the safety net you want during a bounce.

- [ ] What's your approach to logging and error handling in scripts that run unattended, so failures are diagnosable later?

IF ASKED, SAY THIS

Structured logging — JSON lines with a timestamp, level, and context — not bare print() . I catch specific exceptions, not a bare except: , and log the full stack trace on anything unexpected rather than swallowing it. Every run emits one unambiguous final line (“SUCCESS” or “FAILED: reason”) that a

Structured logging — JSON lines with a timestamp, level, and context — not bare `print()`. I catch specific exceptions, not a bare `except:`, and log the full stack trace on anything unexpected rather than swallowing it. Every run emits one unambiguous final line (“SUCCESS” or “FAILED: reason”) that an alerting system can grep for without parsing the whole log.

And logs go to the same central pipeline as any service — stdout captured by Fluent Bit/Vector into Loki or ELK — never a local file on a box that nobody's looking at when it actually fails.

- [ ] Explain Python's GIL and threading vs. multiprocessing — when does this actually matter for an SRE script?

IF ASKED, SAY THIS
The GIL means only one thread executes Python bytecode at a time, so threading doesn't parallelize CPU-bound work — for that you need multiprocessing or a subprocess pool to get real parallel cores.

The GIL means only one thread executes Python bytecode at a time, so `threading` doesn't parallelize CPU-bound work — for that you need `multiprocessing` or a subprocess pool to get real parallel cores.

For I/O-bound automation — firing API calls at 50 hosts in parallel, or SSH-ing to a fleet — threads are still the right, lighter-weight tool, because a thread releases the GIL while it's waiting on network I/O. I've hit this distinction directly writing fan-out scripts against cloud APIs: threading for the parallel API calls, never reaching for multiprocessing unless there's actual CPU-bound work like parsing or computation in the loop.

- [ ] How do you structure and test a Python CLI automation tool so teammates can safely run and extend it?

IF ASKED, SAY THIS
argparse or click for a real CLI surface, with --dry-run as a standard flag on anything that mutates state — that one convention prevents most “accidentally ran it in prod” incidents. I separate pure logic from I/O and side effects so the logic is unit-testable with pytest without needing live AWS/A

`argparse` or `click` for a real CLI surface, with `--dry-run` as a standard flag on anything that mutates state — that one convention prevents most “accidentally ran it in prod” incidents. I separate pure logic from I/O and side effects so the logic is unit-testable with pytest without needing live AWS/Azure/K8s credentials in CI.

Exit codes mean something specific (0 success, 1 general failure, 2 bad usage) so the tool composes cleanly into a pipeline, and a short README with real usage examples — not just `--help` output — is what actually gets someone else using it instead of rewriting their own version.

- [ ] Describe a script or tool you've built that reduced manual/toil work — what did it do, and what was the measured impact?

IF ASKED, SAY THIS
Two concrete examples from recent work: a Python script that reports pod restart counts across namespaces — replacing a manual kubectl sweep whenever instability was suspected — and a bash cleanup script for stale Docker images/containers that used to require someone noticing a build host was low on

Two concrete examples from recent work: a Python script that reports pod restart counts across namespaces — replacing a manual `kubectl` sweep whenever instability was suspected — and a bash cleanup script for stale Docker images/containers that used to require someone noticing a build host was low on disk before acting.

Both are small pieces of a larger pattern of automation-driven toil reduction that contributed to a measured ~60% cut in manual deployment effort in my recent roles; I walk through the fuller story with numbers in the Behavioral section.

- [ ] Outline a Python script that queries Prometheus and flags SLO breaches.

IF ASKED, SAY THIS
In practice this would run on a schedule (cron, or as a lightweight sidecar), and alert() would call PagerDuty or a Slack webhook rather than just printing.

```
import requests

PROM_URL = "http://prometheus:9090/api/v1/query"
QUERY = (
    'sum(rate(http_requests_total{status=~"5.."}[5m])) '
    '/ sum(rate(http_requests_total[5m]))'
)
SLO_THRESHOLD = 0.001  # 99.9% target

def check_slo():
    resp = requests.get(PROM_URL, params={"query": QUERY}, timeout=10)
    resp.raise_for_status()
    result = resp.json()["data"]["result"]
    if not result:
        return
    error_ratio = float(result[0]["value"][1])
```

```
    if error_ratio > SLO_THRESHOLD:
        alert(f"SLO breach: error ratio {error_ratio:.4%} > {SLO_THRESHOLD:.4%}")

def alert(message):
    print(message)  # replace with a webhook/paging call

if __name__ == "__main__":
    check_slo()
```

In practice this would run on a schedule (cron, or as a lightweight sidecar), and `alert()` would call PagerDuty or a Slack webhook rather than just printing.

- [ ] Write a Python snippet to call a REST API and handle failure cases explicitly.

IF ASKED, SAY THIS
Always set a timeout, check status code before parsing the body as JSON, and handle timeout/connection errors as distinct cases rather than a bare except.

Always set a timeout, check status code before parsing the body as JSON, and handle timeout/connection errors as distinct cases rather than a bare except.

```
import requests
try:
    resp = requests.get('https://api.example.com/health', timeout=5)
    if resp.status_code == 200:
        data = resp.json()
    else:
        print(f"Unexpected status: {resp.status_code}")
except requests.exceptions.Timeout:
    print("Request timed out")
except requests.exceptions.RequestException as e:
    print(f"Request failed: {e}")
```

- [ ] What would you actually use Boto3/Python for in this kind of platform-engineering role, versus relying purely on IaC (Terraform/CloudFormation)?

IF ASKED, SAY THIS
IaC for declaring the desired state of infrastructure; Python/Boto3 for genuinely operational logic — automating a multi-step operational runbook, cleaning up stale resources matching a pattern IaC tools aren't built to express, building custom tooling (a Lambda enforcing a tagging policy by remediating non-compliant resources it discover

IaC for declaring the desired state of infrastructure; **Python/Boto3** for genuinely operational logic — automating a multi-step operational runbook, cleaning up stale resources matching a pattern IaC tools aren't built to express, building custom tooling (a Lambda enforcing a tagging policy by remediating non-compliant resources it discovers), or gluing AWS API calls together in a workflow too dynamic/conditional for a static IaC definition.

- [ ] How would you use Python/Boto3 to build a governance or compliance-checking tool across a multi-account environment?

IF ASKED, SAY THIS
A scheduled Lambda (or Step Functions for something multi-stage) using Boto3 with a cross-account role assumed into each member account, checking specific compliance conditions (unencrypted S3 buckets, security groups open on sensitive ports, IAM users without MFA), aggregating findings into a report or feeding Security Hub/a dashboard —

A scheduled Lambda (or Step Functions for something multi-stage) using Boto3 with a cross-account role assumed into each member account, checking specific compliance conditions (unencrypted S3 buckets, security groups open on sensitive ports, IAM users without MFA), aggregating findings into a report or feeding Security Hub/a dashboard — optionally auto-remediating lower-risk findings while flagging higher-risk ones for human review rather than blindly auto-fixing everything.

- [ ] What's your approach to writing Boto3 automation that's actually safe to run against production — idempotency, error handling, dry-run capability?

IF ASKED, SAY THIS
Idempotent by design where possible (check current state before acting, so re-running twice doesn't cause a problem), explicit error handling around API calls rather than an unhandled exception leaving things half-completed (with retry/backoff for AWS API throttling specifically — a real, common failure mode at scale), and a --dry-run mod

Idempotent by design where possible (check current state before acting, so re-running twice doesn't cause a problem), explicit error handling around API calls rather than an unhandled exception leaving things half-completed (with retry/backoff for AWS API throttling specifically — a real, common

failure mode at scale), and a `--dry-run` mode logging what it would do without calling the mutating API — genuinely useful for validating logic against production data before trusting it to act.

- [ ] Bash vs. Python vs. PowerShell — when would you reach for each in this role?

> **IF ASKED, SAY THIS**
> Bash for quick, simple operational tasks and gluing CLI tools where you don't need real data structures — degrades fast past a certain complexity. Python for anything with real logic, needing Boto3 directly, structured data, or that you expect to maintain over time — the default for anything beyond a trivial script. PowerShell specificall

Bash for quick, simple operational tasks and gluing CLI tools where you don't need real data structures — degrades fast past a certain complexity.
Python for anything with real logic, needing Boto3 directly, structured data, or that you expect to maintain over time — the default for anything beyond a trivial script.

PowerShell specifically for Windows-based infrastructure or PowerShell-native client environments (Azure-heavy enterprises especially) — worth working fluency given the JD lists it explicitly, even if Python is the default for most platform automation.

AI/LLM Ops

- [ ] How can AI/LLMs actually be used in SRE work? (The question you should expect to open this topic.)

IF ASKED, SAY THIS
AI/LLMs help across incident management, observability, troubleshooting, automation, RCA, and operational knowledge -- as an assistant that helps engineers respond faster and decide better, not a replacement. Judgment, architecture thinking, stakeholder communication, and risk assessment stay human.

Leading with "assistant, not replacement" then immediately naming 3-4 concrete use cases (not just the concept) is what separates a real answer from a buzzword answer here -- the concrete use-case cards below are exactly that ammunition.

- [ ] How would you use AI to speed up log-based troubleshooting during an incident?

IF ASKED, SAY THIS
Feed the relevant log window to an LLM and ask it to identify the probable root cause -- it detects patterns, summarizes errors, and highlights anomalies across thousands of lines far faster than manual grep/Splunk searching. A real example: raw logs showing scattered DB timeout errors get summarized as "connection pool exhausted, correlates with a deployment 10 minutes earlier that changed pool size" -- turning a 40-minute manual dig into a 5-minute triage.

PRACTICAL NOTE
Investigation time on log-heavy incidents commonly drops from ~40 minutes to ~5 -- but the summary is a hypothesis to verify, not a verdict to act on blind.

- [ ] How would you use AI to help write a postmortem/RCA?

IF ASKED, SAY THIS
Feed the incident timeline (timestamps plus what happened at each) to an LLM and have it draft the Summary / Impact / Root Cause / Corrective Actions sections -- the engineer reviews, corrects, and publishes rather than writing from a blank page during an already-exhausting week.

PRACTICAL NOTE
This mainly saves the drafting time, not the thinking time -- the engineer still owns verifying the root cause is actually correct before it goes out.

- [ ] How can AI help with alert fatigue specifically?

IF ASKED, SAY THIS
Correlate simultaneous alerts (CPU + memory + database + network firing together) and identify they share one root cause, collapsing what would be 100 separate pages into one ticket pointing at the actual failure -- shifting engineer attention from "triage the flood" to "fix the one thing."

PRACTICAL NOTE
This is the single highest-leverage AI use case for MTTA specifically, since most alert fatigue is duplicate signal from one underlying failure, not 100 independent problems.

- [ ] How would AI make a huge, rarely-read runbook actually useful during a P1?

IF ASKED, SAY THIS
Instead of a junior on-call engineer searching a 500-page runbook under pressure, they ask an LLM "why is the payments service down" and get a contextualized next-steps sequence (check pod status, check DB connectivity, check consumer lag, check recent deploys) pulled from the runbook content -- turning static documentation into an interactive guide.

PRACTICAL NOTE
The real win is junior/on-call engineers performing closer to senior-level under pressure, since the AI compresses "where in this huge doc do I even look" into a direct answer.

- [ ] How would you use AI to speed up reading a wall of kubectl describe/logs output?

IF ASKED, SAY THIS
Paste the raw output to an LLM and ask for the root cause -- it condenses a page of kubectl describe events into "Root cause: image pull failure, wrong tag, fix: update the image tag in the deployment" instead of the engineer manually scanning for the one relevant line.

PRACTICAL NOTE

Genuinely fast for well-known failure signatures (ImagePullBackOff, OOMKilled, CrashLoopBackOff); less reliable for a truly novel failure mode the model hasn't effectively seen the pattern for.

- [ ] Can AI shift incident response from reactive to proactive?

IF ASKED, SAY THIS

Monitoring trend lines (steadily climbing CPU plus memory growth plus rising consumer lag) together can flag "this combination predicts an outage in ~2 hours" before any single threshold alert would fire -- catching the compound signal a single-metric alert rule misses.

PRACTICAL NOTE

This is a genuinely different capability from traditional threshold alerting, not just a faster version of it -- it's pattern-matching across multiple simultaneously-trending signals.

- [ ] Can AI replace SREs? What is your view?

IF ASKED, SAY THIS

Answer (use this structure): No. AI can automate repetitive operational tasks and accelerate troubleshooting, but production environments require: Engineering judgment: Deciding whether to rollback vs patch forward during a P1. AI cannot make this call because it does not know the business context. Risk assessment: Eva

Answer (use this structure):

No. AI can automate repetitive operational tasks and accelerate troubleshooting, but production environments require:

Engineering judgment: Deciding whether to rollback vs patch forward during a P1. AI cannot make this call because it does not know the business context.

Risk assessment: Evaluating whether an emergency change during market hours in a financial environment is acceptable. This requires understanding regulatory requirements, customer contracts, and business impact.

Architecture decisions: Choosing between RDS Multi-AZ vs Aurora Global for a specific workload. AI can suggest options but the SRE must evaluate trade-offs against the specific system.

Incident leadership: Managing a P1 bridge call with 20 engineers and 10 business stakeholders. Communication, coordination, prioritisation under pressure is a human skill.

Stakeholder communication: Explaining a 23-minute payment outage to the CFO in plain English during quarter-end. Empathy and business context matter.

The right framing: AI acts as a force multiplier. A team of 5 SREs with AI operates like a team of 15. It does not reduce the need for experienced SREs — it raises the ceiling on what each SRE can accomplish.

- [ ] How would you use AI in a P1 incident right now based on your background?

IF ASKED, SAY THIS

Practical answer using your actual tools (Splunk, Dynatrace, Prometheus, Grafana, K8s, Terraform): "If a P1 incident occurs and users report slowness, here is exactly what I would do with AI: Step 1 — Collect data: Pull logs from Splunk for the last 30 minutes. Export Dynatrace trace data for affected transactions. Exp

Practical answer using your actual tools (Splunk, Dynatrace, Prometheus, Grafana, K8s, Terraform):

"If a P1 incident occurs and users report slowness, here is exactly what I would do with AI:

Step 1 — Collect data: Pull logs from Splunk for the last 30 minutes. Export Dynatrace trace data for affected transactions. Export Grafana metrics for error rate and P99 latency.

Step 2 — AI analysis: Paste the collected logs and metrics into Claude or ChatGPT with this prompt: 'I have a production payments service experiencing high latency. Here are the logs and metrics from the last 30 minutes. Identify the most likely root cause and suggest diagnostic steps.'

Step 3 — AI output: AI identifies that response times increased immediately after a specific deployment and that database connection pool metrics show exhaustion. It suggests checking DB connection limits and recent Terraform changes.

Step 4 — SRE validates: I confirm the finding in Dynatrace traces, identify the specific deployment, and rollback via Harness.

Step 5 — RCA: I paste the full incident timeline into AI to generate an RCA first draft, which I review and publish.

Result: MTTR improved from 40 minutes to 12 minutes for this class of incident."

- [ ] What are the risks of using AI in SRE and how do you mitigate them?

IF ASKED, SAY THIS

Risk 1 — AI hallucinations producing wrong commands: AI can confidently suggest a Kubernetes command that looks right but deletes the wrong namespace. Mitigation: NEVER allow AI to execute commands directly in production. Always human-review AI-generated commands before running. Treat AI output like untrusted code — re

Risk 1 — AI hallucinations producing wrong commands:
AI can confidently suggest a Kubernetes command that looks right but deletes the wrong namespace. Mitigation: NEVER allow AI to execute commands directly in production. Always human-review AI-generated commands before running. Treat AI output like untrusted code — review before executing.

Risk 2 — Sensitive data leakage:
Pasting production logs with PII (customer names, payment details) into a public LLM sends that data to a third party. Mitigation: Use private deployments (Azure OpenAI, AWS Bedrock, or self-hosted LLaMA). Scrub PII from logs before sending to any AI. Establish a company policy on what data can be shared with AI tools.

Risk 3 — Over-reliance reducing skill development:
Junior engineers who only use AI to debug may never develop the deep Linux or Kubernetes knowledge needed for novel failure modes. Mitigation: Use AI as a learning tool — ask it to explain the reasoning, not just give the answer. Ensure junior engineers understand WHY a fix works.

Risk 4 — Autonomous AI making production changes:
An AI agent with kubectl access could delete pods, scale down services, or apply incorrect manifests. Mitigation: AI should SUGGEST, humans should APPROVE and EXECUTE in production. Zero autonomous AI actions on production systems without human review gate.

- [ ] Which AI tools have you used or would you use for SRE work?

IF ASKED, SAY THIS
Log analysis and Q&A: Claude (Anthropic) or ChatGPT-4: paste log output, get analysis and recommendations. Most SREs already use these informally. Code and IaC generation: GitHub Copilot: in-IDE generation of Terraform, Python scripts, Jenkinsfiles. Significantly faster than writing from scratch. Observability AI featu

Log analysis and Q&A:
Claude (Anthropic) or ChatGPT-4: paste log output, get analysis and recommendations. Most SREs already use these informally.

Code and IaC generation:
GitHub Copilot: in-IDE generation of Terraform, Python scripts, Jenkinsfiles. Significantly faster than writing from scratch.

Observability AI features (built into existing tools):
Dynatrace Davis AI: automatic anomaly detection and root cause analysis. Already integrated into the monitoring stack.
Splunk AI/ML Toolkit: anomaly detection on log data without requiring manual SPL queries.
New Relic AI: AI-powered alert correlation and incident analysis built into the APM platform.
Grafana Sift: ML-driven root cause investigation across metrics, logs, and traces.

Kubernetes AI assistance:
K8sGPT: open-source tool that runs kubectl commands, analyses the output, and explains issues in plain English. Run: 'k8sgpt analyse' and get: 'Pod payments-abc-123 is in CrashLoopBackOff due to missing environment variable DB_PASSWORD.'

Private enterprise AI:
Azure OpenAI or AWS Bedrock: enterprise-grade LLM access where your data stays within your cloud environment. Compliant for financial services where public AI tools are restricted.

- [ ] From an infra/platform perspective, what's actually different about running an LLM-backed or agentic service in production vs. a normal microservice?

IF ASKED, SAY THIS
Three things break traditional APM assumptions. Non-determinism — the same input can produce a different output, so you can't reproduce a bug just by replaying the request; you need to have captured the exact prompt, model version, and parameters at call time. Cost model — latency and cost track token count, not reques

Three things break traditional APM assumptions. Non-determinism — the same input can produce a different output, so you can't reproduce a bug just by replaying the request; you need to have captured the exact prompt, model version, and parameters at call time. Cost model — latency and cost track token count, not request count, so a "slow" request and an "expensive" request are different signals that both need watching. And reliability itself is a different concept: for an agent, "reliable" isn't just up/fast, it includes decision quality, task-completion fidelity, and knowing when to stop and hand off to a human — none of which a standard uptime/latency SLO captures.

- [ ] How would you think about SLOs for an AI agent versus a traditional API service?

IF ASKED, SAY THIS
Keep the infrastructure-layer SLOs you'd always have — availability, latency, error rate on the serving layer itself — but add agent-specific ones: task completion rate (did it actually finish what it was asked, correctly), cost-per-task or cost-per-successful-task (not just cost-per-request), and some proxy for output

Keep the infrastructure-layer SLOs you'd always have — availability, latency, error rate on the serving layer itself — but add agent-specific ones: task completion rate (did it actually finish what it was asked, correctly), cost-per-task or cost-per-successful-task (not just cost-per-request), and some proxy for output quality (an eval score, a human-review sample rate, or a downstream-correction rate). The error budget framing still works, it just burns on more than one axis at once, and the newest thinking treats a rising cost-per-task as a leading indicator worth its own budget — it tends to show degradation a day or two before completion-rate or quality metrics visibly move.

- [ ] What's "agentic SRE," and how is it different from just using AI tools to help with SRE work?

IF ASKED, SAY THIS
Worth being precise here because the term gets used both ways. "AI-assisted SRE" (AIOps evolving with LLMs) is using AI as a tool for existing SRE work — correlating alerts, drafting root-cause hypotheses, summarizing an incident timeline, with a human still deciding and acting. "Agentic SRE" goes further: an agent tha

Worth being precise here because the term gets used both ways. "AI-assisted SRE" (AIOps evolving with LLMs) is using AI as a tool for existing SRE work — correlating alerts, drafting root-cause hypotheses, summarizing an incident timeline, with a human still deciding and acting. "Agentic SRE" goes further: an agent that's actually allowed to take remediation actions autonomously, closing the loop itself. The two production patterns for the latter are a state-machine approach (formalize your failure taxonomy into explicit states/transitions — strong safety guarantees, but only works where failure modes are well-understood) and a closed-loop approach (continuous observe-decide-act cycle, more flexible with ambiguity, but needs stronger runtime guardrails since it's less predictable). The honest, defensible answer in an interview is that governance — being able to explain after the fact what the agent did and why, and being able to stop it before it does the wrong thing — is what actually determines whether either pattern is safe to run, more than which pattern you pick.

- [ ] What would you monitor for cost/token usage in an LLM-based service?

IF ASKED, SAY THIS
Input, output, and (for reasoning models) reasoning token counts per request, broken down by model/endpoint so you can see which call sites are expensive. Cost per task rather than cost per request — a task might legitimately take multiple LLM calls, and per-request cost alone hides a runaway multi-step loop. The genui

Input, output, and (for reasoning models) reasoning token counts per request, broken down by model/endpoint so you can see which call sites are expensive. Cost per task rather than cost per request — a task might legitimately take multiple LLM calls, and per-request cost alone hides a runaway multi-step loop. The genuinely useful signal is trend, not snapshot: a rising token-cost-per-task with stable completion rate is a real leading indicator — the current thinking is it tends to show up roughly a day or two before output quality visibly degrades, because the model or agent is "working harder" to get the same result. That's worth alerting on as its own thing, not just as a cost-control line item.

- [ ] If an AI agent is allowed to take automated remediation actions in production, what guardrails would you put around it?

IF ASKED, SAY THIS
Hard action allowlist (only pre-approved, reversible actions — restart, scale, roll back a specific known-safe deploy — never "run arbitrary command"), a rate/blast-radius cap (max N actions per time window, max % of fleet touched at once), and a hard rule against stacking remediation on top of an already-unstable stat

Hard action allowlist (only pre-approved, reversible actions — restart, scale, roll back a specific known-safe deploy — never "run arbitrary command"), a rate/blast-radius cap (max N actions per time window, max % of fleet touched at once), and a hard rule against stacking remediation on top of an already-unstable state — don't let the agent take a new action while an incident is actively in progress, since that's exactly when compounding a bad decision does the most damage. Every action needs to be logged with the reasoning behind it, not just the action itself, so a human can audit "why did it do that" after the fact — that auditability is what actually determines whether this is safe to run, more than the sophistication of the remediation logic itself.

- [ ] What is MCP (Model Context Protocol), and where does it fit if you're running agent/tool-calling workloads?

IF ASKED, SAY THIS
MCP is a standardized protocol for connecting an LLM/agent to external tools and data sources — instead of every agent framework inventing its own bespoke way to expose "here are the tools/functions this agent can call," MCP gives a common interface so a tool server can be written once and used by any MCP-compatible ag

MCP is a standardized protocol for connecting an LLM/agent to external tools and data sources — instead of every agent framework inventing its own bespoke way to expose "here are the tools/functions this agent can call," MCP gives a common interface so a tool server can be written once and used by any MCP-compatible agent client. From a platform standpoint, that turns "which tools can our internal agents use" into an integration/governance question rather than a one-off code problem per agent — you stand up MCP servers for the systems you want to expose (internal APIs, databases, ticketing) and control access/permissions at that layer. It's a genuinely current, still-evolving area — reasonable to be upfront in an interview that your depth here is conceptual/familiarity-level unless you've actually stood one up.

- [ ] How would you architect observability for a multi-agent system where one agent calls another?

IF ASKED, SAY THIS
The goal is one coherent trace tree per top-level request, not a pile of disconnected spans per agent. That requires explicit trace-context propagation across the agent-to-subagent boundary — most agent frameworks don't do this automatically the way HTTP middleware does, so it has to be wired in deliberately wherever a

The goal is one coherent trace tree per top-level request, not a pile of disconnected spans per agent. That requires explicit trace-context propagation across the agent-to-subagent boundary — most agent frameworks don't do this automatically the way HTTP middleware does, so it has to be wired in deliberately wherever a task gets delegated. Within that, each agent hop, each tool call, and each underlying LLM invocation becomes its own child span with token/cost/latency attributes, so when something goes wrong you can see exactly which agent in the chain made the expensive or wrong call instead of only seeing the aggregate outcome.

- [ ] What data governance/security concerns come with running LLM platforms in a regulated environment?

IF ASKED, SAY THIS
PII/sensitive data ending up in prompts (and therefore potentially in provider logs, telemetry, or training data depending on the provider's data policy) is the big one — same discipline as any other data-classification problem, but easy to miss because a prompt feels like ephemeral text rather than a data flow needing

PII/sensitive data ending up in prompts (and therefore potentially in provider logs, telemetry, or training data depending on the provider's data policy) is the big one — same discipline as any other data-classification problem, but easy to miss because a prompt feels like ephemeral text rather than a data flow needing review. Prompt injection is the newer concern specific to this space — if an agent ingests untrusted content (a webpage, an email, a document) as part of its context, that content can contain instructions trying to hijack the agent's behavior, which is a real attack surface with no fully solved answer yet, only mitigations (sandboxing what actions an agent can take regardless of what it's told, treating tool outputs as untrusted input). Data residency and which region/provider actually processes the request matters the same way it does for any other third-party API call in a regulated environment.

- [ ] Walk through MCP's architecture — host, client, server, and the transport options.

IF ASKED, SAY THIS
Three roles: the host is the application the user actually interacts with (an IDE, an agent runtime); the client lives inside the host and manages a 1:1 connection to a specific server; the server exposes tools, resources, and prompts over that connection, implemented independently of any particular host. A host can ru

Three roles: the host is the application the user actually interacts with (an IDE, an agent runtime); the client lives inside the host and manages a 1:1 connection to a specific server; the server exposes tools, resources, and prompts over that connection, implemented independently of any particular host. A host can run multiple clients simultaneously, each talking to a different server, which is what lets one agent draw on several independent tool providers at once. Transport-wise, stdio is the standard choice for a server running locally on the same machine as the host (simple, no network exposure), while HTTP-based transports (Streamable HTTP, and the older HTTP+SSE) are for a server running remotely — which is the shape that matters for a platform role, since exposing an internal service (Langfuse, CloudWatch) as an MCP server to shared agents means running it as a proper remote, authenticated service, not a local process.

- [ ] How would you design and secure an MCP server that exposes internal observability data to an agent?

IF ASKED, SAY THIS
Same platform-engineering instincts as exposing any internal API, applied to a new interface shape: authenticate the connection (don't run an unauthenticated remote MCP server), scope what tools it exposes tightly to read-only observability queries rather than a broad "run any query" capability, and rate-limit/log ever

Same platform-engineering instincts as exposing any internal API, applied to a new interface shape: authenticate the connection (don't run an unauthenticated remote MCP server), scope what tools it exposes tightly to read-only observability queries rather than a broad "run any query" capability, and rate-limit/log every tool invocation the same way you'd log any other API call — since from an audit perspective, "an agent queried production CloudWatch data" is exactly as auditable an event as "a person did." The specific new risk MCP introduces versus a normal internal API: tool descriptions and outputs become part of the agent's context, so a compromised or badly-designed server could try to inject instructions into that context — treat what a tool returns as untrusted input to the agent, the same way you'd treat any other external, potentially-adversarial data source.

- [ ] Compare multi-agent orchestration frameworks — LangGraph, CrewAI, AutoGen/Semantic Kernel — at a conceptual level.

IF ASKED, SAY THIS
The core difference is how much structure they impose on coordination. LangGraph models the whole multi-agent flow as an explicit state graph — nodes and edges you define, with full control over routing, conditionals, and loops — the most control, the most upfront design work. CrewAI leans toward a role-based abstracti

The core difference is how much structure they impose on coordination. LangGraph models the whole multi-agent flow as an explicit state graph — nodes and edges you define, with full control over routing, conditionals, and loops — the most control, the most upfront design work. CrewAI leans toward a role-based abstraction (define agents with roles/goals/tools, assign them tasks, let a process type like sequential or hierarchical handle coordination) — faster to stand something up, less fine-grained control. AutoGen and Semantic Kernel lean toward conversational, message-passing coordination between agents, closer to how a human team might hand work back and forth. None of these is objectively best — it's a real fit question, and the honest answer given this JD's "familiarity" bar is naming the tradeoff correctly rather than claiming deep production experience with all of them.

- [ ] What's a supervisor/orchestrator pattern in multi-agent systems, and why do most production designs end up needing one?

IF ASKED, SAY THIS
Instead of peer agents freely deciding amongst themselves who does what next, a supervisor agent owns the routing decision — it receives the task, decides which specialist agent (or tool) handles the next step, and evaluates whether the result is good enough to move on or needs another pass. Fully decentralized peer-to

Instead of peer agents freely deciding amongst themselves who does what next, a supervisor agent owns the routing decision — it receives the task, decides which specialist agent (or tool) handles the next step, and evaluates whether the result is good enough to move on or needs another pass. Fully decentralized peer-to-peer agent coordination sounds appealing but tends to break down in practice — no clear point of accountability when something goes wrong, harder to bound cost/scope (nothing is centrally deciding "we've done enough"), and much harder to observe/trace cleanly. A supervisor pattern gives you a natural place to enforce the same guardrails discussed elsewhere in this JD — budget caps, action allowlists, a single point where a human-in-the-loop check can be inserted — which is exactly why most production multi-agent systems converge on some version of it even when the initial design was fully decentralized.

- [ ] If an MCP tool call returns malicious or misleading content, what's actually at risk, and how do you defend against it?

IF ASKED, SAY THIS

What's at risk isn't the tool call itself — it's that the tool's output becomes part of the agent's context for its next decision, so misleading or injected content in a tool result can steer the agent's subsequent actions (a classic indirect prompt-injection path, just via a tool response instead of a webpage or docum

What's at risk isn't the tool call itself — it's that the tool's output becomes part of the agent's context for its next decision, so misleading or injected content in a tool result can steer the agent's subsequent actions (a classic indirect prompt-injection path, just via a tool response instead of a webpage or document). Defense is layered, not a single fix: treat every tool output as untrusted data rather than trusted instructions (don't let content returned by a tool silently expand what the agent is willing to do next), constrain what actions are actually reachable regardless of what the agent is told to do (the allowlist/guardrail pattern again), and log tool inputs/outputs so a compromised or misbehaving tool is something you can actually detect and trace after the fact rather than something invisible.

- [ ] If asked directly whether you've worked hands-on with agentic AI platforms in production, how do you frame that honestly?

IF ASKED, SAY THIS

Given the JD lists this as "familiarity," not a required, deep-hands-on bar, the strongest answer is calibrated honesty rather than overclaiming: name what you do have — solid observability/SRE fundamentals that transfer directly (SLOs, tracing, incident response), and genuine conceptual grounding in what's different a

Given the JD lists this as "familiarity," not a required, deep-hands-on bar, the strongest answer is calibrated honesty rather than overclaiming: name what you do have — solid observability/SRE fundamentals that transfer directly (SLOs, tracing, incident response), and genuine conceptual grounding in what's different about operating these systems — and be direct that deep production hands-on with an agentic platform specifically is the part you'd be ramping into. That's a much stronger answer than fumbling a follow-up question on something you claimed deeper experience with than you have — the same "here's exactly where my depth is and isn't" framing works better than pretending the gap doesn't exist.

- [ ] What is Langfuse, and how is it different from pointing a general-purpose APM tool like Datadog at an LLM application?

IF ASKED, SAY THIS

A general APM tool sees an LLM call as just another opaque HTTP request — duration and status code, nothing more. Langfuse is purpose-built for LLM applications: it natively understands token usage, model parameters, prompt/completion pairs, and evaluation scores as first-class concepts, not something you'd have to bol

A general APM tool sees an LLM call as just another opaque HTTP request — duration and status code, nothing more. Langfuse is purpose-built for LLM applications: it natively understands token usage, model parameters, prompt/completion pairs, and evaluation scores as first-class concepts, not something you'd have to bolt on with custom attributes. It bundles four things a generic APM tool doesn't: tracing built around LLM/agent call structure, prompt management with versioning, an evaluation layer (LLM-as-a-judge, human annotation, datasets), and cost/quality dashboards — open source, and it can be self-hosted, which matters a lot in a regulated environment like this one.

- [ ] Walk through Langfuse's tracing hierarchy — traces, observations, generations, sessions.

IF ASKED, SAY THIS

A trace is one end-to-end run of your application (e.g., one user request). Inside a trace, individual units of work are observations — Langfuse distinguishes generations (an actual LLM call: model, prompt, completion, token counts, cost, latency) from plain spans (everything else: retrieval, embedding, a tool/API call

A trace is one end-to-end run of your application (e.g., one user request). Inside a trace, individual units of work are observations — Langfuse distinguishes generations (an actual LLM call: model, prompt, completion, token counts, cost, latency) from plain spans (everything else: retrieval, embedding, a tool/API call), so you can see a trace as a full tree of nested LLM and non-LLM steps with causal relationships preserved. Sessions group multiple traces together for a multi-turn conversation, and user tracking lets you attribute traces/cost back to a specific end user — useful for both debugging ("show me everything this user's session did") and cost attribution.

- [ ] How does Langfuse relate to OpenTelemetry — do you have to choose one or the other?

IF ASKED, SAY THIS

No — Langfuse is actually built on OpenTelemetry under the hood, so it's complementary rather than competing. You can capture traces via Langfuse's native Python/JS SDKs, via 100+ framework integrations, directly via OTel, or via an LLM gateway like LiteLLM — all of it lands in the same tracing model. In practice that

No — Langfuse is actually built on OpenTelemetry under the hood, so it's complementary rather than competing. You can capture traces via Langfuse's native Python/JS SDKs, via 100+ framework integrations, directly via OTel, or via an LLM gateway like LiteLLM — all of it lands in the same tracing model. In practice that means you can run a standard OTel Collector pipeline for your general infrastructure telemetry and have it also export GenAI-specific spans into Langfuse for the LLM-specific analysis (evals, prompt versioning, cost dashboards) that a generic OTel backend like Tempo or an OTel-only Grafana setup won't give you out of the box.

- [ ] How would you use Langfuse for prompt management in a production system?

IF ASKED, SAY THIS

Prompts get versioned and managed centrally in Langfuse rather than hardcoded in application code — you fetch the current (or a specific labeled) version at runtime, which decouples prompt iteration from a full code deploy. That gives you a few practical wins: a rollback path if a new prompt version regresses in produc

Prompts get versioned and managed centrally in Langfuse rather than hardcoded in application code — you fetch the current (or a specific labeled) version at runtime, which decouples prompt iteration from a full code deploy. That gives you a few practical wins: a rollback path if a new prompt version regresses in production, A/B comparison between versions using the experiments feature, and a playground to test prompt changes against real historical trace data before shipping. For a platform/observability role specifically, the operational angle matters most: prompt changes become an auditable, versioned event you can correlate against a quality or cost regression in the dashboards, the same discipline as tracking any other config change.

- [ ] What is LLM-as-a-Judge evaluation in Langfuse, and when would you use it vs. human annotation?

IF ASKED, SAY THIS
LLM-as-a-Judge uses a separate LLM call to score a trace's output against a rubric (accuracy, relevance, safety, format compliance) automatically, at scale — practical for continuous, high-volume monitoring where human review of every output isn't feasible. Human annotation queues are for the cases judge models are wor

LLM-as-a-Judge uses a separate LLM call to score a trace's output against a rubric (accuracy, relevance, safety, format compliance) automatically, at scale — practical for continuous, high-volume monitoring where human review of every output isn't feasible. Human annotation queues are for the cases judge models are worse at — genuinely subjective quality judgments, or validating that the judge model itself is scoring correctly (you spot-check a sample of judge scores against human ones to catch judge drift). The realistic production pattern is layered: automated judge scoring on everything for continuous signal, human annotation on a sampled subset both for quality assurance on subjective cases and as an ongoing calibration check on the judge.

- [ ] How would you set up alerting on Langfuse data for a production quality regression?

IF ASKED, SAY THIS
Langfuse alerts evaluate a metric over a time window against a threshold and can notify Slack, trigger a GitHub Actions workflow, or call a webhook. The detail worth knowing: alerts support boolean scores directly — the average of a boolean score over a window is just the share of "true" results, so you can alert when,

Langfuse alerts evaluate a metric over a time window against a threshold and can notify Slack, trigger a GitHub Actions workflow, or call a webhook. The detail worth knowing: alerts support boolean scores directly — the average of a boolean score over a window is just the share of "true" results, so you can alert when, say, a hallucination-detection judge's failure rate crosses a limit, the same way you'd alert on an error rate in Prometheus. That closes the loop end to end: evaluators/judges score every output, dashboards show the trend, and alerts turn a slow quality drift into an actionable page before it's a user-facing incident.

- [ ] Self-hosted Langfuse vs. Langfuse Cloud — what would push you toward self-hosting, especially here?

IF ASKED, SAY THIS
Self-hosting (open source, MIT license, deployed via Docker Compose across a few backing services) gives you full data ownership and control over where prompt/completion data physically lives — the deciding factor in a regulated, enterprise-scale environment like this one, where sending prompts (which can contain custo

Self-hosting (open source, MIT license, deployed via Docker Compose across a few backing services) gives you full data ownership and control over where prompt/completion data physically lives — the deciding factor in a regulated, enterprise-scale environment like this one, where sending prompts (which can contain customer or internal financial data) to a third-party SaaS may not clear a data-residency or compliance bar. The tradeoff is operational: self-hosting is genuinely more setup and ongoing maintenance than a managed cloud offering, so it's a real ownership cost, not just a checkbox — worth being upfront in an interview that you understand it's a tradeoff, not a free win.

- [ ] How does the Langfuse MCP Server fit into an agent/IDE workflow, and how does that connect to this role's MCP requirement?

IF ASKED, SAY THIS
Langfuse ships its own MCP server, which exposes traces, evals, and prompts as tools an agent (or an IDE-based coding agent) can query and act on via natural language — instead of a human opening the Langfuse UI to check "why did this fail," an agent can pull the relevant trace directly through MCP as part of its own d

Langfuse ships its own MCP server, which exposes traces, evals, and prompts as tools an agent (or an IDE-based coding agent) can query and act on via natural language — instead of a human opening the Langfuse UI to check "why did this fail," an agent can pull the relevant trace directly through MCP as part of its own diagnostic loop. That's a concrete, current example of exactly what this role's MCP requirement is pointing at: MCP isn't just about agents calling arbitrary internal APIs in the abstract, it's specifically becoming the standard way observability tooling itself gets exposed to agentic systems — worth naming directly if asked how Langfuse and MCP intersect in this role.

Leadership & Behavioral

- [ ] How do you manage/lead your team day-to-day?

IF ASKED, SAY THIS
Structure the answer in three layers: (1) rhythm -- what recurring touchpoints exist (daily stand-up, 1:1s, on-call handoff, retro/postmortem review), (2) how you set direction -- how priorities get decided and communicated, (3) how you stay close to the work without micromanaging -- code/IaC review, on-call shadowing, dashboards you personally watch. Close with your actual philosophy in one line (e.g. "I lead by staying technically credible and removing blockers, not by directing every task").

This is normally the FIRST leadership question in a round, and it's really asking "do you have a coherent operating model, or are you just reacting." A strong answer names concrete mechanisms (weekly 1:1 cadence, a specific on-call handoff ritual, how postmortem action items get tracked) rather than staying abstract.

Bring your own: your actual cadence (1:1 frequency, stand-up format), how your team currently sizes/splits on-call, and one concrete example of a priority call you made and how you communicated it.

- [ ] What's your leadership philosophy, in one or two sentences?

IF ASKED, SAY THIS
A strong one-liner names a PRINCIPLE, not a personality trait: e.g. "I optimize for the team being able to make good decisions without me in the room -- that means clear ownership boundaries, documented context, and being genuinely reachable when something's ambiguous, not being the single point of approval for everything."

Avoid generic answers like "I lead by example" with no mechanism behind them. The strongest versions name a SPECIFIC tension you consciously navigate (autonomy vs. alignment, speed vs. safety, technical depth vs. delegation) and how you personally resolve it.

- [ ] How do you handle disagreement within the team on technical direction -- say, two senior engineers want to solve the same problem differently?

IF ASKED, SAY THIS
Structure: (1) make sure it's actually a disagreement about tradeoffs, not talking past each other -- restate both positions until both people agree you've captured them correctly, (2) surface the actual decision criteria (what does this system need to optimize for -- cost, latency, time-to-ship, maintainability), (3) if it's genuinely close, timebox a decision and move, rather than letting it drag. Close with: disagree-and-commit only works if the losing side felt genuinely heard, not just outvoted.

What interviewers are actually testing here is whether you default to hierarchy ("I decide because I'm the lead") or to reasoning ("we decide based on what the system actually needs") -- the second reads as far more senior, because it scales to situations where you're NOT the most senior person in the room.

- [ ] How do you ensure on-call load is distributed fairly across the team?

IF ASKED, SAY THIS
Concrete mechanisms: rotate on primary/secondary on a fixed, published schedule; track actual page volume and after-hours pages per person over time, not just whose turn it is -- someone can get equal SHIFTS but wildly unequal PAIN if their shifts happen to catch every bad week; have an explicit path to rebalance if one person's rotation is consistently worse; treat chronic pain from a specific alert as a signal to fix the alert, not just rotate the suffering.

The "equal shifts vs. equal pain" distinction is the detail that shows real operational maturity -- a lot of on-call rotations LOOK fair on a calendar while being badly unfair in practice.

- [ ] How do you build psychological safety and a blameless culture on your team, concretely -- not just as a value statement?

IF ASKED, SAY THIS
Concrete mechanisms: postmortems that explicitly separate what happened from who did it in the writing itself (describe actions, not people); you personally model owning your own mistakes visibly and early; process/tooling gaps surfaced by an incident get treated as system failures to fix, not individual failures to punish; you actively ask what made this an easy mistake to make, not just how do we prevent recurrence.

"Blameless" is one of the most overused words in these interviews precisely because it's easy to SAY and hard to actually practice under pressure -- naming a specific mechanism (how postmortems are literally worded, what you do in the room when someone gets defensive) reads as far more credible.

- [ ] How do you mentor and grow junior or mid-level engineers on your team?

IF ASKED, SAY THIS
Structure: pair them on real production work starting with lower-blast-radius tasks and deliberately increasing scope; give them primary ownership of something end-to-end (even small) rather than only ever assisting; be explicit about what the next level actually requires for them specifically; make time for this deliberately rather than assuming it happens organically.

Bring your own: a real example of someone you helped grow and specifically what changed for them -- even a small, honest example is far more credible than an abstract answer with no example at all.

- [ ] How do you handle a decision you disagree with from your own manager or leadership -- say, a deprioritization of reliability work you think is genuinely risky?

IF ASKED, SAY THIS
Structure: state your case once, clearly and with data/risk framing -- what's the actual risk being accepted, quantified if possible; make sure the decision-maker has genuinely heard the tradeoff; then disagree-and-commit -- execute the decision professionally rather than passive-aggressively slow-walking it; keep a paper trail of the risk being flagged, not as CYA but so it's not forgotten if it materializes; know when it crosses a line worth escalating further vs. a normal business tradeoff you disagree with but can live with.

Interviewers ask this specifically to filter out two failure modes: someone who never pushes back, and someone who can't let go once overruled. The disagree-and-commit framing, stated explicitly, answers both at once.

- [ ] How do you identify that someone on your team is underperforming, and what signals do you actually look at?

IF ASKED, SAY THIS
Look for a PATTERN across multiple signals, not one bad week: consistently missed commitments/estimates, code or IaC review taking unusually long or needing unusually heavy correction, on-call shifts where they escalate/miss things a peer at their level wouldn't, disengagement in stand-ups or reviews, or peers quietly starting to route around them on shared work. Explicitly distinguish this from a single bad sprint -- the question is whether it's a TREND.

Naming multiple concrete signal types is what separates a thoughtful answer from a generic one -- it shows you're actually watching for this pattern as part of normal leadership, not only noticing it once someone else complains.

- [ ] Walk through, step by step, how you'd actually handle an underperforming team member.

IF ASKED, SAY THIS
1) Private 1:1 conversation first, not a group setting -- ask genuinely, don't accuse. 2) Diagnose the actual cause together (skill gap? unclear expectations? something personal? motivation/fit?) -- the fix differs completely by cause. 3) Set explicit, specific, time-bound expectations, written down. 4) Provide real support toward those expectations, not just a warning. 5) Check in on a fixed cadence. 6) If there's genuine improvement, keep supporting it; if not, escalate formally rather than letting it drift, which is unfair to the person AND the team.

The ordering matters: start with genuine curiosity/diagnosis, not with the consequence. But don't skip the ending either -- a leader who goes vague on "what if it doesn't improve" reads as conflict-avoidant, which is its own red flag at senior level.

- [ ] What's the difference between a skill gap and a will/motivation gap in an underperforming engineer, and does your approach actually differ?

IF ASKED, SAY THIS
Skill gap: they want to do well but lack the specific ability or context -- fix is training, pairing, mentorship, adjusted scope while they grow.
Will/motivation gap: they have the ability but aren't applying it, often rooted in disengagement or something outside work -- the fix starts with an honest conversation, not more technical training. Applying a skill-gap fix to a will-gap problem, or vice versa, wastes time and doesn't help.

This distinction (sometimes framed as "can't" vs. "won't") is a genuinely useful mental model to name explicitly -- it signals you diagnose before you prescribe, rather than running the same playbook for every case.

- [ ] How do you track or measure a team member's performance -- on what basis, concretely?

IF ASKED, SAY THIS
A mix of qualitative and quantitative signals, explicitly NOT reduced to one number: delivery against commitments over time, code/IaC review quality and how they respond to feedback, on-call performance, collaboration signals (do peers want to work with them, do they help unblock others), and growth trajectory. Explicitly avoid vanity metrics like raw ticket count or lines of code, which are easy to game and don't reflect actual impact.

Naming what you DON'T use (ticket count, lines of code) is as valuable as naming what you do -- it signals you understand why naive metrics fail for engineering work, a common trap less experienced leads fall into.

- [ ] How do you actually deliver a difficult performance conversation?

IF ASKED, SAY THIS
Be direct and specific early -- don't bury the real message under small talk, which just increases anxiety. Use specific, observed examples, not generalizations. Separate the behavior from the person's worth ("this pattern isn't working" not "you're not good enough"). End with a clear, mutual next step, not just a diagnosis.

The "be direct early" point is worth stating explicitly -- a common mistake is softening the message so much the person doesn't register it as serious, which just delays the real conversation to a worse moment later.

- [ ] What if the underperformance turns out to be caused by something personal or external -- burnout, a health issue, something outside work -- rather than a skill or will problem?

IF ASKED, SAY THIS
The approach shifts from performance-management to support: acknowledge it without prying for details they haven't offered, adjust scope/expectations temporarily and explicitly, point them toward whatever support resources actually exist, and revisit the timeline once things stabilize rather than running a rigid clock through an acute personal situation. The team's delivery still needs a plan, but the plan should be humane, not punitive.

This question specifically tests whether your performance-management instinct is rigid-process-first or person-first -- a good answer holds both: the work still needs covering, but the person doesn't get treated as a performance problem when the actual issue is a human one.

- [ ] How do you know when coaching isn't working and it's time to escalate to a formal process (PIP or equivalent)?

IF ASKED, SAY THIS
Concrete signals: you've had the direct conversation, set explicit expectations, provided real support, given a genuine timeframe -- and the pattern hasn't meaningfully changed, or the person disputes there's even a problem despite specific examples. At that point, continuing informal coaching indefinitely isn't kindness, it's avoidance -- unfair to the person (who deserves clarity) and to the rest of the team absorbing the gap.

Framing formal escalation as fairness TO the underperforming person, not just protection of the team, is the more senior framing -- it shows you see a formal process as structured and honest rather than purely punitive.

- [ ] How do you handle a high performer who's technically excellent but a poor team player -- dismissive in reviews, unwilling to help others, hoards knowledge?

IF ASKED, SAY THIS
Name it directly and early rather than tolerating it because the output is good -- the behavior has a real cost (knowledge silos, on-call risk, damage to team trust) even if it doesn't show up in their own individual metrics. Give specific, behavioral feedback. If it doesn't change, be willing to weigh team health against individual output -- a single brilliant, corrosive contributor is rarely worth what they cost the team over time.

This question is testing whether you'll actually address a hard case where the obvious metric argues against acting -- naming that tension explicitly, and stating you'd act anyway, reads as genuinely senior.

- [ ] What have you automated in your role? (How do you structure this answer well?)

IF ASKED, SAY THIS
Don't list tools -- tell 2-3 SPECIFIC stories, each with: the manual pain before (be concrete -- how often, how long it took, who did it), what you built, and the measurable-or-at-least-describable outcome after. One well-told automation story beats a list of ten tool names.

Structure per story: "Before, X happened manually and took roughly Y time / Z people / caused this specific pain. I built [approach] to handle it. After, [concrete outcome]." Keep each story to 60-90 seconds spoken.

Bring your own: your real automations -- a health-check/remediation script, an IaC module that replaced manual provisioning, a CI/CD pipeline stage, a runbook you converted into a script. Use YOUR real before/after numbers, not invented ones.

- [ ] Walk me through one specific automation you built, end to end.

IF ASKED, SAY THIS
Structure: (1) the trigger -- what problem or repeated pain made you decide to build it, (2) the design decision -- why this approach over alternatives, (3) the actual build -- key technical choices worth mentioning, (4) how you validated it was safe before trusting it in production, (5) the outcome and what you'd do differently now.

Step 4 (validation/safety) is the part people most often skip and interviewers most want to hear -- automating something risky WITHOUT a safety mechanism is a red flag at senior level; mentioning how you made it safe to trust (dry-run mode, gradual rollout, a manual override) signals real production judgment.

Bring your own: pick your single best, most complete automation story and rehearse this exact structure for it out loud.

- [ ] How do you decide what's actually worth automating versus what should stay manual?

IF ASKED, SAY THIS
Weigh frequency times time-per-occurrence against the cost/risk of building and MAINTAINING the automation -- a task done twice a year for 10 minutes almost never justifies automation; a task done daily for 20 minutes almost always does. Also weigh risk: automating something you don't yet understand well tends to bake in fragility. Some things stay deliberately manual -- anything where human judgment case-by-case is the actual value, automating that just removes judgment you actually need.

The "sometimes staying manual is the right call" point is worth including explicitly -- an answer that says "I automate everything I can" without nuance reads as less experienced than one that names a genuine case FOR keeping something manual.

- [ ] How do you measure the impact or ROI of an automation you've built?

IF ASKED, SAY THIS
Before/after comparison on the specific pain it targeted: time saved per occurrence times frequency = hours/month reclaimed; error rate for that task before vs. after (manual processes have a human-error rate automation usually reduces); toil percentage of team time if you track it (Google's SRE framework suggests capping toil around 50% of time); and -- often underweighted -- whether it changed WHO can do the task (turning a senior-only process into something anyone on-call can safely run is a real force multiplier).

The "changed who can do the task" angle is a good one to have ready -- it's a real, often-overlooked dimension of automation ROI beyond pure time savings.

- [ ] Tell me about an automation that didn't go as planned.

IF ASKED, SAY THIS
Pick a REAL story with a real lesson, not a disaster with no resolution. Structure: what you built and why, what actually went wrong (be specific -- bad input handling, missing safety check, an edge case), what the actual impact was, and -- most important -- what you changed afterward.

This is a variant of the classic "tell me about a failure" question, applied to automation -- own it clearly and briefly, then spend most of the answer on what you learned and changed. A concrete process change afterward (e.g. "since then I always ship new automation with a dry-run mode first") makes this land as growth, not just a confession.

Bring your own: a real, honest example -- even a small one, or a near-miss you caught before it became a real incident.

- [ ] Build vs. buy -- when do you build custom automation versus adopt an existing tool?

IF ASKED, SAY THIS
Buy/adopt when the problem is well-understood and common industry-wide (a mature tool has already solved edge cases you haven't hit yet) -- CI/CD platforms, monitoring stacks, config management. Build custom when the need is specific enough to your environment that no tool fits well, or when the automation is small/simple enough that a tool's overhead costs more than just writing the script. Also weigh long-term maintenance -- a custom build becomes your team's permanent maintenance burden in a way an adopted tool's core logic usually isn't.

The maintenance-burden point is what senior interviewers listen for -- junior engineers tend to default to "I'll just build it" without weighing that the team now owns that code forever.

- [ ] What does a typical day or week look like for you as an SRE? (Framework for structuring this answer)

IF ASKED, SAY THIS
Break it into buckets rather than a literal hour-by-hour walkthrough: (1) On-call/reactive -- monitoring dashboards, triaging alerts, incident response. (2) Proactive reliability work -- capacity planning, chasing flaky alerts, reducing toil, improving runbooks. (3) Collaboration -- code/IaC review, design discussions, cross-team coordination. (4) Planning/growth -- sprint planning, postmortem follow-through, learning. Then give an honest split (e.g. roughly X% proactive, Y% reactive, shifting heavily reactive during an on-call week).

The bucket structure works whether you're asked about a single day or a general week. Being honest about the reactive/proactive split, rather than claiming it's always calm and planned, actually reads as more credible, since every real SRE role has variability.

Bring your own: your actual real rhythm -- your team's stand-up time, what you check first thing, what a typical on-call vs. non-on-call week looks like for you specifically.

- [ ] How do you split your time between reactive firefighting and proactive reliability work, and how do you protect the proactive side?

IF ASKED, SAY THIS
Name that reactive work will always expand to fill available time if you let it -- the discipline is DELIBERATELY protecting time for proactive work (a fixed weekly block, or a rule like "the on-call person handles reactive load so others can stay heads-down on planned work") rather than assuming proactive work happens whenever things are quiet, which in practice means it rarely happens at all.

Citing the general SRE principle that unmanaged toil trends toward consuming all available time, and needs an active counter-force, is a good, credible anchor for this answer.

- [ ] Walk me through your on-call rotation and what a shift actually involves.

IF ASKED, SAY THIS
Structure: rotation cadence and how primary/secondary works; what a shift START looks like (handoff notes, checking open incidents, reviewing what changed recently); what happens DURING a page (triage severity, initial response, when/how you escalate); what a shift END looks like (handoff, follow-up items logged). Mention your actual alert volume honestly if asked.

Bring your own: your real rotation length, your team's actual escalation policy, and one genuine example of a page you handled, even briefly, to ground the answer in something real.

- [ ] What does "ownership" mean to you in an SRE role, in practice?

IF ASKED, SAY THIS
Ownership means the buck stops with you for a system's reliability even for parts you didn't personally build -- you don't get to say "not my code" during an incident just because someone else wrote it originally. Concretely: proactively noticing and flagging risk before it becomes an incident; following through on postmortem action items even after the immediate pressure is gone; being the person who says "we should fix this properly" rather than just patching around it repeatedly.

This connects to a useful broader framing: "I didn't design this" is a weak answer to an ownership question -- "I own operating, troubleshooting, and evolving this, regardless of who designed it originally" is the stronger one.

- [ ] How do you prioritize your day when there's no active incident?

IF ASKED, SAY THIS
Rough priority order: anything time-sensitive from the on-call handoff first; then the highest-leverage proactive item on your list (the thing that reduces the MOST future toil or risk, not just the oldest ticket); then collaborative commitments (reviews others are blocked on); then your own deeper/longer-term work. Name that you actively re-triage through the day rather than rigidly following a plan that ignores what comes in.

The "don't block teammates" point is worth including specifically -- it signals team-awareness in how you prioritize, not just personal productivity, which is exactly what a senior/lead-level interviewer listens for beyond the technical content.

- [ ] What does "technical leadership" mean to you in an IC (not people-management) role like this one?

IF ASKED, SAY THIS
Setting technical direction and standards that other engineers actually adopt — through good architecture decisions, clear documentation, and code/config reviews that teach rather than just gatekeep — rather than through positional authority you don't have as an IC. Influence earned by consistently being right about tr

Setting technical direction and standards that other engineers actually adopt — through good architecture decisions, clear documentation, and code/config reviews that teach rather than just gatekeep — rather than through positional authority you don't have as an IC. Influence earned by consistently being right about tradeoffs and being someone junior engineers actually want to ask for help, not by title.

- [ ] How do you approach mentoring a junior engineer on a platform team, practically — what does that look like week to week?

IF ASKED, SAY THIS
Pairing on real work rather than abstract lessons (walking through an actual incident triage together, reviewing their Ansible role changes with specific, actionable feedback rather than just approve/reject), giving increasing ownership of well-scoped pieces with a safety net, and being explicit about the "why" behind

Pairing on real work rather than abstract lessons (walking through an actual incident triage together, reviewing their Ansible role changes with specific, actionable feedback rather than just approve/reject), giving increasing ownership of well-scoped pieces with a safety net, and being explicit about the "why" behind platform standards, not just the "what" — the goal is someone who can reason about new situations independently, not just follow a runbook.

- [ ] How would you influence platform standards across teams that don't report to you and may have their own established (different) practices?

IF ASKED, SAY THIS
Lead with the problem the standard solves, not the standard itself — a team resisting a proposed Ansible role structure is usually really objecting to "why should I change what already works for me," addressed by showing the actual cost of fragmentation (duplicated effort, inconsistent security posture) rather than ass

Lead with the problem the standard solves, not the standard itself — a team resisting a proposed Ansible role structure is usually really objecting to "why should I change what already works for me," addressed by showing the actual cost of fragmentation (duplicated effort, inconsistent security posture) rather than asserting authority. Document decisions and reasoning (an ADR-style record) so the standard is legible and debatable on its merits later, not just "because platform team said so."

- [ ] Tell me about how you'd participate in an architecture review for a new engineering tool/service being introduced to the platform.

IF ASKED, SAY THIS
Come with specific questions grounded in operational reality, not just design aesthetics — how does this get monitored, what's the backup/DR story, who's on the hook when it breaks at 2am, what's the failure mode if its main dependency is unavailable, does it fit the existing identity/access model or need a bespoke exc

Come with specific questions grounded in operational reality, not just design aesthetics — how does this get monitored, what's the backup/DR story, who's on the hook when it breaks at 2am, what's the failure mode if its main dependency is unavailable, does it fit the existing identity/access model or need a bespoke exception.

The goal of a good architecture review from a platform-support seat is surfacing the operational cost of a design decision while it's still cheap to change, not after it's already in production.

- [ ] How do you handle a situation where you disagree with a technical roadmap decision made above your level?

IF ASKED, SAY THIS
State the disagreement once, clearly, with the specific concern and its actual operational consequence — not just "I don't like this" — then commit to executing the decided direction well rather than relitigating it repeatedly or quietly under-investing in something you didn't agree with. A concrete example of doing th

State the disagreement once, clearly, with the specific concern and its actual operational consequence — not just "I don't like this" — then commit to executing the decided direction well rather than relitigating it repeatedly or quietly under-investing in something you didn't agree with. A concrete example of doing this in a prior role is a strong answer; otherwise, describe the principle directly and be ready to discuss how you'd apply it.

- [ ] This role explicitly calls out working closely with stakeholders, cross-functional teams, and clients — how do you approach translating business requirements into a technical architecture?

IF ASKED, SAY THIS
Start from the actual business outcome/constraint (what the business needs the integration to achieve, real deadline/budget/risk tolerance) before jumping to a specific AWS service list — a technically elegant architecture that doesn't fit the client's actual constraints (existing tooling, team skillset, timeline) isn't actually the right

Start from the actual business outcome/constraint (what the business needs the integration to achieve, real deadline/budget/risk tolerance) before jumping to a specific AWS service list — a technically elegant architecture that doesn't fit the client's actual constraints (existing tooling, team skillset, timeline) isn't actually the right answer.

Iterative validation — sketch the architecture, walk it through with stakeholders in terms they can evaluate (cost, timeline, risk) — catches misalignment early when it's cheap to adjust, rather than after significant build-out.

- [ ] How would you run an architecture review for a proposed integration or platform change in a consulting/client-facing context specifically?

IF ASKED, SAY THIS
The same operational-reality questions as any architecture review (monitoring, DR, security posture, on-call ownership) plus client-specific dimensions — does this fit within the client's existing AWS environment/governance rather than assuming a greenfield ideal, does it account for the client's own team's ability to operate it after han

The same operational-reality questions as any architecture review (monitoring, DR, security posture, on-call ownership) plus client-specific dimensions — does this fit within the client's existing AWS environment/governance rather than assuming a greenfield ideal, does it account for the client's own team's ability to operate it after handoff, and is the cost model something the client actually signed up for, not a surprise at the end.

- [ ] How do you document architecture decisions in a way that's actually useful to the client's team after your engagement, not just internally to your own team?

IF ASKED, SAY THIS
Architecture Decision Records capturing not just what was decided but why — alternatives considered and rejected, and under what circumstances to revisit — since a client team inheriting the platform needs the reasoning, not just the end state, to maintain and evolve it confidently. Written assuming the reader wasn't in the room for the o

Architecture Decision Records capturing not just what was decided but **why** — alternatives considered and rejected, and under what circumstances to revisit — since a client team inheriting the platform needs the reasoning, not just the end state, to maintain and evolve it confidently.

Written assuming the reader wasn't in the room for the original discussion, kept alongside the code/IaC itself rather than in a separate wiki that drifts out of sync with what's actually deployed.

- [ ] How would you mentor a client's internal team to take over operating a platform you architected, given the consulting/services context this role sits in?

IF ASKED, SAY THIS
Involve their team throughout the build, not just at a handoff meeting at the end — pairing on real implementation, walking through actual incidents as they occur rather than only after the engagement formally ends. Runbooks written for their actual skill level and tooling familiarity, and a defined transition period with your team still

Involve their team throughout the build, not just at a handoff meeting at the end — pairing on real implementation, walking through actual incidents as they occur rather than only after the engagement formally ends. Runbooks written for their actual skill level and tooling familiarity, and a defined transition period with your team still reachable for genuine escalations while their confidence ramps up — a hard handoff on day one of go-live is a common way client platforms degrade shortly after an engagement ends.

Prod Troubleshooting

SCENARIO 3 AM page: a production EKS-hosted service is returning 5xxs to customers. Walk through your first 15 minutes.

IF ASKED, SAY THIS
Confirm scope before touching anything — is this one service, one AZ, or cluster-wide? Pull the alarm's exact dimension and a top-level dashboard (error rate, latency, traffic) for the affected service specifically. Scope determines urgency and who else needs to be pulled in; don

First move
Confirm scope before touching anything — is this one service, one AZ, or cluster-wide? Pull the alarm's exact dimension and a top-level dashboard (error rate, latency, traffic) for the affected service specifically. Scope determines urgency and who else needs to be pulled in; don't start debugging pods before you know if this is actually a single-service problem or a symptom of something bigger (a node group issue, a shared dependency).

What you're checking
`kubectl get pods -n <ns>` for CrashLoopBackOff/pending state, `kubectl describe` and recent events for anything that just changed, and — critically — whether a deploy went out recently. A huge fraction of real 3am pages are "the last deploy broke something," so check the deployment's rollout history and recent commits before assuming it's an infrastructure fault. If a recent deploy is the likely cause, rolling back is almost always faster than root-causing forward under pressure.

Judgment signal
Mitigate first, root-cause after — roll back or scale out to stop customer impact, then investigate why with the pressure off. Trying to find the exact root cause while customers are actively affected, when a rollback would resolve it in two minutes, is the mistake this question is designed to surface.

SCENARIO A shared Terraform module update is queued to merge, and the plan shows it will touch 60 resources across 8 services — far more than expected for what looked like a small change. What do you do before approving?

IF ASKED, SAY THIS
Stop and read the plan output carefully rather than trusting the PR description — specifically distinguish update in-place from replace (force-new) in the plan. A small-looking change to a shared module (e.g. changing an argument that's actually immutable on the underlying resour

First move
Stop and read the plan output carefully rather than trusting the PR description — specifically distinguish `update in-place` from `replace` (force-new) in the plan. A small-looking change to a shared module (e.g. changing an argument that's actually immutable on the underlying resource) can silently force replacement of every resource built from it — that's the difference between "60 minor updates" and "60 resources destroyed and recreated," including possibly stateful ones.

What you're checking
Whether any of the affected resources are stateful/hard-to-recreate (databases, resources holding data or identity that matters — an ALB whose DNS name downstream systems depend on). If replacement is real and touches anything like that, this isn't a normal merge — it needs a maintenance window, explicit sign-off from affected service owners, and likely a staged rollout (apply to one service's copy of the module first, verify, then the rest) rather than one big-bang apply across 8 services simultaneously.

Judgment signal
The instinct to actually read a large plan diff line-by-line before approving — not just check that CI is green — is exactly what this question tests. "Terraform plan passed" and "this plan is safe to apply" are not the same statement.

SCENARIO Finance flags that yesterday's AWS bill is roughly double the daily average, with no known change deployed. How do you find the cause quickly?

IF ASKED, SAY THIS
Cost Explorer, grouped by service, filtered to yesterday specifically — this narrows "the whole bill doubled" to "which specific service's spend actually doubled" in under a minute, which is nearly always faster than guessing based on what changed recently.

First move
Cost Explorer, grouped by service, filtered to yesterday specifically — this narrows "the whole bill doubled" to "which specific service's spend actually doubled" in under a minute, which is nearly always faster than guessing based on what changed recently.

What you're checking
Once the service is identified, the usual suspects in rough order of frequency: an Auto Scaling Group or Karpenter that scaled way up and never scaled back down (a stuck scale-in, or a runaway job that kept triggering scale-out); a Lambda function stuck in a retry loop against a failing downstream, burning invocations; a misconfigured cross-region data transfer (someone pointed traffic at the wrong region, incurring transfer charges that don't exist for same-region traffic); or, if no deploy happened, check whether a scheduled job (a nightly batch, a cron-triggered Step Function) simply processed an unusually large input that day — sometimes it's not a bug, it's real, valid usage that needs a capacity conversation, not a fix.

Judgment signal
Don't assume "no known change deployed" means no change happened — check for autoscaling events and scheduled/triggered jobs specifically, since those are exactly the kind of change that doesn't show up in a deploy log.

SCENARIO A service that's worked fine for months suddenly can't assume a cross-account role — right after the security team rolled out new org-wide SCPs. Where do you look?

IF ASKED, SAY THIS
Given the timing, treat the new SCP as the prime suspect before re-checking IAM policies that haven't changed — this connects directly back to the policy-evaluation-order question: SCPs are a hard ceiling that can silently block an action even though the role's own identity polic

First move

Given the timing, treat the new SCP as the prime suspect before re-checking IAM policies that haven't changed — this connects directly back to the policy-evaluation-order question: SCPs are a hard ceiling that can silently block an action even though the role's own identity policy and the target's trust policy are both still perfectly correct and unchanged.

What you're checking
`sts:AssumeRole` (or the specific action now failing) against the new SCP's explicit denies and allow-list conditions for both the source and destination accounts — a common real cause is a new SCP restricting `sts:AssumeRole` to specific role-name patterns or requiring a condition (like an org ID check) that this particular cross-account trust relationship doesn't satisfy.

Judgment signal
Resist the instinct to "fix" it by loosening the service's own IAM policy — if the policy hasn't changed and the SCP has, widening the identity policy won't fix an SCP-level block and just adds unnecessary permissions. Correlating the timing to the actual change that happened is the whole answer here.

- [ ] SCENARIO GitHub Actions shows the deploy job succeeded and green, but the production behavior clearly hasn't changed. What's your investigation order?

> **IF ASKED, SAY THIS**
> Verify what actually got deployed where — confirm the image tag/commit SHA that's actually running in production right now (via the running task/pod's image, not the workflow log's claim that it deployed) matches the commit you expect. A "successful" deploy job can still have dep

First move
Verify what actually got deployed *where* — confirm the image tag/commit SHA that's actually running in production right now (via the running task/pod's image, not the workflow log's claim that it deployed) matches the commit you expect. A "successful" deploy job can still have deployed to the wrong target, or deployed correctly but a caching layer (CDN, ALB connection reuse, browser cache during testing) is masking the change.

What you're checking
Common real causes: the workflow deployed to the correct ECS service/EKS namespace but the wrong environment (staging config accidentally pointed at what you thought was prod); a rolling deployment is still mid-rollout and old + new versions are both serving traffic, so what you're testing against happened to hit an old replica; or the change is real but sits behind a feature flag that wasn't flipped as part of this deploy.

Judgment signal
"Green in CI" only proves the pipeline's steps returned exit code 0 — it proves nothing about whether the deployed artifact is correct or whether it's actually serving the traffic you're testing. Treating pipeline success as equivalent to "verified in production" is the gap this question is probing.

- [ ] SCENARIO An SQS-triggered Lambda has silently stopped processing — the queue depth is climbing, but there are no Lambda errors in the logs at all.

> **IF ASKED, SAY THIS**
> No errors at all, combined with a growing queue, points away from "the function is failing" and toward "the function isn't being invoked in the first place" — check the event source mapping's state first (aws lambda get-event-source-mapping), not the function's own logs.

First move
No errors at all, combined with a growing queue, points away from "the function is failing" and toward "the function isn't being invoked in the first place" — check the event source mapping's state first (`aws lambda get-event-source-mapping`), not the function's own logs.

What you're checking
An event source mapping can be silently `Disabled` (someone paused it during a prior incident and forgot to re-enable it) or stuck in a failed state if it can no longer assume the execution role's permissions to poll the queue (an IAM change broke `sqs:ReceiveMessage` / `sqs:DeleteMessage` access without touching the function's own logs at all, since it never got invoked to log anything). Also check whether reserved concurrency was set to 0 on the function — a common "temporarily pause processing" move during an incident that's easy to forget to revert.

Judgment signal
Recognizing that "zero errors" is itself a data point, not the absence of one — it should redirect your investigation toward the invocation layer instead of the function's internal logic.

- [ ] SCENARIO A newly-launched AgentCore-based internal assistant is generating a much higher AWS bill than projected within its first week. How do you investigate an agent workload's cost, specifically?

> **IF ASKED, SAY THIS**
> Pull per-session tool-call and token counts from AgentCore Observability, not just the aggregate bill — for agentic workloads the useful unit of cost analysis is "cost per session/conversation," analogous to "cost per request" for a normal API, and that view will show you immedia

First move
Pull per-session tool-call and token counts from AgentCore Observability, not just the aggregate bill — for agentic workloads the useful unit of cost analysis is "cost per session/conversation," analogous to "cost per request" for a normal API, and that view will show you immediately whether cost is spread evenly or concentrated in a small number of runaway sessions.

What you're checking
The most common real cause: an agent stuck in a reasoning loop — repeatedly calling the same tool, or the model second-guessing itself across many turns for a single user request, multiplying both model-token cost and Gateway/tool-invocation cost per session far beyond what a well-scoped interaction should need. Check whether there's a hard cap on tool calls or turns per session at all — if the agent has no bounded retry/loop limit, one confused session can run essentially unbounded.

Judgment signal
This is the "cost is per-token, not per-request" operational difference from Chapter 5 showing up as a real incident — recognizing that traditional per-request cost intuition doesn't transfer directly to agentic workloads, and that the fix is often a guardrail (max tool calls, max turns, a timeout) rather than an infrastructure-level cost optimization.

SCENARIO A compliance audit flags an IAM role that has far more permissions than the service using it appears to need. How do you tighten it without risking an outage?

IF ASKED, SAY THIS
Never edit the live role's policy directly first — pull its actual usage history via IAM Access Analyzer's policy generation (or CloudTrail directly) over a window long enough to capture infrequent-but-legitimate paths, not just typical daily traffic.

First move
Never edit the live role's policy directly first — pull its actual usage history via IAM Access Analyzer's policy generation (or CloudTrail directly) over a window long enough to capture infrequent-but-legitimate paths, not just typical daily traffic.

What you're checking
Specifically look for monthly/quarterly/annual jobs that might not have fired during a short observation window — a permission that looks "unused" over two weeks might be exercised by a month-end batch job or an annual DR test, and removing it based on a too-short window is how "least privilege cleanup" causes an outage weeks later that's hard to trace back to this change.

Judgment signal
Deploy the tightened policy to a copy of the role (or in monitor-only/audit mode if the tooling supports it) first, watch for `AccessDenied` events, and only cut the actual role over once you're confident — treating "reduce this role's permissions" as a change with real regression risk, not a pure cleanup task with no downside.

SCENARIO Sketch a disaster recovery plan for an EKS + RDS stack that needs to survive a full region outage.

IF ASKED, SAY THIS
Establish the actual requirement before designing anything — RTO/RPO drive cost and complexity enormously, and "survive a region outage" means very different architectures depending on whether minutes of downtime and data loss are acceptable versus effectively none.

First move
Establish the actual requirement before designing anything — RTO/RPO drive cost and complexity enormously, and "survive a region outage" means very different architectures depending on whether minutes of downtime and data loss are acceptable versus effectively none.

What you're checking / designing
For RDS: cross-region read replica (async, some replication lag = non-zero RPO) promotable to primary on failover, or Aurora Global Database for tighter RPO (typically sub-second lag) with a faster managed failover path. For EKS: since the control plane itself is regional, DR means a second cluster already running (or ready to be stood up quickly from IaC) in the DR region, with application manifests/Helm values already in the GitOps repo ready to sync — not something built from scratch during the actual incident. Route 53 health-check-based failover routing (or a global accelerator) shifts traffic once the DR region is confirmed healthy.

Judgment signal
The strongest answer states the RTO/RPO trade-off explicitly and ties the architecture to it, rather than reflexively describing the most expensive/complex option (active-active everywhere) as if it's always correct — and mentions that none of this counts as a real DR plan until it's actually been tested with a real failover exercise, not just documented.

SCENARIO You're paged for elevated p99 latency. Every individual dashboard — CPU, memory, error rate — looks completely normal. First 15 minutes?

IF ASKED, SAY THIS
Normal resource metrics with elevated latency points away from "something is overloaded" and toward "something is waiting" — go straight to traces for a slow request rather than continuing to stare at metrics that have already told you they're not the answer.

First move
Normal resource metrics with elevated latency points away from "something is overloaded" and toward "something is waiting" — go straight to traces for a slow request rather than continuing to stare at metrics that have already told you they're not the answer.

What you're checking
A trace for one of the actually-slow requests usually shows the answer directly: a downstream dependency (a database, an external API, another internal service) taking longer than usual even though its own dashboards look fine too — a common cause is connection pool exhaustion (requests queueing for a free connection, which shows up as latency, not as an error, and doesn't move CPU/memory at all) or a slow query that's within "normal" CPU usage but is newly slow due to a changed query plan, missing index, or data growth crossing some threshold.

Judgment signal
Recognizing when to stop trusting aggregate dashboards and go straight to a trace of one specific slow request is the actual skill — dashboards answer "is the service healthy on average," traces answer "why was this one request slow," and this scenario is deliberately built so only the second question has the answer.

SCENARIO Payment API latency spikes right after a deployment. Walk through your triage.

IF ASKED, SAY THIS
Check the deployment timeline first — correlate the exact spike time against the deploy event. If they line up, prefer an immediate rollback over live-debugging under pressure; mitigate first. If rollback isn't instantly possible, check the obvious deploy-related causes: a new N+

Check the deployment timeline first — correlate the exact spike time against the deploy event. If they line up, prefer an immediate rollback over live-debugging under pressure; mitigate first. If rollback isn't instantly possible, check the obvious deploy-related causes: a new N+1 query, a missing index on a new code path, a changed connection-pool or timeout setting, a cold cache right after restart. Use tracing to see exactly where in the request the new latency is concentrated — your app code, or a specific downstream call. Only after mitigating do you dig into the full 'why' in the postmortem.

SCENARIO Order rejection rate rises on the trading platform right at market open. What do you check?

IF ASKED, SAY THIS
Whether it's capacity-related — did traffic exceed autoscaling's reaction speed. Whether the risk/compliance pre-trade check service is slow or erroring under the volume spike, since a slow risk check at high load can itself cause timeouts and rejections. Whether the market-data

Whether it's capacity-related — did traffic exceed autoscaling's reaction speed. Whether the risk/compliance pre-trade check service is slow or erroring under the volume spike, since a slow risk check at high load can itself cause timeouts and rejections. Whether the market-data feed is stale, causing rejections due to pricing that's no longer valid. Connection-pool or thread saturation specifically on the order-matching path. And whether it's isolated to particular instruments or order types — which narrows it toward a data/logic issue — versus system-wide, which points toward infra or capacity.

SCENARIO A chaos experiment reveals a single point of failure in a payment gateway's DB failover. How do you communicate and remediate it?

IF ASKED, SAY THIS
Document the finding precisely — what was tested, what broke, the business impact if it happened for real. Communicate promptly to the service owner and reliability leadership, framed by real urgency: this is now a known gap, so treat it with the same seriousness as 'what if this

Document the finding precisely — what was tested, what broke, the business impact if it happened for real. Communicate promptly to the service owner and reliability leadership, framed by real urgency: this is now a *known* gap, so treat it with the same seriousness as 'what if this happens tomorrow'. Open a tracked remediation item with a named owner and target date, not just a conversation that fades. Consider interim mitigations — tighter monitoring on that specific failure mode — while the real DB-failover redesign is underway, since it may take real time to fix properly. And re-run the same experiment after the fix ships, to actually confirm it's resolved rather than assuming it is.

SCENARIO The error budget for a critical payments service is nearly exhausted mid-quarter. What do you do?

IF ASKED, SAY THIS
Invoke the pre-agreed error-budget policy: pause or slow non-essential feature releases for that service, redirect the team's near-term focus to the top reliability issues actually consuming the budget — usually clear from recent incidents and postmortems — and communicate transp

Invoke the pre-agreed error-budget policy: pause or slow non-essential feature releases for that service, redirect the team's near-term focus to the top reliability issues actually consuming the budget — usually clear from recent incidents and postmortems — and communicate transparently to stakeholders about why velocity is intentionally slowing and for roughly how long. Use the remaining budget deliberately rather than freezing everything indiscriminately — a well-tested, high-value fix can still ship; a risky new feature shouldn't.

SCENARIO A batch settlement job that normally finishes in 2 hours is still running after 5. Walk through debugging it.

IF ASKED, SAY THIS
Check job logs or progress markers to see exactly where it's stuck — which record range or step. Check for a lock or contention issue — is it waiting on a DB lock, or blocked by another concurrent job. Check input volume — is it just legitimately processing more data than usual.

Check job logs or progress markers to see exactly where it's stuck — which record range or step. Check for a lock or contention issue — is it waiting on a DB lock, or blocked by another concurrent job. Check input volume — is it just legitimately processing more data than usual. Check downstream dependency health — is a reconciliation or reporting call to another system hanging. Check the job's own resource metrics — is it CPU/memory-starved or swapping. And check whether it's silently retrying or looping on a subset of bad records rather than actually making forward progress. Given settlement deadlines often carry real regulatory consequences, flag the risk to stakeholders early rather than waiting until the deadline is already missed.

SCENARIO A load test shows current peak is fine, but a projected 3× holiday-season surge would breach SLOs. What's your plan?

IF ASKED, SAY THIS
Prioritize remediation by the biggest bottleneck first, from the test's tier-by-tier breakdown, rather than uniformly over-scaling everything. Work out whether it's a horizontal-scaling problem (often the easy fix — add capacity) or an architectural bottleneck — a single non-scal

Prioritize remediation by the biggest bottleneck first, from the test's tier-by-tier breakdown, rather than uniformly over-scaling everything. Work out whether it's a horizontal-scaling problem (often the easy fix — add capacity) or an architectural bottleneck — a single non-scalable component, or a rate-limited third-party dependency — that capacity alone can't solve. Set a concrete remediation timeline working backward from the holiday date, with buffer left for a follow-up validation load test. And keep a 'surge playbook' ready as a fallback — temporary rate-limiting, queueing, or graceful degradation of non-critical features — in case remediation can't fully close the gap in time.

SCENARIO An alert storm hits during a partial cloud-region outage. How do you manage signal vs. noise?

IF ASKED, SAY THIS
Lean on alert grouping/correlation — if many services are alerting at once, it's very likely one root cause (a shared dependency, or the region itself), not N separate problems, so responders should see one grouped incident, not two hundred individual pages. Escalate straight to

Lean on alert grouping/correlation — if many services are alerting at once, it's very likely one root cause (a shared dependency, or the region itself), not N separate problems, so responders should see one grouped incident, not two hundred individual pages. Escalate straight to the known 'regional degradation' runbook if one exists, and validate that failover triggered rather than debugging each symptom in isolation. Temporarily mute clearly-downstream, already-explained alerts to cut noise for responders while keeping the small set of decision-relevant signals visible. And communicate

broadly, early — silence during an obvious multi-service event erodes stakeholder confidence fast, even a short 'we're aware, investigating' is far better than nothing.

- [ ] SCENARIO A regulator asks for evidence of your DR testing cadence and results. What do you show them?

IF ASKED, SAY THIS
A documented DR/resilience testing schedule and history — dates, scope, what was actually tested. Objective pass/fail results measured against defined RTO/RPO targets for each test, not just 'it went fine'. Tracked remediation items from any gaps found, with evidence they were ac

A documented DR/resilience testing schedule and history — dates, scope, what was actually tested. Objective pass/fail results measured against defined RTO/RPO targets for each test, not just 'it went fine'. Tracked remediation items from any gaps found, with evidence they were actually closed out. And ideally trend evidence showing the testing program itself is continuous and improving — chaos engineering game days count as strong evidence of proactive, frequent resilience validation, not just a single annual box-ticking exercise.

- [ ] SCENARIO The cluster autoscaler isn't scaling fast enough during a traffic spike. What do you investigate?

IF ASKED, SAY THIS
Node provisioning time — cloud instance launch plus join time can be several minutes, and if the spike is faster than that, reactive HPA/Cluster Autoscaler fundamentally can't keep up and you need pre-scaling instead. Whether you're silently hitting a resource quota — a cloud acc

Node provisioning time — cloud instance launch plus join time can be several minutes, and if the spike is faster than that, reactive HPA/Cluster Autoscaler fundamentally can't keep up and you need pre-scaling instead. Whether you're silently hitting a resource quota — a cloud account vCPU limit, or a cluster max-node-count — capping scale-out without an obvious error. Whether pod scheduling is blocked by something other than raw node count — limited availability of the specific instance type in the node pool, taints/tolerations, or a PodDisruptionBudget conflict. And whether HPA's own metric-scraping latency and stabilization-window settings are themselves adding delay before it even decides to scale.

- [ ] SCENARIO A canary shows a slightly elevated error rate, but it looks like it could just be noise. Do you proceed?

IF ASKED, SAY THIS
Don't eyeball it — apply the pre-agreed statistical threshold and minimum sample size/bake time rather than a gut call made under release pressure. If the sample size is too small to be confident either way, extend the bake period or increase canary traffic to get a clearer signa

Don't eyeball it — apply the pre-agreed statistical threshold and minimum sample size/bake time rather than a gut call made under release pressure. If the sample size is too small to be confident either way, extend the bake period or increase canary traffic to get a clearer signal instead of guessing. If it's genuinely within the pre-agreed noise threshold *with* adequate sample size, proceed, but watch closely through the full rollout ramp. If it's borderline or trending worse across the bake window rather than flat noise, don't proceed — investigate first. The real discipline here is having the threshold defined *before* the deploy, not decided in the moment when there's pressure to ship.

- [ ] SCENARIO You discover mid-incident that the runbook you're following is outdated and actively leading responders astray. What do you do?

IF ASKED, SAY THIS
In the moment: acknowledge it's wrong and pivot immediately to first-principles troubleshooting with whoever's most familiar with the system, rather than continuing to follow bad instructions or losing time arguing about why it's wrong. Afterward: treat it as a genuine postmortem

In the moment: acknowledge it's wrong and pivot immediately to first-principles troubleshooting with whoever's most familiar with the system, rather than continuing to follow bad instructions or losing time arguing about why it's wrong. Afterward: treat it as a genuine postmortem finding in its own right — an outdated runbook that actively caused delay is a real contributing factor, not a footnote — fix it immediately, and ask whether there's a broader process gap (no requirement to review runbooks after incidents or architecture changes) that let it go stale in the first place.

- [ ] SCENARIO A downstream card network or payment processor has intermittent failures. How do you design resilience against a dependency you don't control?

IF ASKED, SAY THIS
Aggressive but sane timeouts, so a hanging call to them never hangs your entire request thread pool — this is exactly where a circuit breaker earns its keep, failing fast once the dependency is clearly unhealthy instead of queuing endlessly behind it. Idempotency keys on every ou

Aggressive but sane timeouts, so a hanging call to them never hangs your entire request thread pool — this is exactly where a **circuit breaker** earns its keep, failing fast once the dependency is clearly unhealthy instead of queuing endlessly behind it. Idempotency keys on every outbound call, so a retry can never cause a duplicate charge. A secondary processor as a fallback path where the business allows multi-processor routing. Graceful degradation on your side — queue the transaction for later with clear customer messaging, rather than a hard failure. And proactive monitoring of the dependency's own status/health feeding your alerting, so you learn about their incident before your own metrics degrade far enough to page you independently.

- [ ] SCENARIO You're paged at 2 AM: p99 latency on the fraud-scoring endpoint has tripled.

IF ASKED, SAY THIS
First: check whether it's a traffic-volume story (legitimate spike the autoscaler hasn't caught up to yet) versus a genuine degradation — CloudWatch invocation count alongside latency tells you which. Contain: if it's autoscaling lag, manually bump instance count/min-capacity rat

First: check whether it's a traffic-volume story (legitimate spike the autoscaler hasn't caught up to yet) versus a genuine degradation — CloudWatch invocation count alongside latency tells you which. **Contain:** if it's autoscaling lag, manually bump instance count/min-capacity rather than waiting for the scaling policy to catch up. If it's not traffic-driven, check for a resource-exhaustion signal (memory/CPU near limit — possibly a memory leak from a recent deploy) and consider a fast rollback to the previous known-good variant while you investigate rather than debugging live on a degraded production path. **Then diagnose** with traces to find which hop (feature lookup vs. model inference itself) actually regressed. **Escalate** only if it's still unresolved past your runbook's defined threshold — don't sit on it solo past that point.

- [ ] SCENARIO Your drift monitor fires: model accuracy proxy dropped 15% in the last 6 hours.

IF ASKED, SAY THIS
Don't roll back reflexively — first determine whether this is data drift (upstream distribution shift) or a genuine model/infra problem. Pull PSI/KS-test results on recent feature distributions against baseline. Check whether a known event correlates (a new product launch, an ups

Don't roll back reflexively — first determine whether this is data drift (upstream distribution shift) or a genuine model/infra problem. Pull PSI/KS-test results on recent feature distributions against baseline. Check whether a known event correlates (a new product launch, an upstream schema change, a batch job that ran with bad data). If the drift is real and upstream, the fix is a data-pipeline fix, not a model rollback — rolling back the model wouldn't even help if the input data itself is the problem. If you can't find an upstream cause and the model's own behavior on stable inputs has changed, that's when rollback-while-investigating is the right containment move. **Loop in Data Science** either way — this is a joint call, not a unilateral SRE decision, especially for anything touching a regulated outcome.

- [ ] SCENARIO A canary deployment is clearly degrading key metrics, but the automated rollback didn't trigger.

IF ASKED, SAY THIS
Immediately — manually shift traffic weight back to the previous variant; don't wait to understand why automation failed while customers are on the bad path. Then figure out why the alarm-based rollback didn't fire: most commonly, the CloudWatch alarm's evaluation window/threshol

Immediately — manually shift traffic weight back to the previous variant; don't wait to understand why automation failed while customers are on the bad path. **Then** figure out why the alarm-based rollback didn't fire: most commonly, the CloudWatch alarm's evaluation window/threshold was tuned too loosely for how the metric actually degraded (a real quality drop that doesn't cross a raw error-rate threshold, for instance), or the deployment guardrail wasn't actually wired to the alarm you thought it was. **Follow-up** that matters as much as the fix: this is a gap in your safety net that needs a problem record — if it failed silently once, it'll fail silently again on the next deploy unless the alarm/threshold gets fixed and, ideally, tested against a deliberately-bad synthetic deploy before you trust it again.

- [ ] SCENARIO A SageMaker endpoint update has been stuck in "Updating" for 40 minutes.

IF ASKED, SAY THIS
Check the endpoint's event history/CloudWatch Logs for the update operation — common causes: the new container failed its internal health check on the new instances (so SageMaker won't cut over and is stuck waiting), a capacity issue in the target instance type/AZ, or an IAM perm

Check the endpoint's event history/CloudWatch Logs for the update operation — common causes: the new container failed its internal health check on the new instances (so SageMaker won't cut over and is stuck waiting), a capacity issue in the target instance type/AZ, or an IAM permission gap the execution role hit only on this specific update. The endpoint should still be serving traffic on the *old* variant during this — confirm that's actually true (check live invocation success rate) so you know whether this is "annoying but traffic is fine" or "traffic is degraded too." If it's genuinely stuck past a reasonable window, cancel the update rather than let it hang indefinitely, fix the underlying cause (usually the new container's health check or a resource limit), and redeploy.

- [ ] SCENARIO Predictions look consistently off, and you trace it to a Glue ETL job that's been silently failing for two days.

IF ASKED, SAY THIS
This is a monitoring gap as much as a pipeline failure — a job silently failing for two days means nothing was watching job success/failure status with alerting attached, which is the first thing to fix regardless of this specific incident. Immediate containment: identify exactly

This is a monitoring gap as much as a pipeline failure — a job silently failing for two days means nothing was watching job success/failure status with alerting attached, which is the first thing to fix regardless of this specific incident. Immediate containment: identify exactly which feature(s) went stale and for how long, and whether predictions served during that window should be flagged/re-evaluated (this becomes an incident-scope question for Risk/Compliance if any of those predictions affected a customer decision). Fix the job, backfill the missed data if the source still has it, and verify the feature store actually reflects fresh data before declaring resolved. **Prevention:** a CloudWatch alarm/EventBridge rule on Glue job failure status, not reliance on someone noticing.

- [ ] SCENARIO SageMaker Model Monitor flags a data-quality violation. Real issue, or false positive?

IF ASKED, SAY THIS
Pull the actual constraint violation report — Model Monitor tells you exactly which feature and which check (a null-rate threshold, a type mismatch, a range violation) triggered it, so start there rather than guessing. Cross-check against anything that changed recently: an upstre

Pull the actual constraint violation report — Model Monitor tells you exactly which feature and which check (a null-rate threshold, a type mismatch, a range violation) triggered it, so start there rather than guessing. Cross-check against anything that changed recently: an upstream schema change, a new data source, or a genuinely unusual-but-valid event (a holiday causing an unusual transaction pattern that's real, not broken). If it's a genuine data problem, treat it like the Glue-failure scenario above. If it's a legitimate distribution shift the baseline constraints file hasn't been updated to reflect, the fix is updating the baseline — but that update itself should go through review, since loosening a data-quality check is exactly the kind of change that needs a second set of eyes in a regulated environment, not a quiet one-line config edit.

SCENARIO The fraud model starts blocking a disproportionate number of legitimate transactions — false-positive spike.

IF ASKED, SAY THIS
This one has a customer-trust dimension a normal latency incident doesn't, so speed of communication matters alongside speed of fix. Confirm the spike is real (not a reporting/dashboard artifact) and quantify it — how many customers, how much value, over what window; the business

This one has a customer-trust dimension a normal latency incident doesn't, so speed of communication matters alongside speed of fix. Confirm the spike is real (not a reporting/dashboard artifact) and quantify it — how many customers, how much value, over what window; the business will ask this within minutes. Check whether it correlates with a recent deploy (rollback is the fast lever if so) or an upstream data issue (as in scn2/scn5). If it's a genuine model behavior issue with no fast root cause, the containment decision is a business one, not purely technical: is the false-positive rate bad enough to justify falling back to a more conservative/manual-review-heavy mode while you investigate, accepting some fraud may slip through temporarily. That trade-off decision should follow a pre-agreed escalation path, not be made solo.

SCENARIO A DR failover drill reveals your actual RPO is worse than what's documented.

IF ASKED, SAY THIS
Treat the drill as having done its job — this is exactly why you test instead of trusting the architecture diagram. Quantify the actual gap (measured replication lag vs. documented RPO) and find the specific cause: a replication process that's slower than assumed, a component tha

Treat the drill as having done its job — this is exactly why you test instead of trusting the architecture diagram. Quantify the actual gap (measured replication lag vs. documented RPO) and find the specific cause: a replication process that's slower than assumed, a component that wasn't actually included in the replication scope, or a documented RPO that was never validated in the first place and was simply optimistic. Report it honestly rather than quietly patching the number in the DR doc to match reality — a corrected RPO with a remediation plan and timeline is a much better audit finding than a stale document that turns out to be wrong during an actual disaster. Then fix the underlying gap and re-test before calling it closed.

SCENARIO You discover a model version reached production without going through the approval gate.

IF ASKED, SAY THIS
Treat it as a compliance incident, not just a process hiccup — report it up immediately rather than quietly fixing it and moving on, because in a regulated environment an unapproved model having served live decisions is exactly the kind of thing that needs to be in the incident r

Treat it as a compliance incident, not just a process hiccup — report it up immediately rather than quietly fixing it and moving on, because in a regulated environment an unapproved model having served live decisions is exactly the kind of thing that needs to be in the incident record even if the model itself turned out to be fine. Determine the exposure window and what decisions it made. Find the control gap that let it happen — a pipeline misconfiguration that skipped the gate, a manual deploy that bypassed CI/CD entirely, or a gate that exists on paper but isn't actually technically enforced (see §6's segregation-of-duties question). Fix the control, not just this one instance, and make sure the fix is verified to actually block the same bypass next time.

SCENARIO Overnight cost alert: a SageMaker endpoint's spend tripled with no obvious traffic change.

IF ASKED, SAY THIS
Check instance count first — a stuck or misfiring autoscaling policy scaling out and never back down is the most common cause, sometimes triggered by a noisy scaling metric rather than real load. Check whether a deploy changed the instance type to something larger without an acco

Check instance count first — a stuck or misfiring autoscaling policy scaling out and never back down is the most common cause, sometimes triggered by a noisy scaling metric rather than real load. Check whether a deploy changed the instance type to something larger without an accompanying reason. Rule out a duplicate/orphaned endpoint left running from a test or a previous deployment that never got cleaned up — easy to miss and a surprisingly common source of "mystery" cost. Once identified, fix the immediate cost bleed, then look at whether better scaling-policy tuning or a scheduled scale-down for predictable low-traffic windows would prevent a repeat, rather than treating this as a one-off.

SCENARIO A critical payment service is throwing 500 errors. Walk me through your approach.

IF ASKED, SAY THIS
Structure matters more than speed in a P1. T+0 Establish facts: Open Grafana: error rate %, affected endpoints, traffic volume. Splunk: index=prod service=payments status>=500 | timechart count by endpoint Is it 100% of requests or a subset? Specific endpoint? Specific AZ? T+5 Na

Structure matters more than speed in a P1.

T+0 Establish facts:
Open Grafana: error rate %, affected endpoints, traffic volume.
Splunk: index=prod service=payments status>=500 | timechart count by endpoint
Is it 100% of requests or a subset? Specific endpoint? Specific AZ?

T+5 Narrow scope:
Check Harness for any deployment in the last 2 hours.
Check AWS Health Dashboard and ServiceNow CHG calendar.
DB metrics: connection pool, query latency, deadlocks.
Third-party: payment gateway status page?

T+10 Mitigate:
Deploy timing correlates -> rollback via Harness (one click).
DB connection issue -> restart pool, kill long-running queries.
Traffic spike -> kubectl scale deployment/payments --replicas=15
Feature flag -> disable the affected payment flow.

T+20 Confirm resolution:
Error rate returns to baseline in Grafana.
Splunk alert clears. Manual transaction test passes.

T+30 Document:
Screenshot Grafana and Splunk for RCA evidence.
Raise PRB in ServiceNow. Update CHG to Implemented.

SCENARIO Production Kubernetes pod is stuck in Pending state. How do you debug it?

IF ASKED, SAY THIS

Step 1 - Check pod events kubectl describe pod POD-NAME -n NAMESPACE Look at Events section at the bottom. Common messages: - 0/3 nodes available: insufficient memory -> resource issue - didn't match pod affinity rules -> scheduling constraint - no nodes matched the n

Step 1 - Check pod events
kubectl describe pod POD-NAME -n NAMESPACE
Look at Events section at the bottom.
Common messages:
- 0/3 nodes available: insufficient memory -> resource issue
- didn't match pod affinity rules -> scheduling constraint
- no nodes matched the node selector -> wrong label on node

Step 2 - Check node capacity
kubectl get nodes
kubectl describe node NODE-NAME | grep -A 10 'Allocated resources'
Is the node at 100% memory or CPU requests?

Step 3 - Check resource requests on the pod
kubectl get pod POD-NAME -o yaml | grep -A 5 resources
Is the requested memory higher than any node can offer?

Step 4 - Check PersistentVolumeClaims
kubectl get pvc -n NAMESPACE
If PVC is Pending -> check StorageClass, check if EBS volume available in the right AZ.

Step 5 - Check Taints and Tolerations
kubectl get nodes -o custom-columns=NAME:.metadata.name,TAINTS:.spec.taints
If nodes are tainted, pod needs matching toleration.

Common fixes:
- Add nodes to the cluster (Cluster Autoscaler should do this automatically)
- Reduce resource requests if they are over-provisioned
- Fix PVC binding by checking StorageClass and AZ alignment

SCENARIO Your application logs show 'connection refused' errors to the database. What do you check?

IF ASKED, SAY THIS

Systematic check from application to database: Step 1 - Is the DB instance actually running? AWS Console or CLI: aws rds describe-db-instances --query 'DBInstances[*].DBInstanceStatus' Expected: available. If modifying or rebooting -> wait. Step 2 - Is the application using the r

Systematic check from application to database:

Step 1 - Is the DB instance actually running?
AWS Console or CLI: aws rds describe-db-instances --query 'DBInstances[*].DBInstanceStatus'
Expected: available. If modifying or rebooting -> wait.

Step 2 - Is the application using the right endpoint?
Check app config/environment variables: DB_HOST, DB_PORT.
For RDS: use the cluster endpoint, not a replica endpoint for writes.
Check if a recent deployment changed the environment variable.

Step 3 - Network connectivity
From the application pod:
kubectl exec -it POD-NAME -- nc -zv DB-ENDPOINT 5432
If this fails:
- Check Security Group: does the DB SG allow inbound on port 5432 from the app SG?
- Check NACLs on the DB subnet.
- Check VPC routing: is the app in a different VPC without peering?

Step 4 - Connection pool exhaustion
Check RDS CloudWatch metric: DatabaseConnections.
If at max_connections limit: application is not releasing connections.
Short-term fix: restart the application pods to release stale connections.
Long-term fix: fix the connection leak in code, or add PgBouncer.

Step 5 - RDS failover just happened?
Check RDS Events in CloudWatch for recent failover.
DNS TTL may still be pointing to old primary.
Force DNS refresh: restart application pods.

SCENARIO How do you troubleshoot a memory leak in a production Java application?

IF ASKED, SAY THIS

Memory leaks in Java typically show as gradually increasing heap usage followed by OOM kill. Detection signals: - Grafana: JVM heap used metric rising consistently over hours - Pod restart count increasing (exit code 137 = OOM kill) - GC logs showing longer and longer Full GC pau

Memory leaks in Java typically show as gradually increasing heap usage followed by OOM kill.

Detection signals:
- Grafana: JVM heap used metric rising consistently over hours
- Pod restart count increasing (exit code 137 = OOM kill)
- GC logs showing longer and longer Full GC pauses
- kubectl describe pod POD-NAME shows OOMKilled in last state

Immediate mitigation:
Rolling restart to buy time: kubectl rollout restart deployment/myapp
This restores service but does not fix the root cause.

Investigation:

1. Capture a heap dump before OOM kill:
Add JVM flag: -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp/
kubectl cp POD-NAME:/tmp/heapdump.hprof ./heapdump.hprof

2. Analyse with Eclipse Memory Analyzer (MAT) or VisualVM:
- Look for the top retained heap object
- Common culprits: unbounded cache, event listener not deregistered,
static collection growing forever, ThreadLocal not cleared

3. Compare two heap dumps 30 minutes apart:
Objects growing between dumps = the leak.

4. Check GC logs:
-Xlog:gc*:file=/tmp/gc.log:time,uptime
Increasing Full GC frequency = heap filling up faster than it is cleared.

Long-term fix:
- Fix the leak in code
- Set K8s memory limit = JVM max heap + 200MB overhead (prevent OOM of the node)
- Add Grafana alert: heap usage above 80% for more than 10 minutes

SCENARIO You notice a spike in 429 Too Many Requests errors from a downstream API. How do you handle it?

IF ASKED, SAY THIS

429 means you are being rate-limited by the downstream service. Structured response: Immediate triage: - How long has this been happening? Check Splunk timechart. - Which service is sending too many requests? Check request rate per pod. - Is this a sudden traffic spike or a gradu

429 means you are being rate-limited by the downstream service. Structured response:

Immediate triage:
- How long has this been happening? Check Splunk timechart.
- Which service is sending too many requests? Check request rate per pod.
- Is this a sudden traffic spike or a gradual drift?

Short-term mitigation:
1. Implement exponential backoff with jitter in the calling code:
Retry-After header tells you how long to wait.
Respect it. Do not retry immediately.

2. Add a circuit breaker:
After N consecutive 429s, open the circuit.
Stop hammering the downstream. Serve cached response or graceful degradation.

3. Scale down the number of callers if you have many pods all calling the same API.

Root cause investigation:
- Check if a recent code change added an extra API call per transaction.
- Check if a background job is calling the API in a tight loop without rate limiting.
- Check if the downstream raised their rate limit threshold without notice.

Long-term fixes:
- Implement request rate limiting at the client side using a token bucket.
- Cache downstream responses where the data allows it (TTL-based).
- Contact the downstream provider to increase your rate limit quota.
- Add a Grafana alert on upstream 429 count to catch this earlier next time.

SCENARIO Walk me through how Kubernetes Services actually route traffic.

IF ASKED, SAY THIS

ClusterIP is a stable virtual IP reachable only inside the cluster — kube-proxy programs iptables or IPVS rules on every node to DNAT it to a healthy pod. NodePort opens the same thing on a high port on every node, on top of a ClusterIP. LoadBalancer provisions a cloud load balan

ClusterIP is a stable virtual IP reachable only inside the cluster — kube-proxy programs iptables or IPVS rules on every node to DNAT it to a healthy pod. **NodePort** opens the same thing on a high port on every node, on top of a ClusterIP. **LoadBalancer** provisions a cloud load balancer in front of that NodePort. **Headless** (`clusterIP: None`) skips the virtual IP entirely, DNS returns pod IPs directly, which is what StatefulSets need for stable per-pod addressing. The Service selects pods by label; matching, Ready pods land in the Endpoints (or EndpointSlice) object — that's the actual debugging pivot most people miss.

Remember: ClusterIP → NodePort → LoadBalancer is a stack, each building on the last · Headless exists specifically for StatefulSets needing per-pod DNS identity · `kubectl get endpoints <svc>` being empty = selector typo or pods failing readiness — check this before anything else

- [ ] SCENARIO Ingress vs a service mesh — what problem does each actually solve?

IF ASKED, SAY THIS
They solve different axes of traffic, and conflating them is the mistake to avoid. Ingress is north-south : the entry point for external HTTP(S) traffic into the cluster, and an Ingress controller (nginx, ALB controller) watches Ingress resources and programs routing, host/path r

They solve different axes of traffic, and conflating them is the mistake to avoid. Ingress is **north-south**: the entry point for external HTTP(S) traffic into the cluster, and an Ingress controller (nginx, ALB controller) watches Ingress resources and programs routing, host/path rules, TLS termination, one load balancer fronting many services. A service mesh (Istio/Linkerd) is **east-west**: service-to-service traffic inside the cluster, mutual TLS everywhere, retries/timeouts/circuit-breaking, fine-grained traffic splitting for canaries, and uniform per-service telemetry, all without touching application code. A mesh adds real operational cost, another hop, another thing to upgrade and debug, so the senior answer names that tradeoff: for a handful of services, Ingress plus NetworkPolicies plus app-level retries is leaner; adopt a mesh once you have enough services that manual mTLS and traffic management stop being manageable by hand.

Remember: North-south (Ingress) vs east-west (mesh) — say these words, they signal the taxonomy · Naming the mesh's operational cost unprompted is the senior signal, not just listing its features · 'No Ingress controller installed' is a classic trap — Ingress objects do nothing without one

- [ ] SCENARIO DNS inside the cluster stops resolving for one namespace — others are fine. What's your process?

IF ASKED, SAY THIS
Start with a debug pod in the affected namespace, kubectl run tmp --image=busybox -it -- sh , and try resolving from inside it; that isolates whether it's namespace-scoped or actually cluster-wide and the blast radius was misjudged. Check for a recently-added NetworkPolicy blocki

Start with a debug pod in the affected namespace, `kubectl run tmp --image=busybox -it -- sh` , and try resolving from inside it; that isolates whether it's namespace-scoped or actually cluster-wide and the blast radius was misjudged. Check for a recently-added **NetworkPolicy** blocking egress to CoreDNS on port 53, the most common cause when it's isolated to one namespace. Then check the CoreDNS pods themselves and their ConfigMap for a bad edit, and check namespace-level resource quotas. If a sidecar (Istio) was recently injected into that namespace, check whether its network config is intercepting DNS traffic unexpectedly.

Remember: Debug pod first — confirms scope before theorizing about cause · A new NetworkPolicy blocking port 53 egress is the single most common cause of 'DNS broke for one namespace' · Namespace-scoped symptom → look for something namespace-scoped (policy, quota, injection), not a cluster-wide CoreDNS problem

- [ ] SCENARIO What does a NetworkPolicy actually do, and what's the default behavior without one?

IF ASKED, SAY THIS
With zero NetworkPolicies applied, Kubernetes networking is flat and fully open , any pod can reach any other pod, no isolation at all, which surprises people coming from a traditional firewalled network model. A NetworkPolicy is additive and selector-based: the moment any policy

With zero NetworkPolicies applied, Kubernetes networking is **flat and fully open**, any pod can reach any other pod, no isolation at all, which surprises people coming from a traditional firewalled network model. A NetworkPolicy is additive and selector-based: the moment any policy selects a pod, that pod's traffic becomes default-deny for whatever direction the policy covers, and only explicitly allowed traffic gets through. The classic mistake is writing a policy that only defines ingress rules while assuming egress is still open, or vice versa — both directions need to be explicit if isolation is actually the goal.

Remember: Default with no policy = fully open, not locked down — say this explicitly, it's the part people assume backwards · A policy is default-deny *only for the traffic direction and pods it selects* · Forgetting to cover egress (or ingress) is the most common real-world NetworkPolicy bug

- [ ] SCENARIO Service A can't communicate with Service B — what's your systematic troubleshooting order?

IF ASKED, SAY THIS
Endpoints first, kubectl get endpoints service-b ; empty means the label selector doesn't match any Ready pod, the single most common root cause, and it rules out half the possibilities in ten seconds. If endpoints exist, test from inside the cluster with a debug pod to isolate D

Endpoints first, `kubectl get endpoints service-b` ; empty means the label selector doesn't match any Ready pod, the single most common root cause, and it rules out half the possibilities in ten seconds. If endpoints exist, test from inside the cluster with a debug pod to isolate DNS resolution from actual connectivity, `nslookup service-b` then `curl service-b:port` as two separate checks. If DNS resolves but the connection fails, check NetworkPolicy next, then whether a service mesh sidecar was injected on one side but not the other, mTLS mismatch causes exactly this symptom. Only after ruling all of that out do you look at the application itself.

Remember: Endpoints → DNS → connectivity → NetworkPolicy → mesh/mTLS → app — a fixed order beats jumping around · Separating 'DNS resolves' from 'connection succeeds' as two different checks is what shows methodology · Asymmetric mesh injection (one side has a sidecar, one doesn't) is a real, easy-to-miss cause

SCENARIO Metrics vs logs vs traces — and where does each of Prometheus, Grafana, and Loki actually fit?

IF ASKED, SAY THIS

The three pillars solve different questions. Metrics are cheap numeric time series, best for 'is it healthy right now,' dashboards and alerts. Logs are discrete detailed events, best for 'what exactly happened.' Traces follow one request across every service it touched, best for

The three pillars solve different questions. **Metrics** are cheap numeric time series, best for 'is it healthy right now,' dashboards and alerts. **Logs** are discrete detailed events, best for 'what exactly happened.' **Traces** follow one request across every service it touched, best for 'where is the latency, which service actually failed.' Mapped to tools: **Prometheus stores and queries metrics**. **Grafana doesn't store anything, it visualizes**, querying Prometheus for metrics, Loki for logs, and Tempo/Jaeger for traces, often on the same dashboard. **Loki stores logs**, and deliberately indexes only labels, not full text, so it stays cheap at scale, the opposite tradeoff from something like Elasticsearch.

Remember: Say it as a clean equation: Prometheus = metrics store, Loki = logs store, Grafana = visualizes *both* (and traces) · Grafana is a rendering layer, not a data store — this is exactly the distinction that got marked down · Loki's 'index labels only, not full text' design choice is worth naming — it's why it's cheap

SCENARIO A Grafana dashboard suddenly shows 'No Data' on every panel — how do you fix it?

IF ASKED, SAY THIS

Check the data source connection first, Settings → Data Sources → Test, a URL, auth, or network change breaks every panel at once, the signature of a data-source problem rather than a query problem. If that's healthy, confirm the underlying source actually has data for the select

Check the data source connection first, Settings → Data Sources → Test, a URL, auth, or network change breaks every panel at once, the signature of a data-source problem rather than a query problem. If that's healthy, confirm the underlying source actually has data for the selected time range: a migrated Prometheus won't have backfilled history, so a dashboard defaulting to 'last 6 hours' can look empty even though everything's technically working. Only after both check out do you look at the panel queries themselves for a broken PromQL/LogQL expression, often from a label that got renamed upstream.

Remember: Every panel dead at once → data source problem, not a query problem — check that first, not last · Time range mismatch after a migration is a very common false alarm · Individual panel(s) dead while others work → that's a query problem, a different branch of investigation

SCENARIO CPU crosses 95% but no alert fires — why, and what do you check?

IF ASKED, SAY THIS

Work backwards through the alerting pipeline instead of guessing: does the alerting rule actually exist and is the PromQL expression correct, test it directly in the Prometheus UI, don't assume. If it evaluates correctly, check the for: duration; a spike that didn't sustain long

Work backwards through the alerting pipeline instead of guessing: does the alerting rule actually exist and is the PromQL expression correct, test it directly in the Prometheus UI, don't assume. If it evaluates correctly, check the `for:` duration; a spike that didn't sustain long enough never leaves Pending state. If it fired in Prometheus, check Alertmanager received it, a label mismatch between the alert and the route's matchers means it arrives but never gets routed anywhere. Last, check the notification channel itself, an expired PagerDuty/Slack integration token is a boring but common dead end.

Remember: Work the pipeline backwards: rule exists? → evaluates true? → for: duration met? → reached Alertmanager? → routed? → delivered? · A route label mismatch is a silent failure — the alert fires but nobody ever sees it, and nothing errors · This investigation order is reusable for almost any 'alert should have fired but didn't' question

SCENARIO A production Linux server suddenly becomes slow — what's your systematic method?

IF ASKED, SAY THIS

Use a repeatable method instead of randomly running commands, Brendan Gregg's USE method (Utilization, Saturation, Errors per resource) is the framework worth naming by name. A fast 60-second triage: uptime for the load trend, top / htop for the biggest CPU/memory consumers, vmst

Use a repeatable method instead of randomly running commands, Brendan Gregg's USE method (Utilization, Saturation, Errors per resource) is the framework worth naming by name. A fast 60-second triage: `uptime` for the load trend, `top` / `htop` for the biggest CPU/memory consumers, `vmstat 1` for run queue and I/O wait, `free -m` for memory (watch 'available,' not 'free'), `iostat -x 1` for disk saturation, `ss -s` for connection counts, and `dmesg` / `journalctl -k` for kernel-level errors like OOM kills. Then drill into whichever resource is actually saturated, and correlate the start time against a recent deploy or traffic change, that correlation is usually the fastest path to the real cause.

Remember: Name the **USE method** explicitly — that's the methodology signal, not just the commands · The fixed command order (uptime → top → vmstat → free → iostat → ss → dmesg) is itself the answer · Always end with 'correlate against a recent deploy' — that's the step people skip

SCENARIO Load average is 8 but CPU usage is only 30% on a 4-core box — what's going on?

IF ASKED, SAY THIS

Load average counts processes that are either running or waiting to run, plus anything stuck in uninterruptible sleep, which on Linux specifically includes processes blocked on disk I/O. On 4 cores, load 4 means fully busy; load 8 means twice as much runnable work queued as there

Load average counts processes that are either running or waiting to run, plus anything stuck in uninterruptible sleep, which on Linux specifically includes processes blocked on disk I/O. On 4 cores, load 4 means fully busy; load 8 means twice as much runnable work queued as there is CPU to run it. But if CPU usage is only 30%, the processes aren't CPU-bound, they're blocked, almost always on **disk I/O**. Confirm with `vmstat 1`, watching the `b` (blocked) and `wa` (I/O wait) columns, then `iostat -x 1` for `await` and `%util` per device to find the actual slow disk.

Remember: Linux uniquely counts uninterruptible-sleep (disk-blocked) processes toward load — the one-line explanation · High load + low CPU = I/O bound, almost every time — say this pattern-match directly · `vmstat`'s b/wa columns and `iostat`'s await/%util are the confirming evidence, not just a guess

---

- [ ] SCENARIO Root filesystem hits 100% on a production box — what do you do?

IF ASKED, SAY THIS
df -h confirms which filesystem, then du -h --max-depth=1 /path | sort -rh walks down to find what's actually large, /var/log is the usual suspect, an oversized or un-rotated log. If it's the systemd journal specifically, journalctl --disk-usage shows the size and journalctl --va

`df -h` confirms which filesystem, then `du -h --max-depth=1 /path | sort -rh` walks down to find what's actually large, `/var/log` is the usual suspect, an oversized or un-rotated log. If it's the systemd journal specifically, `journalctl --disk-usage` shows the size and `journalctl --vacuum-size=500M` trims it immediately. Worth knowing separately: disk can show 100% *inode* usage while space looks fine, from huge numbers of tiny files, `df -i` catches that case, which `df -h` alone won't show.

Remember: `df -h` → `du --max-depth=1 | sort -rh` → drill into the specific directory — don't grep blindly · `journalctl --vacuum-size` is the fast, safe first move if the journal is the culprit · Mention inode exhaustion (`df -i`) as the case disk-usage-by-size alone won't catch — it's the detail that shows depth

---

- [ ] SCENARIO A service is down right after a server reboot — how do you debug it?

IF ASKED, SAY THIS
systemctl status first for the immediate state and the last few log lines. If it's not enabled to start on boot at all, that's the whole answer right there, systemctl is-enabled confirms. If it is enabled but failed, journalctl -u -b scopes the log t

`systemctl status <service>` first for the immediate state and the last few log lines. If it's not enabled to start on boot at all, that's the whole answer right there, `systemctl is-enabled <service>` confirms. If it is enabled but failed, `journalctl -u <service> -b` scopes the log to just this boot, which matters because you don't want to be reading yesterday's log entries by mistake. Common reboot-specific causes: a dependency service, a database, a network mount, wasn't up yet when this one tried to start, since systemd's ordering is only as good as the unit file's `After=` / `Requires=` directives being correct.

Remember: The `-b` flag on journalctl scopes to the current boot — easy to forget and easy to misread stale logs without it · 'Not enabled to start on boot' is a real, common, embarrassingly simple answer — check it first · Dependency ordering (After=/Requires=) is the senior-level cause behind most reboot-specific failures

---

- [ ] SCENARIO Server reports Out Of Memory — walk me through it.

IF ASKED, SAY THIS
Check dmesg or journalctl -k for the OOM killer's own log line, it names the exact process it killed and why. The kernel scores every process, influenced by memory usage and oom_score_adj , and kills whichever scores highest when it can't satisfy an allocation and has nothing lef

Check `dmesg` or `journalctl -k` for the OOM killer's own log line, it names the exact process it killed and why. The kernel scores every process, influenced by memory usage and `oom_score_adj`, and kills whichever scores highest when it can't satisfy an allocation and has nothing left to reclaim. In a containerized environment, the fix isn't tweaking `oom_score_adj` per process, it's setting proper memory requests/limits so Kubernetes manages pressure proactively before the kernel-level OOM killer ever has to step in, and right-sizing workloads so a node doesn't approach exhaustion in the first place.

Remember: `dmesg` / `journalctl -k` shows exactly what got killed and why — always check the actual log before theorizing · In Kubernetes specifically, proper requests/limits prevent this proactively — that's the senior answer, not oom_score_adj tuning · Exit code 137 on a container is this same OOM kill, just observed from the orchestrator side

---

- [ ] SCENARIO Quick reference — when do you reach for each Linux tool?

IF ASKED, SAY THIS
vmstat 1 — repeating system-wide snapshot of CPU, memory, swap, and run queue, your first command for 'is anything blocked.' iostat -x 1 — disk I/O throughput and latency per device, when storage is the suspect. sar — like vmstat/iostat but with historical data collection, useful

`vmstat 1` — repeating system-wide snapshot of CPU, memory, swap, and run queue, your first command for 'is anything blocked.' `iostat -x 1` — disk I/O throughput and latency per device, when storage is the suspect. `sar` — like vmstat/iostat but with historical data collection, useful for 'what was happening at 3am last night' after the fact. `journalctl` — the systemd log query tool, `-u service` to scope by unit, `-f` to tail live, `-b` to scope to the current boot. `lsof` — lists open files and sockets per process, the answer to 'device busy' errors or finding what's holding a deleted file's disk space. `ss -tulnp` — what's actually listening on which port, the modern replacement for netstat. `strace` — traces the exact system calls a hung or erroring process is making, for when you need to see precisely what it's stuck on.

Remember: This is the exact vocabulary flagged as missing — work these words into scenario answers naturally, don't just recite the list · `sar`'s distinguishing feature is *historical* data — that's what differentiates it from vmstat/iostat · strace is for a hung/erroring single process; the others are system-wide snapshots —

know which tier you're in

SCENARIO How does service discovery actually work in a microservices/Kubernetes environment?

IF ASKED, SAY THIS

In Kubernetes specifically, service discovery is DNS-based and comes for free: every Service gets a stable DNS name (service-name.namespace.svc.cluster.local), CoreDNS resolves it, and kube-proxy's iptables/IPVS rules route the resulting ClusterIP to a healthy pod, no separate

In Kubernetes specifically, service discovery is DNS-based and comes for free: every Service gets a stable DNS name (`service-name.namespace.svc.cluster.local`), CoreDNS resolves it, and kube-proxy's iptables/IPVS rules route the resulting ClusterIP to a healthy pod, no separate service registry like Consul or Eureka needed unless you're outside Kubernetes or need cross-cluster discovery. The Endpoints/EndpointSlice object actually tracks which pods are currently Ready and eligible to receive traffic; a pod failing its readiness probe drops out of Endpoints automatically without anyone updating a registry by hand.

Remember: Kubernetes gives you discovery via DNS + Endpoints — no Consul/Eureka needed inside the cluster · Readiness probes are what keeps the discovery mechanism accurate — a failing probe self-removes a pod from rotation · Outside Kubernetes (VMs, hybrid), that's when a real service registry (Consul, Eureka) earns its place

SCENARIO A downstream service is slow and it's cascading failures back to your callers — how do you implement a circuit breaker?

IF ASKED, SAY THIS

The pattern: after enough consecutive failures or slow responses from a downstream, stop even attempting calls to it for a cooldown window, and fail fast instead, that protects your own service's threads/connections from being tied up waiting on something already in trouble. In I

The pattern: after enough consecutive failures or slow responses from a downstream, stop even attempting calls to it for a cooldown window, and fail fast instead, that protects your own service's threads/connections from being tied up waiting on something already in trouble. In Istio this is configured declaratively on a DestinationRule: `outlierDetection` ejects a host after N consecutive 5xx errors, and `connectionPool` limits keep pending requests from piling up. Pair it with a real fallback in the app, return cached data or a sane default when the circuit's open, because failing fast is only half the answer; failing *usefully* is the other half.

Remember: The core idea in one line: fail fast on a known-bad dependency instead of queueing behind it · If asked to configure it: outlierDetection (ejection) + connectionPool (pending-request limits) on a DestinationRule · A circuit breaker without an app-level fallback just turns a slow failure into a fast one — mention the fallback

SCENARIO What's the actual danger in adding retries to service-to-service calls?

IF ASKED, SAY THIS

Retries without limits turn one slow dependency into a retry storm , every caller retries, multiplying load on an already-struggling service and making the outage worse, not better. The fix is layered: exponential backoff with jitter so retries don't all land in sync, a retry bud

Retries without limits turn one slow dependency into a **retry storm**, every caller retries, multiplying load on an already-struggling service and making the outage worse, not better. The fix is layered: exponential backoff with jitter so retries don't all land in sync, a retry budget so a service caps total retry volume rather than retrying unboundedly, and, the detail most people miss, only retry **idempotent** operations. Retrying a POST that already partially succeeded server-side is exactly how you get duplicate records, the same failure mode as the SQL duplicate-records question, just triggered from the network layer instead of the database layer.

Remember: Retry storms are the real risk — 'just add retries' is a naive answer without backoff + budget · Idempotency is the gate: retry GETs freely, retry POSTs only if they're built to be safely repeated · Connecting this back to the SQL duplicate-records question is a strong callback if it comes up again

SCENARIO How does distributed tracing actually help you find where latency is coming from?

IF ASKED, SAY THIS

A trace follows one request as it hops across every service it touches, and each hop is a span with its own start/end time, nested under the parent. When a request is slow, you pull that one trace and look for the longest span, that tells you exactly which service, or which speci

A trace follows one request as it hops across every service it touches, and each hop is a span with its own start/end time, nested under the parent. When a request is slow, you pull that one trace and look for the longest span, that tells you exactly which service, or which specific call inside a service, a DB query, an external API, owns the latency, instead of guessing from each service's aggregate metrics separately. The prerequisite most people forget: trace context has to propagate across every hop, if one service in the chain doesn't forward the trace headers, the trace breaks into disconnected fragments and the full picture is lost.

Remember: The workflow: find a slow trace → find the longest span → that's your culprit service/call · Trace context propagation across every hop is the prerequisite — a broken link anywhere breaks the whole trace · This is the tool for 'where' when metrics already told you 'that something is slow'

SCENARIO What does a service mesh add over plain Kubernetes networking, and when is it actually worth it?

IF ASKED, SAY THIS

A mesh injects a sidecar proxy beside every pod to handle service-to-service traffic transparently: automatic mutual TLS between every service with no app code changes, fine-grained traffic control, canary splits, retries, timeouts, circuit breaking, decoupled from application de

A mesh injects a sidecar proxy beside every pod to handle service-to-service traffic transparently: automatic mutual TLS between every service with no app code changes, fine-grained traffic control, canary splits, retries, timeouts, circuit breaking, decoupled from application deployments, and rich per-service telemetry for free. The honest tradeoff: it's another hop per call, another thing to upgrade, and another layer to debug when something breaks, mTLS failures showing up as generic 'connection reset' errors is a real, annoying category of its own. The senior answer names that cost explicitly and ties adoption to actual need, enough services that manual TLS and traffic management by hand has become unmanageable, rather than treating a mesh as a default best practice.

Remember: List the real benefits (mTLS, traffic control, telemetry) — but always pair with the real cost · 'Another hop, another upgrade surface, another debugging layer' is the tradeoff line to have ready · Tie adoption to a concrete trigger (service count / compliance need), not 'because it's modern'

SCENARIO Blue-green vs canary — what's the actual difference and when do you pick each?

IF ASKED, SAY THIS
Blue-green stands up the new version fully, alongside the old one, running at full scale, then flips traffic over in one shot once health checks pass, instant cutover, instant rollback by flipping back, but it needs roughly double the capacity while both versions run. Canary send

Blue-green stands up the new version fully, alongside the old one, running at full scale, then flips traffic over in one shot once health checks pass, instant cutover, instant rollback by flipping back, but it needs roughly double the capacity while both versions run. Canary sends a small percentage of real traffic to the new version, watches real metrics, then ramps gradually or aborts, it's the better risk control because the promote/abort decision can be automated directly off error rate and latency, catching a bad release before it ever sees full traffic. Canary needs good observability and traffic-splitting to do well; blue-green is simpler to reason about but a scale-only bug can slip through without also ramping its traffic gradually.

Remember: Blue-green = instant cutover, ~2x capacity, simpler mental model · Canary = gradual, automatable on real metrics, the stronger default for risk control · Whichever you pick, pair it with fast tested rollback — non-negotiable either way

---

SCENARIO A Jenkins pipeline that normally takes 10 minutes suddenly takes 40 — how do you investigate?

IF ASKED, SAY THIS
Don't guess, open the stage timing breakdown first and find which specific stage grew, rather than assuming it's 'the build' generally. Common causes in rough order of likelihood: a dependency cache got invalidated or evicted (npm/Maven/pip pulling fresh instead of cached), a sha

Don't guess, open the stage timing breakdown first and find which specific stage grew, rather than assuming it's 'the build' generally. Common causes in rough order of likelihood: a dependency cache got invalidated or evicted (npm/Maven/pip pulling fresh instead of cached), a shared Jenkins agent is resource-starved because too many jobs landed on it at once, an external dependency, an artifact repo, a container registry, is responding slowly and a stage is silently waiting on it, or a new test was added that isn't parallelized with the rest. If it's a recurring Monday-morning pattern specifically, that's usually scheduled batch jobs all queuing onto the same agent pool at once, not a one-off fluke.

Remember: Find the specific stage that grew — never diagnose 'the pipeline' as one undifferentiated blob · Cache invalidation and agent resource contention are the two most common real causes, in that order · A day-of-week pattern (e.g. every Monday) points at scheduling/contention, not a code or infra regression

---

SCENARIO How do you actually speed up a slow pipeline, concretely?

IF ASKED, SAY THIS
Profile first to find the real bottleneck, don't optimize by guessing. Then: cache dependencies and Docker layers between runs, order your Dockerfile so rarely-changing layers (base image, dependency install) come before frequently-changing ones (your actual source code), and use

Profile first to find the real bottleneck, don't optimize by guessing. Then: cache dependencies and **Docker layers** between runs, order your Dockerfile so rarely-changing layers (base image, dependency install) come before frequently-changing ones (your actual source code), and use multi-stage builds so build tooling never ships in the final image. Parallelize independent stages and shard tests across workers. In a monorepo, build and test only what actually changed rather than everything every time. Adopt 'build once, promote everywhere,' build one artifact/image and promote the same one through staging and prod instead of rebuilding per environment, which removes redundant work and any risk of environment-specific build drift.

Remember: Docker layer ordering (stable layers first, volatile layers last) is the single highest-leverage caching detail · 'Build once, promote everywhere' eliminates a whole category of redundant rebuilds — name it by name · Profile before optimizing — guessing at the bottleneck is the wrong first move even under time pressure

---

SCENARIO A release just went bad in production — what's the fastest path to rollback?

IF ASKED, SAY THIS
The fastest mitigation is almost never 're-run the pipeline backwards,' it's reverting state directly. With GitOps (Argo CD), that's argocd app rollback to the last known-good synced revision, seconds, no pipeline re-run needed. Without GitOps, it's redeploying the previous known

The fastest mitigation is almost never 're-run the pipeline backwards,' it's reverting state directly. With GitOps (Argo CD), that's `argocd app rollback` to the last known-good synced revision, seconds, no pipeline re-run needed. Without GitOps, it's redeploying the previous known-good image tag or task definition directly, since every image should already be tagged with its Git SHA rather than 'latest,' that previous tag is always sitting right there ready to redeploy. `git revert` on the source is the correct *follow-up* so the bad change doesn't quietly resurface on the next deploy, but it's not the fast mitigation path, the fast path is reverting the running state first, understanding second.

Remember: Fast mitigation = revert running state (argocd rollback / previous image tag); git revert is the follow-up, not the first move · SHA-based image tagging is exactly what makes 'redeploy the previous version' a one-line action instead of a rebuild · This is the same 'roll back first, understand later' principle from the MTTR material — keep the story consistent

SCENARIO How would you add automated deployment gates so a bad release can't fully roll out?

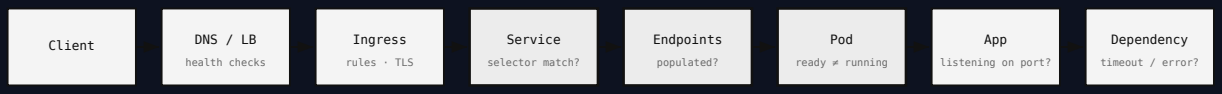
IF ASKED, SAY THIS
Gate each promotion step on real metrics instead of just time elapsed: canary at 10% traffic, an automated analysis step queries error rate and latency against the baseline for a fixed window, and only promotes to 50% then 100% if it passes, auto-abort and roll back if it doesn't

Gate each promotion step on real metrics instead of just time elapsed: canary at 10% traffic, an automated analysis step queries error rate and latency against the baseline for a fixed window, and only promotes to 50% then 100% if it passes, auto-abort and roll back if it doesn't. Argo Rollouts is the concrete tool for this on Kubernetes: it defines the traffic steps and wires an AnalysisTemplate straight to a Prometheus query as the pass/fail gate. Pair this with a broader policy-level gate too: block deployment entirely if the service's error budget is already mostly consumed, so a team can't ship a risky change into a service that's already having a bad week.

Remember: The gate should be a real metrics query against a baseline, not a fixed sleep timer · Argo Rollouts + AnalysisTemplate + Prometheus query is the concrete implementation to name if pressed · An error-budget-aware deployment gate is the senior-level addition beyond basic canary analysis

SCENARIO An AKS/EKS application suddenly starts returning 503 — walk the full troubleshooting path, not just "check the pods."

IF ASKED, SAY THIS
In one lineFollow the actual request path — client → DNS/LB → ingress → service → endpoints → pod → app → dependency — because "pods are running" only answers one hop out of eight.



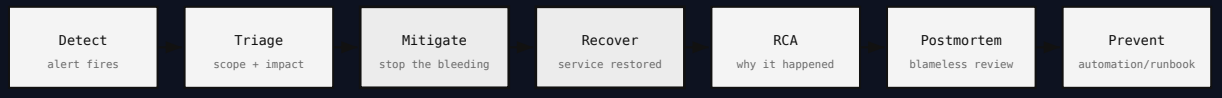
shaded = where a "503 but everything looks fine" usually actually lives

Correlate the exact timestamp against Prometheus metrics and Splunk logs at every hop, not just the pod layer.

- Establish the exact start time first, and whether it's every user, one endpoint, or one region — that scopes the search before touching anything.
- Correlate that timestamp against Grafana metrics, recent deployments, and Splunk logs in parallel with the live investigation, not after.
- At the Kubernetes layer: pod status is necessary but not sufficient — running doesn't mean ready. Check that the Service's selector actually matches the pod labels, and that the Endpoints object is actually populated with healthy pod IPs.
- If pods and endpoints check out, move outward: ingress rules, TLS, and load-balancer target health — a bad ingress rule or an LB health-check misconfiguration produces the exact same symptom as a broken pod.
- If everything upstream is clean, the app itself may not be listening on the expected port, or a downstream dependency (DB, cache, another service) is timing out and the app is surfacing that as a 503.
- Once the cause is identified, stabilize first — rollback, scale, or restore the dependency — and run RCA after the customer is unblocked, not before.

SCENARIO Walk me through your incident management process end to end.

IF ASKED, SAY THIS
In one lineDetect, triage, mitigate, recover, then RCA and postmortem — in that order, because restoring service always outranks finding the perfect root cause in the moment.



immediate priority — restore service before chasing the perfect root cause

RCA and postmortem happen after the customer is unblocked, not instead of it.

- **Detection** — an alert fires, or a customer/monitoring signal surfaces the issue; first move is confirming it's real and establishing scope (one user, one region, everyone).
- **Triage** — assess impact and severity, form initial hypotheses, pull in the right people. Fast, not a deep investigation.
- **Mitigation** — the actual priority: rollback, scale, failover, disable a feature flag, or restore a failed dependency. Stop the customer impact first.
- **Recovery** — confirm the service is genuinely healthy again (SLO recovering, not just the alert clearing) before standing down.
- **RCA** — once stable, dig into the real cause using metrics, logs, deployment history, and dependency behavior.
- **Postmortem** — blameless review, written up with concrete corrective actions and owners, not just a narrative of what happened.

SCENARIO A downstream service starts responding slowly. 20 minutes later, your own service is failing too, and so is everything upstream of you — you never touched your own code. What happened, and how do you prevent it?

IF ASKED, SAY THIS
Classic cascading failure: retries without backoff and no circuit breaker turned one slow dependency into an outage several layers up the call graph.

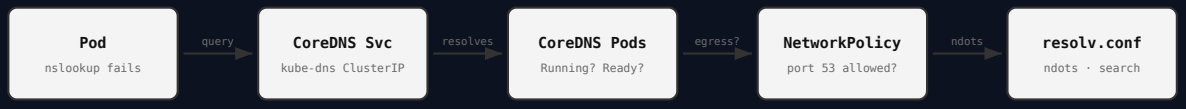


The chain: Service B slows down → callers retry without backoff or a cap → the retry volume (often 2-3x the original load) piles onto B and onto the caller's own thread/connection pool → the pool fills with requests waiting on B → the caller (Service A) now can't serve its OWN healthy requests either → A's callers start retrying too, and the failure propagates upward.

Prevention, in priority order: a circuit breaker that stops calling B once its error/latency rate crosses a threshold (fail fast instead of piling up waiters); bounded retries with exponential backoff and jitter, never unbounded retry loops; timeouts on every outbound call that are shorter than the caller's own SLA budget; and bulkheading — separate connection/thread pools per downstream dependency so one bad dependency can't starve the pool every other call path also uses.

SCENARIO A pod can resolve some in-cluster service names but not others, and only from one namespace. Walk through the diagnostic path.

IF ASKED, SAY THIS
Work outward from the pod: confirm the query even reaches CoreDNS, then check CoreDNS pod health, then NetworkPolicy egress on port 53, then the ndots/search-domain config that decides how the name gets expanded.

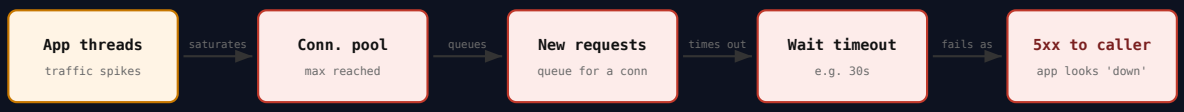


Namespace-scoped failure is the biggest clue — it points at something namespace-specific rather than cluster-wide DNS being broken (if it were fully broken, nothing would resolve anywhere).

- `kubectl exec` into the pod and run `nslookup` / `dig` against the failing name, then against `kubernetes.default` as a control.
- Check the CoreDNS pods are Running and Ready: `kubectl -n kube-system get pods -l k8s-app=kube-dns`.
- Check for a **NetworkPolicy** in the affected namespace that blocks egress to `kube-system` on UDP/TCP 53 — this is the single most common cause of "only this namespace" DNS failures once a team starts locking down NetworkPolicies.
- Check `/etc/resolv.conf` inside the pod for `ndots` and the search-domain list — a short name can resolve differently depending on how many dots it has relative to `ndots`.

SCENARIO Requests start timing out under load and the app logs show it's waiting on the database, but the database itself isn't under heavy CPU/IO load. What's going on?

IF ASKED, SAY THIS
This is almost always connection-pool exhaustion, not database load — the app has more concurrent requests than pool slots, so requests queue for a connection and time out before they ever reach the DB.

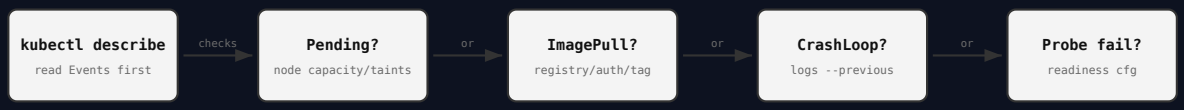


Traffic spikes → the app's configured connection pool (often sized far too conservatively, e.g. 10-20 connections) saturates → new requests queue waiting for a free connection → once the pool's wait-timeout is hit (commonly ~30s), the request fails, and to the caller it looks identical to "the database is slow" even though the DB itself is idle.

Confirm it fast: check the pool's active/idle/waiting metrics (most pool libraries expose these) rather than jumping straight to `EXPLAIN ANALYZE` on the DB. Fix short-term by raising the pool size (bounded by the DB's own `max_connections`); fix properly with a connection pooler in front of the DB (PgBouncer for Postgres, ProxySQL for MySQL) so app-side pool sizing doesn't have to equal DB-side connection budget 1:1.

☐ SCENARIO Give a fast, memorable escalation order for a Pod that's stuck Pending or in CrashLoopBackOff, before diving into any one specific cause. ▲

IF ASKED, SAY THIS
describe first (read Events, don't guess) — then branch: Pending points at scheduling (capacity/taints/affinity), CrashLoopBackOff points at the container itself (check logs --previous, then probes).



- **Always start here:** `kubectl describe pod <pod>` — the Events section at the bottom almost always names the actual blocker (FailedScheduling, ImagePullBackOff, Unhealthy, OOMKilled) so you're not guessing.
- **If Pending:** check node capacity (`kubectl describe node` for allocatable vs requested), taints/tolerations, and node/pod affinity rules — the scheduler couldn't place it somewhere.
- **If ImagePullBackOff:** wrong tag, private registry auth, or the CI pipeline pushed to a different region/registry than the cluster pulls from.
- **If CrashLoopBackOff:** `kubectl logs <pod> --previous` to see why the LAST attempt died, not the current empty one; check exit code via `kubectl describe` (137 = OOMKilled, raise memory limits; 1 = app error, read the log).
- **If it starts but gets killed shortly after:** check readiness/liveness probe config — a liveness probe that's too aggressive (short `initialDelaySeconds`) restarts a pod that just needed more warm-up time.

☐ SCENARIO p99 latency on a checkout API spikes 5x during a traffic surge, but CPU/memory on all pods look normal. Walk through your triage. ▲

IF ASKED, SAY THIS
Since compute resources rule out the obvious cause, go straight to concurrency limits and dependencies: check DB connection pool saturation/wait time first (classic cause of exactly this symptom pattern), then downstream dependency latency (payment gateway, inventory service) — a surge often exposes

Since compute resources rule out the obvious cause, go straight to concurrency limits and dependencies: check DB connection pool saturation/wait time first (classic cause of exactly this symptom pattern), then downstream dependency latency (payment gateway, inventory service) — a surge often exposes a downstream service that was fine at normal load but is now the bottleneck. Check HPA — is it actually scaling, or hitting a max-replica ceiling, or scaling but new pods aren't warm/ready fast enough to help before the surge passes. Pull a trace for one of the actual slow requests rather than reasoning from aggregate dashboards alone — it'll show exactly which hop is eating the time.

☐ SCENARIO A deployment to EKS goes out and 30% of pods immediately fail readiness checks. What's your rollback decision process? ▲

IF ASKED, SAY THIS
30% failing immediately, consistently, is a strong signal of a bad deploy rather than environmental flakiness — that bar (a clear majority-healthy threshold, or any hard failure on a canary subset) should trigger an automatic rollback if you have progressive delivery wired up; if not, this is exactl

30% failing immediately, consistently, is a strong signal of a bad deploy rather than environmental flakiness — that bar (a clear majority-healthy threshold, or any hard failure on a canary subset) should trigger an automatic rollback if you have progressive delivery wired up; if not, this is exactly the "don't debate, roll back now, investigate after" case since a partially-degraded production service is actively hurting users while you investigate.

Roll back first via `kubectl rollout undo` or reverting the GitOps commit, confirm readiness recovers, then pull logs/events from the failed pods afterward to find root cause with the pressure off.

- [ ] SCENARIO Your ELK/OpenSearch cluster hits 85% disk usage and ingestion starts failing. Walk through mitigation and root cause.

IF ASKED, SAY THIS
Immediate: most clusters hit a hard read-only watermark around 90-95% disk — you're close to it, so buy headroom fast by force-deleting the oldest, least-valuable indices per your retention policy (or triggering ILM rollover/deletion manually if it's lagging), not by trying to add capacity first sin

Immediate: most clusters hit a hard read-only watermark around 90-95% disk — you're close to it, so buy headroom fast by force-deleting the oldest, least-valuable indices per your retention policy (or triggering ILM rollover/deletion manually if it's lagging), not by trying to add capacity first since that takes longer than the runway you have left. Once stable: find why disk grew faster than expected — a new high-volume log source that wasn't capacity-planned for, a missing/broken ILM policy that let old indices accumulate past intended retention, or a spike in log volume from an upstream bug (e.g., a service stuck in a tight error-logging loop). Fix the ILM policy gap or add capacity properly, don't just repeat the emergency cleanup next month.

- [ ] SCENARIO An on-call alert fires for "error budget exhausted." Walk through what you do in the next 30 minutes.

IF ASKED, SAY THIS
Error budget exhaustion is a policy trigger, not just an alert — first confirm it's real (not a metric/query bug) by cross-checking against a second signal (actual user reports, a dashboard you trust). If real, the standard SRE response is a feature-freeze on further risky changes to that service un

Error budget exhaustion is a policy trigger, not just an alert — first confirm it's real (not a metric/query bug) by cross-checking against a second signal (actual user reports, a dashboard you trust). If real, the standard SRE response is a feature-freeze on further risky changes to that service until the budget recovers, prioritizing reliability work over new features for that team in the near term. In the moment: identify what's actually burning the budget right now (is it one ongoing incident, or death-by-a-thousand-cuts from many small errors) since those need different responses — one active incident gets normal incident response, chronic burn needs root-cause analysis of the pattern, not a page-and-fix.

- [ ] SCENARIO A Terraform apply in CI partially fails halfway through a multi-resource change. What do you do?

IF ASKED, SAY THIS
Don't panic-retry immediately — first run terraform plan to see the actual current state vs. desired state; Terraform's state file should accurately reflect what did and didn't get created even on a partial failure, so the plan tells you exactly what's left to reconcile. Read the actual error (rate

Don't panic-retry immediately — first run `terraform plan` to see the actual current state vs. desired state; Terraform's state file should accurately reflect what did and didn't get created even on a partial failure, so the plan tells you exactly what's left to reconcile. Read the actual error (rate limiting, a dependency ordering issue, a resource that already exists with a naming conflict) rather than assuming state corruption. If state and reality have genuinely diverged (a resource was created but Terraform's state doesn't know it), `terraform import` reconciles them before re-applying. Re-running `apply` is usually safe once you understand what failed and why — Terraform is designed to be idempotent — but understanding the cause first prevents repeating the same failure in a loop.

- [ ] SCENARIO A downstream team says your service's events "sometimes just don't arrive." How do you debug it end to end?

IF ASKED, SAY THIS
Get specifics first — an actual example (approximate time, expected event, what they were looking for) rather than debugging "sometimes" in the abstract. Then walk the pipeline in order: did your service actually publish it (producer-side logs/metrics for that event type), did it land in the broker

Get specifics first — an actual example (approximate time, expected event, what they were looking for) rather than debugging "sometimes" in the abstract. Then walk the pipeline in order: did your service actually publish it (producer-side logs/metrics for that event type), did it land in the broker (topic/queue metrics — a lag spike or a partition issue can delay delivery long enough to look like loss), is the downstream team's consumer actually subscribed to the right topic/filter (a routing rule or filter policy silently excluding it is a very common "phantom loss" cause), and check the DLQ — a message that failed processing and got dead-lettered looks exactly like "never arrived" from the consumer's side unless someone's watching the DLQ.

- [ ] SCENARIO Security flags a critical CVE in a base image used by 40 running services. Walk through your response plan.

IF ASKED, SAY THIS
Triage before mass-patching: is the vulnerable component actually reachable in each service's real usage, or is exposure limited to services that hit that specific code path — that determines real urgency per-service rather than treating all 40 as equally critical. For genuinely exposed services, pr

Triage before mass-patching: is the vulnerable component actually reachable in each service's real usage, or is exposure limited to services that hit that specific code path — that determines real urgency per-service rather than treating all 40 as equally critical. For genuinely exposed services, prioritize by external-facing/data-sensitivity first. Build the patched base image once, then roll it out via the normal CI/CD pipeline per service rather than manually — this is exactly the case for demonstrating why immutable, pipeline-driven deploys matter, since "rebuild and redeploy 40 services" should be a scripted, low-effort operation, not 40 manual interventions. Track it as an explicit remediation campaign with a deadline, not just "everyone patch when convenient."

- [ ] SCENARIO Your CI/CD pipeline suddenly can't authenticate to AWS via OIDC. Walk through your triage.

IF ASKED, SAY THIS

Check what actually changed first — OIDC breaks almost always from a config drift, not a random failure: did the IAM role's trust policy get modified (condition on repo/branch no longer matches), did the OIDC provider's thumbprint/cert rotate and the trust relationship wasn't updated, or did the wor

Check what actually changed first — OIDC breaks almost always from a config drift, not a random failure: did the IAM role's trust policy get modified (condition on repo/branch no longer matches), did the OIDC provider's thumbprint/cert rotate and the trust relationship wasn't updated, or did the workflow itself change (a renamed repo, a new branch that doesn't match the trust policy's condition). Read the actual STS error message carefully — it usually names exactly which condition failed, which narrows this from a broad investigation to a one-line trust-policy fix in most real cases.

- [ ] SCENARIO A new agentic AI feature starts making expensive, redundant tool calls in a loop. How do you detect and contain it?

IF ASKED, SAY THIS

Detection: this is exactly the "rising cost-per-task while completion rate looks stable" pattern — a per-task token/cost budget with alerting catches it faster than waiting for the monthly bill to reveal it, and a hard per-task max-tool-calls circuit breaker in the agent runtime itself is the actual

Detection: this is exactly the "rising cost-per-task while completion rate looks stable" pattern — a per-task token/cost budget with alerting catches it faster than waiting for the monthly bill to reveal it, and a hard per-task max-tool-calls circuit breaker in the agent runtime itself is the actual containment, not just an alert. Containment in the moment: kill or rate-limit that specific agent/task type first (blast-radius control) rather than shutting down the whole platform, since most agentic incidents are one task pattern misbehaving, not a system-wide failure. Root cause is usually a missing termination condition or an ambiguous instruction the agent is retrying indefinitely trying to satisfy — the fix is tightening the task's stop condition, not just adding a bigger budget cap.

- [ ] SCENARIO A CI/CD pipeline suddenly starts failing at the Terraform apply step with an IAM/permission-denied error, and nobody changed the pipeline. What do you check?

IF ASKED, SAY THIS

“Nobody changed the pipeline” doesn't mean nothing changed — the permission boundary the pipeline depends on is external to the pipeline code itself. Check, in order: has the IAM role/policy the pipeline assumes been modified by someone else (a security team tightening permissions, a policy-as-code

“Nobody changed the pipeline” doesn't mean nothing changed — the permission boundary the pipeline depends on is external to the pipeline code itself. Check, in order: has the IAM role/policy the pipeline assumes been modified by someone else (a security team tightening permissions, a policy-as-code tool auto-remediating something it flagged); has the Terraform config itself started trying to touch a *new* resource type or region the existing policy never covered; has a credential or OIDC trust relationship expired or been rotated; and, for AWS specifically, whether an SCP at the AWS Organizations level changed and is now blocking something the account-level policy still technically allows.

The permission-denied error message itself usually names the exact action and resource that was denied — that's the fastest way to know which of these it actually is rather than guessing.

- [ ] SCENARIO You notice a slow, steady memory increase in a service over several days, a suspected leak. How do you confirm it and what do you do before a fix ships?

IF ASKED, SAY THIS

Confirm with the actual metric trend first — plot container_memory_working_set_bytes (or the language runtime's own heap metric) over several days; a genuine leak shows a monotonic climb that doesn't reset on GC/normal cycling, as opposed to a sawtooth pattern that's just normal allocation/collectio

Confirm with the actual metric trend first — plot `container_memory_working_set_bytes` (or the language runtime's own heap metric) over several days; a genuine leak shows a monotonic climb that doesn't reset on GC/normal cycling, as opposed to a sawtooth pattern that's just normal allocation/collection.

Before a code fix ships: profile it properly rather than guess — a heap dump/profiler snapshot (language-appropriate: `pprof`, a JVM heap dump plus a tool like Eclipse MAT, Python's `tracemalloc`) at two points in time, diffed, to see what's actually accumulating. As a stability stopgap while the real fix is being worked, a scheduled rolling restart (via a CronJob triggering a rollout restart, or a slightly-aggressive memory-based liveness threshold) prevents the leak from reaching OOM in production — clearly labeled as a temporary mitigation in the ticket, not left in place indefinitely as the “fix.”

- [ ] SCENARIO A TLS certificate expired in production and caused an outage. Walk through immediate mitigation and the follow-up to prevent recurrence.

IF ASKED, SAY THIS

Immediate: reissue and deploy the new cert through the fastest safe path — if it's ACM/Key Vault-managed and just needs a redeploy to pick it up, that's minutes; if it's a manually-managed cert, generate/renew via the CA and push it through the normal deployment pipeline, not a manual copy onto a bo

Immediate: reissue and deploy the new cert through the fastest safe path — if it's ACM/Key Vault-managed and just needs a redeploy to pick it up, that's minutes; if it's a manually-managed cert, generate/renew via the CA and push it through the normal deployment pipeline, not a manual copy onto a box under pressure, since that's exactly the kind of untracked manual step that led here in the first place.

Follow-up, in priority order: confirm automated renewal is actually in place going forward (cert-manager, ACM auto-renewal) rather than a calendar reminder; add an independent expiry-monitoring alert (blackbox exporter probing actual cert expiry) that doesn't depend on the renewal system itself working, so a silent renewal failure is caught with real lead time, not discovered at expiry; and in the postmortem, check whether this was a one-off cert or whether other certs in the estate share the same gap, since a single expired cert is often a symptom of a whole class of unmonitored certs.

SCENARIO Two services are each individually healthy, but a request chain between them times out intermittently. How do you find where it's actually breaking?

IF ASKED, SAY THIS
Individually-healthy-but-chain-fails is the textbook case for distributed tracing — if it's in place, pull a trace for one of the failed requests and it usually shows exactly which hop's span has the abnormal latency or the timeout, immediately, rather than needing to guess.

Individually-healthy-but-chain-fails is the textbook case for distributed tracing — if it's in place, pull a trace for one of the failed requests and it usually shows exactly which hop's span has the abnormal latency or the timeout, immediately, rather than needing to guess.

Without tracing already in place: correlate timestamps across both services' logs for the failure window using a shared request ID/correlation ID if one exists; check the network path specifically — intermittent (not constant) failure between two otherwise-healthy services often points at something in between rather than either service itself: a load balancer's idle-timeout being shorter than the app's expected long-poll/keep-alive duration, a connection pool on one side silently exhausted, or a service mesh sidecar's own timeout/retry config fighting with the application's.

SCENARIO You're on-call and get three pages within 10 minutes for different services. Walk through how you prioritize and work them.

IF ASKED, SAY THIS
First, a fast severity/impact triage across all three — not diving deep into the first one that paged, since paging order isn't necessarily impact order. Check each against actual user impact and blast radius (customer-facing vs. internal, one region vs. all, degraded vs. fully down).

First, a fast severity/impact triage across all three — not diving deep into the first one that paged, since paging order isn't necessarily impact order. Check each against actual user impact and blast radius (customer-facing vs. internal, one region vs. all, degraded vs. fully down).

Then a quick check for a shared root cause — three pages in ten minutes is often one underlying issue (a shared dependency, a bad deploy that touched multiple services, a network event) manifesting as multiple alerts, not three independent incidents; five minutes spent confirming or ruling that out can save far more than five minutes later. If they're genuinely independent, work the highest-impact one first, pull in help for the others rather than context-switching solo across all three, and communicate status on all three so nobody's waiting in silence while you triage.

SCENARIO A production service starts returning 5xx errors right after a deployment. Walk through your triage.

IF ASKED, SAY THIS
First question: is it actually the deploy, or coincidental timing? Check the deployment timeline against when errors started — if they line up tightly, that's a strong signal, but confirm rather than assume. If it does look deploy-related, the fastest safe move is usually **rollback first, investiga

First question: is it actually the deploy, or coincidental timing? Check the deployment timeline against when errors started — if they line up tightly, that's a strong signal, but confirm rather than assume. If it does look deploy-related, the fastest safe move is usually **rollback first, investigate after** — restore service, then figure out why, rather than debugging forward while users are actively affected. While that's in flight (or if rollback isn't instant), pull the new version's logs for the actual error/stack trace, check whether it's every request or a specific path/percentage (points at a partial failure — maybe only one of several new tasks is bad, or a specific new code path), and check whether a dependency (a downstream service, a config value, a migration) changed alongside the code, since 'code looks fine in isolation' is a common false lead when the real cause is an environment or dependency mismatch that only manifests in prod.

SCENARIO An ECS service's tasks keep restarting in a loop. How do you find the root cause?

IF ASKED, SAY THIS
Start with ECS service events for the human-readable reason ECS itself already captured, then CloudWatch Logs for that task definition's log group to see the container's actual crash output — between those two, most causes surface immediately. If the container starts but then gets killed, check whet

Start with ECS service events for the human-readable reason ECS itself already captured, then CloudWatch Logs for that task definition's log group to see the container's actual crash output — between those two, most causes surface immediately. If the container starts but then gets killed, check whether it's a **health check failure** (the app takes longer to become ready than the health check's start period allows — very common right after a change that added startup work, like a larger cache warm or a new migration check) versus an actual application crash (an unhandled exception, a missing required environment variable/secret that only shows up in one environment). If it's healthy at first and then dies later, that pattern points more toward a resource constraint — check whether it's hitting its memory limit (an OOM kill shows up as a specific exit code/reason in the task's stopped-reason field) rather than crashing on its own.

SCENARIO The CI/CD pipeline suddenly fails at the deployment stage — nothing in the app code changed since yesterday. How do you investigate?

IF ASKED, SAY THIS
'Nothing changed in the app code' doesn't mean nothing changed — check what *did*: did a base image get updated upstream (if the Dockerfile pulls a mutable tag like `:latest` or `:20` rather than a pinned digest, that's a very common silent-change source), did a dependency in the artifact repository

'Nothing changed in the app code' doesn't mean nothing changed — check what *did*: did a base image get updated upstream (if the Dockerfile pulls a mutable tag like `:latest` or `:20` rather than a pinned digest, that's a very common silent-change source), did a dependency in the artifact repository get a new version resolve that wasn't pinned, did the target environment change (an expiring credential, a security group that got tightened, an IAM policy that changed, a certificate that expired overnight), or did the CI/CD tooling itself get updated (a plugin auto-update, a runner image change). The investigation is really about auditing everything **around** the app code rather than the app code itself, since the pipeline's own inputs and environment are just as capable of breaking a deploy as a code change is — and pipeline failures that 'came from nowhere' are disproportionately likely to be exactly this kind of external drift rather than a mystery.

SCENARIO Terraform apply fails halfway through, leaving a partial state change. How do you recover safely?

IF ASKED, SAY THIS

First, don't panic-retry immediately — read exactly which resources succeeded and which failed from the apply output, since Terraform applies resource-by-resource and state reflects whatever completed before the failure, which is usually a partially-correct, not fully-broken, state. Run `terraform p

First, don't panic-retry immediately — read exactly which resources succeeded and which failed from the apply output, since Terraform applies resource-by-resource and state reflects whatever completed before the failure, which is usually a partially-correct, not fully-broken, state. Run `terraform plan` next to see current reality versus desired state clearly, rather than guessing from the error message alone. If the failure was transient (a rate limit, a brief API error), simply re-running `apply` is often enough — Terraform will pick up from where it left off, only acting on what state says still needs to change. If the failure points at a real conflict (a resource that already exists outside Terraform's knowledge, a dependency ordering issue), fix the underlying cause first, then re-plan and re-apply — resist the urge to hand-edit the state file directly unless it's genuinely the only option, since a slightly wrong manual state edit can turn a partial-apply annoyance into real data loss on the next apply.

SCENARIO Disk usage on a production host is climbing fast and about to hit 100%. Walk through your response.

IF ASKED, SAY THIS

Immediate priority is preventing the outage a full disk causes (a full root/data volume typically stops writes cold, which can crash the application, the database, or the OS's own logging) — so first, quickly identify what's actually consuming space (`du -sh /* | sort -h`, or targeted checks on the

Immediate priority is preventing the outage a full disk causes (a full root/data volume typically stops writes cold, which can crash the application, the database, or the OS's own logging) — so first, quickly identify what's actually consuming space (`du -sh /* | sort -h`, or targeted checks on the usual suspects: application logs, Docker's dangling images/containers/volumes, database WAL/binlog files, a core dump) and clear whatever's safe to clear immediately, buying time. If nothing's safely and quickly reclaimable, an emergency EBS volume expansion (resize the volume, then grow the filesystem — both are online operations on modern Linux, no reboot required) is the fastest real fix. Once stabilized, the actual root cause needs attention — if it's log growth, a missing or misconfigured log rotation policy; if it's a specific process leaking disk (like an app writing debug output it shouldn't in production), a code or config fix — and if it's a recurring pattern, a disk-usage alert with real headroom (alerting at 80%, not 98%) so this becomes a scheduled fix instead of a 2 AM page next time.

SCENARIO An alert fires for elevated latency, but no users are reporting issues. How do you decide whether to act?

IF ASKED, SAY THIS

First, verify it's real and not a monitoring artifact — check whether the underlying metric genuinely moved (a real, sustained shift) versus a single noisy data point, an agent hiccup, or a metric that's measuring something less representative than it seems (an average skewed by one slow outlier req

First, verify it's real and not a monitoring artifact — check whether the underlying metric genuinely moved (a real, sustained shift) versus a single noisy data point, an agent hiccup, or a metric that's measuring something less representative than it seems (an average skewed by one slow outlier request, rather than the p95/p99 the alert should probably be on). Cross-check against a second signal — error rate, request volume, a synthetic check's own latency — since real user-facing latency degradation rarely shows up in exactly one isolated metric and nowhere else. If it's confirmed real but genuinely has no user impact yet (say, latency crept up but is still well within what users would perceive or what the SLO allows), it's reasonable to treat it as a leading indicator worth investigating at normal priority rather than paging everyone — but that's different from dismissing it outright, since 'no reports yet' often just means the impact hasn't reached a threshold users bother reporting, not that there's truly no impact.

SCENARIO A secrets/credential rotation just broke a running application at 2 AM. How do you triage and stabilize?

IF ASKED, SAY THIS

First, confirm that's actually the cause — check the timing of the rotation against when errors started, and check the application's error for something that actually points at auth/credential failure (a 401/403, a connection-refused specifically on the credential-dependent dependency) rather than a

First, confirm that's actually the cause — check the timing of the rotation against when errors started, and check the application's error for something that actually points at auth/credential failure (a 401/403, a connection-refused specifically on the credential-dependent dependency) rather than assuming from timing alone. If confirmed, the fastest stabilization is usually restarting/redeploying the affected service so it picks up the newly-rotated credential fresh — a very common cause of this exact scenario is the app having cached the *old* credential in memory at startup and never reloading it, so the rotation itself succeeded but the running process is still holding a now-invalid value. If a restart doesn't fix it, check whether the rotation actually completed correctly (for something like RDS rotation via Secrets Manager, whether both old and new credentials were meant to work briefly during cutover and that overlap window already closed) or whether it's a permissions issue on the new credential itself. Longer-term fix: either the application needs to reload credentials without a restart, or rotation needs to happen during a low-traffic window with a deliberate rolling-restart step built in, not silently at whatever time was scheduled.

SCENARIO A JFrog Artifactory repository runs out of storage during a critical release window. What do you do right now versus after?

IF ASKED, SAY THIS

Right now: unblock the release first — if there's headroom in a different repository or storage backend, or if Artifactory's storage can be expanded quickly (an underlying EBS volume resize, if self-hosted, or a quick support/quota bump if it's the SaaS offering), that's the fastest path to gett

Right now: unblock the release first — if there's headroom in a different repository or storage backend, or if Artifactory's storage can be expanded quickly (an underlying EBS volume resize, if self-hosted, or a quick support/quota bump if it's the SaaS offering), that's the fastest path to getting the release artifact published without a broader cleanup effort under time pressure. If expansion isn't immediately possible, a targeted emergency cleanup — deleting the most obviously disposable artifacts (very old snapshot builds, known-orphaned uploads) rather than a broad policy run — buys just enough space to get the critical artifact through without risking deleting something the release itself needs. **After**: this is a symptom of retention policies not existing or not being enforced — the real fix is implementing (or fixing) an actual scheduled cleanup policy via AQL so snapshot/dev

repositories don't silently grow until they hit a wall again, plus alerting on storage utilization with real headroom so the next time this trend starts, it's a scheduled maintenance task instead of an emergency during a release window.

- [ ] SCENARIO This kind of platform (proxy voting, trading communications, financial data processing) often runs against hard regulatory deadlines, not just general uptime targets. How does that change how you'd think about an infrastructure incident compared to a typical consumer app outage?

IF ASKED, SAY THIS
The core shift is that 'degraded but recovering' isn't automatically good enough — a proxy voting system that's down for two hours overnight might be a non-event, or it might mean shareholders miss an actual SEC-mandated voting deadline, and the difference is entirely about *when* relative to a fixe

The core shift is that 'degraded but recovering' isn't automatically good enough — a proxy voting system that's down for two hours overnight might be a non-event, or it might mean shareholders miss an actual SEC-mandated voting deadline, and the difference is entirely about *when* relative to a fixed external cutoff, not how long the outage lasted in isolation. That means severity and response urgency for the same technical failure can vary hugely by calendar timing — an incident window worth understanding, rather than treating every outage as equally 'get it back up whenever.' Practically, it argues for building deadline-awareness into monitoring itself (alerting differently, or paging faster, in the days around a known record/vote date than on an ordinary Tuesday), for change-freeze discipline around those windows the same way a retailer freezes deploys before its peak season, and for RCAs on anything that touches those paths to explicitly ask 'could this have caused a missed regulatory deadline,' not just 'did users notice.' It's the same production-support instinct as any other environment, just calibrated to the fact that some failure windows here have a real compliance consequence attached, not only a business/reputational one.

- [ ] SCENARIO A deploy just went out and 5xx errors spike immediately after. How do you confirm it's the deploy before rolling back, and how fast do you act?

IF ASKED, SAY THIS
Correlate the error-spike timestamp precisely against the deploy/canary event, and check whether the errors are isolated to the new version's pods versus baseline (a canary comparison makes this trivial; without one, check pod age/version labels against the error source). If the correlation is clear

Correlate the error-spike timestamp precisely against the deploy/canary event, and check whether the errors are isolated to the new version's pods versus baseline (a canary comparison makes this trivial; without one, check pod age/version labels against the error source). If the correlation is clear and users are actively affected, **roll back first, investigate after** — debugging in production while a known-bad version keeps serving traffic just extends the incident for the sake of root-causing something you can just as well root-cause from the rolled-back state.

- [ ] SCENARIO Prometheus/Alertmanager didn't fire during a real outage — you found out from a customer instead. Walk through your investigation.

IF ASKED, SAY THIS
Check the up metric for the target at the time of the outage — was it even being scraped. Check Alertmanager for an accidental silence or a routing misconfiguration that swallowed the notification. Check whether the alert rule's for: duration was longer than the actual outage lasted — a brief-but-re

Check the `up` metric for the target at the time of the outage — was it even being scraped. Check Alertmanager for an accidental silence or a routing misconfiguration that swallowed the notification. Check whether the alert rule's `for:` duration was longer than the actual outage lasted — a brief-but-real outage can resolve before a 10-minute `for:` clause ever trips. Whatever the cause, this is postmortem-worthy on its own — a monitoring gap that let a real outage go unnoticed is itself the incident, separate from whatever caused the outage.

- [ ] SCENARIO A terraform apply fails partway through against a production account and you're not sure what actually landed. What do you do, step by step?

IF ASKED, SAY THIS
Don't immediately re-run apply, and don't fix it by hand in the console — either compounds the uncertainty about what state you're actually in. Use state list / show and plan to see the real delta between what's recorded and what's desired, cross-check against the cloud console for ground truth, rec

```
terraform state list
terraform show
terraform plan
```

Don't immediately re-run apply, and don't fix it by hand in the console — either compounds the uncertainty about what state you're actually in. Use `state list` / `show` and `plan` to see the real delta between what's recorded and what's desired, cross-check against the cloud console for ground truth, reconcile deliberately, and if the state file itself looks corrupted, restore from a versioned backup rather than guessing forward from an uncertain baseline.

- [ ] SCENARIO A host pages for disk-full at 2am on a system you don't know well. Walk through your first ten minutes.

IF ASKED, SAY THIS
Confirm and locate the full filesystem first, then find what's actually growing — check known culprits (logs, core dumps, container overlay layers) before anything exotic. Buy time safely — rotate, compress, or move data rather than blind-deleting anything you don't yet understand — then once stable

```
df -h
du -sh /var/log/* 2>/dev/null | sort -rh | head
docker system df    # if containers are involved
```

Confirm and locate the full filesystem first, then find what's actually growing — check known culprits (logs, core dumps, container overlay layers) before anything exotic. Buy time safely — rotate, compress, or move data rather than blind-deleting anything you don't yet understand — then once stable, find out *why* it grew (a missing logrotate config, a runaway debug log level left on) so it doesn't page again next week.

- [ ] SCENARIO A new CVE database update causes your scanner to flag hundreds of "critical" findings overnight across the fleet. How do you avoid both panic and complacency?

IF ASKED, SAY THIS
Don't mass-suppress the batch, and don't try to patch all of it by 9am either — both are overreactions to a scanner-config event, not a confirmed incident. Quickly triage for the subset that's internet-facing, on a high-criticality asset, and/or KEV/EPSS-flagged as actively exploited — that subset g

Don't mass-suppress the batch, and don't try to patch all of it by 9am either — both are overreactions to a scanner-config event, not a confirmed incident. Quickly triage for the subset that's internet-facing, on a high-criticality asset, and/or KEV/EPSS-flagged as actively exploited — that subset gets same-day attention. Communicate the situation and the triage plan early so stakeholders aren't blindsided by a scary dashboard number. And treat the batch as a useful prompt to double-check whether your scanning cadence and rule baseline are actually current, since a single database update shouldn't be capable of causing this much noise if tuning is healthy.

- [ ] SCENARIO Your ELK/OpenSearch pipeline falls behind during an incident, so the logs you need aren't there yet. How do you diagnose and fix it, and what do you do about the incident meanwhile?

IF ASKED, SAY THIS
Check the ingest bottleneck — Logstash/ingest-pipeline CPU, Elasticsearch/OpenSearch indexing rate, rejected-execution errors, shard count/sizing relative to write volume. For the incident itself, don't wait on the pipeline to catch up — fall back to direct access (kubectl logs , journalctl on the

Check the ingest bottleneck — Logstash/ingest-pipeline CPU, Elasticsearch/OpenSearch indexing rate, rejected-execution errors, shard count/sizing relative to write volume. For the incident itself, don't wait on the pipeline to catch up — fall back to direct access (`kubectl logs` , `journalctl` on the affected hosts) so response isn't blocked by a secondary tooling failure. Afterward, right-size ingest capacity or shed lower-value log volume so the pipeline has headroom for exactly the kind of traffic spike an incident tends to generate.

- [ ] SCENARIO A Kubernetes node is intermittently NotReady, correlating loosely with high pod density on that node. What's your hypothesis path?

IF ASKED, SAY THIS
Check kubelet-reported node conditions for MemoryPressure/DiskPressure first — the correlation with pod density points straight at resource exhaustion rather than a hardware issue. Look for a pod without resource limits set, consuming more than expected and starving the node. Check whether ResourceQ

Check kubelet-reported node conditions for MemoryPressure/DiskPressure first — the correlation with pod density points straight at resource exhaustion rather than a hardware issue. Look for a pod without resource limits set, consuming more than expected and starving the node. Check whether ResourceQuotas/LimitRanges are actually enforced in the relevant namespace (a policy that exists but isn't applied is as good as no policy). And check PriorityClasses — if critical pods aren't marked with appropriate priority, they may be the ones getting evicted first under pressure instead of the actual offender.

- [ ] SCENARIO Two teams' vulnerability remediation priorities directly conflict — one wants to patch now (a breaking change), the other needs stability through a compliance audit window. How do you resolve it?

IF ASKED, SAY THIS
Assess the vulnerability's actual exploitability and exposure against the audit window's real risk — is this KEV-listed and actively exploited, or a theoretical finding that can safely wait a defined, short window. Look for a compensating control that satisfies the urgency without the breaking chang

Assess the vulnerability's actual exploitability and exposure against the audit window's real risk — is this KEV-listed and actively exploited, or a theoretical finding that can safely wait a defined, short window. Look for a compensating control that satisfies the urgency without the breaking change — a WAF rule or network restriction as a bridge until the audit window closes. And escalate the tradeoff explicitly to whoever owns shared risk for the org, rather than letting the two teams silently disagree and each assume their priority won.

- [ ] SCENARIO A previously-reliable automated remediation script starts making an incident worse instead of better. What do you do in the moment, and after?

IF ASKED, SAY THIS
In the moment: disable the automation immediately (not just let it keep retrying) and switch to manual remediation while you stabilize — an automated action actively causing harm needs a kill switch you can hit fast, which is why every closed-loop remediation should have one designed in from the sta

In the moment: disable the automation immediately (not just let it keep retrying) and switch to manual remediation while you stabilize — an automated action actively causing harm needs a kill switch you can hit fast, which is why every closed-loop remediation should have one designed in from the

start, not added after this happens once. After: treat it exactly like a postmortem-worthy incident on its own — what changed in the environment that the automation's assumptions no longer held, and whether the blast-radius bound on that automation was actually as tight as intended.

SCENARIO Scenario: A Spring Boot microservice's p99 latency jumps from 200ms to 4 seconds right after a routine deployment. Error rate looks normal, CPU and memory look fine at first glance. Walk me through your triage.

IF ASKED, SAY THIS
First check whether it's isolated to the new pods or all pods — canary versus full rollout tells you if it's code-related. Check GC logs and JVM metrics, since a post-deploy latency spike is very often GC-related: a new allocation pattern, a cache miss, or a memory leak in the new build. Check downs

First check whether it's isolated to the new pods or all pods — canary versus full rollout tells you if it's code-related. Check GC logs and JVM metrics, since a post-deploy latency spike is very often GC-related: a new allocation pattern, a cache miss, or a memory leak in the new build. Check downstream dependency latency for a slower query or a newly-introduced N+1 pattern, and check thread and connection pool saturation for blocked threads waiting on a connection. Correlate with a distributed trace to see exactly where the time is going. If it's isolated to the new version and can't be diagnosed quickly, roll back immediately and investigate offline — favor recoverability over root-causing under pressure. Prevention: pre-prod load testing and an APM baseline comparison as a deployment gate.

SCENARIO Scenario: Your GitLab CI/CD pipeline is blocking a critical hotfix from reaching production — a required scan job is failing, but the fix itself doesn't touch the flagged code. What do you do?

IF ASKED, SAY THIS
First confirm the finding is genuinely unrelated — not a new transitive dependency pulled in by the fix, which happens more than expected. If it's a pre-existing, already-tracked, non-critical finding, I'd use a scoped, time-boxed, auditable override tied to a ticket and approved by whoever owns tha

First confirm the finding is genuinely unrelated — not a new transitive dependency pulled in by the fix, which happens more than expected. If it's a pre-existing, already-tracked, non-critical finding, I'd use a scoped, time-boxed, auditable override tied to a ticket and approved by whoever owns that gate, rather than disabling it — the override itself becomes audit evidence. If it's a critical or high finding, it blocks regardless of urgency, and I'd escalate for a parallel fix rather than bypass security to hit a deadline.

SCENARIO Scenario: You're running a canary deployment at 5% traffic and the canary's error rate ticks up slightly, not dramatically, but noticeably above baseline. Do you promote, hold, or roll back?

IF ASKED, SAY THIS
Hold and dig in — "slightly elevated but not dramatic" is exactly the ambiguous zone canary analysis exists for. Check whether the increase is statistically significant given the small sample at 5% traffic, and whether it's concentrated on a specific error type or endpoint versus generally distribut

Hold and dig in — "slightly elevated but not dramatic" is exactly the ambiguous zone canary analysis exists for. Check whether the increase is statistically significant given the small sample at 5% traffic, and whether it's concentrated on a specific error type or endpoint versus generally distributed. A real, attributable signal means roll back and investigate offline; promoting on "hoping it stabilizes" converts a contained 5%-traffic problem into a 100%-traffic incident. If it's noise, widen the canary slightly and re-evaluate rather than deciding on ambiguous data.

SCENARIO Scenario: Kubernetes pods for a critical service are crash-looping during peak traffic. What's your diagnosis sequence?

IF ASKED, SAY THIS
kubectl describe pod and kubectl logs with the previous flag first, to see the actual crash reason and exit code — OOMKilled is extremely common under peak load and immediately tells you it's a resource-limit issue, not a code bug. If OOMKilled, immediate mitigation is raising memory limits or scali

kubectl describe pod and kubectl logs with the previous flag first, to see the actual crash reason and exit code — OOMKilled is extremely common under peak load and immediately tells you it's a resource-limit issue, not a code bug. If OOMKilled, immediate mitigation is raising memory limits or scaling out while root-causing why memory spiked. If it's a different crash, such as a startup dependency being unreachable, I also check readiness and liveness probe configuration, since an overly aggressive liveness probe can itself cause a crash loop under load by killing pods that are just slow, not actually broken — a subtle and common trap.

SCENARIO Scenario: Someone discovers that a database credential was accidentally committed to a GitLab repository three weeks ago. What's your incident response?

IF ASKED, SAY THIS
Treat the credential as compromised immediately regardless of evidence of misuse — rotate it first, investigate second, because investigation takes time and the exposure window needs to close now. Then check GitLab's audit and access logs and the credential's own access logs for use during the expos

Treat the credential as compromised immediately regardless of evidence of misuse — rotate it first, investigate second, because investigation takes time and the exposure window needs to close now. Then check GitLab's audit and access logs and the credential's own access logs for use during the exposure window, remove it from repo history through a proper history rewrite rather than delete-and-recommit, and run a retrospective on why secret-detection scanning didn't catch it so the same gap doesn't repeat elsewhere.

SCENARIO Scenario: Platform leadership tells you your flagship service has been silently burning its error budget for two months and is about to breach its quarterly SLO. How do you handle the conversation and the technical response?

IF ASKED, SAY THIS

Bring the burn-rate data itself to the conversation, not just the conclusion — which error categories are driving the burn, and whether it's concentrated or diffuse, since that shapes both the fix and how I frame it. I'd be direct that the budget is nearly exhausted and what that means per the pre-a

Bring the burn-rate data itself to the conversation, not just the conclusion — which error categories are driving the burn, and whether it's concentrated or diffuse, since that shapes both the fix and how I frame it. I'd be direct that the budget is nearly exhausted and what that means per the pre-agreed policy, typically a shift toward a reliability freeze on risky changes, rather than softening it, and bring a concrete remediation plan with an owner and timeline alongside the bad news.

SCENARIO Scenario: A partner integration goes down and it turns out your API Gateway's rate limiting was misconfigured, throttling a legitimate high-volume partner. How do you handle the immediate fix and the longer-term prevention?

IF ASKED, SAY THIS

Immediate: raise or exempt that partner's rate limit to restore service, after a quick sanity check that this really is legitimate traffic and not a runaway retry loop on their end masquerading as normal volume. Prevention: rate limits tiered per partner based on actual contracted volume rather than

Immediate: raise or exempt that partner's rate limit to restore service, after a quick sanity check that this really is legitimate traffic and not a runaway retry loop on their end masquerading as normal volume. Prevention: rate limits tiered per partner based on actual contracted volume rather than one global default, documented and version-controlled like any other config, with alerting on rate-limit rejections so a misconfiguration surfaces proactively next time instead of via a partner-reported outage.

SCENARIO Scenario: QA confirms a feature works perfectly in staging. It fails immediately in production. What's your approach to finding the gap?

IF ASKED, SAY THIS

Start with what's actually different between the environments, not what's supposed to be different — config values, environment variables, feature flags, external dependency versions, data volume and shape, network policies. This is very often a config or data-shape difference, such as a feature fla

Start with what's actually different between the environments, not what's supposed to be different — config values, environment variables, feature flags, external dependency versions, data volume and shape, network policies. This is very often a config or data-shape difference, such as a feature flag left off or a smaller staging dataset hiding an N+1 query, rather than a code difference, since a pure code issue would have failed in staging too. Long-term, this is itself the argument for staging genuinely mirroring production topology with masked, realistic data.

SCENARIO Scenario: An application team wants to skip your platform's standard CI/CD pipeline and deploy directly to production "just this once" to hit a deadline. How do you respond?

IF ASKED, SAY THIS

No to bypassing production entirely — that gate exists specifically for the moment someone's under deadline pressure, and "just this once" is exactly how standards erode team by team. I'd work with them on what's actually causing the time pressure — if it's the pipeline being genuinely too slow, tha

No to bypassing production entirely — that gate exists specifically for the moment someone's under deadline pressure, and "just this once" is exactly how standards erode team by team. I'd work with them on what's actually causing the time pressure — if it's the pipeline being genuinely too slow, that's real, fixable feedback for the platform team; if it's unrelated schedule pressure, I'd help expedite through a proper fast-track review rather than removing the safety net.

SCENARIO Scenario: You need to choose between Kafka, RabbitMQ, and Temporal for a new multi-step order-processing workflow that needs to survive service restarts and support retries with backoff. Walk me through your decision.

IF ASKED, SAY THIS

"Survive restarts, retries with backoff, multi-step" is really describing durable workflow state, which is Temporal's core purpose — Kafka and RabbitMQ are message transports, not workflow-state engines, so using either means hand-building the state machine and step-tracking on top. I'd only reach f

"Survive restarts, retries with backoff, multi-step" is really describing durable workflow state, which is Temporal's core purpose — Kafka and RabbitMQ are message transports, not workflow-state engines, so using either means hand-building the state machine and step-tracking on top. I'd only reach for the Kafka or RabbitMQ route if the actual need were simpler than described, such as fire-and-forget events rather than genuinely stateful multi-step orchestration with compensation logic; as described, Temporal is the right first-principles fit.

SCENARIO Scenario: You're leading modernization of a legacy Java monolith and the business asks for a timeline for "when the monolith will be fully replaced." How do you answer?

IF ASKED, SAY THIS

I'd push back gently on the framing — a full-replacement date implies a big-bang cutover, the riskiest way to do this and not actually the plan, since strangler fig extracts pieces incrementally with the monolith and new services coexisting for a long transitional period, potentially permanently for

I'd push back gently on the framing — a full-replacement date implies a big-bang cutover, the riskiest way to do this and not actually the plan, since strangler fig extracts pieces incrementally with the monolith and new services coexisting for a long transitional period, potentially permanently for stable low-change components. I'd give a timeline and value delivery per extracted capability instead, and be honest that "fully replaced" may not be the right goal at all if parts of the monolith are stable and low-risk to leave as-is.

SCENARIO Multiple pods independently report the same intermittent latency spike, traced to a shared platform service. Walk through your response.

IF ASKED, SAY THIS
First move: stop treating it as N separate pod issues — open one shared incident channel spanning affected pods immediately, since duplicate parallel investigations waste time and can even conflict (two pods independently "fixing" the same shared dependency). I take or assign incident command since

First move: stop treating it as N separate pod issues — open one shared incident channel spanning affected pods immediately, since duplicate parallel investigations waste time and can even conflict (two pods independently "fixing" the same shared dependency). I take or assign incident command since I'm the only one with visibility across all affected pods. Root cause ownership goes to whoever owns the shared service (likely DevOps & Infra guild if it's platform infrastructure); each pod reports impact and applies their own local mitigation (retries, circuit breakers) in parallel while the root cause gets fixed upstream. Postmortem afterward is guild-level, not pod-level, since the fix needs to prevent recurrence for every pod depending on that shared service, not just the ones that happened to notice first.

SCENARIO A pod ships a change mid-quarter that blows through their error budget. What do you do?

IF ASKED, SAY THIS
Check whether the pod's pre-agreed error budget policy actually triggers here (it should, if it's a real policy) — feature work pauses, reliability work takes priority until budget recovers. I'd have that conversation with the pod lead directly and quickly, not let it linger un-enforced, because an

Check whether the pod's pre-agreed error budget policy actually triggers here (it should, if it's a real policy) — feature work pauses, reliability work takes priority until budget recovers. I'd have that conversation with the pod lead directly and quickly, not let it linger un-enforced, because an error budget policy that's ignored the first time it actually matters loses all credibility for every pod watching how it plays out. I'd also dig into whether this was a one-off bad release or a symptom of a pattern (rushed testing, no canary) worth a guild-level process fix.

SCENARIO You're asked to be the technical judge between two pods proposing different logging solutions. How do you decide?

IF ASKED, SAY THIS
I'd anchor the decision in the guild's actual standard (does one option satisfy the golden-path observability contract better, does one already have wider adoption that makes consolidation cheaper) rather than a pure technical merits debate — the "best" logging tool in isolation matters less than wh

I'd anchor the decision in the guild's actual standard (does one option satisfy the golden-path observability contract better, does one already have wider adoption that makes consolidation cheaper) rather than a pure technical merits debate — the "best" logging tool in isolation matters less than which choice reduces guild-wide fragmentation. If both are genuinely equivalent against the standard, I'd let the pods' own preference and existing familiarity decide rather than imposing my personal pick — forcing a choice I don't have to live with day-to-day is exactly the over-reach that erodes trust in guild judgment.

SCENARIO A critical incident hits a pod running a stack you don't know well. Walk through your first 15 minutes.

IF ASKED, SAY THIS
Same discipline as the incident-management section: I don't try to become the domain expert — I confirm someone's driving the fix and someone's scribing, ask the cross-cutting questions the pod might be too close to ask ("what changed recently," "seen this pattern elsewhere"), track SLO burn rate ag

Same discipline as the incident-management section: I don't try to become the domain expert — I confirm someone's driving the fix and someone's scribing, ask the cross-cutting questions the pod might be too close to ask ("what changed recently," "seen this pattern elsewhere"), track SLO burn rate against a mitigation decision point, and protect the pod from stakeholder noise so they can stay heads-down. My unfamiliarity with their specific stack is exactly why I stay in the coordination role rather than trying to drive the technical fix myself.

SCENARIO A guild standard proposal gets rejected by 3 of 5 pod leads. What's your next move?

IF ASKED, SAY THIS
That's a real signal, not just resistance to push through — I'd go back to the three who rejected it and actually understand why, since "no" from multiple independent pods usually means the standard has a genuine gap, not that three teams are simultaneously being difficult. I'd revise based on that

That's a real signal, not just resistance to push through — I'd go back to the three who rejected it and actually understand why, since "no" from multiple independent pods usually means the standard has a genuine gap, not that three teams are simultaneously being difficult. I'd revise based on that feedback and re-propose, or if the underlying need is proven by the two pods who did want it, offer it as opt-in for now rather than forcing consensus that isn't there. A guild lead who pushes a rejected standard through anyway on authority they don't actually have burns credibility fast.

SCENARIO A cost spike on shared Kubernetes infrastructure traces back to one pod's misconfigured autoscaling. How do you handle it?

IF ASKED, SAY THIS
Immediate technical fix first — cap the HPA/cluster-autoscaler bounds to stop the bleeding — then the conversation with the pod, not the other way around; a cost conversation while the meter's still running gets tense for no reason. Root cause is usually a missing or wrong maxReplicas / resource req

Immediate technical fix first — cap the HPA/cluster-autoscaler bounds to stop the bleeding — then the conversation with the pod, not the other way around; a cost conversation while the meter's still running gets tense for no reason. Root cause is usually a missing or wrong maxReplicas / resource

request, so beyond the immediate fix, I'd add this as a guild-wide lint check (flag any HPA without an explicit max) so the same misconfiguration can't recur silently in another pod.

SCENARIO A DR test reveals RTO is 3x the target. How do you drive the fix without owning the service?

IF ASKED, SAY THIS
Same as the chaos-engineering answer: surface the honest gap immediately to the pod lead and delivery leadership, agree a remediation plan with a real deadline together with the pod (I don't own their service, so I can't unilaterally assign myself the fix), and re-test against that deadline rather t

Same as the chaos-engineering answer: surface the honest gap immediately to the pod lead and delivery leadership, agree a remediation plan with a real deadline together with the pod (I don't own their service, so I can't unilaterally assign myself the fix), and re-test against that deadline rather than letting the finding quietly age into an ignored backlog item. I'd also check whether the gap pattern shows up in other pods with similar architecture, since it rarely is truly isolated.

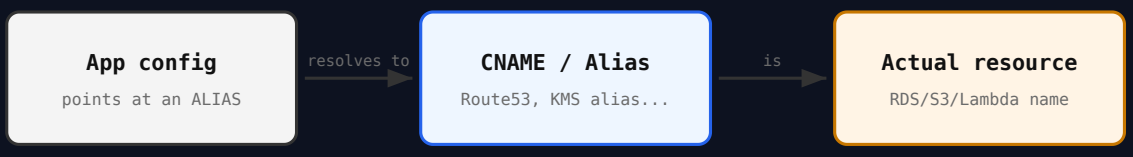
SCENARIO A new client engagement stands up a pod overnight with an aggressive deadline. How do you get baseline reliability practices in without slowing them down?

IF ASKED, SAY THIS
I wouldn't hand them the full standards checklist on day one — that's how a guild becomes the thing that slows down a pod under real deadline pressure, which is exactly the reputation you can't afford. I'd offer the smallest viable slice: the golden-path chart (gets them dashboards/alerts basically

I wouldn't hand them the full standards checklist on day one — that's how a guild becomes the thing that slows down a pod under real deadline pressure, which is exactly the reputation you can't afford. I'd offer the smallest viable slice: the golden-path chart (gets them dashboards/alerts basically for free), a lightweight first SLO, and a point of contact for questions — full guild onboarding (runbooks, guild sync attendance, postmortem process) comes once they're past the initial crunch. Meeting a pod where they actually are, under real constraints, is what makes "influence without authority" credible instead of theoretical.

SCENARIO Someone asks in an interview: "Can we rename an RDS instance in production?" What's the accurate answer?

IF ASKED, SAY THIS
Yes — RDS instances CAN be renamed (modify-db-instance --new-db-instance-identifier, or the console "Modify" flow), but it's rarely done casually in production because the DB endpoint hostname changes with it. The old DNS name is deleted immediately (though it can stay cached briefly) and the new one takes about 10 minutes to become active — so a naive rename is a real, if brief, outage for anything with the old hostname hardcoded.



The senior-level answer: never let apps depend on the raw identifier at all

- The rename itself triggers an instance reboot if applied immediately (or waits for the next maintenance window otherwise) — so there IS a brief availability impact, separate from the endpoint-change issue.
- Read replicas stay attached to the renamed instance; tags and snapshots are unaffected by the rename.
- There's a documented, legitimate production use for renaming: restoring from a snapshot or promoting a read replica, then renaming the NEW instance to take over the OLD instance's name — so it inherits the original endpoint without touching a single application config. Rename the old instance out of the way first, then rename the replacement into its place.

PRACTICAL NOTE
The answer that actually impresses: "Yes, technically — but the right practice is to never let application config point at the raw RDS endpoint at all. Put a Route53 CNAME or a proxy (RDS Proxy) in front of it, so the underlying instance can be renamed, replaced, or failed over without touching a single line of app config."

SCENARIO Someone asks: "Can we rename an S3 bucket?" What's the accurate answer, and what do you actually do instead?

IF ASKED, SAY THIS
No — S3 bucket names are permanent. There is no rename API for a bucket itself (this is different from RDS). The name is part of a namespace shared globally across every AWS account, not just yours, which is exactly why it can't be reassigned in place. The only path: create a new bucket with

the new name, copy every object over (aws s3 sync, or S3 Batch Operations / Cross-Region Replication at large scale), repoint every reference (app config, IAM policies, bucket policies, CloudFront origins, DNS), then delete the old bucket.

Two details worth having ready: first, if you delete a bucket intending to immediately recreate it under the same name, the name doesn't free up instantly — it can take 24-72 hours before another account (or you) can claim it again, since the global namespace has to release it. Second, this is genuinely different from EVERY object inside the bucket — individual object keys CAN be renamed (implemented as copy+delete, so it's not free for large objects), and since 2025 S3 Express One Zone directory buckets specifically support a true atomic RenameObject API for single objects. The bucket's own name is the one thing with no rename path at all.

- [ ] Can you rename an IAM user, an IAM role, and an IAM group — and are they all the same?

IF ASKED, SAY THIS

No, and this is a genuinely good trap: IAM USERS can be renamed (update-user --new-user-name) and IAM GROUPS can be renamed — but IAM ROLES cannot. There's no rename operation for a role at all; to "rename" one, you create a new role with the new name, copy its trust policy and attached permissions, migrate anything that assumes it, then delete the old one.

Worth stating the reasoning, not just the fact: a role's name is baked into its ARN, and that ARN is frequently referenced directly in OTHER accounts' trust policies, resource policies, and cross-account AssumeRole configurations — AWS not supporting a role rename sidesteps a much messier problem than the user/group case, where policies referencing the entity by name get auto-updated for you.

- [ ] Can you change a running EC2 instance's instance type (e.g. t3.medium to m5.large) without recreating it?

IF ASKED, SAY THIS

Yes, but not while running — stop the instance, modify the instance type (console, CLI modify-instance-attribute, or Terraform), then start it again. The instance keeps its instance ID, EBS volumes, private IP (unless it's in a non-default VPC edge case), and any Elastic IP association. What's lost: the public IP (unless using an EIP), and anything stored on instance-store/ephemeral volumes.

The stop-modify-start pattern preserving the instance ID is the detail that matters operationally — this is why resizing is usually far less disruptive than people expect: no new instance ID means no changes needed anywhere that references the instance by ID (Auto Scaling Group attachments, monitoring dashboards, etc.), just a brief stop/start window.

- [ ] Can you change a VPC's CIDR block after creation?

IF ASKED, SAY THIS

You cannot change or remove the PRIMARY CIDR block of an existing VPC — that's fixed at creation. You CAN add secondary CIDR blocks to an existing VPC to grow its address space (this is the standard fix when a cluster starts hitting IP exhaustion, and ties directly into the EKS "not enough IP addresses" scenario elsewhere in this handbook).

This is a genuinely common real-world surprise for teams that undersized their VPC early on — the fix isn't "recreate the VPC," it's "add a secondary CIDR and give new subnets room to grow in it," which is non-disruptive to what's already running.

- [ ] Can you rename a Lambda function, an ECS cluster, an EKS cluster, or a DynamoDB table after creation?

IF ASKED, SAY THIS

No to all four — every one of these names is immutable once created, and in every case the fix is the same shape: create a new one with the desired name, migrate what points at it (triggers/permissions for Lambda; services/tasks for ECS; workloads and IRSA trust relationships for EKS; data via export/import or DynamoDB Streams for a table), then decommission the old one.

Recognizing the PATTERN here is worth more than memorizing each service individually: almost every AWS resource whose name is baked into an ARN or a DNS endpoint is immutable for the same underlying reason — and almost every "gotcha" question in this category has the same shape of answer (no direct rename, create-and-migrate instead, and the senior-level addition is naming a decoupling layer — alias, CNAME, or version-pinned reference — that would have avoided the problem).

- [ ] Can you rename a Security Group after creation?

IF ASKED, SAY THIS

The actual GroupName attribute is immutable once created — you cannot rename it via the API or console after the fact. What people usually mean by "renaming" a security group in the console is actually editing its Name TAG, which is fully mutable at any time (tags in general are always mutable, on virtually every AWS resource) — but that's cosmetic, the underlying GroupName referenced in any rule or automation stays the same.

The tag-vs-attribute distinction this question is really testing generalizes well beyond security groups: a resource's core identifying attribute (name, ID, ARN component) is very often immutable, while its TAGS are almost always mutable — good general instinct to state explicitly when you're not 100% sure whether a specific field is one or the other.

- [ ] You need to rotate/replace the underlying KMS key used to encrypt several resources without updating every application's config. How?

IF ASKED, SAY THIS

Use a KMS alias (e.g. alias/my-app-key) in every application config and IAM policy instead of the raw key ID/ARN — the alias can be re-pointed to a different underlying CMK at any time via update-alias, so the key itself can be rotated, replaced, or regenerated without touching a single line of consumer configuration. This is the exact same indirection pattern as an RDS CNAME, applied to KMS.

This is worth volunteering as the "good practices" answer even if not asked directly — it's the clearest illustration in the whole AWS platform of the general principle underlying this entire category of gotcha question: never let consumers depend on a resource's raw, immutable identity when a mutable alias/indirection layer is available.

- [ ] In Terraform specifically: you rename a resource block in your .tf code (e.g. aws_instance.web to aws_instance.app_server). What happens on the next apply, and how do you prevent it?

IF ASKED, SAY THIS

Terraform sees the old address disappear and a new one appear — its default behavior is to plan a DESTROY of the old resource and a CREATE of the new one, even though nothing about the actual AWS resource needs to change. For a stateful resource (a database, a bucket with data) this is a real-data-loss risk hiding behind a cosmetic code change. The fix: terraform state mv aws_instance.web aws_instance.app_server (or a moved block in newer Terraform versions) tells Terraform "this is the same real resource, just renamed in code" — no destroy/create, no drift.

This is one of the highest-value Terraform gotchas to have cold, because the failure mode is silent and severe: a plan that says "1 to destroy, 1 to add" for what the engineer intended as a harmless rename is exactly the kind of thing that gets rubber-stamp-approved in a PR review by someone who doesn't look closely at resource addresses, and then deletes a production database.

```
# Old way (pre Terraform 1.1):
terraform state mv aws_instance.web aws_instance.app_server

# Terraform 1.1+, declared in code (shows up in the plan, safer/reviewable):
moved {
  from = aws_instance.web
  to   = aws_instance.app_server
}
```

- [ ] Can you rename a Kubernetes Namespace, Deployment, or Pod?

IF ASKED, SAY THIS

No to all three — the name is part of the object's identity in the Kubernetes API and is immutable once created. "Renaming" always means create-new-with-new-name, migrate what points at the old one, delete the old one — for a Namespace that means recreating every object inside it too, since objects belong to exactly one namespace for their whole lifetime.

Namespace "rename" is the one worth elaborating on if asked, because it's the most operationally painful of the three: there's no bulk "move everything to a new namespace" primitive, so in practice it's export all manifests (kubectl get -o yaml across every resource type), edit the namespace field, reapply into the new namespace, verify, then delete the old one — genuinely disruptive for anything stateful (PVCs don't move namespaces cleanly either).

- [ ] Can a Kubernetes PersistentVolumeClaim be resized after creation? Can its StorageClass be changed?

IF ASKED, SAY THIS

Size: yes, but only to increase it, and only if the StorageClass has allowVolumeExpansion: true — you edit the PVC's storage request upward, and depending on the volume plugin the filesystem expansion happens automatically or needs a pod restart to pick it up. StorageClass: no — a PVC's storageClassName is immutable after creation; changing which storage class/backing type a workload uses means provisioning a new PVC and migrating the data (e.g. via a one-off Job that mounts both and copies).

The "increase only, never decrease" constraint on volume expansion is the detail people forget — there's no shrink path at all for a PVC; a volume that's oversized can only be fixed by provisioning a smaller replacement and migrating data over, same shape as every other immutable-identity gotcha in this section.

- [ ] In Azure, can you rename a Storage Account or a Resource Group after creation?

IF ASKED, SAY THIS

No to both, for the same reasons as their AWS equivalents (S3 bucket names, VPC names) — a Storage Account name is globally unique across all of Azure and is baked into its DNS endpoint (mystorageacct.blob.core.windows.net), so it can't be reassigned; a Resource Group name is fixed once created. Both require create-new-and-migrate: a new Storage Account plus AzCopy/Azure Storage Migration for data, or moving resources into a new Resource Group (which Azure DOES support natively via 'Move resource' for many resource types, unlike a true rename).

Worth noting the one asymmetry: Azure's resource "Move" feature lets you relocate most resources into a different (or new) Resource Group without recreating them — which is NOT the same as renaming the group itself, but is a genuinely useful escape hatch that AWS doesn't have a direct equivalent for (there's no "move an S3 bucket into a different structure" operation).

- [ ] Can you rename an Application/Network Load Balancer, or an RDS Aurora cluster identifier?

IF ASKED, SAY THIS

Load balancer name: no — immutable once created, same shape as everything else in this category (create new + shift DNS/target groups over). Aurora cluster identifier: yes, actually — much like a standalone RDS instance, an Aurora DB CLUSTER identifier can be changed via modify-db-cluster --new-db-cluster-identifier, and it carries the same endpoint-change consequence as the instance-level rename.

The ALB/NLB case is a good one to have ready because the workaround is slightly different from the others: since a load balancer's DNS name is what you'd normally point a Route53 alias at anyway, teams that followed that practice from day one don't feel a rename limitation at all — another concrete case for why the alias-layer habit pays off before you need it, not after.

- [ ] You get asked about renaming some AWS resource you've genuinely never looked into. What's the general framework for reasoning through an answer live, instead of freezing?

IF ASKED, SAY THIS
Ask yourself three things out loud: (1) Is this name globally unique across all AWS accounts, or account-scoped? Globally-unique names (S3, sometimes) are almost always unrenamable. (2) Is the name baked into a DNS endpoint or an ARN that other accounts/services might reference directly? If yes, renaming is either unsupported or has real blast radius. (3) Does AWS distinguish this resource's core identifying attribute from its tags? Tags are essentially always mutable regardless of the resource.

Stating this reasoning framework OUT LOUD, even before landing on a confident yes/no, is itself a strong interview answer — it signals you understand WHY these constraints exist rather than having memorized a lookup table, which is exactly what separates a senior answer from a junior one on a question you can't have prepared verbatim.

- [ ] SCENARIO A P1 SLA promises a 4-hour response time, but your team's actual MTTR is averaging 6 hours. How do you close that gap?

IF ASKED, SAY THIS
Don't guess — break the 6 hours down by phase first: detection, escalation, diagnosis, fix. That tells you where to actually invest, rather than applying a generic fix to the wrong phase.

Don't guess — break the 6 hours down by phase first: how much is detection, how much is escalation, how much is diagnosis, how much is the actual fix. That tells you where to invest. Common fixes from there: improve alerting fidelity so real P1s aren't buried in noise, build runbooks for the top recurring incident types to cut diagnosis time, review whether on-call is overwhelmed or under-equipped, and run game days to rehearse the response so the process itself isn't the bottleneck under pressure.

- Break the total time into phases **before** proposing fixes — shows diagnosis, not guessing
- Four usual suspects: alerting fidelity, runbooks, on-call load, rehearsal

- [ ] SCENARIO A secret got committed and pushed to the repo. What are your steps?

IF ASKED, SAY THIS
Rotate first, clean up second — the credential is compromised the moment it's pushed, whether or not you can scrub history, so revoke and rotate immediately. Only after rotation do you deal with the repo itself (purge from history, force-push with the team's awareness).

Rotate first, clean up second — the exposed credential is compromised the moment it's pushed, whether or not you can scrub history, so revoke and rotate it immediately. Then check audit logs for any use of that credential during the exposure window. Only after rotation do you deal with the repo itself — purge it from history with a tool like BFG or filter-repo and force-push with the team's awareness, since it rewrites SHAs. Prevention afterward: pre-commit secret scanning such as git-secrets, truffleHog, or gitleaks, and moving to short-lived, workload-issued credentials instead of long-lived ones wherever that's viable.

- Rotate immediately — don't wait for history cleanup
- Check audit logs for actual misuse during the exposure window
- History rewrite is cleanup, not the fix; pre-commit scanning is the actual prevention

- [ ] SCENARIO A production Java service is unresponsive under load, but the process is still alive (not crashed). What's the very first thing you do?

IF ASKED, SAY THIS
Take a thread dump BEFORE touching anything else (jstack , or kill -3 which prints it to stdout/the log file without killing the process). Restarting first destroys the only evidence that explains WHY it hung — a thread dump captures every thread's exact state at that moment, which is almost always enough to diagnose a deadlock, thread-pool exhaustion, or a stuck downstream call.

Take 2-3 thread dumps a few seconds apart, not just one — if the same threads are stuck in the same place across all of them, that's a strong signal of a genuine hang (deadlock, infinite loop, blocked I/O) rather than just a momentarily busy system.

- [ ] How do you actually read a thread dump to find a deadlock?

IF ASKED, SAY THIS
jstack's own output does the hard work for you — search for "Found one Java-level deadlock" near the bottom, which names the exact threads and locks involved. Manually, the pattern is two (or more) threads each shown as BLOCKED, each holding a lock the other is waiting for — cross-reference the "waiting to lock <0x...>" line in one thread against the "locked <0x...>" line in the other.

Knowing that jstack literally detects and reports deadlocks for you, rather than needing to manually trace lock ownership across dozens of threads, is worth stating explicitly — it's a fast, close-to-free diagnostic step, not laborious detective work.

```
jstack <pid> > threaddump.txt
```

```
grep -A 20 "Found one Java-level deadlock" threaddump.txt
```

- [ ] A Java process is pegging one CPU core, but you don't know which part of the application is responsible. How do you find the exact thread — and even the exact line of code?

IF ASKED, SAY THIS
top -H -p shows per-thread CPU usage within that process, giving you a thread ID (TID) in decimal. Convert it to hexadecimal (printf '%x\n'), then grep a thread dump taken at the same moment for that hex ID in the nid= field — that thread's stack trace in the dump shows exactly what code it was executing.

This decimal-to-hex thread-ID correlation is one of the highest-value, most concrete Java troubleshooting skills to have ready — it turns "the app is using 100% CPU somewhere" into "here's the exact stack trace of the thread doing it," and it's a very common live-diagnosis question precisely because it's a real, repeatable technique rather than trivia.

```
top -H -p 12345          # find the busy thread's TID, e.g. 12401
printf '%x\n' 12401       # convert to hex: 3071
jstack 12345 | grep -A 30 'nid=0x3071'
```

- [ ] What are the different OutOfMemoryError types, and why does treating them all as "the app needs more memory" miss the actual problem?

IF ASKED, SAY THIS
Java heap space -- the heap itself is full of live (still-referenced) objects; Metaspace -- too many loaded classes (common with classloader leaks in app servers doing hot-redeploys); GC overhead limit exceeded -- the JVM is spending almost all its time GCing and reclaiming almost no memory, meaning it's nearly out even before hitting the hard limit; unable to create native thread -- NOT a heap problem at all, it's hit an OS-level thread or memory-map limit; direct buffer memory -- off-heap NIO buffers exhausted. Blindly raising -Xmx fixes exactly one of these and does nothing for the other four.

"Unable to create native thread" is the one people most often misdiagnose as a memory issue — it's actually an OS-level limit (ulimit -u, or the kernel's max thread count) or a thread-pool misconfiguration creating unbounded threads, and raising JVM heap size has zero effect on it.

- [ ] How do you actually diagnose a memory leak in a running Java application, beyond "take a heap dump and look at it"?

IF ASKED, SAY THIS
Capture 2-3 heap dumps spaced out over time (jmap -dump, or trigger automatically on OOM via -XX:+HeapDumpOnOutOfMemoryError) and compare growth between them, not just one snapshot — a leak shows up as a specific object type's RETAINED SIZE growing steadily across dumps while the app's actual workload stays flat. In the analysis (Eclipse MAT or similar), the dominator tree shows what's actually holding the memory, and "path to GC roots" on the biggest suspect shows WHY it's still referenced and can't be collected.

The most common real root causes worth naming: static collections (a Map or List declared static that things get added to but never removed from), unclosed resources (streams/connections not in a try-with-resources), listener/callback registration without matching deregistration, and ThreadLocal values never cleared in a pooled-thread environment (the thread gets reused, but the stale ThreadLocal value doesn't).

```
jmap -dump:live,format=b,file=heap.hprof <pid>
# or automatically on OOM:
# -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/var/dumps/
```

- [ ] Your service has stop-the-world GC pauses long enough to cause request timeouts. How do you actually diagnose and address it?

IF ASKED, SAY THIS
Enable GC logging (-Xlog:gc* in modern JDKs) and read actual pause durations and frequency rather than guessing — that tells you whether it's a few very long pauses (often a full GC, sign of genuine memory pressure or a bad heap-sizing choice) or many small pauses adding up (often just an undersized young generation causing excessive minor GCs). The fix depends entirely on which: right-size generations, or in many cases just switching collector (G1 is the modern default for balanced throughput/latency; ZGC or Shenandoah specifically target very low, near-constant pause times regardless of heap size for latency-critical services).

Naming ZGC/Shenandoah specifically (not just G1) signals current knowledge — they're the right answer for a genuinely latency-sensitive service (e.g. a trading system) where even G1's typically-short pauses are still too long, since both target sub-millisecond pauses even on very large heaps.

- [ ] ClassNotFoundException vs. NoClassDefFoundError -- what's the actual difference, and what does each point you toward?

IF ASKED, SAY THIS
ClassNotFoundException is thrown when code explicitly tries to load a class by name at runtime (e.g. Class.forName(...)) and it isn't on the classpath at all -- check the classpath/JAR is actually present. NoClassDefFoundError is different and more confusing: the class WAS available at COMPILE time (the code compiled fine), but isn't available now at RUNTIME -- classic causes are a JAR that's present in the build but excluded from the deployed artifact, a version mismatch between build and deploy environments, or a static initializer that threw an exception the first time the class loaded, permanently marking it unusable for the rest of that JVM's life.

The static-initializer-failure cause is the one most people don't know and it's a great one to mention — if a class's static block throws once, NoClassDefFoundError will keep appearing for every SUBSEQUENT reference to that class for the life of the JVM, even though the actual root cause was a single one-time exception, not a missing file.

- [ ] Why do containerized Java applications sometimes get OOMKilled by Kubernetes even though the JVM itself never reports an OutOfMemoryError?

IF ASKED, SAY THIS
The JVM's own heap limit and the CONTAINER's memory limit are two separate things -- if the JVM thinks it has more room than the container actually allows (heap sized for the underlying host's memory rather than the container's cgroup limit), the JVM stays well within ITS OWN idea of the heap limit while total process memory (heap + metaspace + thread stacks + native memory) still exceeds the container's cap, and Kubernetes' OOM killer terminates the container from OUTSIDE the JVM -- so nothing inside the JVM ever gets a chance to log an OutOfMemoryError.

Modern JDKs (10+) default to respecting cgroup limits automatically (-XX:+UseContainerSupport, on by default), but an older JDK, or explicit -Xmx flags that ignore the container limit, reintroduce exactly this problem — this connects directly to the general Kubernetes OOMKilled troubleshooting elsewhere in this handbook, but the ROOT CAUSE here is specifically a JVM/container memory-limit mismatch, not a generic "pod needs more memory" issue.

- [ ] A Spring Boot service intermittently fails to get a database connection under load, even though the database itself isn't overloaded. What do you check on the Java side specifically?

IF ASKED, SAY THIS
HikariCP (Spring Boot's default connection pool) exhaustion -- check the pool's configured maximumPoolSize against actual concurrent demand, and check for CONNECTION LEAKS (code paths that acquire a connection but don't release it back to the pool, e.g. a missing try-with-resources or an exception path that skips cleanup) using HikariCP's own leak-detection-threshold setting, which logs a stack trace for any connection held longer than a configured duration.

Setting leakDetectionThreshold in HikariCP proactively (rather than only after the incident) is worth mentioning — it turns a mysterious intermittent exhaustion into a stack trace pointing at the exact code path holding connections too long, the same before/after value it provides elsewhere in this handbook's connection-pool-exhaustion content, just Java/HikariCP-specific here.

```
spring.datasource.hikari.maximum-pool-size=20
spring.datasource.hikari.leak-detection-threshold=30000  # ms; logs a stack trace for long-held connections
```

- [ ] A Spring Boot service has a very slow startup time in production, well beyond what it takes locally. What's typically responsible?

IF ASKED, SAY THIS
Classpath scanning and bean initialization at scale -- Spring's component scan and auto-configuration have to inspect every candidate class, and this scales with classpath size, so a service that's accumulated many dependencies over time slows startup even without new business logic. Common fixes: narrow @ComponentScan to actual package roots instead of scanning broadly, prefer lazy initialization for beans not needed immediately (spring.main.lazy-initialization=true, though this trades startup speed for first-request latency), and audit for genuinely unused dependencies inflating the classpath.

The lazy-initialization tradeoff is worth naming explicitly rather than presenting it as a free win — it moves cost from startup to the first request that actually needs that bean, which matters a lot for a service with a strict readiness-probe SLA versus one where slightly slower first-requests are acceptable.

- [ ] How would you define connection pool standards, such as HikariCP, for Spring Boot services to prevent production incidents?

IF ASKED, SAY THIS
Standardize maximumPoolSize sized to actual DB capacity divided across expected service instances (not one-size-fits-all), a connectionTimeout so callers fail fast rather than queue forever, and leak-detection-threshold enabled by default across all services so a leak surfaces as a log line, not a mystery outage.

Pool size sized to actual downstream capacity and expected concurrency, not a copy-pasted default — oversized pools can overwhelm the database under load, undersized pools cause request queuing under normal traffic. I'd mandate connection timeout, validation, and leak-detection settings are explicitly configured rather than left at framework defaults, and monitor pool utilization, since connection pool exhaustion is one of the most common mystery-latency-spike root causes in Java services.

- [ ] How do you prevent one misbehaving downstream service from taking down others in a Spring Boot microservices platform?

IF ASKED, SAY THIS
A circuit breaker (Resilience4j is the modern standard for Spring Boot, replacing the now-unmaintained Hystrix) wraps calls to a downstream dependency and trips open after a failure threshold, failing fast instead of letting every caller thread queue up waiting on a dependency that's already struggling — the exact mechanism that prevents the cascading-failure pattern covered elsewhere in this handbook.

Circuit breakers and bulkheads via Resilience4j, isolating thread and connection pools per downstream dependency so one slow dependency can't exhaust resources needed for others, sensible timeouts on every outbound call, and rate limiting at the gateway layer. This is a platform-level standard I'd enforce via a shared starter library rather than leave to each team to remember.

- [ ] How do you handle dependency management risk, such as CVEs in transitive Maven dependencies, across many Java services?

> **IF ASKED, SAY THIS**
> Automated dependency scanning in CI (OWASP Dependency-Check, Snyk, or GitHub Dependabot) against every build, not a periodic manual audit, plus a defined SLA for how fast a critical CVE must be patched once flagged — the same pipeline-scanning principle covered in the Security tab, applied specifically to the Maven/Gradle dependency graph where transitive (indirect) dependencies are the ones teams most often lose visibility into.

Automated dependency scanning in every pipeline with severity-based gates, a platform-owned bill of materials or parent POM pinning known-good versions of common libraries so teams aren't each resolving version conflicts independently, and a defined SLA for patching critical CVEs — for example 48 to 72 hours — tracked centrally so it doesn't silently slip on individual team backlogs.

- [ ] SCENARIO Walk through troubleshooting: an EC2 instance in one VPC can't connect to an RDS instance.

> **IF ASKED, SAY THIS**
> Work outward systematically rather than guessing: same VPC, or peered/Transit-Gateway-connected? Security group on the RDS instance actually allowing inbound from the EC2 instance's security group (SG-to-SG references are the more maintainable pattern versus CIDR ranges, worth mentioning). NACLs at the subnet boundary

Work outward systematically rather than guessing: same VPC, or peered/Transit-Gateway-connected? Security group on the RDS instance actually allowing inbound from the EC2 instance's security group (SG-to-SG references are the more maintainable pattern versus CIDR ranges, worth mentioning). NACLs at the subnet boundary not blocking it. Route tables on both subnets actually routing to each other. DNS resolving the RDS endpoint correctly.

And confirm the RDS instance itself isn't paused, in a maintenance window, or has exhausted max connections — sometimes the network path is fine and the database itself is the actual blocker.

- [ ] SCENARIO A production database's CPU is spiking — walk through your triage.

> **IF ASKED, SAY THIS**
> Check Performance Insights or CloudWatch for what's actually driving it — a specific slow query, a spike in connection count, or lock contention look very different and need different fixes. Check for a recent deploy or schema change correlating with the onset. Check connection count against the instance's max — a conn

Check Performance Insights or CloudWatch for what's actually driving it — a specific slow query, a spike in connection count, or lock contention look very different and need different fixes. Check for a recent deploy or schema change correlating with the onset.

Check connection count against the instance's max — a connection leak from an application not releasing connections properly is one of the most common real-world causes, often mistaken for "the database needs to be bigger" when it's actually an application-side bug. If it's genuinely query load, check for a missing index or a newly-inefficient query plan (often after a data volume change) before reaching for a bigger instance class.

- [ ] SCENARIO Walk through connection pooling — why it matters, and how you'd diagnose a connection pool exhaustion issue.

> **IF ASKED, SAY THIS**
> Databases have a hard cap on concurrent connections, each with real memory/resource cost — without pooling, a burst of application instances or threads can exhaust that cap directly, or waste it on mostly-idle connections. A pooler (RDS Proxy, PgBouncer, or an application-level pool) multiplexes many logical applicatio

Databases have a hard cap on concurrent connections, each with real memory/resource cost — without pooling, a burst of application instances or threads can exhaust that cap directly, or waste it on mostly-idle connections. A pooler (RDS Proxy, PgBouncer, or an application-level pool) multiplexes many logical application connections onto fewer actual database connections.

Diagnosing exhaustion: check current connection count against max, check for connections stuck idle-in-transaction (a common leak — a transaction opened and never committed/rolled back), and check whether a recent traffic spike or a change in application instance count is the actual driver before assuming the pool itself is misconfigured.

- [ ] SCENARIO A deployment succeeded but the application is throwing errors for new users only

> **IF ASKED, SAY THIS**
> A 'successful' pipeline only confirms the deploy mechanics worked — it says nothing about whether a new code path's dependencies (config, secrets, a downstream service) are actually correctly wired for this release specifically.

Scenario: The pipeline reports a successful deploy, but new user signups are failing while existing users are fine. Walk through your process.

Reframe: A 'successful' pipeline only confirms the deploy mechanics worked — it says nothing about whether a new code path's dependencies (config, secrets, a downstream service) are actually correctly wired for this release specifically.

What to check: Whether a new environment variable/secret the new code expects was actually added to this environment — extremely common when app code and config live in separate places and one gets updated without the other. Also check whether a downstream dependency (e.g. an email/verification service) needs a corresponding change that wasn't included.

Likely root cause: A config/secret gap between what the new code expects and what's actually deployed, isolated to exactly the code path new users hit that existing users don't (e.g. account creation vs. login).

SCENARIO A Terraform apply partially failed midway through a production infrastructure change

IF ASKED, SAY THIS

Terraform state reflects what actually succeeded before the failure — re-running plan first shows exactly the remaining delta, rather than blindly re-running apply, which risks compounding the problem if the root cause (a quota limit, a naming conflict) isn't understood yet.

Scenario: Your terraform apply against production failed partway through. What's your recovery process?

Don't panic-retry: Terraform state reflects what actually succeeded before the failure — re-running plan first shows exactly the remaining delta, rather than blindly re-running apply, which risks compounding the problem if the root cause (a quota limit, a naming conflict) isn't understood yet.

Fix the actual cause first: Address whatever caused the failure (a quota limit, a dependency conflict, a config error) before re-applying — re-running the same broken config just fails again, possibly leaving state in a worse partial condition.

Recovery tools if state itself is inconsistent: terraform import to sync a resource that was actually created but lost from state; terraform state rm as a last resort to remove a broken resource from state so it can be manually cleaned up and recreated fresh. Always back up state (terraform state pull) before any destructive state operation.

SCENARIO Multiple teams claim an issue isn't theirs during an active outage

IF ASKED, SAY THIS

Table the ownership debate entirely during the active incident — whoever can execute the fastest workaround acts first, regardless of whose 'fault' it turns out to be.

Scenario: During a live Sev1, the app team, database team, and network team are each pointing at another team. How do you drive resolution?

Live-outage rule: Table the ownership debate entirely during the active incident — whoever can execute the fastest workaround acts first, regardless of whose 'fault' it turns out to be.

Get to evidence fast: Pull timestamp-correlated data from logs/metrics/Git history across all three layers — the actual data usually resolves the disagreement quickly, since it shows what changed and when, rather than three competing opinions.

After resolution: Ownership gets resolved in the blameless postmortem using that same data, and a recurring ownership dispute between the same teams is itself a signal that architectural or process boundaries need clarifying — not just something to re-litigate every time it happens.

Tricky Questions

- [ ] What is the difference between SRE and DevOps? Are they the same thing?

> **IF ASKED, SAY THIS**
> They are related but not the same: DevOps is a cultural movement and set of practices: break the wall between Dev and Ops, automate everything, deliver frequently, take ownership of what you build. It is a philosophy, not a specific job role. SRE is Google's specific implementation of DevOps princip

They are related but not the same:

DevOps is a cultural movement and set of practices: break the wall between Dev and Ops, automate everything, deliver frequently, take ownership of what you build. It is a philosophy, not a specific job role.

SRE is Google's specific implementation of DevOps principles. It is a prescriptive engineering discipline with specific practices: SLOs, error budgets, toil elimination, blameless postmortems. SRE answers the question: "How do you operationalise DevOps with rigour and measurement?"

Key difference in practice:
DevOps teams may define reliability goals loosely ("we want high availability"). SRE teams define them mathematically (99.9% availability = 43.8 minutes downtime/month error budget). Error budget policy then drives engineering decisions: if budget is at risk, stop shipping features and fix reliability.

What to say in an interview: "DevOps is the cultural philosophy — build it, run it, automate it. SRE is the engineering discipline that makes reliability measurable and actionable. I apply SRE practices — SLOs, error budgets, blameless postmortems, and toil elimination — within a DevOps cultural framework where my team owns the full lifecycle of the services we build."

- [ ] How do you build a culture of reliability in a team that has never practised SRE?

> **IF ASKED, SAY THIS**
> This is asked to test whether you can lead change, not just implement tools: Step 1 — Measure first, preach later: Do not walk in talking about SLOs. Measure what is actually happening: how many incidents per week, average MTTR, how many alerts per day, how many hours the team spends on manual toil.

This is asked to test whether you can lead change, not just implement tools:

Step 1 — Measure first, preach later: Do not walk in talking about SLOs. Measure what is actually happening: how many incidents per week, average MTTR, how many alerts per day, how many hours the team spends on manual toil. Show the data. Data creates urgency without blame.

Step 2 — Start with one small win: Pick the most painful recurring incident. Fix the root cause permanently. Document what you did. Show the before/after: "We used to get paged 3 times a week for this. Now: zero." This builds credibility and trust.

Step 3 — Introduce blameless postmortems: After the next incident, run a blameless RCA meeting. Focus on system conditions, not people. For many teams, this is the single most culturally significant SRE practice. When engineers stop fearing blame, they report problems earlier and honestly.

Step 4 — Make SLOs real: Define one SLO for the most important service. Track it on a dashboard everyone can see. When the error budget is at 50%, have a conversation: should we ship the planned feature or fix reliability? This makes reliability a business conversation, not an engineering complaint.

Step 5 — Reduce on-call pain: Fix the noisiest alerts. Teams that trust their SRE lead are ones whose on-call rotation gets better, not worse, over time.

- [ ] What is toil and how do you decide what to automate first?

> **IF ASKED, SAY THIS**
> Toil definition (Google SRE book): Manual, repetitive operational work that scales linearly with service growth, has no lasting value, and could be automated. Running a script to restart a service every Monday morning is toil. Writing the script that automatically detects and restarts it is eliminat

Toil definition (Google SRE book): Manual, repetitive operational work that scales linearly with service growth, has no lasting value, and could be automated. Running a script to restart a service every Monday morning is toil. Writing the script that automatically detects and restarts it is eliminating toil.

How to identify toil: Track every operational task your team does for two weeks. Log: what was done, how long it took, and whether the same thing happened before. Anything appearing more than once per week is a toil candidate.

Prioritisation framework (2x2 matrix):
High frequency + High time cost = Automate immediately (e.g. manual incident ticket creation)
High frequency + Low time cost = Automate next (e.g. daily disk check email)
Low frequency + High time cost = Automate eventually (e.g. quarterly SSL cert renewal)
Low frequency + Low time cost = Probably not worth automating

What I automated in my last role: Our team was spending 6 hours/week manually creating ServiceNow incident tickets from monitoring alerts. I built a Python Lambda: CloudWatch alert → SNS → Lambda → auto-create ServiceNow INC with pre-populated severity, CI from CMDB, and runbook link. MTTA dropped from 8 minutes to under 2 minutes. 300 hours/year reclaimed for actual engineering work. Two other teams adopted the same pattern.

- [ ] Explain the concept of error budget and how you use it to make release decisions.

IF ASKED, SAY THIS
Error budget math: SLO = 99.9% availability → Error budget = 0.1% = 43.8 minutes/month. If your service has already been unavailable for 44 minutes this month, the error budget is exhausted. How error budget drives decisions (this is the key insight): Error budget bridges the gap between engineering

Error budget math: SLO = 99.9% availability → Error budget = 0.1% = 43.8 minutes/month. If your service has already been unavailable for 44 minutes this month, the error budget is exhausted.

How error budget drives decisions (this is the key insight):
Error budget bridges the gap between engineering (who wants to ship features fast) and reliability (who wants to prevent incidents). Instead of "ops says no" and "dev says ship", you have a shared budget that both teams agree governs decisions.

Budget healthy (above 50% remaining): Ship features aggressively. Experiment. Take calculated risks. Reliability is fine.
Budget warning (below 50% with 2 weeks left): Reliability review meeting. Be cautious about risky releases. Start working on known reliability issues.
Budget exhausted: Feature releases FREEZE. Every engineer focuses on reliability. No exceptions. This is the policy that needs business leadership sign-off before SRE can enforce it.

Real example to share: "In a previous role, our payments service error budget was at 90% consumed by the 20th of the month. The product team wanted to release a new payment flow feature on the 22nd. We used the error budget conversation to explain: if this release causes any incidents, we breach the SLA we have committed to customers. We delayed by one sprint, used that time to fix two known reliability issues, and the next month we had budget to spare."

- [ ] What is the difference between monitoring and observability? Why does it matter?

IF ASKED, SAY THIS
Monitoring: Collecting predefined metrics and alerting when they breach thresholds. You know what questions to ask in advance. You build dashboards for things you already know can go wrong. Observability: The ability to understand the internal state of a system from its external outputs (metrics, lo

Monitoring: Collecting predefined metrics and alerting when they breach thresholds. You know what questions to ask in advance. You build dashboards for things you already know can go wrong.

Observability: The ability to understand the internal state of a system from its external outputs (metrics, logs, traces), without needing to know in advance what might go wrong. An observable system lets you ask NEW questions about its behaviour during an incident.

Why the distinction matters in practice:
With monitoring only: your error rate alert fires. You check the dashboards you already built. If the problem matches a pattern you anticipated, you find the cause. If it is a new failure mode, you are blind.

With observability: during the same incident, you can query across logs, metrics, and traces ad-hoc. "Show me all requests that took over 2 seconds AND had a DB query count above 10 AND came from European users." This question was not anticipated when the dashboards were built. Observability makes it answerable.

The three pillars:
Metrics: tell you something is wrong. Error rate up.
Logs: tell you what happened. Stack trace showing which exception.
Traces: tell you where in the system it happened. Which microservice, which DB query, which downstream call.

Interview answer tip: "We had monitoring in place but not observability. We had dashboards for the scenarios we anticipated. When a novel failure mode appeared — a specific combination of user region, payment method, and time of day — our dashboards showed nothing wrong because we had not built that panel. I introduced distributed tracing with New Relic APM, which let us trace individual user sessions and find the exact code path that was failing. Monitoring tells you the house is on fire. Observability tells you which room, why, and how the fire started."

- [ ] Your P99 latency is rising but error rate is still zero. How do you investigate?

IF ASKED, SAY THIS
P99 rising without errors is a canary signal of an emerging problem. Structured approach: Step 1 — Characterise the latency spike: What percentile? P50 flat but P99 rising = outlier requests are slow, not all requests. Which endpoints? histogram_quantile(0.99, sum(rate(http_request_duration_seconds_

P99 rising without errors is a canary signal of an emerging problem. Structured approach:

Step 1 — Characterise the latency spike: What percentile? P50 flat but P99 rising = outlier requests are slow, not all requests. Which endpoints? `histogram_quantile(0.99, sum(rate(http_request_duration_seconds_bucket[5m])) by (le, handler))` When did it start? Correlate with the Grafana deployment annotations.

Step 2 — Correlate with deployments: Check Harness or Jenkins for any deployment in the last 2 hours. If timing matches, consider rollback before investigating further. "If in doubt, roll it out."

Step 3 — Check the database: 70% of latency issues trace to the database. PostgreSQL: `SELECT pid, now()-query_start AS duration, left(query,100) FROM pg_stat_activity WHERE state != 'idle' ORDER BY duration DESC;` Look for queries over 1 second. Look for lock waits.

Step 4 — Check downstream dependencies: Is any external API (payment gateway, fraud service) experiencing latency? Check your service mesh (Istio) or New Relic APM for latency per dependency.

Step 5 — Check thread pool / connection pool: If thread pool queue is growing, requests are waiting for a thread. This shows as P99 latency without errors (the request eventually gets a thread and succeeds, just slowly). Check JVM thread pool metrics or hikaricp_connections_active in Prometheus.

Step 6 — Check GC pressure (Java): A full GC pause of 500ms causes P99 to spike without errors. Check GC logs: `jcmd PID GC.heap_info` or look at jvm_gc_pause_seconds_sum in Prometheus.

- [ ] How do you design SLOs for a service you have never worked with before?

> **IF ASKED, SAY THIS**
> SLO design for an unknown service — a common scenario when joining a new team: Step 1 — Understand the user journey: What does the user actually do? For a payment service: user clicks pay → transaction is initiated → result is returned. The SLI must measure what the user experiences, not what the in

SLO design for an unknown service — a common scenario when joining a new team:

Step 1 — Understand the user journey: What does the user actually do? For a payment service: user clicks pay → transaction is initiated → result is returned. The SLI must measure what the user experiences, not what the infrastructure is doing.

Step 2 — Look at historical data: Before setting a target, know what you are actually delivering. Query Prometheus for the last 90 days of availability and latency. Do not set an SLO of 99.9% if you have been delivering 98.5% — you will immediately be in breach.

Step 3 — Consult with the business: What does the customer contract (SLA) commit to? Your internal SLO should be tighter than the external SLA to give a buffer. If SLA is 99.5%, internal SLO might be 99.9%.

Step 4 — Define good events clearly: A good request is one that: returned HTTP 200 AND completed in under 500ms AND the response contained valid data. Make this concrete. Edge cases matter: is a 200 with an error in the body a good event? Probably not.

Step 5 — Start conservative: Begin with achievable SLOs. It is better to have a 99.5% SLO you always meet than a 99.99% SLO you breach every month. Tighten the SLO over time as reliability improves.

Step 6 — Get stakeholder agreement: SLOs only have power when engineering, product, and business all agree on the error budget policy. Without that agreement, the SLO is just a metric, not a decision-making tool.

- [ ] What is Prometheus scrape interval and how does it affect alert accuracy?

> **IF ASKED, SAY THIS**
> Scrape interval: How often Prometheus pulls metrics from a target. Default is 15 seconds. Every 15 seconds, Prometheus sends GET to /metrics and stores the response. How it affects alert accuracy: If your error rate spikes for 10 seconds and recovers, Prometheus may miss it entirely if the spike hap

Scrape interval: How often Prometheus pulls metrics from a target. Default is 15 seconds. Every 15 seconds, Prometheus sends GET to /metrics and stores the response.

How it affects alert accuracy:
If your error rate spikes for 10 seconds and recovers, Prometheus may miss it entirely if the spike happens between scrapes. This is the fundamental sampling tradeoff: shorter scrape interval = more accurate detection = more storage and CPU load.

The rate() function and scrape interval: `rate(http_requests_total[5m])` calculates per-second rate over the last 5 minutes. The window must be at least 4x the scrape interval for statistical stability. With 15s scrape interval, the minimum reliable window is 60s (4x15s). Using [30s] with a 15s scrape interval will give noisy, inaccurate results.

$__rate_interval in Grafana: Always use this instead of a hardcoded window. Grafana automatically calculates the correct interval based on the scrape interval configured in Prometheus. This means your dashboards work correctly whether the scrape interval is 15s or 30s without manual editing.

For_clause in alerts: `for: 5m` means the condition must be true for 5 consecutive minutes before firing. This prevents flapping on brief spikes. The tradeoff: a real outage takes 5 minutes to trigger the alert. For financial services, we use `for: 2m` for critical alerts to balance false positives with detection speed.

Recording rules for performance: Complex PromQL queries run on every scrape interval across all alert rules. For expensive queries used in dashboards and alerts, create recording rules that pre-compute the result every 30 seconds. This dramatically reduces Prometheus query load at scale.

- [ ] Walk me through your exact process when you get paged for a P1 at 3am.

> **IF ASKED, SAY THIS**
> This is the most common senior SRE interview question. The interviewer wants a structured, confident answer: T+0 — Acknowledge (within 2 minutes): PagerDuty acknowledgement stops the escalation chain. Open the incident Slack channel immediately: "IC: [my name], Scribe: [assign colleague], T+0: [time

This is the most common senior SRE interview question. The interviewer wants a structured, confident answer:

T+0 — Acknowledge (within 2 minutes): PagerDuty acknowledgement stops the escalation chain. Open the incident Slack channel immediately: "IC: [my name], Scribe: [assign colleague], T+0: [time]. Investigating." Even if you know nothing yet, you have established command.

T+2 — Quick triage (2 minutes): One look at Grafana. I need three numbers: error rate (how bad?), traffic rate (is this a traffic spike?), and which service (what is the blast radius?). I do NOT start reading logs yet. Logs come after I understand the scope.

T+5 — Communicate to stakeholders: Status page update: "We are investigating an issue affecting [service]. Updates every 15 minutes." Business bridge notification if it is revenue-impacting. This happens WHILE technical investigation continues on a parallel track.

T+10 — Narrow to a hypothesis: One check: was there a deployment in the last 2 hours? If yes, rollback is my first mitigation action. I do not wait to confirm the deployment caused it — the cost of a false rollback is low, the cost of not rolling back if it is the cause is high.

T+15 — Mitigate, not fix: The goal is to stop the bleeding. Rollback, scale out, failover, feature flag off. I am not finding the root cause yet. I am restoring service.

T+20 — Confirm resolution: Grafana shows error rate returning to baseline. Manual end-to-end test. Status page update: "Issue resolved. Post-mortem within 48 hours."

Throughout: Scribe documents everything in real time. Every command run, every finding, every decision. This becomes the raw material for the postmortem.

- [ ] What is a blameless postmortem and how do you run one effectively?

> **IF ASKED, SAY THIS**
> Why blameless: If engineers fear punishment for mistakes, they hide problems. Hidden problems escalate. You want engineers to report failures immediately and honestly. Blameless postmortems create that safety. Blameless does NOT mean consequence-free — it means the focus is on systemic causes, not o

Why blameless: If engineers fear punishment for mistakes, they hide problems. Hidden problems escalate. You want engineers to report failures immediately and honestly. Blameless postmortems create that safety. Blameless does NOT mean consequence-free — it means the focus is on systemic causes, not on finding a scapegoat.

What a good postmortem contains:
1. Incident summary: what broke, for how long, customer impact in business terms, not technical terms
2. Timeline: exact timestamps from logs and monitoring. Not "around 9am" but "09:14:32 UTC"
3. Root cause: technical cause. Use 5 Whys to get to the actual root, not just the immediate trigger
4. Contributing factors: things that made the incident worse or harder to detect
5. What went well: rollback worked fast, on-call responded in 90 seconds, runbook was accurate
6. What went poorly: alert fired 10 minutes late, runbook was outdated, DB team unreachable
7. Action items: specific, owned, dated. "Add connection pool alert" owned by Saroj, due in 3 days — not "improve monitoring"

Running the meeting: Separate the data collection (fact gathering) from the interpretation (why analysis). First, build the timeline collaboratively using logs. Everyone contributes what they saw and did. Second, run the 5 Whys analysis. Third, brainstorm action items without assigning ownership in the meeting — assign async afterwards.

Common mistake: Listing 20 action items that never get done because they have no owner and no deadline. Three action items that actually get completed are worth more than twenty that get forgotten.

- [ ] How do you handle a situation where a teammate made a mistake that caused a production incident?

> **IF ASKED, SAY THIS**
> This is a leadership and culture question disguised as a technical one: In the moment (during the incident): Focus entirely on resolution. Never name the person in the incident bridge. Never say "X did this" or "X broke it." Say "a deployment" or "a configuration change." The incident is not the tim

This is a leadership and culture question disguised as a technical one:

In the moment (during the incident): Focus entirely on resolution. Never name the person in the incident bridge. Never say "X did this" or "X broke it." Say "a deployment" or "a configuration change." The incident is not the time for accountability — it is the time for recovery.

In the postmortem: Redirect any blame language immediately. If someone says "X deployed without testing," reframe: "The system allowed a deployment without automated testing gates. How do we add that gate?" The postmortem question is always: what system condition allowed this to happen, regardless of which human triggered it?

After the postmortem: Privately check in with the teammate. Incidents that involve a teammate's change can be emotionally difficult even in blameless cultures. A simple "these things happen to all of us, the system had gaps that made this possible" goes a long way.

The systemic fix conversation: Use the incident as evidence for a process or tooling improvement. "Our CI pipeline did not catch this type of issue. Let us add an integration test." This turns the individual mistake into a team learning without blame.

What NOT to do: Never use the postmortem to demonstrate that you personally would not have made this mistake. Never say "if they had followed the runbook this would not have happened" as a conclusion — instead ask "why was the runbook not followed?" and fix the process that caused that.

- [ ] You have 3 replicas across 3 AZs. An AZ fails. What happens exactly and how long does recovery take?

> **IF ASKED, SAY THIS**
> This tests understanding of K8s failure handling at the infrastructure level: Immediate impact (T+0): One AZ fails. The node in that AZ becomes unreachable. Your load balancer (ALB) has health checks. ALB detects the unhealthy target within 2-3 health check intervals (~30-45 seconds) and stops sendi

This tests understanding of K8s failure handling at the infrastructure level:

Immediate impact (T+0): One AZ fails. The node in that AZ becomes unreachable. Your load balancer (ALB) has health checks. ALB detects the unhealthy target within 2-3 health check intervals (~30-45 seconds) and stops sending traffic to pods on that node. So traffic is immediately degraded but 2 of 3 pods (in the other AZs) are still serving.

K8s node failure detection (T+0 to T+40s): kubelet stops sending heartbeats. API server waits node-monitor-grace-period (default 40 seconds) before marking the node NotReady.

Pod eviction (T+40s to T+5m30s): Node marked NotReady. K8s waits pod-eviction-timeout (default 5 minutes) before evicting pods — this prevents unnecessary pod shuffling for brief network hiccups.

ReplicaSet controller acts (T+5m30s): Sees 2 running pods, desired is 3. Creates a new pod. Scheduler tries to assign it to a healthy node in the remaining AZs.

New pod starts (T+5m30s to T+7m): Pod starts, passes startup probe, passes readiness probe, is added to ALB target group. Full capacity restored.

Total impact window: 30-45 seconds of reduced capacity (2/3 pods) from when the AZ fails until ALB removes the unhealthy target. Full capacity restored after ~7 minutes.

How to make this faster: Reduce pod-eviction-timeout to 30 seconds if you need faster recovery. Add topology spread constraints so the new pod is guaranteed to go to a different AZ. Ensure Cluster Autoscaler is running so new nodes are provisioned if the remaining AZs do not have enough capacity.

- [ ] How do you implement and test K8s resource quotas and limit ranges in a multi-tenant cluster?

IF ASKED, SAY THIS
Multi-tenant K8s resource management is a common senior SRE question: ResourceQuota — namespace-level limits: apiVersion: v1 kind: ResourceQuota metadata: name: payments-quota namespace: production spec: hard: requests.cpu: "20" requests.memory: 40Gi limits.cpu: "40" limits.mem

Multi-tenant K8s resource management is a common senior SRE question:

ResourceQuota — namespace-level limits: `apiVersion: v1 kind: ResourceQuota metadata: name: payments-quota namespace: production spec: hard: requests.cpu: "20" requests.memory: 40Gi limits.cpu: "40" limits.memory: 80Gi pods: "100" services: "20" persistentvolumeclaims: "10"`
This prevents any single team from consuming all cluster resources. If the payments namespace tries to schedule a pod that would exceed these quotas, the pod stays Pending.

LimitRange — per-pod defaults and boundaries: `apiVersion: v1 kind: LimitRange metadata: name: payments-limits namespace: production spec: limits: - type: Container default: # applied if container has no limits cpu: "500m" memory: "512Mi" defaultRequest: # applied if container has no requests cpu: "100m" memory: "128Mi" max: # no container can exceed this cpu: "4" memory: "8Gi"`

Why LimitRange is critical: Without it, a developer deploys a container with no resource limits. It consumes all available CPU on the node. Other pods are throttled or evicted. LimitRange ensures every container has at minimum the default values.

Testing quotas: Deploy a pod that requests more than the quota allows. Verify it stays Pending. Check events: `kubectl get events -n production | grep "exceeded quota"`. Alert when namespace quota reaches 80%: query `kube_resourcequota{type="used"} / kube_resourcequota{type="hard"} > 0.8` in Prometheus.

- [ ] What happens when a Kubernetes node runs out of memory and what is the eviction sequence?

IF ASKED, SAY THIS
This is a common production scenario that tests deep K8s knowledge: Node memory pressure sequence: Stage 1 — kubelet detects memory pressure: kubelet monitors node resource usage every 10 seconds. When available memory drops below eviction-hard threshold (default 100Mi or 10%): node is tainted with

This is a common production scenario that tests deep K8s knowledge:

Node memory pressure sequence:

Stage 1 — kubelet detects memory pressure: kubelet monitors node resource usage every 10 seconds. When available memory drops below eviction-hard threshold (default 100Mi or 10%): node is tainted with NoSchedule. No new pods scheduled on this node. Existing pods continue.

Stage 2 — kubelet starts evicting pods: Pods are evicted in priority order based on QoS class:
First evicted: BestEffort pods (no requests or limits set). These have the lowest priority.
Second evicted: Burstable pods (requests less than limits) that are using more than their requested memory.
Last evicted: Guaranteed pods (requests equal limits). These are protected until the node is truly out of memory.

Within each QoS class: Pods using the most memory ABOVE their request are evicted first. A pod using 2x its memory request is evicted before a pod using 1.1x.

OOM Killer: If kubelet cannot evict fast enough and the kernel runs out of memory, the OOM killer activates. It kills processes (not pods gracefully) — often killing the container with exit code 137. This is the worst case and leaves containers in a bad state.

Prevention:
1. Always set memory limits on every container. This makes all pods Guaranteed or Burstable. BestEffort pods are dangerous in production.
2. Set memory limits slightly above memory requests (10-20% headroom). This allows the JVM/runtime to use extra memory for GC without immediately hitting the OOM killer.
3. Monitor node memory with alert at 80%. Give yourself time to scale before hitting eviction territory.
4. Use VPA recommendations to right-size memory requests based on actual usage.

- [ ] How do you handle database schema migrations in a zero-downtime deployment pipeline?

IF ASKED, SAY THIS
DB migrations during zero-downtime deployments is one of the hardest engineering problems. The wrong approach causes data loss or downtime: The core problem: During a rolling deploy, old pods (using old schema) and new pods (expecting new schema) run simultaneously. If you run a migration that drops

DB migrations during zero-downtime deployments is one of the hardest engineering problems. The wrong approach causes data loss or downtime:

The core problem: During a rolling deploy, old pods (using old schema) and new pods (expecting new schema) run simultaneously. If you run a migration that drops a column while old pods are running, those old pods crash when they try to read the dropped column.

The Expand-Contract pattern (3 phases across 3 deployments):

Phase 1 — Expand (backward compatible migration): Only ADD things. Add the new column with a nullable type or sensible default. Example: ADD COLUMN new_payment_method VARCHAR(50) NULL Old code: ignores the new column (it does not know about it yet). No crash. New code: can be deployed and uses the new column. Migration runs as a K8s Job before the new pods start (init container pattern).

Phase 2 — Deploy new application version: New pods use the new_payment_method column. Old pods that may still be running during rolling update also work because the old column still exists. Both old and new code work with the expanded schema.

Phase 3 — Contract (cleanup migration, next sprint): After confirming all old pods are gone and the new version is stable (verify rollout complete, monitor for 24-48 hours): run the cleanup migration. DROP old column. ALTER column to NOT NULL if needed. This runs as a separate deployment, not the same one as Phase 1.

Migration tools: Flyway and Liquibase manage migration history and ordering. They track which migrations have run and prevent re-running. Run them as K8s Jobs with `backoffLimit: 3` in the CI/CD pipeline before pod rollout.

Never do this: RENAME COLUMN or DROP COLUMN in the same deployment that introduces the new code. That is a guaranteed production incident.

- [ ] How do you implement feature flags and how do they relate to your CI/CD pipeline?

IF ASKED, SAY THIS
Feature flags are the bridge between code deployment and feature release: What feature flags enable: Decouple deployment (putting code in production) from release (making features available to users). You can deploy code to production daily, but control who sees new features via flag configuration —

Feature flags are the bridge between code deployment and feature release:

What feature flags enable: Decouple deployment (putting code in production) from release (making features available to users). You can deploy code to production daily, but control who sees new features via flag configuration — no deployment needed to toggle a feature.

Flag types and use cases:
Release flags: new feature in development, disabled by default, enabled when ready. Zero-risk deployment.
Experiment flags: A/B test — 50% of users see variant A, 50% see variant B. Measure conversion.
Ops flags: kill switch for a problematic feature. Disable instantly without deployment.
Permission flags: enable feature for specific users/accounts (early access, beta testers).

Implementation options:
LaunchDarkly (SaaS): most mature, real-time flag evaluation, percentage rollouts, user targeting. Enterprise cost.
Unleash (open source): self-hosted, full-featured, Prometheus metrics built-in.
AWS AppConfig: native AWS, integrates with CloudWatch for automatic rollback if error rate increases after flag change.
Simple approach for small teams: config file in S3, Lambda reads it on a schedule, pods reload config without restart.

In CI/CD pipeline context: All new features behind flags by default. Pipeline deploys freely to production. Feature enabled via flag for 1% of users first → monitor → 10% → 50% → 100%. If metrics degrade at any stage, disable the flag (seconds, not a deployment). Harness Continuous Verification can monitor after a flag change, not just after a deployment.

Important: Flag cleanup is technical debt. Every flag added must have a cleanup ticket. Flags live at most 2 sprints. Old flags that no one dares remove are a codebase smell.

- [ ] Explain the difference between horizontal and vertical scaling on AWS and when to use each.

IF ASKED, SAY THIS
Vertical scaling (scale up): Replace the instance with a larger one. t3.medium → r6i.4xlarge. More CPU, more RAM, more network bandwidth. When vertical scaling makes sense: - Stateful workloads that cannot easily be distributed (single-node legacy databases, monolithic apps) - When the application i

Vertical scaling (scale up): Replace the instance with a larger one. t3.medium → r6i.4xlarge. More CPU, more RAM, more network bandwidth.

When vertical scaling makes sense: - Stateful workloads that cannot easily be distributed (single-node legacy databases, monolithic apps)
- When the application is not designed for parallelism (single-threaded workloads)
- Quick fix: if you need more capacity immediately and cannot quickly change the architecture

Problems with vertical scaling: - Hard limit: you cannot scale beyond the largest instance type
- Downtime: changing instance type requires stopping the instance (unless using AWS Graviton and live resize)
- Single point of failure: all traffic on one (larger) instance

Horizontal scaling (scale out): Add more instances of the same size. 1 × r6i.4xlarge → 10 × r6i.xlarge.

When horizontal scaling makes sense: - Stateless applications (web servers, API servers, microservices)
- When you need high availability (multiple instances across AZs)
- When traffic is variable (Auto Scaling Groups scale in and out)
- Cost: 10 × smaller instances are often cheaper and more available than 1 × huge instance

On EKS specifically: Horizontal = HPA (add pods) + Cluster Autoscaler (add nodes). Vertical = VPA (resize pod CPU/memory requests). For most microservices on Kubernetes, horizontal scaling is the right answer because pods are stateless and designed to run in parallel.

Database exception: Read replicas are a form of horizontal scaling for reads. The primary database often needs vertical scaling as it cannot be horizontally scaled for writes without sharding.

- [ ] What is the difference between AWS S3 storage classes and how do you use lifecycle policies to manage cost?

> **IF ASKED, SAY THIS**
> S3 storage classes and use cases: S3 Standard: Default. 3 AZ redundancy. Millisecond access. Most expensive. Use for: active data accessed frequently — application assets, current data, frequently accessed logs. S3 Standard-IA (Infrequent Access): Same 3 AZ redundancy. Same millisecond access. Cheap

S3 storage classes and use cases:

S3 Standard: Default. 3 AZ redundancy. Millisecond access. Most expensive. Use for: active data accessed frequently — application assets, current data, frequently accessed logs.

S3 Standard-IA (Infrequent Access): Same 3 AZ redundancy. Same millisecond access. Cheaper storage cost but retrieval fee per GB. Use for: backups you might need, older data still needed occasionally. Break-even vs Standard at about 30 days average access frequency.

S3 One Zone-IA: Single AZ only. Cheaper than Standard-IA. Data loss risk if AZ fails. Use for: reproducible data you can regenerate (thumbnails, derived analytics), non-critical backups where cost matters more than availability.

S3 Glacier Instant Retrieval: Archive storage. Millisecond retrieval. Very cheap. Use for: archive data accessed a few times per year. Quarterly reports, compliance archives.

S3 Glacier Flexible Retrieval: Hours to retrieve (expedited: minutes for a premium). Cheapest long-term archive. Use for: compliance data retention (7 year retention in financial services). You know you will almost never retrieve it.

S3 Intelligent-Tiering: Automatically moves objects between access tiers based on usage patterns. Small monitoring fee per object. Best for: data with unpredictable access patterns where you do not want to manually manage lifecycle rules.

Lifecycle policy example for financial logs: `Days 0-30: S3 Standard (active investigation access) Days 30-90: S3 Standard-IA (occasional access for audits) Days 90-365: S3 Glacier Instant Retrieval (rare access, compliance holds) Days 365+: S3 Glacier Flexible Retrieval (7-year legal retention, almost never accessed)`

This single policy on an S3 bucket can reduce storage costs by 80-90% for log data without any manual management.

- [ ] How does AWS IAM Policy evaluation work? What is the difference between identity-based and resource-based policies?

> **IF ASKED, SAY THIS**
> IAM policy evaluation is a common interview question that many candidates get wrong: Identity-based policies: Attached to IAM users, groups, or roles. They define what that identity can do. Example: An EC2 instance role with a policy allowing s3:GetObject on a specific bucket. Resource-based policie

IAM policy evaluation is a common interview question that many candidates get wrong:

Identity-based policies: Attached to IAM users, groups, or roles. They define what that identity can do.
Example: An EC2 instance role with a policy allowing s3:GetObject on a specific bucket.

Resource-based policies: Attached to AWS resources (S3 bucket policy, SQS queue policy, KMS key policy). They define who can access that resource. They can grant access to principals from OTHER accounts (cross-account access).

Policy evaluation order (this is what interviewers test):
1. Explicit DENY wins everything. If any policy at any level has an explicit Deny, access is denied. Period.
2. SCPs (Service Control Policies): if the account is in an AWS Organization, SCP must allow the action. Even if the user's IAM policy allows it, if the SCP denies it, access is denied.
3. Permission boundaries: if set on a role, the role can only do what both the identity policy AND the permission boundary allow.
4. Identity-based policies: what the user/role is allowed to do.
5. Resource-based policies: for S3 cross-account, if the bucket policy grants access to an external account, that account can access even without an IAM policy on their side.

Default is DENY: Everything is denied unless explicitly allowed. You do not need an explicit Deny for things you have not allowed.

Common mistake: Assuming that because an IAM role has s3:* permissions, it can access any S3 bucket. But if there is an SCP at the organisation level that restricts S3 access to buckets in ap-south-1 only, the role cannot access buckets in us-east-1 even though the identity policy would allow it.

- [ ] How do you test Terraform code before applying it to production?

> **IF ASKED, SAY THIS**
> Terraform testing is often overlooked but critical in regulated environments: Level 1 — Static analysis (seconds, runs in CI on every commit): terraform validate — checks syntax and internal consistency terraform fmt -check — enforces consistent formatting tflint — linting rules (deprecated resource

Terraform testing is often overlooked but critical in regulated environments:

Level 1 — Static analysis (seconds, runs in CI on every commit): `terraform validate` — checks syntax and internal consistency `terraform fmt -check` — enforces consistent formatting `tflint` — linting rules (deprecated resources, AWS naming conventions, unused variables) `checkov` or `tfsec` — security scanning: flag unencrypted S3, public security groups, missing deletion protection

Level 2 — Plan review (minutes, runs in CI on pull requests): `terraform plan -out=tfplan` stored as artifact Plan is posted as PR comment automatically via Atlantis or Terraform Cloud Humans review the plan before merging. Any unexpected destroy action = mandatory manual review.

Level 3 — Integration testing (minutes to hours, runs in staging): Terratest (Go library): write Go tests that apply Terraform, test the outputs, then destroy. Example: apply the module, verify the RDS endpoint is accessible, verify security groups are correct, then destroy.

Level 4 — Policy as Code (runs in CI, blocks apply if violated): OPA or Sentinel policies: - All S3 buckets must have encryption enabled - All EC2 instances must have deletion_protection or lifecycle prevent_destroy - All resources must have mandatory tags (Environment, Owner, Cost-Center) If policy fails, the CI pipeline blocks the merge. No human review needed for known violations.

Level 5 — Drift detection (runs daily in production): Scheduled pipeline: `terraform plan -refresh-only` and alert if output shows any changes. Any deviation from state = a notification to the SRE team to investigate.

- [ ] What is Terraform state locking and what happens if a lock is stuck?

IF ASKED, SAY THIS
Why state locking exists: Terraform state tracks what infrastructure exists. If two engineers run terraform apply simultaneously, they can corrupt the state file: both read the current state, make conflicting changes, and write back — the second write overwrites the first, losing infrastructure trac

Why state locking exists: Terraform state tracks what infrastructure exists. If two engineers run `terraform apply` simultaneously, they can corrupt the state file: both read the current state, make conflicting changes, and write back — the second write overwrites the first, losing infrastructure tracking.

How locking works with S3+DynamoDB backend: When any Terraform operation that modifies state begins (plan with refresh, apply, destroy): Terraform writes a lock record to the DynamoDB table. The record contains: who holds the lock, when it was acquired, which operation.
If another operation tries to acquire the lock while it is held: it fails immediately with "Error acquiring the state lock" and shows who holds it.
When the operation completes: lock is released from DynamoDB.

What to do if a lock is stuck: Locks get stuck when: a Terraform process crashes mid-run, network drops during apply, engineer force-kills the terminal.

```
# First: verify that NO active Terraform operation is running # Check with the engineer whose name is in the lock # Check CI/CD systems for running jobs # Only after confirming it is safe: terraform force-unlock LOCK-ID
```

Finding the lock ID: The error message when you try to run Terraform shows the lock ID. Or query DynamoDB directly:
`aws dynamodb get-item --table-name terraform-state-lock --key '{"LockID":{"S":"prod/payments/terraform.tfstate"}}'`

In financial services: Never force-unlock without verbal confirmation from the person who holds the lock or from their manager. A stuck lock may indicate an apply is still running in a non-terminal state. Unlocking while an apply is in progress corrupts the state file.

- [ ] What is the container build cache and how do you design Dockerfiles to maximise it?

IF ASKED, SAY THIS
Build cache is what makes Docker builds fast or slow: How the cache works: Docker evaluates each instruction in the Dockerfile. If the instruction and all its inputs (file contents, environment) are identical to a previous build, Docker reuses the cached layer. If any input changes, that layer and a

Build cache is what makes Docker builds fast or slow:

How the cache works: Docker evaluates each instruction in the Dockerfile. If the instruction and all its inputs (file contents, environment) are identical to a previous build, Docker reuses the cached layer. If any input changes, that layer and all layers below it are invalidated.

The critical rule — put things that change least at the top:

```
Wrong order (cache breaks on every source code change): COPY . . # copies everything including source - changes constantly RUN mvn install # re-downloads all dependencies every build (10+ minutes!) Correct order: COPY pom.xml . # copies only the dependency manifest - rarely changes RUN mvn dependency:go-offline # downloads deps, cached unless pom.xml changes COPY src ./src # copies source - changes frequently but deps already cached RUN mvn package # only compilation, not download - fast
```

Why this matters in practice: Without proper cache ordering: developer changes one line of code → Docker invalidates the cache at `COPY . .` → re-downloads all Maven/npm dependencies → 15-minute build. With correct ordering: dependency cache is preserved → build takes 90 seconds.

Multistage build cache sharing: Docker BuildKit allows using the cache from previous builds via registry: `docker build --cache-from=myregistry/myapp:cache .`. This shares the cache across different CI agents instead of each agent rebuilding from scratch.

Invalidating cache intentionally: Sometimes you want fresh dependencies (security patches). Use `docker build --no-cache .` or pass a build argument that changes: `ARG CACHE_BUST=$(date +%s)`

- [ ] How do you scan Docker images for vulnerabilities in CI/CD and what do you do with the results?

IF ASKED, SAY THIS
Container vulnerability scanning in a production CI/CD pipeline: Tool choice — Trivy (open source, recommended): Fast (~30 seconds for a 500MB image), zero config, scans OS packages AND language libraries (Maven, npm, pip), integrated with Docker, free. trivy image --exit-code 1 --severity HIGH,CRIT

Container vulnerability scanning in a production CI/CD pipeline:

Tool choice — Trivy (open source, recommended): Fast (~30 seconds for a 500MB image), zero config, scans OS packages AND language libraries (Maven, npm, pip), integrated with Docker, free. `trivy image --exit-code 1 --severity HIGH,CRITICAL myapp:1.2.3` exit-code 1 = pipeline fails on HIGH or CRITICAL CVEs. This is your quality gate.

What to do with scan results:
Immediately actionable: CVE in a library you directly depend on → update the library version in pom.xml/package.json, rebuild and rescan.
OS-level CVEs: update the base image. If using ubuntu:22.04, pin to the latest patched version. Use Dependabot or Renovate to automate base image updates.

CVE in a transitive dependency (library your library depends on): harder to fix. Options: update to a version of your library that uses a safe transitive version, or suppress with documented justification.

Suppression policy (important in regulated environments): Not all CVEs require immediate action. Create a .trivyignore file for CVEs that:
- Have no fix available yet
- Do not affect the codepath you use from that library
- Have been reviewed and accepted by the security team
Each suppression must have a documented justification and expiry date. This prevents .trivyignore from becoming a graveyard of unreviewed suppressions.

Beyond Trivy — Snyk in the IDE: Developers see vulnerabilities as they type in their IDE, not after a 30-minute build. Shift-left security: fix problems before they reach the pipeline.

Image signing with Cosign: After scanning and pushing a clean image, sign it: `cosign sign --key cosign.key myregistry/myapp:1.2.3` . In production, configure OPA/Kyverno to reject any pod spec that references an unsigned image. This ensures only images built and scanned by your CI pipeline can run in production.

- [ ] How do you implement zero-trust networking in a Kubernetes cluster?

> **IF ASKED, SAY THIS**
> Zero-trust = trust nobody by default, verify every request, assume breach: Layer 1 — Network Policy (L3/L4 firewall): Default deny all: create a NetworkPolicy with podSelector: {} (all pods) and policyTypes: [Ingress, Egress] but no rules. Now no pod can talk to any other pod by default. Then explic

Zero-trust = trust nobody by default, verify every request, assume breach:

Layer 1 — Network Policy (L3/L4 firewall): `Default deny all: create a NetworkPolicy with podSelector: {} (all pods) and policyTypes: [Ingress, Egress] but no rules. Now no pod can talk to any other pod by default.` Then explicitly allow only required communication paths: payments-service can call fraud-service on port 8080 payments-service can call postgres on port 5432 auth-service can call payments-service on port 8080 Nothing else is allowed. Any new service is denied by default until a policy is written.

Layer 2 — mTLS with Istio or Cilium: Mutual TLS means both the client and server authenticate each other using certificates. Without mTLS, even within the cluster, pod-to-pod traffic is plaintext. With mTLS, traffic is encrypted and authenticated — even a compromised pod cannot intercept or impersonate another service's traffic.

Layer 3 — Service Account and RBAC: Every pod runs with a specific Kubernetes ServiceAccount. RBAC policies define exactly what that ServiceAccount can do in the K8s API (which secrets it can read, which configmaps, etc). Ensure no pod has cluster-admin. Apply least-privilege: a payments service should not be able to read secrets from the auth namespace.

Layer 4 — IRSA/Workload Identity (cloud permissions): Each pod assumes a specific AWS IAM role/Azure Managed Identity with only the permissions it needs. Payments service: read from secrets manager, write to SQS. Auth service: read from DynamoDB, write to CloudWatch. No shared service account with broad permissions.

Layer 5 — Runtime security with Falco: Falco detects suspicious behaviour at runtime: a shell spawned inside a container, a binary written to disk, unexpected network connections. Alert on these events via PagerDuty. Even if a container is compromised, Falco detects anomalous behaviour.

- [ ] How do you handle PCI-DSS or SOX compliance requirements in your infrastructure?

> **IF ASKED, SAY THIS**
> Compliance requirements turn into specific engineering controls: PCI-DSS key controls and their implementation: Requirement: Protect cardholder data (CHD) Implementation: Tokenisation — never store PANs in your application. Use a PCI-certified tokenisation vault (Spreedly, Basis Theory, or build you

Compliance requirements turn into specific engineering controls:

PCI-DSS key controls and their implementation:

Requirement: Protect cardholder data (CHD) Implementation: Tokenisation — never store PANs in your application. Use a PCI-certified tokenisation vault (Spreedly, Basis Theory, or build your own in a hardened CDE environment). Your application only ever sees tokens, never card numbers. Macie on S3 to detect if card numbers accidentally end up in logs.

Requirement: Restrict access to system components Implementation: Separate AWS account for the Cardholder Data Environment. VPC with no internet access. All access through PrivateLink or Direct Connect only. No SSH keys — use AWS Systems Manager Session Manager for emergency access (all commands logged to CloudTrail).

Requirement: Monitor and log all access Implementation: CloudTrail on all accounts with immutable log storage (S3 with MFA delete). VPC Flow Logs. All application logs shipped to centrally managed SIEM. Log retention: minimum 12 months immediately accessible, 3 years total.

Requirement: Regular security testing Implementation: Quarterly penetration testing by an approved scanning vendor (ASV). Trivy in CI for continuous CVE scanning. AWS Config rules for continuous compliance assessment.

SOX key controls:
Change management: every production change has a ServiceNow CHG with approvers documented. Harness auto-creates and closes CHGs.
Segregation of duties: developers cannot deploy to production. SRE deploys. Separate approval required.
Evidence collection: Terraform plan outputs, Harness pipeline logs, CloudTrail API calls — all preserved for auditor review.

Audit evidence in practice: Auditors will ask "show me every change made to production system X in the last 90 days." Your answer: pull CloudTrail logs filtered by the resource ARN. Pull ServiceNow CHG records. Pull Harness pipeline execution history. All three corroborate each other.

- [ ] Stopping vs. terminating an EC2 instance — what happens to its data and its IP address in each case?

IF ASKED, SAY THIS
Stop: the root EBS volume (and any other EBS volumes not explicitly set to delete-on-stop, which isn't even a normal option — EBS persists through a stop by default) survives, and the instance can be started again. The trap: the public IP (unless it's an Elastic IP) is released on stop and a new one

Stop: the root EBS volume (and any other EBS volumes not explicitly set to delete-on-stop, which isn't even a normal option — EBS persists through a stop by default) survives, and the instance can be started again. The trap: the **public IP** (unless it's an Elastic IP) is released on stop and a **new** one is assigned on start — the private IP within the VPC subnet, by contrast, stays the same. A reboot changes neither.

Terminate: permanent. Attached EBS volumes are deleted too, *unless* "Delete on Termination" was explicitly unchecked for that volume at launch. Instance-store-backed instances (no EBS root) can't even be stopped — only terminated — and lose all data immediately since there's no persistent volume underneath them at all. Confusing these two failure modes (data survives vs. data gone, IP stable vs. IP changes) is a very common gap.

- [ ] Security Groups vs. NACLs — beyond ‘SG is stateful, NACL is stateless,’ what actually changes about how you write rules?

IF ASKED, SAY THIS
Because SGs are stateful, one inbound rule (say, port 443) is enough — the return traffic is auto-permitted. NACLs are stateless, so you need explicit rules in both directions : inbound 443 and outbound to the ephemeral port range (typically 1024–65535) for the response traffic. Forgetting that outb

Because SGs are stateful, one inbound rule (say, port 443) is enough — the return traffic is auto-permitted. NACLs are stateless, so you need **explicit rules in both directions**: inbound 443 *and* outbound to the ephemeral port range (typically 1024–65535) for the response traffic. Forgetting that outbound ephemeral-port rule is the single most common NACL misconfiguration — it looks like "the security group is fine, why is this still blocked" for hours until someone remembers NACLs exist.

The other structural difference: SGs only support **allow** rules (you omit a rule rather than explicitly deny); NACLs support explicit **allow and deny**, evaluated in rule-number order, first match wins — which also means NACL rule ordering is something you have to actively manage, unlike SGs where order is irrelevant.

- [ ] IMDSv1 vs. IMDSv2 — why does this actually matter for security, not just as a checkbox?

IF ASKED, SAY THIS
IMDSv1 answers a plain, unauthenticated GET request to 169.254.169.254 — which makes it exploitable through an SSRF vulnerability in an application: if an attacker can trick your app server into fetching an attacker-controlled URL, they can just as easily point that same fetch at the metadata endpoi

IMDSv1 answers a plain, unauthenticated GET request to `169.254.169.254` — which makes it exploitable through an SSRF vulnerability in an application: if an attacker can trick your app server into fetching an attacker-controlled URL, they can just as easily point that same fetch at the metadata endpoint and walk away with the instance's IAM credentials. IMDSv2 requires first obtaining a session token via a **PUT** request, which the overwhelming majority of real-world SSRF vulnerabilities (almost always GET-based) simply can't replicate.

This isn't theoretical — it's the actual mechanism behind the 2019 Capital One breach. In production, IMDSv2 should be *enforced* at the instance/launch-template level (`HttpTokens: required`), not just left available alongside v1, because "available" doesn't stop an attacker from just using the still-open v1 path.

- [ ] Is S3 eventually consistent or strongly consistent? Has this changed recently?

IF ASKED, SAY THIS
Since December 2020, S3 provides strong read-after-write consistency for all operations — PUTs and DELETEs — automatically, with no configuration needed and no performance tradeoff. This is a genuinely good trap question: a lot of older documentation, blog posts, and candidates' own memorized answer

Since December 2020, S3 provides **strong read-after-write consistency** for all operations — PUTs and DELETEs — automatically, with no configuration needed and no performance tradeoff. This is a genuinely good trap question: a lot of older documentation, blog posts, and candidates' own memorized answers still say "eventually consistent," which was true before the change but hasn't been for several years now.

Worth stating both halves in an interview — not just "it's strongly consistent now," but that you know it *used to be* eventually consistent and know roughly when and why that changed, since that's what actually demonstrates current, not memorized, knowledge.

- [ ] Is VPC peering transitive? What's the practical design implication?

IF ASKED, SAY THIS
No — explicitly not. If VPC A peers with VPC B, and VPC B peers with VPC C, traffic from A cannot reach C through B, full stop, no configuration fixes this. Each pair that needs to talk needs its own direct peering connection, which becomes an O(n²) mesh problem fast as the number of VPCs grows.

No — explicitly not. If VPC A peers with VPC B, and VPC B peers with VPC C, traffic from A cannot reach C through B, full stop, no configuration fixes this. Each pair that needs to talk needs its own direct peering connection, which becomes an O(n²) mesh problem fast as the number of VPCs grows.

The actual answer for anything beyond a handful of VPCs is **AWS Transit Gateway** — a hub-and-spoke model where every VPC peers once with the Transit Gateway, and the Transit Gateway's route tables control who can reach whom, which does support the multi-hop routing that peering alone never will.

- [ ] If there's an explicit Allow in an identity policy and an explicit Deny in an SCP, what wins — and what's commonly misunderstood about SCPs?

IF ASKED, SAY THIS

Explicit Deny always wins, full stop, regardless of which layer it's in. But the deeper, commonly-missed point: an SCP is a ceiling, not a grant — an Allow in an SCP doesn't actually grant anyone anything by itself; it only permits the IAM policies underneath it to grant that action, if they choose

Explicit Deny always wins, full stop, regardless of which layer it's in. But the deeper, commonly-missed point: an **SCP is a ceiling, not a grant** — an Allow in an SCP doesn't actually grant anyone anything by itself; it only *permits* the IAM policies underneath it to grant that action, if they choose to. A principal with a full-access IAM policy but no SCP allowing that service still can't use it; a principal with an SCP that allows everything but no IAM policy granting anything still can't do anything either. Both layers have to say yes independently, and either one saying an explicit no ends it immediately.

- [ ] Can you resize a VPC's CIDR block after creation?

IF ASKED, SAY THIS

You cannot change or shrink the primary CIDR block once the VPC exists. What you can do — and this is the part that trips up anyone whose knowledge is a few years stale — is add secondary CIDR blocks to extend the address space, subject to some restrictions (no overlap with the existing range, and s

You cannot change or shrink the **primary** CIDR block once the VPC exists. What you *can* do — and this is the part that trips up anyone whose knowledge is a few years stale — is add **secondary** CIDR blocks to extend the address space, subject to some restrictions (no overlap with the existing range, and some limitations interacting with certain peering configurations). A flat "no, never" answer is now only half-true; the accurate answer is "not the primary block, but you can extend it with secondaries."

- [ ] RDS Multi-AZ vs. Read Replica — what's the actual difference, and what's the classic confusion?

IF ASKED, SAY THIS

Multi-AZ is for high availability : a synchronous standby in a different AZ that automatically fails over if the primary has an issue. The standby is not queryable — you cannot read from it (with the exception of certain engines' newer "Multi-AZ with two readable standbys" option, a distinct feature

Multi-AZ is for **high availability**: a synchronous standby in a different AZ that automatically fails over if the primary has an issue. The standby is not queryable — you cannot read from it (with the exception of certain engines' newer "Multi-AZ with two readable standbys" option, a distinct feature from classic Multi-AZ). **Read Replica** is for **read scaling**: an asynchronous, independently-queryable copy you point read traffic at directly — but it's not an automatic failover target, promotion is a manual (or manually-triggered) action, and there's replication lag to account for.

The classic mistake, both ways: assuming a Read Replica gives you HA (it doesn't, without manual promotion and accepting lost in-flight writes), or assuming the Multi-AZ standby can absorb read load (by default, it can't — it's sitting there purely for failover).

- [ ] Is a NAT Gateway a single point of failure? What's the cost gotcha people consistently miss?

IF ASKED, SAY THIS

A NAT Gateway is AZ-scoped . Deploy just one, and every private subnet routed through it loses internet egress if that specific AZ has an issue — even if the rest of your architecture is genuinely Multi-AZ. The fix is one NAT Gateway per AZ, each serving only the private subnets in its own AZ.

A NAT Gateway is **AZ-scoped**. Deploy just one, and every private subnet routed through it loses internet egress if that specific AZ has an issue — even if the rest of your architecture is genuinely Multi-AZ. The fix is one NAT Gateway per AZ, each serving only the private subnets in its own AZ.

The cost gotcha: NAT Gateway bills both an hourly rate *and* a per-GB data processing charge on everything that flows through it. For any workload moving real data volume, that per-GB charge routinely ends up being the dominant, surprising line item — exactly the kind of thing worth checking first during a "why did the bill spike" investigation, and a very common candidate for the S3/DynamoDB Gateway VPC endpoint optimization below.

- [ ] Gateway vs. Interface VPC endpoints — why does the distinction matter for cost, not just architecture?

IF ASKED, SAY THIS

Gateway endpoints (S3 and DynamoDB only) are free — they work by adding an entry to a route table, no ENI involved, no hourly charge, no per-GB charge. Interface endpoints (most other AWS services, via PrivateLink) create an ENI in your subnet and bill both an hourly rate and a per-GB data processin

Gateway endpoints (S3 and DynamoDB only) are **free** — they work by adding an entry to a route table, no ENI involved, no hourly charge, no per-GB charge. **Interface endpoints** (most other AWS services, via PrivateLink) create an ENI in your subnet and bill both an hourly rate and a per-GB data processing charge — the same cost shape as a NAT Gateway.

The practical payoff: if traffic to S3 or DynamoDB is currently routing out through a NAT Gateway, switching it to a Gateway endpoint instead is close to a pure win — same or better latency, zero endpoint cost, and it removes that traffic from the NAT Gateway's billed data-processing volume entirely. This is one of the first things I'd check in a cost audit for any account with meaningful S3 traffic from private subnets.

- [ ] What's the classic ELB timeout gotcha that causes intermittent 502s that look like a random backend issue?

IF ASKED, SAY THIS

If the application's own keep-alive timeout is shorter than what the load balancer expects — or more precisely, if the ALB's idle timeout (default 60s) is shorter than the backend's keep-alive , or the backend's keep-alive is shorter than the ALB assumes — the ALB can try to reuse a connection the b

If the application's own keep-alive timeout is *shorter* than what the load balancer expects — or more precisely, if the **ALB's idle timeout (default 60s) is shorter than the backend's keep-alive**, or the backend's keep-alive is shorter than the ALB assumes — the ALB can try to reuse a connection the backend has already silently closed, producing an intermittent 502 that has nothing to do with application logic and everything to do with a timeout mismatch.

AWS's own guidance is specific: the **backend's keep-alive timeout should be set longer than the load balancer's idle timeout**, not the other way around — that way the LB is never the side left holding a connection the backend has already dropped. This is a genuinely common root cause behind "intermittent, unreproducible 502s under load" tickets that get misdiagnosed as application bugs for far longer than they should.

- [ ] When do you actually get charged for an Elastic IP?

IF ASKED, SAY THIS
An Elastic IP is free only while it's attached to a running instance. Allocate one and leave it unattached, or attach it to a stopped instance, and it starts accruing an hourly charge — a small amount per address, but it adds up silently across an account where people allocate EIPs for testing and f

An Elastic IP is free **only** while it's attached to a running instance. Allocate one and leave it unattached, or attach it to a *stopped* instance, and it starts accruing an hourly charge — a small amount per address, but it adds up silently across an account where people allocate EIPs for testing and forget to release them. It's a genuinely routine, easy finding in any cost audit, and a good concrete example to have ready if asked about cost optimization.

- [ ] Does attaching a Lambda function to a VPC still cause a major cold-start penalty?

IF ASKED, SAY THIS
Historically, yes — badly. Before 2019, a VPC-attached Lambda had to create and attach an ENI at invocation time, adding cold starts of 10 seconds or more, which is why "avoid VPC-attached Lambda unless you truly need it" was standard advice for years.

Historically, yes — badly. Before 2019, a VPC-attached Lambda had to create and attach an ENI at invocation time, adding cold starts of 10 seconds or more, which is why "avoid VPC-attached Lambda unless you truly need it" was standard advice for years.

AWS fixed the underlying mechanism in 2019 with improved VPC networking (shared, pre-warmed ENIs via Hyperplane) — VPC-attached Lambda cold starts are now much closer to non-VPC cold starts, not dramatically worse. This is another "know both the historical gotcha and that it's resolved" question — candidates who only learned the old advice and never revisited it will confidently give an outdated answer.

- [ ] If credentials exist in an environment variable, a shared credentials file, and an EC2 instance role all at once, which one actually gets used?

IF ASKED, SAY THIS
Precedence, highest to lowest: explicit CLI parameters → environment variables (AWS_ACCESS_KEY_ID , etc.) → shared credentials file (~/.aws/credentials) → shared config file (if enabled) → container credentials (ECS task role) → EC2 instance profile credentials, checked last as the fallback.

Precedence, highest to lowest: explicit CLI parameters → environment variables (`AWS_ACCESS_KEY_ID` , etc.) → shared credentials file (`~/.aws/credentials`) → shared config file (if enabled) → container credentials (ECS task role) → EC2 instance profile credentials, checked last as the fallback.

This matters constantly in real debugging — "why is my script hitting the wrong account" is very often a stale `AWS ACCESS KEY ID` environment variable left over from an earlier session, silently overriding the instance role you assumed was in effect. `aws sts get-caller-identity` (see the AWS CLI & Python section) is the fast way to cut through the guessing and see exactly which credential actually won.

- [ ] Does ‘us-east-1a’ mean the same physical location in every AWS account?

IF ASKED, SAY THIS
No — and this genuinely surprises even experienced engineers. AWS deliberately randomizes the mapping of AZ name (like us-east-1a) to physical data center per account , specifically to spread load evenly and avoid every account piling into the "first" AZ. The AZ ID (like use1-az1), by contrast, is

No — and this genuinely surprises even experienced engineers. AWS deliberately randomizes the mapping of **AZ name** (like `us-east-1a`) to physical data center *per account*, specifically to spread load evenly and avoid every account piling into the "first" AZ. The **AZ ID** (like `use1-az1`), by contrast, *is* consistent across accounts — that's the identifier to use when coordinating physical placement across accounts, such as optimizing for low-latency peering or planning Direct Connect topology.

This is a great one to have ready as a "here's something genuinely non-obvious I know" answer — it's true, it's useful, and it's the kind of detail that only comes from having actually worked across multiple AWS accounts, not from reading a single overview article.

- [ ] S3 bucket policy vs. IAM policy vs. ACL — if they conflict, what wins, and when do you actually use each?

IF ASKED, SAY THIS
Same rule as any IAM-adjacent decision: an explicit Deny anywhere wins. Beyond that, they serve different jobs: IAM policies attach to a principal and control what that principal can do across many resources; bucket policies attach to the bucket itself and are the only one of the three that can gran

Same rule as any IAM-adjacent decision: an explicit Deny anywhere wins. Beyond that, they serve different jobs: **IAM policies** attach to a principal and control what that principal can do across many resources; **bucket policies** attach to the bucket itself and are the only one of the three that can grant **cross-account** access cleanly; **ACLs** are the oldest, coarsest mechanism, and AWS now actively recommends disabling them entirely (S3 Object Ownership set to "Bucket owner enforced" on new buckets) in favor of policies.

Practical rule of thumb: use bucket policies specifically for cross-account grants or public-access statements, IAM policies for everything else, and treat ACLs as something you only encounter in legacy buckets, not something you'd design into something new.

- [ ] What is envelope encryption, and why doesn't KMS just encrypt your files directly?

IF ASKED, SAY THIS
KMS's Encrypt API has a hard 4KB limit on the data it will directly encrypt — nowhere near enough for a file, an EBS volume, or a database. For anything larger, AWS uses envelope encryption : KMS generates a data key and returns both a plaintext copy and a KMS-encrypted copy of it; the plaintext dat

KMS's `Encrypt` API has a hard 4KB limit on the data it will directly encrypt — nowhere near enough for a file, an EBS volume, or a database. For anything larger, AWS uses **envelope encryption**: KMS generates a data key and returns both a plaintext copy and a KMS-encrypted copy of it; the *plaintext* data key encrypts your actual large payload locally, fast, with standard symmetric AES; the plaintext key is then discarded, and only the KMS-encrypted version is stored alongside your encrypted data.

Decryption reverses this — KMS decrypts the small wrapped data key (a call well within the 4KB limit), and that decrypted key then decrypts the large payload locally. This is exactly why encrypting a multi-GB EBS volume or S3 object with a KMS key doesn't mean the entire payload round-trips through KMS — only the tiny data key ever does.

- [ ] An ASG considers an instance 'healthy' while its application is actually down and failing every load balancer health check. What's going on?

IF ASKED, SAY THIS
By default, an Auto Scaling Group can be configured with EC2-only health checks — which only ask "is the instance itself running," a check that stays green even if the application process on it has crashed or is failing every ELB health check and receiving zero real traffic. The ASG happily leaves t

By default, an Auto Scaling Group can be configured with **EC2-only** health checks — which only ask "is the instance itself running," a check that stays green even if the application process on it has crashed or is failing every ELB health check and receiving zero real traffic. The ASG happily leaves that dead-to-traffic instance in place indefinitely because, from its (EC2-only) point of view, nothing is wrong.

The fix is setting the ASG's health check type to **ELB** explicitly, so it inherits the load balancer's application-level view of health, not just the instance-level one, and will actually replace an instance that's up but useless. Also worth knowing: the health check **grace period** exists specifically so a legitimately slow-starting instance isn't killed by the ASG before it's had a fair chance to pass its very first health check.

- [ ] Walk through the actual mechanics of a cross-account AssumeRole call — where exactly is trust established?

IF ASKED, SAY THIS
Two independent policy checks, on two different sides of the account boundary, and both have to say yes: the target role's trust policy (in the account being assumed into) names which principals — a specific role or user ARN in the calling account — are allowed to assume it, which is the "who's allo

Two independent policy checks, on two different sides of the account boundary, and **both** have to say yes: the target role's **trust policy** (in the account being assumed into) names which principals — a specific role or user ARN in the calling account — are allowed to assume it, which is the "who's allowed through the door" check. Separately, the **calling principal** needs its own identity-based policy granting it `sts:AssumeRole` on that specific target role ARN — the "are you allowed to even try the door" check on the caller's own side.

Miss either one and the assumption fails — a trust policy alone doesn't grant the caller permission to attempt it, and a caller's own AssumeRole permission doesn't matter if the target doesn't trust them. Once both align and the assumption succeeds, you get temporary STS credentials scoped entirely to whatever the *target role's* permission policy allows — the calling principal's own original permissions are irrelevant from that point on.

- [ ] What are the three EC2 placement group types, and what's the actual tradeoff each one makes?

IF ASKED, SAY THIS
Cluster — instances packed onto the same underlying rack/hardware for the lowest possible latency and highest throughput between them, at the cost of fault tolerance: a hardware failure can take the whole group down together. Fits tightly-coupled HPC or latency-sensitive workloads that need the perf

Cluster — instances packed onto the same underlying rack/hardware for the lowest possible latency and highest throughput between them, at the cost of fault tolerance: a hardware failure can take the whole group down together. Fits tightly-coupled HPC or latency-sensitive workloads that need the performance and can tolerate that shared-failure risk.

Spread — instances forced onto genuinely distinct underlying hardware (max 7 per AZ), maximizing fault isolation for a small number of critical instances — the opposite tradeoff from Cluster.

Partition — instances grouped into logical partitions, each on separate racks, used by large distributed systems (Hadoop, Cassandra, Kafka) that already handle partition-level failure in their own replication logic — it balances some intra-partition locality against inter-partition fault isolation, the middle ground between the other two. The one-line summary: Cluster trades fault tolerance for performance, Spread trades performance for fault tolerance, Partition is the large-distributed-system compromise between them.

- [ ] You remove an item from the middle of a list used with count, or switch a resource from count to for_each. What actually happens, and why does it matter?

IF ASKED, SAY THIS
count addresses resources by INDEX (aws_instance.this[0], [1], [2]...). Remove the middle item from the source list and every subsequent index shifts -- Terraform doesn't see "one item removed," it sees every resource after that point as a different resource, and plans to destroy and recreate all of them. for_each addresses by KEY instead, so removing one map/set entry only touches that one entry -- nothing else moves.

This is one of the most commonly tested Terraform gotchas precisely because the blast radius is so much bigger than it looks in the diff -- a one-line change to a list can produce a plan that destroys and recreates a dozen unrelated-looking resources. The practical rule: use for_each (over a map or a set of strings) for anything where individual items might be added/removed out of order, and reserve count for genuinely fixed-size, order-independent replication.

- [ ] Does marking a Terraform variable sensitive = true keep its value out of the state file?

IF ASKED, SAY THIS
No -- sensitive = true only redacts the value from CLI output (plan/apply/output text) and from logs. The actual value is still written to the state file in plain text, because Terraform needs the real value there to detect drift and plan future changes. Anyone with read access to the state file can see it regardless of the sensitive flag.

The fix isn't the sensitive flag -- it's securing the state file itself: use a remote backend with encryption at rest (S3 with encrypt=true and a KMS key, or a backend that encrypts automatically like HCP Terraform), and tightly restrict who/what can read it. For genuinely not wanting a value persisted at all, Terraform 1.10+ added ephemeral variables/outputs and 1.11+ added write-only resource arguments (for supporting providers) -- both of these actually keep the value OUT of state entirely, which sensitive = true never did. Worth naming both the classic gotcha and the modern fix if it comes up -- it signals current knowledge, not memorized-from-2020 knowledge.

- [ ] Two engineers run terraform apply on the same state at the same time. What actually stops this from corrupting the state, and what happens if it's not configured?

IF ASKED, SAY THIS
State locking -- a remote backend (S3+DynamoDB historically, or a backend with native locking) takes a lock before any operation that writes state, so the second apply blocks/errors instead of racing the first. Without locking configured (e.g. plain local state, or a backend that doesn't support it), two concurrent applies CAN corrupt the state file -- the classic failure mode is state showing resources that don't match reality, or losing track of a resource that both operations touched.

This is exactly why "local state in a shared team repo" is a real anti-pattern independent of the git-merge-conflict problem -- there's no locking mechanism protecting concurrent runs at all, only whatever discipline the team imposes manually.

- [ ] terraform plan shows no changes, but you know someone modified a resource manually in the console. What's going on, and how do you fix it?

IF ASKED, SAY THIS
This shouldn't normally happen if refresh is working -- modern Terraform runs an implicit refresh as part of plan by default, comparing real infrastructure state against what's recorded, and WOULD show the drift. If it's not showing drift you expect, check whether refresh is explicitly disabled (-refresh=false) in the workflow/CI config, or use terraform apply -refresh-only to reconcile state with reality as a standalone step before deciding whether to accept the drift into state or plan a change to revert it.

The -refresh-only workflow is worth naming specifically: it lets you SEE what drifted and choose to update state to match reality (accept the manual change) without simultaneously planning any other changes -- cleanly separating "reconcile state" from "make new changes," which a plain apply conflates.

- [ ] How do you tell, just from reading a terraform plan output, whether a change will happen in place or will destroy and recreate the resource?

IF ASKED, SAY THIS
The symbol in front of the resource: ~ means update in place, -/+ means destroy and recreate. When it's -/+, Terraform also annotates the SPECIFIC attribute that forced it with "# forces replacement" right next to that attribute in the plan -- that's the line to read to understand WHY, rather than assuming the whole resource just happens to need recreating.

Reading plan output for the replacement reason, not just the top-line "1 to add, 1 to destroy" summary, is exactly the habit that would have caught a rename-triggers-replacement mistake (see the resource-identity gotchas elsewhere in this handbook) before it ever reached apply.

- [ ] Terraform workspaces -- do different workspaces give you isolated environments (like separate AWS accounts or credentials)?

IF ASKED, SAY THIS
No, and this trips people up: all workspaces under one backend configuration share the SAME backend (same S3 bucket/DynamoDB table, same provider credentials unless you explicitly parameterize them) -- workspaces only give you separate STATE FILES within that shared backend. If you want genuinely isolated environments (different accounts, different credentials), you need separate backend configs / separate root modules, not just separate workspaces.

A common real mistake this causes: assuming a "prod" workspace is safely isolated from "dev," then discovering both were quietly running against the same AWS credentials the whole time because nothing in the workspace mechanism itself enforces credential separation -- that has to be done deliberately, e.g. via separate provider configurations or a wrapper script that swaps credentials per workspace.

- [ ] You run terraform destroy -target=aws_instance.web to remove just one resource. Is the blast radius actually limited to that resource?

IF ASKED, SAY THIS
Not necessarily -- -target also pulls in anything that resource DEPENDS ON in some Terraform versions/scenarios, and more importantly, using -target regularly for routine operations is itself considered an anti-pattern precisely because it's easy to underestimate what's connected to what. HashiCorp's own guidance is to treat -target as a break-glass tool for recovering from a bad state, not an everyday way to scope changes.

The safer way to scope an intentional destroy of one resource is to remove it from the .tf configuration and run a normal plan/apply -- that way the full dependency graph is evaluated the normal way and shows you everything that's actually affected, rather than trusting -target to have scoped it correctly.

- [ ] What does terraform import actually do, and what's the gotcha people hit the first time they use it?

  IF ASKED, SAY THIS
  import brings an EXISTING real resource under Terraform's management by adding it to the state file -- but it does not write the matching resource block in your .tf code for you (in the classic workflow). If you import without first writing a resource block that matches, the very next plan will show Terraform wanting to destroy the "unmanaged" resource and create a new one, because as far as your CONFIG is concerned, nothing exists yet.

  The correct order: write the resource block first (even a mostly-empty one), THEN import, then run plan and iteratively adjust the resource block's arguments until plan shows no changes -- that convergence is how you confirm the config now accurately describes the real resource. Newer Terraform (1.5+) also supports import blocks that can generate starter config for you, reducing this manual back-and-forth.

- [ ] You run kubectl delete pod on a pod that's part of a Deployment. What actually happens?

  IF ASKED, SAY THIS
  The pod disappears and is immediately replaced by a new one -- because the Deployment's ReplicaSet controller continuously reconciles "desired replica count" against "actual running pods," and deleting one just drops the count by one for a fraction of a second before the controller creates a replacement. To actually reduce capacity, you scale the Deployment down; to actually stop the workload, you delete the Deployment (or scale to 0), not individual pods.

  This is a great one to lead with when asked "what's a Kubernetes gotcha" because it directly demonstrates understanding the CONTROLLER PATTERN that underlies almost everything in Kubernetes -- you're not managing individual objects, you're managing desired state that a controller continuously reconciles toward.

- [ ] You set resource limits on a container but never set requests. What does Kubernetes actually do?

  IF ASKED, SAY THIS
  If limits are set but requests are omitted, Kubernetes defaults requests to EQUAL the limits for that resource -- so the pod is scheduled as if it needs the full limit amount, not as if it needs nothing. This surprises people who assume omitting requests means "unconstrained" or "best effort" scheduling.

  The practical consequence: setting a generous limit "just in case" without a lower request can waste huge amounts of schedulable capacity across a cluster, since the scheduler reserves the full limit on every node placement even if the pod normally uses far less -- this is a real, common cause of clusters that look full but aren't actually using their allocated resources.

- [ ] You update a ConfigMap that's mounted into running pods. Do the pods pick up the change automatically?

  IF ASKED, SAY THIS
  Depends entirely on HOW it's consumed. Mounted as environment variables: no -- env vars are read once at container start, updating the ConfigMap does nothing until the pod restarts. Mounted as a VOLUME: the file on disk does eventually update on its own (via the kubelet's periodic sync), but the application inside the container still needs to notice the file changed and reload -- Kubernetes doesn't restart the pod or notify the app for you either way.

  This is why teams commonly add a checksum/hash of the ConfigMap's content as a pod template annotation (common in Helm charts) -- changing the ConfigMap changes the hash, which changes the pod template, which correctly triggers a real rolling restart through the normal Deployment mechanism, rather than relying on the app to watch for file changes itself.

- [ ] A Service has no endpoints -- kubectl get endpoints shows an empty list even though pods are running. What's the single most common cause?

  IF ASKED, SAY THIS
  The Service's selector doesn't actually match the labels on the pods you think it's routing to -- a typo, a label that got changed on the Deployment's pod template without updating the Service, or comparing app=web (Service) against app: web-app (pods) are all classic versions of this. kubectl describe service shows the selector; kubectl get pods --show-labels shows what's actually on the pods -- compare them directly rather than guessing.

  The second most common cause, worth naming as the follow-up: the pods exist and labels match, but none of them are passing their readiness probe -- Kubernetes only adds a pod to a Service's endpoints once it's Ready, not just Running, so an unhealthy-but-running pod fleet still shows zero endpoints.

- [ ] kubectl get pods shows a pod as Running. Does that mean the application inside is healthy and ready for traffic?

  IF ASKED, SAY THIS
  No -- Running only means the container process started and hasn't exited. It says nothing about whether the app finished initializing, is passing health checks, or is actually able to serve requests. Whether it's safe to route traffic to is governed entirely by the readiness probe (Ready column in kubectl get pods, separate from the Status column).

  This Running-vs-Ready distinction is exactly why a bad deploy can look fine at a glance ("pods show Running!") while actually serving errors or timing out -- always check the READY column (e.g. 1/1) alongside STATUS, not status alone, when verifying a rollout.

- [ ] You run kubectl apply -f deploy.yaml without specifying a namespace. Where does it go, and why does this matter operationally?

IF ASKED, SAY THIS
It goes to whatever namespace your current kubectl context is set to -- which defaults to the default namespace if nothing else was configured. This is a common, genuinely dangerous CI/CD mistake: a pipeline that forgets to pass -n (or set the context correctly) can silently deploy into the wrong namespace, or worse, the wrong CLUSTER if the context also points somewhere unexpected.

Production-safe practice: never rely on the ambient default context in automation -- explicitly pass --namespace and --context (or an equivalent kubeconfig scoping) in every CI/CD step that touches kubectl, so a misconfigured runner can't silently target the wrong place.

- [ ] kubectl top nodes / kubectl top pods returns an error or no data. What's missing?

IF ASKED, SAY THIS
metrics-server isn't installed -- it's NOT built into Kubernetes by default on most distributions (some managed offerings like EKS install it via an add-on, but it's not automatic everywhere). kubectl top, and anything relying on the same metrics API (like the Horizontal Pod Autoscaler's CPU/memory-based scaling), both depend on metrics-server being deployed and healthy.

Worth connecting explicitly: if HPA isn't scaling on CPU/memory as expected, checking whether metrics-server is even running is a legitimate first diagnostic step, not just a kubectl top curiosity -- the same missing piece breaks both.

- [ ] A toleration lets a pod schedule onto a tainted node. Does that mean the pod WILL be scheduled there?

IF ASKED, SAY THIS
No -- a toleration only REMOVES a barrier, it doesn't create a preference or requirement. A pod with a toleration for a taint is merely ALLOWED to land on that node; the scheduler is still free to place it anywhere else that also fits. To actually force placement onto specific nodes, you need nodeAffinity or nodeSelector in addition to the toleration.

This distinction (permission vs. requirement) is the same shape as several other Kubernetes gotchas in this list -- taints/tolerations control what's ALLOWED, affinity/selectors control what's PREFERRED or REQUIRED, and conflating the two is a very common source of "why didn't my pod land where I expected" confusion.

- [ ] You're draining a node for maintenance and kubectl drain hangs indefinitely. What's a common cause specific to production clusters with availability guarantees configured?

IF ASKED, SAY THIS
A PodDisruptionBudget (PDB) on the workload is blocking the eviction -- if a Deployment has a PDB requiring, say, at least 2 replicas always available, and draining this node would take it below that minimum, the drain will not evict those pods and will hang waiting rather than violate the budget. This is the PDB doing exactly its job: protecting availability during voluntary disruptions.

The fix isn't to force through it by default -- first confirm the PDB's minimum is actually still satisfiable elsewhere (e.g. scale up temporarily, or verify other nodes can absorb the pods) rather than reaching straight for --disable-eviction or force-deleting, which defeats the exact protection the PDB exists to provide.

- [ ] A namespace or pod gets stuck in Terminating state indefinitely. What's usually holding it there?

IF ASKED, SAY THIS
A finalizer -- a marker on the object telling Kubernetes "don't fully delete this until some external controller confirms cleanup is done" (common with custom resources, some storage/networking controllers, or namespaces containing resources whose controller has since crashed or been removed). If that controller is gone or broken, the finalizer never gets removed, and the object sits in Terminating forever.

The diagnostic path: kubectl get <object> -o yaml and look at the metadata.finalizers list to see what's actually blocking deletion, then investigate whether the owning controller is healthy. Manually patching out the finalizer force-completes the deletion but should be a deliberate last resort, since it skips whatever cleanup that finalizer was supposed to guarantee.

- [ ] A pipeline goes fully green -- build, tests, deploy step all pass. Does that guarantee the deployment is actually healthy in production?

IF ASKED, SAY THIS
No -- a green pipeline confirms the deploy STEP executed without erroring (e.g. kubectl apply succeeded, or the deploy API call returned 200). It says nothing about whether the new version is actually serving traffic correctly afterward. This is exactly why deploy pipelines that matter add a post-deploy verification gate (smoke tests, checking error rates for N minutes, or a canary-analysis step) rather than treating "the deploy command didn't error" as proof of success.

Conflating these two is one of the most common reasons a bad release ships and nobody notices for 20 minutes -- the pipeline dashboard says success while the actual service is erroring, because nothing in the pipeline was checking application health after the deploy command returned.

- [ ] You push a new image tagged :latest and update nothing else. Does Kubernetes pick up the new image?

IF ASKED, SAY THIS
Not reliably -- if the node already has an image cached with that exact tag and the effective imagePullPolicy doesn't force a re-pull, the kubelet may decide it already has "latest" and skip pulling the new one. The reliable fix is to never deploy on a mutable tag at all -- use a unique tag per build (git

SHA, semantic version, build number) so every deploy is an explicit, unambiguous image reference, and never depend on getting imagePullPolicy exactly right everywhere instead.

This is a very commonly cited real production incident pattern: a hotfix gets pushed to :latest, the deploy pipeline runs, everything reports success, and the OLD broken code is still running because no node was forced to actually re-pull. Unique immutable tags eliminate the ambiguity entirely.

- [ ] A pipeline step pipes one command into another (cmd1 | cmd2) and the step is marked as passed even though cmd1 actually failed. What's going on?

IF ASKED, SAY THIS
By default, a shell pipeline's exit code is the exit code of the LAST command only -- if cmd1 fails but cmd2 still runs and succeeds, the overall pipe reports success, silently swallowing cmd1's failure. The fix is set -o pipefail (bash) at the top of the script/step, which makes the pipeline's exit code reflect the first non-zero exit anywhere in the chain, not just the last command.

This is genuinely sneaky because the pipeline LOOKS like it's doing the right thing (checking exit codes) while actually checking the wrong command's exit code -- worth having as a go-to answer for "describe a subtle CI/CD bug you've seen or would watch for."

```
#!/bin/bash
set -euo pipefail   # pipefail specifically catches the case above

some_command_that_might_fail | grep "pattern"
```

- [ ] A command that fails prints a full curl command, including an embedded API token, straight into the pipeline's build log. What's the actual fix, not just "be careful"?

IF ASKED, SAY THIS
Secrets should never be interpolated directly into a command that gets echoed/logged -- mask them at the CI platform level (most CI systems auto-redact known secret variables from logs, but only if they were injected as a registered SECRET, not a plain env var), avoid verbose flags on commands carrying credentials in the URL or headers, and prefer passing secrets via files or stdin over command-line arguments.

The registered-secret-vs-plain-env-var distinction is the detail worth stating precisely: most CI platforms only apply automatic log-redaction to variables explicitly marked as secrets in their UI/config -- a value merely stored in a regular environment variable, even if it "looks" secret, won't be masked, which is exactly how this class of leak happens even on teams that think they're being careful.

- [ ] Two pipeline runs on the same branch trigger close together (e.g. two quick commits) and both start deploying to the same environment. What's the risk, and how do you prevent it?

IF ASKED, SAY THIS
A race condition -- whichever deploy finishes last "wins" regardless of which commit is actually newer, potentially leaving an OLDER version live even though the pipeline reports the newest commit deployed. Prevent it with concurrency control at the pipeline level -- most CI platforms support a concurrency group keyed by environment/branch that cancels or queues overlapping runs, rather than trusting commit timing to sort itself out.

GitHub Actions' concurrency key with cancel-in-progress, or an equivalent lock in other platforms, is the concrete mechanism to name here -- the general principle (serialize deploys to the same target) matters more than the exact syntax, but naming a real mechanism shows you've actually configured this, not just reasoned about it abstractly.

- [ ] Your team has branch protection requiring PR review before merging to main. Does that guarantee no unreviewed code reaches main?

IF ASKED, SAY THIS
Not necessarily -- check whether admins/service accounts are exempted from the protection rule (a common default or convenience setting), and whether the CI/CD pipeline's own deploy credentials could push directly, bypassing the PR requirement entirely. A protection rule is only as strong as its enforcement scope; "admins can bypass" is often on by default and quietly undermines the whole control.

This is worth raising proactively in a security-flavored interview answer -- auditing WHO can bypass a control you believe is enforced is exactly the kind of thing that separates "I set up branch protection" from "I verified branch protection actually protects the branch."

- [ ] A pipeline checks out a repo that uses git submodules, and the build mysteriously fails referencing files that should exist but don't. What's commonly missing?

IF ASKED, SAY THIS
A plain checkout doesn't pull submodule contents by default -- the submodule directory exists but is empty (just a reference to a commit, not the actual files) unless the checkout step explicitly initializes and updates submodules (most CI checkout actions have a submodules option; the raw git equivalent is git submodule update --init --recursive).

This one is easy to miss specifically because it works fine on every developer's local machine (where they cloned with the recurse-submodules flag, or ran the init command once, ages ago) and only breaks in the clean-checkout environment of CI -- a good instinct whenever something is "fine locally, broken in CI" is to ask what the CI checkout step is actually NOT doing that a full local clone did.

- [ ] You exec into a running container and make a change (install a package, edit a file). Does that change survive a docker restart? Does it survive docker rm?

IF ASKED, SAY THIS
Survives restart: yes -- the container's writable layer persists across stop/start, so anything changed via exec is still there after a restart. Survives docker rm: no -- removing the container destroys its writable layer permanently; only what's in the original image, or on a mounted volume, comes back if you run a new container from the same image.

The practical lesson this tests: never treat exec'd-in changes as a substitute for actually updating the Dockerfile and rebuilding -- anything that matters needs to be baked into the image (or persisted via a volume), because the container itself is meant to be disposable.

- [ ] COPY vs ADD in a Dockerfile -- why do most style guides say to prefer COPY?

IF ASKED, SAY THIS
ADD does everything COPY does, PLUS two extra behaviors that surprise people: it auto-extracts local tar archives into the destination, and it can fetch a remote URL directly. Both are rarely what you actually want in a reproducible build, and the implicit tar-extraction especially has caused real security issues. COPY does exactly one thing -- copy files -- which is easier to reason about and audit.

The one legitimate common use for ADD is exactly the tar-auto-extract behavior when you deliberately want it (e.g. unpacking a downloaded release tarball as part of the build) -- otherwise, defaulting to COPY and being explicit about extraction/download steps elsewhere in the Dockerfile is the safer habit.

- [ ] Why does the ORDER of instructions in a Dockerfile matter for build speed, beyond just "organize it neatly"?

IF ASKED, SAY THIS
Docker caches each layer, and invalidates that layer's cache -- and every layer after it -- the moment anything in that instruction changes. Put rarely-changing steps (installing OS packages, copying a dependency manifest and installing dependencies) BEFORE frequently-changing steps (copying application source code), so a source-code change only invalidates the last couple of layers instead of forcing a full reinstall of every dependency on every build.

The classic concrete pattern: copy the dependency manifest and run the install FIRST, THEN copy the rest of the source -- as long as the manifest hasn't changed, that install layer stays cached across every subsequent source-only change, which is often the difference between a 3-second build and a 3-minute one.

- [ ] docker build seems to take a while just to START, before any instruction even runs. What's happening, and what's the fix?

IF ASKED, SAY THIS
Docker sends the entire build CONTEXT (the directory you're building from) to the Docker daemon before executing anything -- a directory containing node_modules, .git history, or other large/irrelevant content gets uploaded every single build even if nothing in it is actually used, and that transfer time shows up before the first instruction even starts. Fix: a .dockerignore file excluding anything the build doesn't genuinely need.

This is functionally identical in spirit to .gitignore, and forgetting it is one of the most common, easily-fixed sources of slow local Docker builds -- worth checking .dockerignore exists and is actually comprehensive as a first troubleshooting step whenever a build feels unexpectedly slow to even begin.

- [ ] When both ENTRYPOINT and CMD are set in a Dockerfile, how do they actually combine, and what happens if you pass arguments at docker run time?

IF ASKED, SAY THIS
ENTRYPOINT defines the fixed executable that always runs; CMD supplies its DEFAULT arguments, which get appended after ENTRYPOINT. Anything passed at the end of a docker run command overrides CMD's arguments entirely (not ENTRYPOINT itself) -- so the container's fixed command still runs, but with whatever new arguments you supplied instead of the Dockerfile's defaults.

The mental model that makes this click: think of ENTRYPOINT as the command name and CMD as its default arg list -- docker run's trailing arguments are just a new arg list for that same fixed command, exactly like overriding default arguments to any CLI tool.